🐍 Apuntes para el examen de Python · Máster Big Data & IA (Tajamar)

Héctor Miguel Tudela · Examen práctico de Python · 13 de noviembre de 2026

Este documento reúne **todo** el material de Python del módulo: una chuleta rápida, el **modelo de examen 16.1 resuelto entero**, las **prácticas 16, 15 y 13 resueltas** (enunciado del profesor + código + salida real) y la **teoría de las clases 18 a 22** con sus ejemplos.

> Todas las soluciones se han **ejecutado** y su salida coincide con la *salida esperada* del enunciado. Usa **Ctrl+F** para buscar por nombre de clase, función o concepto (por ejemplo `__iadd__`, `MRO`, `reduce`, `RecursionError`).

Índice

1. **Chuleta rápida** – plantilla de clase, métodos mágicos, guiones bajos, atributos de clase/instancia, alias y copias, herencia y MRO, composición, comprehensions, funcional, recursión, parámetros, estructuras, formato, módulos, errores y checklist.
2. **Modelo de examen 16.1 resuelto** – secciones 1 a 10 (lectura de código, comprehensions, funcional, recursión, clases, sobrecarga, herencia, refactorización, integrador y proyecto modular `hoteles_etl` + mini app con paquetes).
3. **Práctica 16 resuelta** – OOP y programación funcional (clases 20 y 21) + ejercicios extra de la clase 20 (P5, P6, 3.1, 3.2).
4. **Práctica 15 resuelta** – métodos y clases (B1-B7, I1-I6, A1-A5, R1, J1 Hundir la flota).
5. **Práctica 13 resuelta** – sprints de Python: listas, tuplas, sets, diccionarios, comprehensions, funciones y recursión (ejemplos del profesor + ejercicios propuestos resueltos).
6. **Teoría de las clases 18 a 22** – clases y objetos, OOP parte 2, herencia y polimorfismo, programación funcional, programación modular y paquetes.
7. **Examen de práctica · Modelo A** – 10 ejercicios (uno por sección) con enunciados y solucionario.
8. **Examen de práctica · Modelo B** – 10 ejercicios (uno por sección) con enunciados y solucionario.

Índice detallado

Índice

Parte 1 · Chuleta rápida para el examen
- 1.1 Plantilla de una clase completa
- 1.2 Métodos mágicos (sobrecarga de operadores y conversiones)
- 1.3 Convenciones de nombres (guiones bajos)
- 1.4 Atributo de clase vs. atributo de instancia (TRAMPA FAVORITA)
- 1.5 Referencias, alias y copias
- 1.6 Herencia, super() y polimorfismo
- 1.7 Herencia múltiple y MRO
- 1.8 Composición (práctica 15: sin herencia)
- 1.9 Comprehensions
- 1.10 Programación funcional
- 1.11 Recursión
- 1.12 Funciones: parámetros
- 1.13 Estructuras de datos (práctica 13)
- 1.14 Conversión de tipos y formato
- 1.15 Programación modular y paquetes
- 1.16 Errores típicos y su tipo
- 1.17 Checklist del examen

Parte 2 · Modelo de examen 16.1 resuelto
- Indicaciones generales
- Sección 1. Lectura de código: ¿qué se muestra por pantalla?
  - Ejercicio 1.1 · El contador de tiendas
  - Ejercicio 1.2 · Dos listas que parecen tres

Ejercicio 1.3 · Filtrar o clasificar
Ejercicio 1.4 · Cuenta atrás
Ejercicio 1.5 · Recepción de un hotel
Ejercicio 1.6 · Atributos con guiones bajos
Ejercicio 1.7 · Funciones que fabrican funciones
Ejercicio 1.8 · La cafetería y la herencia múltiple
Sección 2. Comprehensions
Ejercicio 2.1 · Precios con IVA
Ejercicio 2.2 · Clasificar paquetes
Ejercicio 2.3 · Ciudades sin repetir
Ejercicio 2.4 · Butacas libres en un cine
Ejercicio 2.5 · Unir las cestas de la compra
Sección 3. Programación funcional
Ejercicio 3.1 · Funciones lambda
Ejercicio 3.2 · map()
Ejercicio 3.3 · filter()
Ejercicio 3.4 · reduce()
Ejercicio 3.5 · Funciones que reciben funciones
Ejercicio 3.6 · Funciones que devuelven funciones
Sección 4. Recursión
Ejercicio 4.1 · Sumar los dígitos de un código
Ejercicio 4.2 · Total de una lista de gastos
Ejercicio 4.3 · ¿Es un palíndromo?
Ejercicio 4.4 · Cajas dentro de cajas
Ejercicio 4.5 · Iterativo frente a recursivo
Sección 5. Clases y objetos
Ejercicio 5.1 · Artículos de una tienda de deportes
Ejercicio 5.2 · Cuentas de un banco
Ejercicio 5.3 · Carrito de la compra (composición)
Ejercicio 5.4 · Lista de objetos
Sección 6. Sobrecarga de operadores y conversión de tipos
Ejercicio 6.1 · Sumar y comparar importes
Ejercicio 6.2 · Lista de reproducción
Ejercicio 6.3 · Precios escritos a mano
Sección 7. Herencia, polimorfismo y herencia múltiple
Ejercicio 7.1 · Peajes de una autopista
Ejercicio 7.2 · Suscripciones de una plataforma de series
Ejercicio 7.3 · Impresora multifunción (herencia múltiple)
Ejercicio 7.4 · Permisos de usuarios en una aplicación (herencia múltiple con super())
Sección 8. Refactorización: de código espagueti a clases
Parte A. Analizar el código original
Parte D. Crear una clase que agrupe las reservas (composición)
Parte F. Ampliación
Restricciones
Sección 9. Ejercicio integrador: restaurante de comida a domicilio
Parte A. Jerarquía de clases
Parte C. Comprehensions
Parte F. Informe final
Sección 10: Programación Modular
Estructura mínima exigida
Requisitos por módulo
Reglas de importación

Parte 3 · Práctica 16 resuelta (OOP y programación funcional · clases 20 y 21)

Clase 20:
Ejercicio P1. Sistema de gestión de empleados
Ejercicio P2. Refactorización de un sistema de dispositivos IoT
Ejercicio P3. Gestión de recursos de un laboratorio de Big Data
Ejercicio P4: Polimorfismo en un sistema de procesamiento de datos
Parte A. Crear la superclase
Parte C. Crear los objetos
Parte D. Aplicar polimorfismo
Parte E. Procesar múltiples archivos
Parte F. Ampliación del sistema
Preguntas
Clase 21
Ejercicio Propuesto P1: Sistema de análisis de envíos logísticos

Enunciado
Reglas del negocio
Parte A. Crear la jerarquía de clases
Parte B. Crear objetos y aplicar polimorfismo
Parte C. Aplicar comprehensions
Parte D. Aplicar programación funcional
Parte E. Mostrar el informe final
8. Restricciones
Ejercicio propuesto P2. Procesamiento de pedidos de una tienda online
Ejercicio propuesto P3. Análisis de ejecuciones de pipelines de datos
Ejercicios extra de la clase 20 (herencia múltiple)
Ejercicio de comprobación
3.2. Herencia Múltiple: El “Súper Héroe con Doble Poder”
Ejercicio P5: Sistema de vehículos inteligentes con herencia múltiple
Parte A. Crear las clases base
Parte B. Crear una clase mediante herencia múltiple
Parte C. Aplicar super() en herencia múltiple
Parte D. Crear objetos
Parte E. Utilizar los métodos heredados
Salida esperada
Parte F. Investigar el orden de resolución de métodos (MRO)
Restricciones del ejercicio
Ejercicio P6: Refactorización de un sistema de incidencias informáticas
Parte A. Analizar el código original
Parte B. Crear dos clases base
Parte C. Aplicar herencia múltiple
Parte D. Crear tres clases especializadas
Parte E. Reutilizar métodos con super()
Parte F. Aplicar polimorfismo
Salida esperada
Parte G. Ampliación
Restricciones

Parte 4 · Práctica 15 resuelta (OOP · métodos y clases)
Instrucciones
Sección 1 - Nivel básico
Ejercicio B1 - Ficha de empleado del equipo de datos
Ejercicio B2 - Pool de conexiones a una base de datos
Ejercicio B3 - Plantilla de prompt reutilizable · IA Generativa
Ejercicio B4 - Coste de las llamadas a un LLM · IA Generativa
Ejercicio B5 - Climatización de un centro de datos
Ejercicio B6 - Peso de los envíos de un almacén
Ejercicio B7 - Valoraciones de clientes desde una encuesta
Sección 2 - Nivel intermedio
Ejercicio I1 - Historial de conversación de un asistente · IA Generativa
Ejercicio I2 - Cuotas de tokens por equipo - IA Generativa
Ejercicio I3 - Control de calidad de un lote de ingesta - Ingeniería de Datos
Ejercicio I4 - Reservas de salas de reuniones
Ejercicio I5 - Depura el registro de equipos de proyecto
Ejercicio I6 - Precios en varias divisas
Sección 3 - Nivel avanzado
Ejercicio A1 - Mini framework de pipelines ETL - Ingeniería de Datos
Ejercicio A2 - Simulador de MapReduce sobre logs web - Big Data
Ejercicio A3 - Regresión lineal desde cero - Machine Learning
Ejercicio A4 - Motor RAG mínimo para la base de conocimiento de RR. HH. - IA Generativa
Ejercicio A5 - Planificador de proyectos con dependencias
Sección 4 - Refactorización a programación orientada a objetos
Ejercicio R1 - Refactorizar el gestor de pedidos de una distribuidora
Sección 5 - Juego interactivo
Ejercicio J1 - Juego interactivo: Hundir la flota

Parte 5 · Práctica 13 resuelta (Sprints de Python)
Sección 1. Listas
Ejemplos resueltos
Ejercicios propuestos
Ejercicio propuesto 1.1
Ejercicio propuesto 1.2

Ejercicio propuesto 1.3
Ejercicio propuesto 1.4
Ejercicio propuesto 1.5
Ejercicio propuesto 1.6
Ejercicio propuesto 1.7
Sección 2. Tuplas
Ejemplos resueltos
Ejercicios propuestos
Ejercicio propuesto 2.1
Ejercicio propuesto 2.2
Ejercicio propuesto 2.3
Ejercicio propuesto 2.4
Ejercicio propuesto 2.5
Ejercicio propuesto 2.6
Sección 3. Set
Ejemplos resueltos
Ejercicios propuestos
Ejercicio propuesto 3.1
Ejercicio propuesto 3.2
Ejercicio propuesto 3.3
Ejercicio propuesto 3.4
Ejercicio propuesto 3.5
Ejercicio propuesto 3.6
Sección 4. Diccionarios
Ejemplos resueltos
Ejercicios propuestos
Ejercicio propuesto 4.1
Ejercicio propuesto 4.2
Ejercicio propuesto 4.3
Ejercicio propuesto 4.4
Ejercicio propuesto 4.5
Ejercicio propuesto 4.6
Ejercicio propuesto 4.7
Sección 5. Comprehensions
Ejemplos resueltos
Ejercicios propuestos
Ejercicio propuesto 5.1
Ejercicio propuesto 5.2
Ejercicio propuesto 5.3
Ejercicio propuesto 5.4
Ejercicio propuesto 5.5
Ejercicio propuesto 5.6
Ejercicio propuesto 5.7
Sección 6. Funciones
Ejemplos resueltos
Ejercicios propuestos
Ejercicio propuesto 6.1
Ejercicio propuesto 6.2
Ejercicio propuesto 6.3
Ejercicio propuesto 6.4
Ejercicio propuesto 6.5
Ejercicio propuesto 6.6
Ejercicio propuesto 6.7
Ejercicio propuesto 6.8
Ejercicio propuesto 6.9
Ejercicio propuesto 6.10
Sección 7. Recursión
Ejemplos resueltos
Ejercicios propuestos
Ejercicio propuesto 7.1
Ejercicio propuesto 7.2
Ejercicio propuesto 7.3
Ejercicio propuesto 7.4
Ejercicio propuesto 7.5
Ejercicio propuesto 7.6
Ejercicio propuesto 7.7

Ejercicio propuesto 7.8
Ejercicio propuesto 7.9
Ejercicio propuesto 7.10

Parte 6 · Teoría de las clases 18 a 22 (material del profesor)
Clase 18 · Clases y objetos
Clase 19 · OOP parte 2
Clase 20 · Herencia y polimorfismo
Clase 21 · Programación funcional
Clase 22 · Programación modular y paquetes

Parte 7 · Examen de práctica · Modelo A
Indicaciones
Parte I · Enunciados
Ejercicio 1 · Lectura de código · Pedidos de una tienda
Ejercicio 2 · Comprehensions · Ventas de una cadena de tiendas
Ejercicio 3 · Programación funcional · Vuelos de una aerolínea
Ejercicio 4 · Recursión · Máximo común divisor y máximo de una lista
Ejercicio 5 · Clases y objetos · Gestor de tareas (composición)
Ejercicio 6 · Sobrecarga de operadores · Fracciones
Ejercicio 7 · Herencia y herencia múltiple · Alquiler de vehículos
Ejercicio 8 · Refactorización · Cuotas de un gimnasio
Ejercicio 9 · Ejercicio integrador · Tienda de videojuegos
Ejercicio 10 · Programación modular · El paquete tienda
Parte II · Solucionario
Solución 1 · Lectura de código · Pedidos de una tienda
Solución 2 · Comprehensions · Ventas de una cadena de tiendas
Solución 3 · Programación funcional · Vuelos de una aerolínea
Solución 4 · Recursión · Máximo común divisor y máximo de una lista
Solución 5 · Clases y objetos · Gestor de tareas (composición)
Solución 6 · Sobrecarga de operadores · Fracciones
Solución 7 · Herencia y herencia múltiple · Alquiler de vehículos
Solución 8 · Refactorización · Cuotas de un gimnasio
Solución 9 · Ejercicio integrador · Tienda de videojuegos
Solución 10 · Programación modular · El paquete tienda

Parte 8 · Examen de práctica · Modelo B
Indicaciones
Parte I · Enunciados
Ejercicio 1 · Lectura de código · La cadena de super() rota
Ejercicio 2 · Comprehensions · Logs de un servidor web
Ejercicio 3 · Programación funcional · Liga de fútbol
Ejercicio 4 · Recursión · Binario, listas crecientes y presupuestos
Ejercicio 5 · Clases y objetos · Hotel y habitaciones (composición)
Ejercicio 6 · Sobrecarga de operadores · Temperaturas
Ejercicio 7 · Herencia y polimorfismo · Envío de notificaciones
Ejercicio 8 · Refactorización · Matrículas de una academia de idiomas
Ejercicio 9 · Ejercicio integrador · Taquilla de un cine
Ejercicio 10 · Programación modular · Del script al paquete almacen
Parte II · Solucionario
Solución 1 · Lectura de código · La cadena de super() rota
Solución 2 · Comprehensions · Logs de un servidor web
Solución 3 · Programación funcional · Liga de fútbol
Solución 4 · Recursión · Binario, listas crecientes y presupuestos
Solución 5 · Clases y objetos · Hotel y habitaciones (composición)
Solución 6 · Sobrecarga de operadores · Temperaturas
Solución 7 · Herencia y polimorfismo · Envío de notificaciones
Solución 8 · Refactorización · Matrículas de una academia de idiomas
Solución 9 · Ejercicio integrador · Taquilla de un cine
Solución 10 · Programación modular · Del script al paquete almacen

Parte 1 · Chuleta rápida para el examen

> Lo esencial de las clases 18-22 y de las prácticas 13, 15 y 16 en pocas páginas. Si dudas en el examen, **busca aquí primero** (Ctrl+F) y después en los ejercicios resueltos.

1.1 Plantilla de una clase completa

```python
class Producto:
    IVA = 0.21                  # atributo de CLASE (constante compartida)
    creados = 0                 # contador de clase

    def __init__(self, nombre, precio, stock=0):   # constructor
        self.nombre = nombre    # atributo de INSTANCIA (público)
        self._precio = precio   # "protegido": convención, se puede tocar pero no se debe
        self.__stock = stock    # "privado": name mangling → _Producto__stock
        self.etiquetas = []     # ¡las listas SIEMPRE dentro de __init__!
        Producto.creados += 1   # ¡escribir en la CLASE, nunca self.creados += 1!

    def precio_final(self):                 # método de instancia (recibe self)
        return round(self._precio * (1 + Producto.IVA), 2)

    @classmethod
    def total_creados(cls):                 # recibe la clase (cls)
        return cls.creados

    @staticmethod
    def desde_texto(texto):                 # ni self ni cls: función "guardada" en la clase
        nombre, precio = texto.split(";")
        return Producto(nombre, float(precio.replace(",", ".")))

    @property
    def stock(self):                        # se usa SIN paréntesis: p.stock
        return self.__stock

    def __str__(self):                      # print(p) / str(p) → para el usuario
        return f"{self.nombre} - {self.precio_final():.2f} €"

    def __repr__(self):                     # repr(p) / dentro de listas → para el programador
        return f"Producto({self.nombre!r}, {self._precio})"
```

1.2 Métodos mágicos (sobrecarga de operadores y conversiones)

Quieres que funcione…	Método	Devuelve / ojo
`print(obj)`, `str(obj)`, f-string	`__str__(self)`	`str`. Si no existe, usa `__repr__`
`repr(obj)`, `print([obj1, obj2])`	`__repr__(self)`	`str`. Las **listas** se imprimen con `__repr__`
`a + b`	`__add__(self, otro)`	Un objeto **nuevo** (`return Clase(...)`)
`a - b`, `a * n`	`__sub__`, `__mul__`	Objeto nuevo
`a += x`	`__iadd__(self, x)`	**`return self`** (si no, `a` pasa a valer `None`)
`a == b`	`__eq__(self, otro)`	`bool`. Sin él, `==` compara identidad (`is`)
`a < b`, `sorted()`, `min()`, `max()`	`__lt__(self, otro)`	`bool`. Con `__lt__` ya funcionan `sorted/min/max`
`a <= b`, `a > b`	`__le__`, `__gt__`	`bool`
`len(obj)`	`__len__(self)`	`int`
`x in obj`	`__contains__(self, x)`	`bool`

Quieres que funcione…	Método	Devuelve / ojo
`bool(obj)`, `if obj:`, `not obj`	`__bool__(self)`	`bool` (si no existe, usa `__len__`)
`float(obj)` / `int(obj)`	`__float__` / `__int__`	`float` / `int`
`obj[i]`	`__getitem__(self, i)`	El elemento
`for x in obj`	`__iter__(self)`	Un iterador (`return iter(self._lista)`)

```python
class Importe:
    def __init__(self, cantidad):
        self.cantidad = cantidad
    def __add__(self, otro):
        return Importe(self.cantidad + otro.cantidad)
    def __eq__(self, otro):
        return round(self.cantidad, 2) == round(otro.cantidad, 2)   # cuidado con los float
    def __lt__(self, otro):
        return self.cantidad < otro.cantidad
    def __str__(self):
        return f"{self.cantidad:.2f} €"
    def __float__(self):
        return float(self.cantidad)
```

1.3 Convenciones de nombres (guiones bajos)

Nombre	Significado	¿Accesible desde fuera?
`nombre`	Público	Sí
`_nombre`	Uso interno / protegido (convención)	Sí, pero **no se debe**
`__nombre`	Privado: *name mangling* → se guarda como `_Clase__nombre`	`obj.__nombre` → **AttributeError**; `obj._Clase__nombre` → funciona
`__nombre__`	Método especial (dunder) de Python	Sí
`MAX_CONEXIONES`	Constante (convención: mayúsculas)	Sí

`hasattr(obj, "__pin")` es `False` y `vars(obj)` muestra `'_Tarjeta__pin'`. El doble guion bajo **no hace el atributo realmente privado**, solo le cambia el nombre.

1.4 Atributo de clase vs. atributo de instancia (TRAMPA FAVORITA)

```python
class Tienda:
    total = 0                   # de clase
    items = []                  # ⚠️ lista de clase: COMPARTIDA por todos los objetos
    def __init__(self):
        Tienda.total += 1       # ✅ modifica la clase
        # self.total += 1       # ❌ lee la clase y CREA un atributo de instancia que la tapa

t1, t2 = Tienda(), Tienda()
t1.total = 10                   # crea un atributo de INSTANCIA en t1 (no cambia la clase)
print(Tienda.total, t1.total, t2.total)   # 2 10 2
```

- **Búsqueda de atributos:** primero el objeto (`vars(obj)`), después la clase, después los padres (MRO).
- `obj.x = ...` **siempre** escribe en el objeto. `obj.lista.append(...)` **modifica** lo que encuentre (si es la lista de clase, la de todos).
- `vars(obj)` → diccionario con los atributos de **instancia**. `dir(obj)` → todo (incluidos métodos y heredados).

1.5 Referencias, alias y copias

```python
a = [1, 2, 3]
b = a          # ALIAS: mismo objeto → b.append(4) también cambia a
c = a[:]       # COPIA (también list(a) o a.copy()) → objeto nuevo
a is b, a is c, a == c       # True False True   (is = mismo objeto, == = mismo contenido)
```

```
x = 5; y = x; y += 1          # inmutables (int, str, tuple): += crea un objeto NUEVO → x sigue en 5
```

- **Mutables:** `list`, `dict`, `set`, objetos propios. **Inmutables:** `int`, `float`, `str`, `tuple`, `bool`, `None`.
- ⚠️ **Valor por defecto mutable:** `def f(x, lista=[])` comparte la lista entre llamadas. Usa `lista=None` y dentro `lista = [] if lista is None else lista`.
- ⚠️ **No borrar de una lista mientras se recorre** (se salta elementos): crea una lista nueva con una comprehension o recorre una copia `for x in lista[:]`.

1.6 Herencia, `super()` y polimorfismo

```
class Vehiculo:
    def __init__(self, matricula, km):
        self.matricula, self.km = matricula, km
    def peaje(self):
        return self.km * 0.10

class Camion(Vehiculo):                        # hereda
    def __init__(self, matricula, km, ejes):
        super().__init__(matricula, km)        # reutiliza el constructor del padre
        self.ejes = ejes                       # atributo nuevo
    def peaje(self):                           # SOBRESCRIBE
        return super().peaje() + 2 * self.ejes   # AMPLÍA el del padre

for v in [Vehiculo("A", 100), Camion("B", 100, 4)]:
    print(v.peaje())                           # POLIMORFISMO: misma llamada, distinto comportamiento
```

- **Ampliar** = llamar a `super().metodo()` y añadir. **Sustituir** = no llamar a `super()`.
- Si la subclase **no define** `__init__`, hereda el del padre (P4: subclases que solo sobrescriben `procesar()`).
- `isinstance(obj, Clase)` → `True` también para subclases. `type(obj).__name__` → nombre de la clase real (`"Camion"`).
- `type(self).__name__.replace("Reserva", "").upper()` → `"VUELO"` (truco del ejercicio 8).
- **Duck typing:** no hace falta herencia; basta con que los objetos tengan el **mismo método** (`guardar()`, `leer()`). Si no lo tienen → `AttributeError`.
- **Ventaja del polimorfismo:** añadir un tipo nuevo = **añadir una clase**, sin tocar los bucles ni las funciones que ya existían (sin `if/elif` por tipo ni `isinstance`).

1.7 Herencia múltiple y MRO

```
class Usuario:
    def __init__(self, nombre):
        self.nombre = nombre
        print("Configurando Usuario")
    def permisos(self): return ["leer"]

class Editor(Usuario):
    def __init__(self, nombre):
        super().__init__(nombre); print("Configurando Editor")
    def permisos(self): return super().permisos() + ["editar"]

class Moderador(Usuario):
    def __init__(self, nombre):
        super().__init__(nombre); print("Configurando Moderador")
    def permisos(self): return super().permisos() + ["borrar comentarios"]

class Administrador(Editor, Moderador):
    def __init__(self, nombre):
        super().__init__(nombre); print("Configurando Administrador")
    def permisos(self): return super().permisos() + ["gestionar usuarios"]

Administrador("Marina")
# Configurando Usuario / Moderador / Editor / Administrador   (al revés: cada uno imprime DESPUÉS de super())
print([c.__name__ for c in Administrador.__mro__])
# ['Administrador', 'Editor', 'Moderador', 'Usuario', 'object']
```

Reglas del MRO (Method Resolution Order):

1. Primero la propia clase, después los padres **de izquierda a derecha** según la declaración, y la clase común (`Usuario`) **al final y una sola vez**; termina en `object`.
2. `super()` **no** significa "mi padre": significa "**la siguiente clase del MRO del objeto**". En un `Administrador`, el `super()` de `Editor` llama a `Moderador`.
3. Los resultados que se construyen "a la vuelta" (`super().x() + [...]`) salen en **orden inverso** al MRO: `['leer', 'borrar comentarios', 'editar', 'gestionar usuarios']`.
4. Si una clase de la cadena **no llama a `super()`**, la cadena se corta y las siguientes clases del MRO no se ejecutan.
5. Cambiar el orden `class A(Moderador, Editor)` cambia el MRO y por tanto el orden de salida.

Inicialización cooperativa con `**kwargs` (P5): cada clase coge sus parámetros y pasa el resto.

```
class GPS:
    def __init__(self, latitud, longitud, **kwargs):
        self.latitud, self.longitud = latitud, longitud
        super().__init__(**kwargs)
class VehiculoInteligente(Vehiculo, GPS):     # Vehiculo también recibe **kwargs y llama a super()
    def __init__(self, flota, **kwargs):
        self.flota = flota
        super().__init__(**kwargs)
VehiculoInteligente(flota="F1", matricula="1234", modelo="X", latitud=40.4, longitud=-3.7)
```

1.8 Composición (práctica 15: sin herencia)

Un objeto **contiene** otros objetos en sus atributos ("tiene un"), frente a la herencia ("es un").

```
class Carrito:
    def __init__(self):
        self.lineas = []                        # lista de (articulo, cantidad)
    def agregar(self, articulo, cantidad):
        self.lineas.append((articulo, cantidad))
    def total(self):
        return sum(a.precio * c for a, c in self.lineas)
```

1.9 Comprehensions

```
[x * 2 for x in datos if x > 0]                    # if AL FINAL = FILTRA (puede haber menos)
["ALTO" if x >= 5 else "BAJO" for x in datos]      # if/else AL PRINCIPIO = TRANSFORMA (mismo nº)
[n for fila in matriz for n in fila]               # aplanar: 1º for = externo, 2º = interno
[[fila[c] for fila in m] for c in range(len(m[0]))]  # anidada (transponer)
{c.strip().capitalize() for c in ciudades}         # set: elimina repetidos (sin orden)
{k: v * 2 for k, v in d.items() if v > 0}          # dict
{p: lista.count(p) for p in lista}                 # contar apariciones
sum(x for x in datos)                              # expresión generadora (sin corchetes)
"COMPLETA" if n == 0 else "ÚLTIMAS" if n < 10 else "DISPONIBLE"   # ternario encadenado
```

1.10 Programación funcional

```
from functools import reduce

doble = lambda x: x * 2                                 # lambda: UNA expresión, sin return
list(map(lambda c: c["km"], carreras))                  # map: TRANSFORMA cada elemento
list(filter(lambda c: not c["tarjeta"], carreras))      # filter: SELECCIONA (True se queda)
reduce(lambda acum, c: acum + c["importe"], carreras, 0)  # reduce: ACUMULA (0 = valor inicial)
reduce(lambda a, b: a if a["km"] >= b["km"] else b, carreras)   # el máximo con reduce
sorted(carreras, key=lambda c: c["km"], reverse=True)    # ordenar (devuelve lista NUEVA)
max(recursos, key=lambda r: r.consumo())                # el objeto con mayor valor
max(diccionario, key=diccionario.get)                   # la CLAVE con mayor valor

def aplicar(funcion, valor):          # función que RECIBE una función (orden superior)
    return funcion(valor)
aplicar(redondear, 12.345)            # se pasa SIN paréntesis
```

```python
def crear_descuento(porcentaje):      # función que DEVUELVE una función (closure)
    def descontar(importe):
        return importe * (1 - porcentaje / 100)   # "recuerda" porcentaje
    return descontar                  # sin paréntesis
descontar_10 = crear_descuento(10); descontar_10(100)   # 90.0
```

- `map()` y `filter()` devuelven **iteradores**: usa `list(...)` para verlos o reutilizarlos (solo se recorren una vez).
- `reduce` **sin** valor inicial usa el primer elemento como acumulador. Con lista vacía y sin inicial → `TypeError`.

1.11 Recursión

```python
def suma_digitos(n):
    if n < 10:                        # 1) CASO BASE (detiene la recursión)
        return n
    return n % 10 + suma_digitos(n // 10) # 2) CASO RECURSIVO (problema más pequeño)

def total(lista):                     # sobre listas: primero + resto
    return 0 if not lista else lista[0] + total(lista[1:])

def es_palindromo(t):
    if len(t) <= 1: return True
    return t[0] == t[-1] and es_palindromo(t[1:-1])

def peso_total(caja):                 # estructuras anidadas: for + recursión en las sublistas
    return sum(peso_total(e) if isinstance(e, list) else e for e in caja)
```

- Sin caso base (o si nunca se alcanza) → **`RecursionError: maximum recursion depth exceeded`**. Límite por defecto ≈ 1000 llamadas (`sys.getrecursionlimit()`), por eso `ahorro_recursivo(1000, 5000)` falla y la versión iterativa no.
- En diccionarios/listas anidados: `isinstance(nodo, dict)` → recorre `nodo.values()`; `isinstance(nodo, list)` → recorre la lista; si no, es valor simple (caso base).

1.12 Funciones: parámetros

```python
def f(a, b, *args, modo="append", **kwargs): ...
# orden: posicionales → *args (tupla) → solo-por-nombre → **kwargs (diccionario)
f(1, 2, 3, 4, modo="x", extra=5)     # a=1 b=2 args=(3,4) modo='x' kwargs={'extra':5}
f(*lista, **dic)                     # DESEMPAQUETAR al llamar
return a, b                          # devuelve una TUPLA → x, y = f()
```

- Falta un posicional obligatorio → `TypeError: f() missing 1 required positional argument: 'b'`.
- Mismo parámetro por posición y por nombre → `TypeError: f() got multiple values for argument 'a'`.
- Lo que va después de `*args` **solo** se puede pasar por nombre (si lo pasas por posición acaba dentro de `args`).

1.13 Estructuras de datos (práctica 13)

```python
# LISTA (ordenada, mutable)
l.append(x); l.insert(i, x); l.extend(otra); l.remove(x); l.pop(); l.pop(i)
l.index(x); l.count(x); l.sort(); sorted(l); l.reverse(); l[::-1]; l[-2:]; len(l)
# TUPLA (ordenada, inmutable; sirve de clave de dict o elemento de set)
t = (1, 2); a, b = t; for x, y in lista_tuplas: ...; (1, 10, 0) > (1, 9, 7)  # compara por posiciones
# SET (sin orden, sin repetidos)
a | b (unión)  a & b (intersección)  a - b (diferencia)  a ^ b (simétrica)
s.add(x); s |= otro; a.isdisjoint(b); a <= b (subconjunto)
# DICCIONARIO
d.get(k, defecto); d.setdefault(k, []).append(x); d.items(); d.keys(); d.values()
d[k] = d.get(k, 0) + 1                  # contador
{**d1, **d2} o d1 | d2                  # fusionar: el último gana
sorted(d.items(), key=lambda kv: kv[1], reverse=True)   # ordenar por valor
from operator import itemgetter; sorted(d.items(), key=itemgetter(1))
```

1.14 Conversión de tipos y formato

```
int("12"); float("12.5"); float("12,50".replace(",", ".")); str(3.0) → "3.0"
int(3.9) → 3 (trunca)   round(3.5) → 4   round(4.5) → 4 (¡redondeo bancario al par!)
bool(0), bool(""), bool([]), bool(None) → False ; todo lo demás → True
int("80.00") → ValueError         # captúralo con try/except ValueError

f"{x:.2f}"   # 2 decimales            f"{x:>9.2f}"  # ancho 9 alineado a la derecha
f"{s:<10}"   # ancho 10 a la izquierda  f"{n:02d}"   # 2 dígitos con cero: 09
f"{x:,}"     # separador de miles     f"{obj!r}"   # usa repr
```

1.15 Programación modular y paquetes

```
import modulo                       # modulo.funcion()
from modulo import funcion          # funcion()
from modulo import funcion as f     # alias
import paquete.sub.modulo
from .config import TASA            # import RELATIVO (dentro de un paquete)
from ..modelos.socio import Socio   # .. = subir un nivel

if __name__ == "__main__":          # solo se ejecuta si lanzas ESTE archivo (no al importarlo)
    pruebas()
```

- Un **paquete** es una carpeta con `__init__.py` (la "recepción"): ahí se exponen las funciones públicas (`from .orquestador import ejecutar_pipeline`), `__version__` y `__all__`.
- Al importar un módulo **se ejecuta entero una sola vez** (después queda en caché en `sys.modules`).
- Python busca los módulos en `sys.path`: primero la carpeta del script. Un archivo propio llamado `random.py` **tapa** al módulo estándar.
- Ejecutar un módulo de un paquete con imports relativos: `python -m paquete.sub.modulo` desde la carpeta raíz.
- Las dependencias van en un solo sentido (sin imports circulares): `config → modelos → transformacion/carga → orquestador → __init__ → main`.

1.16 Errores típicos y su tipo

Error	Cuándo
`AttributeError`	El objeto no tiene ese atributo/método (`t.__pin`, duck typing fallido)
`TypeError`	Argumentos mal pasados, operar tipos incompatibles (`"3" + 4`), objeto no llamable
`NameError`	Variable o función no definida
`ValueError`	Valor con tipo correcto pero contenido inválido (`int("abc")`, `lista.remove(99)`)
`KeyError`	Clave inexistente en un diccionario (`d["x"]`; usa `d.get("x")`)
`IndexError`	Índice fuera de rango
`ZeroDivisionError`	División entre 0
`RecursionError`	Recursión sin caso base o demasiado profunda
`NotImplementedError`	Método "abstracto" que la subclase debía implementar

```
try:
    valor = int(texto)
except ValueError as e:
    print("Error:", e)
```

1.17 Checklist del examen

1. Autocompletado de IA **desactivado** (Copilot `*` = `false`, Inline Suggest y Next Edit Suggestions desmarcados). El IntelliSense normal sí se puede usar.
2. Puedes escribir en un `.py` con celdas `# %%` y pasarlo luego al `.ipynb` que se entrega.
3. En los ejercicios de "¿qué se imprime?", escribe la predicción como **comentario** (`#`) o en una celda Markdown, no como código.
4. Compara tu salida con la **salida esperada carácter a carácter** (espacios, decimales, `€`).
5. Antes de entregar: **Reiniciar kernel → Run All** y comprobar que todas las celdas se ejecutan de arriba abajo.

Parte 2 · Modelo de examen 16.1 resuelto

Indicaciones generales

- Esta practica no hay que entregarla.
- En el examen real saldrán de 10 a 15 preguntas como máximo y serán del mismo estilo y estructura de este modelo.
- Falta añadir preguntas del tema de programación modular.

Sección 1. Lectura de código: ¿qué se muestra por pantalla?

En esta sección **no hay que programar**. Lee cada fragmento y escribe lo que mostrará por pantalla **antes de ejecutarlo**. Después, ejecútalo y comprueba si acertaste. Si no acertaste, explica con tus palabras por qué.

Ejercicio 1.1 · El contador de tiendas

```
class Tienda:
    total_tiendas = 0

    def __init__(self, nombre):
        self.nombre = nombre
        Tienda.total_tiendas += 1

t1 = Tienda("Centro")
t2 = Tienda("Norte")
t1.total_tiendas = 10

print(Tienda.total_tiendas, t1.total_tiendas, t2.total_tiendas)
```

Pregunta extra: ¿por qué `t1` y `t2` no muestran el mismo número?

✅ Solución

```
class Tienda:
    total_tiendas = 0          # atributo de CLASE (compartido)

    def __init__(self, nombre):
        self.nombre = nombre
        Tienda.total_tiendas += 1   # se suma en la clase → 1, luego 2

t1 = Tienda("Centro")
t2 = Tienda("Norte")
t1.total_tiendas = 10   # NO toca la clase: crea un atributo de INSTANCIA en t1 que "tapa" al de clase

print(Tienda.total_tiendas, t1.total_tiendas, t2.total_tiendas)
# RESPUESTA: 2 10 2
# Python busca primero en el objeto y, si no lo encuentra, en la clase.
# t1 tiene su propio total_tiendas (10); t2 no, así que lee el de la clase (2).
```

🖥️ Salida:

```
2 10 2
```

Ejercicio 1.2 · Dos listas que parecen tres

```
pedidos_a = [101, 102, 103]
pedidos_b = pedidos_a
pedidos_c = pedidos_a[:]
```

```
pedidos_b.append(104)
pedidos_c.append(999)

print(pedidos_a)
print(pedidos_b)
print(pedidos_c)
print(pedidos_a is pedidos_b, pedidos_a is pedidos_c, pedidos_a == pedidos_b)
```

✅ **Solución**

```
pedidos_a = [101, 102, 103]
pedidos_b = pedidos_a        # ALIAS: el mismo objeto lista (dos nombres)
pedidos_c = pedidos_a[:]     # COPIA: objeto nuevo con los mismos elementos

pedidos_b.append(104)        # cambia la lista compartida por a y b
pedidos_c.append(999)        # solo cambia la copia

print(pedidos_a)
print(pedidos_b)
print(pedidos_c)
print(pedidos_a is pedidos_b, pedidos_a is pedidos_c, pedidos_a == pedidos_b)
# is  → ¿es el MISMO objeto en memoria?   == → ¿tienen el mismo CONTENIDO?
```

🖥️ **Salida:**

```
[101, 102, 103, 104]
[101, 102, 103, 104]
[101, 102, 103, 999]
True False True
```

Ejercicio 1.3 · Filtrar o clasificar

```
valoraciones = [4, 7, 5, 9, 3]

resultado_1 = [v for v in valoraciones if v >= 5]
resultado_2 = ["POSITIVA" if v >= 5 else "NEGATIVA" for v in valoraciones]

print(resultado_1)
print(resultado_2)
```

Pregunta extra: ¿por qué el `if` está en una posición distinta en cada comprehension?

✅ **Solución**

```
valoraciones = [4, 7, 5, 9, 3]

resultado_1 = [v for v in valoraciones if v >= 5]                          # if al FINAL = FILTRA
resultado_2 = ["POSITIVA" if v >= 5 else "NEGATIVA" for v in valoraciones]  # if/else al PRINCIPIO =
TRANSFORMA

print(resultado_1)
print(resultado_2)
# El if del final decide si el elemento ENTRA (puede haber menos elementos).
# El if/else del principio es una expresión ternaria que ELIGE el valor (mismo nº de elementos).
```

🖥️ **Salida:**

```
[7, 5, 9]
['NEGATIVA', 'POSITIVA', 'POSITIVA', 'POSITIVA', 'NEGATIVA']
```

Ejercicio 1.4 · Cuenta atrás

```
def cuenta_atras(n):
    if n == 0:
        return "¡Despegue!"
```

```
    return str(n) + " " + cuenta_atras(n - 1)

print(cuenta_atras(3))
```

Pregunta extra: ¿qué ocurriría si se borran las dos líneas del `if`?

✅ Solución

```
def cuenta_atras(n):
    if n == 0:                  # CASO BASE: detiene la recursión
        return "¡Despegue!"
    return str(n) + " " + cuenta_atras(n - 1)   # caso recursivo: problema más pequeño

print(cuenta_atras(3))
# "3 " + ("2 " + ("1 " + "¡Despegue!"))
# Sin el if no hay caso base: se llama infinitamente (3, 2, 1, 0, -1, ...) hasta superar
# el límite de llamadas de Python (~1000) → RecursionError: maximum recursion depth exceeded
```

🖥️ Salida:

```
3 2 1 ¡Despegue!
```

Ejercicio 1.5 · Recepción de un hotel

```
class Huesped:
    def bienvenida(self):
        return "Bienvenido"

class HuespedVIP(Huesped):
    def bienvenida(self):
        return super().bienvenida() + " - Tiene champán en la habitación"

class HuespedGrupo(Huesped):
    def bienvenida(self):
        return "Bienvenidos, el guía les espera en el vestíbulo"

for huesped in [Huesped(), HuespedVIP(), HuespedGrupo()]:
    print(huesped.bienvenida())
```

✅ Solución

```
class Huesped:
    def bienvenida(self):
        return "Bienvenido"

class HuespedVIP(Huesped):
    def bienvenida(self):
        # super() ejecuta el método de la clase PADRE y AMPLÍA su resultado
        return super().bienvenida() + " - Tiene champán en la habitación"

class HuespedGrupo(Huesped):
    def bienvenida(self):
        # SUSTITUYE por completo el método del padre (no usa super)
        return "Bienvenidos, el guía les espera en el vestíbulo"

for huesped in [Huesped(), HuespedVIP(), HuespedGrupo()]:
    print(huesped.bienvenida())   # POLIMORFISMO: misma llamada, comportamiento distinto
```

🖥️ Salida:

```
Bienvenido
Bienvenido - Tiene champán en la habitación
Bienvenidos, el guía les espera en el vestíbulo
```

Ejercicio 1.6 · Atributos con guiones bajos

```
class Tarjeta:
    def __init__(self):
        self.titular = "Ana"
        self._saldo = 500
        self.__pin = 1234

t = Tarjeta()
print(t.titular)        # Línea A
print(t._saldo)         # Línea B
print(t._Tarjeta__pin)  # Línea C
print(t.__pin)          # Línea D
```

Indica qué líneas funcionan, cuál produce un error y de qué tipo es ese error.

✅ Solución

```
class Tarjeta:
    def __init__(self):
        self.titular = "Ana"     # público
        self._saldo = 500        # "protegido" (convención: no tocar desde fuera, pero se puede)
        self.__pin = 1234        # name mangling → se guarda como _Tarjeta__pin

t = Tarjeta()
print(t.titular)        # Línea A → Ana
print(t._saldo)         # Línea B → 500
print(t._Tarjeta__pin)  # Línea C → 1234 (nombre real tras el name mangling)
try:
    print(t.__pin)      # Línea D → ERROR
except AttributeError as e:
    print("AttributeError:", e)
# Funcionan A, B y C. La D da AttributeError porque "__pin" no existe con ese nombre.
# El doble guion bajo NO hace el atributo realmente privado: solo le cambia el nombre.
```

🖥️ Salida:

```
Ana
500
1234
AttributeError: 'Tarjeta' object has no attribute '__pin'
```

Ejercicio 1.7 · Funciones que fabrican funciones

```
def crear_etiqueta(prefijo):
    def etiquetar(codigo):
        return prefijo + "-" + str(codigo)
    return etiquetar

factura = crear_etiqueta("FAC")
albaran = crear_etiqueta("ALB")

print(factura(15), albaran(7), factura(albaran(1)))
```

✅ Solución

```
def crear_etiqueta(prefijo):
    def etiquetar(codigo):
        return prefijo + "-" + str(codigo)   # CLOSURE: recuerda "prefijo"
    return etiquetar                          # devuelve la función (sin paréntesis)

factura = crear_etiqueta("FAC")
albaran = crear_etiqueta("ALB")

print(factura(15), albaran(7), factura(albaran(1)))
# factura(albaran(1)) → albaran(1) = "ALB-1" → factura("ALB-1") = "FAC-ALB-1"
```

🖥️ **Salida:**

```
FAC-15 ALB-7 FAC-ALB-1
```

Ejercicio 1.8 · La cafetería y la herencia múltiple

```python
class Bebida:
    def ingredientes(self):
        return ["café"]

class ConLeche(Bebida):
    def ingredientes(self):
        return super().ingredientes() + ["leche"]

class ConHielo(Bebida):
    def ingredientes(self):
        return super().ingredientes() + ["hielo"]

class Frappe(ConLeche, ConHielo):
    def ingredientes(self):
        return super().ingredientes() + ["nata"]

class CafeFrio(ConHielo, ConLeche):
    pass

print(Frappe().ingredientes())
print(CafeFrio().ingredientes())
print([clase.__name__ for clase in Frappe.__mro__])
```

Preguntas extra:

1. Dentro de `ConLeche`, `super()` no llama a `Bebida`. ¿A qué clase llama en un objeto `Frappe` y por qué?
2. ¿Por qué `"café"` aparece una sola vez si `ConLeche` y `ConHielo` heredan las dos de `Bebida`?
3. ¿Por qué cambia el orden de los ingredientes en `CafeFrio`?

✅ **Solución**

```python
class Bebida:
    def ingredientes(self):
        return ["café"]

class ConLeche(Bebida):
    def ingredientes(self):
        return super().ingredientes() + ["leche"]

class ConHielo(Bebida):
    def ingredientes(self):
        return super().ingredientes() + ["hielo"]

class Frappe(ConLeche, ConHielo):
    def ingredientes(self):
        return super().ingredientes() + ["nata"]

class CafeFrio(ConHielo, ConLeche):
    pass

print(Frappe().ingredientes())
print(CafeFrio().ingredientes())
print([clase.__name__ for clase in Frappe.__mro__])
# 1) super() NO significa "mi padre", sino "la SIGUIENTE clase en el MRO del objeto".
#    MRO de Frappe: Frappe → ConLeche → ConHielo → Bebida → object.
#    Así que en un Frappe, el super() de ConLeche llama a ConHielo.
# 2) "café" sale una vez porque Bebida aparece UNA sola vez en el MRO (al final),
#    así que su método se ejecuta una sola vez.
# 3) CafeFrio(ConHielo, ConLeche) → MRO: CafeFrio → ConHielo → ConLeche → Bebida.
#    Las listas se construyen "de vuelta": café + leche + hielo.
```

🖥️ **Salida:**

```
['café', 'hielo', 'leche', 'nata']
['café', 'leche', 'hielo']
['Frappe', 'ConLeche', 'ConHielo', 'Bebida', 'object']
```

Sección 2. Comprehensions

Ejercicio 2.1 · Precios con IVA

Una tienda guarda los precios **sin IVA** de sus productos:

```
precios = [8.50, 12.00, 3.20, 45.90, 19.99, 7.00]
```

Usa **una sola list comprehension** para obtener los precios **con IVA (21 %)**, pero solo de los productos que cuestan **más de 10 €** sin IVA. Redondea cada precio a 2 decimales.

Salida esperada

```
[14.52, 55.54, 24.19]
```

✅ **Solución**

```
precios = [8.50, 12.00, 3.20, 45.90, 19.99, 7.00]

precios_iva = [round(p * 1.21, 2) for p in precios if p > 10]
print(precios_iva)
```

🖥️ **Salida:**

```
[14.52, 55.54, 24.19]
```

Ejercicio 2.2 · Clasificar paquetes

Una empresa de mensajería tiene el peso de cada paquete en kilos:

```
pesos = [0.5, 3.2, 12.0, 1.1, 25.4, 7.8]
```

Crea con una list comprehension una lista en la que cada paquete aparezca como `"LIGERO"` si pesa menos de 5 kg o como `"PESADO"` en caso contrario. Después, muestra cuántos paquetes son pesados.

Salida esperada

```
['LIGERO', 'LIGERO', 'PESADO', 'LIGERO', 'PESADO', 'PESADO']
Paquetes pesados: 3
```

✅ **Solución**

```
pesos = [0.5, 3.2, 12.0, 1.1, 25.4, 7.8]

clasificacion = ["LIGERO" if p < 5 else "PESADO" for p in pesos]
print(clasificacion)
print("Paquetes pesados:", clasificacion.count("PESADO"))
```

🖥️ **Salida:**

```
['LIGERO', 'LIGERO', 'PESADO', 'LIGERO', 'PESADO', 'PESADO']
Paquetes pesados: 3
```

Ejercicio 2.3 · Ciudades sin repetir

En un formulario de clientes, las ciudades se escribieron de forma poco cuidada:

```
ciudades = ["Madrid", " madrid", "SEVILLA", "Valencia ", "sevilla", "Bilbao", "MADRID"]
```

Con **una set comprehension**, obtén las ciudades distintas, sin espacios sobrantes y con solo la primera letra en mayúscula. Muestra el resultado ordenado alfabéticamente y el número de ciudades distintas.

Salida esperada

```
['Bilbao', 'Madrid', 'Sevilla', 'Valencia']
Ciudades distintas: 4
```

✅ Solución

```
ciudades = ["Madrid", " madrid", "SEVILLA", "Valencia ", "sevilla", "Bilbao", "MADRID"]

distintas = {c.strip().capitalize() for c in ciudades}   # set → elimina repetidos
print(sorted(distintas))
print("Ciudades distintas:", len(distintas))
```

🖥️ Salida:

```
['Bilbao', 'Madrid', 'Sevilla', 'Valencia']
Ciudades distintas: 4
```

Ejercicio 2.4 · Butacas libres en un cine

Un cine guarda cuántas butacas quedan libres en cada sala para la sesión de esta noche:

```
butacas_libres = {"Sala 1": 0, "Sala 2": 34, "Sala 3": 6, "Sala 4": 0, "Sala 5": 120}
```

Con **comprehensions**, crea:

4. Una **lista** con los nombres de las salas completas (0 butacas libres). Para esta usa una list comprehension.
5. Un diccionario que asocie cada sala con un estado:

- `"COMPLETA"` si tiene 0 butacas libres;
- `"ÚLTIMAS PLAZAS"` si tiene menos de 10;
- `"DISPONIBLE"` en cualquier otro caso.

6. Un diccionario solo con las salas que **no** están completas, en el que el valor sea el **porcentaje de ocupación**, sabiendo que todas las salas tienen 150 butacas. Redondea a 1 decimal. **Salida esperada**

```
['Sala 1', 'Sala 4']
{'Sala 1': 'COMPLETA', 'Sala 2': 'DISPONIBLE', 'Sala 3': 'ÚLTIMAS PLAZAS', 'Sala 4': 'COMPLETA', 'Sala 5':
'DISPONIBLE'}
{'Sala 2': 77.3, 'Sala 3': 96.0, 'Sala 5': 20.0}
```

✅ Solución

```
butacas_libres = {"Sala 1": 0, "Sala 2": 34, "Sala 3": 6, "Sala 4": 0, "Sala 5": 120}
CAPACIDAD = 150

completas = [sala for sala, libres in butacas_libres.items() if libres == 0]

estados = {
    sala: "COMPLETA" if libres == 0 else "ÚLTIMAS PLAZAS" if libres < 10 else "DISPONIBLE"
    for sala, libres in butacas_libres.items()
}

ocupacion = {
    sala: round((CAPACIDAD - libres) / CAPACIDAD * 100, 1)
    for sala, libres in butacas_libres.items()
    if libres > 0
}

print(completas)
```

```
print(estados)
print(ocupacion)
```

🖥️ **Salida:**

```
['Sala 1', 'Sala 4']
{'Sala 1': 'COMPLETA', 'Sala 2': 'DISPONIBLE', 'Sala 3': 'ÚLTIMAS PLAZAS', 'Sala 4': 'COMPLETA', 'Sala 5':
'DISPONIBLE'}
{'Sala 2': 77.3, 'Sala 3': 96.0, 'Sala 5': 20.0}
```

Ejercicio 2.5 · Unir las cestas de la compra

Un supermercado online guarda los productos de cada cesta en una lista independiente:

```
cestas = [["pan", "leche"], ["huevos"], [], ["leche", "café", "pan"]]
```

7. Con una list comprehension **con dos** `for`, crea una única lista con todos los productos de todas las cestas.
8. Con una dictionary comprehension, crea un diccionario que indique cuántas veces aparece cada producto. Pista: puedes usar el método `count()` de las listas. **Salida esperada**

```
['pan', 'leche', 'huevos', 'leche', 'café', 'pan']
{'pan': 2, 'leche': 2, 'huevos': 1, 'café': 1}
```

✅ **Solución**

```
cestas = [["pan", "leche"], ["huevos"], [], ["leche", "café", "pan"]]

todos = [producto for cesta in cestas for producto in cesta]   # 1º for = externo, 2º = interno
conteo = {producto: todos.count(producto) for producto in todos}  # claves repetidas se sobrescriben

print(todos)
print(conteo)
```

🖥️ **Salida:**

```
['pan', 'leche', 'huevos', 'leche', 'café', 'pan']
{'pan': 2, 'leche': 2, 'huevos': 1, 'café': 1}
```

Sección 3. Programación funcional

Una empresa de taxis registra los trayectos del día. Trabaja con estos datos en todos los ejercicios de la sección:

```
carreras = [
    {"id": 1, "pasajero": "Raúl",    "km": 12.0, "importe": 18.5, "tarjeta": True},
    {"id": 2, "pasajero": "Sofía",   "km": 3.5,  "importe": 7.0,  "tarjeta": False},
    {"id": 3, "pasajero": "Iván",    "km": 28.0, "importe": 42.0, "tarjeta": True},
    {"id": 4, "pasajero": "Claudia", "km": 8.0,  "importe": 13.5, "tarjeta": True},
    {"id": 5, "pasajero": "Hugo",    "km": 15.0, "importe": 24.0, "tarjeta": False},
]
```

> El campo `tarjeta` indica si el pasajero pagó con tarjeta (`True`) o en efectivo (`False`).

Ejercicio 3.1 · Funciones lambda

9. Crea una función lambda llamada `con_suplemento` que reciba un importe y le sume 2.50 € de suplemento nocturno.
10. Crea una función lambda llamada `es_larga` que reciba unos kilómetros y devuelva `True` si son 10 o más.
11. Usa `sorted()` con una lambda para ordenar las carreras **de más a menos kilómetros** y muestra solo el nombre del pasajero. **Salida esperada**

```
21.0
False
['Iván', 'Hugo', 'Raúl', 'Claudia', 'Sofía']
```

> La primera línea muestra `con_suplemento(18.5)` y la segunda, `es_larga(3.5)`.

✅ Solución

```python
con_suplemento = lambda importe: importe + 2.50
es_larga = lambda km: km >= 10

print(con_suplemento(18.5))
print(es_larga(3.5))

ordenadas = sorted(carreras, key=lambda c: c["km"], reverse=True)
print([c["pasajero"] for c in ordenadas])
```

🖥️ Salida:

```
21.0
False
['Iván', 'Hugo', 'Raúl', 'Claudia', 'Sofía']
```

Ejercicio 3.2 · `map()`

Usa `map()` con una lambda para obtener:

12. La lista de kilómetros de todas las carreras.
13. Una lista de textos con el formato `"Carrera 1: 18.5 €"`. **Salida esperada**

```
[12.0, 3.5, 28.0, 8.0, 15.0]
['Carrera 1: 18.5 €', 'Carrera 2: 7.0 €', 'Carrera 3: 42.0 €', 'Carrera 4: 13.5 €', 'Carrera 5: 24.0 €']
```

✅ Solución

```python
kms = list(map(lambda c: c["km"], carreras))
textos = list(map(lambda c: f"Carrera {c['id']}: {c['importe']} €", carreras))

print(kms)
print(textos)
```

🖥️ Salida:

```
[12.0, 3.5, 28.0, 8.0, 15.0]
['Carrera 1: 18.5 €', 'Carrera 2: 7.0 €', 'Carrera 3: 42.0 €', 'Carrera 4: 13.5 €', 'Carrera 5: 24.0 €']
```

Ejercicio 3.3 · `filter()`

Usa `filter()` con una lambda para obtener:

14. Las carreras pagadas **en efectivo**. Muestra solo el nombre del pasajero.
15. Las carreras pagadas **con tarjeta y de 10 km o más**. Muestra solo su `id`. **Salida esperada**

```
['Sofía', 'Hugo']
[1, 3]
```

✅ Solución

```python
efectivo = list(filter(lambda c: not c["tarjeta"], carreras))
print([c["pasajero"] for c in efectivo])
```

```
tarjeta_largas = list(filter(lambda c: c["tarjeta"] and c["km"] >= 10, carreras))
print([c["id"] for c in tarjeta_largas])
```

🖥️ Salida:

```
['Sofía', 'Hugo']
[1, 3]
```

Ejercicio 3.4 · `reduce()`

Importa `reduce` desde el módulo `functools` y úsalo para calcular:

16. La **recaudación total** del día.
17. La **carrera más larga**. Muestra el nombre del pasajero.
18. Los **kilómetros recorridos en carreras pagadas con tarjeta**. Combina `filter()` y `reduce()`. **Salida esperada**

```
Recaudación total: 105.0 €
Carrera más larga: Iván
Km pagados con tarjeta: 48.0
```

✅ Solución

```
from functools import reduce

recaudacion = reduce(lambda acum, c: acum + c["importe"], carreras, 0)
mas_larga = reduce(lambda a, b: a if a["km"] >= b["km"] else b, carreras)   # sin valor inicial
km_tarjeta = reduce(lambda acum, c: acum + c["km"],
                    filter(lambda c: c["tarjeta"], carreras), 0)

print(f"Recaudación total: {recaudacion} €")
print("Carrera más larga:", mas_larga["pasajero"])
print("Km pagados con tarjeta:", km_tarjeta)
```

🖥️ Salida:

```
Recaudación total: 105.0 €
Carrera más larga: Iván
Km pagados con tarjeta: 48.0
```

Ejercicio 3.5 · Funciones que reciben funciones

Escribe una función `transformar_importes(carreras, operacion)` que reciba la lista de carreras y **una función**, y devuelva una lista con el resultado de aplicar esa función al importe de cada carrera. No uses `map()`: resuélvelo con una list comprehension.

Pruébala con:

- una función normal `a_dolares(x)`, que convierta euros a dólares con un cambio de 1.08 y redondee a 2 decimales;
- una lambda que añada un 10 % de propina, también redondeada a 2 decimales. **Salida esperada**

```
[19.98, 7.56, 45.36, 14.58, 25.92]
[20.35, 7.7, 46.2, 14.85, 26.4]
```

✅ Solución

```
def transformar_importes(carreras, operacion):
    """Aplica 'operacion' (una función) al importe de cada carrera."""
    return [operacion(c["importe"]) for c in carreras]

def a_dolares(x):
    return round(x * 1.08, 2)
```

```
print(transformar_importes(carreras, a_dolares))                    # función normal (sin paréntesis)
print(transformar_importes(carreras, lambda x: round(x * 1.10, 2)))  # lambda
```

🖥️ Salida:

```
[19.98, 7.56, 45.36, 14.58, 25.92]
[20.35, 7.7, 46.2, 14.85, 26.4]
```

Ejercicio 3.6 · Funciones que devuelven funciones

La empresa cobra un precio por kilómetro distinto según la franja horaria. Escribe una función `crear_tarifa(precio_km)` que **devuelva otra función**. Esa función recibirá unos kilómetros y devolverá el precio del trayecto, redondeado a 2 decimales.

```
tarifa_diurna = crear_tarifa(1.10)
tarifa_nocturna = crear_tarifa(1.35)

print(tarifa_diurna(10))
print(tarifa_nocturna(10))
print([tarifa_nocturna(c["km"]) for c in carreras])
```

Salida esperada

```
11.0
13.5
[16.2, 4.73, 37.8, 10.8, 20.25]
```

✅ Solución

```
def crear_tarifa(precio_km):
    def calcular(km):
        return round(km * precio_km, 2)   # closure: recuerda precio_km
    return calcular

tarifa_diurna = crear_tarifa(1.10)
tarifa_nocturna = crear_tarifa(1.35)

print(tarifa_diurna(10))
print(tarifa_nocturna(10))
print([tarifa_nocturna(c["km"]) for c in carreras])
```

🖥️ Salida:

```
11.0
13.5
[16.2, 4.73, 37.8, 10.8, 20.25]
```

Sección 4. Recursión

En esta sección **no puedes usar bucles** (`for` ni `while`) dentro de las funciones recursivas, salvo que el enunciado lo permita. Cada función debe tener un caso base claro.

Ejercicio 4.1 · Sumar los dígitos de un código

Escribe una función recursiva `suma_digitos(n)` que reciba un número entero positivo y devuelva la suma de sus cifras.

Pista: `n % 10` da la última cifra y `n // 10` quita la última cifra.

```
print(suma_digitos(4528))
print(suma_digitos(7))
```

Salida esperada

```
19
7
```

✅ Solución

```python
def suma_digitos(n):
    if n < 10:                          # caso base: una sola cifra
        return n
    return n % 10 + suma_digitos(n // 10)   # última cifra + suma del resto

print(suma_digitos(4528))
print(suma_digitos(7))
```

🖥️ Salida:

```
19
7
```

Ejercicio 4.2 · Total de una lista de gastos

Escribe una función recursiva `total_gastos(gastos)` que sume todos los valores de una lista. Usa la idea: *el total es el primer elemento más el total del resto de la lista*. El caso base es la lista vacía.

```python
print(total_gastos([12.5, 30, 7.5, 50]))
print(total_gastos([]))
```

Salida esperada

```
100.0
0
```

✅ Solución

```python
def total_gastos(gastos):
    if not gastos:                      # caso base: lista vacía
        return 0
    return gastos[0] + total_gastos(gastos[1:])   # primero + total del resto

print(total_gastos([12.5, 30, 7.5, 50]))
print(total_gastos([]))
```

🖥️ Salida:

```
100.0
0
```

Ejercicio 4.3 · ¿Es un palíndromo?

Un palíndromo es una palabra que se lee igual al derecho y al revés. Escribe una función recursiva `es_palindromo(texto)` que devuelva `True` o `False`.

Pista: compara el primer y el último carácter y, si coinciden, vuelve a llamar a la función con el texto sin esos dos caracteres.

```python
print(es_palindromo("reconocer"))
print(es_palindromo("radar"))
print(es_palindromo("python"))
```

Salida esperada

```
True
True
```

```
False
```

✅ Solución

```python
def es_palindromo(texto):
    if len(texto) <= 1:                # caso base: 0 o 1 letras
        return True
    if texto[0] != texto[-1]:          # caso base: extremos distintos
        return False
    return es_palindromo(texto[1:-1])  # quitar primer y último carácter

print(es_palindromo("reconocer"))
print(es_palindromo("radar"))
print(es_palindromo("python"))
```

🖥️ Salida:

```
True
True
False
```

Ejercicio 4.4 · Cajas dentro de cajas

Un almacén prepara envíos en los que unas cajas pueden ir **dentro de otras cajas**. Cada caja se representa con una lista y cada artículo con un número que indica su peso en kilos:

```python
envio = [2, [3, 1], [[4, 5], 2], [], 6]
```

En este envío hay una caja exterior con un artículo de 2 kg, una caja con dos artículos, una caja que contiene otra caja, una caja vacía y un artículo de 6 kg.

19. Escribe una función recursiva `peso_total(caja)` que devuelva los kilos que pesa todo el contenido.
20. Escribe una función recursiva `contar_articulos(caja)` que devuelva cuántos artículos hay en total. Las cajas no cuentan como artículos. Pista: usa `isinstance(elemento, list)` para saber si un elemento es una caja. En este ejercicio **sí puedes usar un `for`** para recorrer el contenido de cada caja, porque las cajas interiores se tratan con la llamada recursiva.

Salida esperada

```
Peso total: 23 kg
Número de artículos: 7
```

✅ Solución

```python
envio = [2, [3, 1], [[4, 5], 2], [], 6]

def peso_total(caja):
    total = 0
    for elemento in caja:
        if isinstance(elemento, list):
            total += peso_total(elemento)   # caja interior → recursión
        else:
            total += elemento               # artículo
    return total                            # caja vacía → 0 (caso base implícito)

def contar_articulos(caja):
    total = 0
    for elemento in caja:
        if isinstance(elemento, list):
            total += contar_articulos(elemento)
        else:
            total += 1
    return total

print(f"Peso total: {peso_total(envio)} kg")
print(f"Número de artículos: {contar_articulos(envio)}")
```

🖥️ **Salida:**

```
Peso total: 23 kg
Número de artículos: 7
```

Ejercicio 4.5 · Iterativo frente a recursivo

Una persona ahorra dinero en una cuenta que da un **2 % de interés al año**. Escribe dos funciones que calculen el dinero que tendrá tras `n` años, partiendo de un capital inicial:

- `ahorro_iterativo(capital, anios)`, con un bucle;
- `ahorro_recursivo(capital, anios)`, sin bucles. Redondea el resultado a 2 decimales al mostrarlo.

```
print(round(ahorro_iterativo(1000, 5), 2))
print(round(ahorro_recursivo(1000, 5), 2))
```

Salida esperada

```
1104.08
1104.08
```

Pregunta: ¿qué ocurriría con la versión recursiva si se llamara con `anios = 5000`?

✅ **Solución**

```python
def ahorro_iterativo(capital, anios):
    for _ in range(anios):
        capital = capital * 1.02
    return capital

def ahorro_recursivo(capital, anios):
    if anios == 0:                      # caso base
        return capital
    return ahorro_recursivo(capital * 1.02, anios - 1)

print(round(ahorro_iterativo(1000, 5), 2))
print(round(ahorro_recursivo(1000, 5), 2))
# Con anios = 5000 la versión recursiva lanza RecursionError: Python limita
# las llamadas encadenadas (~1000, ver sys.getrecursionlimit()). La iterativa funciona.
```

🖥️ **Salida:**

```
1104.08
1104.08
```

Sección 5. Clases y objetos

Ejercicio 5.1 · Artículos de una tienda de deportes

Crea una clase `Articulo` con:

- **Atributos de instancia:** `nombre`, `precio` y `stock`. El `stock` tiene valor por defecto 0.
- **Método `valor_inventario()`:** devuelve el precio multiplicado por el stock.
- **Método `vender(cantidad)`:**
- si hay stock suficiente, lo resta y devuelve `True`;
- si no lo hay, no cambia nada y devuelve `False`.
- **Método `reponer(cantidad)`:** suma unidades al stock.
- **Método `__str__`:** devuelve un texto como `Balón de fútbol | 25.0 € | 10 uds.`

```
a1 = Articulo("Balón de fútbol", 25.0, 10)
a2 = Articulo("Cinta de pádel", 9.5)
```

```
print(a1)
print(a2)
print(a1.vender(3))
print(a2.vender(1))
a2.reponer(20)
print(a1)
print(a2)
print(a1.valor_inventario(), a2.valor_inventario())
```

Salida esperada

```
Balón de fútbol | 25.0 € | 10 uds.
Cinta de pádel | 9.5 € | 0 uds.
True
False
Balón de fútbol | 25.0 € | 7 uds.
Cinta de pádel | 9.5 € | 20 uds.
175.0 190.0
```

✅ **Solución**

```
class Articulo:
    def __init__(self, nombre, precio, stock=0):
        self.nombre = nombre
        self.precio = precio
        self.stock = stock

    def valor_inventario(self):
        return self.precio * self.stock

    def vender(self, cantidad):
        if cantidad <= self.stock:
            self.stock -= cantidad
            return True
        return False

    def reponer(self, cantidad):
        self.stock += cantidad

    def __str__(self):
        return f"{self.nombre} | {self.precio} € | {self.stock} uds."

a1 = Articulo("Balón de fútbol", 25.0, 10)
a2 = Articulo("Cinta de pádel", 9.5)

print(a1)
print(a2)
print(a1.vender(3))
print(a2.vender(1))
a2.reponer(20)
print(a1)
print(a2)
print(a1.valor_inventario(), a2.valor_inventario())
```

🖥️ **Salida:**

```
Balón de fútbol | 25.0 € | 10 uds.
Cinta de pádel | 9.5 € | 0 uds.
True
False
Balón de fútbol | 25.0 € | 7 uds.
Cinta de pádel | 9.5 € | 20 uds.
175.0 190.0
```

Ejercicio 5.2 · Cuentas de un banco

Crea una clase `Cuenta` con:

- **Atributo de clase `total_cuentas`:** cuenta cuántas cuentas se han creado.

- **Atributo de clase `COMISION_RETIRADA`:** vale 1.5 €.
- **Atributos de instancia:** `titular` y `_saldo`. El saldo inicial vale 0.0 si no se indica. El guion bajo indica que el saldo no debe modificarse directamente desde fuera de la clase.
- **Método `ingresar(importe)`:** suma el importe al saldo solo si es positivo.
- **Método `retirar(importe)`:**
- resta el importe **más la comisión**, solo si hay saldo suficiente para ambos;
- devuelve `True` o `False` según se haya podido retirar o no.
- **Método `consultar_saldo()`:** devuelve el saldo.
- **Método de clase `cuentas_creadas()`:** usa `@classmethod` y devuelve el valor de `total_cuentas`.
- **Método `__repr__`:** devuelve un texto como `Cuenta('Laura', 150.0)`.

```
c1 = Cuenta("Laura", 100.0)
c2 = Cuenta("Diego")

c1.ingresar(50)
c1.ingresar(-20)
print(c1.retirar(48.5))
print(c2.retirar(10))
print(c1, c2)
print("Cuentas creadas:", Cuenta.cuentas_creadas())
```

Salida esperada

```
True
False
Cuenta('Laura', 100.0) Cuenta('Diego', 0.0)
Cuentas creadas: 2
```

✅ Solución

```
class Cuenta:
    total_cuentas = 0            # atributo de clase
    COMISION_RETIRADA = 1.5      # constante de clase

    def __init__(self, titular, saldo=0.0):
        self.titular = titular
        self._saldo = saldo
        Cuenta.total_cuentas += 1

    def ingresar(self, importe):
        if importe > 0:
            self._saldo += importe

    def retirar(self, importe):
        coste = importe + Cuenta.COMISION_RETIRADA
        if coste <= self._saldo:
            self._saldo -= coste
            return True
        return False

    def consultar_saldo(self):
        return self._saldo

    @classmethod
    def cuentas_creadas(cls):     # recibe la CLASE (cls), no el objeto
        return cls.total_cuentas

    def __repr__(self):
        return f"Cuenta('{self.titular}', {self._saldo})"

c1 = Cuenta("Laura", 100.0)
c2 = Cuenta("Diego")

c1.ingresar(50)
c1.ingresar(-20)
print(c1.retirar(48.5))
print(c2.retirar(10))
print(c1, c2)                     # print usa __str__; si no existe, usa __repr__
print("Cuentas creadas:", Cuenta.cuentas_creadas())
```

🖥️ **Salida:**

```
True
False
Cuenta('Laura', 100.0) Cuenta('Diego', 0.0)
Cuentas creadas: 2
```

Ejercicio 5.3 · Carrito de la compra (composición)

Usando la clase `Articulo` del ejercicio 5.1, crea una clase `Carrito` que **contenga** una lista de líneas de compra. Cada línea es una tupla `(articulo, cantidad)`.

- **Método `agregar(articulo, cantidad)`:** añade una línea al carrito.
- **Método `total()`:** devuelve el importe total del carrito.
- **Método `resumen()`:** muestra cada línea con el formato `3 x Balón de fútbol = 75.0 €` y, al final, el total.

```
carrito = Carrito()
carrito.agregar(Articulo("Balón de fútbol", 25.0, 10), 3)
carrito.agregar(Articulo("Cinta de pádel", 9.5, 20), 2)
carrito.agregar(Articulo("Botella deportiva", 15.0, 5), 1)
carrito.resumen()
```

Salida esperada

```
3 x Balón de fútbol = 75.0 €
2 x Cinta de pádel = 19.0 €
1 x Botella deportiva = 15.0 €
TOTAL: 109.0 €
```

✅ **Solución**

```
# Usa la clase Articulo del 5.1 (composición: el Carrito CONTIENE artículos)
class Carrito:
    def __init__(self):
        self.lineas = []          # lista de tuplas (articulo, cantidad)

    def agregar(self, articulo, cantidad):
        self.lineas.append((articulo, cantidad))

    def total(self):
        return sum(art.precio * cant for art, cant in self.lineas)

    def resumen(self):
        for art, cant in self.lineas:
            print(f"{cant} x {art.nombre} = {art.precio * cant} €")
        print(f"TOTAL: {self.total()} €")

carrito = Carrito()
carrito.agregar(Articulo("Balón de fútbol", 25.0, 10), 3)
carrito.agregar(Articulo("Cinta de pádel", 9.5, 20), 2)
carrito.agregar(Articulo("Botella deportiva", 15.0, 5), 1)
carrito.resumen()
```

🖥️ **Salida:**

```
3 x Balón de fútbol = 75.0 €
2 x Cinta de pádel = 19.0 €
1 x Botella deportiva = 15.0 €
TOTAL: 109.0 €
```

Ejercicio 5.4 · Lista de objetos

Con la clase `Articulo` del ejercicio 5.1 y esta lista:

```
catalogo = [
    Articulo("Esterilla", 30.0, 12),
    Articulo("Comba", 12.0, 0),
    Articulo("Bicicleta estática", 180.0, 3),
    Articulo("Mancuernas", 45.0, 0),
]
```

Resuelve cada apartado en **una sola línea**, usando comprehensions o `sorted()`:

21. Los nombres de los artículos sin stock.
22. Un diccionario `nombre → valor_inventario()`.
23. El catálogo ordenado por precio de menor a mayor. Muestra cada artículo con `print()`. **Salida esperada**

```
['Comba', 'Mancuernas']
{'Esterilla': 360.0, 'Comba': 0.0, 'Bicicleta estática': 540.0, 'Mancuernas': 0.0}
Comba | 12.0 € | 0 uds.
Esterilla | 30.0 € | 12 uds.
Mancuernas | 45.0 € | 0 uds.
Bicicleta estática | 180.0 € | 3 uds.
```

✅ Solución

```
catalogo = [
    Articulo("Esterilla", 30.0, 12),
    Articulo("Comba", 12.0, 0),
    Articulo("Bicicleta estática", 180.0, 3),
    Articulo("Mancuernas", 45.0, 0),
]

print([a.nombre for a in catalogo if a.stock == 0])
print({a.nombre: a.valor_inventario() for a in catalogo})
[print(a) for a in sorted(catalogo, key=lambda a: a.precio)]   # una línea (o un for normal)
```

🖥️ Salida:

```
['Comba', 'Mancuernas']
{'Esterilla': 360.0, 'Comba': 0.0, 'Bicicleta estática': 540.0, 'Mancuernas': 0.0}
Comba | 12.0 € | 0 uds.
Esterilla | 30.0 € | 12 uds.
Mancuernas | 45.0 € | 0 uds.
Bicicleta estática | 180.0 € | 3 uds.
```

Sección 6. Sobrecarga de operadores y conversión de tipos

Ejercicio 6.1 · Sumar y comparar importes

Crea una clase `Importe` que guarde una `cantidad` en euros. Haz que funcionen estas operaciones:

- `+` entre dos importes, que devuelve un **nuevo** `Importe`;
- `==` y `<`, que comparan las cantidades;
- `str()`, que muestra el importe con 2 decimales y el símbolo `€`;
- `float()`, que devuelve la cantidad como número decimal.

```
a = Importe(19.99)
b = Importe(5.01)
c = Importe(25)

total = a + b
print(total)
print(total == c)
print(a < b)
print(float(a) * 2)
```

Salida esperada

```
25.00 €
True
False
39.98
```

✅ Solución

```python
class Importe:
    def __init__(self, cantidad):
        self.cantidad = cantidad

    def __add__(self, otro):              # a + b
        return Importe(self.cantidad + otro.cantidad)   # devuelve un objeto NUEVO

    def __eq__(self, otro):               # a == b
        return round(self.cantidad, 2) == round(otro.cantidad, 2)

    def __lt__(self, otro):               # a < b
        return self.cantidad < otro.cantidad

    def __str__(self):                    # str(a) / print(a)
        return f"{self.cantidad:.2f} €"

    def __float__(self):                  # float(a)
        return float(self.cantidad)

a = Importe(19.99)
b = Importe(5.01)
c = Importe(25)

total = a + b
print(total)
print(total == c)
print(a < b)
print(float(a) * 2)
```

🖥️ Salida:

```
25.00 €
True
False
39.98
```

Ejercicio 6.2 · Lista de reproducción

Crea una clase `ListaReproduccion` con un atributo `nombre` y una lista interna de canciones. Haz que funcionen estas operaciones:

- `len(lista)`, que devuelve el número de canciones;
- `"titulo" in lista`, que indica si una canción está en la lista;
- `lista += "titulo"`, que añade una canción y devuelve la propia lista;
- `str(lista)`, que muestra un texto como `Viaje (3 canciones)`.

```python
viaje = ListaReproduccion("Viaje")
viaje += "Canción A"
viaje += "Canción B"
viaje += "Canción C"

print(viaje)
print(len(viaje))
print("Canción B" in viaje)
print("Canción Z" in viaje)
```

Salida esperada

```
Viaje (3 canciones)
3
```

```
True
False
```

✅ Solución

```python
class ListaReproduccion:
    def __init__(self, nombre):
        self.nombre = nombre
        self._canciones = []

    def __len__(self):                    # len(lista)
        return len(self._canciones)

    def __contains__(self, titulo):       # "x" in lista
        return titulo in self._canciones

    def __iadd__(self, titulo):           # lista += "x"
        self._canciones.append(titulo)
        return self                       # ¡OBLIGATORIO! si no, viaje pasaría a valer None

    def __str__(self):
        return f"{self.nombre} ({len(self)} canciones)"

viaje = ListaReproduccion("Viaje")
viaje += "Canción A"
viaje += "Canción B"
viaje += "Canción C"

print(viaje)
print(len(viaje))
print("Canción B" in viaje)
print("Canción Z" in viaje)
```

🖥️ Salida:

```
Viaje (3 canciones)
3
True
False
```

Ejercicio 6.3 · Precios escritos a mano

Un restaurante recibe los precios de su carta como texto, con coma decimal y el símbolo del euro, por ejemplo `"12,50 €"`.

24. Crea una clase `Precio` que guarde el valor en euros como número decimal.
25. Añade un **método estático** `desde_texto(texto)` que reciba un texto como `"7,25 €"` y devuelva un objeto `Precio`.
26. Haz que:

- `float(precio)` devuelva el valor en euros;
- `bool(precio)` devuelva `False` si el precio es 0 (plato gratuito) y `True` en caso contrario;
- `str(precio)` muestre el formato `7.25 €`, con 2 decimales.

```python
carta = ["12,50 €", "0,00 €", "7,25 €", "18,00 €"]
precios = [Precio.desde_texto(t) for t in carta]

print([float(p) for p in precios])
print("Total carta:", sum(float(p) for p in precios))
print("Platos gratuitos:", len([p for p in precios if not p]))
print(precios[2])
```

Salida esperada

```
[12.5, 0.0, 7.25, 18.0]
Total carta: 37.75
```

```
Platos gratuitos: 1
7.25 €
```

✅ **Solución**

```python
class Precio:
    def __init__(self, euros):
        self.euros = float(euros)

    @staticmethod
    def desde_texto(texto):               # no usa self ni cls
        limpio = texto.replace("€", "").strip().replace(",", ".")
        return Precio(float(limpio))

    def __float__(self):
        return self.euros

    def __bool__(self):                   # bool(p) / if p / not p
        return self.euros != 0

    def __str__(self):
        return f"{self.euros:.2f} €"

carta = ["12,50 €", "0,00 €", "7,25 €", "18,00 €"]
precios = [Precio.desde_texto(t) for t in carta]

print([float(p) for p in precios])
print("Total carta:", sum(float(p) for p in precios))
print("Platos gratuitos:", len([p for p in precios if not p]))
print(precios[2])
```

🖥️ **Salida:**

```
[12.5, 0.0, 7.25, 18.0]
Total carta: 37.75
Platos gratuitos: 1
7.25 €
```

Sección 7. Herencia, polimorfismo y herencia múltiple

Ejercicio 7.1 · Peajes de una autopista

Una autopista cobra peajes distintos según el tipo de vehículo.

27. Crea una superclase `Vehiculo` con los atributos `matricula` y `kilometros`, y un método `peaje()` que devuelva 0.10 € por kilómetro.
28. Crea estas subclases, que **sobrescriben** `peaje()`:

    - `Moto`: paga la mitad que un vehículo normal;
    - `Camion`: tiene un atributo adicional `ejes` y paga lo mismo que un vehículo normal **más 2 € por cada eje**. Usa `super()` tanto en el constructor como en `peaje()`.

29. Crea una lista con un `Vehiculo`, una `Moto` y un `Camion` y recórrela con **un único bucle**, mostrando la matrícula y el peaje redondeado a 2 decimales.
30. Muestra el total recaudado.

```python
vehiculos = [
    Vehiculo("1234-ABC", 100),
    Moto("5678-DEF", 100),
    Camion("9012-GHI", 100, 4),
]
```

Salida esperada

```
1234-ABC → 10.0 €
5678-DEF → 5.0 €
```

```
9012-GHI → 18.0 €
Total recaudado: 33.0 €
```

Pregunta: ¿en qué parte de tu código se aplica el polimorfismo?

✅ Solución

```python
class Vehiculo:
    def __init__(self, matricula, kilometros):
        self.matricula = matricula
        self.kilometros = kilometros

    def peaje(self):
        return self.kilometros * 0.10

class Moto(Vehiculo):
    def peaje(self):
        return super().peaje() / 2

class Camion(Vehiculo):
    def __init__(self, matricula, kilometros, ejes):
        super().__init__(matricula, kilometros)   # reutiliza el constructor del padre
        self.ejes = ejes

    def peaje(self):
        return super().peaje() + 2 * self.ejes

vehiculos = [
    Vehiculo("1234-ABC", 100),
    Moto("5678-DEF", 100),
    Camion("9012-GHI", 100, 4),
]

total = 0
for v in vehiculos:
    importe = round(v.peaje(), 2)       # ← POLIMORFISMO: misma llamada, cada clase su versión
    print(f"{v.matricula} → {importe} €")
    total += importe
print(f"Total recaudado: {round(total, 2)} €")
```

🖥️ Salida:

```
1234-ABC → 10.0 €
5678-DEF → 5.0 €
9012-GHI → 18.0 €
Total recaudado: 33.0 €
```

Ejercicio 7.2 · Suscripciones de una plataforma de series

Una plataforma de series ofrece varios tipos de suscripción.

31. Crea una superclase `Suscripcion` con `usuario` y `cuota_base`, y los métodos:

- `precio_mensual()`, que devuelve la cuota base;
- `ficha()`, que devuelve `"<usuario> (<tipo>): <precio> €"`. `<tipo>` es el nombre de la clase y `<precio>` se muestra con 2 decimales. Pista: `type(self).__name__`.

32. Crea estas subclases:

- `Familiar`: recibe además `perfiles_extra` y cuesta la cuota base más 3 € por cada perfil extra;
- `Anual`: tiene un 15 % de descuento sobre la cuota base;
- `ConAnuncios`: cuesta el 60 % de la cuota base.

33. Recorre la lista de suscripciones mostrando la ficha de cada una y calcula los ingresos mensuales totales con `sum()` y una comprehension.

```python
suscripciones = [
    Suscripcion("Carlos", 12),
```

```
    Familiar("Lucía", 12, 2),
    Anual("Sergio", 12),
    ConAnuncios("Nerea", 12),
]
```

Salida esperada

```
Carlos (Suscripcion): 12.00 €
Lucía (Familiar): 18.00 €
Sergio (Anual): 10.20 €
Nerea (ConAnuncios): 7.20 €
Ingresos mensuales: 47.40 €
```

✅ **Solución**

```python
class Suscripcion:
    def __init__(self, usuario, cuota_base):
        self.usuario = usuario
        self.cuota_base = cuota_base

    def precio_mensual(self):
        return self.cuota_base

    def ficha(self):
        return f"{self.usuario} ({type(self).__name__}): {self.precio_mensual():.2f} €"

class Familiar(Suscripcion):
    def __init__(self, usuario, cuota_base, perfiles_extra):
        super().__init__(usuario, cuota_base)
        self.perfiles_extra = perfiles_extra

    def precio_mensual(self):
        return super().precio_mensual() + 3 * self.perfiles_extra

class Anual(Suscripcion):
    def precio_mensual(self):
        return super().precio_mensual() * 0.85

class ConAnuncios(Suscripcion):
    def precio_mensual(self):
        return super().precio_mensual() * 0.60

suscripciones = [
    Suscripcion("Carlos", 12),
    Familiar("Lucía", 12, 2),
    Anual("Sergio", 12),
    ConAnuncios("Nerea", 12),
]

for s in suscripciones:
    print(s.ficha())
ingresos = sum(s.precio_mensual() for s in suscripciones)
print(f"Ingresos mensuales: {ingresos:.2f} €")
```

🖥️ **Salida:**

```
Carlos (Suscripcion): 12.00 €
Lucía (Familiar): 18.00 €
Sergio (Anual): 10.20 €
Nerea (ConAnuncios): 7.20 €
Ingresos mensuales: 47.40 €
```

Ejercicio 7.3 · Impresora multifunción (herencia múltiple)

34. Crea una clase `EquipoOficina` con un atributo `modelo` y un método `encender()` que muestre `"<modelo>: encendido"`.
35. Crea dos clases que hereden de `EquipoOficina`:

- `Impresora`, con un método `imprimir(documento)` que muestre `"Imprimiendo <documento>"`;

- `Escaner`, con un método `escanear(documento)` que muestre `"Escaneando <documento>"`.

36. Crea una clase `Multifuncion` que herede de `Impresora` **y** de `Escaner`, con un método `fotocopiar(documento)` que reutilice los dos anteriores.
37. Además, cada clase debe sobrescribir un método `descripcion()` que use `super()`, para que en un objeto `Multifuncion` se obtenga la descripción completa.

```
equipo = Multifuncion("OfiPro 300")
equipo.encender()
equipo.fotocopiar("contrato.pdf")
print(equipo.descripcion())
print([clase.__name__ for clase in Multifuncion.__mro__])
```

Salida esperada

```
OfiPro 300: encendido
Escaneando contrato.pdf
Imprimiendo contrato.pdf
Equipo de oficina + Escáner + Impresora + Multifunción
['Multifuncion', 'Impresora', 'Escaner', 'EquipoOficina', 'object']
```

Pregunta: explica con tus palabras por qué la descripción sale en ese orden. Relaciónalo con el MRO que se muestra en la última línea.

✅ **Solución**

```
class EquipoOficina:
    def __init__(self, modelo):
        self.modelo = modelo

    def encender(self):
        print(f"{self.modelo}: encendido")

    def descripcion(self):
        return "Equipo de oficina"

class Impresora(EquipoOficina):
    def imprimir(self, documento):
        print(f"Imprimiendo {documento}")

    def descripcion(self):
        return super().descripcion() + " + Impresora"

class Escaner(EquipoOficina):
    def escanear(self, documento):
        print(f"Escaneando {documento}")

    def descripcion(self):
        return super().descripcion() + " + Escáner"

class Multifuncion(Impresora, Escaner):
    def fotocopiar(self, documento):
        self.escanear(documento)      # reutiliza Escaner
        self.imprimir(documento)      # reutiliza Impresora

    def descripcion(self):
        return super().descripcion() + " + Multifunción"

equipo = Multifuncion("OfiPro 300")
equipo.encender()
equipo.fotocopiar("contrato.pdf")
print(equipo.descripcion())
print([clase.__name__ for clase in Multifuncion.__mro__])
# Cada super() llama a la SIGUIENTE clase del MRO: Multifuncion → Impresora → Escaner → EquipoOficina.
# La cadena llega hasta EquipoOficina y el texto se construye "a la vuelta":
# "Equipo de oficina" + " + Escáner" + " + Impresora" + " + Multifunción".
```

🖥️ **Salida:**

```
OfiPro 300: encendido
Escaneando contrato.pdf
Imprimiendo contrato.pdf
Equipo de oficina + Escáner + Impresora + Multifunción
['Multifuncion', 'Impresora', 'Escaner', 'EquipoOficina', 'object']
```

Ejercicio 7.4 · Permisos de usuarios en una aplicación (herencia múltiple con `super()`)

Una aplicación de gestión de contenidos tiene varios tipos de usuario. Cada tipo **suma** permisos a los del tipo del que hereda.

Parte A. Crear las clases

Clase	Hereda de	Permiso que añade
`Usuario`	—	`"leer"`
`Editor`	`Usuario`	`"editar"`
`Moderador`	`Usuario`	`"borrar comentarios"`
`Administrador`	`Editor` y `Moderador`, en ese orden	`"gestionar usuarios"`

Requisitos:

38. Todas las clases reciben solo el parámetro `nombre`. `Usuario` lo guarda como atributo.
39. El constructor de **cada** clase llama primero a `super().__init__(nombre)` y **después** muestra `"Configurando <Clase>"`. `Usuario` no llama a `super()` y solo muestra su mensaje.
40. Cada clase tiene un método `permisos()` que devuelve la lista de permisos. Las subclases **no repiten** los permisos de sus padres: los obtienen con `super().permisos()` y les añaden el suyo.
41. `Usuario` tiene además un método `puede(accion)` que devuelve `True` si la acción está en `self.permisos()`.

Parte B. Probar la clase

```
admin = Administrador("Marina")
print(admin.permisos())
print(admin.puede("borrar comentarios"), admin.puede("borrar la base de datos"))
print([clase.__name__ for clase in Administrador.__mro__])
```

Salida esperada

```
Configurando Usuario
Configurando Moderador
Configurando Editor
Configurando Administrador
['leer', 'borrar comentarios', 'editar', 'gestionar usuarios']
True False
['Administrador', 'Editor', 'Moderador', 'Usuario', 'object']
```

Parte C. Preguntas

42. ¿Por qué `"Configurando Usuario"` aparece **una sola vez**, si `Editor` y `Moderador` llaman los dos a `super().__init__()`?
43. Cambia temporalmente `Editor.permisos()` para que devuelva directamente `["leer", "editar"]`, sin usar `super()`. ¿Qué permiso pierde el administrador? ¿Por qué?
44. Cambia el orden de la herencia a `class Administrador(Moderador, Editor)`. ¿Qué cambia en la salida?

✅ Solución

```
class Usuario:
    def __init__(self, nombre):
        self.nombre = nombre
        print("Configurando Usuario")

    def permisos(self):
        return ["leer"]

    def puede(self, accion):
```

```
        return accion in self.permisos()

class Editor(Usuario):
    def __init__(self, nombre):
        super().__init__(nombre)
        print("Configurando Editor")

    def permisos(self):
        return super().permisos() + ["editar"]

class Moderador(Usuario):
    def __init__(self, nombre):
        super().__init__(nombre)
        print("Configurando Moderador")

    def permisos(self):
        return super().permisos() + ["borrar comentarios"]

class Administrador(Editor, Moderador):
    def __init__(self, nombre):
        super().__init__(nombre)
        print("Configurando Administrador")

    def permisos(self):
        return super().permisos() + ["gestionar usuarios"]

admin = Administrador("Marina")
print(admin.permisos())
print(admin.puede("borrar comentarios"), admin.puede("borrar la base de datos"))
print([clase.__name__ for clase in Administrador.__mro__])
# C1) MRO = Administrador → Editor → Moderador → Usuario. Cada super() salta a la siguiente
#     del MRO (el de Editor va a Moderador, no a Usuario), así que Usuario se ejecuta UNA vez.
#     Los mensajes salen al revés porque cada clase imprime DESPUÉS de llamar a super().
# C2) Si Editor.permisos() devuelve ["leer", "editar"] sin super(), se corta la cadena:
#     nunca se llama a Moderador.permisos() → el admin pierde "borrar comentarios".
# C3) Con class Administrador(Moderador, Editor): MRO Admin → Moderador → Editor → Usuario.
#     Mensajes: Usuario, Editor, Moderador, Administrador.
#     Permisos: ['leer', 'editar', 'borrar comentarios', 'gestionar usuarios'].
```

🖥️ **Salida:**

```
Configurando Usuario
Configurando Moderador
Configurando Editor
Configurando Administrador
['leer', 'borrar comentarios', 'editar', 'gestionar usuarios']
True False
['Administrador', 'Editor', 'Moderador', 'Usuario', 'object']
```

Sección 8. Refactorización: de código espagueti a clases

Una agencia de viajes tiene un programa que **funciona**, pero está escrito todo seguido. Tiene listas con posiciones difíciles de recordar, muchos `if/elif` según el tipo de reserva y cálculos repetidos. Cada vez que la agencia quiere vender un nuevo tipo de reserva hay que tocar el programa en varios sitios.

Tu tarea es **reorganizarlo usando clases, objetos, herencia y polimorfismo**, sin cambiar lo que muestra por pantalla.

Parte A. Analizar el código original

Copia el código, ejecútalo y anota su salida. **No lo modifiques todavía.**

```
# Programa original de la agencia de viajes
# Cada reserva: [tipo, cliente, cantidad, precio_unitario, destino]
reservas = [
    ["vuelo", "Ana López", 2, 120.0, "Madrid-Roma"],
    ["hotel", "Luis Pérez", 3, 85.0, "Hotel Mar Azul"],
    ["tren", "Marta Gil", 1, 45.0, "Madrid-Sevilla"],
    ["vuelo", "Pablo Ruiz", 1, 300.0, "Madrid-Nueva York"],
```

```
    ["hotel", "Eva Sanz", 2, 140.0, "Hotel Centro"],
]

total = 0
for r in reservas:
    if r[0] == "vuelo":
        precio = r[2] * r[3]
        precio = precio + 25
        if r[3] > 200:
            precio = precio + r[2] * 30
        print("VUELO | " + r[1] + " | " + r[4] + " | " + str(r[2]) + " pasajeros | " + str(round(precio, 2))
+ " €")
    elif r[0] == "hotel":
        precio = r[2] * r[3]
        if r[2] >= 3:
            precio = precio * 0.9
        precio = precio + r[2] * 1.5
        print("HOTEL | " + r[1] + " | " + r[4] + " | " + str(r[2]) + " noches | " + str(round(precio, 2)) + "
€")
    elif r[0] == "tren":
        precio = r[2] * r[3]
        print("TREN | " + r[1] + " | " + r[4] + " | " + str(r[2]) + " billetes | " + str(round(precio, 2)) +
" €")
    total = total + precio

print("TOTAL AGENCIA: " + str(round(total, 2)) + " €")

# Contar reservas de más de 200 €: hay que repetir todos los cálculos
caras = 0
for r in reservas:
    if r[0] == "vuelo":
        precio = r[2] * r[3] + 25
        if r[3] > 200:
            precio = precio + r[2] * 30
    elif r[0] == "hotel":
        precio = r[2] * r[3]
        if r[2] >= 3:
            precio = precio * 0.9
        precio = precio + r[2] * 1.5
    elif r[0] == "tren":
        precio = r[2] * r[3]
    if precio > 200:
        caras = caras + 1

print("Reservas de más de 200 €: " + str(caras))
```

Las reglas de precio que esconde el código son:

Tipo	Cantidad significa	Cálculo del precio
Vuelo	pasajeros	pasajeros × precio + 25 € de tasas. Si el billete cuesta más de 200 €, se suman 30 € por pasajero.
Hotel	noches	noches × precio. Con 3 noches o más, un 10 % de descuento. Después se suma 1.5 € por noche de tasa turística.
Tren	billetes	billetes × precio.

Antes de programar, responde:

45. ¿Qué significan `r[2]` y `r[3]`? ¿Es fácil saberlo sin leer el comentario?
46. ¿En cuántos sitios distintos está escrita la regla del precio de un hotel?
47. Si mañana la agencia vende **cruceros**, ¿cuántas partes del programa habría que tocar? #### Parte B. Crear la superclase

Crea una clase `Reserva` con:

- **Atributos:** `cliente`, `destino`, `cantidad` y `precio_unitario`.
- **Método `precio_base()`:** devuelve cantidad × precio unitario.

- **Método `precio_total()`:** en la superclase devuelve el precio base.
- **Método `unidad()`:** devuelve el texto de la unidad, por ejemplo `"unidades"`.
- **Método `__str__`:** devuelve la línea completa con el formato del programa original. El tipo (`VUELO`, `HOTEL`…) se obtiene a partir del nombre de la clase, y la unidad, con `unidad()`. #### Parte C. Crear las subclases

Crea `ReservaVuelo`, `ReservaHotel` y `ReservaTren`. Cada una debe:

- sobrescribir `unidad()`;
- sobrescribir `precio_total()` **solo si lo necesita**, reutilizando `precio_base()` o `super().precio_total()` en lugar de repetir la multiplicación. Pista para el tipo: si la clase se llama `ReservaVuelo`, puedes obtener `"VUELO"` con `type(self).__name__.replace("Reserva", "").upper()`.

Parte D. Crear una clase que agrupe las reservas (composición)

Crea una clase `Agencia` que **contenga** una lista de reservas y tenga estos métodos:

- `agregar(reserva)`;
- `mostrar()`, que imprime cada reserva;
- `total()`, que devuelve la suma de los precios totales;
- `contar_superiores_a(importe)`, que devuelve cuántas reservas superan ese importe. #### Parte E. Reescribir el programa principal

```
agencia = Agencia()
agencia.agregar(ReservaVuelo("Ana López", "Madrid-Roma", 2, 120.0))
agencia.agregar(ReservaHotel("Luis Pérez", "Hotel Mar Azul", 3, 85.0))
agencia.agregar(ReservaTren("Marta Gil", "Madrid-Sevilla", 1, 45.0))
agencia.agregar(ReservaVuelo("Pablo Ruiz", "Madrid-Nueva York", 1, 300.0))
agencia.agregar(ReservaHotel("Eva Sanz", "Hotel Centro", 2, 140.0))

agencia.mostrar()
print(f"TOTAL AGENCIA:{round(agencia.total(), 2)} €")
print(f"Reservas de más de 200 €:{agencia.contar_superiores_a(200)}")
```

Salida esperada (idéntica a la del programa original)

```
VUELO | Ana López | Madrid-Roma | 2 pasajeros | 265.0 €
HOTEL | Luis Pérez | Hotel Mar Azul | 3 noches | 234.0 €
TREN | Marta Gil | Madrid-Sevilla | 1 billetes | 45.0 €
VUELO | Pablo Ruiz | Madrid-Nueva York | 1 pasajeros | 355.0 €
HOTEL | Eva Sanz | Hotel Centro | 2 noches | 283.0 €
TOTAL AGENCIA: 1182.0 €
Reservas de más de 200 €: 4
```

Parte F. Ampliación

La agencia empieza a vender **cruceros**. La cantidad son los camarotes y el precio es camarotes × precio más un 12 % de tasas portuarias. Crea `ReservaCrucero` y añade esta reserva:

```
agencia.agregar(ReservaCrucero("Jorge Vidal", "Crucero Mediterráneo", 1, 900.0))
```

Salida esperada (solo las líneas nuevas o que cambian)

```
CRUCERO | Jorge Vidal | Crucero Mediterráneo | 1 camarotes | 1008.0 €
TOTAL AGENCIA: 2190.0 €
Reservas de más de 200 €: 5
```

Restricciones

- En el programa final **no puede quedar ningún `if/elif` que pregunte por el tipo de reserva**.
- No se puede usar `isinstance()` para decidir cómo calcular el precio.
- El cálculo de cada tipo de reserva debe estar escrito **en un único sitio**.

- Para añadir el crucero **no debes modificar** ni `Reserva` ni `Agencia`. **Pregunta final:** compara tu respuesta a la pregunta 3 de la Parte A con lo que has tenido que hacer en la Parte F. ¿Qué ha cambiado?

Respuestas Parte A:

1. `r[2]` es la cantidad (pasajeros/noches/billetes) y `r[3]` el precio unitario. Sin el comentario es imposible saberlo: los índices no dicen nada (*números mágicos*).
2. La regla del hotel está escrita en **2 sitios** (en el bucle que imprime y en el que cuenta las caras). Si cambia, hay que acordarse de cambiar los dos.
3. Para añadir cruceros habría que tocar **3 partes**: el `if/elif` del primer bucle, el del segundo bucle y el texto/unidad de impresión.

✅ Solución completa de la sección 8

```
class Reserva:
    def __init__(self, cliente, destino, cantidad, precio_unitario):
        self.cliente = cliente
        self.destino = destino
        self.cantidad = cantidad
        self.precio_unitario = precio_unitario

    def precio_base(self):
        return self.cantidad * self.precio_unitario

    def precio_total(self):
        return self.precio_base()

    def unidad(self):
        return "unidades"

    def __str__(self):
        tipo = type(self).__name__.replace("Reserva", "").upper()
        return (f"{tipo} | {self.cliente} | {self.destino} | "
                f"{self.cantidad} {self.unidad()} | {round(self.precio_total(), 2)} €")

class ReservaVuelo(Reserva):
    def unidad(self):
        return "pasajeros"

    def precio_total(self):
        precio = super().precio_total() + 25            # tasas
        if self.precio_unitario > 200:
            precio += self.cantidad * 30                # suplemento billete caro
        return precio

class ReservaHotel(Reserva):
    def unidad(self):
        return "noches"

    def precio_total(self):
        precio = self.precio_base()
        if self.cantidad >= 3:
            precio *= 0.9                               # 10 % descuento
        return precio + self.cantidad * 1.5             # tasa turística

class ReservaTren(Reserva):
    def unidad(self):
        return "billetes"
    # precio_total() no se sobrescribe: el del padre ya sirve

class Agencia:                                          # COMPOSICIÓN
    def __init__(self):
        self.reservas = []

    def agregar(self, reserva):
        self.reservas.append(reserva)

    def mostrar(self):
        for r in self.reservas:
            print(r)                                    # polimorfismo vía __str__/precio_total
```

```
    def total(self):
        return sum(r.precio_total() for r in self.reservas)

    def contar_superiores_a(self, importe):
        return len([r for r in self.reservas if r.precio_total() > importe])

agencia = Agencia()
agencia.agregar(ReservaVuelo("Ana López", "Madrid-Roma", 2, 120.0))
agencia.agregar(ReservaHotel("Luis Pérez", "Hotel Mar Azul", 3, 85.0))
agencia.agregar(ReservaTren("Marta Gil", "Madrid-Sevilla", 1, 45.0))
agencia.agregar(ReservaVuelo("Pablo Ruiz", "Madrid-Nueva York", 1, 300.0))
agencia.agregar(ReservaHotel("Eva Sanz", "Hotel Centro", 2, 140.0))

agencia.mostrar()
print(f"TOTAL AGENCIA: {round(agencia.total(), 2)} €")
print(f"Reservas de más de 200 €: {agencia.contar_superiores_a(200)}")

# ---- Parte F: ampliación SIN tocar Reserva ni Agencia ----
class ReservaCrucero(Reserva):
    def unidad(self):
        return "camarotes"

    def precio_total(self):
        return self.precio_base() * 1.12                # 12 % tasas portuarias

print("--- Con crucero ---")
agencia.agregar(ReservaCrucero("Jorge Vidal", "Crucero Mediterráneo", 1, 900.0))
agencia.mostrar()
print(f"TOTAL AGENCIA: {round(agencia.total(), 2)} €")
print(f"Reservas de más de 200 €: {agencia.contar_superiores_a(200)}")
# Pregunta final: antes había que tocar 3 sitios; ahora basta con AÑADIR una clase nueva.
# El resto del código (Agencia, bucles) funciona sin cambios gracias al polimorfismo.
```

🖥️ **Salida:**

```
VUELO | Ana López | Madrid-Roma | 2 pasajeros | 265.0 €
HOTEL | Luis Pérez | Hotel Mar Azul | 3 noches | 234.0 €
TREN | Marta Gil | Madrid-Sevilla | 1 billetes | 45.0 €
VUELO | Pablo Ruiz | Madrid-Nueva York | 1 pasajeros | 355.0 €
HOTEL | Eva Sanz | Hotel Centro | 2 noches | 283.0 €
TOTAL AGENCIA: 1182.0 €
Reservas de más de 200 €: 4
--- Con crucero ---
VUELO | Ana López | Madrid-Roma | 2 pasajeros | 265.0 €
HOTEL | Luis Pérez | Hotel Mar Azul | 3 noches | 234.0 €
TREN | Marta Gil | Madrid-Sevilla | 1 billetes | 45.0 €
VUELO | Pablo Ruiz | Madrid-Nueva York | 1 pasajeros | 355.0 €
HOTEL | Eva Sanz | Hotel Centro | 2 noches | 283.0 €
CRUCERO | Jorge Vidal | Crucero Mediterráneo | 1 camarotes | 1008.0 €
TOTAL AGENCIA: 2190.0 €
Reservas de más de 200 €: 5
```

Sección 9. Ejercicio integrador: restaurante de comida a domicilio

Un restaurante quiere un pequeño programa para analizar los pedidos del día. Este ejercicio combina clases, herencia, polimorfismo, comprehensions, programación funcional y recursión.

Parte A. Jerarquía de clases

48. Crea una superclase `Plato` con `nombre` y `precio`, un método `precio_final()` que devuelva el precio y un método `__str__` con el formato `Nombre - 12.50 €`, usando `precio_final()`.
49. Crea estas subclases:

- `PlatoVegano`: tiene un 10 % de descuento sobre su precio;
- `PlatoGourmet`: tiene un recargo fijo de 3 €. #### Parte B. Datos del día

```
pedidos = [
    {"cliente": "Ana",   "plato": PlatoVegano("Ensalada quinoa", 11.0),  "unidades": 2},
    {"cliente": "Luis",  "plato": Plato("Hamburguesa", 9.5),             "unidades": 1},
```

```
    {"cliente": "Marta", "plato": PlatoGourmet("Risotto trufa", 18.0),   "unidades": 1},
    {"cliente": "Ana",   "plato": Plato("Pizza margarita", 10.0),       "unidades": 3},
    {"cliente": "Pablo", "plato": PlatoVegano("Curry garbanzos", 12.0),  "unidades": 1},
]
```

Parte C. Comprehensions

50. Una **lista** con el importe de cada pedido (`precio_final() × unidades`), redondeado a 2 decimales.
51. Un **set** con los clientes distintos, ordenado alfabéticamente al mostrarlo.
52. Un **diccionario** `nombre del plato → precio final` de todos los platos. #### Parte D. Programación funcional
53. Con `filter()`, obtén los pedidos de **platos veganos**. Muestra el nombre del plato. Pista: `isinstance()`.
54. Con `map()` y `reduce()`, calcula la **facturación total** del día.
55. Crea una función `crear_filtro_minimo(minimo)` que devuelva otra función. Úsala para obtener los pedidos cuyo importe sea **igual o superior a 20 €** y muestra el nombre del cliente. #### Parte E. Recursión

Escribe una función recursiva `contar_unidades(pedidos)` que devuelva el total de unidades vendidas, **sin usar bucles**.

Parte F. Informe final

Muestra un informe con este aspecto:

```
=== CARTA DEL DÍA ===
Ensalada quinoa - 9.90 €
Hamburguesa - 9.50 €
Risotto trufa - 21.00 €
Pizza margarita - 10.00 €
Curry garbanzos - 10.80 €

Importes por pedido: [19.8, 9.5, 21.0, 30.0, 10.8]
Clientes distintos: ['Ana', 'Luis', 'Marta', 'Pablo']
Precios finales: {'Ensalada quinoa': 9.9, 'Hamburguesa': 9.5, 'Risotto trufa': 21.0, 'Pizza margarita': 10.0,
'Curry garbanzos': 10.8}
Platos veganos pedidos: ['Ensalada quinoa', 'Curry garbanzos']
Facturación total: 91.1 €
Pedidos de 20 € o más: ['Marta', 'Ana']
Unidades vendidas: 8
```

Pregunta final: si mañana el restaurante añade una clase `PlatoInfantil` con un precio fijo de 6 €, ¿qué partes del programa tendrías que cambiar y cuáles funcionarían sin tocarlas? Justifica tu respuesta.

✅ Solución completa de la sección 9

```python
from functools import reduce

# --- Parte A: jerarquía ---
class Plato:
    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.precio = precio

    def precio_final(self):
        return self.precio

    def __str__(self):
        return f"{self.nombre} - {self.precio_final():.2f} €"

class PlatoVegano(Plato):
    def precio_final(self):
        return round(super().precio_final() * 0.90, 2)

class PlatoGourmet(Plato):
```

```
    def precio_final(self):
        return super().precio_final() + 3

# --- Parte B: datos ---
pedidos = [
    {"cliente": "Ana",   "plato": PlatoVegano("Ensalada quinoa", 11.0),  "unidades": 2},
    {"cliente": "Luis",  "plato": Plato("Hamburguesa", 9.5),             "unidades": 1},
    {"cliente": "Marta", "plato": PlatoGourmet("Risotto trufa", 18.0),   "unidades": 1},
    {"cliente": "Ana",   "plato": Plato("Pizza margarita", 10.0),        "unidades": 3},
    {"cliente": "Pablo", "plato": PlatoVegano("Curry garbanzos", 12.0),  "unidades": 1},
]

importe = lambda p: round(p["plato"].precio_final() * p["unidades"], 2)

# --- Parte C: comprehensions ---
importes = [importe(p) for p in pedidos]
clientes = {p["cliente"] for p in pedidos}
precios_finales = {p["plato"].nombre: p["plato"].precio_final() for p in pedidos}

# --- Parte D: programación funcional ---
veganos = list(filter(lambda p: isinstance(p["plato"], PlatoVegano), pedidos))
facturacion = reduce(lambda acum, x: acum + x, map(importe, pedidos), 0)

def crear_filtro_minimo(minimo):
    def cumple(pedido):
        return importe(pedido) >= minimo
    return cumple

minimo_20 = crear_filtro_minimo(20)
pedidos_20 = [p["cliente"] for p in pedidos if minimo_20(p)]

# --- Parte E: recursión ---
def contar_unidades(pedidos):
    if not pedidos:
        return 0
    return pedidos[0]["unidades"] + contar_unidades(pedidos[1:])

# --- Parte F: informe ---
print("=== CARTA DEL DÍA ===")
for p in pedidos:
    print(p["plato"])
print()
print("Importes por pedido:", importes)
print("Clientes distintos:", sorted(clientes))
print("Precios finales:", precios_finales)
print("Platos veganos pedidos:", [p["plato"].nombre for p in veganos])
print(f"Facturación total: {round(facturacion, 2)} €")
print("Pedidos de 20 € o más:", pedidos_20)
print("Unidades vendidas:", contar_unidades(pedidos))
# Pregunta final: con PlatoInfantil (precio fijo 6 €) solo hay que AÑADIR la subclase
# sobrescribiendo precio_final(). Comprehensions, map/reduce, filtros y recursión siguen
# funcionando porque llaman a precio_final() (polimorfismo). Solo cambiaría el filtro de
# veganos si quisiéramos uno equivalente para infantiles (usa isinstance).
```

🖥️ Salida:

```
=== CARTA DEL DÍA ===
Ensalada quinoa - 9.90 €
Hamburguesa - 9.50 €
Risotto trufa - 21.00 €
Pizza margarita - 10.00 €
Curry garbanzos - 10.80 €

Importes por pedido: [19.8, 9.5, 21.0, 30.0, 10.8]
Clientes distintos: ['Ana', 'Luis', 'Marta', 'Pablo']
Precios finales: {'Ensalada quinoa': 9.9, 'Hamburguesa': 9.5, 'Risotto trufa': 21.0, 'Pizza margarita': 10.0,
'Curry garbanzos': 10.8}
Platos veganos pedidos: ['Ensalada quinoa', 'Curry garbanzos']
Facturación total: 91.1 €
Pedidos de 20 € o más: ['Marta', 'Ana']
Unidades vendidas: 8
```

Sección 10: Programación Modular

> **Hoteles Brisa** es una cadena con tres hoteles: *Costa Azul*, *Sierra Nevada* y *Centro Histórico*. Un analista de revenue ha preparado en un notebook el **informe de ingresos por hotel**: 1. Lee las reservas del día, que llegan en formato CSV. 2. Descarta las reservas con el número de noches o el precio vacíos o mal escritos. 3. Descarta las reservas anuladas. 4. Calcula el importe de cada reserva: noches × precio por noche, con un **10 % de descuento para los socios** del programa de fidelización, más una **tasa turística de 2,50 € por noche** (la tasa no tiene descuento). 5. Suma noches e ingresos por hotel, ordena de mayor a menor ingreso y calcula el **ingreso medio por noche**. 6. Muestra el informe por pantalla y lo guarda en `informe_hoteles.csv`. La dirección quiere que el informe se ejecute **automáticamente cada noche**. El trabajo consiste en convertir el notebook en un proyecto Python modular que haga **exactamente lo mismo**.

> Crear el notebook en VS Code o Jupyter con las siguientes celdas y ejecutarlo para comprobar la salida antes de empezar a modularizar.

Celda [1] · Cargar los datos

```
datos = """id,cliente,hotel,tipo,noches,precio_noche,estado
R001, Carmen Vidal ,costa azul,socio,3,95.00,confirmada
R002,Andrés Molina,SIERRA NEVADA,general,2,120.00,anulada
R003,Lucía Ferrer,Costa Azul,general,5,95.00,confirmada
R004,Raúl Ortega,centro histórico,socio,,80.00,confirmada
R005,Nuria Campos,Sierra Nevada,general,4,120.00,check-out
R006,Tomás Reyes,centro histórico,general,1,8O.00,confirmada
R007,Irene Navarro,Centro Histórico,socio,2,80.00,confirmada
R008,Óscar Prieto,sierra nevada,socio,3,120.00,ANULADA
R009,Paula Serrano,Sierra Nevada,general,2,135.50,confirmada"""
lineas = datos.split("\n")[1:]
print("Reservas leídas:", len(lineas))
```

```
Reservas leídas: 9
```

Celda [2] · Limpiar y descartar errores

```
reservas = []
errores = []
for l in lineas:
    c = [x.strip() for x in l.split(",")]
    try:
        n = int(c[4])
        p = float(c[5])
        reservas.append([c[0], c[1].title(), c[2].title(), c[3].lower(), n, p, c[6].lower()])
    except ValueError:
        errores.append(c[0])
print("Reservas con errores:", errores)
```

```
Reservas con errores: ['R004', 'R006']
```

> En `R006` el precio está escrito con una letra **O** en lugar de un cero: `8O.00`.

Celda [3] · Quitar las reservas anuladas

```
anuladas = [r[0] for r in reservas if r[6] == "anulada"]
reservas = [r for r in reservas if r[6] != "anulada"]
print("Reservas anuladas:", anuladas)
print("Reservas válidas:", len(reservas))
```

```
Reservas anuladas: ['R002', 'R008']
```

```
Reservas válidas: 5
```

Celda [4] · Calcular el importe de cada reserva

```python
def es_socio(r):
    return r[3] == "socio"

for r in reservas:
    base = r[4] * r[5]
    if es_socio(r):
        base = base * 0.90
    r.append(round(base + 2.50 * r[4], 2))
print(reservas[0])
```

```
['R001', 'Carmen Vidal', 'Costa Azul', 'socio', 3, 95.0, 'confirmada', 264.0]
```

Celda [5] · Agregar por hotel

```python
ingresos = {}
noches = {}
for r in reservas:
    ingresos[r[2]] = round(ingresos.get(r[2], 0) + r[7], 2)
    noches[r[2]] = noches.get(r[2], 0) + r[4]
ingresos = dict(sorted(ingresos.items(), key=lambda x: x[1], reverse=True))
```

Celda [6] · Mostrar el informe

```python
print("Informe de ingresos por hotel:")
for h, t in ingresos.items():
    print(f"  {h:<17}{noches[h]:>3} noches {t:>9.2f} €   media/noche {t/noches[h]:>7.2f} €")
print(f"  {'TOTAL':<17}{sum(noches.values()):>3} noches {sum(ingresos.values()):>9.2f} €")
```

```
Informe de ingresos por hotel:
  Sierra Nevada      6 noches    766.00 €   media/noche  127.67 €
  Costa Azul         8 noches    751.50 €   media/noche   93.94 €
  Centro Histórico   2 noches    149.00 €   media/noche   74.50 €
  TOTAL             16 noches   1666.50 €
```

Celda [7] · Guardar el informe en CSV

```python
with open("informe_hoteles.csv", "w", encoding="utf-8") as f:
    f.write("hotel,noches,ingresos\n")
    for h, t in ingresos.items():
        f.write(f"{h},{noches[h]},{t}\n")
print("Informe guardado en informe_hoteles.csv")
```

```
Informe guardado en informe_hoteles.csv
```

Contenido de `informe_hoteles.csv`:

```
hotel,noches,ingresos
Sierra Nevada,6,766.0
Costa Azul,8,751.5
Centro Histórico,2,149.0
```

Antes de escribir código, completar esta tabla con **al menos cinco problemas** concretos del notebook, indicando la celda donde se ve cada uno. Usar como guía la tabla de la Sección 2 del Ejemplo 9.2.

Problema en el notebook	Dónde se ve (celda y línea)	Consecuencia en producción
…	…	…

> 💡 Pistas para buscar problemas: - ¿Qué pasa si se ejecuta dos veces la celda 4? ¿Y si se salta la celda 3? - ¿Qué significan `r[3]`, `r[4]` o `r[7]`? - ¿Dónde están escritos el `0.90`, el `2.50`, el `"anulada"` y el `"socio"`? -

> ¿Por qué `es_socio()` está definida en mitad del notebook y depende de la posición `r[3]`? - ¿Se podría usar la limpieza de reservas en otro informe sin copiar y pegar?

Completar la tabla que indica **a qué módulo y a qué función, clase o constante** va a parar cada celda:

Celda del notebook	Módulo de producción	Función / clase / constante
Celda 1: cargar los datos		
Celda 2: limpiar y descartar errores		
Celda 3: quitar anuladas		
Celda 4: calcular importes		
Celda 5: agregar por hotel		
Celda 6: mostrar el informe		
Celda 7: guardar en CSV		
`0.90`, `2.50`, `"anulada"`, `"socio"`, nombre del CSV		

Estructura mínima exigida

```
hoteles_etl/
├── main.py                ← punto de entrada
└── etl/                   ← paquete del pipeline
    ├── __init__.py        ← recepción del paquete
    ├── config.py          ← reglas de negocio y parámetros
    ├── modelos.py         ← clase Reserva
    ├── extraccion.py      ← E: obtener las líneas en bruto
    ├── transformacion.py  ← T: parsear, validar, convertir, filtrar
    ├── carga.py           ← L: clase InformeHoteles + exportar a CSV
    └── orquestador.py     ← une E, T y L en el orden correcto
```

Requisitos por módulo

Módulo	Requisitos
`config.py`	Solo constantes en mayúsculas: descuento de socio, tasa turística por noche, estado de anulación, tipo de cliente socio y nombre del fichero de salida. Ningún otro módulo puede tener esos valores escritos «a mano»
`modelos.py`	Clase `Reserva` con atributos con nombre (`id_reserva`, `cliente`, `hotel`, `tipo`, `noches`, `precio_noche`, `estado`). Una `@property` `es_socio` y una `@property` `importe_total` que **se calcula** al pedirla (no se guarda como columna). Método `__repr__`
`extraccion.py`	Función `leer_lineas(texto)` que devuelve las líneas sin la cabecera
`transformacion.py`	Funciones **puras**: `parsear(linea)`, `es_valida(campos)`, `a_reserva(campos)` y `transformar(lineas)`. `transformar()` devuelve las reservas activas, los ids con errores y los ids anulados. Usar al menos un `map()`, un `filter()` con `lambda` y una comprehension
`carga.py`	Clase `InformeHoteles` con `cargar(reservas)`, `__len__`, `por_hotel()` (devuelve, ordenado de mayor a menor ingreso, un diccionario `{hotel: (noches, ingresos)}`) y `total()`. Función `exportar_csv(informe, ruta)` que genera el CSV
`orquestador.py`	Función `ejecutar_pipeline(texto)` que devuelve un diccionario con `leidas`, `errores`, `anuladas` e `informe`

Módulo	Requisitos
`__init__.py`	Expone `ejecutar_pipeline` y `exportar_csv`. Define `__version__ = "1.0.0"` y `__all__`
`main.py`	Contiene los datos de entrada, una función `mostrar_informe(resultado)`, una función `main()` y el bloque `if __name__ == "__main__":`. **Solo importa del paquete `etl`**, nunca de sus módulos internos

Reglas de importación

- Dentro del paquete `etl` se usan **imports relativos** (`from .config import TASA_TURISTICA`).
- Desde `main.py` se usan **imports absolutos** a través de la recepción (`from etl import ejecutar_pipeline, exportar_csv`).
- No puede haber **imports circulares**: el sentido de las dependencias es `config` → `modelos` → `transformacion` / `carga` → `orquestador` → `__init__` → `main`.

Desde la carpeta del proyecto:

```
cd hoteles_etl
python main.py
```

Salida esperada (debe ser **idéntica** a la del notebook, sin la fila de la celda 4):

```
Reservas leídas: 9
Reservas con errores: ['R004', 'R006']
Reservas anuladas: ['R002', 'R008']
Reservas válidas: 5
Informe de ingresos por hotel:
  Sierra Nevada      6 noches    766.00 €   media/noche  127.67 €
  Costa Azul         8 noches    751.50 €   media/noche   93.94 €
  Centro Histórico   2 noches    149.00 €   media/noche   74.50 €
  TOTAL             16 noches   1666.50 €
Informe guardado en informe_hoteles.csv
```

El fichero `informe_hoteles.csv` generado debe tener **el mismo contenido** que el del notebook.

Crear un archivo `pruebas_manuales.py` en la carpeta `hoteles_etl` que importe **solo** las funciones de `etl.transformacion` y `etl.modelos` y compruebe estos casos sin ejecutar el pipeline completo:

```
from etl.transformacion import parsear, es_valida, a_reserva

print(es_valida(parsear("R010,Eva Lara,costa azul,general,2,95.00,confirmada")))
print(es_valida(parsear("R011,Eva Lara,costa azul,general,2,95,50,confirmada")))
print(es_valida(parsear("R012,Eva Lara,costa azul,general,,95.00,confirmada")))
print(a_reserva(parsear("R013, Eva Lara ,COSTA AZUL,Socio,2,100.00,confirmada")).importe_total)
```

Salida esperada:

```
True
False
False
185.0
```

> Explicar en un comentario por qué la línea `R011` (precio escrito con **coma decimal**) habría pasado sin error en el notebook y qué importe equivocado habría sumado.

---

56. **El notebook importa el paquete.** Crear un notebook nuevo `analisis.ipynb` en la carpeta `hoteles_etl` que, en lugar de repetir el código, importe `ejecutar_pipeline` y muestre el hotel con mayor ingreso medio por noche.

Diseñar y programar una **mini aplicación de temática libre** cuyo código esté organizado en **paquetes y subpaquetes**. La aplicación debe seguir la forma modular y ejecutarse desde un único `main.py`.

Requisito estructural mínimo:

- Al menos **3 paquetes** de primer nivel.
- Al menos **2 subpaquetes dentro de cada paquete**.
- Al menos **1 módulo con código en cada subpaquete**.

Se puede elegir cualquiera de estas o proponer otra (con el visto bueno del profesor):

Aplicación	Posibles paquetes
Gestor de un gimnasio	`socios`, `actividades`, `facturacion`
Alquiler de vehículos	`flota`, `reservas`, `tarifas`
Tienda online de electrónica	`catalogo`, `pedidos`, `envios`
Clínica veterinaria	`pacientes`, `citas`, `cobros`
Taller mecánico	`vehiculos`, `reparaciones`, `presupuestos`
Agencia de viajes	`destinos`, `reservas`, `pagos`
Gestión de un parking	`plazas`, `accesos`, `tarificacion`
Monitor de sensores de una fábrica	`sensores`, `alertas`, `informes`

> Este ejemplo usa la temática **restaurante** solo para mostrar el tipo de organización esperada. Esta temática **no se puede elegir**.

```
app_restaurante/
├── main.py
├── restaurante/                    ← paquete raíz de la aplicación
│   ├── __init__.py
│   ├── config.py
│   ├── carta/                      ← paquete 1
│   │   ├── __init__.py
│   │   ├── modelos/                ← subpaquete 1.1
│   │   │   ├── __init__.py
│   │   │   └── platos.py           (Plato, Entrante, Principal, Postre)
│   │   └── servicios/              ← subpaquete 1.2
│   │       ├── __init__.py
│   │       └── filtros.py          (filtrar por precio, alérgenos…)
│   ├── comandas/                   ← paquete 2
│   │   ├── __init__.py
│   │   ├── modelos/                ← subpaquete 2.1
│   │   │   ├── __init__.py
│   │   │   └── comanda.py          (Mesa, Comanda, LineaComanda)
│   │   └── gestion/                ← subpaquete 2.2
│   │       ├── __init__.py
│   │       └── cocina.py           (cola de comandas, tiempos)
│   └── caja/                       ← paquete 3
│       ├── __init__.py
│       ├── calculos/               ← subpaquete 3.1
│       │   ├── __init__.py
│       │   ├── impuestos.py
│       │   └── descuentos.py
│       └── informes/               ← subpaquete 3.2
│           ├── __init__.py
│           └── cierre.py           (cierre de caja del día)
└── tests_manuales.py
```

> Los tres paquetes (`carta`, `comandas`, `caja`) pueden colgar de un **paquete raíz** (`restaurante`) o estar directamente al lado de `main.py`. Las dos opciones son válidas; con paquete raíz los imports quedan `from restaurante.caja.calculos.impuestos import ...`.

- Al menos **3 módulos** incluyen un bloque `if __name__ == "__main__":` con pruebas propias que imprimen resultados.

✅ Solución de la sección 10

Tabla 1 · Problemas del notebook

Problema en el notebook	Dónde se ve	Consecuencia en producción
Datos escritos dentro del código	Celda 1, `datos = """..."""`	Para procesar otro día hay que editar el código
Índices mágicos (`r[3]`, `r[4]`, `r[7]`)	Celdas 2-6	Nadie sabe qué es cada posición; un cambio de columnas rompe todo en silencio
Constantes escritas a mano (`0.90`, `2.50`, `"anulada"`, `"socio"`, nombre del CSV)	Celdas 3, 4 y 7	Cambiar la tasa obliga a buscarla por todo el código; fácil olvidar una
Celda 4 modifica las reservas con `append`	Celda 4, `r.append(...)`	Si se ejecuta dos veces, cada reserva tiene 2 importes; `r[7]` deja de ser fiable
Orden de celdas obligatorio y sin control	Celdas 3 → 4 → 5	Si se salta la 3, se suman las anuladas sin ningún error
`es_socio()` definida a mitad y atada a `r[3]`	Celda 4	No se puede reutilizar ni probar por separado
Validación incompleta (solo `int`/`float` de 2 campos)	Celda 2	Una línea con coma decimal (`95,50`) pasa como válida con datos erróneos
Nada es reutilizable ni probable aisladamente	Todo	Para otro informe hay que copiar y pegar

Tabla 2 · De cada celda a su módulo

Celda del notebook	Módulo de producción	Función / clase / constante
Celda 1: cargar los datos	`extraccion.py` (+ datos en `main.py`)	`leer_lineas(texto)` / `DATOS`
Celda 2: limpiar y descartar errores	`transformacion.py`	`parsear()`, `es_valida()`, `a_reserva()`
Celda 3: quitar anuladas	`transformacion.py`	`transformar()` (con `filter` + `lambda`)
Celda 4: calcular importes	`modelos.py`	`Reserva.importe_total` (`@property`)
Celda 5: agregar por hotel	`carga.py`	`InformeHoteles.por_hotel()` y `total()`
Celda 6: mostrar el informe	`main.py`	`mostrar_informe(resultado)`
Celda 7: guardar en CSV	`carga.py`	`exportar_csv(informe, ruta)`
`0.90`, `2.50`, `"anulada"`, `"socio"`, nombre del CSV	`config.py`	`DESCUENTO_SOCIO`, `TASA_TURISTICA`, `ESTADO_ANULADA`, `TIPO_SOCIO`, `FICHERO_SALIDA`

Dependencias (sin imports circulares): `config` → `modelos` → `transformacion` / `carga` → `orquestador` → `__init__` → `main`.

Estructura del proyecto `hoteles_etl/`:

```
hoteles_etl/
├── main.py
├── pruebas_manuales.py
├── analisis.py            (contenido de la celda de analisis.ipynb)
└── etl/
    ├── __init__.py
    ├── config.py
    ├── modelos.py
    ├── extraccion.py
    ├── transformacion.py
```

```
    ├── carga.py
    └── orquestador.py
```

`hoteles_etl/etl/config.py`

```python
"""Reglas de negocio y parámetros. Solo constantes en MAYÚSCULAS."""
DESCUENTO_SOCIO = 0.10          # 10 % de descuento a socios
TASA_TURISTICA = 2.50           # € por noche (sin descuento)
ESTADO_ANULADA = "anulada"
TIPO_SOCIO = "socio"
FICHERO_SALIDA = "informe_hoteles.csv"
NUM_CAMPOS = 7
```

`hoteles_etl/etl/modelos.py`

```python
"""Modelo de datos: una reserva con atributos con nombre (adiós a r[3], r[4]...)."""
from .config import DESCUENTO_SOCIO, TASA_TURISTICA, TIPO_SOCIO


class Reserva:
    def __init__(self, id_reserva, cliente, hotel, tipo, noches, precio_noche, estado):
        self.id_reserva = id_reserva
        self.cliente = cliente
        self.hotel = hotel
        self.tipo = tipo
        self.noches = noches
        self.precio_noche = precio_noche
        self.estado = estado

    @property
    def es_socio(self):
        return self.tipo == TIPO_SOCIO

    @property
    def importe_total(self):
        """Se CALCULA cada vez que se pide (no se guarda): no se puede 'duplicar'."""
        base = self.noches * self.precio_noche
        if self.es_socio:
            base *= (1 - DESCUENTO_SOCIO)
        return round(base + TASA_TURISTICA * self.noches, 2)

    def __repr__(self):
        return (f"Reserva({self.id_reserva!r}, {self.cliente!r}, {self.hotel!r}, "
                f"{self.tipo!r}, {self.noches}, {self.precio_noche}, {self.estado!r})")
```

`hoteles_etl/etl/extraccion.py`

```python
"""E: obtener las líneas en bruto."""


def leer_lineas(texto):
    """Devuelve las líneas de datos, sin la cabecera."""
    return texto.strip().split("\n")[1:]
```

`hoteles_etl/etl/transformacion.py`

```python
"""T: funciones PURAS (no modifican nada externo, mismo input → mismo output)."""
from .config import ESTADO_ANULADA, NUM_CAMPOS
from .modelos import Reserva


def parsear(linea):
    return [campo.strip() for campo in linea.split(",")]          # comprehension


def es_valida(campos):
    if len(campos) != NUM_CAMPOS:          # p. ej. "95,50" partiría el precio en 2 campos
        return False
    try:
        int(campos[4])
```

```
        float(campos[5])
        return True
    except ValueError:
        return False


def a_reserva(campos):
    return Reserva(campos[0], campos[1].title(), campos[2].title(), campos[3].lower(),
                   int(campos[4]), float(campos[5]), campos[6].lower())


def transformar(lineas):
    """Devuelve (reservas_activas, ids_con_errores, ids_anulados)."""
    campos = list(map(parsear, lineas))                                    # map
    errores = [c[0] for c in campos if not es_valida(c)]
    reservas = [a_reserva(c) for c in campos if es_valida(c)]
    anuladas = [r.id_reserva for r in reservas if r.estado == ESTADO_ANULADA]
    activas = list(filter(lambda r: r.estado != ESTADO_ANULADA, reservas))  # filter + lambda
    return activas, errores, anuladas
```

`hoteles_etl/etl/carga.py`

```
"""L: agregación por hotel y exportación a CSV."""


class InformeHoteles:
    def __init__(self):
        self._reservas = []

    def cargar(self, reservas):
        self._reservas.extend(reservas)

    def __len__(self):
        return len(self._reservas)

    def por_hotel(self):
        """{hotel: (noches, ingresos)} ordenado de mayor a menor ingreso."""
        agregado = {}
        for r in self._reservas:
            noches, ingresos = agregado.get(r.hotel, (0, 0))
            agregado[r.hotel] = (noches + r.noches, round(ingresos + r.importe_total, 2))
        return dict(sorted(agregado.items(), key=lambda x: x[1][1], reverse=True))

    def total(self):
        """(noches_totales, ingresos_totales)."""
        datos = self.por_hotel().values()
        return sum(n for n, _ in datos), round(sum(t for _, t in datos), 2)


def exportar_csv(informe, ruta):
    with open(ruta, "w", encoding="utf-8") as f:
        f.write("hotel,noches,ingresos\n")
        for hotel, (noches, ingresos) in informe.por_hotel().items():
            f.write(f"{hotel},{noches},{ingresos}\n")
```

`hoteles_etl/etl/orquestador.py`

```
"""Une E → T → L en el orden correcto (imposible saltarse un paso)."""
from .extraccion import leer_lineas
from .transformacion import transformar
from .carga import InformeHoteles


def ejecutar_pipeline(texto):
    lineas = leer_lineas(texto)
    activas, errores, anuladas = transformar(lineas)
    informe = InformeHoteles()
    informe.cargar(activas)
    return {"leidas": len(lineas), "errores": errores,
            "anuladas": anuladas, "informe": informe}
```

`hoteles_etl/etl/__init__.py`

```
"""Recepción del paquete etl: lo que se puede usar desde fuera."""
from .orquestador import ejecutar_pipeline
from .carga import exportar_csv
from .config import FICHERO_SALIDA

__version__ = "1.0.0"
__all__ = ["ejecutar_pipeline", "exportar_csv", "FICHERO_SALIDA"]
```

`hoteles_etl/main.py`

```
"""Punto de entrada. Solo importa del paquete etl (nunca de sus módulos internos)."""
from etl import ejecutar_pipeline, exportar_csv, FICHERO_SALIDA

DATOS = """id,cliente,hotel,tipo,noches,precio_noche,estado
R001, Carmen Vidal ,costa azul,socio,3,95.00,confirmada
R002,Andrés Molina,SIERRA NEVADA,general,2,120.00,anulada
R003,Lucía Ferrer,Costa Azul,general,5,95.00,confirmada
R004,Raúl Ortega,centro histórico,socio,,80.00,confirmada
R005,Nuria Campos,Sierra Nevada,general,4,120.00,check-out
R006,Tomás Reyes,centro histórico,general,1,8O.00,confirmada
R007,Irene Navarro,Centro Histórico,socio,2,80.00,confirmada
R008,Óscar Prieto,sierra nevada,socio,3,120.00,ANULADA
R009,Paula Serrano,Sierra Nevada,general,2,135.50,confirmada"""


def mostrar_informe(resultado):
    informe = resultado["informe"]
    print("Reservas leídas:", resultado["leidas"])
    print("Reservas con errores:", resultado["errores"])
    print("Reservas anuladas:", resultado["anuladas"])
    print("Reservas válidas:", len(informe))
    print("Informe de ingresos por hotel:")
    for h, (n, t) in informe.por_hotel().items():
        print(f"  {h:<17}{n:>3} noches {t:>9.2f} €   media/noche {t/n:>7.2f} €")
    noches, total = informe.total()
    print(f"  {'TOTAL':<17}{noches:>3} noches {total:>9.2f} €")


def main():
    resultado = ejecutar_pipeline(DATOS)
    mostrar_informe(resultado)
    exportar_csv(resultado["informe"], FICHERO_SALIDA)
    print(f"Informe guardado en {FICHERO_SALIDA}")


if __name__ == "__main__":
    main()
```

`hoteles_etl/pruebas_manuales.py`

```
from etl.transformacion import parsear, es_valida, a_reserva

print(es_valida(parsear("R010,Eva Lara,costa azul,general,2,95.00,confirmada")))
print(es_valida(parsear("R011,Eva Lara,costa azul,general,2,95,50,confirmada")))
print(es_valida(parsear("R012,Eva Lara,costa azul,general,,95.00,confirmada")))
print(a_reserva(parsear("R013, Eva Lara ,COSTA AZUL,Socio,2,100.00,confirmada")).importe_total)

# R011: con coma decimal el split(",") parte "95,50" en "95" y "50" → 8 campos.
# El notebook solo miraba c[4] y c[5] ("2" y "95"), que convierten sin error, así que la
# habría aceptado con precio 95.0 (en vez de 95.50) y estado "50" (no "anulada"):
# habría sumado 2*95 + 2*2.50 = 195.0 € en vez de 196.0 €. Aquí se rechaza por tener 8 campos.
```

`hoteles_etl/analisis.py`

```
# Contenido de la celda de analisis.ipynb: el notebook IMPORTA el paquete, no repite código
from main import DATOS
from etl import ejecutar_pipeline
```

```
resultado = ejecutar_pipeline(DATOS)
medias = {h: t / n for h, (n, t) in resultado["informe"].por_hotel().items()}
mejor = max(medias, key=medias.get)
print(f"Hotel con mayor ingreso medio por noche: {mejor} ({medias[mejor]:.2f} €)")
```

🖥️ **Salida de `python main.py`:**

```
Reservas leídas: 9
Reservas con errores: ['R004', 'R006']
Reservas anuladas: ['R002', 'R008']
Reservas válidas: 5
Informe de ingresos por hotel:
  Sierra Nevada      6 noches    766.00 €   media/noche  127.67 €
  Costa Azul         8 noches    751.50 €   media/noche   93.94 €
  Centro Histórico   2 noches    149.00 €   media/noche   74.50 €
  TOTAL             16 noches   1666.50 €
Informe guardado en informe_hoteles.csv
```

Contenido de `informe_hoteles.csv`:

```
hotel,noches,ingresos
Sierra Nevada,6,766.0
Costa Azul,8,751.5
Centro Histórico,2,149.0
```

🖥️ **Salida de `python pruebas_manuales.py`:**

```
True
False
False
185.0
```

🖥️ **Salida de `analisis.ipynb`:**

```
Hotel con mayor ingreso medio por noche: Sierra Nevada (127.67 €)
```

✅ Mini aplicación con paquetes y subpaquetes (temática: gestor de un gimnasio)

3 paquetes (`socios`, `actividades`, `facturacion`), cada uno con 2 subpaquetes y al menos un módulo con código; 3 módulos con bloque `if __name__ == "__main__":`.

```
app_gimnasio/
├── main.py
└── gimnasio/                      ← paquete raíz
    ├── __init__.py
    ├── config.py
    ├── socios/                    ← paquete 1
    │   ├── __init__.py
    │   ├── modelos/socio.py       (Socio, SocioPremium)
    │   └── servicios/registro.py  (alta)
    ├── actividades/               ← paquete 2
    │   ├── __init__.py
    │   ├── modelos/clase.py       (Clase)
    │   └── planificacion/horario.py (crear_horario, ocupacion)
    └── facturacion/               ← paquete 3
        ├── __init__.py
        ├── calculos/importes.py   (importe_socio, total)
        └── informes/cierre.py     (informe_mensual)
(cada subpaquete lleva su __init__.py)
```

`app_gimnasio/gimnasio/config.py`

```
CUOTA_BASICA = 30.0
CUOTA_PREMIUM = 50.0
IVA = 0.21
DESCUENTO_ESTUDIANTE = 0.20
```

`app_gimnasio/gimnasio/socios/modelos/socio.py`

```
from ...config import CUOTA_BASICA, CUOTA_PREMIUM   # import relativo: 3 puntos = subir 2 niveles


class Socio:
    def __init__(self, dni, nombre, estudiante=False):
        self.dni, self.nombre, self.estudiante = dni, nombre, estudiante

    def cuota(self):
        return CUOTA_BASICA

    def __repr__(self):
        return f"{type(self).__name__}({self.nombre!r})"


class SocioPremium(Socio):
    def cuota(self):
        return CUOTA_PREMIUM
```

app_gimnasio/gimnasio/socios/servicios/registro.py

```
from ..modelos.socio import Socio, SocioPremium


def alta(datos):
    """Crea socios a partir de tuplas (dni, nombre, premium, estudiante)."""
    return [(SocioPremium if p else Socio)(d, n, e) for d, n, p, e in datos]


if __name__ == "__main__":      # se ejecuta con: python -m gimnasio.socios.servicios.registro
    print(alta([("1A", "Ana", True, False), ("2B", "Luis", False, True)]))
```

app_gimnasio/gimnasio/actividades/modelos/clase.py

```
class Clase:
    def __init__(self, nombre, aforo):
        self.nombre, self.aforo, self.inscritos = nombre, aforo, []

    def inscribir(self, socio):
        if len(self.inscritos) < self.aforo and socio not in self.inscritos:
            self.inscritos.append(socio)
            return True
        return False

    def __len__(self):
        return len(self.inscritos)
```

app_gimnasio/gimnasio/actividades/planificacion/horario.py

```
from ..modelos.clase import Clase


def crear_horario(nombres_aforos):
    return {nombre: Clase(nombre, aforo) for nombre, aforo in nombres_aforos}


def ocupacion(horario):
    return {n: f"{len(c)}/{c.aforo}" for n, c in horario.items()}


if __name__ == "__main__":
    h = crear_horario([("Yoga", 2), ("Spinning", 10)])
    h["Yoga"].inscribir("Ana")
    print(ocupacion(h))
```

app_gimnasio/gimnasio/facturacion/calculos/importes.py

```
from functools import reduce
from ...config import IVA, DESCUENTO_ESTUDIANTE
```

```
def importe_socio(socio):
    base = socio.cuota() * (1 - DESCUENTO_ESTUDIANTE) if socio.estudiante else socio.cuota()
    return round(base * (1 + IVA), 2)


def total(socios):
    return round(reduce(lambda acum, s: acum + importe_socio(s), socios, 0), 2)


if __name__ == "__main__":
    class _S:                       # objeto falso para probar (duck typing)
        estudiante = True
        def cuota(self): return 30.0
    print(importe_socio(_S()))      # 30*0.8*1.21 = 29.04
```

`app_gimnasio/gimnasio/facturacion/informes/cierre.py`

```
from ..calculos.importes import importe_socio, total


def informe_mensual(socios):
    lineas = [f"{s.nombre:<8}{type(s).__name__:<14}{importe_socio(s):>7.2f} €" for s in socios]
    return "\n".join(lineas + [f"{'TOTAL':<22}{total(socios):>7.2f} €"])
```

`app_gimnasio/main.py`

```
from gimnasio.socios.servicios.registro import alta
from gimnasio.actividades.planificacion.horario import crear_horario, ocupacion
from gimnasio.facturacion.informes.cierre import informe_mensual


def main():
    socios = alta([("1A", "Ana", True, False), ("2B", "Luis", False, True), ("3C", "Eva", False, False)])
    horario = crear_horario([("Yoga", 2), ("Spinning", 10)])
    for s in socios:
        horario["Yoga"].inscribir(s)
    print("Ocupación:", ocupacion(horario))
    print(informe_mensual(socios))


if __name__ == "__main__":
    main()
```

🖥️ Salida de `python main.py`:

```
Ocupación: {'Yoga': '2/2', 'Spinning': '0/10'}
Ana     SocioPremium    60.50 €
Luis    Socio           29.04 €
Eva     Socio           36.30 €
TOTAL                  125.84 €
```

🖥️ Pruebas propias de los módulos (`python -m gimnasio.socios.servicios.registro`, `...planificacion.horario`, `...calculos.importes`):

```
[SocioPremium('Ana'), Socio('Luis')]
{'Yoga': '1/2', 'Spinning': '0/10'}
29.04
```

Parte 3 · Práctica 16 resuelta (OOP y programación funcional · clases 20 y 21)

Clase 20:

> Ejercicio 1 . Tomando como referencia el ejemplo anterior crear una Superclase llamada Vehículo con sus respectivos atributos comunes. Luego crear al menos tres subclases (con sus atributos adicionales) del tipo: Coche, Moto, Camión, Autobús. Crear por lo menos un objeto de cada subclase y muestra en pantalla los resultados.

✅ Solución del Ejercicio 1 (Vehículo y subclases)

```python
# Ejercicio 1 (clase 20): superclase Vehiculo + subclases Coche, Moto, Camion, Autobus
class Vehiculo:
    def __init__(self, matricula, marca, modelo, anio):
        self.matricula = matricula
        self.marca = marca
        self.modelo = modelo
        self.anio = anio

    def mostrar_informacion(self):
        print(f"[{type(self).__name__}] {self.marca} {self.modelo} ({self.anio}) - {self.matricula}")

class Coche(Vehiculo):
    def __init__(self, matricula, marca, modelo, anio, puertas):
        super().__init__(matricula, marca, modelo, anio)   # reutiliza el constructor del padre
        self.puertas = puertas                             # atributo adicional

    def mostrar_informacion(self):
        super().mostrar_informacion()                      # parte común
        print(f"   Puertas: {self.puertas}")               # parte específica

class Moto(Vehiculo):
    def __init__(self, matricula, marca, modelo, anio, cilindrada):
        super().__init__(matricula, marca, modelo, anio)
        self.cilindrada = cilindrada

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"   Cilindrada: {self.cilindrada} cc")

class Camion(Vehiculo):
    def __init__(self, matricula, marca, modelo, anio, carga_max_t):
        super().__init__(matricula, marca, modelo, anio)
        self.carga_max_t = carga_max_t

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"   Carga máxima: {self.carga_max_t} t")

class Autobus(Vehiculo):
    def __init__(self, matricula, marca, modelo, anio, plazas):
        super().__init__(matricula, marca, modelo, anio)
        self.plazas = plazas

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"   Plazas: {self.plazas}")

flota = [
    Coche("1111-AAA", "Seat", "León", 2021, 5),
    Moto("2222-BBB", "Yamaha", "MT-07", 2022, 689),
    Camion("3333-CCC", "Volvo", "FH16", 2019, 40),
    Autobus("4444-DDD", "Irizar", "i6", 2020, 55),
]
```

```
for v in flota:
    v.mostrar_informacion()
```

🖥️ **Salida:**

```
[Coche] Seat León (2021) - 1111-AAA
   Puertas: 5
[Moto] Yamaha MT-07 (2022) - 2222-BBB
   Cilindrada: 689 cc
[Camion] Volvo FH16 (2019) - 3333-CCC
   Carga máxima: 40 t
[Autobus] Irizar i6 (2020) - 4444-DDD
   Plazas: 55
```

Ejercicio P1. Sistema de gestión de empleados

Enunciado

> Una empresa tecnológica necesita desarrollar un programa para gestionar la información de sus empleados. Actualmente cuenta con tres tipos de trabajadores: desarrolladores, analistas de datos y administradores de sistemas.

Debes desarrollar una estructura de clases basada en herencia que permita representar a los distintos trabajadores sin repetir atributos ni métodos comunes.

Requisitos

1. Crear una superclase `Empleado` con los atributos `id_empleado`, `nombre`, `salario_base` y `departamento`.
2. Implementar un método `mostrar_informacion()` que muestre los datos del empleado.
3. Crear tres subclases:

- `Desarrollador`: añadir los atributos `lenguaje_principal` y `nivel`.
- `AnalistaDatos`: añadir los atributos `herramienta_bi` y `especialidad`.
- `AdministradorSistemas`: añadir los atributos `sistema_operativo` y `certificacion`.

4. Sobrescribir `mostrar_informacion()` en las tres subclases para incluir sus atributos específicos.
5. Crear al menos dos objetos de cada subclase.
6. Almacenar los seis objetos en una única lista llamada `empleados`.
7. Recorrer la lista e imprimir la información de todos los empleados. ##### Salida esperada (orientativa)

```
ID: E001
Nombre: Ana López
Departamento: Desarrollo
Salario base: 2800 €
Lenguaje principal: Python
Nivel: Senior

ID: E002
Nombre: Carlos Pérez
Departamento: Datos
Salario base: 2600 €
Herramienta BI: Power BI
Especialidad: Análisis de ventas

ID: E003
Nombre: Laura Gómez
Departamento: Infraestructura
Salario base: 2900 €
Sistema operativo: Linux
Certificación: AZ-104
```

La salida anterior representa únicamente tres empleados de ejemplo. Tu programa debe mostrar los seis.

✅ Solución

```python
class Empleado:
    def __init__(self, id_empleado, nombre, salario_base, departamento):
        self.id_empleado = id_empleado
        self.nombre = nombre
        self.salario_base = salario_base
        self.departamento = departamento

    def mostrar_informacion(self):
        print(f"ID: {self.id_empleado}")
        print(f"Nombre: {self.nombre}")
        print(f"Departamento: {self.departamento}")
        print(f"Salario base: {self.salario_base} €")

class Desarrollador(Empleado):
    def __init__(self, id_empleado, nombre, salario_base, departamento, lenguaje_principal, nivel):
        super().__init__(id_empleado, nombre, salario_base, departamento)
        self.lenguaje_principal = lenguaje_principal
        self.nivel = nivel

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"Lenguaje principal: {self.lenguaje_principal}")
        print(f"Nivel: {self.nivel}")

class AnalistaDatos(Empleado):
    def __init__(self, id_empleado, nombre, salario_base, departamento, herramienta_bi, especialidad):
        super().__init__(id_empleado, nombre, salario_base, departamento)
        self.herramienta_bi = herramienta_bi
        self.especialidad = especialidad

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"Herramienta BI: {self.herramienta_bi}")
        print(f"Especialidad: {self.especialidad}")

class AdministradorSistemas(Empleado):
    def __init__(self, id_empleado, nombre, salario_base, departamento, sistema_operativo, certificacion):
        super().__init__(id_empleado, nombre, salario_base, departamento)
        self.sistema_operativo = sistema_operativo
        self.certificacion = certificacion

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"Sistema operativo: {self.sistema_operativo}")
        print(f"Certificación: {self.certificacion}")

empleados = [
    Desarrollador("E001", "Ana López", 2800, "Desarrollo", "Python", "Senior"),
    AnalistaDatos("E002", "Carlos Pérez", 2600, "Datos", "Power BI", "Análisis de ventas"),
    AdministradorSistemas("E003", "Laura Gómez", 2900, "Infraestructura", "Linux", "AZ-104"),
    Desarrollador("E004", "Pablo Ruiz", 2400, "Desarrollo", "Java", "Junior"),
    AnalistaDatos("E005", "Marta Sanz", 2700, "Datos", "Tableau", "Marketing"),
    AdministradorSistemas("E006", "Jorge Gil", 3000, "Infraestructura", "Windows Server", "AZ-305"),
]
for e in empleados:
    e.mostrar_informacion()   # polimorfismo
    print()
```

🖥️ Salida:

```
ID: E001
Nombre: Ana López
Departamento: Desarrollo
Salario base: 2800 €
Lenguaje principal: Python
Nivel: Senior

ID: E002
Nombre: Carlos Pérez
Departamento: Datos
```

```
Salario base: 2600 €
Herramienta BI: Power BI
Especialidad: Análisis de ventas

ID: E003
Nombre: Laura Gómez
Departamento: Infraestructura
Salario base: 2900 €
Sistema operativo: Linux
Certificación: AZ-104

ID: E004
Nombre: Pablo Ruiz
Departamento: Desarrollo
Salario base: 2400 €
Lenguaje principal: Java
Nivel: Junior

ID: E005
Nombre: Marta Sanz
Departamento: Datos
Salario base: 2700 €
Herramienta BI: Tableau
Especialidad: Marketing

ID: E006
Nombre: Jorge Gil
Departamento: Infraestructura
Salario base: 3000 €
Sistema operativo: Windows Server
Certificación: AZ-305
```

Ejercicio P2. Refactorización de un sistema de dispositivos IoT

Enunciado

> Una empresa especializada en soluciones de *Smart Agriculture* ha desarrollado un programa para gestionar los dispositivos IoT instalados en diferentes explotaciones agrícolas. El programa funciona, pero presenta un problema de diseño: **todos los tipos de sensores se representan mediante una única clase**, que contiene atributos que no corresponden a todos los dispositivos.

Tu tarea consiste en **refactorizar el código utilizando herencia**, creando una superclase y cuatro clases hijas.

Código original que debes refactorizar

```python
class DispositivoIoT:
    def __init__(
        self,
        device_id,
        nombre,
        ubicacion,
        tipo,
        valor,
        unidad,
        profundidad=None,
        calibracion=None,
        altitud=None,
        precision=None,
        orientacion=None,
        velocidad_maxima=None,
        capacidad=None,
        nivel_bateria=None
    ):
        self.device_id = device_id
        self.nombre = nombre
        self.ubicacion = ubicacion
        self.tipo = tipo
```

```
        self.valor = valor
        self.unidad = unidad

        self.profundidad = profundidad
        self.calibracion = calibracion
        self.altitud = altitud
        self.precision = precision
        self.orientacion = orientacion
        self.velocidad_maxima = velocidad_maxima
        self.capacidad = capacidad
        self.nivel_bateria = nivel_bateria

    def mostrar_informacion(self):
        print("ID:", self.device_id)
        print("Nombre:", self.nombre)
        print("Ubicación:", self.ubicacion)
        print("Tipo:", self.tipo)
        print("Valor:", self.valor, self.unidad)

        if self.tipo == "HUMEDAD":
            print("Profundidad:", self.profundidad)
            print("Calibración:", self.calibracion)

        elif self.tipo == "TEMPERATURA":
            print("Altitud:", self.altitud)
            print("Precisión:", self.precision)

        elif self.tipo == "VIENTO":
            print("Orientación:", self.orientacion)
            print("Velocidad máxima:", self.velocidad_maxima)

        elif self.tipo == "TANQUE":
            print("Capacidad:", self.capacidad)
            print("Nivel de batería:", self.nivel_bateria)


sensor1 = DispositivoIoT(
    "DEV001", "Sensor de suelo", "Parcela A",
    "HUMEDAD", 45, "%",
    profundidad=30, calibracion="Automática"
)

sensor2 = DispositivoIoT(
    "DEV002", "Sensor ambiental", "Parcela B",
    "TEMPERATURA", 26, "°C",
    altitud=650, precision=0.5
)

sensor3 = DispositivoIoT(
    "DEV003", "Anemómetro", "Parcela C",
    "VIENTO", 18, "km/h",
    orientacion="Norte", velocidad_maxima=80
)

sensor4 = DispositivoIoT(
    "DEV004", "Tanque principal", "Parcela A",
    "TANQUE", 75, "%",
    capacidad=5000, nivel_bateria=90
)

dispositivos = [sensor1, sensor2, sensor3, sensor4]

for dispositivo in dispositivos:
    dispositivo.mostrar_informacion()
    print("-" * 40)
```

Requisitos de refactorización

Debes transformar el programa original en una estructura de cinco clases:

La clase `DispositivoIoT` debe contener únicamente los atributos compartidos por todos los dispositivos: `device_id`, `nombre`, `ubicacion`, `valor` y `unidad`.

También debe implementar el método `mostrar_informacion()`.

Cada subclase debe añadir exclusivamente los atributos que necesita:

Subclase	Atributos adicionales
`SensorHumedad`	`profundidad`, `calibracion`
`SensorTemperatura`	`altitud`, `precision`
`SensorViento`	`orientacion`, `velocidad_maxima`
`SensorTanque`	`capacidad`, `nivel_bateria`

Cada subclase debe sobrescribir el método `mostrar_informacion()` para mostrar los atributos comunes y los específicos de su dispositivo.

8. Crear al menos dos objetos de cada subclase, obteniendo un total de ocho dispositivos.
9. Almacenar todos los objetos en una única lista llamada `dispositivos`.
10. Recorrer la lista y ejecutar `mostrar_informacion()` para cada dispositivo.
11. Utilizar `isinstance()` para identificar el tipo de dispositivo y mostrar un mensaje personalizado.
12. Comprobar que los objetos no contienen atributos innecesarios utilizando `vars()`. ##### Salida esperada (orientativa)

```
ID: DEV001
Nombre: Sensor de suelo
Ubicación: Parcela A
Valor: 45 %
Profundidad: 30 cm
Calibración: Automática
Categoría: Sensor de humedad
----------------------------------------

ID: DEV002
Nombre: Sensor ambiental
Ubicación: Parcela B
Valor: 26 °C
Altitud: 650 m
Precisión: 0.5 °C
Categoría: Sensor de temperatura
----------------------------------------
```

Resultado esperado: El nuevo programa debe conservar la funcionalidad original, eliminar los atributos innecesarios y aprovechar la herencia para reducir la duplicación de código.

✅ Solución

```python
class DispositivoIoT:
    def __init__(self, device_id, nombre, ubicacion, valor, unidad):
        self.device_id = device_id
        self.nombre = nombre
        self.ubicacion = ubicacion
        self.valor = valor
        self.unidad = unidad

    def mostrar_informacion(self):
        print("ID:", self.device_id)
        print("Nombre:", self.nombre)
        print("Ubicación:", self.ubicacion)
        print("Valor:", self.valor, self.unidad)

class SensorHumedad(DispositivoIoT):
    def __init__(self, device_id, nombre, ubicacion, valor, unidad, profundidad, calibracion):
        super().__init__(device_id, nombre, ubicacion, valor, unidad)
        self.profundidad = profundidad
        self.calibracion = calibracion

    def mostrar_informacion(self):
```

```
        super().mostrar_informacion()
        print("Profundidad:", self.profundidad, "cm")
        print("Calibración:", self.calibracion)

class SensorTemperatura(DispositivoIoT):
    def __init__(self, device_id, nombre, ubicacion, valor, unidad, altitud, precision):
        super().__init__(device_id, nombre, ubicacion, valor, unidad)
        self.altitud = altitud
        self.precision = precision

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print("Altitud:", self.altitud, "m")
        print("Precisión:", self.precision, "°C")

class SensorViento(DispositivoIoT):
    def __init__(self, device_id, nombre, ubicacion, valor, unidad, orientacion, velocidad_maxima):
        super().__init__(device_id, nombre, ubicacion, valor, unidad)
        self.orientacion = orientacion
        self.velocidad_maxima = velocidad_maxima

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print("Orientación:", self.orientacion)
        print("Velocidad máxima:", self.velocidad_maxima, "km/h")

class SensorTanque(DispositivoIoT):
    def __init__(self, device_id, nombre, ubicacion, valor, unidad, capacidad, nivel_bateria):
        super().__init__(device_id, nombre, ubicacion, valor, unidad)
        self.capacidad = capacidad
        self.nivel_bateria = nivel_bateria

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print("Capacidad:", self.capacidad, "L")
        print("Nivel de batería:", self.nivel_bateria, "%")

dispositivos = [
    SensorHumedad("DEV001", "Sensor de suelo", "Parcela A", 45, "%", 30, "Automática"),
    SensorTemperatura("DEV002", "Sensor ambiental", "Parcela B", 26, "°C", 650, 0.5),
    SensorViento("DEV003", "Anemómetro", "Parcela C", 18, "km/h", "Norte", 80),
    SensorTanque("DEV004", "Tanque principal", "Parcela A", 75, "%", 5000, 90),
    SensorHumedad("DEV005", "Sensor de raíz", "Parcela D", 38, "%", 50, "Manual"),
    SensorTemperatura("DEV006", "Sensor invernadero", "Parcela E", 31, "°C", 640, 0.2),
    SensorViento("DEV007", "Veleta", "Parcela F", 25, "km/h", "Sur", 95),
    SensorTanque("DEV008", "Tanque auxiliar", "Parcela B", 40, "%", 2000, 65),
]

for d in dispositivos:
    d.mostrar_informacion()
    # isinstance() para un mensaje personalizado (lo pide el enunciado)
    if isinstance(d, SensorHumedad):
        print("Categoría: Sensor de humedad")
    elif isinstance(d, SensorTemperatura):
        print("Categoría: Sensor de temperatura")
    elif isinstance(d, SensorViento):
        print("Categoría: Sensor de viento")
    elif isinstance(d, SensorTanque):
        print("Categoría: Sensor de tanque")
    print("-" * 40)

# vars() → diccionario con los atributos REALES del objeto: ya no hay atributos None sobrantes
print(vars(dispositivos[0]))
print(vars(dispositivos[3]))
print("Atributos por objeto:", {d.device_id: len(vars(d)) for d in dispositivos})
```

🖥️ Salida:

```
ID: DEV001
Nombre: Sensor de suelo
Ubicación: Parcela A
Valor: 45 %
Profundidad: 30 cm
```

```
Calibración: Automática
Categoría: Sensor de humedad
----------------------------------------
ID: DEV002
Nombre: Sensor ambiental
Ubicación: Parcela B
Valor: 26 °C
Altitud: 650 m
Precisión: 0.5 °C
Categoría: Sensor de temperatura
----------------------------------------
ID: DEV003
Nombre: Anemómetro
Ubicación: Parcela C
Valor: 18 km/h
Orientación: Norte
Velocidad máxima: 80 km/h
Categoría: Sensor de viento
----------------------------------------
ID: DEV004
Nombre: Tanque principal
Ubicación: Parcela A
Valor: 75 %
Capacidad: 5000 L
Nivel de batería: 90 %
Categoría: Sensor de tanque
----------------------------------------
ID: DEV005
Nombre: Sensor de raíz
Ubicación: Parcela D
Valor: 38 %
Profundidad: 50 cm
Calibración: Manual
Categoría: Sensor de humedad
----------------------------------------
ID: DEV006
Nombre: Sensor invernadero
Ubicación: Parcela E
Valor: 31 °C
Altitud: 640 m
Precisión: 0.2 °C
Categoría: Sensor de temperatura
----------------------------------------
ID: DEV007
Nombre: Veleta
Ubicación: Parcela F
Valor: 25 km/h
Orientación: Sur
Velocidad máxima: 95 km/h
Categoría: Sensor de viento
----------------------------------------
ID: DEV008
Nombre: Tanque auxiliar
Ubicación: Parcela B
Valor: 40 %
Capacidad: 2000 L
Nivel de batería: 65 %
Categoría: Sensor de tanque
----------------------------------------
{'device_id': 'DEV001', 'nombre': 'Sensor de suelo', 'ubicacion': 'Parcela A', 'valor': 45, 'unidad': '%',
'profundidad': 30, 'calibracion': 'Automática'}
{'device_id': 'DEV004', 'nombre': 'Tanque principal', 'ubicacion': 'Parcela A', 'valor': 75, 'unidad': '%',
'capacidad': 5000, 'nivel_bateria': 90}
Atributos por objeto: {'DEV001': 7, 'DEV002': 7, 'DEV003': 7, 'DEV004': 7, 'DEV005': 7, 'DEV006': 7,
'DEV007': 7, 'DEV008': 7}
```

Ejercicio P3. Gestión de recursos de un laboratorio de Big Data

Enunciado

> Un centro de formación tecnológica necesita desarrollar una aplicación para gestionar los recursos informáticos utilizados en sus laboratorios de Big Data e Inteligencia Artificial. El laboratorio dispone de

> ordenadores, servidores y dispositivos de procesamiento Edge AI.

Debes implementar un sistema basado en herencia para organizar estos recursos, consultar su información y calcular su consumo eléctrico.

Requisitos

Desarrollar una clase `RecursoComputacional` con los siguientes atributos:

- `id_recurso`: identificador único.
- `nombre`: nombre del equipo.
- `fabricante`: fabricante del dispositivo.
- `potencia_w`: potencia eléctrica en vatios.
- `horas_uso`: horas acumuladas de funcionamiento. La superclase debe incluir estos métodos:
- `mostrar_informacion()`: muestra los atributos comunes.
- `calcular_consumo()`: devuelve el consumo eléctrico acumulado en kWh, calculado como `(potencia_w * horas_uso) / 1000`. | Subclase | Atributos específicos | | — | — | | `Ordenador` | `procesador`, `ram_gb`, `sistema_operativo` | | `Servidor` | `num_cpu`, `ram_gb`, `almacenamiento_tb` | | `DispositivoEdge` | `modelo_gpu`, `memoria_gpu_gb`, `tipo_inferencia` |

Cada subclase debe sobrescribir el método `mostrar_informacion()` para mostrar sus atributos específicos.

13. Crear al menos dos objetos de cada subclase.
14. Guardar todos los objetos en una lista `recursos`.
15. Recorrer la lista para mostrar la información de cada recurso.
16. Calcular el consumo eléctrico individual de cada dispositivo.
17. Calcular el consumo total acumulado de todos los recursos.
18. Utilizar `isinstance()` para contar cuántos objetos hay de cada subclase.
19. Identificar qué dispositivo tiene el mayor consumo eléctrico. ##### Salida esperada (orientativa)

```
========== LABORATORIO BIG DATA ==========

Recurso: PC-01
Tipo: Ordenador
Fabricante: Dell
Procesador: Intel Core i7
RAM: 32 GB
Sistema operativo: Windows 11
Consumo acumulado: 12.5 kWh

Recurso: SRV-01
Tipo: Servidor
Fabricante: HPE
Número de CPU: 2
RAM: 128 GB
Almacenamiento: 4 TB
Consumo acumulado: 42.0 kWh

========== RESUMEN DEL LABORATORIO ==========

Ordenadores: 2
Servidores: 2
Dispositivos Edge: 2

Consumo total: 78.5 kWh
Dispositivo de mayor consumo: SRV-01
```

> Los valores anteriores son orientativos y deben corresponder a los datos de los objetos que crees.

> 📖 Ejercicio 2. Crea 2 ejemplos de polimorfismo mediante ***duck typing*** **.**

✅ Solución

```
class RecursoComputacional:
    def __init__(self, id_recurso, nombre, fabricante, potencia_w, horas_uso):
        self.id_recurso = id_recurso
        self.nombre = nombre
        self.fabricante = fabricante
        self.potencia_w = potencia_w
        self.horas_uso = horas_uso

    def mostrar_informacion(self):
        print(f"Recurso: {self.id_recurso}")
        print(f"Tipo: {type(self).__name__}")
        print(f"Fabricante: {self.fabricante}")

    def calcular_consumo(self):
        return (self.potencia_w * self.horas_uso) / 1000      # kWh (se hereda sin cambios)

class Ordenador(RecursoComputacional):
    def __init__(self, id_recurso, nombre, fabricante, potencia_w, horas_uso,
                 procesador, ram_gb, sistema_operativo):
        super().__init__(id_recurso, nombre, fabricante, potencia_w, horas_uso)
        self.procesador = procesador
        self.ram_gb = ram_gb
        self.sistema_operativo = sistema_operativo

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"Procesador: {self.procesador}")
        print(f"RAM: {self.ram_gb} GB")
        print(f"Sistema operativo: {self.sistema_operativo}")

class Servidor(RecursoComputacional):
    def __init__(self, id_recurso, nombre, fabricante, potencia_w, horas_uso,
                 num_cpu, ram_gb, almacenamiento_tb):
        super().__init__(id_recurso, nombre, fabricante, potencia_w, horas_uso)
        self.num_cpu = num_cpu
        self.ram_gb = ram_gb
        self.almacenamiento_tb = almacenamiento_tb

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"Número de CPU: {self.num_cpu}")
        print(f"RAM: {self.ram_gb} GB")
        print(f"Almacenamiento: {self.almacenamiento_tb} TB")

class DispositivoEdge(RecursoComputacional):
    def __init__(self, id_recurso, nombre, fabricante, potencia_w, horas_uso,
                 modelo_gpu, memoria_gpu_gb, tipo_inferencia):
        super().__init__(id_recurso, nombre, fabricante, potencia_w, horas_uso)
        self.modelo_gpu = modelo_gpu
        self.memoria_gpu_gb = memoria_gpu_gb
        self.tipo_inferencia = tipo_inferencia

    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"GPU: {self.modelo_gpu} ({self.memoria_gpu_gb} GB)")
        print(f"Inferencia: {self.tipo_inferencia}")

recursos = [
    Ordenador("PC-01", "Puesto 1", "Dell", 250, 50, "Intel Core i7", 32, "Windows 11"),
    Ordenador("PC-02", "Puesto 2", "HP", 200, 30, "AMD Ryzen 7", 16, "Ubuntu 24.04"),
    Servidor("SRV-01", "Nodo Spark", "HPE", 600, 70, 2, 128, 4),
    Servidor("SRV-02", "Nodo HDFS", "Dell", 500, 20, 1, 64, 8),
    DispositivoEdge("EDGE-01", "Cámara IA", "NVIDIA", 30, 100, "Jetson Orin Nano", 8, "Visión"),
    DispositivoEdge("EDGE-02", "Robot IA", "NVIDIA", 50, 100, "Jetson AGX Orin", 32, "Voz"),
]

print("========== LABORATORIO BIG DATA ==========\n")
```

```python
for r in recursos:
    r.mostrar_informacion()
    print(f"Consumo acumulado: {r.calcular_consumo()} kWh\n")

print("========== RESUMEN DEL LABORATORIO ==========\n")
print("Ordenadores:", sum(1 for r in recursos if isinstance(r, Ordenador)))
print("Servidores:", sum(1 for r in recursos if isinstance(r, Servidor)))
print("Dispositivos Edge:", sum(1 for r in recursos if isinstance(r, DispositivoEdge)))
print(f"\nConsumo total: {sum(r.calcular_consumo() for r in recursos)} kWh")
mayor = max(recursos, key=lambda r: r.calcular_consumo())
print("Dispositivo de mayor consumo:", mayor.id_recurso)
```

🖥️ Salida:

```
========== LABORATORIO BIG DATA ==========

Recurso: PC-01
Tipo: Ordenador
Fabricante: Dell
Procesador: Intel Core i7
RAM: 32 GB
Sistema operativo: Windows 11
Consumo acumulado: 12.5 kWh

Recurso: PC-02
Tipo: Ordenador
Fabricante: HP
Procesador: AMD Ryzen 7
RAM: 16 GB
Sistema operativo: Ubuntu 24.04
Consumo acumulado: 6.0 kWh

Recurso: SRV-01
Tipo: Servidor
Fabricante: HPE
Número de CPU: 2
RAM: 128 GB
Almacenamiento: 4 TB
Consumo acumulado: 42.0 kWh

Recurso: SRV-02
Tipo: Servidor
Fabricante: Dell
Número de CPU: 1
RAM: 64 GB
Almacenamiento: 8 TB
Consumo acumulado: 10.0 kWh

Recurso: EDGE-01
Tipo: DispositivoEdge
Fabricante: NVIDIA
GPU: Jetson Orin Nano (8 GB)
Inferencia: Visión
Consumo acumulado: 3.0 kWh

Recurso: EDGE-02
Tipo: DispositivoEdge
Fabricante: NVIDIA
GPU: Jetson AGX Orin (32 GB)
Inferencia: Voz
Consumo acumulado: 5.0 kWh

========== RESUMEN DEL LABORATORIO ==========

Ordenadores: 2
Servidores: 2
Dispositivos Edge: 2

Consumo total: 78.5 kWh
Dispositivo de mayor consumo: SRV-01
```

✅ Solución del Ejercicio 2 (duck typing)

```python
# Ejercicio 2: polimorfismo por DUCK TYPING ("si camina como un pato y hace cuac, es un pato").
# Las clases NO comparten padre: solo comparten el NOMBRE del método.

# --- Ejemplo 1: exportadores de informes ---
class ExportadorCSV:
    def guardar(self, datos):
        return "CSV  → " + ",".join(map(str, datos))

class ExportadorJSON:
    def guardar(self, datos):
        return f"JSON → {{\"datos\": {list(datos)}}}"

class ExportadorPantalla:
    def guardar(self, datos):
        return f"PANTALLA → {len(datos)} valores: {datos}"

def exportar(exportador, datos):      # no sabe (ni le importa) de qué clase es
    print(exportador.guardar(datos))

for exp in [ExportadorCSV(), ExportadorJSON(), ExportadorPantalla()]:
    exportar(exp, [10, 20, 30])

# --- Ejemplo 2: fuentes de datos con el método leer() ---
class FicheroLocal:
    def leer(self):
        return ["fila1", "fila2"]

class APIRest:
    def leer(self):
        return [{"id": 1}, {"id": 2}, {"id": 3}]

class Sensor:
    def leer(self):
        return [21.5]

for fuente in [FicheroLocal(), APIRest(), Sensor()]:
    print(f"{type(fuente).__name__}: {len(fuente.leer())} registros")

# Un objeto SIN el método rompe el "contrato" → AttributeError
try:
    exportar("soy un texto", [1])
except AttributeError as e:
    print("AttributeError:", e)
```

🖥️ Salida:

```
CSV  → 10,20,30
JSON → {"datos": [10, 20, 30]}
PANTALLA → 3 valores: [10, 20, 30]
FicheroLocal: 2 registros
APIRest: 3 registros
Sensor: 1 registros
AttributeError: 'str' object has no attribute 'guardar'
```

Ejercicio P4: Polimorfismo en un sistema de procesamiento de datos

> 📖 #### 📌 Enunciado

Una empresa de Ingeniería de Datos necesita desarrollar una aplicación que permita procesar archivos procedentes de diferentes fuentes.

Actualmente, la empresa trabaja con tres formatos de archivos: **CSV, JSON y TXT**.

Cada tipo de archivo requiere un procesamiento diferente, pero todos deben permitir ejecutar una misma acción: `procesar()`.

Debes desarrollar una solución utilizando **programación orientada a objetos, herencia simple y polimorfismo**, sin utilizar `super()` ni herencia múltiple.

Parte A. Crear la superclase

Crea una clase llamada `ArchivoDatos` que contenga los siguientes atributos:

Atributo	Descripción
`nombre`	Nombre del archivo.
`tamano`	Tamaño del archivo en MB.

La clase debe implementar estos métodos:

- `mostrar_informacion()`: muestra el nombre y el tamaño del archivo.
- `procesar()`: muestra el mensaje `"Procesamiento genérico de archivo"`. #### Parte B. Crear las subclases

Crea tres clases hijas que hereden de `ArchivoDatos`.

Cada clase debe sobrescribir el método `procesar()` para mostrar un comportamiento diferente.

Subclase	Comportamiento de `procesar()`
`ArchivoCSV`	Lee registros separados por comas y los transforma en filas.
`ArchivoJSON`	Interpreta objetos JSON y extrae sus propiedades.
`ArchivoTXT`	Lee las líneas de texto y realiza una limpieza básica.

Cada clase hija debe utilizar los atributos heredados `nombre` y `tamano`.

Parte C. Crear los objetos

Crea los siguientes objetos:

```
archivo1 = ArchivoCSV("ventas.csv", 15)
archivo2 = ArchivoJSON("sensores.json", 8)
archivo3 = ArchivoTXT("logs.txt", 25)
```

Parte D. Aplicar polimorfismo

Crea una función llamada `ejecutar_procesamiento()` que reciba un objeto de cualquiera de las tres subclases.

La función debe ejecutar los métodos:

```
archivo.mostrar_informacion()
archivo.procesar()
```

Importante: La función no debe utilizar `if`, `elif`, `isinstance()` ni `type()` para determinar qué clase de archivo está procesando.

Parte E. Procesar múltiples archivos

Crea una lista que contenga los tres objetos anteriores.

Recorre la lista utilizando un bucle `for` y llama a la función `ejecutar_procesamiento()` para cada objeto.

Salida esperada

```
Nombre: ventas.csv
Tamaño: 15 MB
Procesando archivo CSV: ventas.csv
Leyendo registros separados por comas...

Nombre: sensores.json
Tamaño: 8 MB
```

```
Procesando archivo JSON: sensores.json
Extrayendo propiedades de los objetos JSON...

Nombre: logs.txt
Tamaño: 25 MB
Procesando archivo TXT: logs.txt
Leyendo líneas y realizando limpieza básica...
```

Parte F. Ampliación del sistema

La empresa comienza a recibir archivos de un nuevo formato: **XML**.

Debes incorporar una nueva clase llamada `ArchivoXML` que herede de `ArchivoDatos`.

Esta clase debe sobrescribir `procesar()` para mostrar:

```
Procesando archivo XML: configuracion.xml
Analizando etiquetas y atributos XML...
```

Crea un objeto `ArchivoXML`, añádelo a la lista anterior y comprueba que puede procesarse correctamente.

Condición importante: No puedes modificar la función `ejecutar_procesamiento()` para incorporar el nuevo formato.

Preguntas

20. ¿Qué métodos heredan las clases hijas de `ArchivoDatos`?
21. ¿Por qué las subclases sobrescriben el método `procesar()`?
22. ¿Dónde se aplica exactamente el polimorfismo en este ejercicio?
23. ¿Por qué no necesitamos utilizar `if` o `elif` para identificar cada tipo de archivo?
24. ¿Qué ventaja ofrece el polimorfismo cuando incorporamos la clase `ArchivoXML`?

> **Restricciones del ejercicio:** se permite herencia simple, sobrescritura de métodos, listas de objetos y funciones que reciben objetos como argumentos. No se permite utilizar `super()`, herencia múltiple ni estructuras condicionales para seleccionar el comportamiento de cada subclase.

Nota: No es necesario leer archivos reales del disco. El procesamiento se simulará mostrando los mensajes correspondientes mediante `print()`.

Respuestas a las preguntas:

1. Heredan `__init__()`, `mostrar_informacion()` y `procesar()` (este último lo sobrescriben).
2. Porque cada formato se procesa de forma distinta: la versión genérica no sirve.
3. En la llamada `archivo.procesar()` dentro de `ejecutar_procesamiento()`: la misma línea ejecuta un método distinto según la clase real del objeto.
4. Porque Python decide en tiempo de ejecución qué método usar según la clase del objeto (enlace dinámico); cada clase “sabe” procesarse.
5. Se añade un formato nuevo creando una clase, **sin modificar** el código existente (principio abierto/cerrado).

✅ Solución

```python
class ArchivoDatos:
    def __init__(self, nombre, tamano):
        self.nombre = nombre
        self.tamano = tamano

    def mostrar_informacion(self):
        print(f"Nombre: {self.nombre}")
        print(f"Tamaño: {self.tamano} MB")

    def procesar(self):
```

```
        print("Procesamiento genérico de archivo")

# Las subclases NO definen __init__ (lo heredan) y NO usan super(): solo sobrescriben procesar()
class ArchivoCSV(ArchivoDatos):
    def procesar(self):
        print(f"Procesando archivo CSV: {self.nombre}")
        print("Leyendo registros separados por comas...")

class ArchivoJSON(ArchivoDatos):
    def procesar(self):
        print(f"Procesando archivo JSON: {self.nombre}")
        print("Extrayendo propiedades de los objetos JSON...")

class ArchivoTXT(ArchivoDatos):
    def procesar(self):
        print(f"Procesando archivo TXT: {self.nombre}")
        print("Leyendo líneas y realizando limpieza básica...")

def ejecutar_procesamiento(archivo):     # sin if / isinstance / type
    archivo.mostrar_informacion()
    archivo.procesar()                   # ← AQUÍ está el polimorfismo

archivo1 = ArchivoCSV("ventas.csv", 15)
archivo2 = ArchivoJSON("sensores.json", 8)
archivo3 = ArchivoTXT("logs.txt", 25)
archivos = [archivo1, archivo2, archivo3]

for archivo in archivos:
    ejecutar_procesamiento(archivo)
    print()

# Parte F: nuevo formato SIN tocar ejecutar_procesamiento()
class ArchivoXML(ArchivoDatos):
    def procesar(self):
        print(f"Procesando archivo XML: {self.nombre}")
        print("Analizando etiquetas y atributos XML...")

archivos.append(ArchivoXML("configuracion.xml", 2))
ejecutar_procesamiento(archivos[-1])
```

🖥️ Salida:

```
Nombre: ventas.csv
Tamaño: 15 MB
Procesando archivo CSV: ventas.csv
Leyendo registros separados por comas...

Nombre: sensores.json
Tamaño: 8 MB
Procesando archivo JSON: sensores.json
Extrayendo propiedades de los objetos JSON...

Nombre: logs.txt
Tamaño: 25 MB
Procesando archivo TXT: logs.txt
Leyendo líneas y realizando limpieza básica...

Nombre: configuracion.xml
Tamaño: 2 MB
Procesando archivo XML: configuracion.xml
Analizando etiquetas y atributos XML...
```

Clase 21

Ejercicio Propuesto P1: Sistema de análisis de envíos logísticos

Enunciado

> 📖 Una empresa de logística recibe diariamente pedidos de envío. Cada pedido tiene un identificador, un destino y un peso, pero **el coste de transporte depende del tipo de envío**. El equipo de Ingeniería de Datos quiere analizar estos pedidos sin modificar los registros originales. Para ello, debes construir un programa que combine:

- **Programación orientada a objetos:** para representar los envíos.
- **Herencia y polimorfismo:** para calcular sus costes según el tipo.
- **Comprehensions:** para filtrar y transformar colecciones de objetos.
- **Programación funcional:** para pasar funciones como argumentos y crear funciones que devuelvan otras funciones. El resultado será un pequeño informe con los envíos caros, los tipos de envío disponibles y los importes finales tras aplicar un descuento.

Reglas del negocio

Todos los envíos incluyen los siguientes datos:

Atributo	Descripción
`envio_id`	Identificador único, por ejemplo `E01`
`destino`	Ciudad de destino
`peso`	Peso del paquete en kilogramos

Hay tres modalidades, con las siguientes tarifas:

Tipo de envío	Regla para calcular el coste (€)
Nacional	`peso * 2 + 3`
Internacional	`peso * 2 + 15`
Urgente	`(peso * 2) * 1.5`

> **Importante:** Las tarifas son ficticias y se utilizan únicamente con fines educativos. Los importes se expresan en euros.

Parte A. Crear la jerarquía de clases

A.1. Superclase `Envio`

Implementa una clase `Envio` que:

25. Reciba `envio_id`, `destino` y `peso` en su constructor.
26. Almacene esos datos como atributos de instancia.
27. Implemente `calcular_coste()` con la tarifa base `peso * 2`.
28. Implemente `mostrar_informacion()`, que devuelva un **string** con el identificador, el destino y el peso.
    ##### A.2. Subclases

Crea tres clases hijas de `Envio`:

- `EnvioNacional`
- `EnvioInternacional`
- `EnvioUrgente` Cada subclase debe sobrescribir `calcular_coste()` para aplicar su tarifa. **Usa `super().calcular_coste()`** para obtener el coste base, sin repetir el cálculo `peso * 2`.

No es necesario añadir atributos nuevos a estas subclases ni redefinir sus constructores.

Parte B. Crear objetos y aplicar polimorfismo

Una vez implementadas las clases, crea **exactamente estos siete objetos**, en el orden indicado:

```
# Ejecuta este bloque después de definir las cuatro clases.
envios = [
    EnvioNacional("E01", "Madrid", 2),
    EnvioInternacional("E02", "París", 3),
    EnvioUrgente("E03", "Sevilla", 4),
    EnvioNacional("E04", "Valencia", 5),
    EnvioInternacional("E05", "Roma", 1),
    EnvioUrgente("E06", "Bilbao", 2),
    EnvioNacional("E07", "Málaga", 1),
]
```

Recorre la colección y muestra para cada objeto su identificador y su coste. Debes utilizar **la misma llamada `envio.calcular_coste()`** independientemente de la clase concreta del objeto.

Pregunta para responder en un comentario de código: ¿Por qué esta llamada ejecuta una versión diferente de `calcular_coste()` dependiendo del objeto?

Parte C. Aplicar comprehensions

Resuelve los tres apartados **sin utilizar `for` convencionales para construir las colecciones**. Puedes usar `for` dentro de las comprehensions.

C.1. List comprehension: envíos de coste elevado

Crea una lista `envios_caros` con los **identificadores** de los envíos cuyo coste sea **mayor o igual a 12 €**.

Usa `calcular_coste()` dentro de la condición.

C.2. Set comprehension: modalidades utilizadas

Crea un conjunto `tipos_envio` con los nombres de las clases de los objetos que aparecen en `envios`.

Pista: puedes obtener el nombre de la clase de un objeto con `type(objeto).__name__`.

C.3. Dictionary comprehension: costes por envío

Crea un diccionario `costes_por_envio` que relacione cada `envio_id` con su coste calculado. Incluye **todos** los envíos.

Parte D. Aplicar programación funcional

D.1. Funciones como argumentos

Implementa la siguiente función genérica:

```
def aplicar_operacion(funcion, valor):
    # Completar
    pass
```

Debe recibir una función y un valor, ejecutar la función recibida sobre ese valor y devolver el resultado.

Después, crea una función `redondear_euros(importe)` que devuelva el importe redondeado a dos decimales.

Comprueba que puedes llamar a `aplicar_operacion(redondear_euros, 12.345)` y obtener `12.35`.

D.2. Función que devuelve otra función

Implementa `crear_descuento(porcentaje)`. Esta función debe **devolver otra función** que reciba un importe y aplique el descuento indicado.

Ejemplo del comportamiento requerido:

```
descontar_10 = crear_descuento(10)
print(descontar_10(100))  # 90.0
```

La función interna debe conservar el porcentaje recibido por la función externa.

D.3. Combinación de programación funcional y dictionary comprehension

Construye `costes_con_descuento`, un nuevo diccionario que:

29. Incluya **solo** los envíos cuyo coste original sea mayor o igual a 12 €.
30. Use el identificador del envío como clave.
31. Use como valor el coste tras aplicar un **10 % de descuento**.
32. Calcule ese valor combinando `aplicar_operacion()`, la función creada con `crear_descuento(10)` y `redondear_euros()`.
33. No modifique ningún objeto ni el diccionario `costes_por_envio`. > Puedes realizar las dos operaciones sobre el valor con llamadas anidadas a `aplicar_operacion()`.

Parte E. Mostrar el informe final

El programa debe imprimir los siguientes apartados, en este orden:

34. Coste calculado de cada envío.
35. Identificadores de `envios_caros`.
36. Modalidades de `tipos_envio`.
37. Diccionario `costes_por_envio`.
38. Diccionario `costes_con_descuento`. Para presentar `tipos_envio` de forma ordenada, utiliza `sorted(tipos_envio)`; el conjunto original no tiene un orden garantizado.

Resultados esperados

Puedes utilizar estos resultados para comprobar tu implementación. **No son una solución del código**.

```
E01: 7.0 €
E02: 21.0 €
E03: 12.0 €
E04: 13.0 €
E05: 17.0 €
E06: 6.0 €
E07: 5.0 €

Envíos caros: ['E02', 'E03', 'E04', 'E05']

Tipos de envío: ['EnvioInternacional', 'EnvioNacional', 'EnvioUrgente']

Costes por envío: {'E01': 7.0, 'E02': 21.0, 'E03': 12.0, 'E04': 13.0, 'E05': 17.0, 'E06': 6.0, 'E07': 5.0}

Costes con descuento: {'E02': 18.9, 'E03': 10.8, 'E04': 11.7, 'E05': 15.3}
```

8. Restricciones

- Deben existir **cuatro clases**: una superclase y tres subclases.
- Las tres subclases deben usar **herencia, sobrescritura de métodos y** `super()`.
- El cálculo del coste debe realizarse mediante **polimorfismo**; evita cadenas de `if/elif` que comprueben el tipo de envío.

- Debes emplear al menos **una list comprehension, una set comprehension y dos dictionary comprehensions**.
- Debes incluir **una función que reciba otra función** y **una función que devuelva otra función**.
- No utilices `map()`, `filter()`, `reduce()`, Pandas ni bibliotecas externas: trabaja con los conceptos estudiados.
- Los objetos y los datos originales no deben modificarse durante la generación del informe.

✅ Solución

```
class Envio:
    def __init__(self, envio_id, destino, peso):
        self.envio_id = envio_id
        self.destino = destino
        self.peso = peso

    def calcular_coste(self):
        return self.peso * 2                          # tarifa base

    def mostrar_informacion(self):
        return f"{self.envio_id} → {self.destino} ({self.peso} kg)"

class EnvioNacional(Envio):
    def calcular_coste(self):
        return super().calcular_coste() + 3

class EnvioInternacional(Envio):
    def calcular_coste(self):
        return super().calcular_coste() + 15

class EnvioUrgente(Envio):
    def calcular_coste(self):
        return super().calcular_coste() * 1.5

envios = [
    EnvioNacional("E01", "Madrid", 2),
    EnvioInternacional("E02", "París", 3),
    EnvioUrgente("E03", "Sevilla", 4),
    EnvioNacional("E04", "Valencia", 5),
    EnvioInternacional("E05", "Roma", 1),
    EnvioUrgente("E06", "Bilbao", 2),
    EnvioNacional("E07", "Málaga", 1),
]

# B. Polimorfismo: la misma llamada envio.calcular_coste() ejecuta la versión de la CLASE
#    REAL del objeto; Python la busca primero en la subclase (enlace dinámico).
for envio in envios:
    print(f"{envio.envio_id}: {float(envio.calcular_coste())} €")

# C. Comprehensions
envios_caros = [e.envio_id for e in envios if e.calcular_coste() >= 12]
tipos_envio = {type(e).__name__ for e in envios}
costes_por_envio = {e.envio_id: float(e.calcular_coste()) for e in envios}

# D. Programación funcional
def aplicar_operacion(funcion, valor):            # recibe una función
    return funcion(valor)

def redondear_euros(importe):
    return round(importe, 2)

def crear_descuento(porcentaje):                  # devuelve una función (closure)
    def descontar(importe):
        return importe * (1 - porcentaje / 100)
    return descontar

descontar_10 = crear_descuento(10)
costes_con_descuento = {
    e.envio_id: aplicar_operacion(redondear_euros, aplicar_operacion(descontar_10, e.calcular_coste()))
    for e in envios
    if e.calcular_coste() >= 12
}
```

```
# E. Informe
print("\nEnvíos caros:", envios_caros)
print("\nTipos de envío:", sorted(tipos_envio))
print("\nCostes por envío:", costes_por_envio)
print("\nCostes con descuento:", costes_con_descuento)
print("\nComprobaciones:", aplicar_operacion(redondear_euros, 12.345), descontar_10(100))
```

🖥️ **Salida:**

```
E01: 7.0 €
E02: 21.0 €
E03: 12.0 €
E04: 13.0 €
E05: 17.0 €
E06: 6.0 €
E07: 5.0 €

Envíos caros: ['E02', 'E03', 'E04', 'E05']

Tipos de envío: ['EnvioInternacional', 'EnvioNacional', 'EnvioUrgente']

Costes por envío: {'E01': 7.0, 'E02': 21.0, 'E03': 12.0, 'E04': 13.0, 'E05': 17.0, 'E06': 6.0, 'E07': 5.0}

Costes con descuento: {'E02': 18.9, 'E03': 10.8, 'E04': 11.7, 'E05': 15.3}

Comprobaciones: 12.35 90.0
```

Ejercicio propuesto P2. Procesamiento de pedidos de una tienda online

Enunciado

> 📖 Una tienda online almacena los pedidos recibidos durante una jornada en una lista de diccionarios. El departamento de análisis necesita seleccionar los pedidos completados, calcular su importe con impuestos y obtener el total de facturación correspondiente. Desarrolla un programa en Python utilizando **funciones `lambda` y las funciones de orden superior `filter()`, `map()` y `reduce()`**.

Datos iniciales

```
pedidos = [
    {"pedido_id": "P001", "cliente": "Ana", "importe": 120.0, "estado": "completado"},
    {"pedido_id": "P002", "cliente": "Luis", "importe": 80.0, "estado": "pendiente"},
    {"pedido_id": "P003", "cliente": "Marta", "importe": 250.0, "estado": "completado"},
    {"pedido_id": "P004", "cliente": "Pedro", "importe": 60.0, "estado": "cancelado"},
    {"pedido_id": "P005", "cliente": "Lucía", "importe": 150.0, "estado": "completado"},
]
```

Actividades

39. **Filtrado con `filter()`**. Obtén una nueva lista que contenga únicamente los pedidos cuyo `estado` sea `"completado"`. Utiliza una función `lambda` como criterio de selección.
40. **Transformación con `map()`**. A partir de los pedidos completados, genera una nueva lista de diccionarios con los campos `pedido_id`, `cliente` e `importe_final`. El `importe_final` será el importe original más un **21 % de IVA**. Redondea el resultado a dos decimales. No modifiques los diccionarios originales.
41. **Acumulación con `reduce()`**. Calcula la suma de todos los `importe_final` de los pedidos completados. Usa un acumulador inicial de `0`.
42. **Función `lambda` independiente**. Define una función `lambda` que reciba un importe y devuelva `"Alto"` si es mayor o igual a 200 €, y `"Normal"` en caso contrario. Aplícala a los importes finales mediante `map()` para generar una lista de clasificaciones.
43. **Presentación**. Muestra los pedidos completados, los pedidos transformados, las clasificaciones y la facturación total. ##### Requisitos

- Utiliza `lambda` al menos en las operaciones de filtrado, transformación y clasificación.
- Utiliza obligatoriamente `filter()`, `map()` y `reduce()`.
- Importa `reduce` desde `functools`.
- Convierte a lista los resultados de `filter()` y `map()` cuando necesites mostrarlos o reutilizarlos.
- Mantén sin cambios la lista original `pedidos`. ##### Comprobaciones

Al ejecutar el programa, verifica que:

- Se seleccionan **3 pedidos completados**.
- Los importes finales son **145.20 €, 302.50 € y 181.50 €**, en el mismo orden que los pedidos originales.
- La facturación total con IVA es **629.20 €**.
- Las clasificaciones son **`["Normal", "Alto", "Normal"]`**.

✅ **Solución**

```
from functools import reduce

pedidos = [
    {"pedido_id": "P001", "cliente": "Ana", "importe": 120.0, "estado": "completado"},
    {"pedido_id": "P002", "cliente": "Luis", "importe": 80.0, "estado": "pendiente"},
    {"pedido_id": "P003", "cliente": "Marta", "importe": 250.0, "estado": "completado"},
    {"pedido_id": "P004", "cliente": "Pedro", "importe": 60.0, "estado": "cancelado"},
    {"pedido_id": "P005", "cliente": "Lucía", "importe": 150.0, "estado": "completado"},
]

# 1. filter + lambda
completados = list(filter(lambda p: p["estado"] == "completado", pedidos))

# 2. map + lambda → diccionarios NUEVOS (no se modifican los originales)
transformados = list(map(lambda p: {"pedido_id": p["pedido_id"],
                                    "cliente": p["cliente"],
                                    "importe_final": round(p["importe"] * 1.21, 2)},
                         completados))

# 3. reduce con acumulador inicial 0
facturacion = reduce(lambda acum, p: acum + p["importe_final"], transformados, 0)

# 4. lambda independiente + map
clasificar = lambda importe: "Alto" if importe >= 200 else "Normal"
clasificaciones = list(map(clasificar, [p["importe_final"] for p in transformados]))

# 5. Presentación
print("Pedidos completados:", [p["pedido_id"] for p in completados])
print("Pedidos transformados:")
for p in transformados:
    print("  ", p)
print("Clasificaciones:", clasificaciones)
print(f"Facturación total: {facturacion:.2f} €")
```

🖥️ **Salida:**

```
Pedidos completados: ['P001', 'P003', 'P005']
Pedidos transformados:
   {'pedido_id': 'P001', 'cliente': 'Ana', 'importe_final': 145.2}
   {'pedido_id': 'P003', 'cliente': 'Marta', 'importe_final': 302.5}
   {'pedido_id': 'P005', 'cliente': 'Lucía', 'importe_final': 181.5}
Clasificaciones: ['Normal', 'Alto', 'Normal']
Facturación total: 629.20 €
```

Ejercicio propuesto P3. Análisis de ejecuciones de pipelines de datos

> 📖 ##### Enunciado

Un equipo de ingeniería de datos registra la duración, el volumen procesado y el resultado de las ejecuciones de varios pipelines ETL. El objetivo es elaborar un pequeño informe con las ejecuciones correctas, los datos procesados y el consumo de tiempo de ejecución.

Desarrolla el procesamiento utilizando **`lambda`**, **`filter()`**, **`map()`**, **`reduce()`** y **`sorted()`**, evitando modificar los registros originales.

Datos iniciales

```
ejecuciones = [
    {"pipeline": "ventas", "duracion_min": 18, "registros": 12000, "estado": "ok"},
    {"pipeline": "clientes", "duracion_min": 32, "registros": 8000, "estado": "ok"},
    {"pipeline": "inventario", "duracion_min": 12, "registros": 0, "estado": "error"},
    {"pipeline": "facturas", "duracion_min": 45, "registros": 15000, "estado": "ok"},
    {"pipeline": "devoluciones", "duracion_min": 26, "registros": 4000, "estado": "ok"},
    {"pipeline": "proveedores", "duracion_min": 15, "registros": 0, "estado": "error"},
]
```

Actividades

44. **Filtrado de ejecuciones exitosas**. Usa `filter()` y una función `lambda` para obtener únicamente los registros cuyo `estado` sea `"ok"`.
45. **Detección de ejecuciones lentas**. A partir de las ejecuciones exitosas, crea otra lista con aquellas cuya duración sea **superior a 25 minutos**.
46. **Transformación de datos**. Utiliza `map()` y `lambda` para crear una nueva lista de diccionarios con estos campos:

- `pipeline`: nombre del pipeline.
- `duracion_min`: duración original.
- `duracion_seg`: duración convertida a segundos.
- `registros`: número de registros procesados.

47. **Total de registros procesados**. Utiliza `reduce()` con valor inicial `0` para sumar los registros de todas las ejecuciones exitosas.
48. **Tiempo total de procesamiento**. Utiliza otra llamada a `reduce()` para sumar la duración, en minutos, de las ejecuciones exitosas.
49. **Ordenación**. Ordena los pipelines exitosos de **mayor a menor duración** mediante `sorted()` y una función `lambda` aplicada al parámetro `key`.
50. **Informe final**. Muestra el número de ejecuciones exitosas, los pipelines lentos, la lista transformada, el total de registros, el tiempo acumulado y los nombres de los pipelines ordenados por duración. ##### Requisitos

- Utiliza **al menos dos llamadas a `filter()`**, una a `map()`, dos a `reduce()` y una a `sorted()`.
- Emplea funciones `lambda` en las operaciones anteriores.
- Usa `list()` para materializar los resultados que corresponda.
- No utilices bibliotecas externas como `pandas` o `numpy`.
- No modifiques la lista original `ejecuciones`. ##### Comprobaciones

Al ejecutar el programa, verifica que:

- Existen **4 ejecuciones exitosas**.
- Los pipelines lentos son **`clientes`**, **`facturas`** y **`devoluciones`**.
- Se procesaron **39 000 registros** en las ejecuciones exitosas.
- El tiempo total de estas ejecuciones es **121 minutos**.
- El orden de mayor a menor duración es: **`facturas`**, **`clientes`**, **`devoluciones`**, **`ventas`**.

✅ Solución

```
from functools import reduce

ejecuciones = [
    {"pipeline": "ventas", "duracion_min": 18, "registros": 12000, "estado": "ok"},
    {"pipeline": "clientes", "duracion_min": 32, "registros": 8000, "estado": "ok"},
    {"pipeline": "inventario", "duracion_min": 12, "registros": 0, "estado": "error"},
    {"pipeline": "facturas", "duracion_min": 45, "registros": 15000, "estado": "ok"},
    {"pipeline": "devoluciones", "duracion_min": 26, "registros": 4000, "estado": "ok"},
    {"pipeline": "proveedores", "duracion_min": 15, "registros": 0, "estado": "error"},
```

```
]

exitosas = list(filter(lambda e: e["estado"] == "ok", ejecuciones))          # filter 1
lentas = list(filter(lambda e: e["duracion_min"] > 25, exitosas))            # filter 2
transformadas = list(map(lambda e: {"pipeline": e["pipeline"],
                                    "duracion_min": e["duracion_min"],
                                    "duracion_seg": e["duracion_min"] * 60,
                                    "registros": e["registros"]}, exitosas))
total_registros = reduce(lambda acum, e: acum + e["registros"], exitosas, 0)
total_minutos = reduce(lambda acum, e: acum + e["duracion_min"], exitosas, 0)
ordenadas = sorted(exitosas, key=lambda e: e["duracion_min"], reverse=True)

print("Ejecuciones exitosas:", len(exitosas))
print("Pipelines lentos:", [e["pipeline"] for e in lentas])
print("Transformadas:")
for t in transformadas:
    print("  ", t)
print("Total registros:", total_registros)
print("Tiempo total:", total_minutos, "min")
print("Orden por duración:", [e["pipeline"] for e in ordenadas])
```

🖥️ **Salida:**

```
Ejecuciones exitosas: 4
Pipelines lentos: ['clientes', 'facturas', 'devoluciones']
Transformadas:
   {'pipeline': 'ventas', 'duracion_min': 18, 'duracion_seg': 1080, 'registros': 12000}
   {'pipeline': 'clientes', 'duracion_min': 32, 'duracion_seg': 1920, 'registros': 8000}
   {'pipeline': 'facturas', 'duracion_min': 45, 'duracion_seg': 2700, 'registros': 15000}
   {'pipeline': 'devoluciones', 'duracion_min': 26, 'duracion_seg': 1560, 'registros': 4000}
Total registros: 39000
Tiempo total: 121 min
Orden por duración: ['facturas', 'clientes', 'devoluciones', 'ventas']
```

Ejercicios extra de la clase 20 (herencia múltiple)

Ejercicio de comprobación

¿Qué sucede si cambiamos el orden de herencia?

```
class SensorInteligente(Conectividad, Sensor):
    def ejecutar(self):
        print("1. SensorInteligente: iniciar operación")
        super().ejecutar()
```

Pregunta: ¿Cuál sería ahora el orden de ejecución?

Respuesta:

```
SensorInteligente → Conectividad → Sensor → Dispositivo → object
```

La salida sería:

```
1. SensorInteligente: iniciar operación
3. Conectividad: enviar datos a la nube
2. Sensor: obtener mediciones
4. Dispositivo: funcionamiento básico
```

> **Nota:** Los números de las frases son etiquetas escritas en los `print()` para el primer orden de herencia. Al invertir las clases base, se conservan esas etiquetas, aunque el orden real cambia.

> 📖 En herencia múltiple, `super()` **permite que varias clases colaboren ejecutando sus métodos en el orden establecido por el MRO**. La idea fundamental es que `super()` **no llama necesariamente al padre directo; continúa con el siguiente método del MRO**. Para que este mecanismo funcione de forma

> cooperativa, los métodos de las clases participantes deben ser compatibles y utilizar `super()` de manera consistente.

> **Ejercicio 3**. *Ejemplos resueltos para revisar, corregir y mejorar añadiendo nuevos atributos, métodos y herencias. #### 3.1. Polimorfismo: El “Botón Mágico”*

Imagina que tienes un botón mágico que, al presionarlo, hace que diferentes objetos hagan su “sonido” característico. Un perro ladra, un gato maúlla y un pato cuaquea. No tienes que saber de antemano qué animal es, ¡el botón simplemente le dice “haz tu sonido” y cada uno lo hace a su manera!

En programación, el **polimorfismo** significa que objetos de diferentes tipos pueden responder a la misma “orden” (o método) de maneras diferentes, pero adecuadas a su propio tipo.

Concepto Clave: Tener un método con el mismo nombre en diferentes clases, y cada clase lo implementa a su manera.

```python
# Nuestro plano general para cualquier animal
class Animal:
    def hacer_sonido(self):
        # Este es un sonido genérico, cada animal lo cambiará
        pass

# Nuestro plano para perros
class Perro(Animal): # Perro es un tipo de Animal
    def hacer_sonido(self):
        return "¡Guau! ¡Guau!"

# Nuestro plano para gatos
class Gato(Animal): # Gato es un tipo de Animal
    def hacer_sonido(self):
        return "¡Miau! ¡Miau!"

# Nuestro plano para patos
class Pato(Animal): # Pato es un tipo de Animal
    def hacer_sonido(self):
        return "¡Cuac! ¡Cuac!"

# --- Demostración del Polimorfismo ---

# Creamos algunos animales (instancias)
mi_perro = Perro()
mi_gato = Gato()
mi_pato = Pato()

# Los ponemos todos en una lista, ¡sin importar el tipo exacto!
animales_en_el_zoo = [mi_perro, mi_gato, mi_pato]

print("--- El botón mágico para hacer sonidos ---")

# Ahora, recorremos la lista y le decimos a cada animal: "¡Haz tu sonido!"
# No necesitamos saber si es un perro, gato o pato.
# Cada uno sabe cómo responder a 'hacer_sonido'.
for animal in animales_en_el_zoo:
    print(f"El animal hace: {animal.hacer_sonido()}")

print("\n--- Salida Esperada ---")
print("El animal hace: ¡Guau! ¡Guau!")
print("El animal hace: ¡Miau! ¡Miau!")
print("El animal hace: ¡Cuac! ¡Cuac!")
```

> **¿Por qué es útil el Polimorfismo?** - **Código más limpio:** No tienes que escribir `if es_perro: haz_guau() elif es_gato: haz_miau()`. Simplemente dices `animal.hacer_sonido()`. - **Fácil de expandir:** Si mañana añades una clase `Vaca` con su `hacer_sonido()`, tu “botón mágico” (el bucle `for`) seguirá funcionando sin cambios.

✅ **Versión revisada y mejorada**

```
# Ejercicio 3.1 mejorado: método base que avisa si no se implementa, atributo nombre,
# un método común heredado y una clase nueva (Vaca) que no obliga a tocar el bucle.
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

    def hacer_sonido(self):
        raise NotImplementedError("Cada animal debe implementar hacer_sonido()")

    def presentarse(self):                      # común: usa el método polimórfico
        return f"{self.nombre} ({type(self).__name__}) dice {self.hacer_sonido()}"

class Perro(Animal):
    def hacer_sonido(self):
        return "¡Guau! ¡Guau!"

class Gato(Animal):
    def hacer_sonido(self):
        return "¡Miau! ¡Miau!"

class Pato(Animal):
    def hacer_sonido(self):
        return "¡Cuac! ¡Cuac!"

class Vaca(Animal):                             # ampliación sin tocar el bucle
    def hacer_sonido(self):
        return "¡Muuu!"

for animal in [Perro("Toby"), Gato("Misi"), Pato("Donald"), Vaca("Lola")]:
    print(animal.presentarse())
```

🖥️ **Salida:**

```
Toby (Perro) dice ¡Guau! ¡Guau!
Misi (Gato) dice ¡Miau! ¡Miau!
Donald (Pato) dice ¡Cuac! ¡Cuac!
Lola (Vaca) dice ¡Muuu!
```

3.2. Herencia Múltiple: El “Súper Héroe con Doble Poder”

Imagina que quieres crear un nuevo súper héroe que tiene poderes de dos súper héroes diferentes. Por ejemplo, quieres un héroe que pueda volar (como Superman) y que pueda lanzar telarañas (como Spiderman).

La **herencia múltiple** en Python permite que una clase (tu nuevo súper héroe) herede características (atributos y métodos) de *más de una* clase “padre” (Superman y Spiderman).

```
# Clase Padre 1: Héroe Volador
class HeroeVolador:
    def __init__(self):
        print("Poder: Capacidad de Volar!")

    def volar(self):
        return "Estoy volando por el cielo."

# Clase Padre 2: Héroe Lanzador de Telarañas
class HeroeTelarana:
    def __init__(self):
        print("Poder: Capacidad de Lanzar Telarañas!")

    def lanzar_telarana(self):
        return "¡Zas! Telaraña lanzada."

# Clase Hija: Nuestro Súper Héroe con doble poder
# Hereda de HeroeVolador Y HeroeTelarana
class SúperHeroeCombinado(HeroeVolador, HeroeTelarana):
    def __init__(self, nombre):
        # Llamar a los constructores de las clases padre
```

```
        # En herencia múltiple, se llama el constructor de la primera clase listada
        super().__init__() # Llama al __init__ de HeroeVolador en este caso
        HeroeTelarana.__init__(self) # Debemos llamar explícitamente al otro constructor si tiene uno.

        self.nombre = nombre
        print(f"¡{self.nombre} ha nacido con poderes combinados!")

    def presentarse(self):
        return f"Hola, soy {self.nombre} y tengo poderes de volar y lanzar telarañas."

# --- Demostración de la Herencia Múltiple ---

# Creamos una instancia de nuestro Súper Héroe Combinado
mi_super_heroe = SúperHeroeCombinado("Capitán Combinado")

print(f"\n{mi_super_heroe.presentarse()}")
print(f"¡Mira, puede! {mi_super_heroe.volar()}")
print(f"¡Y también puede! {mi_super_heroe.lanzar_telarana()}")

print("\n--- Salida Esperada ---")
print("Poder: Capacidad de Volar!")
print("Poder: Capacidad de Lanzar Telarañas!")
print("¡Capitán Combinado ha nacido con poderes combinados!")
print("Hola, soy Capitán Combinado y tengo poderes de volar y lanzar telarañas.")
print("¡Mira, puede! Estoy volando por el cielo.")
print("¡Y también puede! ¡Zas! Telaraña lanzada.")
```

✅ Versión corregida (herencia cooperativa)

```
# Ejercicio 3.2 corregido: en vez de llamar a HeroeTelarana.__init__ a mano, herencia
# múltiple COOPERATIVA: todas las clases llaman a super().__init__(**kwargs).
class HeroeVolador:
    def __init__(self, **kwargs):
        print("Poder: Capacidad de Volar!")
        super().__init__(**kwargs)

    def volar(self):
        return "Estoy volando por el cielo."

class HeroeTelarana:
    def __init__(self, **kwargs):
        print("Poder: Capacidad de Lanzar Telarañas!")
        super().__init__(**kwargs)

    def lanzar_telarana(self):
        return "¡Zas! Telaraña lanzada."

class SuperHeroeCombinado(HeroeVolador, HeroeTelarana):
    def __init__(self, nombre):
        super().__init__()          # recorre el MRO: HeroeVolador → HeroeTelarana → object
        self.nombre = nombre
        print(f"¡{self.nombre} ha nacido con poderes combinados!")

    def presentarse(self):
        return f"Hola, soy {self.nombre} y tengo poderes de volar y lanzar telarañas."

heroe = SuperHeroeCombinado("Capitán Combinado")
print(f"\n{heroe.presentarse()}")
print(f"¡Mira, puede! {heroe.volar()}")
print(f"¡Y también puede! {heroe.lanzar_telarana()}")
print([c.__name__ for c in SuperHeroeCombinado.__mro__])
```

🖥️ Salida:

```
Poder: Capacidad de Volar!
Poder: Capacidad de Lanzar Telarañas!
¡Capitán Combinado ha nacido con poderes combinados!

Hola, soy Capitán Combinado y tengo poderes de volar y lanzar telarañas.
¡Mira, puede! Estoy volando por el cielo.
```

```
¡Y también puede! ¡Zas! Telaraña lanzada.
['SuperHeroeCombinado', 'HeroeVolador', 'HeroeTelarana', 'object']
```

Ejercicio P5: Sistema de vehículos inteligentes con herencia múltiple

> 📖 ### 📌 Enunciado

Una empresa de logística está desarrollando una plataforma para gestionar una flota de vehículos inteligentes. Estos vehículos combinan diferentes tecnologías, como sistemas de geolocalización GPS, conectividad a Internet y sensores de diagnóstico.

La empresa necesita desarrollar una aplicación utilizando **Programación Orientada a Objetos (POO)** que permita representar estos vehículos y sus funcionalidades.

Debes implementar una solución utilizando **herencia múltiple de atributos y métodos**, incorporando también el uso de `super()`.

Parte A. Crear las clases base

Debes crear las siguientes cuatro clases independientes.

Clase	Atributos	Métodos
`Vehiculo`	`matricula`, `modelo`	`mostrar_informacion()`
`SistemaGPS`	`latitud`, `longitud`	`mostrar_ubicacion()`
`SistemaConectividad`	`direccion_ip`, `conectado`	`mostrar_conexion()`
`SistemaDiagnostico`	`nivel_bateria`, `temperatura_motor`	`mostrar_diagnostico()`

Cada clase debe implementar su propio constructor `__init__()` para inicializar los atributos correspondientes. Los métodos deben mostrar en pantalla la información de los atributos de cada clase.

Parte B. Crear una clase mediante herencia múltiple

Crea una clase llamada `VehiculoInteligente` que herede simultáneamente de las cuatro clases anteriores. La declaración debe ser:

```python
class VehiculoInteligente(
    Vehiculo,
    SistemaGPS,
    SistemaConectividad,
    SistemaDiagnostico
):
    pass
```

Sustituye `pass` por la implementación necesaria.

La nueva clase debe incorporar un atributo propio:

- `identificador_flota`: identificador único del vehículo dentro de la flota. Además, debe implementar el método `mostrar_estado_completo()`, que permita visualizar toda la información del vehículo utilizando los métodos heredados.

Parte C. Aplicar `super()` en herencia múltiple

Modifica los constructores de las cuatro clases base para que utilicen `super().__init__()` y permitan inicializar correctamente todos los atributos cuando se cree un objeto `VehiculoInteligente`.

Para conseguirlo, utiliza **inicialización cooperativa**: cada constructor recibe sus propios atributos y transmite los restantes al siguiente constructor mediante `super()`.

Puedes utilizar `**kwargs` para pasar los argumentos que corresponden a las demás clases.

Ejemplo orientativo:

```
class Vehiculo:
    def __init__(self, matricula, modelo, **kwargs):
        self.matricula = matricula
        self.modelo = modelo
        super().__init__(**kwargs)
```

Importante: En este ejercicio no debes llamar directamente a los constructores de las clases base utilizando expresiones como `Vehiculo.__init__(self, ...)`. Debes utilizar `super()` para respetar el orden de resolución de métodos de Python.

Parte D. Crear objetos

Crea dos objetos de la clase `VehiculoInteligente` con los siguientes datos:

Atributo	Vehículo 1	Vehículo 2
`identificador_flota`	FLT-001	FLT-002
`matricula`	1234-ABC	5678-DEF
`modelo`	Tesla Model 3	Mercedes eSprinter
`latitud`	40.4168	41.3874
`longitud`	-3.7038	2.1686
`direccion_ip`	192.168.1.10	192.168.1.11
`conectado`	True	False
`nivel_bateria`	85	42
`temperatura_motor`	45	65

Parte E. Utilizar los métodos heredados

Para cada vehículo, ejecuta los siguientes métodos:

```
vehiculo1.mostrar_informacion()
vehiculo1.mostrar_ubicacion()
vehiculo1.mostrar_conexion()
vehiculo1.mostrar_diagnostico()
vehiculo1.mostrar_estado_completo()
```

Comprueba que el objeto puede acceder a los atributos y métodos definidos en todas las clases base.

Salida esperada

Al ejecutar `vehiculo1.mostrar_estado_completo()`, el resultado debe ser similar al siguiente:

```
====================================
    ESTADO DEL VEHÍCULO INTELIGENTE
====================================

Identificador: FLT-001

--- INFORMACIÓN DEL VEHÍCULO ---
Matrícula: 1234-ABC
Modelo: Tesla Model 3

--- SISTEMA GPS ---
Latitud: 40.4168
Longitud: -3.7038

--- CONECTIVIDAD ---
Dirección IP: 192.168.1.10
Estado: Conectado

--- DIAGNÓSTICO ---
```

```
Nivel de batería: 85%
Temperatura del motor: 45 °C
```

Parte F. Investigar el orden de resolución de métodos (MRO)

Python utiliza un mecanismo llamado *Method Resolution Order* (MRO) para determinar el orden en el que busca y ejecuta métodos cuando existe herencia múltiple.

Ejecuta:

```
print(VehiculoInteligente.__mro__)
```

Después, realiza lo siguiente:

29. Identifica las clases que aparecen en el resultado.
30. Comprueba si el orden coincide con el utilizado en la declaración de `VehiculoInteligente`.
31. Explica qué papel desempeña `object` en la cadena de herencia.
32. Explica por qué es importante utilizar `super()` cuando varias clases participan en la inicialización de un objeto. ### Parte G. Ampliación: Sobrescritura de métodos con `super()`

Añade un método llamado `mostrar_informacion()` a la clase `VehiculoInteligente`, sobrescribiendo el método heredado de `Vehiculo`.

El nuevo método debe:

33. Ejecutar primero el método original `mostrar_informacion()` de la clase base utilizando `super()`.
34. Mostrar después el identificador de la flota. La salida esperada será:

```
Matrícula: 1234-ABC
Modelo: Tesla Model 3
Identificador de flota: FLT-001
```

Comprueba qué sucede cuando ejecutas:

```
vehiculo1.mostrar_informacion()
```

Restricciones del ejercicio

- Debes utilizar al menos **cuatro clases base** y una clase hija con herencia múltiple.
- Todas las clases base deben definir atributos y métodos propios.
- Debes utilizar `super()` para inicializar los atributos mediante herencia múltiple cooperativa.
- Debes aplicar sobrescritura de métodos en la parte G.
- Debes comprobar el orden de resolución de métodos mediante `__mro__`.
- No debes utilizar librerías externas.
- No es necesario conectarse a vehículos reales ni implementar sistemas GPS o comunicaciones reales: todo el funcionamiento será simulado.

✅ Solución

```python
class Vehiculo:
    def __init__(self, matricula, modelo, **kwargs):
        self.matricula = matricula
        self.modelo = modelo
        super().__init__(**kwargs)        # pasa el resto a la siguiente clase del MRO

    def mostrar_informacion(self):
        print(f"Matrícula: {self.matricula}")
        print(f"Modelo: {self.modelo}")

class SistemaGPS:
    def __init__(self, latitud, longitud, **kwargs):
        self.latitud = latitud
        self.longitud = longitud
        super().__init__(**kwargs)
```

```python
    def mostrar_ubicacion(self):
        print(f"Latitud: {self.latitud}")
        print(f"Longitud: {self.longitud}")

class SistemaConectividad:
    def __init__(self, direccion_ip, conectado, **kwargs):
        self.direccion_ip = direccion_ip
        self.conectado = conectado
        super().__init__(**kwargs)

    def mostrar_conexion(self):
        print(f"Dirección IP: {self.direccion_ip}")
        print(f"Estado: {'Conectado' if self.conectado else 'Desconectado'}")

class SistemaDiagnostico:
    def __init__(self, nivel_bateria, temperatura_motor, **kwargs):
        self.nivel_bateria = nivel_bateria
        self.temperatura_motor = temperatura_motor
        super().__init__(**kwargs)        # aquí kwargs ya está vacío → object.__init__()

    def mostrar_diagnostico(self):
        print(f"Nivel de batería: {self.nivel_bateria}%")
        print(f"Temperatura del motor: {self.temperatura_motor} °C")

class VehiculoInteligente(Vehiculo, SistemaGPS, SistemaConectividad, SistemaDiagnostico):
    def __init__(self, identificador_flota, **kwargs):
        self.identificador_flota = identificador_flota
        super().__init__(**kwargs)        # inicialización cooperativa (sin X.__init__(self, ...))

    def mostrar_estado_completo(self):
        print("=" * 36)
        print("    ESTADO DEL VEHÍCULO INTELIGENTE")
        print("=" * 36)
        print(f"\nIdentificador: {self.identificador_flota}")
        print("\n--- INFORMACIÓN DEL VEHÍCULO ---")
        Vehiculo.mostrar_informacion(self)   # la versión base (sin la línea de flota de la Parte G)
        print("\n--- SISTEMA GPS ---")
        self.mostrar_ubicacion()
        print("\n--- CONECTIVIDAD ---")
        self.mostrar_conexion()
        print("\n--- DIAGNÓSTICO ---")
        self.mostrar_diagnostico()

    # Parte G: sobrescritura reutilizando el método del padre
    def mostrar_informacion(self):
        super().mostrar_informacion()
        print(f"Identificador de flota: {self.identificador_flota}")

vehiculo1 = VehiculoInteligente(
    identificador_flota="FLT-001", matricula="1234-ABC", modelo="Tesla Model 3",
    latitud=40.4168, longitud=-3.7038, direccion_ip="192.168.1.10", conectado=True,
    nivel_bateria=85, temperatura_motor=45)
vehiculo2 = VehiculoInteligente(
    identificador_flota="FLT-002", matricula="5678-DEF", modelo="Mercedes eSprinter",
    latitud=41.3874, longitud=2.1686, direccion_ip="192.168.1.11", conectado=False,
    nivel_bateria=42, temperatura_motor=65)

vehiculo1.mostrar_estado_completo()
print()
vehiculo2.mostrar_conexion()
print()
vehiculo1.mostrar_informacion()          # Parte G
print()
print([c.__name__ for c in VehiculoInteligente.__mro__])
# Parte F: el MRO coincide con el orden de la declaración y termina en object, la raíz de
# todas las clases (su __init__ es el último eslabón de la cadena). super() garantiza que
# cada constructor se ejecuta UNA vez y en ese orden, pasando los **kwargs restantes.
```

🖥️ **Salida:**

```
==================================
    ESTADO DEL VEHÍCULO INTELIGENTE
==================================

Identificador: FLT-001

--- INFORMACIÓN DEL VEHÍCULO ---
Matrícula: 1234-ABC
Modelo: Tesla Model 3

--- SISTEMA GPS ---
Latitud: 40.4168
Longitud: -3.7038

--- CONECTIVIDAD ---
Dirección IP: 192.168.1.10
Estado: Conectado

--- DIAGNÓSTICO ---
Nivel de batería: 85%
Temperatura del motor: 45 °C

Dirección IP: 192.168.1.11
Estado: Desconectado

Matrícula: 1234-ABC
Modelo: Tesla Model 3
Identificador de flota: FLT-001

['VehiculoInteligente', 'Vehiculo', 'SistemaGPS', 'SistemaConectividad', 'SistemaDiagnostico', 'object']
```

Ejercicio P6: Refactorización de un sistema de incidencias informáticas

> 📖 ### 📌 Enunciado

El departamento de informática de una empresa utiliza un programa para gestionar incidencias de sus empleados. Las incidencias pueden ser de **hardware**, **software** o **redes**, y cada tipo necesita una respuesta diferente.

El programa actual funciona, pero su estructura es especialmente difícil de mantener: guarda la información en **listas paralelas**, repite bloques enteros de instrucciones, mezcla la presentación con la lógica de negocio y utiliza condiciones anidadas innecesarias.

Tu tarea consiste en refactorizarlo mediante Programación Orientada a Objetos (POO), aplicando herencia múltiple, polimorfismo y `super()`. El programa refactorizado debe producir la misma información que el original.

> **Importante:** No debes resolver el ejercicio creando únicamente una clase grande. Debes distribuir los atributos y métodos entre las clases indicadas.

Parte A. Analizar el código original

Archivo: `incidencias.py`

```python
# Programa original: funciona, pero está muy mal organizado.
# NO debes corregirlo todavía: primero ejecútalo y analízalo.

numeros = [101, 102, 103, 104]
empleados = ["Ana", "Luis", "Marta", "Pedro"]
tipos = ["hardware", "software", "redes", "software"]
prioridades = ["alta", "media", "alta", "baja"]
descripciones = [
    "El ordenador no enciende",
    "La aplicación se bloquea",
    "No hay conexión a Internet",
    "No puedo actualizar el programa"
```

```
]

posicion = 0

while posicion < len(numeros):
    numero_actual = numeros[posicion]
    nombre_actual = empleados[posicion]
    problema_actual = descripciones[posicion]
    tipo_actual = tipos[posicion]
    prioridad_actual = prioridades[posicion]

    # Datos comunes, repetidos para cada tipo de incidencia.
    if tipo_actual == "hardware":
        print("-----------------------------")
        print("Incidencia:", numero_actual)
        print("Empleado:", nombre_actual)
        print("Descripción:", problema_actual)

        if prioridad_actual == "alta":
            texto_prioridad = "Prioridad: ALTA - atención urgente"
        else:
            if prioridad_actual == "media":
                texto_prioridad = "Prioridad: MEDIA - atención normal"
            else:
                texto_prioridad = "Prioridad: BAJA - atención no urgente"
        print(texto_prioridad)
        print("Solución: revisar componentes físicos")
        print("Incidencia registrada")

    else:
        if tipo_actual == "software":
            print("-----------------------------")
            print("Incidencia:", numero_actual)
            print("Empleado:", nombre_actual)
            print("Descripción:", problema_actual)

            # La misma lógica de prioridad vuelve a escribirse.
            if prioridad_actual == "alta":
                print("Prioridad: ALTA - atención urgente")
            elif prioridad_actual == "media":
                print("Prioridad: MEDIA - atención normal")
            else:
                print("Prioridad: BAJA - atención no urgente")

            print("Solución: comprobar y actualizar la aplicación")
            print("Incidencia registrada")

        else:
            if tipo_actual == "redes":
                print("-----------------------------")
                print("Incidencia:", numero_actual)
                print("Empleado:", nombre_actual)
                print("Descripción:", problema_actual)

                # Una tercera copia del mismo procesamiento.
                if prioridad_actual == "alta":
                    print("Prioridad: ALTA - atención urgente")
                elif prioridad_actual == "media":
                    print("Prioridad: MEDIA - atención normal")
                else:
                    print("Prioridad: BAJA - atención no urgente")

                print("Solución: verificar la conexión de red")
                print("Incidencia registrada")

    posicion = posicion + 1
```

Antes de modificarlo, **ejecuta el programa y verifica su salida**. Después, identifica: ¿qué instrucciones son comunes a todas las incidencias?, ¿qué bloques se han copiado tres veces?, ¿qué ocurre si las listas paralelas dejan de tener la misma longitud?, ¿qué instrucciones dependen de la prioridad? y ¿cuáles dependen del tipo de incidencia?

> **Importante:** No debes añadir nuevas funcionalidades ni cambiar los mensajes. El reto consiste en **conservar el comportamiento y mejorar la estructura**.

Parte B. Crear dos clases base

Crea estas **dos clases independientes**:

Clase	Atributos	Métodos
`Incidencia`	`numero`, `empleado`, `descripcion`	`mostrar_informacion()`, `resolver()`
`GestionPrioridad`	`prioridad`	`mostrar_prioridad()`

La clase `Incidencia` debe inicializar sus tres atributos. Su método `mostrar_informacion()` debe mostrar el número, el empleado y la descripción. El método `resolver()` debe mostrar inicialmente `Solución: revisión general de la incidencia`.

La clase `GestionPrioridad` debe inicializar `prioridad` y su método `mostrar_prioridad()` debe mostrar el mensaje correspondiente a `alta`, `media` o `baja`, tal como aparece en el programa original.

Parte C. Aplicar herencia múltiple

Crea la clase `TicketSoporte`, que herede de ambas clases base:

```
class TicketSoporte(Incidencia, GestionPrioridad):
    pass
```

Sustituye `pass` e implementa un constructor que permita inicializar los cuatro atributos heredados. Para mantener el ejercicio en un nivel básico, puedes llamar explícitamente a los constructores de las dos clases base:

```
Incidencia.__init__(self, numero, empleado, descripcion)
GestionPrioridad.__init__(self, prioridad)
```

Añade un método `mostrar_detalles()` que ejecute `mostrar_informacion()` y `mostrar_prioridad()` para mostrar los datos comunes de cada ticket.

Parte D. Crear tres clases especializadas

Crea las siguientes clases, todas hijas de `TicketSoporte`:

- `TicketHardware`
- `TicketSoftware`
- `TicketRedes` Cada clase debe **sobrescribir** el método `resolver()` con el mensaje que le corresponda:

Clase	Mensaje de `resolver()`
`TicketHardware`	`Solución: revisar componentes físicos`
`TicketSoftware`	`Solución: comprobar y actualizar la aplicación`
`TicketRedes`	`Solución: verificar la conexión de red`

No repitas los atributos ni los métodos comunes en estas clases.

Parte E. Reutilizar métodos con `super()`

En cada una de las tres clases especializadas, sobrescribe `mostrar_detalles()` para que:

35. Ejecute el método heredado `mostrar_detalles()` mediante `super()`.
36. Ejecute `self.resolver()`.
37. Muestre `Incidencia registrada`. **Pista:** el comienzo del método será:

```
def mostrar_detalles(self):
    super().mostrar_detalles()
```

```
    # Completa aquí las instrucciones restantes
```

En esta parte, `super()` se emplea para **reutilizar el método heredado**; no es necesario usarlo para coordinar los constructores de las dos clases base.

Parte F. Aplicar polimorfismo

Crea una función llamada `atender_incidencia(ticket)` que reciba un objeto de cualquiera de las tres clases especializadas y ejecute:

```
ticket.mostrar_detalles()
```

Condición: la función no debe utilizar `if`, `elif`, `type()` ni `isinstance()` para identificar la clase del ticket.

Crea cuatro objetos correspondientes a las cuatro incidencias del programa original, guárdalos en una lista y recórrela con un `for` para llamar a `atender_incidencia()`.

Salida esperada

```
Incidencia: 101
Empleado: Ana
Descripción: El ordenador no enciende
Prioridad: ALTA - atención urgente
Solución: revisar componentes físicos
Incidencia registrada
------------------------------
Incidencia: 102
Empleado: Luis
Descripción: La aplicación se bloquea
Prioridad: MEDIA - atención normal
Solución: comprobar y actualizar la aplicación
Incidencia registrada
------------------------------
Incidencia: 103
Empleado: Marta
Descripción: No hay conexión a Internet
Prioridad: ALTA - atención urgente
Solución: verificar la conexión de red
Incidencia registrada
------------------------------
Incidencia: 104
Empleado: Pedro
Descripción: No puedo actualizar el programa
Prioridad: BAJA - atención no urgente
Solución: comprobar y actualizar la aplicación
Incidencia registrada
```

> Puedes mostrar la línea separadora antes de cada ticket, como en el código original. Lo importante es conservar los mensajes y el orden de la información.

Parte G. Ampliación

La empresa empieza a recibir incidencias de **seguridad informática**.

Crea `TicketSeguridad` como nueva subclase de `TicketSoporte`. Debe sobrescribir `resolver()` para mostrar:

```
Solución: comprobar accesos y cambiar credenciales
```

Crea un ticket de seguridad con estos datos:

- Número: `105`
- Empleado: `Sara`
- Descripción: `Se ha detectado un acceso sospechoso`
- Prioridad: `alta` Añade el objeto a la lista y comprueba que puede atenderse **sin modificar `atender_incidencia()`**.

Restricciones

- Utiliza **dos clases base**, una clase que aplique **herencia múltiple** y al menos **tres clases especializadas**.
- Utiliza atributos, constructores y métodos heredados.
- Aplica sobrescritura de métodos y `super()`.
- Aplica polimorfismo en la función que atiende los tickets.
- No utilices condiciones para distinguir clases de tickets durante su atención.
- No utilices librerías externas ni bases de datos.
- Simula la atención mediante mensajes `print()`; no es necesario resolver incidencias reales.

Análisis del código original (Parte A): son comunes las 3 líneas de datos (número, empleado, descripción), el bloque de prioridad y "Incidencia registrada"; esos bloques están copiados 3 veces. Si las listas paralelas tienen distinta longitud → `IndexError` o datos mezclados entre empleados. La prioridad depende de `prioridad`; la solución depende del tipo.

✅ Solución

```
class Incidencia:
    def __init__(self, numero, empleado, descripcion):
        self.numero = numero
        self.empleado = empleado
        self.descripcion = descripcion

    def mostrar_informacion(self):
        print("Incidencia:", self.numero)
        print("Empleado:", self.empleado)
        print("Descripción:", self.descripcion)

    def resolver(self):
        print("Solución: revisión general de la incidencia")

class GestionPrioridad:
    MENSAJES = {                          # diccionario en vez de if/else anidados
        "alta": "Prioridad: ALTA - atención urgente",
        "media": "Prioridad: MEDIA - atención normal",
        "baja": "Prioridad: BAJA - atención no urgente",
    }

    def __init__(self, prioridad):
        self.prioridad = prioridad

    def mostrar_prioridad(self):
        print(GestionPrioridad.MENSAJES[self.prioridad])

class TicketSoporte(Incidencia, GestionPrioridad):
    def __init__(self, numero, empleado, descripcion, prioridad):
        Incidencia.__init__(self, numero, empleado, descripcion)
        GestionPrioridad.__init__(self, prioridad)

    def mostrar_detalles(self):
        self.mostrar_informacion()
        self.mostrar_prioridad()

class TicketHardware(TicketSoporte):
    def resolver(self):
        print("Solución: revisar componentes físicos")

    def mostrar_detalles(self):
        super().mostrar_detalles()
        self.resolver()
        print("Incidencia registrada")

class TicketSoftware(TicketSoporte):
    def resolver(self):
        print("Solución: comprobar y actualizar la aplicación")

    def mostrar_detalles(self):
        super().mostrar_detalles()
        self.resolver()
```

```
        print("Incidencia registrada")

class TicketRedes(TicketSoporte):
    def resolver(self):
        print("Solución: verificar la conexión de red")

    def mostrar_detalles(self):
        super().mostrar_detalles()
        self.resolver()
        print("Incidencia registrada")

def atender_incidencia(ticket):           # polimorfismo, sin if/type/isinstance
    print("-----------------------------")
    ticket.mostrar_detalles()

tickets = [
    TicketHardware(101, "Ana", "El ordenador no enciende", "alta"),
    TicketSoftware(102, "Luis", "La aplicación se bloquea", "media"),
    TicketRedes(103, "Marta", "No hay conexión a Internet", "alta"),
    TicketSoftware(104, "Pedro", "No puedo actualizar el programa", "baja"),
]

# Parte G: nuevo tipo sin tocar atender_incidencia()
class TicketSeguridad(TicketSoporte):
    def resolver(self):
        print("Solución: comprobar accesos y cambiar credenciales")

    def mostrar_detalles(self):
        super().mostrar_detalles()
        self.resolver()
        print("Incidencia registrada")

tickets.append(TicketSeguridad(105, "Sara", "Se ha detectado un acceso sospechoso", "alta"))

for t in tickets:
    atender_incidencia(t)
```

🖥️ Salida:

```
-----------------------------
Incidencia: 101
Empleado: Ana
Descripción: El ordenador no enciende
Prioridad: ALTA - atención urgente
Solución: revisar componentes físicos
Incidencia registrada
-----------------------------
Incidencia: 102
Empleado: Luis
Descripción: La aplicación se bloquea
Prioridad: MEDIA - atención normal
Solución: comprobar y actualizar la aplicación
Incidencia registrada
-----------------------------
Incidencia: 103
Empleado: Marta
Descripción: No hay conexión a Internet
Prioridad: ALTA - atención urgente
Solución: verificar la conexión de red
Incidencia registrada
-----------------------------
Incidencia: 104
Empleado: Pedro
Descripción: No puedo actualizar el programa
Prioridad: BAJA - atención no urgente
Solución: comprobar y actualizar la aplicación
Incidencia registrada
-----------------------------
Incidencia: 105
Empleado: Sara
Descripción: Se ha detectado un acceso sospechoso
Prioridad: ALTA - atención urgente
```

```
Solución: comprobar accesos y cambiar credenciales
Incidencia registrada
```

Parte 4 · Práctica 15 resuelta (OOP · métodos y clases)

Instrucciones

- **Regla general: no está permitido usar herencia ni polimorfismo.** Ninguna clase hereda de otra (`class A(B):` está prohibido). Cuando una clase necesite a otra, usa **composición**: un objeto guarda otros objetos en sus atributos.
- Solo se usa la biblioteca estándar de Python (`math` y `random` cuando el ejercicio lo indique).
- Entregar solo el vínculo a tu repo de Github que tenga, además de esta practica, los ejemplos resueltos de la clases 18 y 19 > 💡 No hay un número mínimo de ejercicios propuestos para entregar, haz los que quieras.

Sección 1 - Nivel básico

Ejercicio B1 - Ficha de empleado del equipo de datos

El departamento de RR. HH. quiere dejar de gestionar las fichas de empleados con diccionarios sueltos. Crea la clase `Empleado`:

- Constructor con `nombre`, `departamento` y `salario_bruto` (anual, en euros). Además crea el atributo `proyectos`, que empieza siendo una **lista vacía**.
- Método `asignar_proyecto(proyecto)`: añade el proyecto a la lista **solo si no estaba ya asignado**.
- Método `salario_neto_mensual(retencion=0.15)`: aplica la retención al salario bruto, lo reparte en **14 pagas** y lo devuelve redondeado a 2 decimales.
- `__str__` con el formato `"Lucía Ortega (Data Platform) · 2 proyectos"`. **Código de prueba:**

```
lucia = Empleado("Lucía Ortega", "Data Platform", 42000)
mario = Empleado("Mario Gil", "BI", 35000)
lucia.asignar_proyecto("Migración a Lakehouse")
lucia.asignar_proyecto("Catálogo de datos")
lucia.asignar_proyecto("Migración a Lakehouse")
print(lucia)
print(mario)
print(lucia.proyectos)
print(lucia.salario_neto_mensual())
print(lucia.salario_neto_mensual(0.20))
print(mario.salario_neto_mensual())
```

Salida esperada:

```
Lucía Ortega (Data Platform) · 2 proyectos
Mario Gil (BI) · 0 proyectos
['Migración a Lakehouse', 'Catálogo de datos']
2550.0
2400.0
2125.0
```

✅ Solución

```
class Empleado:
    def __init__(self, nombre, departamento, salario_bruto):
        self.nombre = nombre
        self.departamento = departamento
        self.salario_bruto = salario_bruto
        self.proyectos = []                     # lista NUEVA para cada objeto

    def asignar_proyecto(self, proyecto):
        if proyecto not in self.proyectos:
            self.proyectos.append(proyecto)
```

```
    def salario_neto_mensual(self, retencion=0.15):
        return round(self.salario_bruto * (1 - retencion) / 14, 2)

    def __str__(self):
        return f"{self.nombre} ({self.departamento}) · {len(self.proyectos)} proyectos"

lucia = Empleado("Lucía Ortega", "Data Platform", 42000)
mario = Empleado("Mario Gil", "BI", 35000)
lucia.asignar_proyecto("Migración a Lakehouse")
lucia.asignar_proyecto("Catálogo de datos")
lucia.asignar_proyecto("Migración a Lakehouse")
print(lucia)
print(mario)
print(lucia.proyectos)
print(lucia.salario_neto_mensual())
print(lucia.salario_neto_mensual(0.20))
print(mario.salario_neto_mensual())
```

🖥️ **Salida:**

```
Lucía Ortega (Data Platform) · 2 proyectos
Mario Gil (BI) · 0 proyectos
['Migración a Lakehouse', 'Catálogo de datos']
2550.0
2400.0
2125.0
```

Ejercicio B2 - Pool de conexiones a una base de datos

El servidor PostgreSQL de producción solo admite **3 conexiones simultáneas** desde los procesos ETL. Crea la clase `ConexionBD`:

- Atributo de clase `MAX_CONEXIONES = 3` y atributo de clase `abiertas = 0`.
- Constructor con `servidor`. Cada conexión empieza con el atributo `activa = False`.
- Método `abrir()`:
- Si la conexión ya estaba activa, devuelve `"ya estaba abierta"`.
- Si se ha alcanzado el máximo de conexiones, devuelve `"rechazada: límite alcanzado"`.
- En otro caso la activa, suma 1 a `abiertas` **de la clase** y devuelve `"abierta"`.
- Método `cerrar()`: si estaba activa, la desactiva, resta 1 a `abiertas` y devuelve `"cerrada"`; si no, devuelve `"no estaba abierta"`.
- Método de clase `estado_pool()` (con `@classmethod`) que devuelve `"2/3 conexiones en uso"`. **Código de prueba:**

```
etl_ventas = ConexionBD("pg-prod-01")
etl_stock = ConexionBD("pg-prod-01")
etl_crm = ConexionBD("pg-prod-01")
etl_logs = ConexionBD("pg-prod-01")
print(etl_ventas.abrir(), "|", etl_stock.abrir(), "|", etl_crm.abrir())
print(etl_logs.abrir())
print(etl_ventas.abrir())
print(ConexionBD.estado_pool())
print(etl_stock.cerrar())
print(etl_logs.abrir())
print(etl_stock.cerrar())
print(ConexionBD.estado_pool())
```

Salida esperada:

```
abierta | abierta | abierta
rechazada: límite alcanzado
ya estaba abierta
3/3 conexiones en uso
cerrada
abierta
no estaba abierta
3/3 conexiones en uso
```

✅ **Solución**

```python
class ConexionBD:
    MAX_CONEXIONES = 3
    abiertas = 0                                # contador COMPARTIDO por todas

    def __init__(self, servidor):
        self.servidor = servidor
        self.activa = False

    def abrir(self):
        if self.activa:
            return "ya estaba abierta"
        if ConexionBD.abiertas >= ConexionBD.MAX_CONEXIONES:
            return "rechazada: límite alcanzado"
        self.activa = True
        ConexionBD.abiertas += 1                # ¡en la CLASE, no en self!
        return "abierta"

    def cerrar(self):
        if not self.activa:
            return "no estaba abierta"
        self.activa = False
        ConexionBD.abiertas -= 1
        return "cerrada"

    @classmethod
    def estado_pool(cls):
        return f"{cls.abiertas}/{cls.MAX_CONEXIONES} conexiones en uso"

etl_ventas = ConexionBD("pg-prod-01")
etl_stock = ConexionBD("pg-prod-01")
etl_crm = ConexionBD("pg-prod-01")
etl_logs = ConexionBD("pg-prod-01")
print(etl_ventas.abrir(), "|", etl_stock.abrir(), "|", etl_crm.abrir())
print(etl_logs.abrir())
print(etl_ventas.abrir())
print(ConexionBD.estado_pool())
print(etl_stock.cerrar())
print(etl_logs.abrir())
print(etl_stock.cerrar())
print(ConexionBD.estado_pool())
```

🖥️ **Salida:**

```
abierta | abierta | abierta
rechazada: límite alcanzado
ya estaba abierta
3/3 conexiones en uso
cerrada
abierta
no estaba abierta
3/3 conexiones en uso
```

Ejercicio B3 - Plantilla de prompt reutilizable · IA Generativa

El equipo de atención al cliente de una cadena de electrónica usa un LLM para clasificar reseñas. Cada analista escribe el prompt a mano y los resultados son inconsistentes. Crea la clase `PlantillaPrompt` para que todos usen la misma estructura:

- Constructor con `rol`, `tarea` y `formato_salida` (por defecto `"texto libre"`). Crea también la lista `restricciones`, vacía.
- Método `agregar_restriccion(texto)`: añade una restricción a la lista.
- Método `construir(entrada)`: devuelve **un único texto** con varias líneas, en este orden:

1. `Actúa como <rol>.`
2. `Tarea: <tarea>`
3. Si hay restricciones, la línea `Restricciones:` y una línea `<restricción>` por cada una.
4. `Formato de salida: <formato_salida>`

5. `Entrada: """<entrada>"""`

- `__repr__` con el formato `PlantillaPrompt(rol='analista de experiencia de cliente', restricciones=2)`. Pista: construye una lista de líneas y únelas con `"\n".join(lineas)`.

Código de prueba:

```
clasificador = PlantillaPrompt(
    "analista de experiencia de cliente",
    "clasifica el sentimiento de la reseña como POSITIVO, NEUTRO o NEGATIVO",
    "JSON con las claves sentimiento y motivo",
)
clasificador.agregar_restriccion("No inventes información que no aparezca en la reseña")
clasificador.agregar_restriccion("El motivo debe tener como máximo 15 palabras")
print(repr(clasificador))
print(clasificador.construir("El portátil llegó en 24 h pero la batería dura la mitad de lo anunciado."))
print("---")
resumen = PlantillaPrompt("redactor técnico", "resume la incidencia en una frase")
print(resumen.construir("El TPV de la tienda 14 no imprime tickets desde las 10:00."))
```

Salida esperada:

```
PlantillaPrompt(rol='analista de experiencia de cliente', restricciones=2)
Actúa como analista de experiencia de cliente.
Tarea: clasifica el sentimiento de la reseña como POSITIVO, NEUTRO o NEGATIVO
Restricciones:
- No inventes información que no aparezca en la reseña
- El motivo debe tener como máximo 15 palabras
Formato de salida: JSON con las claves sentimiento y motivo
Entrada: """El portátil llegó en 24 h pero la batería dura la mitad de lo anunciado."""

---
Actúa como redactor técnico.
Tarea: resume la incidencia en una frase
Formato de salida: texto libre
Entrada: """El TPV de la tienda 14 no imprime tickets desde las 10:00."""
```

✅ Solución

```
class PlantillaPrompt:
    def __init__(self, rol, tarea, formato_salida="texto libre"):
        self.rol = rol
        self.tarea = tarea
        self.formato_salida = formato_salida
        self.restricciones = []

    def agregar_restriccion(self, texto):
        self.restricciones.append(texto)

    def construir(self, entrada):
        lineas = [f"Actúa como {self.rol}.", f"Tarea: {self.tarea}"]
        if self.restricciones:
            lineas.append("Restricciones:")
            lineas += [f"- {r}" for r in self.restricciones]
        lineas.append(f"Formato de salida: {self.formato_salida}")
        lineas.append(f'Entrada: """{entrada}"""')
        return "\n".join(lineas)

    def __repr__(self):
        return f"PlantillaPrompt(rol='{self.rol}', restricciones={len(self.restricciones)})"

clasificador = PlantillaPrompt(
    "analista de experiencia de cliente",
    "clasifica el sentimiento de la reseña como POSITIVO, NEUTRO o NEGATIVO",
    "JSON con las claves sentimiento y motivo",
)
clasificador.agregar_restriccion("No inventes información que no aparezca en la reseña")
clasificador.agregar_restriccion("El motivo debe tener como máximo 15 palabras")
print(repr(clasificador))
print(clasificador.construir("El portátil llegó en 24 h pero la batería dura la mitad de lo anunciado."))
```

```
print("---")
resumen = PlantillaPrompt("redactor técnico", "resume la incidencia en una frase")
print(resumen.construir("El TPV de la tienda 14 no imprime tickets desde las 10:00."))
```

🖥️ Salida:

```
PlantillaPrompt(rol='analista de experiencia de cliente', restricciones=2)
Actúa como analista de experiencia de cliente.
Tarea: clasifica el sentimiento de la reseña como POSITIVO, NEUTRO o NEGATIVO
Restricciones:
- No inventes información que no aparezca en la reseña
- El motivo debe tener como máximo 15 palabras
Formato de salida: JSON con las claves sentimiento y motivo
Entrada: """El portátil llegó en 24 h pero la batería dura la mitad de lo anunciado."""
---
Actúa como redactor técnico.
Tarea: resume la incidencia en una frase
Formato de salida: texto libre
Entrada: """El TPV de la tienda 14 no imprime tickets desde las 10:00."""
```

Ejercicio B4 - Coste de las llamadas a un LLM · IA Generativa

Finanzas quiere saber cuánto cuesta cada llamada al proveedor de modelos de lenguaje. El proveedor cobra en dólares **por cada millón de tokens**, con un precio distinto para los tokens de entrada y los de salida. Crea la clase `LlamadaLLM`:

- Atributo de clase `PRECIOS`, un diccionario `{modelo: (precio_entrada, precio_salida)}`: | Modelo | Entrada (/M tokens) | Salida (/M tokens) | | | — | — | — | | `modelo-lite` | 0.20 | 0.80 | | `modelo-pro` | 3.00 | 12.00 |
- Atributo de clase `total_llamadas = 0`, que se incrementa cada vez que se crea una llamada.
- Constructor con `modelo`, `tokens_entrada` y `tokens_salida`.
- Método `coste()`: devuelve el coste en dólares redondeado a **6 decimales**.
- Método `tokens_totales()`.
- `__repr__` con el formato `LlamadaLLM(modelo-lite, 1500 tokens, $0.00048)`. **Código de prueba:**

```
llamadas = [
    LlamadaLLM("modelo-lite", 1200, 300),
    LlamadaLLM("modelo-pro", 1200, 300),
    LlamadaLLM("modelo-lite", 8000, 1500),
    LlamadaLLM("modelo-pro", 25000, 4000),
]
for llamada in llamadas:
    print(llamada)
print("Llamadas registradas:", LlamadaLLM.total_llamadas)
print(f"Coste total: ${round(sum([l.coste() for l in llamadas]), 6)}")
coste_pro = sum([l.coste() for l in llamadas if l.modelo == "modelo-pro"])
print("Peso de modelo-pro:", round(100 * coste_pro / sum([l.coste() for l in llamadas]), 1), "%")
```

Salida esperada:

```
LlamadaLLM(modelo-lite, 1500 tokens, $0.00048)
LlamadaLLM(modelo-pro, 1500 tokens, $0.0072)
LlamadaLLM(modelo-lite, 9500 tokens, $0.0028)
LlamadaLLM(modelo-pro, 29000 tokens, $0.123)
Llamadas registradas: 4
Coste total: $0.13348
Peso de modelo-pro: 97.5 %
```

✅ Solución

```
class LlamadaLLM:
    PRECIOS = {"modelo-lite": (0.20, 0.80), "modelo-pro": (3.00, 12.00)}   # $ por millón
    total_llamadas = 0
```

```
    def __init__(self, modelo, tokens_entrada, tokens_salida):
        self.modelo = modelo
        self.tokens_entrada = tokens_entrada
        self.tokens_salida = tokens_salida
        LlamadaLLM.total_llamadas += 1

    def coste(self):
        precio_in, precio_out = LlamadaLLM.PRECIOS[self.modelo]
        return round((self.tokens_entrada * precio_in + self.tokens_salida * precio_out) / 1_000_000, 6)

    def tokens_totales(self):
        return self.tokens_entrada + self.tokens_salida

    def __repr__(self):
        return f"LlamadaLLM({self.modelo}, {self.tokens_totales()} tokens, ${self.coste()})"

llamadas = [
    LlamadaLLM("modelo-lite", 1200, 300),
    LlamadaLLM("modelo-pro", 1200, 300),
    LlamadaLLM("modelo-lite", 8000, 1500),
    LlamadaLLM("modelo-pro", 25000, 4000),
]
for llamada in llamadas:
    print(llamada)
print("Llamadas registradas:", LlamadaLLM.total_llamadas)
print(f"Coste total: ${round(sum([l.coste() for l in llamadas]), 6)}")
coste_pro = sum([l.coste() for l in llamadas if l.modelo == "modelo-pro"])
print("Peso de modelo-pro:", round(100 * coste_pro / sum([l.coste() for l in llamadas]), 1), "%")
```

🖥️ **Salida:**

```
LlamadaLLM(modelo-lite, 1500 tokens, $0.00048)
LlamadaLLM(modelo-pro, 1500 tokens, $0.0072)
LlamadaLLM(modelo-lite, 9500 tokens, $0.0028)
LlamadaLLM(modelo-pro, 29000 tokens, $0.123)
Llamadas registradas: 4
Coste total: $0.13348
Peso de modelo-pro: 97.5 %
```

Ejercicio B5 - Climatización de un centro de datos

La consigna de temperatura de cada sala del CPD solo puede estar entre **18 y 27 °C**. Nadie debe poder cambiarla «a mano» desde fuera del objeto. Crea la clase `Climatizador` aplicando las convenciones de nombres:

- Atributos de clase `TEMP_MIN = 18` y `TEMP_MAX = 27`.
- Constructor con `sala` (público) y `consigna` inicial. La consigna se guarda en el atributo **privado** `__consigna`, ya limitada al rango permitido. Crea también el atributo de **uso interno** `_historial`, una lista con la consigna inicial.
- Método **de uso interno** `_limitar(valor)`: devuelve el valor ajustado al rango (si es menor que el mínimo, el mínimo; si es mayor que el máximo, el máximo).
- Método `ajustar(grados)`: suma (o resta, si es negativo) los grados a la consigna, la limita con `_limitar`, la añade al historial y devuelve la nueva consigna.
- Método `consigna()`: devuelve el valor actual.
- `__str__`: `"Sala A · 22 °C · 3 cambios"` (los cambios son las entradas del historial menos la inicial).
  Responde en un comentario al final del archivo: ¿por qué `hasattr(sala_a, "__consigna")` devuelve `False` si el atributo existe?

Código de prueba:

```
sala_a = Climatizador("A", 22)
sala_b = Climatizador("B", 31)
print(sala_a.ajustar(-3), sala_a.ajustar(-4), sala_a.ajustar(6))
print(sala_a)
print(sala_b)
print(sala_a._historial)
print(vars(sala_b))
print(hasattr(sala_a, "__consigna"), hasattr(sala_a, "_Climatizador__consigna"))
```

Salida esperada:

```
19 18 24
Sala A · 24 °C · 3 cambios
Sala B · 27 °C · 0 cambios
[22, 19, 18, 24]
{'sala': 'B', '_Climatizador__consigna': 27, '_historial': [27]}
False True
```

✅ Solución

```python
class Climatizador:
    TEMP_MIN = 18
    TEMP_MAX = 27

    def __init__(self, sala, consigna):
        self.sala = sala                                # público
        self.__consigna = self._limitar(consigna)       # privado → _Climatizador__consigna
        self._historial = [self.__consigna]             # uso interno

    def _limitar(self, valor):
        return max(Climatizador.TEMP_MIN, min(Climatizador.TEMP_MAX, valor))

    def ajustar(self, grados):
        self.__consigna = self._limitar(self.__consigna + grados)
        self._historial.append(self.__consigna)
        return self.__consigna

    def consigna(self):
        return self.__consigna

    def __str__(self):
        return f"Sala {self.sala} · {self.__consigna} °C · {len(self._historial) - 1} cambios"

sala_a = Climatizador("A", 22)
sala_b = Climatizador("B", 31)
print(sala_a.ajustar(-3), sala_a.ajustar(-4), sala_a.ajustar(6))
print(sala_a)
print(sala_b)
print(sala_a._historial)
print(vars(sala_b))
print(hasattr(sala_a, "__consigna"), hasattr(sala_a, "_Climatizador__consigna"))
# hasattr(sala_a, "__consigna") es False porque, dentro de la clase, Python renombra
# self.__consigna a self._Climatizador__consigna (name mangling). Con ese nombre sí existe.
```

🖥️ Salida:

```
19 18 24
Sala A · 24 °C · 3 cambios
Sala B · 27 °C · 0 cambios
[22, 19, 18, 24]
{'sala': 'B', '_Climatizador__consigna': 27, '_historial': [27]}
False True
```

Ejercicio B6 - Peso de los envíos de un almacén

El almacén de una empresa de e-commerce agrupa paquetes en palés y necesita sumar, comparar y ordenar pesos. Crea la clase `PesoEnvio`:

- Constructor con `gramos` (entero). Si llega un valor negativo, se guarda `0`.
- Sobrecarga `+` para que la suma de dos pesos devuelva un **objeto nuevo** `PesoEnvio`.
- Sobrecarga `==` y `<` comparando los gramos.
- `__str__`: si el peso es de 1000 g o más, se muestra en kilos con 2 decimales (`"1.25 kg"`); si no, en gramos (`"850 g"`).
- `__repr__`: `PesoEnvio(850)`. **Código de prueba:**

```
paquetes = [PesoEnvio(850), PesoEnvio(1250), PesoEnvio(400), PesoEnvio(-30), PesoEnvio(2300)]
print(paquetes[0] + paquetes[2])
print(paquetes[0] + paquetes[1])
pale = PesoEnvio(0)
for p in paquetes:
    pale = pale + p
print("Palé:", pale)
print(sorted(paquetes))
print("Más pesado:", max(paquetes))
print(PesoEnvio(1250) == paquetes[1], PesoEnvio(1250) is paquetes[1])
```

Salida esperada:

```
1.25 kg
2.10 kg
Palé: 4.80 kg
[PesoEnvio(0), PesoEnvio(400), PesoEnvio(850), PesoEnvio(1250), PesoEnvio(2300)]
Más pesado: 2.30 kg
True False
```

✅ Solución

```
class PesoEnvio:
    def __init__(self, gramos):
        self.gramos = max(0, gramos)

    def __add__(self, otro):
        return PesoEnvio(self.gramos + otro.gramos)

    def __eq__(self, otro):
        return self.gramos == otro.gramos

    def __lt__(self, otro):                     # sorted(), min() y max() lo usan
        return self.gramos < otro.gramos

    def __str__(self):
        if self.gramos >= 1000:
            return f"{self.gramos / 1000:.2f} kg"
        return f"{self.gramos} g"

    def __repr__(self):
        return f"PesoEnvio({self.gramos})"

paquetes = [PesoEnvio(850), PesoEnvio(1250), PesoEnvio(400), PesoEnvio(-30), PesoEnvio(2300)]
print(paquetes[0] + paquetes[2])
print(paquetes[0] + paquetes[1])
pale = PesoEnvio(0)
for p in paquetes:
    pale = pale + p
print("Palé:", pale)
print(sorted(paquetes))                         # una lista se imprime con __repr__
print("Más pesado:", max(paquetes))
print(PesoEnvio(1250) == paquetes[1], PesoEnvio(1250) is paquetes[1])
```

🖥️ Salida:

```
1.25 kg
2.10 kg
Palé: 4.80 kg
[PesoEnvio(0), PesoEnvio(400), PesoEnvio(850), PesoEnvio(1250), PesoEnvio(2300)]
Más pesado: 2.30 kg
True False
```

Ejercicio B7 - Valoraciones de clientes desde una encuesta

La herramienta de encuestas exporta las valoraciones como texto: `"4,5/5"`, `"3/5"`, `"0/5"` (el `0` significa *sin valorar*). Crea la clase `Valoracion`:

- El constructor recibe el texto, toma la parte anterior a la `/`, cambia la coma por punto y guarda el valor como `float` en el atributo **privado** `__nota`.
- `float(v)`: devuelve la nota.
- `int(v)`: devuelve la nota **redondeada** a entero.
- `bool(v)`: `False` si la nota es 0 (sin valorar).
- `str(v)`: tantas `★` como `int(v)` y tantas `☆` como falten hasta 5, seguido de la nota entre paréntesis: `"★★★★☆ (4.5)"`. **Código de prueba:**

```
respuestas = ["4,5/5", "3/5", "0/5", "2,4/5", "5/5", "0/5"]
valoraciones = [Valoracion(r) for r in respuestas]
for v in valoraciones:
    print(str(v), float(v), int(v), bool(v))
validas = [v for v in valoraciones if v]
print("Respuestas válidas:", len(validas), "de", len(valoraciones))
print("Media:", round(sum([float(v) for v in validas]) / len(validas), 2))
```

Salida esperada:

```
★★★★☆ (4.5) 4.5 4 True
★★★☆☆ (3.0) 3.0 3 True
☆☆☆☆☆ (0.0) 0.0 0 False
★★☆☆☆ (2.4) 2.4 2 True
★★★★★ (5.0) 5.0 5 True
☆☆☆☆☆ (0.0) 0.0 0 False
Respuestas válidas: 4 de 6
Media: 3.73
```

✅ Solución

```
class Valoracion:
    def __init__(self, texto):
        self.__nota = float(texto.split("/")[0].replace(",", "."))

    def __float__(self):
        return self.__nota

    def __int__(self):
        return round(self.__nota)        # round(4.5) = 4 (redondeo bancario)

    def __bool__(self):
        return self.__nota != 0

    def __str__(self):
        estrellas = int(self)
        return "★" * estrellas + "☆" * (5 - estrellas) + f" ({self.__nota})"

respuestas = ["4,5/5", "3/5", "0/5", "2,4/5", "5/5", "0/5"]
valoraciones = [Valoracion(r) for r in respuestas]
for v in valoraciones:
    print(str(v), float(v), int(v), bool(v))
validas = [v for v in valoraciones if v]
print("Respuestas válidas:", len(validas), "de", len(valoraciones))
print("Media:", round(sum([float(v) for v in validas]) / len(validas), 2))
```

🖥️ Salida:

```
★★★★☆ (4.5) 4.5 4 True
★★★☆☆ (3.0) 3.0 3 True
☆☆☆☆☆ (0.0) 0.0 0 False
★★☆☆☆ (2.4) 2.4 2 True
★★★★★ (5.0) 5.0 5 True
☆☆☆☆☆ (0.0) 0.0 0 False
Respuestas válidas: 4 de 6
Media: 3.73
```

Sección 2 - Nivel intermedio

Ejercicio I1 - Historial de conversación de un asistente · IA Generativa

Una aseguradora está construyendo un asistente para tramitar partes de siniestros. Las API de los LLM reciben el historial como una lista de diccionarios `{"role": ..., "content": ...}` y tienen una **ventana de contexto** limitada. Modela el historial con **dos clases relacionadas por composición**.

Clase `Mensaje`

- Atributo de clase `ROLES_VALIDOS = ("system", "user", "assistant")`.
- Constructor con `rol` y `contenido`.
- Método `tokens()`: estimación sencilla, **1 token por palabra** (`len(contenido.split())`).
- Método `a_dict()`: devuelve `{"role": rol, "content": contenido}`.
- `__str__`: `"[user] Quiero dar un parte de mi coche"`. **Clase `Conversacion`**
- Constructor con `max_tokens`. Guarda los mensajes en una lista **privada** `__mensajes`.
- Método `agregar(rol, contenido)`: si el rol no está en `Mensaje.ROLES_VALIDOS`, no añade nada y devuelve `False`. Si es válido, crea el `Mensaje`, lo añade, llama a `_recortar()` y devuelve `True`.
- `+=` recibe una **tupla** `(rol, contenido)` y usa `agregar`.
- Método `tokens_totales()`.
- Método de uso interno `_recortar()`: mientras los tokens totales superen `max_tokens`, elimina el **mensaje más antiguo que no sea `system`** (el mensaje de sistema nunca se borra).
- `len(conv)`: número de mensajes. `palabra in conv`: `True` si la palabra aparece en algún mensaje, sin distinguir mayúsculas.
- Método `para_api()`: lista de diccionarios de todos los mensajes.
- `__str__`: un mensaje por línea. **Código de prueba:**

```
conv = Conversacion(max_tokens=30)
conv += ("system", "Eres el asistente de siniestros de Seguros Brisamar")
conv += ("user", "Quiero dar un parte de mi coche")
conv += ("assistant", "Claro, ¿cuál es la matrícula del vehículo?")
print(conv.agregar("cliente", "Hola"))
print(len(conv), conv.tokens_totales())
conv += ("user", "Es la 4821 KLM y el golpe fue en un parking")
print(len(conv), conv.tokens_totales())
print(conv)
print("matrícula" in conv, "COCHE" in conv)
print(conv.para_api()[0])
```

Salida esperada:

```
False
3 22
3 26
[system] Eres el asistente de siniestros de Seguros Brisamar
[assistant] Claro, ¿cuál es la matrícula del vehículo?
[user] Es la 4821 KLM y el golpe fue en un parking
True False
{'role': 'system', 'content': 'Eres el asistente de siniestros de Seguros Brisamar'}
```

✅ Solución

```
class Mensaje:
    ROLES_VALIDOS = ("system", "user", "assistant")
```

```
    def __init__(self, rol, contenido):
        self.rol = rol
        self.contenido = contenido

    def tokens(self):
        return len(self.contenido.split())

    def a_dict(self):
        return {"role": self.rol, "content": self.contenido}

    def __str__(self):
        return f"[{self.rol}] {self.contenido}"

class Conversacion:                                  # COMPOSICIÓN: contiene Mensajes
    def __init__(self, max_tokens):
        self.max_tokens = max_tokens
        self.__mensajes = []

    def agregar(self, rol, contenido):
        if rol not in Mensaje.ROLES_VALIDOS:
            return False
        self.__mensajes.append(Mensaje(rol, contenido))
        self._recortar()
        return True

    def __iadd__(self, tupla):
        self.agregar(*tupla)                         # desempaqueta (rol, contenido)
        return self

    def tokens_totales(self):
        return sum(m.tokens() for m in self.__mensajes)

    def _recortar(self):
        while self.tokens_totales() > self.max_tokens:
            for i, m in enumerate(self.__mensajes):
                if m.rol != "system":
                    del self.__mensajes[i]           # el más antiguo que no es system
                    break
            else:
                break                                # solo quedan system: no se puede recortar

    def __len__(self):
        return len(self.__mensajes)

    def __contains__(self, palabra):
        return any(palabra.lower() in m.contenido.lower() for m in self.__mensajes)

    def para_api(self):
        return [m.a_dict() for m in self.__mensajes]

    def __str__(self):
        return "\n".join(str(m) for m in self.__mensajes)

conv = Conversacion(max_tokens=30)
conv += ("system", "Eres el asistente de siniestros de Seguros Brisamar")
conv += ("user", "Quiero dar un parte de mi coche")
conv += ("assistant", "Claro, ¿cuál es la matrícula del vehículo?")
print(conv.agregar("cliente", "Hola"))
print(len(conv), conv.tokens_totales())
conv += ("user", "Es la 4821 KLM y el golpe fue en un parking")
print(len(conv), conv.tokens_totales())
print(conv)
print("matrícula" in conv, "COCHE" in conv)
print(conv.para_api()[0])
```

🖥️ Salida:

```
False
3 22
3 26
[system] Eres el asistente de siniestros de Seguros Brisamar
[assistant] Claro, ¿cuál es la matrícula del vehículo?
[user] Es la 4821 KLM y el golpe fue en un parking
```

```
True False
{'role': 'system', 'content': 'Eres el asistente de siniestros de Seguros Brisamar'}
```

Ejercicio I2 - Cuotas de tokens por equipo - IA Generativa

Una consultora da acceso a un LLM corporativo a varios equipos y cada uno tiene una **cuota mensual de tokens**. La plataforma debe rechazar las peticiones que superen la cuota y mostrar un panel de consumo. Crea la clase `CuotaEquipo`:

- Atributo de clase `peticiones_rechazadas = 0` (para **todos** los equipos).
- Constructor con `equipo` y `limite_tokens`. Guarda los consumos en una lista **privada** `__consumos`, donde cada consumo es una tupla `(caso_uso, tokens)`.
- Método `consumido()` y método `restante()`.
- Método `registrar(caso_uso, tokens)`: si cabe en lo que queda, guarda el consumo y devuelve `True`; si no, suma 1 a `peticiones_rechazadas` **de la clase** y devuelve `False`.
- Método `porcentaje()`: porcentaje de la cuota consumido, redondeado a 1 decimal.
- Método `top_caso_uso()`: el caso de uso que más tokens ha consumido (suma por caso de uso). Si no hay consumos, `None`.
- `bool(cuota)`: `True` mientras quede algún token disponible.
- `<` compara por porcentaje consumido (para poder ordenar el panel).
- `__str__`: nombre del equipo con ancho 10, una barra de 10 caracteres (`#` por cada 10 % consumido, redondeando hacia abajo, y `.` para el resto) y el porcentaje: `"Marketing [######....] 64.0 %"`. **Código de prueba:**

```
marketing = CuotaEquipo("Marketing", 50_000)
legal = CuotaEquipo("Legal", 20_000)
soporte = CuotaEquipo("Soporte", 80_000)

marketing.registrar("copys campaña", 18_000)
marketing.registrar("traducciones", 9_000)
marketing.registrar("copys campaña", 5_000)
legal.registrar("resumen contratos", 15_000)
print(legal.registrar("resumen contratos", 6_000))
print(legal.registrar("cláusulas", 5_000))
soporte.registrar("respuestas tickets", 12_000)

panel = sorted([marketing, legal, soporte])
panel.reverse()
for cuota in panel:
    print(cuota)
print("Restante Legal:", legal.restante(), "| ¿Puede seguir?", bool(legal))
print("Top Marketing:", marketing.top_caso_uso())
print("Rechazadas en total:", CuotaEquipo.peticiones_rechazadas)
print(CuotaEquipo("I+D", 10_000).top_caso_uso())
```

Salida esperada:

```
False
True
Legal      [##########] 100.0 %
Marketing  [######....] 64.0 %
Soporte    [#.........] 15.0 %
Restante Legal: 0 | ¿Puede seguir? False
Top Marketing: copys campaña
Rechazadas en total: 1
None
```

✅ Solución

```
class CuotaEquipo:
    peticiones_rechazadas = 0

    def __init__(self, equipo, limite_tokens):
        self.equipo = equipo
        self.limite_tokens = limite_tokens
        self.__consumos = []                    # [(caso_uso, tokens), ...]
```

```
    def consumido(self):
        return sum(t for _, t in self.__consumos)

    def restante(self):
        return self.limite_tokens - self.consumido()

    def registrar(self, caso_uso, tokens):
        if tokens <= self.restante():
            self.__consumos.append((caso_uso, tokens))
            return True
        CuotaEquipo.peticiones_rechazadas += 1
        return False

    def porcentaje(self):
        return round(self.consumido() / self.limite_tokens * 100, 1)

    def top_caso_uso(self):
        if not self.__consumos:
            return None
        por_caso = {}
        for caso, t in self.__consumos:
            por_caso[caso] = por_caso.get(caso, 0) + t
        return max(por_caso, key=por_caso.get)

    def __bool__(self):
        return self.restante() > 0

    def __lt__(self, otra):
        return self.porcentaje() < otra.porcentaje()

    def __str__(self):
        llenas = int(self.porcentaje() // 10)
        return f"{self.equipo:<10} [{'#' * llenas}{'.' * (10 - llenas)}] {self.porcentaje()} %"

marketing = CuotaEquipo("Marketing", 50_000)
legal = CuotaEquipo("Legal", 20_000)
soporte = CuotaEquipo("Soporte", 80_000)

marketing.registrar("copys campaña", 18_000)
marketing.registrar("traducciones", 9_000)
marketing.registrar("copys campaña", 5_000)
legal.registrar("resumen contratos", 15_000)
print(legal.registrar("resumen contratos", 6_000))
print(legal.registrar("cláusulas", 5_000))
soporte.registrar("respuestas tickets", 12_000)

panel = sorted([marketing, legal, soporte])
panel.reverse()
for cuota in panel:
    print(cuota)
print("Restante Legal:", legal.restante(), "| ¿Puede seguir?", bool(legal))
print("Top Marketing:", marketing.top_caso_uso())
print("Rechazadas en total:", CuotaEquipo.peticiones_rechazadas)
print(CuotaEquipo("I+D", 10_000).top_caso_uso())
```

🖥️ Salida:

```
False
True
Legal      [##########] 100.0 %
Marketing  [######....] 64.0 %
Soporte    [#.........] 15.0 %
Restante Legal: 0 | ¿Puede seguir? False
Top Marketing: copys campaña
Rechazadas en total: 1
None
```

Ejercicio I3 - Control de calidad de un lote de ingesta - Ingeniería de Datos

Antes de cargar los pedidos en la capa *silver* del data lake, el equipo de Data Engineering quiere separar los registros válidos de los rechazados e informar del motivo. Implementa **dos clases relacionadas por composición**.

Clase `ReglaCalidad`

- Constructor con `columna`, `tipo` (texto: `"int"`, `"float"` o `"str"`), `obligatoria` (por defecto `True`) y `minimo` (por defecto `None`).
- Método `comprobar(registro)`: recibe un diccionario y devuelve un **texto con el error** o `None` si la regla se cumple. Comprueba en este orden:

1. Si la columna no está o vale `None`: si es obligatoria, `"<columna>: falta"`; si no es obligatoria, la regla se cumple.
2. Si `type(valor).__name__` no coincide con `tipo`: `"<columna>: se esperaba <tipo>"`.
3. Si hay mínimo y el valor es menor: `"<columna>: menor que <minimo>"`.

- `__repr__`: `Regla(importe:float>=0.01)` o, sin mínimo y opcional, `Regla(cupon:str?)` (`?` indica opcional).

Clase `ValidadorLote`

- Constructor con `nombre`. Guarda una lista **privada** de reglas y dos listas públicas vacías: `validos` y `rechazados`.
- `+=` añade una `ReglaCalidad`. `len(validador)` devuelve el número de reglas.
- Método `validar(registro)`: devuelve la lista de errores de **todas** las reglas.
- Método `procesar(lote)`: vacía `validos` y `rechazados` y recorre el lote. Los registros sin errores van a `validos`; los demás van a `rechazados` como tupla `(registro, errores)`.
- Método `tasa_calidad()`: porcentaje de válidos (1 decimal).
- `__str__`: informe con el formato de la salida esperada. **Código de prueba:**

```
validador = ValidadorLote("pedidos_silver")
validador += ReglaCalidad("id_pedido", "int")
validador += ReglaCalidad("cliente", "str")
validador += ReglaCalidad("importe", "float", minimo=0.01)
validador += ReglaCalidad("unidades", "int", minimo=1)
validador += ReglaCalidad("cupon", "str", obligatoria=False)

lote = [
    {"id_pedido": 5001, "cliente": "C-17", "importe": 89.9, "unidades": 2},
    {"id_pedido": 5002, "cliente": "C-03", "importe": "45,00", "unidades": 1},
    {"id_pedido": 5003, "importe": 12.5, "unidades": 0, "cupon": "BF26"},
    {"id_pedido": 5004, "cliente": "C-17", "importe": 230.0, "unidades": 4, "cupon": "BF26"},
    {"id_pedido": 5005, "cliente": "C-88", "importe": -5.0, "unidades": 1, "cupon": 10},
]
validador.procesar(lote)
print(validador)
print([r["id_pedido"] for r in validador.validos])
print(ReglaCalidad("importe", "float", minimo=0.01), ReglaCalidad("cupon", "str", obligatoria=False))
```

Salida esperada:

```
Validador pedidos_silver · 5 reglas
Válidos: 2 | Rechazados: 3 | Calidad: 40.0 %
  - 5002: importe: se esperaba float
  - 5003: cliente: falta; unidades: menor que 1
  - 5005: importe: menor que 0.01; cupon: se esperaba str
[5001, 5004]
Regla(importe:float>=0.01) Regla(cupon:str?)
```

✅ Solución

```python
class ReglaCalidad:
    def __init__(self, columna, tipo, obligatoria=True, minimo=None):
        self.columna = columna
        self.tipo = tipo
        self.obligatoria = obligatoria
        self.minimo = minimo

    def comprobar(self, registro):
        valor = registro.get(self.columna)
        if valor is None:
            return f"{self.columna}: falta" if self.obligatoria else None
        if type(valor).__name__ != self.tipo:
            return f"{self.columna}: se esperaba {self.tipo}"
        if self.minimo is not None and valor < self.minimo:
            return f"{self.columna}: menor que {self.minimo}"
        return None

    def __repr__(self):
        minimo = f">={self.minimo}" if self.minimo is not None else ""
        opcional = "" if self.obligatoria else "?"
        return f"Regla({self.columna}:{self.tipo}{minimo}{opcional})"

class ValidadorLote:
    def __init__(self, nombre):
        self.nombre = nombre
        self.__reglas = []
        self.validos = []
        self.rechazados = []

    def __iadd__(self, regla):
        self.__reglas.append(regla)
        return self

    def __len__(self):
        return len(self.__reglas)

    def validar(self, registro):
        errores = [r.comprobar(registro) for r in self.__reglas]
        return [e for e in errores if e is not None]

    def procesar(self, lote):
        self.validos, self.rechazados = [], []
        for registro in lote:
            errores = self.validar(registro)
            if errores:
                self.rechazados.append((registro, errores))
            else:
                self.validos.append(registro)

    def tasa_calidad(self):
        total = len(self.validos) + len(self.rechazados)
        return round(len(self.validos) / total * 100, 1) if total else 0.0

    def __str__(self):
        lineas = [f"Validador {self.nombre} · {len(self)} reglas",
                  f"Válidos: {len(self.validos)} | Rechazados: {len(self.rechazados)} | "
                  f"Calidad: {self.tasa_calidad()} %"]
        lineas += [f"  - {reg['id_pedido']}: {'; '.join(err)}" for reg, err in self.rechazados]
        return "\n".join(lineas)

validador = ValidadorLote("pedidos_silver")
validador += ReglaCalidad("id_pedido", "int")
validador += ReglaCalidad("cliente", "str")
validador += ReglaCalidad("importe", "float", minimo=0.01)
validador += ReglaCalidad("unidades", "int", minimo=1)
validador += ReglaCalidad("cupon", "str", obligatoria=False)

lote = [
    {"id_pedido": 5001, "cliente": "C-17", "importe": 89.9, "unidades": 2},
    {"id_pedido": 5002, "cliente": "C-03", "importe": "45,00", "unidades": 1},
    {"id_pedido": 5003, "importe": 12.5, "unidades": 0, "cupon": "BF26"},
    {"id_pedido": 5004, "cliente": "C-17", "importe": 230.0, "unidades": 4, "cupon": "BF26"},
    {"id_pedido": 5005, "cliente": "C-88", "importe": -5.0, "unidades": 1, "cupon": 10},
```

```
]
validador.procesar(lote)
print(validador)
print([r["id_pedido"] for r in validador.validos])
print(ReglaCalidad("importe", "float", minimo=0.01), ReglaCalidad("cupon", "str", obligatoria=False))
```

🖥️ **Salida:**

```
Validador pedidos_silver · 5 reglas
Válidos: 2 | Rechazados: 3 | Calidad: 40.0 %
  - 5002: importe: se esperaba float
  - 5003: cliente: falta; unidades: menor que 1
  - 5005: importe: menor que 0.01; cupon: se esperaba str
[5001, 5004]
Regla(importe:float>=0.01) Regla(cupon:str?)
```

Ejercicio I4 - Reservas de salas de reuniones

La oficina central de una empresa de logística tiene salas que se reservan por franjas horarias y continuamente hay reservas solapadas. Implementa:

Clase `FranjaHoraria`

- Constructor con `inicio` y `fin` como texto `"HH:MM"`. Internamente guarda los **minutos desde las 00:00** en dos atributos privados.
- Método de uso interno `_a_minutos(texto)` que hace esa conversión.
- `len(franja)`: duración en minutos.
- `"HH:MM" in franja`: `True` si la hora está dentro de la franja (incluye el inicio, **excluye** el fin).
- Método `solapa(otra)`: `True` si las dos franjas comparten algún minuto.
- `<` compara por hora de inicio.
- `__str__`: `"09:30-10:15"`. **Clase `Sala`**
- Constructor con `nombre` y `capacidad`. Lista **privada** de reservas; cada reserva es una tupla `(franja, responsable, asistentes)`.
- Método `reservar(inicio, fin, responsable, asistentes)` que devuelve un texto:
- `"RECHAZADA: aforo"` si los asistentes superan la capacidad.
- `"RECHAZADA: solapa con <franja> (<responsable>)"` si choca con una reserva existente.
- `"OK <franja>"` si se guarda.
- Método `ocupada_a(hora)`: responsable de la reserva que ocupa la sala a esa hora, o `None`.
- Método `ocupacion(jornada_min=600)`: porcentaje de una jornada de 10 h reservado (1 decimal).
- Método `agenda()`: texto con las reservas **ordenadas por hora**, una por línea: `"09:00-10:00 · Ana (6 pers.)"`. **Código de prueba:**

```
sala = Sala("Atlas", 8)
print(sala.reservar("10:00", "11:30", "Ana", 6))
print(sala.reservar("09:00", "10:00", "Luis", 4))
print(sala.reservar("11:00", "12:00", "Marta", 5))
print(sala.reservar("12:00", "12:45", "Iker", 12))
print(sala.reservar("15:15", "16:00", "Iker", 7))
print(sala.agenda())
print(sala.ocupada_a("10:59"), sala.ocupada_a("11:30"), sala.ocupada_a("15:15"))
print("Ocupación:", sala.ocupacion(), "%")
f = FranjaHoraria("08:45", "09:20")
print(f, len(f), "09:20" in f)
```

Salida esperada:

```
OK 10:00-11:30
OK 09:00-10:00
RECHAZADA: solapa con 10:00-11:30 (Ana)
RECHAZADA: aforo
OK 15:15-16:00
09:00-10:00 · Luis (4 pers.)
10:00-11:30 · Ana (6 pers.)
15:15-16:00 · Iker (7 pers.)
Ana None Iker
Ocupación: 32.5 %
08:45-09:20 35 False
```

✅ Solución

```python
class FranjaHoraria:
    def __init__(self, inicio, fin):
        self.__inicio = self._a_minutos(inicio)
        self.__fin = self._a_minutos(fin)

    def _a_minutos(self, texto):
        horas, minutos = texto.split(":")
        return int(horas) * 60 + int(minutos)

    def __len__(self):
        return self.__fin - self.__inicio

    def __contains__(self, hora):
        return self.__inicio <= self._a_minutos(hora) < self.__fin   # incluye inicio, excluye fin

    def solapa(self, otra):
        return self.__inicio < otra.__fin and otra.__inicio < self.__fin

    def __lt__(self, otra):
        return self.__inicio < otra.__inicio

    def __str__(self):
        a, b = self.__inicio, self.__fin
        return f"{a // 60:02d}:{a % 60:02d}-{b // 60:02d}:{b % 60:02d}"

class Sala:
    def __init__(self, nombre, capacidad):
        self.nombre = nombre
        self.capacidad = capacidad
        self.__reservas = []                          # [(franja, responsable, asistentes)]

    def reservar(self, inicio, fin, responsable, asistentes):
        if asistentes > self.capacidad:
            return "RECHAZADA: aforo"
        nueva = FranjaHoraria(inicio, fin)
        for franja, resp, _ in self.__reservas:
            if nueva.solapa(franja):
                return f"RECHAZADA: solapa con {franja} ({resp})"
        self.__reservas.append((nueva, responsable, asistentes))
        return f"OK {nueva}"

    def ocupada_a(self, hora):
        for franja, resp, _ in self.__reservas:
            if hora in franja:
                return resp
        return None

    def ocupacion(self, jornada_min=600):
        return round(sum(len(f) for f, _, _ in self.__reservas) / jornada_min * 100, 1)

    def agenda(self):
        ordenadas = sorted(self.__reservas, key=lambda r: r[0])     # usa __lt__ de FranjaHoraria
        return "\n".join(f"{f} · {resp} ({n} pers.)" for f, resp, n in ordenadas)

sala = Sala("Atlas", 8)
```

```
print(sala.reservar("10:00", "11:30", "Ana", 6))
print(sala.reservar("09:00", "10:00", "Luis", 4))
print(sala.reservar("11:00", "12:00", "Marta", 5))
print(sala.reservar("12:00", "12:45", "Iker", 12))
print(sala.reservar("15:15", "16:00", "Iker", 7))
print(sala.agenda())
print(sala.ocupada_a("10:59"), sala.ocupada_a("11:30"), sala.ocupada_a("15:15"))
print("Ocupación:", sala.ocupacion(), "%")
f = FranjaHoraria("08:45", "09:20")
print(f, len(f), "09:20" in f)
```

🖥️ **Salida:**

```
OK 10:00-11:30
OK 09:00-10:00
RECHAZADA: solapa con 10:00-11:30 (Ana)
RECHAZADA: aforo
OK 15:15-16:00
09:00-10:00 · Luis (4 pers.)
10:00-11:30 · Ana (6 pers.)
15:15-16:00 · Iker (7 pers.)
Ana None Iker
Ocupación: 32.5 %
08:45-09:20 35 False
```

Ejercicio I5 - Depura el registro de equipos de proyecto

La PMO de una consultora registra los equipos de proyecto y las horas asignadas. Se quiere:

- Que cada equipo tenga **sus propios** miembros y **sus propias** horas.
- Que la clase lleve la cuenta de **todos** los equipos creados (`equipos_creados`) y de las horas de **toda** la empresa (`horas_empresa`). El programador que escribió la clase cometió **tres errores** relacionados con los atributos de clase y de instancia:

```
class EquipoProyecto:
    miembros = []
    horas_empresa = 0
    equipos_creados = 0

    def __init__(self, nombre):
        self.nombre = nombre
        self.horas = 0
        self.equipos_creados += 1

    def incorporar(self, persona, horas):
        self.miembros.append(persona)
        self.horas += horas
        self.horas_empresa += horas

    def __str__(self):
        return f"{self.nombre}:{self.miembros} ·{self.horas} h"
```

Al ejecutarlo con el código de prueba de abajo, la salida es esta (incorrecta):

```
Data Lake: ['Ana', 'Luis', 'Sara'] · 200 h
Churn ML: ['Ana', 'Luis', 'Sara'] · 60 h
0 0
True
{'nombre': 'Data Lake', 'horas': 200, 'equipos_creados': 1, 'horas_empresa': 200}
```

Se pide:

1. **Antes de tocar nada**, escribe en un comentario al principio del archivo por qué cada línea de la salida es incorrecta.
2. Explica en el comentario por qué `vars(datos)` contiene `equipos_creados` y `horas_empresa` si están declarados como atributos **de clase**.
3. Corrige la clase para que el código de prueba produzca la salida esperada. **Código de prueba:**

```
datos = EquipoProyecto("Data Lake")
ml = EquipoProyecto("Churn ML")
datos.incorporar("Ana", 120)
datos.incorporar("Luis", 80)
ml.incorporar("Sara", 60)
print(datos)
print(ml)
print(EquipoProyecto.equipos_creados, EquipoProyecto.horas_empresa)
print(datos.miembros is ml.miembros)
print(vars(datos))
```

Salida esperada del código corregido:

```
Data Lake: ['Ana', 'Luis'] · 200 h
Churn ML: ['Sara'] · 60 h
2 260
False
{'nombre': 'Data Lake', 'horas': 200, 'miembros': ['Ana', 'Luis']}
```

✅ Solución

```
# DIAGNÓSTICO (salida incorrecta):
# 1) Las dos listas de miembros son iguales (['Ana','Luis','Sara']): "miembros = []" es un
#    atributo de CLASE y append() modifica esa única lista compartida por todos los equipos.
# 2) "0 0": self.equipos_creados += 1 y self.horas_empresa += horas LEEN el valor de la clase
#    pero ESCRIBEN un atributo nuevo en la instancia → la clase se queda en 0.
# 3) "True": datos.miembros y ml.miembros son el mismo objeto (la lista de clase).
# 4) vars(datos) muestra equipos_creados y horas_empresa porque "self.x += 1" equivale a
#    "self.x = self.x + 1": la asignación con self. crea un atributo de INSTANCIA que tapa al
#    de clase. Y no muestra miembros porque ese nunca se asignó con self.
class EquipoProyecto:
    horas_empresa = 0                # de clase: toda la empresa
    equipos_creados = 0              # de clase: todos los equipos

    def __init__(self, nombre):
        self.nombre = nombre
        self.horas = 0
        self.miembros = []           # CORRECCIÓN 1: lista propia de cada equipo
        EquipoProyecto.equipos_creados += 1      # CORRECCIÓN 2: escribir en la clase

    def incorporar(self, persona, horas):
        self.miembros.append(persona)
        self.horas += horas
        EquipoProyecto.horas_empresa += horas    # CORRECCIÓN 3: escribir en la clase

    def __str__(self):
        return f"{self.nombre}: {self.miembros} · {self.horas} h"

datos = EquipoProyecto("Data Lake")
ml = EquipoProyecto("Churn ML")
datos.incorporar("Ana", 120)
datos.incorporar("Luis", 80)
ml.incorporar("Sara", 60)
print(datos)
print(ml)
print(EquipoProyecto.equipos_creados, EquipoProyecto.horas_empresa)
print(datos.miembros is ml.miembros)
print(vars(datos))
```

🖥️ Salida:

```
Data Lake: ['Ana', 'Luis'] · 200 h
Churn ML: ['Sara'] · 60 h
2 260
False
{'nombre': 'Data Lake', 'horas': 200, 'miembros': ['Ana', 'Luis']}
```

Ejercicio I6 - Precios en varias divisas

Un marketplace vende en euros, dólares y libras. Hasta ahora los precios se guardaban como `float` y se sumaban sin convertir, con resultados absurdos. Crea la clase `Precio`:

- Atributo de clase `TASAS_A_EUR = {"EUR": 1.0, "USD": 0.92, "GBP": 1.17}` (cuántos euros vale 1 unidad de cada divisa).
- Constructor con `cantidad` y `divisa` (por defecto `"EUR"`). La cantidad se guarda redondeada a 2 decimales.
- Método `en_euros()`: la cantidad convertida a euros (sin redondear).
- Método `convertir_a(divisa)`: devuelve un **`Precio` nuevo** en la divisa indicada, redondeado a 2 decimales.
- `+`: suma dos precios y devuelve el resultado **en la divisa del primero**.
- : multiplica un precio por un número.
- `==` y `<`: comparan el valor **en euros** (para `==`, redondeado a 2 decimales).
- `float(precio)`: el valor en euros redondeado a 2 decimales.
- Método de clase `actualizar_tasa(divisa, tasa)` con `@classmethod`.
- `__str__`: `"12.50 USD"` (siempre 2 decimales). **Código de prueba:**

```
auriculares = Precio(59.90)
cable = Precio(12.50, "USD")
funda = Precio(15, "GBP")
print(auriculares + cable)
print(cable + auriculares)
print(funda.convertir_a("EUR"), funda.convertir_a("USD"))
print(cable * 3)
print(Precio(9.20) == Precio(10, "USD"), Precio(9.20) is Precio(10, "USD"))
catalogo = [auriculares, cable, funda]
print("Más barato:", min(catalogo), "| Más caro:", max(catalogo))
print("Total en EUR:", round(sum([float(p) for p in catalogo]), 2))
Precio.actualizar_tasa("USD", 0.95)
print(cable.convertir_a("EUR"), float(cable))
```

Salida esperada:

```
71.40 EUR
77.61 USD
17.55 EUR 19.08 USD
37.50 USD
True False
Más barato: 12.50 USD | Más caro: 59.90 EUR
Total en EUR: 88.95
11.88 EUR 11.88
```

✅ Solución

```
class Precio:
    TASAS_A_EUR = {"EUR": 1.0, "USD": 0.92, "GBP": 1.17}

    def __init__(self, cantidad, divisa="EUR"):
        self.cantidad = round(cantidad, 2)
        self.divisa = divisa

    def en_euros(self):
        return self.cantidad * Precio.TASAS_A_EUR[self.divisa]

    def convertir_a(self, divisa):
        return Precio(self.en_euros() / Precio.TASAS_A_EUR[divisa], divisa)

    def __add__(self, otro):                    # resultado en la divisa del PRIMERO
        return Precio(self.cantidad + otro.convertir_a(self.divisa).cantidad, self.divisa)

    def __mul__(self, numero):
        return Precio(self.cantidad * numero, self.divisa)

    def __eq__(self, otro):
        return round(self.en_euros(), 2) == round(otro.en_euros(), 2)

    def __lt__(self, otro):
```

```
        return self.en_euros() < otro.en_euros()

    def __float__(self):
        return round(self.en_euros(), 2)

    @classmethod
    def actualizar_tasa(cls, divisa, tasa):
        cls.TASAS_A_EUR[divisa] = tasa

    def __str__(self):
        return f"{self.cantidad:.2f} {self.divisa}"

auriculares = Precio(59.90)
cable = Precio(12.50, "USD")
funda = Precio(15, "GBP")
print(auriculares + cable)
print(cable + auriculares)
print(funda.convertir_a("EUR"), funda.convertir_a("USD"))
print(cable * 3)
print(Precio(9.20) == Precio(10, "USD"), Precio(9.20) is Precio(10, "USD"))
catalogo = [auriculares, cable, funda]
print("Más barato:", min(catalogo), "| Más caro:", max(catalogo))
print("Total en EUR:", round(sum([float(p) for p in catalogo]), 2))
Precio.actualizar_tasa("USD", 0.95)
print(cable.convertir_a("EUR"), float(cable))
```

🖥️ **Salida:**

```
71.40 EUR
77.61 USD
17.55 EUR 19.08 USD
37.50 USD
True False
Más barato: 12.50 USD | Más caro: 59.90 EUR
Total en EUR: 88.95
11.88 EUR 11.88
```

Sección 3 - Nivel avanzado

Ejercicio A1 - Mini framework de pipelines ETL - Ingeniería de Datos

El equipo de datos de una cadena de supermercados quiere un pequeño framework propio para encadenar pasos de limpieza sobre registros (diccionarios) y medir cuántos registros entran y salen de cada paso. Aprovecha que **en Python las funciones son objetos**: se pueden guardar en atributos y llamar después.

Clase `Etapa`

- Atributo de clase `TIPOS = ("transformar", "filtrar")`.
- Constructor con `nombre`, `funcion` y `tipo` (por defecto `"transformar"`).
- En una etapa `transformar`, `funcion(registro)` devuelve el registro modificado.
- En una etapa `filtrar`, `funcion(registro)` devuelve `True` si el registro **se queda**.
- Método `aplicar(registros)`: recibe una lista y devuelve una lista **nueva**. Las transformaciones se aplican sobre una **copia** de cada registro (`dict(registro)`) para no modificar los datos de entrada.
- `__repr__`: `Etapa(limpiar_tienda, transformar)`. **Clase `Pipeline`**
- Constructor con `nombre`. Lista **privada** de etapas y lista **privada** de métricas.
- `+=` añade una `Etapa`. Si se intenta añadir algo que no es `Etapa` (compruébalo con `type(x).__name__`), se ignora.
- `len(pipeline)`: número de etapas.
- Método `ejecutar(registros)`: aplica las etapas en orden, guarda en las métricas una tupla `(nombre_etapa, entrada, salida)` por etapa y devuelve el resultado final. Cada ejecución reinicia las métricas.

- Método `informe()`: texto con una línea por etapa y una línea final con los descartados en total (ver salida).
- `__str__`: `"Pipeline ventas_diarias: limpiar_tienda → solo_completados → importe_con_iva → solo_grandes"`. **Código de prueba:**

```
def limpiar_tienda(r):
    r["tienda"] = r["tienda"].strip().upper()
    return r

def es_completado(r):
    return r["estado"] == "completado"

def con_iva(r):
    r["importe_iva"] = round(r["importe"] * 1.21, 2)
    return r

def es_grande(r):
    return r["importe_iva"] >= 50

ventas = [
    {"id": 1, "tienda": " mad-01", "estado": "completado", "importe": 120.0},
    {"id": 2, "tienda": "bcn-02 ", "estado": "cancelado", "importe": 80.0},
    {"id": 3, "tienda": "mad-01", "estado": "completado", "importe": 35.5},
    {"id": 4, "tienda": " vlc-03 ", "estado": "completado", "importe": 41.4},
    {"id": 5, "tienda": "bcn-02", "estado": "devuelto", "importe": 15.0},
    {"id": 6, "tienda": "vlc-03", "estado": "completado", "importe": 260.0},
]

p = Pipeline("ventas_diarias")
p += Etapa("limpiar_tienda", limpiar_tienda)
p += Etapa("solo_completados", es_completado, "filtrar")
p += Etapa("importe_con_iva", con_iva)
p += Etapa("solo_grandes", es_grande, "filtrar")
p += "esto no es una etapa"
print(p, "|", len(p), "etapas")
resultado = p.ejecutar(ventas)
for r in resultado:
    print(r)
print(p.informe())
print(ventas[0])
print(type(limpiar_tienda).__name__, Etapa("x", con_iva))
```

Salida esperada:

```
Pipeline ventas_diarias: limpiar_tienda → solo_completados → importe_con_iva → solo_grandes | 4 etapas
{'id': 1, 'tienda': 'MAD-01', 'estado': 'completado', 'importe': 120.0, 'importe_iva': 145.2}
{'id': 4, 'tienda': 'VLC-03', 'estado': 'completado', 'importe': 41.4, 'importe_iva': 50.09}
{'id': 6, 'tienda': 'VLC-03', 'estado': 'completado', 'importe': 260.0, 'importe_iva': 314.6}
Informe de ventas_diarias
  limpiar_tienda       6 →   6  (-0)
  solo_completados     6 →   4  (-2)
  importe_con_iva      4 →   4  (-0)
  solo_grandes         4 →   3  (-1)
  Descartados en total: 3
{'id': 1, 'tienda': ' mad-01', 'estado': 'completado', 'importe': 120.0}
function Etapa(x, transformar)
```

✅ Solución

```
class Etapa:
    TIPOS = ("transformar", "filtrar")

    def __init__(self, nombre, funcion, tipo="transformar"):
        self.nombre = nombre
        self.funcion = funcion          # ¡una función guardada en un atributo!
        self.tipo = tipo

    def aplicar(self, registros):
        if self.tipo == "filtrar":
            return [r for r in registros if self.funcion(r)]
```

```
        return [self.funcion(dict(r)) for r in registros]    # copia → no toca la entrada

    def __repr__(self):
        return f"Etapa({self.nombre}, {self.tipo})"

class Pipeline:
    def __init__(self, nombre):
        self.nombre = nombre
        self.__etapas = []
        self.__metricas = []

    def __iadd__(self, etapa):
        if type(etapa).__name__ == "Etapa":
            self.__etapas.append(etapa)
        return self

    def __len__(self):
        return len(self.__etapas)

    def ejecutar(self, registros):
        self.__metricas = []
        datos = registros
        for etapa in self.__etapas:
            salida = etapa.aplicar(datos)
            self.__metricas.append((etapa.nombre, len(datos), len(salida)))
            datos = salida
        return datos

    def informe(self):
        lineas = [f"Informe de {self.nombre}"]
        lineas += [f"  {n:<20}{e:>2} → {s:>3}  (-{e - s})" for n, e, s in self.__metricas]
        descartados = self.__metricas[0][1] - self.__metricas[-1][2] if self.__metricas else 0
        lineas.append(f"  Descartados en total: {descartados}")
        return "\n".join(lineas)

    def __str__(self):
        return f"Pipeline {self.nombre}: " + " → ".join(e.nombre for e in self.__etapas)

def limpiar_tienda(r):
    r["tienda"] = r["tienda"].strip().upper()
    return r

def es_completado(r):
    return r["estado"] == "completado"

def con_iva(r):
    r["importe_iva"] = round(r["importe"] * 1.21, 2)
    return r

def es_grande(r):
    return r["importe_iva"] >= 50

ventas = [
    {"id": 1, "tienda": " mad-01", "estado": "completado", "importe": 120.0},
    {"id": 2, "tienda": "bcn-02 ", "estado": "cancelado", "importe": 80.0},
    {"id": 3, "tienda": "mad-01", "estado": "completado", "importe": 35.5},
    {"id": 4, "tienda": " vlc-03 ", "estado": "completado", "importe": 41.4},
    {"id": 5, "tienda": "bcn-02", "estado": "devuelto", "importe": 15.0},
    {"id": 6, "tienda": "vlc-03", "estado": "completado", "importe": 260.0},
]

p = Pipeline("ventas_diarias")
p += Etapa("limpiar_tienda", limpiar_tienda)
p += Etapa("solo_completados", es_completado, "filtrar")
p += Etapa("importe_con_iva", con_iva)
p += Etapa("solo_grandes", es_grande, "filtrar")
p += "esto no es una etapa"
print(p, "|", len(p), "etapas")
resultado = p.ejecutar(ventas)
for r in resultado:
    print(r)
print(p.informe())
```

```
print(ventas[0])
print(type(limpiar_tienda).__name__, Etapa("x", con_iva))
```

🖥️ **Salida:**

```
Pipeline ventas_diarias: limpiar_tienda → solo_completados → importe_con_iva → solo_grandes | 4 etapas
{'id': 1, 'tienda': 'MAD-01', 'estado': 'completado', 'importe': 120.0, 'importe_iva': 145.2}
{'id': 4, 'tienda': 'VLC-03', 'estado': 'completado', 'importe': 41.4, 'importe_iva': 50.09}
{'id': 6, 'tienda': 'VLC-03', 'estado': 'completado', 'importe': 260.0, 'importe_iva': 314.6}
Informe de ventas_diarias
  limpiar_tienda       6 →   6  (-0)
  solo_completados     6 →   4  (-2)
  importe_con_iva      4 →   4  (-0)
  solo_grandes         4 →   3  (-1)
  Descartados en total: 3
{'id': 1, 'tienda': ' mad-01', 'estado': 'completado', 'importe': 120.0}
function Etapa(x, transformar)
```

Ejercicio A2 - Simulador de MapReduce sobre logs web - Big Data

Antes de lanzar trabajos en el clúster Spark, el equipo de Big Data de un e-commerce quiere que los nuevos ingenieros entiendan cómo funciona el modelo **MapReduce**: los datos se reparten en **particiones**, cada partición se procesa por separado (*map*), se agregan resultados parciales en cada nodo (*combiner*) y finalmente se fusionan (*reduce*). El tráfico de red del *shuffle* depende de cuántos pares viajan entre nodos.

Cada línea de log tiene 5 campos separados por espacios: `timestamp país endpoint estado bytes`.

Clase `ConteoParcial` (resultado agregado clave → valor)

- Constructor sin parámetros: crea un diccionario **privado** vacío.
- Método `sumar(clave, valor)`.
- Método `get(clave)`: valor de la clave o `0`.
- `+`: devuelve un `ConteoParcial` **nuevo** que fusiona los dos (sumando los valores de las claves comunes). No modifica los originales.
- `len()`: número de claves distintas. Método `total()`: suma de todos los valores.
- Método `top(n)`: lista de las `n` tuplas `(clave, valor)` con mayor valor; a igual valor, por orden alfabético de clave. **Clase `Particion`**
- Constructor con `id_particion` y `lineas`. `len()` devuelve el número de líneas.
- Método `mapear(funcion_map)`: aplica la función a cada línea y devuelve **la lista de todos los pares** generados. La función de map devuelve una **lista** de pares `(clave, valor)` (vacía si la línea no es válida). **Clase `TrabajoMapReduce`**
- Constructor con `nombre`, `funcion_map` y `usar_combinador` (por defecto `True`).
- Método `ejecutar(particiones)`:
- Para cada partición: obtiene los pares y los agrega en un `ConteoParcial` local.
- Pares enviados al shuffle por la partición: si hay combinador, el número de **claves** del parcial; si no, el número de **pares**.
- Fusiona todos los parciales con `+` y devuelve el `ConteoParcial` final.
- Guarda en una lista **privada** una tupla por partición `(id, lineas, pares, enviados)`.
- Método `estadisticas()`: texto como el de la salida esperada. Escribe además tres **funciones globales de map** (las líneas mal formadas, que no tienen 5 campos, no generan pares):
- `peticiones_por_pais(linea)` → `[(pais, 1)]`.
- `bytes_ok_por_endpoint(linea)` → `[(endpoint, bytes)]` solo si el estado es `200`.

- `errores_por_endpoint(linea)` → `[(endpoint, 1)]` solo si el estado es `500` o superior. **Código de prueba:**

```
particiones = [
    Particion("p0", ["10:01 ES /home 200 5120", "10:01 ES /checkout 200 830",
                     "10:02 FR /home 200 4980", "10:02 ES /home 500 0"]),
    Particion("p1", ["10:03 ES /search 200 2210", "10:03 PT /home 200 5003",
                     "linea corrupta", "10:04 ES /checkout 503 0", "10:04 FR /search 200 1990"]),
    Particion("p2", ["10:05 ES /home 200 5120", "10:05 ES /home 200 5120",
                     "10:06 ES /home 200 5120", "10:06 FR /checkout 500 0"]),
]

paises = TrabajoMapReduce("peticiones_por_pais", peticiones_por_pais)
resultado = paises.ejecutar(particiones)
print(paises.estadisticas())
print("Top 2:", resultado.top(2), "| Total:", resultado.total(), "| PT:", resultado.get("PT"), "| IT:",
resultado.get("IT"))

sin_comb = TrabajoMapReduce("peticiones_por_pais", peticiones_por_pais, usar_combinador=False)
resultado_2 = sin_comb.ejecutar(particiones)
print(sin_comb.estadisticas())
print("Mismo resultado:", resultado_2.top(10) == resultado.top(10))

volumen = TrabajoMapReduce("bytes_ok", bytes_ok_por_endpoint).ejecutar(particiones)
print("Bytes OK:", volumen.top(3))
errores = TrabajoMapReduce("errores", errores_por_endpoint).ejecutar(particiones)
print("Errores:", errores.top(5), "| endpoints con error:", len(errores))
```

Salida esperada:

```
Trabajo peticiones_por_pais (con combinador)
  p0: 4 líneas → 4 pares → 2 enviados
  p1: 5 líneas → 4 pares → 3 enviados
  p2: 4 líneas → 4 pares → 2 enviados
  Shuffle total: 7 pares
Top 2: [('ES', 8), ('FR', 3)] | Total: 12 | PT: 1 | IT: 0
Trabajo peticiones_por_pais (sin combinador)
  p0: 4 líneas → 4 pares → 4 enviados
  p1: 5 líneas → 4 pares → 4 enviados
  p2: 4 líneas → 4 pares → 4 enviados
  Shuffle total: 12 pares
Mismo resultado: True
Bytes OK: [('/home', 30463), ('/search', 4200), ('/checkout', 830)]
Errores: [('/checkout', 2), ('/home', 1)] | endpoints con error: 2
```

✅ Solución

```
class ConteoParcial:
    def __init__(self):
        self.__datos = {}

    def sumar(self, clave, valor):
        self.__datos[clave] = self.__datos.get(clave, 0) + valor

    def get(self, clave):
        return self.__datos.get(clave, 0)

    def items(self):
        return self.__datos.items()

    def __add__(self, otro):
        nuevo = ConteoParcial()
        for origen in (self, otro):
            for clave, valor in origen.items():
                nuevo.sumar(clave, valor)
        return nuevo

    def __len__(self):
        return len(self.__datos)

    def total(self):
        return sum(self.__datos.values())
```

```
    def top(self, n):
        return sorted(self.__datos.items(), key=lambda kv: (-kv[1], kv[0]))[:n]

class Particion:
    def __init__(self, id_particion, lineas):
        self.id_particion = id_particion
        self.lineas = lineas

    def __len__(self):
        return len(self.lineas)

    def mapear(self, funcion_map):
        return [par for linea in self.lineas for par in funcion_map(linea)]

class TrabajoMapReduce:
    def __init__(self, nombre, funcion_map, usar_combinador=True):
        self.nombre = nombre
        self.funcion_map = funcion_map
        self.usar_combinador = usar_combinador
        self.__stats = []

    def ejecutar(self, particiones):
        self.__stats = []
        final = ConteoParcial()
        for part in particiones:
            pares = part.mapear(self.funcion_map)                # MAP
            local = ConteoParcial()
            for clave, valor in pares:
                local.sumar(clave, valor)                         # COMBINER
            enviados = len(local) if self.usar_combinador else len(pares)
            self.__stats.append((part.id_particion, len(part), len(pares), enviados))
            final = final + local                                 # REDUCE
        return final

    def estadisticas(self):
        modo = "con" if self.usar_combinador else "sin"
        lineas = [f"Trabajo {self.nombre} ({modo} combinador)"]
        lineas += [f"  {i}: {l} líneas → {p} pares → {e} enviados" for i, l, p, e in self.__stats]
        lineas.append(f"  Shuffle total: {sum(s[3] for s in self.__stats)} pares")
        return "\n".join(lineas)

def _campos(linea):
    campos = linea.split()
    return campos if len(campos) == 5 else None

def peticiones_por_pais(linea):
    c = _campos(linea)
    return [(c[1], 1)] if c else []

def bytes_ok_por_endpoint(linea):
    c = _campos(linea)
    return [(c[2], int(c[4]))] if c and c[3] == "200" else []

def errores_por_endpoint(linea):
    c = _campos(linea)
    return [(c[2], 1)] if c and int(c[3]) >= 500 else []

particiones = [
    Particion("p0", ["10:01 ES /home 200 5120", "10:01 ES /checkout 200 830",
                     "10:02 FR /home 200 4980", "10:02 ES /home 500 0"]),
    Particion("p1", ["10:03 ES /search 200 2210", "10:03 PT /home 200 5003",
                     "linea corrupta", "10:04 ES /checkout 503 0", "10:04 FR /search 200 1990"]),
    Particion("p2", ["10:05 ES /home 200 5120", "10:05 ES /home 200 5120",
                     "10:06 ES /home 200 5120", "10:06 FR /checkout 500 0"]),
]

paises = TrabajoMapReduce("peticiones_por_pais", peticiones_por_pais)
resultado = paises.ejecutar(particiones)
print(paises.estadisticas())
print("Top 2:", resultado.top(2), "| Total:", resultado.total(), "| PT:", resultado.get("PT"), "| IT:",
resultado.get("IT"))
```

```
sin_comb = TrabajoMapReduce("peticiones_por_pais", peticiones_por_pais, usar_combinador=False)
resultado_2 = sin_comb.ejecutar(particiones)
print(sin_comb.estadisticas())
print("Mismo resultado:", resultado_2.top(10) == resultado.top(10))

volumen = TrabajoMapReduce("bytes_ok", bytes_ok_por_endpoint).ejecutar(particiones)
print("Bytes OK:", volumen.top(3))
errores = TrabajoMapReduce("errores", errores_por_endpoint).ejecutar(particiones)
print("Errores:", errores.top(5), "| endpoints con error:", len(errores))
```

🖥️ **Salida:**

```
Trabajo peticiones_por_pais (con combinador)
  p0: 4 líneas → 4 pares → 2 enviados
  p1: 5 líneas → 4 pares → 3 enviados
  p2: 4 líneas → 4 pares → 2 enviados
  Shuffle total: 7 pares
Top 2: [('ES', 8), ('FR', 3)] | Total: 12 | PT: 1 | IT: 0
Trabajo peticiones_por_pais (sin combinador)
  p0: 4 líneas → 4 pares → 4 enviados
  p1: 5 líneas → 4 pares → 4 enviados
  p2: 4 líneas → 4 pares → 4 enviados
  Shuffle total: 12 pares
Mismo resultado: True
Bytes OK: [('/home', 30463), ('/search', 4200), ('/checkout', 830)]
Errores: [('/checkout', 2), ('/home', 1)] | endpoints con error: 2
```

Ejercicio A3 - Regresión lineal desde cero - Machine Learning

El equipo de marketing de una cadena de gimnasios quiere estimar cuántas **altas mensuales** se consiguen según la **inversión en publicidad digital** (miles de euros). Antes de usar scikit-learn, vas a programar el modelo a mano para entender qué hay dentro de `fit()` y `predict()`.

Fórmulas de mínimos cuadrados para `y = pendiente · x + intercepto`:

- `pendiente = Σ (xᵢ − x̄)(yᵢ − ȳ) / Σ (xᵢ − x̄)²`
- `intercepto = ȳ − pendiente · x̄`
- `R² = 1 − Σ (yᵢ − ŷᵢ)² / Σ (yᵢ − ȳ)²`
- `MAE = Σ |yᵢ − ŷᵢ| / n` **Clase `ConjuntoDatos`**
- Constructor con `x`, `y` (listas de igual longitud) y `nombre_x`, `nombre_y`.
- `len()`: número de observaciones.
- Método `dividir(proporcion_train=0.75)`: devuelve una tupla `(train, test)` de **dos objetos `ConjuntoDatos` nuevos**. Las primeras `int(len * proporcion)` observaciones van a train y el resto a test (sin desordenar, para que el resultado sea reproducible).
- `__repr__`: `ConjuntoDatos(inversion_k€ → altas, n=12)`. **Clase `RegresionLinealSimple`**
- Constructor sin datos: `pendiente` e `intercepto` valen `None`. Atributo de clase `modelos_entrenados = 0`.
- Método `entrenar(datos)`: calcula pendiente e intercepto con las fórmulas, suma 1 a `modelos_entrenados` y **devuelve `self`** (así se puede escribir `modelo = RegresionLinealSimple().entrenar(train)`).
- Método `predecir(x)`: si `x` es una lista, devuelve la lista de predicciones; si es un número, una sola predicción. Redondea a 2 decimales. Comprueba el tipo con `type(x).__name__`.
- Métodos `r2(datos)` y `mae(datos)`, ambos redondeados a 3 decimales.
- `bool(modelo)`: `True` si el modelo está entrenado.
- `__str__`: `"altas = 18.42 · inversion_k€ + 31.05"` (2 decimales) o `"Modelo sin entrenar"`. **Código de prueba:**

```
inversion = [2.0, 3.5, 1.0, 4.0, 5.5, 3.0, 6.0, 2.5, 4.5, 7.0, 5.0, 1.5]
altas = [68, 95, 49, 112, 133, 88, 150, 74, 120, 162, 128, 60]
datos = ConjuntoDatos(inversion, altas, "inversion_k€", "altas")
train, test = datos.dividir(0.75)
print(datos, train, test)

modelo = RegresionLinealSimple()
print(modelo, bool(modelo))
modelo = modelo.entrenar(train)
print(modelo, bool(modelo))
print("R² train:", modelo.r2(train), "| R² test:", modelo.r2(test))
print("MAE test:", modelo.mae(test), "altas")
print("Predicción test:", modelo.predecir(test.x), "| real:", test.y)
print("¿Y si invertimos 8 k€?", modelo.predecir(8.0))
completo = RegresionLinealSimple().entrenar(datos)
print(completo, "| modelos entrenados:", RegresionLinealSimple.modelos_entrenados)
```

Salida esperada:

```
ConjuntoDatos(inversion_k€ → altas, n=12) ConjuntoDatos(inversion_k€ → altas, n=9) ConjuntoDatos(inversion_k€
→ altas, n=3)
Modelo sin entrenar False
altas = 20.03 · inversion_k€ + 27.55 True
R² train: 0.991 | R² test: 0.993
MAE test: 2.82 altas
Predicción test: [167.77, 127.71, 57.6] | real: [162, 128, 60]
¿Y si invertimos 8 k€? 187.81
altas = 19.42 · inversion_k€ + 29.62 | modelos entrenados: 2
```

✅ Solución

```
class ConjuntoDatos:
    def __init__(self, x, y, nombre_x, nombre_y):
        self.x, self.y = x, y
        self.nombre_x, self.nombre_y = nombre_x, nombre_y

    def __len__(self):
        return len(self.x)

    def dividir(self, proporcion_train=0.75):
        corte = int(len(self) * proporcion_train)
        train = ConjuntoDatos(self.x[:corte], self.y[:corte], self.nombre_x, self.nombre_y)
        test = ConjuntoDatos(self.x[corte:], self.y[corte:], self.nombre_x, self.nombre_y)
        return train, test

    def __repr__(self):
        return f"ConjuntoDatos({self.nombre_x} → {self.nombre_y}, n={len(self)})"

class RegresionLinealSimple:
    modelos_entrenados = 0

    def __init__(self):
        self.pendiente = None
        self.intercepto = None

    def entrenar(self, datos):
        n = len(datos)
        mx, my = sum(datos.x) / n, sum(datos.y) / n
        num = sum((xi - mx) * (yi - my) for xi, yi in zip(datos.x, datos.y))
        den = sum((xi - mx) ** 2 for xi in datos.x)
        self.pendiente = num / den
        self.intercepto = my - self.pendiente * mx
        self.nombre_x, self.nombre_y = datos.nombre_x, datos.nombre_y
        RegresionLinealSimple.modelos_entrenados += 1
        return self                                  # permite encadenar

    def _predice(self, x):
        return self.pendiente * x + self.intercepto

    def predecir(self, x):
```

```
        if type(x).__name__ == "list":
            return [round(self._predice(v), 2) for v in x]
        return round(self._predice(x), 2)

    def r2(self, datos):
        my = sum(datos.y) / len(datos)
        ss_res = sum((yi - self._predice(xi)) ** 2 for xi, yi in zip(datos.x, datos.y))
        ss_tot = sum((yi - my) ** 2 for yi in datos.y)
        return round(1 - ss_res / ss_tot, 3)

    def mae(self, datos):
        return round(sum(abs(yi - self._predice(xi)) for xi, yi in zip(datos.x, datos.y)) / len(datos), 3)

    def __bool__(self):
        return self.pendiente is not None

    def __str__(self):
        if not self:
            return "Modelo sin entrenar"
        return f"{self.nombre_y} = {self.pendiente:.2f} · {self.nombre_x} + {self.intercepto:.2f}"

inversion = [2.0, 3.5, 1.0, 4.0, 5.5, 3.0, 6.0, 2.5, 4.5, 7.0, 5.0, 1.5]
altas = [68, 95, 49, 112, 133, 88, 150, 74, 120, 162, 128, 60]
datos = ConjuntoDatos(inversion, altas, "inversion_k€", "altas")
train, test = datos.dividir(0.75)
print(datos, train, test)

modelo = RegresionLinealSimple()
print(modelo, bool(modelo))
modelo = modelo.entrenar(train)
print(modelo, bool(modelo))
print("R² train:", modelo.r2(train), "| R² test:", modelo.r2(test))
print("MAE test:", modelo.mae(test), "altas")
print("Predicción test:", modelo.predecir(test.x), "| real:", test.y)
print("¿Y si invertimos 8 k€?", modelo.predecir(8.0))
completo = RegresionLinealSimple().entrenar(datos)
print(completo, "| modelos entrenados:", RegresionLinealSimple.modelos_entrenados)
```

🖥️ **Salida:**

```
ConjuntoDatos(inversion_k€ → altas, n=12) ConjuntoDatos(inversion_k€ → altas, n=9) ConjuntoDatos(inversion_k€
→ altas, n=3)
Modelo sin entrenar False
altas = 20.03 · inversion_k€ + 27.55 True
R² train: 0.991 | R² test: 0.993
MAE test: 2.82 altas
Predicción test: [167.77, 127.71, 57.6] | real: [162, 128, 60]
¿Y si invertimos 8 k€? 187.81
altas = 19.42 · inversion_k€ + 29.62 | modelos entrenados: 2
```

Ejercicio A4 - Motor RAG mínimo para la base de conocimiento de RR. HH. - IA Generativa

Una empresa de 800 empleados quiere un asistente que responda dudas sobre sus políticas internas **sin inventar**. La técnica es **RAG** (*Retrieval-Augmented Generation*): antes de llamar al LLM se buscan los fragmentos más parecidos a la pregunta y se insertan en el prompt como contexto. En producción se usan *embeddings* y una base vectorial; aquí vas a construir una versión didáctica con **bolsa de palabras** y **similitud del coseno**.

Clase `Documento`

- Atributo de clase `STOPWORDS = {"el", "la", "los", "las", "de", "del", "en", "y", "a", "un", "una", "por", "para", "con", "que", "se", "al", "es", "lo", "mi", "tengo", "hay", "cuántos", "cuántas", "cuál", "cómo"}`.
- Constructor con `id_doc`, `titulo` y `texto`.
- Método `terminos()`: devuelve un diccionario `{palabra: frecuencia}` del texto en minúsculas, quitando los signos `.,;:¿?¡!()` y las stopwords.

- `__repr__`: `Documento(POL-01: Teletrabajo)`. **Clase `IndiceDocumental`**
- Constructor sin parámetros. Diccionario **privado** `{id_doc: Documento}`.
- `+=` añade un `Documento`. `len()` y `id_doc in indice`.
- Función de la clase **sin `self`** `similitud(v1, v2)`: similitud del coseno entre dos diccionarios de frecuencias. Si alguno está vacío, devuelve `0.0`.
- `coseno = Σ v1[t]·v2[t] / (√Σ v1[t]² · √Σ v2[t]²)`. Usa `math.sqrt`.
- Método `buscar(consulta, k=2, umbral=0.1)`: convierte la consulta a términos **con la misma limpieza que los documentos** (pista: crea un `Documento` temporal), calcula la similitud con cada documento y devuelve una lista de tuplas `(documento, puntuacion)` ordenada de mayor a menor, con como máximo `k` resultados y solo los que superen el umbral. Redondea la puntuación a 3 decimales. **Clase `AsistenteRAG`**
- Constructor con `indice` e `instrucciones` (texto del mensaje de sistema).
- Método `construir_prompt(pregunta, k=2)`:
- Si la búsqueda no devuelve nada, el prompt incluye `CONTEXTO: (sin documentos relevantes)` para que el modelo responda que no lo sabe.
- Si hay resultados, un bloque `[id_doc] texto` por documento.
- Formato completo en la salida esperada.
- Método `fuentes(pregunta, k=2)`: lista de los `id_doc` usados. **Código de prueba:**

```
indice = IndiceDocumental()
indice += Documento("POL-01", "Teletrabajo", "El teletrabajo se permite dos días por semana previa aprobación del responsable.")
indice += Documento("POL-02", "Vacaciones", "Cada empleado dispone de 23 días laborables de vacaciones al año. Las vacaciones se solicitan en el portal.")
indice += Documento("POL-03", "Gastos de viaje", "Los gastos de viaje se reembolsan con ticket. El límite de dieta es de 45 euros por día.")
indice += Documento("POL-04", "Formación", "La empresa financia certificaciones técnicas hasta 1500 euros al año.")
print(len(indice), "POL-03" in indice, "POL-99" in indice)
print(Documento("X", "", "¿Cuántos días de vacaciones tengo al año?").terminos())

for doc, puntuacion in indice.buscar("¿Cuántos días de vacaciones tengo al año?", k=3):
    print(doc, puntuacion)

asistente = AsistenteRAG(indice, "Responde solo con el contexto. Si no está, di que no lo sabes. Cita las fuentes.")
print(asistente.construir_prompt("¿Cuál es el límite de dieta en un viaje?", k=1))
print(asistente.fuentes("¿Cuántos días de vacaciones tengo al año?"))
print(asistente.construir_prompt("¿Hay parking gratuito?"))
print(IndiceDocumental.similitud({"a": 1}, {}))
```

Salida esperada:

```
4 True False
{'días': 1, 'vacaciones': 1, 'año': 1}
Documento(POL-02: Vacaciones) 0.641
Documento(POL-01: Teletrabajo) 0.204
Documento(POL-04: Formación) 0.204
SISTEMA: Responde solo con el contexto. Si no está, di que no lo sabes. Cita las fuentes.
CONTEXTO:
[POL-03] Los gastos de viaje se reembolsan con ticket. El límite de dieta es de 45 euros por día.
PREGUNTA: ¿Cuál es el límite de dieta en un viaje?
['POL-02', 'POL-01']
SISTEMA: Responde solo con el contexto. Si no está, di que no lo sabes. Cita las fuentes.
CONTEXTO: (sin documentos relevantes)
PREGUNTA: ¿Hay parking gratuito?
0.0
```

✅ Solución

```
import math

class Documento:
    STOPWORDS = {"el", "la", "los", "las", "de", "del", "en", "y", "a", "un", "una", "por",
                 "para", "con", "que", "se", "al", "es", "lo", "mi", "tengo", "hay",
                 "cuántos", "cuántas", "cuál", "cómo"}

    def __init__(self, id_doc, titulo, texto):
        self.id_doc, self.titulo, self.texto = id_doc, titulo, texto

    def terminos(self):
        limpio = self.texto.lower()
        for signo in ".,;:¿?¡!()":
            limpio = limpio.replace(signo, "")
        frecuencias = {}
        for palabra in limpio.split():
            if palabra not in Documento.STOPWORDS:
                frecuencias[palabra] = frecuencias.get(palabra, 0) + 1
        return frecuencias

    def __repr__(self):
        return f"Documento({self.id_doc}: {self.titulo})"

class IndiceDocumental:
    def __init__(self):
        self.__docs = {}

    def __iadd__(self, doc):
        self.__docs[doc.id_doc] = doc
        return self

    def __len__(self):
        return len(self.__docs)

    def __contains__(self, id_doc):
        return id_doc in self.__docs

    def similitud(v1, v2):                      # función de la clase SIN self
        if not v1 or not v2:
            return 0.0
        producto = sum(v1[t] * v2.get(t, 0) for t in v1)
        norma1 = math.sqrt(sum(v * v for v in v1.values()))
        norma2 = math.sqrt(sum(v * v for v in v2.values()))
        return producto / (norma1 * norma2)

    def buscar(self, consulta, k=2, umbral=0.1):
        v_consulta = Documento("_", "", consulta).terminos()
        puntuados = [(doc, round(IndiceDocumental.similitud(v_consulta, doc.terminos()), 3))
                     for doc in self.__docs.values()]
        relevantes = [(d, s) for d, s in puntuados if s > umbral]
        return sorted(relevantes, key=lambda par: par[1], reverse=True)[:k]

class AsistenteRAG:
    def __init__(self, indice, instrucciones):
        self.indice = indice
        self.instrucciones = instrucciones

    def construir_prompt(self, pregunta, k=2):
        resultados = self.indice.buscar(pregunta, k)
        lineas = [f"SISTEMA: {self.instrucciones}"]
        if resultados:
            lineas.append("CONTEXTO:")
            lineas += [f"[{d.id_doc}] {d.texto}" for d, _ in resultados]
        else:
            lineas.append("CONTEXTO: (sin documentos relevantes)")
        lineas.append(f"PREGUNTA: {pregunta}")
        return "\n".join(lineas)

    def fuentes(self, pregunta, k=2):
        return [d.id_doc for d, _ in self.indice.buscar(pregunta, k)]

indice = IndiceDocumental()
indice += Documento("POL-01", "Teletrabajo", "El teletrabajo se permite dos días por semana previa aprobación
```

```
del responsable.")
indice += Documento("POL-02", "Vacaciones", "Cada empleado dispone de 23 días laborables de vacaciones al
año. Las vacaciones se solicitan en el portal.")
indice += Documento("POL-03", "Gastos de viaje", "Los gastos de viaje se reembolsan con ticket. El límite de
dieta es de 45 euros por día.")
indice += Documento("POL-04", "Formación", "La empresa financia certificaciones técnicas hasta 1500 euros al
año.")
print(len(indice), "POL-03" in indice, "POL-99" in indice)
print(Documento("X", "", "¿Cuántos días de vacaciones tengo al año?").terminos())

for doc, puntuacion in indice.buscar("¿Cuántos días de vacaciones tengo al año?", k=3):
    print(doc, puntuacion)

asistente = AsistenteRAG(indice, "Responde solo con el contexto. Si no está, di que no lo sabes. Cita las
fuentes.")
print(asistente.construir_prompt("¿Cuál es el límite de dieta en un viaje?", k=1))
print(asistente.fuentes("¿Cuántos días de vacaciones tengo al año?"))
print(asistente.construir_prompt("¿Hay parking gratuito?"))
print(IndiceDocumental.similitud({"a": 1}, {}))
```

🖥️ **Salida:**

```
4 True False
{'días': 1, 'vacaciones': 1, 'año': 1}
Documento(POL-02: Vacaciones) 0.641
Documento(POL-01: Teletrabajo) 0.204
Documento(POL-04: Formación) 0.204
SISTEMA: Responde solo con el contexto. Si no está, di que no lo sabes. Cita las fuentes.
CONTEXTO:
[POL-03] Los gastos de viaje se reembolsan con ticket. El límite de dieta es de 45 euros por día.
PREGUNTA: ¿Cuál es el límite de dieta en un viaje?
['POL-02', 'POL-01']
SISTEMA: Responde solo con el contexto. Si no está, di que no lo sabes. Cita las fuentes.
CONTEXTO: (sin documentos relevantes)
PREGUNTA: ¿Hay parking gratuito?
0.0
```

Ejercicio A5 - Planificador de proyectos con dependencias

Una constructora digitaliza la planificación de sus obras. Cada tarea dura un número de días y no puede empezar hasta que terminen sus **dependencias**. Hay que calcular un orden de ejecución válido, el día de inicio y fin de cada tarea y la duración total del proyecto.

Clase `Tarea`

- Constructor con `codigo`, `descripcion`, `duracion` (días) y `dependencias` (lista de códigos, por defecto vacía). Atención: **no uses `[]` como valor por defecto** del parámetro; usa `None` y crea la lista dentro.
- Atributos `inicio` y `fin`, que empiezan en `None`.
- `__repr__`: `Tarea(T3, 5d)`. **Clase `Proyecto`**
- Constructor con `nombre`. Diccionario **privado** `{codigo: Tarea}`.
- `+=` añade una tarea. `len()` y `codigo in proyecto`.
- Método `dependencias_rotas()`: lista ordenada de textos `"T7 → T9"` para cada dependencia que apunta a una tarea que no existe.
- Método `orden_ejecucion()`: devuelve la lista de códigos en un orden válido. Algoritmo:

1. Mientras queden tareas pendientes, busca las que tengan **todas** sus dependencias ya colocadas.
2. De ellas, coloca **la de código menor** (orden alfabético) y repite.
3. Si en una vuelta ninguna tarea está lista, hay un **ciclo**: devuelve `None`.

- Método `planificar()`: recorre el orden y asigna a cada tarea `inicio` = mayor `fin` de sus dependencias (0 si no tiene) y `fin = inicio + duracion`. Devuelve `True` si pudo planificar o `False` si hay ciclo o dependencias rotas.
- Método `duracion_total()`: el mayor `fin`.

- Método `ruta_critica()`: empezando por la tarea que termina la última, va hacia atrás eligiendo en cada paso la dependencia cuyo `fin` coincide con el `inicio` de la tarea actual (si hay varias, la de código menor). Devuelve la lista de códigos desde el principio hasta el final.
- Método `diagrama()`: una línea por tarea, en orden de ejecución, con un diagrama de Gantt en texto (ver salida). **Código de prueba:**

```
obra = Proyecto("Nave logística Getafe")
obra += Tarea("T1", "Licencias", 4, ["T8"])
obra += Tarea("T2", "Cimentación", 6, ["T1"])
obra += Tarea("T3", "Estructura", 8, ["T2"])
obra += Tarea("T4", "Acometida eléctrica", 3, ["T1"])
obra += Tarea("T5", "Cubierta", 5, ["T3"])
obra += Tarea("T6", "Instalaciones", 7, ["T3", "T4"])
obra += Tarea("T7", "Acabados", 4, ["T5", "T6"])
obra += Tarea("T8", "Proyecto técnico", 3)
print(len(obra), "T6" in obra, obra.dependencias_rotas())
print(obra.orden_ejecucion())
print(obra.planificar())
print(obra.diagrama())
print("Duración total:", obra.duracion_total(), "días")
print("Ruta crítica:", " → ".join(obra.ruta_critica()))

ciclo = Proyecto("Con ciclo")
ciclo += Tarea("A", "Diseño", 2, ["C"])
ciclo += Tarea("B", "Revisión", 1, ["A"])
ciclo += Tarea("C", "Aprobación", 1, ["B"])
print(ciclo.orden_ejecucion(), ciclo.planificar())

roto = Proyecto("Incompleto")
roto += Tarea("X1", "Pedido", 2, ["X0", "X9"])
print(roto.dependencias_rotas(), roto.planificar())
print(Tarea("Z1", "Prueba", 1).dependencias is Tarea("Z2", "Prueba", 1).dependencias)
```

Salida esperada:

```
8 True []
['T8', 'T1', 'T2', 'T3', 'T4', 'T5', 'T6', 'T7']
True
T8  Proyecto técnico     |███                                  | 0-3
T1  Licencias            |   ████                              | 3-7
T2  Cimentación          |       ██████                        | 7-13
T3  Estructura           |             ████████                | 13-21
T4  Acometida eléctrica  |       ███                           | 7-10
T5  Cubierta             |                     █████           | 21-26
T6  Instalaciones        |                     ███████         | 21-28
T7  Acabados             |                            ████| 28-32
Duración total: 32 días
Ruta crítica: T8 → T1 → T2 → T3 → T6 → T7
None False
['X1 → X0', 'X1 → X9'] False
False
```

✅ Solución

```
class Tarea:
    def __init__(self, codigo, descripcion, duracion, dependencias=None):
        self.codigo = codigo
        self.descripcion = descripcion
        self.duracion = duracion
        self.dependencias = dependencias if dependencias is not None else []   # ¡nunca [] por defecto!
        self.inicio = None
        self.fin = None

    def __repr__(self):
        return f"Tarea({self.codigo}, {self.duracion}d)"

class Proyecto:
    def __init__(self, nombre):
        self.nombre = nombre
        self.__tareas = {}
```

```
    def __iadd__(self, tarea):
        self.__tareas[tarea.codigo] = tarea
        return self

    def __len__(self):
        return len(self.__tareas)

    def __contains__(self, codigo):
        return codigo in self.__tareas

    def dependencias_rotas(self):
        return sorted(f"{t.codigo} → {d}" for t in self.__tareas.values()
                      for d in t.dependencias if d not in self.__tareas)

    def orden_ejecucion(self):
        colocadas, pendientes = [], set(self.__tareas)
        while pendientes:
            listas = [c for c in pendientes
                      if all(d in colocadas for d in self.__tareas[c].dependencias)]
            if not listas:
                return None                              # ciclo (o dependencia imposible)
            elegida = min(listas)
            colocadas.append(elegida)
            pendientes.remove(elegida)
        return colocadas

    def planificar(self):
        orden = self.orden_ejecucion()
        if orden is None or self.dependencias_rotas():
            return False
        for codigo in orden:
            t = self.__tareas[codigo]
            t.inicio = max((self.__tareas[d].fin for d in t.dependencias), default=0)
            t.fin = t.inicio + t.duracion
        return True

    def duracion_total(self):
        return max(t.fin for t in self.__tareas.values())

    def ruta_critica(self):
        actual = max(self.__tareas.values(), key=lambda t: t.fin)
        ruta = [actual.codigo]
        while actual.dependencias:
            candidatas = sorted(d for d in actual.dependencias
                                if self.__tareas[d].fin == actual.inicio)
            actual = self.__tareas[candidatas[0]]
            ruta.append(actual.codigo)
        return ruta[::-1]

    def diagrama(self):
        total = self.duracion_total()
        lineas = []
        for codigo in self.orden_ejecucion():
            t = self.__tareas[codigo]
            barra = " " * t.inicio + "█" * t.duracion + " " * (total - t.fin)
            lineas.append(f"{t.codigo:<4}{t.descripcion:<21}|{barra}| {t.inicio}-{t.fin}")
        return "\n".join(lineas)

obra = Proyecto("Nave logística Getafe")
obra += Tarea("T1", "Licencias", 4, ["T8"])
obra += Tarea("T2", "Cimentación", 6, ["T1"])
obra += Tarea("T3", "Estructura", 8, ["T2"])
obra += Tarea("T4", "Acometida eléctrica", 3, ["T1"])
obra += Tarea("T5", "Cubierta", 5, ["T3"])
obra += Tarea("T6", "Instalaciones", 7, ["T3", "T4"])
obra += Tarea("T7", "Acabados", 4, ["T5", "T6"])
obra += Tarea("T8", "Proyecto técnico", 3)
print(len(obra), "T6" in obra, obra.dependencias_rotas())
print(obra.orden_ejecucion())
print(obra.planificar())
print(obra.diagrama())
print("Duración total:", obra.duracion_total(), "días")
```

```
print("Ruta crítica:", " → ".join(obra.ruta_critica()))

ciclo = Proyecto("Con ciclo")
ciclo += Tarea("A", "Diseño", 2, ["C"])
ciclo += Tarea("B", "Revisión", 1, ["A"])
ciclo += Tarea("C", "Aprobación", 1, ["B"])
print(ciclo.orden_ejecucion(), ciclo.planificar())

roto = Proyecto("Incompleto")
roto += Tarea("X1", "Pedido", 2, ["X0", "X9"])
print(roto.dependencias_rotas(), roto.planificar())
print(Tarea("Z1", "Prueba", 1).dependencias is Tarea("Z2", "Prueba", 1).dependencias)
```

🖥️ **Salida:**

```
8 True []
['T8', 'T1', 'T2', 'T3', 'T4', 'T5', 'T6', 'T7']
True
T8  Proyecto técnico     |██                                | 0-3
T1  Licencias            |  ████                            | 3-7
T2  Cimentación          |      ██████                      | 7-13
T3  Estructura           |            ████████              | 13-21
T4  Acometida eléctrica  |      ███                         | 7-10
T5  Cubierta             |                      ████        | 21-26
T6  Instalaciones        |                      ██████      | 21-28
T7  Acabados             |                            ████| 28-32
Duración total: 32 días
Ruta crítica: T8 → T1 → T2 → T3 → T6 → T7
None False
['X1 → X0', 'X1 → X9'] False
False
```

Sección 4 - Refactorización a programación orientada a objetos

Ejercicio R1 - Refactorizar el gestor de pedidos de una distribuidora

Una distribuidora de bebidas para hostelería (canal **horeca**) y supermercados (canal **retail**) factura con un script escrito en 2017 por un becario que ya no está en la empresa. «Funciona», pero nadie se atreve a tocarlo: cada vez que cambia una regla de descuento hay que buscar en varios sitios y siempre se rompe algo. Te encargan **reescribirlo con programación orientada a objetos sin cambiar su comportamiento**.

Restricción: no está permitido usar herencia ni polimorfismo. Toda la reutilización se consigue con **composición** (objetos que contienen o usan otros objetos).

Código heredado:

```
# gestion_pedidos.py · versión heredada (2017). "Funciona, no tocar".
p_cod = ["AG-01", "RF-02", "ZM-03", "CV-04", "VN-05"]
p_nom = ["Agua 1,5L pack 6", "Refresco cola pack 12", "Zumo naranja 1L", "Cerveza pack 24", "Vino tinto
crianza"]
p_pre = [3.20, 9.60, 1.85, 18.90, 7.45]
p_stk = [120, 40, 60, 15, 30]
c_cod = ["CL-1", "CL-2", "CL-3"]
c_nom = ["Bar El Puerto", "Hotel Mirador", "Supermercado Sol"]
c_tip = ["horeca", "horeca", "retail"]
c_vip = [False, True, False]
peds = [
    ["CL-1", [["AG-01", 10], ["CV-04", 4], ["VN-05", 6]]],
    ["CL-2", [["RF-02", 20], ["VN-05", 12], ["XX-99", 1]]],
    ["CL-3", [["AG-01", 100], ["ZM-03", 30], ["CV-04", 10]]],
    ["CL-9", [["AG-01", 1]]],
    ["CL-1", [["CV-04", 5], ["ZM-03", 12]]],
    ["CL-2", [["CV-04", 3]]],
]
tot_emp = 0
n_ok = 0
n_ko = 0
for i in range(len(peds)):
    ped = peds[i]
```

```
    ci = -1
    for j in range(len(c_cod)):
        if c_cod[j] == ped[0]:
            ci = j
    if ci == -1:
        print("Pedido " + str(i + 1) + ": cliente " + ped[0] + " desconocido")
        n_ko = n_ko + 1
    else:
        print("Pedido " + str(i + 1) + " - " + c_nom[ci])
        sub = 0
        for l in ped[1]:
            pi = -1
            for k in range(len(p_cod)):
                if p_cod[k] == l[0]:
                    pi = k
            if pi == -1:
                print("   ! producto " + l[0] + " no existe")
            else:
                if p_stk[pi] >= l[1]:
                    p_stk[pi] = p_stk[pi] - l[1]
                    imp = p_pre[pi] * l[1]
                    sub = sub + imp
                    print("   " + p_nom[pi] + " x" + str(l[1]) + " = " + str(round(imp, 2)))
                else:
                    print("   ! sin stock de " + p_nom[pi] + " (quedan " + str(p_stk[pi]) + ")")
        d = 0
        if c_tip[ci] == "horeca":
            if sub > 100:
                d = 0.05
            if c_vip[ci] == True:
                d = d + 0.05
        else:
            if sub > 300:
                d = 0.08
            if c_vip[ci] == True:
                d = d + 0.05
        desc = sub * d
        base = sub - desc
        iva = base * 0.21
        tot = base + iva
        if sub > 0:
            print("   Subtotal: " + str(round(sub, 2)) + " | Dto: " + str(round(desc, 2)) + " | IVA: " +
str(round(iva, 2)) + " | Total: " + str(round(tot, 2)))
            tot_emp = tot_emp + tot
            n_ok = n_ok + 1
        else:
            print("   Pedido vacío")
            n_ko = n_ko + 1
print("==============================")
print("Pedidos facturados: " + str(n_ok) + " | Rechazados: " + str(n_ko))
print("Facturación total: " + str(round(tot_emp, 2)))
print("Stock bajo (<20):")
for k in range(len(p_cod)):
    if p_stk[k] < 20:
        print("   " + p_cod[k] + " " + p_nom[k] + ": " + str(p_stk[k]))
```

Salida del código heredado:

```
Pedido 1 - Bar El Puerto
   Agua 1,5L pack 6 x10 = 32.0
   Cerveza pack 24 x4 = 75.6
   Vino tinto crianza x6 = 44.7
   Subtotal: 152.3 | Dto: 7.62 | IVA: 30.38 | Total: 175.07
Pedido 2 - Hotel Mirador
   Refresco cola pack 12 x20 = 192.0
   Vino tinto crianza x12 = 89.4
   ! producto XX-99 no existe
   Subtotal: 281.4 | Dto: 28.14 | IVA: 53.18 | Total: 306.44
Pedido 3 - Supermercado Sol
   Agua 1,5L pack 6 x100 = 320.0
   Zumo naranja 1L x30 = 55.5
   Cerveza pack 24 x10 = 189.0
   Subtotal: 564.5 | Dto: 45.16 | IVA: 109.06 | Total: 628.4
```

```
Pedido 4: cliente CL-9 desconocido
Pedido 5 - Bar El Puerto
   ! sin stock de Cerveza pack 24 (quedan 1)
   Zumo naranja 1L x12 = 22.2
   Subtotal: 22.2 | Dto: 0.0 | IVA: 4.66 | Total: 26.86
Pedido 6 - Hotel Mirador
   ! sin stock de Cerveza pack 24 (quedan 1)
   Pedido vacío
============================
Pedidos facturados: 4 | Rechazados: 2
Facturación total: 1136.78
Stock bajo (<20):
   AG-01 Agua 1,5L pack 6: 10
   ZM-03 Zumo naranja 1L: 18
   CV-04 Cerveza pack 24: 1
   VN-05 Vino tinto crianza: 12
```

Reglas de negocio que esconde el código (descúbrelas tú también leyéndolo):

- Canal `horeca`: 5 % de descuento si el subtotal supera 100 €. Canal `retail`: 8 % si supera 300 €.
- Los clientes VIP suman un 5 % extra de descuento, en cualquier canal.
- El IVA es del 21 % sobre la base (subtotal − descuento).
- Si no hay stock suficiente de un producto, esa línea no se sirve. Si ninguna línea se sirve, el pedido es vacío y cuenta como rechazado. **Se pide:**

4. **Diagnóstico.** Al principio de `gestion_pedidos_oop.py`, en un comentario, enumera **al menos 6 problemas** de diseño del código heredado (por ejemplo: listas paralelas, búsquedas repetidas, números mágicos...).
5. **Diseño.** Implementa como mínimo estas clases (los nombres de los métodos los decides tú): | Clase | Responsabilidad | | — | — | | `Producto` | Código, nombre, precio y **stock privado**. Sabe si tiene stock suficiente y descuenta unidades. | | `Cliente` | Código, nombre, canal y si es VIP. Sabe **qué porcentaje de descuento le corresponde** para un subtotal. Las reglas por canal son un **atributo de clase**. | | `Pedido` | Cliente, líneas servidas o con incidencia, subtotal, descuento, IVA y total. El IVA es un **atributo de clase**. `bool(pedido)` indica si tiene algo facturable y `str(pedido)` produce su bloque de texto. | | `Distribuidora` | Catálogo de productos y clientes (diccionarios **privados** por código), procesa pedidos, acumula la facturación y genera el informe final. |
6. **Prohibido** en tu versión: listas paralelas, variables globales que se modifiquen dentro de bucles, búsquedas con `for j in range(len(...))` para encontrar un elemento y números sueltos (`0.21`, `100`, `0.05`, `20`...) fuera de atributos de clase.
7. **Test de caracterización.** La salida de tu programa debe ser **idéntica carácter a carácter** a la del código heredado. Compruébalo en PowerShell:

```
python gestion_pedidos_original.py > salida_original.txt
python gestion_pedidos_oop.py > salida_oop.txt
Compare-Object (Get-Content salida_original.txt) (Get-Content salida_oop.txt)
```

Si `Compare-Object` no muestra nada, las dos salidas son iguales. 8. **Prueba de extensibilidad.** Cuando termines, responde en un comentario: si mañana se crea el canal `online` (3 % de descuento a partir de 50 €), ¿cuántas líneas tienes que cambiar en tu versión? ¿Y en la original?

✅ Solución

```python
# gestion_pedidos_oop.py
# DIAGNÓSTICO del código heredado:
#  1. Listas paralelas (p_cod, p_nom, p_pre...): un producto está repartido en 4 listas.
#  2. Búsquedas lineales repetidas con for j in range(len(...)) en vez de diccionarios.
#  3. Números mágicos sueltos: 0.21, 100, 300, 0.05, 0.08, 20.
#  4. Reglas de descuento duplicadas (horeca/retail repiten la suma del VIP).
#  5. Lógica y presentación mezcladas (cálculos y prints en el mismo bloque).
#  6. Variables globales que se modifican dentro de bucles (tot_emp, n_ok, n_ko, p_stk).
#  7. Índices crípticos (ped[0], l[1], ci, pi) y nombres de una letra.
#  8. Comparaciones redundantes (c_vip[ci] == True) y todo en un único bloque sin funciones.

class Producto:
    def __init__(self, codigo, nombre, precio, stock):
        self.codigo, self.nombre, self.precio = codigo, nombre, precio
        self.__stock = stock
```

```
    @property
    def stock(self):
        return self.__stock

    def hay_stock(self, cantidad):
        return self.__stock >= cantidad

    def retirar(self, cantidad):
        self.__stock -= cantidad

class Cliente:
    REGLAS_CANAL = {"horeca": (100, 0.05), "retail": (300, 0.08)}   # canal: (umbral, %)
    EXTRA_VIP = 0.05

    def __init__(self, codigo, nombre, canal, vip=False):
        self.codigo, self.nombre, self.canal, self.vip = codigo, nombre, canal, vip

    def porcentaje_descuento(self, subtotal):
        umbral, porcentaje = Cliente.REGLAS_CANAL[self.canal]
        descuento = porcentaje if subtotal > umbral else 0
        return descuento + (Cliente.EXTRA_VIP if self.vip else 0)

class Pedido:
    IVA = 0.21
    SANGRIA = "    "

    def __init__(self, numero, cliente):
        self.numero, self.cliente = numero, cliente
        self.__lineas = []          # textos en orden (servidas e incidencias)
        self.subtotal = 0

    def servir(self, producto, cantidad):
        if not producto.hay_stock(cantidad):
            self.__lineas.append(f"! sin stock de {producto.nombre} (quedan {producto.stock})")
            return
        producto.retirar(cantidad)
        importe = producto.precio * cantidad
        self.subtotal += importe
        self.__lineas.append(f"{producto.nombre} x{cantidad} = {round(importe, 2)}")

    def incidencia(self, texto):
        self.__lineas.append(texto)

    @property
    def descuento(self):
        return self.subtotal * self.cliente.porcentaje_descuento(self.subtotal)

    @property
    def iva(self):
        return (self.subtotal - self.descuento) * Pedido.IVA

    @property
    def total(self):
        return self.subtotal - self.descuento + self.iva

    def __bool__(self):
        return self.subtotal > 0

    def __str__(self):
        lineas = [f"Pedido {self.numero} - {self.cliente.nombre}"]
        lineas += [Pedido.SANGRIA + l for l in self.__lineas]
        if self:
            lineas.append(f"{Pedido.SANGRIA}Subtotal: {round(self.subtotal, 2)} | Dto: {round(self.descuento,
2)}"
                          f" | IVA: {round(self.iva, 2)} | Total: {round(self.total, 2)}")
        else:
            lineas.append(Pedido.SANGRIA + "Pedido vacío")
        return "\n".join(lineas)

class Distribuidora:
    UMBRAL_STOCK_BAJO = 20
```

```
    def __init__(self):
        self.__productos = {}
        self.__clientes = {}
        self.facturacion = 0
        self.facturados = 0
        self.rechazados = 0

    def alta_producto(self, producto):
        self.__productos[producto.codigo] = producto

    def alta_cliente(self, cliente):
        self.__clientes[cliente.codigo] = cliente

    def procesar(self, numero, cod_cliente, lineas):
        cliente = self.__clientes.get(cod_cliente)
        if cliente is None:
            print(f"Pedido {numero}: cliente {cod_cliente} desconocido")
            self.rechazados += 1
            return
        pedido = Pedido(numero, cliente)
        for cod_producto, cantidad in lineas:
            producto = self.__productos.get(cod_producto)
            if producto is None:
                pedido.incidencia(f"! producto {cod_producto} no existe")
            else:
                pedido.servir(producto, cantidad)
        print(pedido)
        if pedido:
            self.facturacion += pedido.total
            self.facturados += 1
        else:
            self.rechazados += 1

    def informe(self):
        print("=" * 30)
        print(f"Pedidos facturados: {self.facturados} | Rechazados: {self.rechazados}")
        print(f"Facturación total: {round(self.facturacion, 2)}")
        print(f"Stock bajo (<{Distribuidora.UMBRAL_STOCK_BAJO}):")
        for p in self.__productos.values():
            if p.stock < Distribuidora.UMBRAL_STOCK_BAJO:
                print(f"   {p.codigo} {p.nombre}: {p.stock}")

dist = Distribuidora()
for datos in [("AG-01", "Agua 1,5L pack 6", 3.20, 120), ("RF-02", "Refresco cola pack 12", 9.60, 40),
              ("ZM-03", "Zumo naranja 1L", 1.85, 60), ("CV-04", "Cerveza pack 24", 18.90, 15),
              ("VN-05", "Vino tinto crianza", 7.45, 30)]:
    dist.alta_producto(Producto(*datos))
for datos in [("CL-1", "Bar El Puerto", "horeca", False), ("CL-2", "Hotel Mirador", "horeca", True),
              ("CL-3", "Supermercado Sol", "retail", False)]:
    dist.alta_cliente(Cliente(*datos))

pedidos = [
    ("CL-1", [("AG-01", 10), ("CV-04", 4), ("VN-05", 6)]),
    ("CL-2", [("RF-02", 20), ("VN-05", 12), ("XX-99", 1)]),
    ("CL-3", [("AG-01", 100), ("ZM-03", 30), ("CV-04", 10)]),
    ("CL-9", [("AG-01", 1)]),
    ("CL-1", [("CV-04", 5), ("ZM-03", 12)]),
    ("CL-2", [("CV-04", 3)]),
]
for numero, (cod_cliente, lineas) in enumerate(pedidos, start=1):
    dist.procesar(numero, cod_cliente, lineas)
dist.informe()
# EXTENSIBILIDAD: canal online (3 % desde 50 €) → 1 línea en mi versión:
#   REGLAS_CANAL = {..., "online": (50, 0.03)}. En la original habría que añadir otra rama
#   if/else completa (≈5 líneas) y repetir la suma del VIP.
```

🖥️ Salida:

```
Pedido 1 - Bar El Puerto
   Agua 1,5L pack 6 x10 = 32.0
   Cerveza pack 24 x4 = 75.6
   Vino tinto crianza x6 = 44.7
   Subtotal: 152.3 | Dto: 7.62 | IVA: 30.38 | Total: 175.07
```

```
Pedido 2 - Hotel Mirador
   Refresco cola pack 12 x20 = 192.0
   Vino tinto crianza x12 = 89.4
   ! producto XX-99 no existe
   Subtotal: 281.4 | Dto: 28.14 | IVA: 53.18 | Total: 306.44
Pedido 3 - Supermercado Sol
   Agua 1,5L pack 6 x100 = 320.0
   Zumo naranja 1L x30 = 55.5
   Cerveza pack 24 x10 = 189.0
   Subtotal: 564.5 | Dto: 45.16 | IVA: 109.06 | Total: 628.4
Pedido 4: cliente CL-9 desconocido
Pedido 5 - Bar El Puerto
   ! sin stock de Cerveza pack 24 (quedan 1)
   Zumo naranja 1L x12 = 22.2
   Subtotal: 22.2 | Dto: 0.0 | IVA: 4.66 | Total: 26.86
Pedido 6 - Hotel Mirador
   ! sin stock de Cerveza pack 24 (quedan 1)
   Pedido vacío
==============================
Pedidos facturados: 4 | Rechazados: 2
Facturación total: 1136.78
Stock bajo (<20):
   AG-01 Agua 1,5L pack 6: 10
   ZM-03 Zumo naranja 1L: 18
   CV-04 Cerveza pack 24: 1
   VN-05 Vino tinto crianza: 12
```

Sección 5 - Juego interactivo

> 💡 Puedes hacer tu propio juego si quieres. No tiene que ser obligatoriamente el ejercicio J1

Ejercicio J1 - Juego interactivo: Hundir la flota

Vas a programar una versión de un jugador de **Hundir la flota** que se juega en la terminal. El ordenador esconde una flota en un tablero de 6 × 6 y el jugador dispara escribiendo coordenadas como `B4` hasta hundir todos los barcos o quedarse sin munición.

Restricción: no está permitido usar herencia ni polimorfismo. El diseño se basa en clases independientes que colaboran por **composición**.

Reglas del juego

- Tablero de 6 × 6. Columnas `AF` y filas `16`.
- Flota (atributo de clase de `Partida`): `Destructor` (3 casillas), `Fragata` (2) y `Lancha` (2). Se colocan al azar, en horizontal o vertical, sin salirse del tablero y sin solaparse.
- El jugador tiene **20 disparos**.
- Resultados de un disparo: `Agua.`, `¡Tocado!` o `¡Hundido! Barco hundido: <nombre>.`
- Disparar a una casilla ya disparada **no gasta munición**. Una coordenada mal escrita tampoco.
- Comandos especiales: `mapa` (dibuja el tablero sin mostrar los barcos), `flota` (estado de cada barco) y `salir`.
- La partida termina al hundir toda la flota (victoria), al agotar la munición o al escribir `salir`. Al terminar se muestra un resumen con las estadísticas y el tablero con los barcos al descubierto.
- Símbolos del tablero: `·` desconocido, `~` agua, `X` tocado, `#` barco hundido, `B` barco (solo en el resumen final). **Diseño obligatorio**

Clase	Responsabilidad	Detalles mínimos
`Barco`	Un barco de la flota	Nombre, tamaño, lista de posiciones `(fila, columna)` y un `set` **privado** de impactos. `ocupa(pos)`, `recibir_impacto(pos)`, `hundido()`, `len(barco)` y `__str__` (`"Fragata (2) · 1/2 impactos"` o `"Fragata (2) · hundido"`).
`Tablero`	La cuadrícula y los barcos	Tamaño, lista de barcos y diccionario **privado** de disparos. Los símbolos son

Clase	Responsabilidad	Detalles mínimos
		atributos de clase. Coloca barcos al azar, procesa un disparo, sabe si toda la flota está hundida y se dibuja como texto.
`Jugador`	Estadísticas del jugador	Nombre, munición, disparos y aciertos. `precision()`, `bool(jugador)` (`True` si le queda munición) y `__str__`.
`Partida`	Coordina el juego	**Tiene** un `Tablero` y un `Jugador`. Convierte el texto `B4` en `(fila, columna)` con un método de uso interno, procesa un turno, sabe si la partida ha terminado y contiene el bucle con `input()`.

Requisitos técnicos

9. El constructor de `Partida` recibe un parámetro opcional `semilla`. Crea con él un **objeto** generador de números aleatorios: `random.Random(semilla)`. Con la misma semilla los barcos quedan siempre en el mismo sitio, lo que permite probar el juego.
10. Separa la lógica de la interacción: el método `turno(texto)` recibe una coordenada y **devuelve** el mensaje (no hace `print` ni `input`). Solo `jugar()` usa `input()` y `print()`.
11. Al final del archivo, protege el arranque con `if __name__ == "__main__":`, pidiendo el nombre del jugador y creando la partida con `semilla=7`.
12. Añade un bloque de **pruebas sin teclado** (por ejemplo, en una función `probar()`) que cree una partida con `semilla=7` y `municion=3`, llame tres veces a `turno()` y compruebe que la partida termina. **Ejemplo de partida** (con `semilla=7`; tu tablero puede ser diferente si colocas los barcos con otro algoritmo, pero el formato debe ser este):

```
Nombre del jugador: Ana
HUNDIR LA FLOTA · tablero 6x6 · 3 barcos · 20 disparos
Comandos: coordenada (B4), 'mapa', 'flota', 'salir'
[20] Disparo: A1
Agua.
[19] Disparo: D2
¡Tocado!
[18] Disparo: D3
¡Tocado!
[17] Disparo: d4
¡Hundido! Barco hundido: Destructor.
[16] Disparo: D4
Ya habías disparado ahí. No gastas munición.
[16] Disparo: Z9
Coordenada no válida: 'Z9'. Usa letra + número, por ejemplo B4.
[16] Disparo: mapa
    A B C D E F
 1  ~ · · · · ·
 2  · · · # · ·
 3  · · · # · ·
 4  · · · # · ·
 5  · · · · · ·
 6  · · · · · ·
[16] Disparo: E3
¡Tocado!
[15] Disparo: F3
¡Hundido! Barco hundido: Lancha.
[14] Disparo: flota
  - Destructor hundido
  - Fragata a flote
  - Lancha hundido
[14] Disparo: E1
¡Tocado!
[13] Disparo: F1
¡Hundido! Barco hundido: Fragata.

=== FIN DE LA PARTIDA ===
¡Victoria, Ana! Flota enemiga hundida.
Ana · disparos: 8 · aciertos: 7 · precisión: 87.5 % · munición: 12
```

```
  - Destructor (3) · hundido
  - Fragata (2) · hundido
  - Lancha (2) · hundido
    A B C D E F
 1  ~ · · · # #
 2  · · · # · ·
 3  · · · # # #
 4  · · · # · ·
 5  · · · · · ·
 6  · · · · · ·
```

Ampliaciones interesantes:

- Impedir que dos barcos queden en casillas contiguas.
- Elegir el tamaño del tablero y la dificultad (munición) al empezar.
- Guardar un ranking de partidas en una lista de objetos `Jugador` ordenada por precisión (sobrecarga `<`).
- Modo «radar»: una vez por partida, el comando `radar B4` indica cuántas casillas con barco hay en el cuadrado 3 × 3 alrededor de la coordenada.

✅ Solución

```
import random

class Barco:
    def __init__(self, nombre, tamano, posiciones):
        self.nombre = nombre
        self.tamano = tamano
        self.posiciones = posiciones          # [(fila, col), ...]
        self.__impactos = set()

    def ocupa(self, pos):
        return pos in self.posiciones

    def recibir_impacto(self, pos):
        if self.ocupa(pos):
            self.__impactos.add(pos)

    def hundido(self):
        return len(self.__impactos) == self.tamano

    def __len__(self):
        return self.tamano

    def __str__(self):
        estado = "hundido" if self.hundido() else f"{len(self.__impactos)}/{self.tamano} impactos"
        return f"{self.nombre} ({self.tamano}) · {estado}"

class Tablero:
    DESCONOCIDO, AGUA, TOCADO, HUNDIDO, BARCO = "·", "~", "X", "#", "B"
    LETRAS = "ABCDEF"

    def __init__(self, tamano, rng):
        self.tamano = tamano
        self.rng = rng                        # objeto random.Random
        self.barcos = []
        self.__disparos = {}                  # {(fila, col): resultado}

    def colocar(self, nombre, tamano):
        while True:
            horizontal = self.rng.choice([True, False])
            fila = self.rng.randrange(self.tamano - (0 if horizontal else tamano - 1))
            col = self.rng.randrange(self.tamano - (tamano - 1 if horizontal else 0))
            posiciones = [(fila, col + i) if horizontal else (fila + i, col) for i in range(tamano)]
            if not any(b.ocupa(p) for b in self.barcos for p in posiciones):
                self.barcos.append(Barco(nombre, tamano, posiciones))
                return

    def ya_disparado(self, pos):
        return pos in self.__disparos

    def disparar(self, pos):
        for barco in self.barcos:
```

```
            if barco.ocupa(pos):
                barco.recibir_impacto(pos)
                self.__disparos[pos] = "tocado"
                if barco.hundido():
                    return f"¡Hundido! Barco hundido: {barco.nombre}."
                return "¡Tocado!"
        self.__disparos[pos] = "agua"
        return "Agua."

    def flota_hundida(self):
        return all(b.hundido() for b in self.barcos)

    def _simbolo(self, pos, revelar):
        barco = next((b for b in self.barcos if b.ocupa(pos)), None)
        if pos in self.__disparos:
            if barco is None:
                return Tablero.AGUA
            return Tablero.HUNDIDO if barco.hundido() else Tablero.TOCADO
        return Tablero.BARCO if (revelar and barco) else Tablero.DESCONOCIDO

    def dibujar(self, revelar=False):
        lineas = ["    " + " ".join(Tablero.LETRAS[:self.tamano])]
        for f in range(self.tamano):
            celdas = " ".join(self._simbolo((f, c), revelar) for c in range(self.tamano))
            lineas.append(f"{f + 1:>2}  {celdas}")
        return "\n".join(lineas)

class Jugador:
    def __init__(self, nombre, municion):
        self.nombre, self.municion = nombre, municion
        self.disparos, self.aciertos = 0, 0

    def precision(self):
        return round(self.aciertos / self.disparos * 100, 1) if self.disparos else 0.0

    def __bool__(self):
        return self.municion > 0

    def __str__(self):
        return (f"{self.nombre} · disparos: {self.disparos} · aciertos: {self.aciertos} · "
                f"precisión: {self.precision()} % · munición: {self.municion}")

class Partida:
    FLOTA = [("Destructor", 3), ("Fragata", 2), ("Lancha", 2)]
    TAMANO = 6

    def __init__(self, nombre="Jugador", municion=20, semilla=None):
        self.tablero = Tablero(Partida.TAMANO, random.Random(semilla))   # composición
        for nombre_barco, tam in Partida.FLOTA:
            self.tablero.colocar(nombre_barco, tam)
        self.jugador = Jugador(nombre, municion)
        self.abandonada = False

    def _a_posicion(self, texto):
        texto = texto.strip().upper()
        if len(texto) != 2 or texto[0] not in Tablero.LETRAS or not texto[1].isdigit():
            return None
        fila = int(texto[1]) - 1
        return (fila, Tablero.LETRAS.index(texto[0])) if 0 <= fila < Partida.TAMANO else None

    def turno(self, texto):                    # lógica pura: devuelve texto, sin print/input
        comando = texto.strip().lower()
        if comando == "mapa":
            return self.tablero.dibujar()
        if comando == "flota":
            return "\n".join(f"  - {b.nombre} {'hundido' if b.hundido() else 'a flote'}"
                             for b in self.tablero.barcos)
        if comando == "salir":
            self.abandonada = True
            return "Partida abandonada."
        pos = self._a_posicion(texto)
        if pos is None:
            return f"Coordenada no válida: '{texto}'. Usa letra + número, por ejemplo B4."
```

```
        if self.tablero.ya_disparado(pos):
            return "Ya habías disparado ahí. No gastas munición."
        self.jugador.municion -= 1
        self.jugador.disparos += 1
        resultado = self.tablero.disparar(pos)
        if resultado != "Agua.":
            self.jugador.aciertos += 1
        return resultado

    def terminada(self):
        return self.abandonada or self.tablero.flota_hundida() or not self.jugador

    def resumen(self):
        lineas = ["\n=== FIN DE LA PARTIDA ==="]
        if self.tablero.flota_hundida():
            lineas.append(f"¡Victoria, {self.jugador.nombre}! Flota enemiga hundida.")
        else:
            lineas.append("Derrota: quedan barcos a flote.")
        lineas.append(str(self.jugador))
        lineas += [f"  - {b}" for b in self.tablero.barcos]
        lineas.append(self.tablero.dibujar(revelar=True))
        return "\n".join(lineas)

    def jugar(self):
        print(f"HUNDIR LA FLOTA · tablero {Partida.TAMANO}x{Partida.TAMANO} · "
              f"{len(Partida.FLOTA)} barcos · {self.jugador.municion} disparos")
        print("Comandos: coordenada (B4), 'mapa', 'flota', 'salir'")
        while not self.terminada():
            texto = input(f"[{self.jugador.municion}] Disparo: ")
            print(self.turno(texto))
        print(self.resumen())

def probar():
    p = Partida("Test", municion=3, semilla=7)
    for coord in ["A1", "A2", "A3"]:
        p.turno(coord)
    assert p.terminada() and p.jugador.municion == 0
    print("Prueba OK:", p.jugador)

probar()
if __name__ == "__main__":
    nombre = input("Nombre del jugador: ")
    Partida(nombre, semilla=7).jugar()
```

🖥️ Salida (prueba sin teclado + partida de ejemplo con semilla=7):

```
Prueba OK: Test · disparos: 3 · aciertos: 0 · precisión: 0.0 % · munición: 0
Nombre del jugador: Ana
HUNDIR LA FLOTA · tablero 6x6 · 3 barcos · 20 disparos
Comandos: coordenada (B4), 'mapa', 'flota', 'salir'
[20] Disparo: A1
Agua.
[19] Disparo: D2
¡Tocado!
[18] Disparo: D3
¡Tocado!
[17] Disparo: d4
¡Hundido! Barco hundido: Destructor.
[16] Disparo: D4
Ya habías disparado ahí. No gastas munición.
[16] Disparo: Z9
Coordenada no válida: 'Z9'. Usa letra + número, por ejemplo B4.
[16] Disparo: mapa
    A B C D E F
 1  ~ · · · · ·
 2  · · · # · ·
 3  · · · # · ·
 4  · · · # · ·
 5  · · · · · ·
 6  · · · · · ·
[16] Disparo: E3
¡Tocado!
[15] Disparo: F3
```

```
¡Hundido! Barco hundido: Lancha.
[14] Disparo: flota
  - Destructor hundido
  - Fragata a flote
  - Lancha hundido
[14] Disparo: E1
¡Tocado!
[13] Disparo: F1
¡Hundido! Barco hundido: Fragata.

=== FIN DE LA PARTIDA ===
¡Victoria, Ana! Flota enemiga hundida.
Ana · disparos: 8 · aciertos: 7 · precisión: 87.5 % · munición: 12
  - Destructor (3) · hundido
  - Fragata (2) · hundido
  - Lancha (2) · hundido
    A B C D E F
 1  ~ · · · # #
 2  · · · # · ·
 3  · · · # # #
 4  · · · # · ·
 5  · · · · · ·
 6  · · · · · ·
```

Parte 5 · Práctica 13 resuelta (Sprints de Python)

> **Instrucciones de entrega:** - Enlace al repositorio de Github. El formato de entrega son notebooks de Jupyter (.ipynb). - Se entrega un notebook por cada sprint, por ejemplo: sprint1.ipynb , sprint2.ipynb, sprint3.ipynb, sprint4.ipynb. - No hace falta entregar los sprints en markdown ni pdf

Página relacionada: 🚵Sprint_1-Estructuras básicas **Temas:** listas, tuplas, set y diccionarios

En cada sección hay primero ejemplos resueltos y después ejercicios propuestos. Todos trabajan con situaciones habituales en un equipo de datos: lotes de ingesta, logs, registros JSON, esquemas o particiones de un data lake.

Cada ejemplo incluye el código, la explicación paso a paso y la salida esperada. Cada ejercicio propuesto incluye los datos de partida y la salida que debe producir tu solución, para que puedas comprobarla. Copia los datos tal cual en tu notebook o en tu script.

Sección 1. Listas

Una lista es una colección **ordenada y modificable**. En ingeniería de datos aparece en todas partes: los ficheros pendientes de un lote, las filas leídas de un CSV, las métricas de una ejecución o la cola de tareas de un orquestador.

Ejemplos resueltos

Cada noche llegan a la zona de aterrizaje los ficheros de ventas del día. El proceso de ingesta mantiene una lista con los ficheros pendientes.

```
ficheros = ["ventas_0101.csv", "ventas_0102.csv", "ventas_0104.csv"]

# Llega un fichero nuevo: va al final del lote
ficheros.append("ventas_0105.csv")

# Llega con retraso el fichero del día 3: lo colocamos en su posición
ficheros.insert(2, "ventas_0103.csv")
print(ficheros)

print("Ficheros en el lote:", len(ficheros))
print("Primero:", ficheros[0], "| Último:", ficheros[-1])
print("Dos últimos:", ficheros[-2:])

# El fichero del día 1 ya se procesó: lo retiramos
ficheros.remove("ventas_0101.csv")
print(ficheros)
print("Posición de ventas_0104.csv:", ficheros.index("ventas_0104.csv"))
```

Paso a paso

1. `append()` añade siempre al final de la lista.
2. `insert(2, ...)` coloca el elemento en la posición 2 y desplaza el resto a la derecha. Así el fichero que llegó tarde queda en su sitio.
3. `len()` devuelve el número de elementos. Los índices empiezan en 0, y los negativos cuentan desde el final: `[-1]` es el último.
4. El *slicing* `[-2:]` devuelve una **lista nueva** con los dos últimos elementos.
5. `remove()` elimina la primera aparición del valor indicado. Si el valor no existe, lanza `ValueError`.
6. `index()` devuelve la posición del valor. Como ya hemos quitado el primer fichero, `ventas_0104.csv` pasa a la posición 2. **Salida esperada**

```
['ventas_0101.csv', 'ventas_0102.csv', 'ventas_0103.csv', 'ventas_0104.csv', 'ventas_0105.csv']
Ficheros en el lote: 5
Primero: ventas_0101.csv | Último: ventas_0105.csv
Dos últimos: ['ventas_0104.csv', 'ventas_0105.csv']
```

```
['ventas_0102.csv', 'ventas_0103.csv', 'ventas_0104.csv', 'ventas_0105.csv']
Posición de ventas_0104.csv: 2
```

Un pipeline lee datos de una API y guarda cuánto tarda cada petición. Queremos un pequeño informe de rendimiento.

```
latencias_ms = [120, 85, 430, 95, 110, 1500, 88, 102, 97, 640]

total = sum(latencias_ms)
media = total / len(latencias_ms)
print("Peticiones:", len(latencias_ms))
print("Mínima:", min(latencias_ms), "ms | Máxima:", max(latencias_ms), "ms")
print("Media:", round(media, 1), "ms")

# sorted() devuelve una lista NUEVA: la original no cambia
ordenadas = sorted(latencias_ms)
n = len(ordenadas)
mediana = (ordenadas[n // 2 - 1] + ordenadas[n // 2]) / 2   # n es par
print("Mediana:", mediana, "ms")

print("Tres más lentas:", sorted(latencias_ms, reverse=True)[:3])
print("Original sin tocar:", latencias_ms)
```

Paso a paso

7. `sum()`, `min()`, `max()` y `len()` son funciones integradas que aceptan una lista.
8. La media (326,7 ms) se dispara por dos peticiones muy lentas (1500 y 640 ms). Por eso en monitorización se mira también la **mediana**, que no se deja arrastrar por los valores extremos.
9. Para la mediana hace falta ordenar. `sorted()` devuelve una lista nueva y deja la original intacta. Con 10 valores, la mediana es la media de los dos centrales: posiciones 4 y 5 de la lista ordenada (102 y 110).
10. `sorted(..., reverse=True)[:3]` ordena de mayor a menor y se queda con los tres primeros.
11. La última línea confirma que `latencias_ms` conserva su orden original. Si hubiéramos usado `latencias_ms.sort()`, la lista original sí habría cambiado. **Salida esperada**

```
Peticiones: 10
Mínima: 85 ms | Máxima: 1500 ms
Media: 326.7 ms
Mediana: 106.0 ms
Tres más lentas: [1500, 640, 430]
Original sin tocar: [120, 85, 430, 95, 110, 1500, 88, 102, 97, 640]
```

En una arquitectura medallón, la capa **bronce** guarda los datos tal como llegan y la capa **plata** guarda los datos limpios. La regla es no modificar nunca bronce.

```
capa_bronce = [100, -5, 230, 0, 75]

# Error habitual: parece una copia, pero es la MISMA lista
capa_plata = capa_bronce
capa_plata.remove(-5)
print("Bronce:", capa_bronce)
print("Plata: ", capa_plata)
print("¿Misma lista?", capa_plata is capa_bronce)

# Forma correcta: copiar la lista
capa_bronce = [100, -5, 230, 0, 75]
capa_plata = capa_bronce.copy()       # también vale capa_bronce[:]
capa_plata.remove(-5)
print("Bronce:", capa_bronce)
print("Plata: ", capa_plata)
print("¿Misma lista?", capa_plata is capa_bronce)
```

Paso a paso

12. `capa_plata = capa_bronce` **no copia** la lista. Crea un segundo nombre (un *alias*) para el mismo objeto en memoria.
13. Por eso, al quitar el `5` desde `capa_plata`, también desaparece de `capa_bronce`. El operador `is` lo confirma: devuelve `True` porque ambas variables apuntan al mismo objeto.

14. Con `.copy()` (o `[:]`) se crea una lista nueva con el mismo contenido. Ahora los cambios en plata no afectan a bronce. > Este error es muy frecuente en los pipelines: una función de limpieza recibe una lista y, sin querer, modifica los datos originales.

Salida esperada

```
Bronce: [100, 230, 0, 75]
Plata:  [100, 230, 0, 75]
¿Misma lista? True
Bronce: [100, -5, 230, 0, 75]
Plata:  [100, 230, 0, 75]
¿Misma lista? False
```

Un orquestador ejecuta los jobs en el orden en que llegan, como una **cola**. En cambio, cuando hay que deshacer migraciones de base de datos, se revierte primero la última aplicada, como en una **pila**.

```
from collections import deque

# Cola FIFO: los jobs se ejecutan en el orden en que llegan
cola_jobs = deque()
cola_jobs.append("ingesta_clientes")
cola_jobs.append("ingesta_pedidos")
cola_jobs.append("calculo_kpis")
print("Cola:", list(cola_jobs))

siguiente = cola_jobs.popleft()
print("Se ejecuta:", siguiente)
print("Pendientes:", list(cola_jobs))

# Pila LIFO: las migraciones se deshacen en orden inverso
migraciones = []
migraciones.append("v1_crear_tablas")
migraciones.append("v2_add_columna_email")
migraciones.append("v3_crear_indice")
print("Aplicadas:", migraciones)

while migraciones:                      # una lista vacía equivale a False
    print("Revirtiendo:", migraciones.pop())
print("Quedan:", migraciones)
```

Paso a paso

15. Quitar el primer elemento de una lista con `pop(0)` es lento en listas grandes, porque Python tiene que desplazar todos los demás. Para colas se usa `deque` del módulo `collections`, cuyo `popleft()` es inmediato.
16. Para la pila basta una lista normal: `append()` apila y `pop()` sin argumento desapila el último elemento.
17. `while migraciones:` se repite mientras la lista tenga elementos, porque una lista vacía se evalúa como `False`. **Salida esperada**

```
Cola: ['ingesta_clientes', 'ingesta_pedidos', 'calculo_kpis']
Se ejecuta: ingesta_clientes
Pendientes: ['ingesta_pedidos', 'calculo_kpis']
Aplicadas: ['v1_crear_tablas', 'v2_add_columna_email', 'v3_crear_indice']
Revirtiendo: v3_crear_indice
Revirtiendo: v2_add_columna_email
Revirtiendo: v1_crear_tablas
Quedan: []
```

Cuando se lee un CSV sin librerías, cada fila es una lista y el fichero completo es una lista de listas.

```
from operator import itemgetter

# Cada fila: [fecha, tienda, importe]
filas = [
    ["2026-10-01", "MAD01", 1250.0],
    ["2026-10-01", "BCN02", 980.5],
    ["2026-10-02", "MAD01", 1530.0],
    ["2026-10-02", "VLC03", 410.0],
    ["2026-10-03", "BCN02", 1105.0],
]
```

```
print("Primera fila:", filas[0])
print("Tienda de la segunda fila:", filas[1][1])

importes = []
for fila in filas:
    importes.append(fila[2])
print("Columna importe:", importes)
print("Total vendido:", sum(importes))

por_importe = sorted(filas, key=itemgetter(2), reverse=True)
print("Venta más alta:", por_importe[0])
print("Venta más baja:", por_importe[-1])
```

Paso a paso

18. `filas[1][1]` accede primero a la fila 1 y, dentro de ella, a la columna 1 (la tienda).
19. Para extraer una columna recorremos las filas y guardamos el elemento 2 de cada una.
20. `itemgetter(2)` (del módulo `operator`) crea una función que, dada una fila, devuelve su elemento 2. `sorted()` la usa como **criterio de ordenación** mediante el parámetro `key`. **Salida esperada**

```
Primera fila: ['2026-10-01', 'MAD01', 1250.0]
Tienda de la segunda fila: BCN02
Columna importe: [1250.0, 980.5, 1530.0, 410.0, 1105.0]
Total vendido: 5275.5
Venta más alta: ['2026-10-02', 'MAD01', 1530.0]
Venta más baja: ['2026-10-02', 'VLC03', 410.0]
```

HDFS no guarda los ficheros enteros: los divide en **bloques** de 128 MB y guarda **3 copias** (réplicas) de cada bloque en nodos distintos (DataNodes). Así, si un nodo falla, los datos siguen disponibles en otro.

```
tamano_fichero_mb = 700
tamano_bloque_mb = 128
replicas = 3
nodos = ["datanode1", "datanode2", "datanode3", "datanode4"]

# 1. Trocear el fichero: todos los bloques de 128 MB salvo el último
bloques = []
restante = tamano_fichero_mb
while restante > 0:
    bloques.append(min(tamano_bloque_mb, restante))
    restante -= tamano_bloque_mb
print("Bloques (MB):", bloques)
print("Número de bloques:", len(bloques))

# 2. Una lista vacía por nodo: lo que guardará cada DataNode
contenido = []
for _ in nodos:
    contenido.append([])

# 3. Repartir las 3 réplicas de cada bloque en nodos distintos
for b in range(len(bloques)):
    for r in range(replicas):
        destino = (b + r) % len(nodos)
        contenido[destino].append(f"blk{b}")

for i in range(len(nodos)):
    print(nodos[i], contenido[i])
print("Espacio ocupado en el clúster:", sum(bloques) * replicas, "MB")
```

Paso a paso

21. El bucle `while` va restando 128 MB hasta agotar el fichero. `min()` hace que el último bloque tenga solo lo que queda (60 MB). Los bloques no se rellenan hasta 128 MB.
22. `contenido` es una lista de listas: una lista por cada DataNode.
23. La réplica `r` del bloque `b` va al nodo `(b + r) % 4`. El operador `%` hace que, al llegar al último nodo, se vuelva a empezar por el primero, y garantiza que las 3 réplicas de un bloque caen en nodos distintos.
24. Un fichero de 700 MB ocupa 2100 MB en el clúster. Es el precio de la tolerancia a fallos. **Salida esperada**

```
Bloques (MB): [128, 128, 128, 128, 128, 60]
Número de bloques: 6
datanode1 ['blk0', 'blk2', 'blk3', 'blk4']
datanode2 ['blk0', 'blk1', 'blk3', 'blk4', 'blk5']
datanode3 ['blk0', 'blk1', 'blk2', 'blk4', 'blk5']
datanode4 ['blk1', 'blk2', 'blk3', 'blk5']
Espacio ocupado en el clúster: 2100 MB
```

En Spark, cada partición la procesa una tarea, y una etapa no termina hasta que acaba **la tarea más lenta**. Si unas particiones tienen muchos más datos que otras (*data skew*), todo el job espera por ellas.

```python
registros_por_particion = [10250, 9870, 10110, 48300, 9930, 10020, 9990, 51200]

total = sum(registros_por_particion)
media = total / len(registros_por_particion)
print("Particiones:", len(registros_por_particion), "| Registros:", total)
print("Media por partición:", round(media))

mayor = max(registros_por_particion)
print("Partición más grande:", registros_por_particion.index(mayor), "con", mayor, "registros")
print("Factor de skew (máx / media):", round(mayor / media, 2))

sesgadas = []
for i in range(len(registros_por_particion)):
    if registros_por_particion[i] > 2 * media:
        sesgadas.append(i)
print("Particiones con más del doble de la media:", sesgadas)

dos_mayores = sorted(registros_por_particion, reverse=True)[:2]
print("Las 2 mayores concentran el", round(sum(dos_mayores) / total * 100, 1), "% de los datos")
```

Paso a paso

25. Con la suma y la longitud de la lista se obtiene la media de registros por partición.
26. El **factor de skew** compara la partición más grande con la media. Un valor cercano a 1 indica un reparto equilibrado; aquí es 2,57.
27. El bucle con `range(len(...))` recorre los índices para poder informar del **número de partición**, no solo de su tamaño.
28. Dos particiones de ocho concentran más de la mitad de los datos: esas dos tareas marcarán la duración del job. La solución habitual es reparticionar (Ejercicio 1.6). **Salida esperada**

```
Particiones: 8 | Registros: 159670
Media por partición: 19959
Partición más grande: 7 con 51200 registros
Factor de skew (máx / media): 2.57
Particiones con más del doble de la media: [3, 7]
Las 2 mayores concentran el 62.3 % de los datos
```

Ejercicios propuestos

Ejercicio propuesto 1.1

Un sensor de temperatura envía `-999.0` cuando falla, y a veces envía valores imposibles. El rango físico válido es de -40 a 85 ºC, ambos incluidos.

```python
lecturas = [21.5, 22.1, -999.0, 23.4, 150.2, 22.8, -999.0, 24.0, 21.9, 87.3]
```

Construye dos listas, `validas` y `descartadas`, y muestra:

- Las dos listas.
- El porcentaje de lecturas descartadas, con un decimal.
- La media de las lecturas válidas, con dos decimales.
- Cuántas veces aparece el código de fallo `999.0` (usa un método de lista). **Salida esperada**

```
Válidas: [21.5, 22.1, 23.4, 22.8, 24.0, 21.9]
Descartadas: [-999.0, 150.2, -999.0, 87.3]
Porcentaje descartado: 40.0 %
Media de las válidas: 22.62
Códigos -999.0 (sensor caído): 2
```

✅ Solución

```python
lecturas = [21.5, 22.1, -999.0, 23.4, 150.2, 22.8, -999.0, 24.0, 21.9, 87.3]

validas, descartadas = [], []
for valor in lecturas:
    if -40 <= valor <= 85:
        validas.append(valor)
    else:
        descartadas.append(valor)

print("Válidas:", validas)
print("Descartadas:", descartadas)
print(f"Porcentaje descartado: {len(descartadas) / len(lecturas) * 100:.1f} %")
print(f"Media de las válidas: {sum(validas) / len(validas):.2f}")
print("Códigos -999.0 (sensor caído):", lecturas.count(-999.0))
```

🖥️ Salida:

```
Válidas: [21.5, 22.1, 23.4, 22.8, 24.0, 21.9]
Descartadas: [-999.0, 150.2, -999.0, 87.3]
Porcentaje descartado: 40.0 %
Media de las válidas: 22.62
Códigos -999.0 (sensor caído): 2
```

Ejercicio propuesto 1.2

Una API de clientes devuelve identificadores repetidos. Hay que eliminarlos **conservando el orden en que aparecen por primera vez**. Resuélvelo solo con listas (los `set` se ven en la Sección 3).

```python
ids_recibidos = ["C003", "C001", "C003", "C007", "C001", "C002", "C007", "C003"]
```

Salida esperada

```
Únicos en orden de llegada: ['C003', 'C001', 'C007', 'C002']
Duplicados eliminados: 4
```

✅ Solución

```python
ids_recibidos = ["C003", "C001", "C003", "C007", "C001", "C002", "C007", "C003"]

unicos = []
for cid in ids_recibidos:
    if cid not in unicos:
        unicos.append(cid)

print("Únicos en orden de llegada:", unicos)
print("Duplicados eliminados:", len(ids_recibidos) - len(unicos))
```

🖥️ Salida:

```
Únicos en orden de llegada: ['C003', 'C001', 'C007', 'C002']
Duplicados eliminados: 4
```

Ejercicio propuesto 1.3

La API de destino acepta como máximo 5 registros por petición. Divide la lista en lotes con *slicing* y muestra cada lote con su número y su tamaño. El último lote puede ser más pequeño. Calcula el número de lotes con una

fórmula, sin contar a mano.

```
registros = list(range(1, 24))   # 23 registros: 1, 2, ..., 23
tamano_lote = 5
```

Salida esperada

```
Lote 1: 5 registros -> [1, 2, 3, 4, 5]
Lote 2: 5 registros -> [6, 7, 8, 9, 10]
Lote 3: 5 registros -> [11, 12, 13, 14, 15]
Lote 4: 5 registros -> [16, 17, 18, 19, 20]
Lote 5: 3 registros -> [21, 22, 23]
Número de lotes: 5
```

✅ **Solución**

```
registros = list(range(1, 24))
tamano_lote = 5

num_lotes = (len(registros) + tamano_lote - 1) // tamano_lote    # división "hacia arriba"
for i in range(num_lotes):
    lote = registros[i * tamano_lote:(i + 1) * tamano_lote]
    print(f"Lote {i + 1}: {len(lote)} registros -> {lote}")
print("Número de lotes:", num_lotes)
```

🖥️ **Salida:**

```
Lote 1: 5 registros -> [1, 2, 3, 4, 5]
Lote 2: 5 registros -> [6, 7, 8, 9, 10]
Lote 3: 5 registros -> [11, 12, 13, 14, 15]
Lote 4: 5 registros -> [16, 17, 18, 19, 20]
Lote 5: 3 registros -> [21, 22, 23]
Número de lotes: 5
```

Ejercicio propuesto 1.4

Un compañero ha escrito este código para eliminar los importes negativos de un lote:

```
importes = [120, -10, -35, 80, -5, -60, 200]
for x in importes:
    if x < 0:
        importes.remove(x)
print(importes)
```

Y obtiene esta salida:

```
[120, -35, 80, -60, 200]
```

29. Explica por qué siguen apareciendo `35` y `60`. Pista: piensa qué le ocurre al índice interno del `for` cuando se elimina un elemento.
30. Corrige el código **sin modificar la lista mientras se recorre**. **Salida esperada tras la corrección**

```
[120, 80, 200]
```

✅ **Solución**

```
# 1) Al borrar un elemento, los siguientes se desplazan una posición a la izquierda, pero el
#    índice interno del for sigue avanzando: el elemento que ocupa el hueco (-35, -60) se SALTA.
# 2) Corrección: construir una lista NUEVA (o recorrer una copia: for x in importes[:]).
importes = [120, -10, -35, 80, -5, -60, 200]
importes = [x for x in importes if x >= 0]
print(importes)
```

🖥️ **Salida:**

```
[120, 80, 200]
```

Ejercicio propuesto 1.5

```
# [pedido_id, tienda, importe, estado]
pedidos = [
    ["P01", "MAD01", 250.0, "ENTREGADO"],
    ["P02", "BCN02", 89.9, "PENDIENTE"],
    ["P03", "MAD01", 1200.0, "ENTREGADO"],
    ["P04", "VLC03", 45.5, "CANCELADO"],
    ["P05", "MAD01", 310.0, "PENDIENTE"],
    ["P06", "BCN02", 760.0, "ENTREGADO"],
    ["P07", "MAD01", 99.0, "ENTREGADO"],
]
```

Escribe el código necesario para:

31. Obtener la lista de importes de los pedidos `ENTREGADO` de la tienda `MAD01` y su total.
32. Obtener los identificadores de los 3 pedidos de mayor importe, sea cual sea su estado.
33. Cambiar **en la propia lista** el estado `PENDIENTE` por `EN_CURSO` y mostrar la columna de estados. **Salida esperada**

```
Importes entregados en MAD01: [250.0, 1200.0, 99.0] -> total 1549.0
Top 3 por importe: ['P03', 'P06', 'P05']
Estados actualizados: ['ENTREGADO', 'EN_CURSO', 'ENTREGADO', 'CANCELADO', 'EN_CURSO', 'ENTREGADO',
'ENTREGADO']
```

✅ Solución

```
pedidos = [
    ["P01", "MAD01", 250.0, "ENTREGADO"],
    ["P02", "BCN02", 89.9, "PENDIENTE"],
    ["P03", "MAD01", 1200.0, "ENTREGADO"],
    ["P04", "VLC03", 45.5, "CANCELADO"],
    ["P05", "MAD01", 310.0, "PENDIENTE"],
    ["P06", "BCN02", 760.0, "ENTREGADO"],
    ["P07", "MAD01", 99.0, "ENTREGADO"],
]

importes_mad = [p[2] for p in pedidos if p[1] == "MAD01" and p[3] == "ENTREGADO"]
print(f"Importes entregados en MAD01: {importes_mad} -> total {sum(importes_mad)}")

top3 = sorted(pedidos, key=lambda p: p[2], reverse=True)[:3]
print("Top 3 por importe:", [p[0] for p in top3])

for p in pedidos:                       # modificar en la propia lista
    if p[3] == "PENDIENTE":
        p[3] = "EN_CURSO"
print("Estados actualizados:", [p[3] for p in pedidos])
```

🖥️ Salida:

```
Importes entregados en MAD01: [250.0, 1200.0, 99.0] -> total 1549.0
Top 3 por importe: ['P03', 'P06', 'P05']
Estados actualizados: ['ENTREGADO', 'EN_CURSO', 'ENTREGADO', 'CANCELADO', 'EN_CURSO', 'ENTREGADO',
'ENTREGADO']
```

Ejercicio propuesto 1.6

La función `repartition()` de Spark redistribuye los registros para equilibrar las particiones. Vamos a simular la versión más sencilla, el reparto *round-robin*.

```
# Tres particiones de entrada muy desequilibradas
entrada = [
    [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112],
```

```
    [113, 114],
    [115, 116, 117, 118, 119, 120],
]
num_particiones = 4
```

34. Junta todos los registros en una sola lista, en orden. Puedes usar `extend()`.
35. Crea 4 particiones vacías y asigna el registro número `i` a la partición `i % num_particiones`.
36. Muestra el contenido de cada partición y compara el factor de skew (máximo / media) antes y después, con dos decimales. **Salida esperada**

```
Partición 0: [101, 105, 109, 113, 117]
Partición 1: [102, 106, 110, 114, 118]
Partición 2: [103, 107, 111, 115, 119]
Partición 3: [104, 108, 112, 116, 120]
Tamaños antes: [12, 2, 6] -> skew 1.8
Tamaños después: [5, 5, 5, 5] -> skew 1.0
```

✅ **Solución**

```
entrada = [
    [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112],
    [113, 114],
    [115, 116, 117, 118, 119, 120],
]
num_particiones = 4

todos = []
for parte in entrada:
    todos.extend(parte)

particiones = [[] for _ in range(num_particiones)]
for i, registro in enumerate(todos):
    particiones[i % num_particiones].append(registro)

for i, p in enumerate(particiones):
    print(f"Partición {i}: {p}")

def skew(tamanos):
    return round(max(tamanos) / (sum(tamanos) / len(tamanos)), 2)

antes = [len(p) for p in entrada]
despues = [len(p) for p in particiones]
print(f"Tamaños antes: {antes} -> skew {skew(antes)}")
print(f"Tamaños después: {despues} -> skew {skew(despues)}")
```

🖥️ **Salida:**

```
Partición 0: [101, 105, 109, 113, 117]
Partición 1: [102, 106, 110, 114, 118]
Partición 2: [103, 107, 111, 115, 119]
Partición 3: [104, 108, 112, 116, 120]
Tamaños antes: [12, 2, 6] -> skew 1.8
Tamaños después: [5, 5, 5, 5] -> skew 1.0
```

Ejercicio propuesto 1.7

Un sistema de streaming cuenta los eventos que llegan cada minuto. Para suavizar el ruido se calcula una **media móvil** con una ventana de 3 minutos.

```
eventos_por_minuto = [120, 135, 128, 410, 140, 132, 125, 390, 395, 130]
ventana = 3
```

37. Calcula con *slicing* la media de cada ventana de 3 minutos consecutivos (minutos 0-2, 1-3, 2-4...), redondeada a un decimal.
38. Indica qué ventana tiene la media más alta.
39. Detecta los minutos con **pico**: aquellos cuyo valor supera el doble de la media de los 3 minutos anteriores. Empieza a comprobar en el minuto 3. **Salida esperada**

```
Medias móviles: [127.7, 224.3, 226.0, 227.3, 132.3, 215.7, 303.3, 305.0]
Ventanas calculadas: 8
Ventana más cargada: minutos 7-9 (media 305.0)
Minutos con pico: [3, 7]
```

✅ Solución

```python
eventos_por_minuto = [120, 135, 128, 410, 140, 132, 125, 390, 395, 130]
ventana = 3

medias = []
for i in range(len(eventos_por_minuto) - ventana + 1):
    trozo = eventos_por_minuto[i:i + ventana]
    medias.append(round(sum(trozo) / ventana, 1))
print("Medias móviles:", medias)
print("Ventanas calculadas:", len(medias))

i_max = medias.index(max(medias))
print(f"Ventana más cargada: minutos {i_max}-{i_max + ventana - 1} (media {max(medias)})")

picos = []
for m in range(3, len(eventos_por_minuto)):
    media_previa = sum(eventos_por_minuto[m - 3:m]) / 3
    if eventos_por_minuto[m] > 2 * media_previa:
        picos.append(m)
print("Minutos con pico:", picos)
```

🖥️ Salida:

```
Medias móviles: [127.7, 224.3, 226.0, 227.3, 132.3, 215.7, 303.3, 305.0]
Ventanas calculadas: 8
Ventana más cargada: minutos 7-9 (media 305.0)
Minutos con pico: [3, 7]
```

Sección 2. Tuplas

Una tupla es una colección **ordenada e inmutable**: una vez creada, no se puede cambiar. Por eso encaja muy bien para representar un registro cerrado, una clave compuesta o una fila que ningún paso del pipeline debe alterar por accidente.

Ejemplos resueltos

```python
evento = ("2026-10-01T08:15:00", "sensor_07", 21.4)

print("Evento completo:", evento)
print("Sensor:", evento[1])
print("Campos:", len(evento))

# Desempaquetado: cada campo en su variable
momento, sensor, temperatura = evento
print(sensor, "midió", temperatura, "ºC a las", momento[11:16])

# Una tupla no se puede modificar:
# evento[2] = 22.0   ->  TypeError: 'tuple' object does not support item assignment

# Para "corregir" un valor se construye una tupla nueva
evento_corregido = evento[:2] + (22.0,)
print("Original: ", evento)
print("Corregido:", evento_corregido)

un_campo = ("sensor_07",)    # con coma: tupla de un elemento
sin_coma = ("sensor_07")     # sin coma: es solo un str entre paréntesis
print(type(un_campo).__name__, type(sin_coma).__name__)
```

Paso a paso

40. Se accede por índice igual que en una lista, y `len()` funciona igual.

41. El **desempaquetado** asigna cada campo a una variable. Debe haber tantas variables como elementos.
42. `momento[11:16]` extrae la hora (`08:15`) del texto con formato ISO.
43. Intentar asignar `evento[2] = 22.0` produce un `TypeError`. Para "corregir" un valor se construye una tupla nueva concatenando trozos: `evento[:2] + (22.0,)`.
44. Una tupla de un solo elemento necesita la coma final. Sin la coma, los paréntesis solo agrupan y el resultado es un `str`. **Salida esperada**

```
Evento completo: ('2026-10-01T08:15:00', 'sensor_07', 21.4)
Sensor: sensor_07
Campos: 3
sensor_07 midió 21.4 ºC a las 08:15
Original:  ('2026-10-01T08:15:00', 'sensor_07', 21.4)
Corregido: ('2026-10-01T08:15:00', 'sensor_07', 22.0)
tuple str
```

```
from operator import itemgetter

lecturas = [
    ("sensor_01", "Madrid", 21.4),
    ("sensor_02", "Sevilla", 31.8),
    ("sensor_03", "Bilbao", 17.2),
    ("sensor_04", "Sevilla", 29.5),
    ("sensor_05", "Madrid", 24.9),
]

# Desempaquetado directo en el for
for sensor, ciudad, temp in lecturas:
    if temp > 25:
        print("ALERTA", sensor, ciudad, temp)

ordenadas = sorted(lecturas, key=itemgetter(2))
print("Más fría:  ", ordenadas[0])
print("Más cálida:", ordenadas[-1])

# Ordenar por ciudad y, dentro de cada ciudad, por temperatura
for fila in sorted(lecturas, key=itemgetter(1, 2)):
    print(fila)
```

Paso a paso

45. `for sensor, ciudad, temp in lecturas` desempaqueta cada tupla directamente en el bucle, sin usar índices.
46. `itemgetter(2)` ordena por temperatura.
47. `itemgetter(1, 2)` ordena por **dos criterios**: primero ciudad y, si hay empate, temperatura. Funciona porque las tuplas se comparan elemento a elemento, de izquierda a derecha. **Salida esperada**

```
ALERTA sensor_02 Sevilla 31.8
ALERTA sensor_04 Sevilla 29.5
Más fría:   ('sensor_03', 'Bilbao', 17.2)
Más cálida: ('sensor_02', 'Sevilla', 31.8)
('sensor_03', 'Bilbao', 17.2)
('sensor_01', 'Madrid', 21.4)
('sensor_05', 'Madrid', 24.9)
('sensor_04', 'Sevilla', 29.5)
('sensor_02', 'Sevilla', 31.8)
```

```
particiones = ("2026-09-26", "2026-09-27", "2026-09-28", "2026-09-29", "2026-09-30")

primera, *intermedias, ultima = particiones
print("Desde:", primera, "hasta:", ultima)
print("Intermedias:", intermedias)

# Duración de un job en segundos -> horas, minutos y segundos
duracion = 7384
horas, resto = divmod(duracion, 3600)
minutos, segundos = divmod(resto, 60)
print(f"El job tardó{horas} h{minutos} min{segundos} s")

# Intercambio de variables en una sola línea
```

```
origen, destino = "staging", "produccion"
origen, destino = destino, origen
print(origen, destino)
```

Paso a paso

48. `primera, *intermedias, ultima` asigna el primer y el último elemento a sus variables. Lo que queda en medio va a `intermedias`, que siempre es una **lista**.
49. `divmod(a, b)` devuelve una tupla `(cociente, resto)`. 7384 s entre 3600 da 2 h y sobran 184 s. Esos 184 s son 3 min y 4 s.
50. `origen, destino = destino, origen` funciona porque Python primero crea la tupla `(destino, origen)` y después la desempaqueta. No hace falta una variable auxiliar. **Salida esperada**

```
Desde: 2026-09-26 hasta: 2026-09-30
Intermedias: ['2026-09-27', '2026-09-28', '2026-09-29']
El job tardó 2 h 3 min 4 s
produccion staging
```

Los formatos como Parquet guardan los datos **por columnas**, mientras que un CSV o una API los entregan **por filas**. `zip()` permite pasar de un formato al otro.

```
ids = [101, 102, 103, 104]
clientes = ["Acme", "Globex", "Initech", "Umbrella"]
importes = [1200.0, 340.5, 980.0, 15.0]

# De columnas a filas: cada fila es una tupla
filas = list(zip(ids, clientes, importes))
print("Filas:")
for fila in filas:
    print(" ", fila)

# De filas a columnas: zip(*filas) desempaqueta las filas
col_ids, col_clientes, col_importes = zip(*filas)
print("Columna clientes:", col_clientes)
print("Total importes:", sum(col_importes))
```

Paso a paso

51. `zip(ids, clientes, importes)` toma un elemento de cada lista en cada vuelta y los junta en una tupla.
52. `zip(*filas)` hace la operación inversa. El pasa cada fila como un argumento separado, y `zip` agrupa los primeros elementos de todas las filas, después los segundos, etc.
53. Cada columna que devuelve `zip` es una tupla, por eso `col_clientes` aparece entre paréntesis. **Salida esperada**

```
Filas:
  (101, 'Acme', 1200.0)
  (102, 'Globex', 340.5)
  (103, 'Initech', 980.0)
  (104, 'Umbrella', 15.0)
Columna clientes: ('Acme', 'Globex', 'Initech', 'Umbrella')
Total importes: 2535.5
```

MapReduce, el modelo de procesamiento de Hadoop, trabaja siempre con **pares clave-valor**, y en Python un par es una tupla. Vamos a simular un conteo de palabras sobre líneas de log en tres fases.

```
from operator import itemgetter

# Cada línea la procesaría un nodo distinto del clúster
splits = [
    "ERROR disco lleno en datanode3",
    "WARN latencia alta en datanode1",
    "ERROR timeout en datanode1",
]

# MAP: cada palabra se emite como una tupla (clave, 1)
pares = []
for linea in splits:
    for palabra in linea.split():
```

```
        pares.append((palabra, 1))
print("Pares emitidos:", len(pares))
print("Primeros pares:", pares[:3])

# SHUFFLE / SORT: al ordenar por clave, las tuplas iguales quedan juntas
pares.sort(key=itemgetter(0))

# REDUCE: sumar los valores de cada grupo de claves consecutivas
resultado = []
clave_actual, suma = pares[0]
for clave, valor in pares[1:]:
    if clave == clave_actual:
        suma += valor
    else:
        resultado.append((clave_actual, suma))
        clave_actual, suma = clave, valor
resultado.append((clave_actual, suma))

print("Claves distintas:", len(resultado))
for clave, total in resultado:
    if total > 1:
        print(clave, total)
```

Paso a paso

54. **Map:** cada nodo procesa su trozo de datos (*split*) y emite una tupla `(palabra, 1)` por cada palabra, sin contar nada todavía.
55. **Shuffle / sort:** el sistema ordena los pares por clave para que las tuplas de una misma palabra queden juntas. En un clúster, esta fase mueve datos por la red entre nodos y es la más costosa.
56. **Reduce:** se recorren los pares ordenados. Mientras la clave se repite, se acumula; cuando cambia, se guarda el total de la clave anterior y se empieza a contar la nueva.
57. El último grupo se añade después del bucle, porque no hay ninguna clave nueva que provoque su guardado.
58. Python ordena las mayúsculas antes que las minúsculas, por eso `ERROR` aparece primero. **Salida esperada**

```
Pares emitidos: 14
Primeros pares: [('ERROR', 1), ('disco', 1), ('lleno', 1)]
Claves distintas: 10
ERROR 2
datanode1 2
en 3
```

Kafka identifica cada mensaje por tres datos: el topic, la partición y su posición dentro de ella (*offset*). El **lag** es el número de mensajes que el consumidor todavía no ha procesado.

```
from operator import itemgetter

# (topic, partición, offset): último mensaje escrito en el broker
ultimos = [("pedidos", 0, 15230), ("pedidos", 1, 14980), ("pedidos", 2, 15410)]
# (topic, partición, offset): último mensaje confirmado por el consumidor
confirmados = [("pedidos", 0, 15228), ("pedidos", 1, 13100), ("pedidos", 2, 15410)]

lags = []
for (topic, particion, fin), (_, _, leido) in zip(ultimos, confirmados):
    lags.append((topic, particion, fin - leido))

for lag in sorted(lags, key=itemgetter(2), reverse=True):
    print(lag)

total = 0
for _, _, pendientes in lags:
    total += pendientes
print("Lag total del consumidor:", total)
print("Partición más retrasada:", max(lags, key=itemgetter(2))[:2])
```

Paso a paso

59. `zip()` empareja la tupla de cada partición en las dos listas. Funciona porque ambas listas están en el mismo orden de partición.
60. En el `for` se desempaquetan **dos tuplas a la vez**. El guion bajo `_` indica los campos que no necesitamos.
61. Lag = offset escrito − offset confirmado. La partición 1 acumula 1880 mensajes pendientes: el consumidor asignado a ella va retrasado.
62. `max(...)[:2]` toma la tupla con más lag y se queda con sus dos primeros campos, topic y partición. **Salida esperada**

```
('pedidos', 1, 1880)
('pedidos', 0, 2)
('pedidos', 2, 0)
Lag total del consumidor: 1882
Partición más retrasada: ('pedidos', 1)
```

Ejercicios propuestos

Ejercicio propuesto 2.1

```python
# (pedido_id, cliente, importe, estado)
pedidos = [
    (5001, "Acme", 250.0, "ENTREGADO"),
    (5002, "Globex", 1340.0, "ENTREGADO"),
    (5003, "Initech", 89.9, "CANCELADO"),
    (5004, "Acme", 610.0, "ENTREGADO"),
    (5005, "Umbrella", 45.0, "CANCELADO"),
    (5006, "Globex", 300.0, "PENDIENTE"),
]
```

Recorre los pedidos con desempaquetado en el `for` y muestra:

- La facturación de los pedidos `ENTREGADO`.
- El id y el cliente del pedido de mayor importe. Usa `max()` con `key`.
- Una lista de tuplas `(cliente, importe)` con los pedidos `CANCELADO`.
- Cuántos pedidos hay en cada estado. Guarda los estados en una lista y usa `count()`. **Salida esperada**

```
Facturación entregada: 2200.0
Pedido de mayor importe: 5002 de Globex
Cancelados: [('Initech', 89.9), ('Umbrella', 45.0)]
ENTREGADO: 3 | CANCELADO: 2 | PENDIENTE: 1
```

✅ Solución

```python
pedidos = [
    (5001, "Acme", 250.0, "ENTREGADO"),
    (5002, "Globex", 1340.0, "ENTREGADO"),
    (5003, "Initech", 89.9, "CANCELADO"),
    (5004, "Acme", 610.0, "ENTREGADO"),
    (5005, "Umbrella", 45.0, "CANCELADO"),
    (5006, "Globex", 300.0, "PENDIENTE"),
]

facturacion, cancelados, estados = 0, [], []
for pid, cliente, importe, estado in pedidos:          # desempaquetado en el for
    if estado == "ENTREGADO":
        facturacion += importe
    if estado == "CANCELADO":
        cancelados.append((cliente, importe))
    estados.append(estado)

pid, cliente, _, _ = max(pedidos, key=lambda p: p[2])
print("Facturación entregada:", facturacion)
print(f"Pedido de mayor importe: {pid} de {cliente}")
print("Cancelados:", cancelados)
print(" | ".join(f"{e}: {estados.count(e)}" for e in ("ENTREGADO", "CANCELADO", "PENDIENTE")))
```

🖥️ Salida:

```
Facturación entregada: 2200.0
Pedido de mayor importe: 5002 de Globex
Cancelados: [('Initech', 89.9), ('Umbrella', 45.0)]
ENTREGADO: 3 | CANCELADO: 2 | PENDIENTE: 1
```

Ejercicio propuesto 2.2

El esquema de una tabla se describe como una tupla de tuplas `(columna, tipo)`. Comprueba cada fila con `isinstance()` e informa de cada valor que no tenga el tipo esperado.

```python
esquema = (("id", int), ("cliente", str), ("importe", float), ("activo", bool))

filas = [
    (1, "Acme", 120.5, True),
    (2, "Globex", "340", True),
    ("3", "Initech", 99.9, False),
    (4, "Umbrella", 15.0, "no"),
]
```

Pistas: `enumerate(filas, start=1)` numera las filas desde 1, y `zip(esquema, fila)` empareja cada columna del esquema con su valor. El nombre de un tipo se obtiene con `tipo.__name__`.

Salida esperada

```
Fila 2: 'importe' esperaba float y recibió str
Fila 3: 'id' esperaba int y recibió str
Fila 4: 'activo' esperaba bool y recibió str
Filas: 4 | Errores: 3
```

✅ Solución

```python
esquema = (("id", int), ("cliente", str), ("importe", float), ("activo", bool))
filas = [
    (1, "Acme", 120.5, True),
    (2, "Globex", "340", True),
    ("3", "Initech", 99.9, False),
    (4, "Umbrella", 15.0, "no"),
]

errores = 0
for n, fila in enumerate(filas, start=1):
    for (columna, tipo), valor in zip(esquema, fila):
        if not isinstance(valor, tipo):
            print(f"Fila {n}: '{columna}' esperaba {tipo.__name__} y recibió {type(valor).__name__}")
            errores += 1
print(f"Filas: {len(filas)} | Errores: {errores}")
```

🖥️ Salida:

```
Fila 2: 'importe' esperaba float y recibió str
Fila 3: 'id' esperaba int y recibió str
Fila 4: 'activo' esperaba bool y recibió str
Filas: 4 | Errores: 3
```

Ejercicio propuesto 2.3

Un registro de esquemas guarda versiones como texto. Compararlas como texto da resultados erróneos, porque `"1.10.0"` se considera menor que `"1.9.7"`.

```python
versiones = ["1.4.2", "1.10.0", "1.9.7", "2.0.0", "1.10.3"]
```

63. Muestra el resultado incorrecto al comparar y ordenar como texto.
64. Convierte cada versión en una tupla de enteros `(mayor, menor, parche)`.
65. Ordénalas, obtén la última y muéstrala con el formato `v2.0.0`. **Salida esperada**

```
Como texto, ¿1.10.0 > 1.9.7? False
Ordenadas como texto: ['1.10.0', '1.10.3', '1.4.2', '1.9.7', '2.0.0']
Como tuplas: [(1, 4, 2), (1, 10, 0), (1, 9, 7), (2, 0, 0), (1, 10, 3)]
Ordenadas: [(1, 4, 2), (1, 9, 7), (1, 10, 0), (1, 10, 3), (2, 0, 0)]
Última versión: v2.0.0
¿1.10.0 es posterior a 1.9.7? True
```

✅ Solución

```python
versiones = ["1.4.2", "1.10.0", "1.9.7", "2.0.0", "1.10.3"]

print("Como texto, ¿1.10.0 > 1.9.7?", "1.10.0" > "1.9.7")   # compara carácter a carácter
print("Ordenadas como texto:", sorted(versiones))

tuplas = [tuple(int(p) for p in v.split(".")) for v in versiones]
print("Como tuplas:", tuplas)
ordenadas = sorted(tuplas)                                    # las tuplas se comparan elemento a elemento
print("Ordenadas:", ordenadas)
ultima = ordenadas[-1]
print(f"Última versión: v{ultima[0]}.{ultima[1]}.{ultima[2]}")
print("¿1.10.0 es posterior a 1.9.7?", (1, 10, 0) > (1, 9, 7))
```

🖥️ Salida:

```
Como texto, ¿1.10.0 > 1.9.7? False
Ordenadas como texto: ['1.10.0', '1.10.3', '1.4.2', '1.9.7', '2.0.0']
Como tuplas: [(1, 4, 2), (1, 10, 0), (1, 9, 7), (2, 0, 0), (1, 10, 3)]
Ordenadas: [(1, 4, 2), (1, 9, 7), (1, 10, 0), (1, 10, 3), (2, 0, 0)]
Última versión: v2.0.0
¿1.10.0 es posterior a 1.9.7? True
```

Ejercicio propuesto 2.4

Cada tupla indica el minuto del día en que arranca y termina un job. Dos jobs que comparten clúster no deberían solaparse.

```python
# (job, minuto_inicio, minuto_fin) dentro del día
ejecuciones = [
    ("job_c", 130, 170),
    ("job_a", 60, 95),
    ("job_d", 165, 200),
    ("job_b", 90, 120),
]
```

Ordena las ejecuciones por hora de inicio. Después compara cada ejecución con la siguiente: hay solape si la siguiente arranca antes de que termine la actual. Para cada solape, muestra los jobs implicados y los minutos de solape.

Salida esperada

```
Orden de arranque: ['job_a', 'job_b', 'job_c', 'job_d']
SOLAPE: job_b arranca en 90 y job_a termina en 95 (5 min)
SOLAPE: job_d arranca en 165 y job_c termina en 170 (5 min)
Solapes detectados: 2
```

✅ Solución

```python
ejecuciones = [
    ("job_c", 130, 170),
    ("job_a", 60, 95),
    ("job_d", 165, 200),
    ("job_b", 90, 120),
]

orden = sorted(ejecuciones, key=lambda e: e[1])
print("Orden de arranque:", [e[0] for e in orden])
```

```
solapes = 0
for actual, siguiente in zip(orden, orden[1:]):          # pares consecutivos
    if siguiente[1] < actual[2]:
        print(f"SOLAPE: {siguiente[0]} arranca en {siguiente[1]} y {actual[0]} "
              f"termina en {actual[2]} ({actual[2] - siguiente[1]} min)")
        solapes += 1
print("Solapes detectados:", solapes)
```

🖥️ Salida:

```
Orden de arranque: ['job_a', 'job_b', 'job_c', 'job_d']
SOLAPE: job_b arranca en 90 y job_a termina en 95 (5 min)
SOLAPE: job_d arranca en 165 y job_c termina en 170 (5 min)
Solapes detectados: 2
```

Ejercicio propuesto 2.5

Cada línea de log tiene el formato `método ruta código_http bytes`.

```
# método ruta código_http bytes
logs = [
    "GET /pedidos 200 5120",
    "GET /clientes 404 310",
    "POST /pedidos 201 880",
    "GET /pedidos 200 4980",
    "GET /stock 500 120",
    "GET /clientes 200 2210",
    "GET /stock 500 95",
    "POST /pedidos 201 910",
]
```

Simula un job MapReduce como el del Ejemplo 2.5:

66. **Map:** emite una tupla `(código, bytes)` por línea. Los bytes deben ser `int`.
67. **Shuffle / sort:** ordena los pares por código.
68. **Reduce:** obtén una lista de tuplas `(código, peticiones, bytes_totales)` y muéstrala. **Salida esperada**

```
Fase map: [('200', 5120), ('404', 310), ('201', 880)] ...
200: 3 peticiones, 12310 bytes
201: 2 peticiones, 1790 bytes
404: 1 peticiones, 310 bytes
500: 2 peticiones, 215 bytes
```

✅ Solución

```
logs = [
    "GET /pedidos 200 5120", "GET /clientes 404 310", "POST /pedidos 201 880",
    "GET /pedidos 200 4980", "GET /stock 500 120", "GET /clientes 200 2210",
    "GET /stock 500 95", "POST /pedidos 201 910",
]

# MAP
pares = []
for linea in logs:
    _, _, codigo, nbytes = linea.split()
    pares.append((codigo, int(nbytes)))
print("Fase map:", pares[:3], "...")

# SHUFFLE / SORT
pares.sort()

# REDUCE: los pares del mismo código están juntos tras ordenar
resultado = []
for codigo, nbytes in pares:
    if resultado and resultado[-1][0] == codigo:
        c, n, total = resultado[-1]
        resultado[-1] = (c, n + 1, total + nbytes)      # las tuplas no se modifican: se reemplazan
    else:
```

```
        resultado.append((codigo, 1, nbytes))

for codigo, peticiones, total in resultado:
    print(f"{codigo}: {peticiones} peticiones, {total} bytes")
```

🖥️ **Salida:**

```
Fase map: [('200', 5120), ('404', 310), ('201', 880)] ...
200: 3 peticiones, 12310 bytes
201: 2 peticiones, 1790 bytes
404: 1 peticiones, 310 bytes
500: 2 peticiones, 215 bytes
```

Ejercicio propuesto 2.6

Cuando una consulta filtra por fecha sobre una tabla particionada, el motor solo lee las carpetas de las fechas pedidas y descarta el resto sin abrirlas. Esto se llama *partition pruning*.

```
# (año, mes, día, número de ficheros) de cada partición de la tabla ventas
particiones = [
    (2026, 9, 27, 14), (2026, 9, 28, 12), (2026, 9, 29, 15), (2026, 9, 30, 13),
    (2026, 10, 1, 16), (2026, 10, 2, 11), (2026, 10, 3, 12),
]
desde = (2026, 9, 29)
hasta = (2026, 10, 1)
```

69. Selecciona las particiones cuya fecha `(año, mes, día)` esté entre `desde` y `hasta`, ambos incluidos. Compara tuplas directamente, sin convertirlas a texto.
70. Muestra la ruta de cada partición seleccionada con el formato `ventas/year=AAAA/month=MM/day=DD` y su número de ficheros.
71. Indica cuántos ficheros se leen del total y qué porcentaje se ha descartado. **Salida esperada**

```
ventas/year=2026/month=09/day=29 -> 15 ficheros
ventas/year=2026/month=09/day=30 -> 13 ficheros
ventas/year=2026/month=10/day=01 -> 16 ficheros
Ficheros leídos: 44 de 93
Ficheros descartados: 52.7 %
```

✅ **Solución**

```
particiones = [
    (2026, 9, 27, 14), (2026, 9, 28, 12), (2026, 9, 29, 15), (2026, 9, 30, 13),
    (2026, 10, 1, 16), (2026, 10, 2, 11), (2026, 10, 3, 12),
]
desde = (2026, 9, 29)
hasta = (2026, 10, 1)

leidos = 0
for anio, mes, dia, ficheros in particiones:
    if desde <= (anio, mes, dia) <= hasta:              # comparación directa de tuplas
        print(f"ventas/year={anio}/month={mes:02d}/day={dia:02d} -> {ficheros} ficheros")
        leidos += ficheros

total = sum(p[3] for p in particiones)
print(f"Ficheros leídos: {leidos} de {total}")
print(f"Ficheros descartados: {(total - leidos) / total * 100:.1f} %")
```

🖥️ **Salida:**

```
ventas/year=2026/month=09/day=29 -> 15 ficheros
ventas/year=2026/month=09/day=30 -> 13 ficheros
ventas/year=2026/month=10/day=01 -> 16 ficheros
Ficheros leídos: 44 de 93
Ficheros descartados: 52.7 %
```

Sección 3. Set

Un `set` es una colección **sin orden y sin duplicados**. Sus dos usos principales en ingeniería de datos son **deduplicar** y **comparar colecciones**: qué registros faltan, cuáles sobran y cuáles coinciden. Además, comprobar si un elemento pertenece a un `set` es mucho más rápido que buscarlo en una lista.

> Como un `set` no tiene orden, los ejemplos usan `sorted()` para mostrarlos. Así la salida es siempre la misma.

Ejemplos resueltos

```
crm = ["C001", "C002", "C003", "C002", "C004", "C001"]
ecommerce = ["C003", "C005", "C004", "C006", "C005"]

print("Registros en CRM:", len(crm), "| únicos:", len(set(crm)))

clientes = set(crm)
clientes.update(ecommerce)      # añade todos los elementos de la otra fuente
print("Clientes únicos totales:", len(clientes))
print(sorted(clientes))

clientes.add("C007")
clientes.add("C001")            # ya existe: no cambia nada
clientes.discard("C999")        # no existe: discard() no da error
print("Tras las altas:", len(clientes))
```

Paso a paso

72. `set(crm)` elimina automáticamente los duplicados: 6 registros se quedan en 4 clientes.
73. `update()` añade todos los elementos de otra colección. Los repetidos se ignoran.
74. `add()` añade un elemento. Si ya existía, no pasa nada.
75. `discard()` elimina un elemento sin dar error si no existe. `remove()`, en cambio, lanzaría `KeyError`. **Salida esperada**

```
Registros en CRM: 6 | únicos: 4
Clientes únicos totales: 6
['C001', 'C002', 'C003', 'C004', 'C005', 'C006']
Tras las altas: 7
```

Después de una carga, hay que comprobar que todos los ids del sistema origen han llegado al data warehouse.

```
origen = {101, 102, 103, 104, 105, 106}
destino = {101, 102, 104, 106, 107}

print("Cargados correctamente:", sorted(origen & destino))
print("Faltan en destino:     ", sorted(origen - destino))
print("Sobran en destino:     ", sorted(destino - origen))
print("Discrepancias totales: ", sorted(origen ^ destino))
print("¿Carga completa?", origen <= destino)
```

Paso a paso

Operador	Operación	Pregunta que responde
`a & b`	Intersección	¿Qué está en los dos?
`a - b`	Diferencia	¿Qué está en `a` pero no en `b`?
`a ^ b`	Diferencia simétrica	¿Qué está solo en uno de los dos?
`a <= b`	Subconjunto	¿Está todo `a` dentro de `b`?

Los ids 103 y 105 no se cargaron, y el 107 está en destino aunque no viene del origen. Puede ser un registro antiguo que debería haberse borrado.

Salida esperada

```
Cargados correctamente: [101, 102, 104, 106]
Faltan en destino:      [103, 105]
Sobran en destino:      [107]
Discrepancias totales:  [103, 105, 107]
¿Carga completa? False
```

```
esperadas = {"id_pedido", "fecha", "cliente", "importe", "moneda"}
recibidas = ["id_pedido", "fecha", "importe", "cliente", "canal", "importe"]

recibidas_set = set(recibidas)
print("¿Columnas duplicadas en origen?", len(recibidas) != len(recibidas_set))
print("Faltan:", sorted(esperadas - recibidas_set))
print("Nuevas:", sorted(recibidas_set - esperadas))
print("¿Esquema válido?", esperadas.issubset(recibidas_set))
```

Paso a paso

76. Si al convertir la lista en `set` se reduce su tamaño, había columnas duplicadas: `importe` aparece dos veces.
77. `esperadas - recibidas_set` indica qué columnas faltan, y la resta inversa indica las columnas nuevas.
78. `issubset()` es la versión con método del operador `<=`. **Salida esperada**

```
¿Columnas duplicadas en origen? True
Faltan: ['moneda']
Nuevas: ['canal']
¿Esquema válido? False
```

```
ips_bloqueadas = {"10.0.0.13", "192.168.1.50", "172.16.4.2"}

accesos = [
    ("08:01", "10.0.0.7"),
    ("08:02", "192.168.1.50"),
    ("08:02", "10.0.0.8"),
    ("08:03", "172.16.4.2"),
    ("08:05", "192.168.1.50"),
]

sospechosos = []
ips_vistas = set()
for hora, ip in accesos:
    ips_vistas.add(ip)
    if ip in ips_bloqueadas:          # búsqueda muy rápida en un set
        sospechosos.append((hora, ip))

print("Accesos bloqueados:", len(sospechosos))
for s in sospechosos:
    print(" ", s)
print("IPs distintas vistas:", len(ips_vistas))
print("Bloqueadas que no han aparecido:", ips_bloqueadas - ips_vistas)
```

Paso a paso

79. `ip in ips_bloqueadas` es casi instantáneo aunque el `set` tenga millones de IPs, porque internamente se busca por *hash*. En una lista habría que revisar los elementos uno a uno.
80. Un `set` puede contener tuplas, porque son inmutables, pero no listas. `{[1, 2]}` produce `TypeError: unhashable type: 'list'`.
81. Al restar las IPs vistas de las bloqueadas sabemos qué IPs de la lista negra no han aparecido en este log. **Salida esperada**

```
Accesos bloqueados: 3
  ('08:02', '192.168.1.50')
  ('08:03', '172.16.4.2')
  ('08:05', '192.168.1.50')
IPs distintas vistas: 4
Bloqueadas que no han aparecido: {'10.0.0.13'}
```

Contar elementos distintos (`COUNT(DISTINCT ...)`) es una de las operaciones más delicadas en un sistema distribuido, porque un mismo usuario puede aparecer en varias particiones.

```
# Usuarios que generaron eventos, repartidos en 3 particiones
particion_0 = ["u1", "u2", "u3", "u2", "u4"]
particion_1 = ["u3", "u5", "u1", "u6"]
particion_2 = ["u2", "u7", "u5", "u5"]

unicos_0 = set(particion_0)
unicos_1 = set(particion_1)
unicos_2 = set(particion_2)
print("Únicos por partición:", len(unicos_0), len(unicos_1), len(unicos_2))

suma_parcial = len(unicos_0) + len(unicos_1) + len(unicos_2)
print("Suma de los únicos parciales:", suma_parcial, "(incorrecto)")

globales = unicos_0 | unicos_1 | unicos_2
print("Únicos globales:", len(globales), sorted(globales))
print("Usuarios contados de más:", suma_parcial - len(globales))
```

Paso a paso

82. Cada partición calcula su `set` de usuarios únicos de forma independiente, como haría cada nodo del clúster.
83. **Sumar los tamaños de los `set` parciales es un error**: `u1`, `u2`, `u3` y `u5` aparecen en más de una partición y se cuentan varias veces.
84. El resultado correcto exige **unir** los `set` (`|`) y contar después. En un clúster, esto obliga a mover los identificadores por la red hasta un mismo nodo.
85. Con miles de millones de usuarios, unir los `set` exactos es muy caro. Por eso los motores de Big Data ofrecen conteos aproximados, como `approx_count_distinct()` en Spark. **Salida esperada**

```
Únicos por partición: 4 4 3
Suma de los únicos parciales: 11 (incorrecto)
Únicos globales: 7 ['u1', 'u2', 'u3', 'u4', 'u5', 'u6', 'u7']
Usuarios contados de más: 4
```

Muchos sistemas de mensajería garantizan que cada evento llega **al menos una vez** (*at-least-once*). Si hay un fallo, el evento se reenvía, así que el consumidor debe descartar los repetidos para no escribirlos dos veces.

```
procesados = set()        # ids de eventos ya escritos en destino

micro_lotes = [
    ["e01", "e02", "e03"],
    ["e03", "e04", "e05"],          # e03 se reenvió tras un fallo
    ["e05", "e06", "e02", "e07"],   # e05 y e02 llegan otra vez
]

for num, lote in enumerate(micro_lotes, start=1):
    nuevos = []
    for evento in lote:
        if evento not in procesados:
            nuevos.append(evento)
            procesados.add(evento)
    descartados = len(lote) - len(nuevos)
    print(f"Lote{num}: recibidos{len(lote)}, escritos{nuevos}, descartados{descartados}")

print("Eventos escritos en total:", len(procesados))
```

Paso a paso

86. `procesados` es un `set` que se mantiene entre micro-lotes y recuerda qué eventos ya se han escrito.
87. Para cada evento, `in` comprueba de forma inmediata si ya se procesó. Si es nuevo, se escribe y se añade al `set`.
88. `enumerate(..., start=1)` numera los lotes desde 1.
89. Gracias a esta comprobación, procesar varias veces el mismo evento produce el mismo resultado. Esta propiedad se llama **idempotencia**.

90. En un sistema real este `set` no puede crecer sin límite. Spark Structured Streaming, por ejemplo, olvida los ids más antiguos que un tiempo configurado (*watermark*). **Salida esperada**

```
Lote 1: recibidos 3, escritos ['e01', 'e02', 'e03'], descartados 0
Lote 2: recibidos 3, escritos ['e04', 'e05'], descartados 1
Lote 3: recibidos 4, escritos ['e06', 'e07'], descartados 2
Eventos escritos en total: 7
```

Ejercicios propuestos

Ejercicio propuesto 3.1

Cada `set` contiene los usuarios que abrieron la app ese día.

```
lunes = {"u01", "u02", "u03", "u04", "u05", "u06"}
martes = {"u02", "u03", "u05", "u07", "u08"}
miercoles = {"u01", "u03", "u05", "u08", "u09"}
```

Calcula, solo con operadores de `set`:

- Los usuarios activos los tres días.
- Cuántos usuarios distintos estuvieron activos al menos un día.
- Los que solo entraron el lunes.
- Los usuarios nuevos del miércoles, es decir, los que no estuvieron ni el lunes ni el martes.
- La retención del lunes al martes: porcentaje de usuarios del lunes que volvieron el martes. **Salida esperada**

```
Activos los tres días: ['u03', 'u05']
Activos al menos un día: 9
Solo el lunes: ['u04', 'u06']
Nuevos el miércoles: ['u09']
Retención lunes -> martes: 50.0 %
```

✅ Solución

```
lunes = {"u01", "u02", "u03", "u04", "u05", "u06"}
martes = {"u02", "u03", "u05", "u07", "u08"}
miercoles = {"u01", "u03", "u05", "u08", "u09"}

print("Activos los tres días:", sorted(lunes & martes & miercoles))      # intersección
print("Activos al menos un día:", len(lunes | martes | miercoles))      # unión
print("Solo el lunes:", sorted(lunes - martes - miercoles))             # diferencia
print("Nuevos el miércoles:", sorted(miercoles - (lunes | martes)))
print(f"Retención lunes -> martes: {len(lunes & martes) / len(lunes) * 100:.1f} %")
```

🖥️ Salida:

```
Activos los tres días: ['u03', 'u05']
Activos al menos un día: 9
Solo el lunes: ['u04', 'u06']
Nuevos el miércoles: ['u09']
Retención lunes -> martes: 50.0 %
```

Ejercicio propuesto 3.2

Antes de publicar datasets en una zona compartida, el equipo de gobierno del dato pide revisar si contienen columnas con información personal (PII).

```
pii = {"dni", "email", "telefono", "iban", "fecha_nacimiento"}

datasets = [
    ("ventas", {"id_venta", "fecha", "importe", "id_cliente"}),
    ("clientes", {"id_cliente", "nombre", "email", "telefono", "ciudad"}),
    ("pagos", {"id_pago", "iban", "importe", "fecha"}),
```

```
    ("logs_web", {"timestamp", "url", "user_agent"}),
]
```

Para cada dataset, usa `isdisjoint()` para saber si está libre de PII. Si no lo está, muestra qué columnas sensibles contiene. Al final, indica cuántos datasets necesitan enmascarado.

Salida esperada

```
ventas    sin PII
clientes  PII -> ['email', 'telefono']
pagos     PII -> ['iban']
logs_web  sin PII
Datasets que requieren enmascarado: 2
```

✅ **Solución**

```
pii = {"dni", "email", "telefono", "iban", "fecha_nacimiento"}
datasets = [
    ("ventas", {"id_venta", "fecha", "importe", "id_cliente"}),
    ("clientes", {"id_cliente", "nombre", "email", "telefono", "ciudad"}),
    ("pagos", {"id_pago", "iban", "importe", "fecha"}),
    ("logs_web", {"timestamp", "url", "user_agent"}),
]

a_enmascarar = 0
for nombre, columnas in datasets:
    if columnas.isdisjoint(pii):                    # True si no comparten ningún elemento
        print(f"{nombre:<9} sin PII")
    else:
        print(f"{nombre:<9} PII -> {sorted(columnas & pii)}")
        a_enmascarar += 1
print("Datasets que requieren enmascarado:", a_enmascarar)
```

🖥️ **Salida:**

```
ventas    sin PII
clientes  PII -> ['email', 'telefono']
pagos     PII -> ['iban']
logs_web  sin PII
Datasets que requieren enmascarado: 2
```

Ejercicio propuesto 3.3

`set(transacciones)` elimina los duplicados, pero no dice **cuáles** estaban repetidos. Usa dos `set`, `vistos` y `repetidos`, en un solo recorrido.

```
transacciones = ["T10", "T11", "T12", "T11", "T13", "T14", "T12", "T11", "T15"]
```

Salida esperada

```
Repetidos: ['T11', 'T12']
Únicos: 6 de 9
```

✅ **Solución**

```
transacciones = ["T10", "T11", "T12", "T11", "T13", "T14", "T12", "T11", "T15"]

vistos, repetidos = set(), set()
for t in transacciones:
    if t in vistos:            # búsqueda en set: O(1)
        repetidos.add(t)
    vistos.add(t)

print("Repetidos:", sorted(repetidos))
print(f"Únicos: {len(vistos)} de {len(transacciones)}")
```

🖥️ **Salida:**

```
Repetidos: ['T11', 'T12']
Únicos: 6 de 9
```

Ejercicio propuesto 3.4

```
rutas_hoy = [("MAD", "BCN"), ("BCN", "MAD"), ("MAD", "LIS"), ("MAD", "BCN"),
             ("SVQ", "MAD"), ("LIS", "MAD"), ("BIO", "PMI")]
rutas_manana = [("PMI", "BIO"), ("MAD", "VLC")]
```

91. Crea un `set` de rutas distintas. Una ruta es una tupla `(origen, destino)`.
92. Obtén las rutas que no tienen vuelta, es decir, aquellas en las que `(destino, origen)` no está en el `set`.
93. Incorpora las rutas de mañana con el operador `|=` y repite el cálculo. **Salida esperada**

```
Rutas distintas hoy: 6
Sin vuelta: [('BIO', 'PMI'), ('SVQ', 'MAD')]
Rutas acumuladas: 8
Sin vuelta tras añadir mañana: [('MAD', 'VLC'), ('SVQ', 'MAD')]
```

✅ **Solución**

```
rutas_hoy = [("MAD", "BCN"), ("BCN", "MAD"), ("MAD", "LIS"), ("MAD", "BCN"),
             ("SVQ", "MAD"), ("LIS", "MAD"), ("BIO", "PMI")]
rutas_manana = [("PMI", "BIO"), ("MAD", "VLC")]

rutas = set(rutas_hoy)                        # las tuplas son inmutables → pueden ir en un set
print("Rutas distintas hoy:", len(rutas))
print("Sin vuelta:", sorted(r for r in rutas if (r[1], r[0]) not in rutas))

rutas |= set(rutas_manana)
print("Rutas acumuladas:", len(rutas))
print("Sin vuelta tras añadir mañana:", sorted(r for r in rutas if (r[1], r[0]) not in rutas))
```

🖥️ **Salida:**

```
Rutas distintas hoy: 6
Sin vuelta: [('BIO', 'PMI'), ('SVQ', 'MAD')]
Rutas acumuladas: 8
Sin vuelta tras añadir mañana: [('MAD', 'VLC'), ('SVQ', 'MAD')]
```

Ejercicio propuesto 3.5

Un job diario escribe una carpeta por fecha en HDFS y, al terminar con éxito, deja dentro un fichero vacío llamado `_SUCCESS`. Una carpeta sin ese marcador puede tener datos a medias.

```
esperadas = {"2026-10-01", "2026-10-02", "2026-10-03", "2026-10-04",
             "2026-10-05", "2026-10-06", "2026-10-07"}
# Carpetas que existen en HDFS
presentes = {"2026-09-30", "2026-10-01", "2026-10-02", "2026-10-03",
             "2026-10-05", "2026-10-06", "2026-10-07"}
# Carpetas que contienen el marcador _SUCCESS (escritura terminada)
con_success = {"2026-09-30", "2026-10-01", "2026-10-02", "2026-10-05", "2026-10-07"}
```

Usando solo operaciones de `set`, obtén:

- Las particiones esperadas que no existen.
- Las particiones esperadas que existen pero no tienen `_SUCCESS`.
- La lista de particiones a reprocesar (unión de las dos anteriores).
- Las carpetas que existen pero no corresponden al rango esperado.
- Cuántas particiones esperadas están completas y qué porcentaje representan. **Salida esperada**

```
Faltan: ['2026-10-04']
Incompletas: ['2026-10-03', '2026-10-06']
```

```
A reprocesar: ['2026-10-03', '2026-10-04', '2026-10-06']
Fuera de rango: ['2026-09-30']
Completas: 4 de 7 (57.1 %)
```

✅ **Solución**

```python
esperadas = {"2026-10-01", "2026-10-02", "2026-10-03", "2026-10-04",
             "2026-10-05", "2026-10-06", "2026-10-07"}
presentes = {"2026-09-30", "2026-10-01", "2026-10-02", "2026-10-03",
             "2026-10-05", "2026-10-06", "2026-10-07"}
con_success = {"2026-09-30", "2026-10-01", "2026-10-02", "2026-10-05", "2026-10-07"}

faltan = esperadas - presentes
incompletas = (esperadas & presentes) - con_success
reprocesar = faltan | incompletas
fuera = presentes - esperadas
completas = esperadas & con_success

print("Faltan:", sorted(faltan))
print("Incompletas:", sorted(incompletas))
print("A reprocesar:", sorted(reprocesar))
print("Fuera de rango:", sorted(fuera))
print(f"Completas: {len(completas)} de {len(esperadas)} ({len(completas) / len(esperadas) * 100:.1f} %)")
```

🖥️ **Salida:**

```
Faltan: ['2026-10-04']
Incompletas: ['2026-10-03', '2026-10-06']
A reprocesar: ['2026-10-03', '2026-10-04', '2026-10-06']
Fuera de rango: ['2026-09-30']
Completas: 4 de 7 (57.1 %)
```

Ejercicio propuesto 3.6

Dos DataNodes del clúster han dejado de responder. El NameNode necesita saber qué bloques han perdido réplicas.

```python
nodos_vivos = {"dn1", "dn2", "dn4", "dn5"}

# (bloque, set de nodos que guardan una réplica)
bloques = [
    ("blk_001", {"dn1", "dn2", "dn3"}),
    ("blk_002", {"dn3", "dn6", "dn4"}),
    ("blk_003", {"dn2", "dn4", "dn5"}),
    ("blk_004", {"dn3", "dn6", "dn1"}),
    ("blk_005", {"dn6", "dn3", "dn2"}),
    ("blk_006", {"dn3", "dn6"}),
]
```

94. Para cada bloque, calcula sus réplicas vivas (intersección con `nodos_vivos`) y muéstralas ordenadas.
95. Reúne en un `set` todos los nodos que aparecen en los bloques (usa `|=`) y obtén los nodos caídos.
96. Clasifica los bloques: **en riesgo** si les queda 1 réplica viva y **perdidos** si no les queda ninguna. **Salida esperada**

```
blk_001: 2 réplicas vivas ['dn1', 'dn2']
blk_002: 1 réplicas vivas ['dn4']
blk_003: 3 réplicas vivas ['dn2', 'dn4', 'dn5']
blk_004: 1 réplicas vivas ['dn1']
blk_005: 1 réplicas vivas ['dn2']
blk_006: 0 réplicas vivas []
Nodos caídos: ['dn3', 'dn6']
Bloques en riesgo: ['blk_002', 'blk_004', 'blk_005']
Bloques perdidos: ['blk_006']
```

✅ **Solución**

```python
nodos_vivos = {"dn1", "dn2", "dn4", "dn5"}
bloques = [
    ("blk_001", {"dn1", "dn2", "dn3"}),
    ("blk_002", {"dn3", "dn6", "dn4"}),
    ("blk_003", {"dn2", "dn4", "dn5"}),
    ("blk_004", {"dn3", "dn6", "dn1"}),
    ("blk_005", {"dn6", "dn3", "dn2"}),
    ("blk_006", {"dn3", "dn6"}),
]

todos_nodos, en_riesgo, perdidos = set(), [], []
for bloque, nodos in bloques:
    vivas = nodos & nodos_vivos
    print(f"{bloque}: {len(vivas)} réplicas vivas {sorted(vivas)}")
    todos_nodos |= nodos
    if len(vivas) == 1:
        en_riesgo.append(bloque)
    elif len(vivas) == 0:
        perdidos.append(bloque)

print("Nodos caídos:", sorted(todos_nodos - nodos_vivos))
print("Bloques en riesgo:", en_riesgo)
print("Bloques perdidos:", perdidos)
```

🖥️ **Salida:**

```
blk_001: 2 réplicas vivas ['dn1', 'dn2']
blk_002: 1 réplicas vivas ['dn4']
blk_003: 3 réplicas vivas ['dn2', 'dn4', 'dn5']
blk_004: 1 réplicas vivas ['dn1']
blk_005: 1 réplicas vivas ['dn2']
blk_006: 0 réplicas vivas []
Nodos caídos: ['dn3', 'dn6']
Bloques en riesgo: ['blk_002', 'blk_004', 'blk_005']
Bloques perdidos: ['blk_006']
```

Sección 4. Diccionarios

Un diccionario guarda pares **clave → valor** y permite acceder a cada valor por su clave de forma muy rápida. Es la estructura que corresponde a un objeto JSON. También es la herramienta básica para contar, agrupar y cruzar datos sin una base de datos.

Ejemplos resueltos

```python
pedido = {
    "id": 50231,
    "cliente": "Acme",
    "importe": 1250.0,
    "moneda": "EUR",
}

print(pedido["cliente"])
print(pedido.get("canal"))               # la clave no existe: None
print(pedido.get("canal", "online"))     # valor por defecto

pedido["canal"] = "tienda"      # añadir
pedido["importe"] = 1190.0      # modificar
moneda = pedido.pop("moneda")   # extraer y eliminar
print("Moneda retirada:", moneda)
print(pedido)

print("Claves:", list(pedido.keys()))
for campo, valor in pedido.items():
    print(f"{campo:<8} ->{valor}")
print("¿Tiene 'importe'?", "importe" in pedido)
```

Paso a paso

97. `pedido["cliente"]` lanza `KeyError` si la clave no existe. `get()` devuelve `None` en ese caso, o el valor por defecto que le indiquemos.
98. Asignar a una clave nueva la **añade**, y asignar a una clave existente la **modifica**.
99. `pop()` elimina la clave y devuelve su valor.
100. `keys()`, `values()` e `items()` permiten recorrer las claves, los valores o los pares. Los diccionarios conservan el orden de inserción.
101. `in` comprueba si existe una **clave**, no un valor. **Salida esperada**

```
Acme
None
online
Moneda retirada: EUR
{'id': 50231, 'cliente': 'Acme', 'importe': 1190.0, 'canal': 'tienda'}
Claves: ['id', 'cliente', 'importe', 'canal']
  id       -> 50231
  cliente  -> Acme
  importe  -> 1190.0
  canal    -> tienda
¿Tiene 'importe'? True
```

```
eventos = ["login", "click", "click", "compra", "login", "click",
           "error", "click", "logout", "error"]

conteo = {}
for e in eventos:
    conteo[e] = conteo.get(e, 0) + 1
print(conteo)

for tipo in sorted(conteo):
    print(f"{tipo:<7}{'#' * conteo[tipo]}{conteo[tipo]}")
```

Paso a paso

102. `conteo.get(e, 0)` devuelve el contador actual o 0 si es la primera vez que aparece el evento. Así no hace falta comprobar antes si la clave existe.
103. `sorted(conteo)` devuelve las **claves** ordenadas alfabéticamente.
104. `'#' * n` repite el carácter `n` veces y produce un histograma de texto. **Salida esperada**

```
{'login': 2, 'click': 4, 'compra': 1, 'error': 2, 'logout': 1}
click   #### 4
compra  # 1
error   ## 2
login   ## 2
logout  # 1
```

```
ventas = [
    ("MAD01", 1250.0), ("BCN02", 980.5), ("MAD01", 1530.0),
    ("VLC03", 410.0), ("BCN02", 1105.0), ("MAD01", 300.0),
]

resumen = {}
for tienda, importe in ventas:
    if tienda not in resumen:
        resumen[tienda] = {"total": 0.0, "pedidos": 0}
    resumen[tienda]["total"] += importe
    resumen[tienda]["pedidos"] += 1

print(resumen)
for tienda, datos in resumen.items():
    media = datos["total"] / datos["pedidos"]
    print(f"{tienda}:{datos['pedidos']} pedidos, total{datos['total']:.2f}, media{media:.2f}")
```

Paso a paso

105. Cada tienda es una clave, y su valor es otro diccionario con dos acumuladores.
106. La primera vez que aparece una tienda se inicializan sus acumuladores. Las siguientes veces solo se actualizan.

107. Es lo mismo que haría en SQL `SELECT tienda, COUNT(*), SUM(importe), AVG(importe) ... GROUP BY tienda`. **Salida esperada**

```
{'MAD01': {'total': 3080.0, 'pedidos': 3}, 'BCN02': {'total': 2085.5, 'pedidos': 2}, 'VLC03': {'total':
410.0, 'pedidos': 1}}
MAD01: 3 pedidos, total 3080.00, media 1026.67
BCN02: 2 pedidos, total 2085.50, media 1042.75
VLC03: 1 pedidos, total 410.00, media 410.00
```

```python
clientes = {
    "C001": {"nombre": "Acme", "pais": "ES"},
    "C002": {"nombre": "Globex", "pais": "PT"},
    "C003": {"nombre": "Initech", "pais": "MX"},
}
pedidos = [("P1", "C001", 300.0), ("P2", "C003", 125.5),
           ("P3", "C009", 80.0), ("P4", "C001", 47.0)]

desconocido = {"nombre": "DESCONOCIDO", "pais": "??"}
for id_pedido, id_cliente, importe in pedidos:
    cli = clientes.get(id_cliente, desconocido)
    print(id_pedido, id_cliente, cli["nombre"], cli["pais"], importe)
```

Paso a paso

108. El diccionario `clientes` funciona como una **tabla de búsqueda** indexada por `id_cliente`. Así funciona internamente un *hash join*.
109. `get(id_cliente, desconocido)` resuelve los clientes que no existen, como `C009`, sin que el proceso falle. Es el equivalente a un `LEFT JOIN` con un valor por defecto. **Salida esperada**

```
P1 C001 Acme ES 300.0
P2 C003 Initech MX 125.5
P3 C009 DESCONOCIDO ?? 80.0
P4 C001 Acme ES 47.0
```

```python
config_base = {
    "formato": "parquet",
    "compresion": "snappy",
    "particiones": 8,
    "destino": {"zona": "plata", "ruta": "/lake/plata"},
}
config_prod = {"particiones": 64, "compresion": "zstd"}

# Fusión: las claves de config_prod sobrescriben a las de config_base
config = {**config_base, **config_prod}
print(config)
print("Particiones:", config["particiones"])
print("Ruta destino:", config["destino"]["ruta"])

# Cuidado: el diccionario anidado "destino" es compartido
config["destino"]["zona"] = "oro"
print("Zona en config_base:", config_base["destino"]["zona"])
```

Paso a paso

110. `{**a, **b}` crea un diccionario nuevo con todas las claves de `a` y de `b`. Si una clave está en los dos, gana el valor de `b`, que está más a la derecha.
111. La fusión es **superficial**: el diccionario anidado `destino` no se copia, sino que se comparte. Por eso, al cambiar `config["destino"]["zona"]` también cambia `config_base`. Es el mismo problema de alias que vimos en el Ejemplo 1.3.
112. Para copiar también los niveles internos se usa `copy.deepcopy()`. **Salida esperada**

```
{'formato': 'parquet', 'compresion': 'zstd', 'particiones': 64, 'destino': {'zona': 'plata', 'ruta':
'/lake/plata'}}
Particiones: 64
Ruta destino: /lake/plata
Zona en config_base: oro
```

Spark y Kafka deciden en qué partición va cada registro calculando un número a partir de su clave (*hash*) y quedándose con el resto de dividirlo entre el número de particiones.

```
import zlib

num_particiones = 3
claves = ["C001", "C002", "C003", "C004", "C005", "C001", "C006", "C003"]

particiones = {}
for clave in claves:
    p = zlib.crc32(clave.encode()) % num_particiones
    particiones.setdefault(p, []).append(clave)

for p in sorted(particiones):
    print(f"Partición{p}:{particiones[p]}")
```

Paso a paso

113. `zlib.crc32()` convierte el texto de la clave (pasado a bytes con `encode()`) en un número entero. Para la misma clave, el número es siempre el mismo.
114. `% num_particiones` convierte ese número en una partición entre 0 y 2.
115. `setdefault(p, [])` devuelve la lista de la partición y, si todavía no existe, la crea vacía. Así se puede hacer `append()` directamente.
116. Todos los registros de `C001` y de `C003` caen siempre en la misma partición. Por eso un `groupBy` o un `join` por cliente puede resolverse dentro de cada partición.
117. No se usa la función `hash()` de Python porque, para los textos, cambia en cada ejecución del programa. Un particionado debe ser estable entre ejecuciones y entre máquinas. **Salida esperada**

```
Partición 0: ['C003', 'C003']
Partición 1: ['C002', 'C005', 'C006']
Partición 2: ['C001', 'C004', 'C001']
```

En el Ejemplo 2.5 cada palabra viajaba por la red como un par `(palabra, 1)`. Un *combiner* hace un conteo **local** en cada nodo antes de enviar nada, y así reduce el volumen de datos del shuffle.

```
splits = [
    ["click", "view", "click", "view", "view", "compra"],
    ["view", "view", "click", "view"],
    ["click", "compra", "view", "click", "click"],
]

# MAP + COMBINER: cada nodo cuenta primero su propio split
parciales = []
for split in splits:
    local = {}
    for evento in split:
        local[evento] = local.get(evento, 0) + 1
    parciales.append(local)
    print("Conteo local:", local)

# REDUCE: se fusionan los conteos parciales
total = {}
for local in parciales:
    for evento, n in local.items():
        total[evento] = total.get(evento, 0) + n
print("Conteo global:", total)

sin_combiner = 0
con_combiner = 0
for i in range(len(splits)):
    sin_combiner += len(splits[i])
    con_combiner += len(parciales[i])
print("Pares enviados por la red sin combiner:", sin_combiner)
print("Pares enviados por la red con combiner:", con_combiner)
```

Paso a paso

118. Cada nodo construye su propio diccionario de conteos con el patrón `get(clave, 0) + 1`.

119. En la fase reduce se recorren los diccionarios parciales y se suman sus valores en un diccionario global.
120. Sin combiner, cada evento viajaría por la red como un par: 15 pares. Con combiner, cada nodo envía un par por cada clave distinta: 8 pares.
121. Esta técnica funciona porque la suma es asociativa: da igual sumar por partes que todo junto. Las operaciones `reduceByKey` de Spark aplican esta optimización automáticamente. **Salida esperada**

```
Conteo local: {'click': 2, 'view': 3, 'compra': 1}
Conteo local: {'view': 3, 'click': 1}
Conteo local: {'click': 3, 'compra': 1, 'view': 1}
Conteo global: {'click': 6, 'view': 7, 'compra': 2}
Pares enviados por la red sin combiner: 15
Pares enviados por la red con combiner: 8
```

Ejercicios propuestos

Ejercicio propuesto 4.1

```python
tienda_region = {
    "MAD01": "Centro", "MAD02": "Centro", "BCN01": "Noreste",
    "VLC01": "Levante", "BCN02": "Noreste", "TOL01": "Centro",
}
```

Construye un diccionario `region → lista de tiendas` y muéstralo con las regiones y las tiendas ordenadas alfabéticamente. Pista: el método `setdefault()` permite crear la lista vacía la primera vez que aparece una región.

Salida esperada

```
Centro -> ['MAD01', 'MAD02', 'TOL01']
Levante -> ['VLC01']
Noreste -> ['BCN01', 'BCN02']
```

✅ **Solución**

```python
tienda_region = {
    "MAD01": "Centro", "MAD02": "Centro", "BCN01": "Noreste",
    "VLC01": "Levante", "BCN02": "Noreste", "TOL01": "Centro",
}

por_region = {}
for tienda, region in tienda_region.items():
    por_region.setdefault(region, []).append(tienda)   # crea la lista solo la 1ª vez

for region in sorted(por_region):
    print(region, "->", sorted(por_region[region]))
```

🖥️ **Salida:**

```
Centro -> ['MAD01', 'MAD02', 'TOL01']
Levante -> ['VLC01']
Noreste -> ['BCN01', 'BCN02']
```

Ejercicio propuesto 4.2

```python
ventas = [
    ("2026-09-15", "MAD01", 200.0),
    ("2026-09-28", "BCN02", 150.0),
    ("2026-10-02", "MAD01", 320.0),
    ("2026-09-30", "MAD01", 80.0),
    ("2026-10-05", "BCN02", 410.0),
    ("2026-10-09", "MAD01", 95.5),
]
```

Usa como clave una tupla `(tienda, mes)`, donde el mes son los 7 primeros caracteres de la fecha. Muestra los totales ordenados por tienda y mes, con dos decimales.

Salida esperada

```
BCN02 2026-09: 150.00
BCN02 2026-10: 410.00
MAD01 2026-09: 280.00
MAD01 2026-10: 415.50
```

✅ Solución

```python
ventas = [
    ("2026-09-15", "MAD01", 200.0), ("2026-09-28", "BCN02", 150.0),
    ("2026-10-02", "MAD01", 320.0), ("2026-09-30", "MAD01", 80.0),
    ("2026-10-05", "BCN02", 410.0), ("2026-10-09", "MAD01", 95.5),
]

totales = {}
for fecha, tienda, importe in ventas:
    clave = (tienda, fecha[:7])                        # tupla como clave (es inmutable)
    totales[clave] = totales.get(clave, 0) + importe

for (tienda, mes), total in sorted(totales.items()):
    print(f"{tienda} {mes}: {total:.2f}")
```

🖥️ Salida:

```
BCN02 2026-09: 150.00
BCN02 2026-10: 410.00
MAD01 2026-09: 280.00
MAD01 2026-10: 415.50
```

Ejercicio propuesto 4.3

Un informe de calidad de datos necesita saber cuántos valores faltan en cada campo. Un valor cuenta como nulo si es `None`, si es una cadena vacía o si la clave no aparece en el registro.

```python
registros = [
    {"id": 1, "email": "ana@acme.com", "ciudad": "Madrid", "edad": 34},
    {"id": 2, "email": None, "ciudad": "Sevilla", "edad": None},
    {"id": 3, "email": "luis@globex.com", "edad": 41},
    {"id": 4, "email": "", "ciudad": None, "edad": 29},
]
campos = ["id", "email", "ciudad", "edad"]
```

Salida esperada

```
id      0 nulos (0 %)
email   2 nulos (50 %)
ciudad  2 nulos (50 %)
edad    1 nulos (25 %)
```

✅ Solución

```python
registros = [
    {"id": 1, "email": "ana@acme.com", "ciudad": "Madrid", "edad": 34},
    {"id": 2, "email": None, "ciudad": "Sevilla", "edad": None},
    {"id": 3, "email": "luis@globex.com", "edad": 41},
    {"id": 4, "email": "", "ciudad": None, "edad": 29},
]
campos = ["id", "email", "ciudad", "edad"]

for campo in campos:
    nulos = 0
    for r in registros:
        if r.get(campo) in (None, ""):                 # get() devuelve None si falta la clave
```

```
            nulos += 1
    print(f"{campo:<7} {nulos} nulos ({nulos * 100 // len(registros)} %)")
```

🖥️ Salida:

```
id      0 nulos (0 %)
email   2 nulos (50 %)
ciudad  2 nulos (50 %)
edad    1 nulos (25 %)
```

Ejercicio propuesto 4.4

```
lineas = [
    ("teclado", 3), ("raton", 10), ("monitor", 2), ("raton", 5),
    ("cable_hdmi", 8), ("teclado", 4), ("monitor", 1), ("cable_hdmi", 9),
]
```

Suma las unidades por producto. Después ordena los pares de `items()` por unidades, de mayor a menor, con `itemgetter(1)`, y muestra el top 3 numerado.

Salida esperada

```
{'teclado': 7, 'raton': 15, 'monitor': 3, 'cable_hdmi': 17}
1. cable_hdmi (17 uds)
2. raton (15 uds)
3. teclado (7 uds)
```

✅ Solución

```
from operator import itemgetter

lineas = [
    ("teclado", 3), ("raton", 10), ("monitor", 2), ("raton", 5),
    ("cable_hdmi", 8), ("teclado", 4), ("monitor", 1), ("cable_hdmi", 9),
]

unidades = {}
for producto, uds in lineas:
    unidades[producto] = unidades.get(producto, 0) + uds
print(unidades)

ranking = sorted(unidades.items(), key=itemgetter(1), reverse=True)   # itemgetter(1) ≈ lambda x: x[1]
for pos, (producto, uds) in enumerate(ranking[:3], start=1):
    print(f"{pos}. {producto} ({uds} uds)")
```

🖥️ Salida:

```
{'teclado': 7, 'raton': 15, 'monitor': 3, 'cable_hdmi': 17}
1. cable_hdmi (17 uds)
2. raton (15 uds)
3. teclado (7 uds)
```

Ejercicio propuesto 4.5

Un pipeline combina tres niveles de configuración. El nivel `ejecucion` tiene prioridad sobre `entorno`, y `entorno` sobre `defaults`.

```
defaults = {"formato": "csv", "separador": ",", "reintentos": 3, "timeout_s": 60}
entorno = {"formato": "parquet", "timeout_s": 120}
ejecucion = {"reintentos": 5}
```

Fusiona los tres niveles en un solo diccionario. Después indica, para cada clave, de qué nivel procede su valor final.

Salida esperada

```
{'formato': 'parquet', 'separador': ',', 'reintentos': 5, 'timeout_s': 120}
formato    = parquet  (viene de entorno)
separador  = ,        (viene de defaults)
reintentos = 5        (viene de ejecucion)
timeout_s  = 120      (viene de entorno)
```

✅ **Solución**

```python
defaults = {"formato": "csv", "separador": ",", "reintentos": 3, "timeout_s": 60}
entorno = {"formato": "parquet", "timeout_s": 120}
ejecucion = {"reintentos": 5}

config = {**defaults, **entorno, **ejecucion}     # el último gana (también: defaults | entorno | ejecucion)
print(config)

niveles = [("ejecucion", ejecucion), ("entorno", entorno), ("defaults", defaults)]
for clave, valor in config.items():
    origen = next(nombre for nombre, nivel in niveles if clave in nivel)
    print(f"{clave:<10} = {str(valor):<8} (viene de {origen})")
```

🖥️ **Salida:**

```
{'formato': 'parquet', 'separador': ',', 'reintentos': 5, 'timeout_s': 120}
formato    = parquet  (viene de entorno)
separador  = ,        (viene de defaults)
reintentos = 5        (viene de ejecucion)
timeout_s  = 120      (viene de entorno)
```

Ejercicio propuesto 4.6

Varios equipos leen el mismo topic, cada uno con su propio grupo de consumidores. Hay que revisar cuál va retrasado.

```python
# Último offset escrito en cada partición del topic "pedidos"
ultimo_offset = {0: 15230, 1: 14980, 2: 15410, 3: 9800}

# Último offset confirmado por cada grupo de consumidores
grupos = {
    "etl_lakehouse": {0: 15230, 1: 14975, 2: 15400, 3: 9800},
    "alertas_fraude": {0: 13900, 1: 14980, 2: 12010, 3: 9790},
    "dashboard_rt": {0: 15229, 1: 14980, 2: 15410},
}
umbral_alerta = 1000
```

Para cada grupo:

122. Construye un diccionario `partición → lag`. Si un grupo no tiene offset confirmado para una partición, considera que no ha leído nada (offset 0).
123. Calcula el lag total y la partición más retrasada. Pista: `max(lags, key=lags.get)` devuelve la clave con el valor más alto.
124. Marca el grupo como `ALERTA` si su lag total supera el umbral, o como `OK` en caso contrario. **Salida esperada**

```
etl_lakehouse   lag    15 | peor partición 2 (10) | OK
alertas_fraude  lag  4740 | peor partición 2 (3400) | ALERTA
dashboard_rt    lag  9801 | peor partición 3 (9800) | ALERTA
```

✅ **Solución**

```python
ultimo_offset = {0: 15230, 1: 14980, 2: 15410, 3: 9800}
grupos = {
    "etl_lakehouse": {0: 15230, 1: 14975, 2: 15400, 3: 9800},
    "alertas_fraude": {0: 13900, 1: 14980, 2: 12010, 3: 9790},
    "dashboard_rt": {0: 15229, 1: 14980, 2: 15410},
```

```
}
umbral_alerta = 1000

for grupo, confirmados in grupos.items():
    lags = {}
    for particion, ultimo in ultimo_offset.items():
        lags[particion] = ultimo - confirmados.get(particion, 0)
    total = sum(lags.values())
    peor = max(lags, key=lags.get)
    estado = "ALERTA" if total > umbral_alerta else "OK"
    print(f"{grupo:<15} lag {total:>5} | peor partición {peor} ({lags[peor]}) | {estado}")
```

🖥️ **Salida:**

```
etl_lakehouse   lag    15 | peor partición 2 (10) | OK
alertas_fraude  lag  4740 | peor partición 2 (3400) | ALERTA
dashboard_rt    lag  9801 | peor partición 3 (9800) | ALERTA
```

Ejercicio propuesto 4.7

Cuando una tabla es muy pequeña, Spark envía una copia completa a cada nodo (*broadcast*). Así cada partición de la tabla grande se cruza localmente, sin mover sus datos por la red.

```
# Tabla pequeña: se envía completa a todos los nodos (broadcast)
paises = {"ES": "Europa", "PT": "Europa", "MX": "América", "AR": "América", "JP": "Asia"}

# Tabla grande, ya repartida en 3 particiones: (id_venta, país, importe)
particiones = [
    [("V1", "ES", 120.0), ("V2", "MX", 80.0), ("V3", "ES", 45.5)],
    [("V4", "JP", 300.0), ("V5", "AR", 60.0), ("V6", "FR", 99.0)],
    [("V7", "PT", 210.0), ("V8", "MX", 150.0)],
]
```

125. Para cada partición, enriquece cada venta con su continente usando el diccionario `paises`. Los países que no estén en él se asignan a `"Desconocido"`.
126. Calcula en cada partición un diccionario local `continente → importe` y muéstralo.
127. Fusiona los resultados locales en un total global y muéstralo ordenado por continente, con dos decimales.
     Salida esperada

```
Partición 0: {'Europa': 165.5, 'América': 80.0}
Partición 1: {'Asia': 300.0, 'América': 60.0, 'Desconocido': 99.0}
Partición 2: {'Europa': 210.0, 'América': 150.0}
Ventas por continente:
  América       290.00
  Asia          300.00
  Desconocido    99.00
  Europa        375.50
```

✅ **Solución**

```
paises = {"ES": "Europa", "PT": "Europa", "MX": "América", "AR": "América", "JP": "Asia"}
particiones = [
    [("V1", "ES", 120.0), ("V2", "MX", 80.0), ("V3", "ES", 45.5)],
    [("V4", "JP", 300.0), ("V5", "AR", 60.0), ("V6", "FR", 99.0)],
    [("V7", "PT", 210.0), ("V8", "MX", 150.0)],
]

global_ = {}
for i, particion in enumerate(particiones):
    local = {}
    for _, pais, importe in particion:
        continente = paises.get(pais, "Desconocido")      # valor por defecto si no existe
        local[continente] = local.get(continente, 0) + importe
    print(f"Partición {i}: {local}")
    for continente, importe in local.items():
        global_[continente] = global_.get(continente, 0) + importe

print("Ventas por continente:")
```

```
for continente in sorted(global_):
    print(f"  {continente:<12}{global_[continente]:>8.2f}")
```

🖥️ **Salida:**

```
Partición 0: {'Europa': 165.5, 'América': 80.0}
Partición 1: {'Asia': 300.0, 'América': 60.0, 'Desconocido': 99.0}
Partición 2: {'Europa': 210.0, 'América': 150.0}
Ventas por continente:
  América       290.00
  Asia          300.00
  Desconocido    99.00
  Europa        375.50
```

Sección 5. Comprehensions

Una *comprehension* construye una lista, un `set` o un diccionario en una sola expresión, a partir de otra colección. Su forma general es:

```
[expresion for elemento in coleccion if condicion]          # lista
{expresion for elemento in coleccion if condicion}          # set
{clave: valor for elemento in coleccion if condicion}       # diccionario
```

Es la forma habitual de escribir en Python las operaciones de **transformar** (`SELECT`) y **filtrar** (`WHERE`) una colección.

Ejemplos resueltos

```
crudos = ["12.5", "8", "", "N/A", "103.75", "  44 ", "-3"]

limpios = [v.strip() for v in crudos]
print(limpios)

numericos = [float(v) for v in limpios if v.replace(".", "", 1).isdigit()]
print(numericos)
```

Paso a paso

1. La primera comprehension solo **transforma**: aplica `strip()` a cada valor.
2. La segunda **filtra y transforma**: se queda con los textos que representan un número y los convierte a `float`.
3. `v.replace(".", "", 1).isdigit()` quita el primer punto decimal y comprueba que lo que queda son dígitos. Por eso `"-3"` se descarta: en este ejemplo, un importe negativo no es válido.
4. La segunda comprehension equivale a este bucle:

```
numericos = []
for v in limpios:
    if v.replace(".", "", 1).isdigit():
        numericos.append(float(v))
```

Salida esperada

```
['12.5', '8', '', 'N/A', '103.75', '44', '-3']
[12.5, 8.0, 103.75, 44.0]
```

```
latencias = [120, 85, 430, 95, 1500, 88]

estado = ["LENTA" if ms > 300 else "OK" for ms in latencias]
print(estado)
print(list(zip(latencias, estado)))
print("Lentas:", estado.count("LENTA"))
```

Paso a paso

5. Cuando la condición sirve para **elegir el valor**, el `if-else` va **antes** del `for`.
6. Cuando la condición sirve para **filtrar elementos**, el `if` va **después** del `for` (Ejemplo 5.1). **Salida esperada**

```
['OK', 'OK', 'LENTA', 'OK', 'LENTA', 'OK']
[(120, 'OK'), (85, 'OK'), (430, 'LENTA'), (95, 'OK'), (1500, 'LENTA'), (88, 'OK')]
Lentas: 2
```

```
lotes = [[101, 102, 103], [104], [], [105, 106]]

registros = [r for lote in lotes for r in lote]
print(registros)
print("Total registros:", len(registros))
```

Paso a paso

Los `for` se leen en el mismo orden que si fueran bucles anidados: el primero es el exterior.

```
registros = []
for lote in lotes:          # primer for: bucle exterior
    for r in lote:          # segundo for: bucle interior
        registros.append(r)
```

Salida esperada

```
[101, 102, 103, 104, 105, 106]
Total registros: 6
```

```
emails = ["ana@Acme.com", "luis@globex.com ", "marta@acme.com",
          "pablo@Initech.io", "eva@globex.com"]

dominios = {e.strip().lower().split("@")[1] for e in emails}
print(sorted(dominios))
print("Empresas distintas:", len(dominios))
```

Paso a paso

7. Para cada email: se quitan los espacios, se pasa a minúsculas y se toma lo que hay después de `@`.
8. Al usar llaves `{}`, el resultado es un `set`. `Acme.com` y `acme.com` quedan unificados tras normalizarlos. **Salida esperada**

```
['acme.com', 'globex.com', 'initech.io']
Empresas distintas: 3
```

```
stock = {"SKU-001": 120, "SKU-002": 0, "SKU-003": 15, "SKU-004": 0, "SKU-005": 340}

agotados = {sku: uds for sku, uds in stock.items() if uds == 0}
print(agotados)

estado = {sku: ("REPONER" if uds < 20 else "OK") for sku, uds in stock.items()}
print(estado)

columnas = ["ID Pedido", "Fecha Alta", "Importe Total"]
renombrar = {c: c.lower().replace(" ", "_") for c in columnas}
print(renombrar)
```

Paso a paso

9. `stock.items()` entrega los pares `(clave, valor)`, que se desempaquetan en `sku, uds`.
10. El primer diccionario **filtra**, el segundo **transforma los valores** y el tercero crea un mapeo para **renombrar columnas** a *snake_case*. **Salida esperada**

```
{'SKU-002': 0, 'SKU-004': 0}
{'SKU-001': 'OK', 'SKU-002': 'REPONER', 'SKU-003': 'REPONER', 'SKU-004': 'REPONER', 'SKU-005': 'OK'}
{'ID Pedido': 'id_pedido', 'Fecha Alta': 'fecha_alta', 'Importe Total': 'importe_total'}
```

Las transformaciones básicas de Spark (`map`, `filter`, `flatMap`, `distinct`) tienen un equivalente directo en las comprehensions. Vamos a procesar un micro-lote de logs con el formato `timestamp|usuario|nivel|etiquetas`.

```
lineas = [
    "2026-10-15T08:00:01|u17|ERROR|pago,timeout",
    "2026-10-15T08:00:02|u03|INFO|login",
    "2026-10-15T08:00:04|u17|ERROR|pago,tarjeta",
    "2026-10-15T08:00:05|u21|WARN|latencia",
    "2026-10-15T08:00:07|u08|ERROR|stock",
]

# map: cada línea se convierte en una lista de campos
registros = [linea.split("|") for linea in lineas]
print("Registros:", len(registros))
print("Primero:", registros[0])

# filter: solo los errores
errores = [r for r in registros if r[2] == "ERROR"]
print("Errores:", len(errores))

# flatMap: cada error produce tantas etiquetas como tenga
etiquetas = [etiqueta for r in errores for etiqueta in r[3].split(",")]
print("Etiquetas:", etiquetas)

# distinct: usuarios afectados
usuarios = {r[1] for r in errores}
print("Usuarios con errores:", sorted(usuarios))
```

Paso a paso

Paso	Comprehension	Equivalente en Spark
Convertir cada elemento	`[f(x) for x in datos]`	`map`
Quedarse con algunos elementos	`[x for x in datos if cond]`	`filter`
Un elemento produce varios	`[y for x in datos for y in f(x)]`	`flatMap`
Eliminar repetidos	`{x for x in datos}`	`distinct`

11. `registros` aplica `split("|")` a cada línea: de 5 textos pasamos a 5 listas de campos.
12. `errores` filtra por el campo 2 (nivel) y se queda con 3 registros.
13. `etiquetas` usa dos `for`: el primero recorre los errores y el segundo recorre las etiquetas de cada error. De 3 registros salen 5 etiquetas.
14. La comprehension de `set` devuelve los usuarios sin repetir: `u17` tuvo dos errores, pero aparece una vez.

Salida esperada

```
Registros: 5
Primero: ['2026-10-15T08:00:01', 'u17', 'ERROR', 'pago,timeout']
Errores: 3
Etiquetas: ['pago', 'timeout', 'pago', 'tarjeta', 'stock']
Usuarios con errores: ['u08', 'u17']
```

En el particionado por hash, cada clave va a la partición `hash(clave) % num_particiones`. Una comprehension de diccionario con otra de lista dentro construye todas las particiones de una vez.

```
import zlib

num_particiones = 3
claves = ["C001", "C002", "C003", "C004", "C005", "C001", "C006", "C003", "C007", "C001"]

# Una lista de claves por partición
particiones = {
    p: [c for c in claves if zlib.crc32(c.encode()) % num_particiones == p]
    for p in range(num_particiones)
}
for p, contenido in particiones.items():
    print(f"Partición{p}:{contenido}")

tamanos = {p: len(contenido) for p, contenido in particiones.items()}
print("Tamaños:", tamanos)
```

```
media = len(claves) / num_particiones
print("Factor de skew:", round(max(tamanos.values()) / media, 2))
```

Paso a paso

15. La comprehension exterior crea una clave por partición (`for p in range(num_particiones)`).
16. Para cada partición, la comprehension interior recorre todas las claves y se queda con las que tienen ese resto. `zlib.crc32()` da siempre el mismo número para la misma clave, así que `C001` cae siempre en la misma partición.
17. Una comprehension se puede escribir en varias líneas para que sea más legible: Python sabe que continúa hasta que se cierran las llaves.
18. `tamanos` se construye con otra comprehension sobre `particiones.items()`, y con ella se calcula el factor de skew (máximo / media).
19. Esta versión recorre la lista de claves una vez por partición. Con millones de claves sería más eficiente un bucle que recorra la lista una sola vez; la comprehension aporta claridad, pero no siempre es la opción más rápida. **Salida esperada**

```
Partición 0: ['C003', 'C003', 'C007']
Partición 1: ['C002', 'C005', 'C006']
Partición 2: ['C001', 'C004', 'C001', 'C001']
Tamaños: {0: 3, 1: 3, 2: 4}
Factor de skew: 1.2
```

Ejercicios propuestos

Ejercicio propuesto 5.1

```
registros = [
    {"id": 1, "cliente": "Acme", "importe": 250.0, "canal": "web"},
    {"id": 2, "cliente": "Globex", "importe": 80.0, "canal": "tienda"},
    {"id": 3, "cliente": "Initech", "importe": 1200.0, "canal": "web"},
    {"id": 4, "cliente": "Umbrella", "importe": 99.99, "canal": "app"},
    {"id": 5, "cliente": "Acme", "importe": 430.0, "canal": "app"},
]
```

Con comprehensions:

20. Crea una lista de diccionarios que contenga solo `id` e `importe` de los registros con importe mayor que 100.
21. Calcula el total de esos importes.
22. Obtén el `set` de canales distintos y muéstralo ordenado. **Salida esperada**

```
[{'id': 1, 'importe': 250.0}, {'id': 3, 'importe': 1200.0}, {'id': 5, 'importe': 430.0}]
Total: 1880.0
Canales: ['app', 'tienda', 'web']
```

✅ Solución

```
registros = [
    {"id": 1, "cliente": "Acme", "importe": 250.0, "canal": "web"},
    {"id": 2, "cliente": "Globex", "importe": 80.0, "canal": "tienda"},
    {"id": 3, "cliente": "Initech", "importe": 1200.0, "canal": "web"},
    {"id": 4, "cliente": "Umbrella", "importe": 99.99, "canal": "app"},
    {"id": 5, "cliente": "Acme", "importe": 430.0, "canal": "app"},
]

grandes = [{"id": r["id"], "importe": r["importe"]} for r in registros if r["importe"] > 100]
print(grandes)
print("Total:", sum(r["importe"] for r in grandes))
print("Canales:", sorted({r["canal"] for r in registros}))
```

🖥️ Salida:

```
[{'id': 1, 'importe': 250.0}, {'id': 3, 'importe': 1200.0}, {'id': 5, 'importe': 430.0}]
Total: 1880.0
Canales: ['app', 'tienda', 'web']
```

Ejercicio propuesto 5.2

```python
# 3 filas x 4 columnas (id, unidades, precio, descuento)
filas = [
    [1, 10, 2.5, 0.0],
    [2, 4, 10.0, 0.1],
    [3, 7, 3.2, 0.0],
]
```

Con una comprehension anidada, construye `columnas`: una lista con una lista por cada columna. No uses `zip()`. Después suma las unidades, que son la columna 1.

Salida esperada

```
[[1, 2, 3], [10, 4, 7], [2.5, 10.0, 3.2], [0.0, 0.1, 0.0]]
Unidades totales: 21
```

✅ Solución

```python
filas = [
    [1, 10, 2.5, 0.0],
    [2, 4, 10.0, 0.1],
    [3, 7, 3.2, 0.0],
]

columnas = [[fila[c] for fila in filas] for c in range(len(filas[0]))]   # transponer
print(columnas)
print("Unidades totales:", sum(columnas[1]))
```

🖥️ Salida:

```
[[1, 2, 3], [10, 4, 7], [2.5, 10.0, 3.2], [0.0, 0.1, 0.0]]
Unidades totales: 21
```

Ejercicio propuesto 5.3

```python
muestra = {"id": 1043, "cliente": "Acme", "importe": 99.9, "activo": True,
           "fecha": "2026-10-01", "descuento": None}
```

Con una comprehension de diccionario, construye `columna → nombre del tipo` (usa `type(v).__name__`). Después obtén con una comprehension de lista las columnas cuyo tipo no se puede inferir porque el valor es `None`.

Salida esperada

```
{'id': 'int', 'cliente': 'str', 'importe': 'float', 'activo': 'bool', 'fecha': 'str', 'descuento':
'NoneType'}
No se puede inferir: ['descuento']
```

✅ Solución

```python
muestra = {"id": 1043, "cliente": "Acme", "importe": 99.9, "activo": True,
           "fecha": "2026-10-01", "descuento": None}

tipos = {col: type(v).__name__ for col, v in muestra.items()}
print(tipos)
print("No se puede inferir:", [col for col, t in tipos.items() if t == "NoneType"])
```

🖥️ Salida:

```
{'id': 'int', 'cliente': 'str', 'importe': 'float', 'activo': 'bool', 'fecha': 'str', 'descuento':
'NoneType'}
No se puede inferir: ['descuento']
```

Ejercicio propuesto 5.4

```
anio = 2026
meses = [9, 10]
dias = [1, 2, 3]
```

Con una sola comprehension, genera las rutas con el formato `ventas/year=AAAA/month=MM/day=DD`. El mes y el día deben tener dos dígitos: usa `{m:02d}` en un f-string.

Salida esperada

```
ventas/year=2026/month=09/day=01
ventas/year=2026/month=09/day=02
ventas/year=2026/month=09/day=03
ventas/year=2026/month=10/day=01
ventas/year=2026/month=10/day=02
ventas/year=2026/month=10/day=03
Particiones: 6
```

✅ Solución

```
anio = 2026
meses = [9, 10]
dias = [1, 2, 3]

rutas = [f"ventas/year={anio}/month={m:02d}/day={d:02d}" for m in meses for d in dias]
for r in rutas:
    print(r)
print("Particiones:", len(rutas))
```

🖥️ Salida:

```
ventas/year=2026/month=09/day=01
ventas/year=2026/month=09/day=02
ventas/year=2026/month=09/day=03
ventas/year=2026/month=10/day=01
ventas/year=2026/month=10/day=02
ventas/year=2026/month=10/day=03
Particiones: 6
```

Ejercicio propuesto 5.5

Este código guarda la duración de los jobs que no terminaron en error:

```
lineas = ["job_a;120;OK", "job_b;45;ERROR", "job_c;300;OK", "job_d;12;OK"]

resultado = {}
for linea in lineas:
    partes = linea.split(";")
    if partes[2] != "ERROR":
        resultado[partes[0]] = int(partes[1])
print(resultado)
```

Reescríbelo como **una sola comprehension de diccionario**. Pista: puedes llamar varias veces a `linea.split(";")`.

Salida esperada

```
{'job_a': 120, 'job_c': 300, 'job_d': 12}
```

✅ **Solución**

```
lineas = ["job_a;120;OK", "job_b;45;ERROR", "job_c;300;OK", "job_d;12;OK"]

resultado = {linea.split(";")[0]: int(linea.split(";")[1])
             for linea in lineas if linea.split(";")[2] != "ERROR"}
print(resultado)
```

🖥️ **Salida:**

```
{'job_a': 120, 'job_c': 300, 'job_d': 12}
```

Ejercicio propuesto 5.6

En un data lake con particionado estilo Hive, la ruta de cada fichero contiene los valores de las columnas de partición con el formato `columna=valor`.

```
rutas = [
    "ventas/year=2026/month=09/day=30/part-0000.parquet",
    "ventas/year=2026/month=10/day=01/part-0000.parquet",
    "ventas/year=2026/month=10/day=01/part-0001.parquet",
    "ventas/year=2026/month=10/day=02/part-0000.parquet",
    "ventas/year=2026/month=10/day=02/part-0001.parquet",
    "ventas/year=2026/month=10/day=02/part-0002.parquet",
]
```

Resuelve cada paso con comprehensions:

23. Construye `metadatos`: una lista con un diccionario por ruta que contenga solo los segmentos `columna=valor`. Muestra el primero y el último.
24. Obtén las particiones distintas como tuplas `(mes, día)`, ordenadas.
25. Cuenta cuántos ficheros pertenecen al mes `"10"`. Pista: `zip(rutas, metadatos)` recorre las dos listas a la vez.
26. Construye un diccionario `"mes-día" → número de ficheros`. **Salida esperada**

```
{'year': '2026', 'month': '09', 'day': '30'}
{'year': '2026', 'month': '10', 'day': '02'}
Particiones distintas: [('09', '30'), ('10', '01'), ('10', '02')]
Ficheros de octubre: 5 de 6
Ficheros por partición: {'09-30': 1, '10-01': 2, '10-02': 3}
```

✅ **Solución**

```
rutas = [
    "ventas/year=2026/month=09/day=30/part-0000.parquet",
    "ventas/year=2026/month=10/day=01/part-0000.parquet",
    "ventas/year=2026/month=10/day=01/part-0001.parquet",
    "ventas/year=2026/month=10/day=02/part-0000.parquet",
    "ventas/year=2026/month=10/day=02/part-0001.parquet",
    "ventas/year=2026/month=10/day=02/part-0002.parquet",
]

metadatos = [{seg.split("=")[0]: seg.split("=")[1] for seg in ruta.split("/") if "=" in seg}
             for ruta in rutas]
print(metadatos[0])
print(metadatos[-1])

particiones = sorted({(m["month"], m["day"]) for m in metadatos})
print("Particiones distintas:", particiones)

octubre = len([r for r, m in zip(rutas, metadatos) if m["month"] == "10"])
print(f"Ficheros de octubre: {octubre} de {len(rutas)}")

claves = [f"{m['month']}-{m['day']}" for m in metadatos]
print("Ficheros por partición:", {k: claves.count(k) for k in claves})
```

🖥️ **Salida:**

```
{'year': '2026', 'month': '09', 'day': '30'}
{'year': '2026', 'month': '10', 'day': '02'}
Particiones distintas: [('09', '30'), ('10', '01'), ('10', '02')]
Ficheros de octubre: 5 de 6
Ficheros por partición: {'09-30': 1, '10-01': 2, '10-02': 3}
```

Ejercicio propuesto 5.7

La función `explode()` de Spark convierte un registro con un array en varias filas, una por cada elemento del array. Vamos a simularla.

```
pedidos = [
    {"id": "P1", "lineas": [{"sku": "A1", "uds": 2}, {"sku": "B7", "uds": 1}]},
    {"id": "P2", "lineas": [{"sku": "A1", "uds": 5}]},
    {"id": "P3", "lineas": []},
    {"id": "P4", "lineas": [{"sku": "C3", "uds": 4}, {"sku": "B7", "uds": 3}, {"sku": "A1", "uds": 1}]},
]
```

27. Con una comprehension de dos `for`, genera una fila `(id_pedido, sku, uds)` por cada línea de pedido y muéstralas.
28. Comprueba qué pedidos desaparecen en el *explode* porque no tienen líneas.
29. Con una comprehension de `set`, obtén los SKU distintos. Después, con una comprehension de diccionario, calcula las unidades totales de cada SKU. **Salida esperada**

```
('P1', 'A1', 2)
('P1', 'B7', 1)
('P2', 'A1', 5)
('P4', 'C3', 4)
('P4', 'B7', 3)
('P4', 'A1', 1)
Filas tras el explode: 6
Pedidos sin líneas: ['P3']
Unidades por SKU: {'A1': 8, 'B7': 4, 'C3': 4}
```

✅ **Solución**

```
pedidos = [
    {"id": "P1", "lineas": [{"sku": "A1", "uds": 2}, {"sku": "B7", "uds": 1}]},
    {"id": "P2", "lineas": [{"sku": "A1", "uds": 5}]},
    {"id": "P3", "lineas": []},
    {"id": "P4", "lineas": [{"sku": "C3", "uds": 4}, {"sku": "B7", "uds": 3}, {"sku": "A1", "uds": 1}]},
]

filas = [(p["id"], l["sku"], l["uds"]) for p in pedidos for l in p["lineas"]]   # explode
for f in filas:
    print(f)
print("Filas tras el explode:", len(filas))
print("Pedidos sin líneas:", [p["id"] for p in pedidos if not p["lineas"]])

skus = {sku for _, sku, _ in filas}
print("Unidades por SKU:", {s: sum(u for _, sku, u in filas if sku == s) for s in sorted(skus)})
```

🖥️ **Salida:**

```
('P1', 'A1', 2)
('P1', 'B7', 1)
('P2', 'A1', 5)
('P4', 'C3', 4)
('P4', 'B7', 3)
('P4', 'A1', 1)
Filas tras el explode: 6
Pedidos sin líneas: ['P3']
Unidades por SKU: {'A1': 8, 'B7': 4, 'C3': 4}
```

Sección 6. Funciones

Una función agrupa un bloque de código con un nombre para poder **reutilizarlo**. Python trae muchas funciones integradas (`len()`, `sorted()`, `max()`…) y permite definir las nuestras con `def`. En ingeniería de datos, cada paso de un pipeline (leer, validar, transformar, cargar) suele ser una función. Así cada paso se puede probar por separado y combinar con otros.

```
def nombre_funcion(parametros):
    """Docstring: qué hace la función."""
    # cuerpo de la función
    return resultado
```

Por convención, los nombres de función van en minúsculas y con las palabras separadas por `_`: `calcular_kpis()`, `validar_registro()`.

Ejemplos resueltos

```
def resumen_lote(valores):
    """Devuelve (n, mínimo, máximo, media) de una lista de números."""
    n = len(valores)
    media = sum(valores) / n
    return n, min(valores), max(valores), round(media, 2)

lote_lunes = [120.0, 85.5, 430.0, 95.25]
lote_martes = [300.0, 310.5]

print(resumen_lote(lote_lunes))
n, minimo, maximo, media = resumen_lote(lote_martes)
print(f"Martes:{n} registros, media{media}")
```

Paso a paso

1. El texto entre triples comillas es el *docstring*: documenta qué hace la función.
2. `return n, min(...), ...` devuelve **una tupla**. El que llama puede guardarla entera o desempaquetarla en variables.
3. La misma función sirve para cualquier lote. Esa es la ventaja de definirla una vez. **Salida esperada**

```
(4, 85.5, 430.0, 182.69)
Martes: 2 registros, media 305.25
```

```
def normalizar(texto, minusculas=True, quitar_espacios=True, relleno=""):
    if texto is None or texto.strip() == "":
        return relleno
    if quitar_espacios:
        texto = texto.strip()
    if minusculas:
        texto = texto.lower()
    return texto

print(repr(normalizar("  Madrid ")))
print(repr(normalizar("  Madrid ", minusculas=False)))
print(repr(normalizar("", relleno="SIN_DATO")))
print(repr(normalizar(None, relleno="SIN_DATO")))
print(repr(normalizar("  BCN  ", quitar_espacios=False)))
```

Paso a paso

4. `minusculas`, `quitar_espacios` y `relleno` tienen **valor por defecto**. Si no se pasan, se usan esos valores.
5. Al llamar con `minusculas=False` (**argumento por nombre**) cambiamos solo esa opción, sin tener que pasar las anteriores en orden.
6. En la definición, los parámetros con valor por defecto deben ir **después** de los obligatorios.
7. `repr()` muestra el texto entre comillas, así que se ven los espacios que quedan. **Salida esperada**

```
'madrid'
'Madrid'
```

```
'SIN_DATO'
'SIN_DATO'
'  bcn  '
```

```python
def publicar_metrica(nombre, valor, unidad):
    print(f"métrica={nombre} valor={valor} unidad={unidad}")

publicar_metrica("filas_cargadas", 15230, "filas")            # por posición
publicar_metrica(valor=42.7, unidad="s", nombre="duracion")   # por nombre: el orden da igual
publicar_metrica("errores", unidad="filas", valor=3)          # mezcla: posicionales primero

# publicar_metrica(nombre="errores", 3, "filas")
#   -> SyntaxError: positional argument follows keyword argument
# publicar_metrica("errores", 3)
#   -> TypeError: publicar_metrica() missing 1 required positional argument: 'unidad'

resultado = publicar_metrica("latencia_p95", 230, "ms")
print("La función devuelve:", resultado)

def formato_fecha():
    return "AAAA-MM-DD"

def formato_fecha():          # redefinición: sustituye a la anterior
    return "DD/MM/AAAA"

print(formato_fecha())
```

Paso a paso

8. **Por posición**, cada valor va al parámetro que ocupa su mismo lugar. Hay que pasar exactamente tantos valores como parámetros obligatorios tenga la función.
9. **Por nombre**, cada valor se asigna al parámetro indicado y el orden da igual.
10. Se pueden mezclar, pero los argumentos posicionales siempre van **antes** que los argumentos por nombre. Si no, Python ni siquiera ejecuta el programa (`SyntaxError`).
11. Si falta un argumento obligatorio, la llamada falla con `TypeError` e indica cuál falta.
12. Una función sin `return` **devuelve** `None`. `publicar_metrica()` imprime, pero no devuelve nada.
13. Si se define dos veces una función con el mismo nombre, la segunda definición sustituye a la primera. En un notebook es una fuente habitual de confusión: se ejecuta la versión de la última celda que se ejecutó.
Salida esperada

```
métrica=filas_cargadas valor=15230 unidad=filas
métrica=duracion valor=42.7 unidad=s
métrica=errores valor=3 unidad=filas
métrica=latencia_p95 valor=230 unidad=ms
La función devuelve: None
DD/MM/AAAA
```

```python
def total_bytes(*tamanos):
    print("  recibido:", tamanos)
    return sum(tamanos)

print(total_bytes(1024, 2048))
print(total_bytes(500, 500, 500, 500))
print(total_bytes())

def log_evento(nivel, mensaje, **contexto):
    linea = f"[{nivel}]{mensaje}"
    for clave, valor in contexto.items():
        linea += f" |{clave}={valor}"
    print(linea)

log_evento("INFO", "Carga finalizada", tabla="ventas", filas=15230)
log_evento("ERROR", "Timeout en origen", origen="api_crm", intento=3, espera_s=30)
log_evento("DEBUG", "Sin contexto")
```

Paso a paso

14. `tamanos` recoge todos los argumentos posicionales en una **tupla**, que puede estar vacía.

15. `*contexto` recoge todos los argumentos por nombre que sobran en un **diccionario**. Es muy útil para logs, porque cada evento puede llevar un contexto distinto.
16. Los nombres `args` y `kwargs` son solo una convención: lo que importa son el y el `*`. **Salida esperada**

```
  recibido: (1024, 2048)
3072
  recibido: (500, 500, 500, 500)
2000
  recibido: ()
0
[INFO] Carga finalizada | tabla=ventas | filas=15230
[ERROR] Timeout en origen | origen=api_crm | intento=3 | espera_s=30
[DEBUG] Sin contexto
```

Cuando una función combina todos los tipos de parámetros, deben aparecer en este orden:

Orden	Tipo	En el ejemplo
1	Posicionales obligatorios	`nombre`, `entorno`
2	Posicionales variables	`*tablas`
3	Solo por nombre	`reintentos`, `notificar`
4	Por nombre variables	`**spark_conf`

```
def lanzar_job(nombre, entorno, *tablas, reintentos=3, notificar=False, **spark_conf):
    print(f"Job:{nombre} [{entorno}]")
    print("  Tablas:", tablas)
    print("  Reintentos:", reintentos, "| Notificar:", notificar)
    print("  Configuración Spark:", spark_conf)

lanzar_job("carga_diaria", "prod", "ventas", "clientes", "stock",
           notificar=True, executor_memory="8g", shuffle_partitions=200)

lanzar_job("recalculo_kpis", "dev")

# Cuidado: el 5 no llega a reintentos, se queda dentro de tablas
lanzar_job("backfill", "dev", "ventas", 5)
```

Paso a paso

17. `nombre` y `entorno` reciben los dos primeros valores. Todos los posicionales que siguen van a la tupla `tablas`.
18. Los parámetros escritos **después** de `tablas` solo se pueden pasar **por nombre**. Por eso `notificar=True` llega a su parámetro.
19. Los argumentos por nombre que no corresponden a ningún parámetro (`executor_memory`, `shuffle_partitions`) van al diccionario `spark_conf`.
20. En la última llamada, el `5` no llega a `reintentos`: al ser posicional, se queda dentro de `tablas`. Para cambiar los reintentos hay que escribir `reintentos=5`. **Salida esperada**

```
Job: carga_diaria [prod]
  Tablas: ('ventas', 'clientes', 'stock')
  Reintentos: 3 | Notificar: True
  Configuración Spark: {'executor_memory': '8g', 'shuffle_partitions': 200}
Job: recalculo_kpis [dev]
  Tablas: ()
  Reintentos: 3 | Notificar: False
  Configuración Spark: {}
Job: backfill [dev]
  Tablas: ('ventas', 5)
  Reintentos: 3 | Notificar: False
  Configuración Spark: {}
```

Es habitual que los parámetros de conexión estén en un fichero de configuración. Al leerlo, llegan a Python como tupla o como diccionario.

```
def cadena_conexion(host, puerto, base_datos, usuario="lector"):
    return f"{usuario}@{host}:{puerto}/{base_datos}"
```

```
params_tupla = ("db-prod.interno", 5432, "ventas")
print(cadena_conexion(*params_tupla))

params_dict = {"host": "db-dev.interno", "puerto": 5433,
               "base_datos": "pruebas", "usuario": "etl"}
print(cadena_conexion(**params_dict))

print(cadena_conexion("db-qa.interno", 5432, base_datos="qa"))

# Sin ** se desempaquetan las CLAVES del diccionario, no los valores
print(cadena_conexion(*params_dict))
```

Paso a paso

21. `params_tupla` reparte los elementos de la tupla como argumentos posicionales, en orden.
22. `*params_dict` reparte el diccionario como argumentos por nombre. Las claves deben coincidir con los nombres de los parámetros.
23. En la tercera llamada se mezclan argumentos posicionales y por nombre. Los posicionales siempre van primero.
24. Con un solo , un diccionario reparte sus **claves** como argumentos posicionales. Por eso la última línea muestra los nombres de los campos en lugar de sus valores. **Salida esperada**

```
lector@db-prod.interno:5432/ventas
etl@db-dev.interno:5433/pruebas
lector@db-qa.interno:5432/qa
usuario@host:puerto/base_datos
```

```
def registrar_error_mal(error, errores=[]):
    errores.append(error)
    return errores

print(registrar_error_mal("fila 3: importe vacío"))
print(registrar_error_mal("fila 9: fecha inválida"))   # ¡arrastra el error anterior!

def registrar_error(error, errores=None):
    if errores is None:
        errores = []
    errores.append(error)
    return errores

print(registrar_error("fila 3: importe vacío"))
print(registrar_error("fila 9: fecha inválida"))
```

Paso a paso

25. El valor por defecto de un parámetro se crea **una sola vez**, al definir la función, no en cada llamada.
26. Por eso, en la versión incorrecta, todas las llamadas comparten la misma lista `errores`, y el error de la fila 3 aparece en la segunda llamada.
27. La forma correcta es usar `None` como valor por defecto y crear la lista dentro de la función. **Salida esperada**

```
['fila 3: importe vacío']
['fila 3: importe vacío', 'fila 9: fecha inválida']
['fila 3: importe vacío']
['fila 9: fecha inválida']
```

Una función se puede definir dentro de otra. La función interna solo existe mientras se ejecuta la externa, así que queda protegida: nadie puede llamarla desde fuera por error.

```
def validar_lote(registros, importe_maximo=10000):
    def es_importe_valido(importe):        # función interna
        return importe is not None and 0 <= importe <= importe_maximo

    validos = 0
    for r in registros:
        if es_importe_valido(r["importe"]):
            validos += 1
    return validos, len(registros) - validos
```

```
lote = [
    {"id": 1, "importe": 250.0},
    {"id": 2, "importe": -4.0},
    {"id": 3, "importe": None},
    {"id": 4, "importe": 12500.0},
    {"id": 5, "importe": 99.9},
]
print("(válidos, rechazados):", validar_lote(lote))
print("Con máximo 20000:     ", validar_lote(lote, importe_maximo=20000))

# es_importe_valido(100)  -> NameError: name 'es_importe_valido' is not defined
```

Paso a paso

28. `es_importe_valido()` se define dentro de `validar_lote()`. Fuera de ella no existe: intentar llamarla produce `NameError`.
29. La función interna puede **leer** las variables de la externa. Usa `importe_maximo` sin recibirlo como parámetro.
30. Por eso, al llamar a `validar_lote()` con `importe_maximo=20000`, la regla interna cambia y el importe de 12500 pasa a ser válido.
31. Es una buena opción para reglas auxiliares que solo tienen sentido dentro de un proceso concreto. **Salida esperada**

```
(válidos, rechazados): (2, 3)
Con máximo 20000:      (3, 2)
```

```
def leer():
    return ["  101;Acme;1200 ", "102;Globex;", "103;initech;300", "  104;ACME;50"]

def parsear(linea):
    id_, cliente, importe = linea.strip().split(";")
    return {
        "id": int(id_),
        "cliente": cliente.title(),
        "importe": float(importe) if importe else None,
    }

def es_valido(registro):
    return registro["importe"] is not None

def ejecutar_pipeline():
    validos, rechazados = [], []
    for linea in leer():
        registro = parsear(linea)
        if es_valido(registro):
            validos.append(registro)
        else:
            rechazados.append(registro)
    return validos, rechazados

ok, ko = ejecutar_pipeline()
print("Válidos:", ok)
print("Rechazados:", ko)
print("Total cargado:", sum([r["importe"] for r in ok]))
```

Paso a paso

32. Cada función hace **una sola cosa**: `leer()` obtiene los datos, `parsear()` convierte una línea en un diccionario, `es_valido()` aplica una regla de calidad y `ejecutar_pipeline()` coordina el proceso.
33. `parsear()` normaliza el nombre del cliente con `title()`, de modo que `ACME` e `initech` pasan a `Acme` e `Initech`.
34. Las filas sin importe no se pierden: se apartan en `rechazados`, igual que una tabla de cuarentena. **Salida esperada**

```
Válidos: [{'id': 101, 'cliente': 'Acme', 'importe': 1200.0}, {'id': 103, 'cliente': 'Initech', 'importe':
300.0}, {'id': 104, 'cliente': 'Acme', 'importe': 50.0}]
```

```
Rechazados: [{'id': 102, 'cliente': 'Globex', 'importe': None}]
Total cargado: 1550.0
```

En Hadoop, el programador solo escribe dos funciones, la de **map** y la de **reduce**, y el framework se encarga de ejecutarlas en paralelo y de mover los datos entre ellas. Vamos a simular un conteo de las palabras más buscadas en una tienda online.

```python
def mapear(linea):
    """Fase map: emite (palabra, 1) por cada palabra buscada."""
    usuario, termino = linea.split(";")
    return [(palabra.lower(), 1) for palabra in termino.split()]

def agrupar(pares):
    """Fase shuffle: reúne todos los valores de cada clave."""
    grupos = {}
    for clave, valor in pares:
        grupos.setdefault(clave, []).append(valor)
    return grupos

def reducir(clave, valores):
    """Fase reduce: combina los valores de una clave."""
    return clave, sum(valores)

def ejecutar_job(lineas):
    pares = []
    for linea in lineas:
        pares.extend(mapear(linea))
    grupos = agrupar(pares)
    return [reducir(clave, valores) for clave, valores in grupos.items()]

busquedas = [
    "u1;Zapatillas running", "u2;zapatillas mujer", "u3;Camiseta running",
    "u1;zapatillas trail", "u4;mochila trail", "u5;Running",
]

print("Map de una línea:", mapear(busquedas[0]))
resultado = ejecutar_job(busquedas)
from operator import itemgetter
ordenado = sorted(sorted(resultado), key=itemgetter(1), reverse=True)
for palabra, total in ordenado:
    print(f"{palabra:<11}{total}")
```

Paso a paso

35. `mapear()` procesa **una sola línea** y devuelve sus pares `(palabra, 1)`. Como no depende de ninguna otra línea, el framework puede ejecutarla en muchos nodos a la vez.
36. `agrupar()` simula el **shuffle**: reúne en una lista todos los valores de una misma clave.
37. `reducir()` recibe **una clave y sus valores** y los combina en un resultado.
38. `ejecutar_job()` hace el papel del framework: llama a las tres fases en orden.
39. Para ordenar por frecuencia y, en caso de empate, alfabéticamente, se ordena primero por palabra y después por total. `sorted()` respeta el orden previo de los empates. **Salida esperada**

```
Map de una línea: [('zapatillas', 1), ('running', 1)]
running     3
zapatillas  3
trail       2
camiseta    1
mochila     1
mujer       1
```

En Spark, una referencia habitual es que cada partición pese unos 128 MB. Con pocas particiones se desaprovechan núcleos del clúster; con demasiadas, se pierde tiempo gestionando tareas muy pequeñas.

```python
import math

def calcular_particiones(tamano_gb, objetivo_mb=128, nucleos=None):
    """Número de particiones para que cada una pese unos objetivo_mb."""
    particiones = math.ceil(tamano_gb * 1024 / objetivo_mb)
    if nucleos and particiones > nucleos:
```

```
        # redondear hacia arriba a un múltiplo del número de núcleos
        particiones = math.ceil(particiones / nucleos) * nucleos
    return particiones

datasets = [("logs_web", 50), ("ventas", 2.5), ("clientes", 0.04)]

for nombre, gb in datasets:
    simple = calcular_particiones(gb)
    ajustado = calcular_particiones(gb, nucleos=48)
    print(f"{nombre:<9}{gb:>5} GB ->{simple:>3} particiones | con 48 núcleos:{ajustado}")

print("logs_web con particiones de 256 MB:", calcular_particiones(50, objetivo_mb=256))
```

Paso a paso

40. `math.ceil()` redondea hacia arriba: si el dataset pesa 40 MB, hace falta al menos una partición.
41. `objetivo_mb` y `nucleos` tienen valores por defecto, así que la función sirve tanto para un cálculo rápido como para uno ajustado al clúster.
42. Si se indican núcleos y hay más particiones que núcleos, el resultado se redondea a un múltiplo de los núcleos. Así la última "oleada" de tareas ocupa el clúster entero: 400 particiones en 48 núcleos dejarían una última oleada a medias, y 432 no.
43. Si hay menos particiones que núcleos (`ventas`, `clientes`), no se ajusta nada: crear particiones casi vacías no aporta. **Salida esperada**

```
logs_web     50 GB -> 400 particiones | con 48 núcleos: 432
ventas      2.5 GB ->  20 particiones | con 48 núcleos: 20
clientes   0.04 GB ->   1 particiones | con 48 núcleos: 1
logs_web con particiones de 256 MB: 200
```

En las aplicaciones de IA Generativa, los prompts no se escriben a mano cada vez: se generan con funciones que combinan una plantilla fija con los datos de cada petición.

```
def construir_prompt(tarea, contexto, *ejemplos, idioma="español",
                     formato="lista", **restricciones):
    partes = [f"Tarea:{tarea}", f"Contexto:{contexto}"]
    if ejemplos:
        partes.append("Ejemplos:")
        for i, ejemplo in enumerate(ejemplos, start=1):
            partes.append(f"{i}.{ejemplo}")
    partes.append(f"Responde en{idioma} con formato{formato}.")
    for clave, valor in restricciones.items():
        partes.append(f"-{clave.replace('_', ' ')}:{valor}")
    return "\n".join(partes)

prompt = construir_prompt(
    "Resumir las incidencias del pipeline nocturno",
    "3 jobs fallaron por timeout en la API del CRM y 1 job tardó el doble de lo habitual.",
    "job_ventas: timeout tras 3 reintentos -> revisar el límite de la API",
    longitud_maxima="5 líneas",
    tono="técnico",
)
print(prompt)
print()
print(construir_prompt("Traducir el mensaje de error", "Connection refused", idioma="inglés",
                       formato="texto plano"))
```

Paso a paso

44. `tarea` y `contexto` son obligatorios: sin ellos el prompt no tiene sentido.
45. `ejemplos` admite cero o más ejemplos (*few-shot*). Si no se pasa ninguno, la sección de ejemplos no aparece.
46. `idioma` y `formato` tienen un valor por defecto y solo se pueden pasar por nombre.
47. `*restricciones` admite cualquier instrucción adicional. El nombre del argumento se convierte en texto sustituyendo `_` por espacios.
48. `"\n".join(partes)` une las líneas con saltos de línea. **Salida esperada**

```
Tarea: Resumir las incidencias del pipeline nocturno
Contexto: 3 jobs fallaron por timeout en la API del CRM y 1 job tardó el doble de lo habitual.
```

```
Ejemplos:
  1. job_ventas: timeout tras 3 reintentos -> revisar el límite de la API
Responde en español con formato lista.
- longitud maxima: 5 líneas
- tono: técnico

Tarea: Traducir el mensaje de error
Contexto: Connection refused
Responde en inglés con formato texto plano.
```

En un sistema RAG (*Retrieval-Augmented Generation*), los documentos se trocean en fragmentos (*chunks*) antes de convertirlos en embeddings. Los fragmentos se **solapan** un poco para que una idea no quede partida entre dos trozos sin contexto.

```python
def dividir_en_chunks(texto, tamano=8, solape=2):
    palabras = texto.split()
    paso = tamano - solape
    chunks = []
    for inicio in range(0, len(palabras), paso):
        trozo = palabras[inicio:inicio + tamano]
        chunks.append({"id": len(chunks), "inicio": inicio, "texto": " ".join(trozo)})
        if inicio + tamano >= len(palabras):
            break
    return chunks

politica = ("La política de devoluciones permite devolver cualquier producto en un plazo "
            "de 30 días. El cliente debe conservar el ticket de compra. Los productos de "
            "higiene no admiten devolución una vez abiertos.")

print("Palabras:", len(politica.split()))
for chunk in dividir_en_chunks(politica):
    print(chunk)
print("Chunks con tamano=12 y solape=3:", len(dividir_en_chunks(politica, tamano=12, solape=3)))
```

Paso a paso

49. Cada chunk tiene `tamano` palabras, y el siguiente empieza `tamano - solape` palabras más adelante. Con 8 y 2, avanza de 6 en 6, así que cada chunk repite las 2 últimas palabras del anterior.
50. Cada chunk se guarda como un diccionario con su `id` y su posición de `inicio`. Esos metadatos permiten saber después de qué parte del documento salió cada fragmento.
51. El `break` detiene el bucle cuando un chunk ya llega al final del texto, para no generar un último fragmento formado solo por el solape.
52. Los sistemas reales trocean por **tokens** en lugar de palabras, pero la lógica es la misma. **Salida esperada**

```
Palabras: 32
{'id': 0, 'inicio': 0, 'texto': 'La política de devoluciones permite devolver cualquier producto'}
{'id': 1, 'inicio': 6, 'texto': 'cualquier producto en un plazo de 30 días.'}
{'id': 2, 'inicio': 12, 'texto': '30 días. El cliente debe conservar el ticket'}
{'id': 3, 'inicio': 18, 'texto': 'el ticket de compra. Los productos de higiene'}
{'id': 4, 'inicio': 24, 'texto': 'de higiene no admiten devolución una vez abiertos.'}
Chunks con tamano=12 y solape=3: 4
```

Ejercicios propuestos

Ejercicio propuesto 6.1

```python
registros = [
    {"id": 1, "cliente": "Acme", "importe": 120.0},
    {"id": 2, "cliente": "", "importe": 80.0},
    {"id": 3, "cliente": "Globex", "importe": -15.0},
    {"cliente": "Initech", "importe": None},
]
obligatorios = ["id", "cliente", "importe"]
```

Escribe `validar_registro(registro, obligatorios)`, que devuelva una tupla `(es_valido, lista_de_errores)`. Un campo obligatorio falla si no existe, si es `None` o si es una cadena vacía. Además, si el importe existe y es negativo, se añade el error `"importe negativo"`. Aplica la función a todos los registros y muestra `?` cuando falte el id.

Salida esperada

```
OK  1 []
KO  2 ["falta 'cliente'"]
KO  3 ['importe negativo']
KO  ? ["falta 'id'", "falta 'importe'"]
```

✅ **Solución**

```
registros = [
    {"id": 1, "cliente": "Acme", "importe": 120.0},
    {"id": 2, "cliente": "", "importe": 80.0},
    {"id": 3, "cliente": "Globex", "importe": -15.0},
    {"cliente": "Initech", "importe": None},
]
obligatorios = ["id", "cliente", "importe"]

def validar_registro(registro, obligatorios):
    """Devuelve (es_valido, lista_de_errores)."""
    errores = [f"falta '{campo}'" for campo in obligatorios if registro.get(campo) in (None, "")]
    importe = registro.get("importe")
    if importe is not None and importe < 0:
        errores.append("importe negativo")
    return len(errores) == 0, errores

for r in registros:
    ok, errores = validar_registro(r, obligatorios)    # desempaquetar la tupla devuelta
    print(f"{'OK' if ok else 'KO'}  {r.get('id', '?')} {errores}")
```

🖥️ **Salida:**

```
OK  1 []
KO  2 ["falta 'cliente'"]
KO  3 ['importe negativo']
KO  ? ["falta 'id'", "falta 'importe'"]
```

Ejercicio propuesto 6.2

Escribe `calcular_kpis(*importes, moneda="EUR", decimales=2)`, que devuelva un diccionario con `n`, `total`, `media` y `moneda`. Si no recibe importes, la media debe ser `None`. Comprueba la función con estas llamadas:

```
print(calcular_kpis(120.5, 80.25, 99.99))
print(calcular_kpis(1000, 2500, moneda="USD", decimales=0))
print(calcular_kpis())
lote = [10.0, 20.0, 30.0]
print(calcular_kpis(*lote, moneda="MXN"))
```

Salida esperada

```
{'n': 3, 'total': 300.74, 'media': 100.25, 'moneda': 'EUR'}
{'n': 2, 'total': 3500, 'media': 1750.0, 'moneda': 'USD'}
{'n': 0, 'total': 0, 'media': None, 'moneda': 'EUR'}
{'n': 3, 'total': 60.0, 'media': 20.0, 'moneda': 'MXN'}
```

✅ **Solución**

```
def calcular_kpis(*importes, moneda="EUR", decimales=2):
    # *importes → tupla con todos los posicionales; moneda/decimales → solo por nombre
    n = len(importes)
    total = round(sum(importes), decimales)
    media = round(sum(importes) / n, decimales) if n else None
```

```python
    return {"n": n, "total": total, "media": media, "moneda": moneda}

print(calcular_kpis(120.5, 80.25, 99.99))
print(calcular_kpis(1000, 2500, moneda="USD", decimales=0))
print(calcular_kpis())
lote = [10.0, 20.0, 30.0]
print(calcular_kpis(*lote, moneda="MXN"))          # * desempaqueta la lista en argumentos
```

🖥️ Salida:

```
{'n': 3, 'total': 300.74, 'media': 100.25, 'moneda': 'EUR'}
{'n': 2, 'total': 3500, 'media': 1750.0, 'moneda': 'USD'}
{'n': 0, 'total': 0, 'media': None, 'moneda': 'EUR'}
{'n': 3, 'total': 60.0, 'media': 20.0, 'moneda': 'MXN'}
```

Ejercicio propuesto 6.3

Escribe `construir_ruta(base, *carpetas, **particiones)`. La función debe unir la base (sin `/` final), las carpetas y las particiones con el formato `clave=valor`, separando cada parte con `/`. Comprueba la función con estas llamadas:

```python
print(construir_ruta("abfss://lake@cuenta.dfs.core.windows.net/", "plata", "ventas",
                     year=2026, month="10", day="15"))
print(construir_ruta("/lake", "bronce", "clientes"))
print(construir_ruta("/lake", region="EU"))
```

Salida esperada

```
abfss://lake@cuenta.dfs.core.windows.net/plata/ventas/year=2026/month=10/day=15
/lake/bronce/clientes
/lake/region=EU
```

✅ Solución

```python
def construir_ruta(base, *carpetas, **particiones):
    partes = [base.rstrip("/")]
    partes += list(carpetas)
    partes += [f"{clave}={valor}" for clave, valor in particiones.items()]
    return "/".join(partes)

print(construir_ruta("abfss://lake@cuenta.dfs.core.windows.net/", "plata", "ventas",
                     year=2026, month="10", day="15"))
print(construir_ruta("/lake", "bronce", "clientes"))
print(construir_ruta("/lake", region="EU"))
```

🖥️ Salida:

```
abfss://lake@cuenta.dfs.core.windows.net/plata/ventas/year=2026/month=10/day=15
/lake/bronce/clientes
/lake/region=EU
```

Ejercicio propuesto 6.4

```python
pedidos = [
    {"id": 1, "canal": "web", "pais": "ES"},
    {"id": 2, "canal": "app", "pais": "PT"},
    {"id": 3, "canal": "web", "pais": "ES"},
    {"id": 4, "canal": "tienda", "pais": "ES"},
    {"id": 5, "canal": "web", "pais": "MX"},
    {"id": 6, "canal": "app"},
]
```

Escribe `contar_por(registros, campo, ausente="SIN_VALOR")`, que cuente cuántos registros hay para cada valor del campo indicado. Los registros que no tengan el campo se cuentan con el valor de `ausente`. Llama a la función

tres veces: por `canal`, por `pais` y por `pais` con `ausente="??"`.

Salida esperada

```
{'web': 3, 'app': 2, 'tienda': 1}
{'ES': 3, 'PT': 1, 'MX': 1, 'SIN_VALOR': 1}
{'ES': 3, 'PT': 1, 'MX': 1, '??': 1}
```

✅ **Solución**

```
pedidos = [
    {"id": 1, "canal": "web", "pais": "ES"},
    {"id": 2, "canal": "app", "pais": "PT"},
    {"id": 3, "canal": "web", "pais": "ES"},
    {"id": 4, "canal": "tienda", "pais": "ES"},
    {"id": 5, "canal": "web", "pais": "MX"},
    {"id": 6, "canal": "app"},
]

def contar_por(registros, campo, ausente="SIN_VALOR"):
    conteo = {}
    for r in registros:
        valor = r.get(campo, ausente)
        conteo[valor] = conteo.get(valor, 0) + 1
    return conteo

print(contar_por(pedidos, "canal"))
print(contar_por(pedidos, "pais"))
print(contar_por(pedidos, "pais", ausente="??"))
```

🖥️ **Salida:**

```
{'web': 3, 'app': 2, 'tienda': 1}
{'ES': 3, 'PT': 1, 'MX': 1, 'SIN_VALOR': 1}
{'ES': 3, 'PT': 1, 'MX': 1, '??': 1}
```

Ejercicio propuesto 6.5

Tienes estas líneas de un fichero de movimientos por tienda: `fecha;tienda;tipo;importe`.

```
lineas_crudas = [
    "2026-10-15;MAD01;  ventas ;1250.0",
    "2026-10-15;BCN02;Ventas;",
    "2026-10-15;MAD01;DEVOLUCIONES;-80.0",
    "2026-10-15;VLC03;ventas;410.0",
    "2026-10-15;BCN02;ventas;980.5",
]
```

Organiza la solución en cuatro funciones:

- `parsear_linea(linea)`: devuelve un diccionario. El tipo se normaliza quitando espacios y pasando a minúsculas. Si no hay importe, se guarda `None`.
- `es_valida(registro)`: comprueba que haya importe.
- `agregar_por_tienda(registros, tipo)`: devuelve un diccionario `tienda → total` con los registros del tipo indicado.
- `informe(lineas)`: coordina el proceso y muestra cuántas líneas se han leído, cuántas son válidas y cuántas se han rechazado, seguido del total de `ventas` por tienda, ordenado. **Salida esperada**

```
Leídas: 5 | Válidas: 4 | Rechazadas: 1
  BCN02: 980.50
  MAD01: 1250.00
  VLC03: 410.00
```

✅ **Solución**

```
lineas_crudas = [
    "2026-10-15;MAD01;  ventas ;1250.0",
    "2026-10-15;BCN02;Ventas;",
    "2026-10-15;MAD01;DEVOLUCIONES;-80.0",
    "2026-10-15;VLC03;ventas;410.0",
    "2026-10-15;BCN02;ventas;980.5",
]

def parsear_linea(linea):
    fecha, tienda, tipo, importe = linea.split(";")
    return {"fecha": fecha, "tienda": tienda, "tipo": tipo.strip().lower(),
            "importe": float(importe) if importe else None}

def es_valida(registro):
    return registro["importe"] is not None

def agregar_por_tienda(registros, tipo):
    totales = {}
    for r in registros:
        if r["tipo"] == tipo:
            totales[r["tienda"]] = totales.get(r["tienda"], 0) + r["importe"]
    return totales

def informe(lineas):
    registros = [parsear_linea(l) for l in lineas]
    validos = [r for r in registros if es_valida(r)]
    print(f"Leídas: {len(registros)} | Válidas: {len(validos)} | Rechazadas: {len(registros) -
len(validos)}")
    for tienda, total in sorted(agregar_por_tienda(validos, "ventas").items()):
        print(f"  {tienda}: {total:.2f}")

informe(lineas_crudas)
```

🖥️ **Salida:**

```
Leídas: 5 | Válidas: 4 | Rechazadas: 1
  BCN02: 980.50
  MAD01: 1250.00
  VLC03: 410.00
```

Ejercicio propuesto 6.6

Sin ejecutar el código, analiza estas seis llamadas a la misma función:

```
def cargar_tabla(tabla, esquema, *columnas, modo="append", **opciones):
    print(f"{esquema}.{tabla} | columnas={columnas} | modo={modo} | opciones={opciones}")

# Llamada 1
cargar_tabla("ventas", "plata")
# Llamada 2
cargar_tabla("ventas", "plata", "id", "importe", modo="overwrite")
# Llamada 3
cargar_tabla(esquema="oro", tabla="kpis", particion="fecha")
# Llamada 4
cargar_tabla("ventas")
# Llamada 5
cargar_tabla("clientes", "plata", "id", "overwrite")
# Llamada 6
cargar_tabla("clientes", "plata", tabla="clientes_v2")
```

53. Indica qué llamadas producen un error y explica por qué.
54. Para las que funcionan, escribe qué imprimen.
55. Una de las llamadas que funcionan tiene un error lógico: hace algo distinto de lo que probablemente pretendía quien la escribió. Identifícala y corrígela. Después, ejecútalas una a una para comprobar tus respuestas.

Salida esperada (los errores se muestran con su mensaje de `TypeError`)

```
Llamada 1: plata.ventas | columnas=() | modo=append | opciones={}
Llamada 2: plata.ventas | columnas=('id', 'importe') | modo=overwrite | opciones={}
Llamada 3: oro.kpis | columnas=() | modo=append | opciones={'particion': 'fecha'}
Llamada 4: ERROR -> cargar_tabla() missing 1 required positional argument: 'esquema'
Llamada 5: plata.clientes | columnas=('id', 'overwrite') | modo=append | opciones={}
Llamada 6: ERROR -> cargar_tabla() got multiple values for argument 'tabla'
```

✅ **Solución**

```python
def cargar_tabla(tabla, esquema, *columnas, modo="append", **opciones):
    print(f"{esquema}.{tabla} | columnas={columnas} | modo={modo} | opciones={opciones}")

llamadas = [
    lambda: cargar_tabla("ventas", "plata"),
    lambda: cargar_tabla("ventas", "plata", "id", "importe", modo="overwrite"),
    lambda: cargar_tabla(esquema="oro", tabla="kpis", particion="fecha"),
    lambda: cargar_tabla("ventas"),
    lambda: cargar_tabla("clientes", "plata", "id", "overwrite"),
    lambda: cargar_tabla("clientes", "plata", tabla="clientes_v2"),
]
for n, llamada in enumerate(llamadas, start=1):
    print(f"Llamada {n}: ", end="")
    try:
        llamada()
    except TypeError as e:
        print("ERROR ->", e)
# Fallan la 4 (falta el posicional obligatorio 'esquema') y la 6 ('tabla' llega dos veces:
# por posición "clientes" y por nombre tabla=...).
# Error lógico: la 5. "overwrite" se trata como una COLUMNA más (cae en *columnas) y el modo
# sigue en append. Corrección: cargar_tabla("clientes", "plata", "id", modo="overwrite").
```

🖥️ **Salida:**

```
Llamada 1: plata.ventas | columnas=() | modo=append | opciones={}
Llamada 2: plata.ventas | columnas=('id', 'importe') | modo=overwrite | opciones={}
Llamada 3: oro.kpis | columnas=() | modo=append | opciones={'particion': 'fecha'}
Llamada 4: ERROR -> cargar_tabla() missing 1 required positional argument: 'esquema'
Llamada 5: plata.clientes | columnas=('id', 'overwrite') | modo=append | opciones={}
Llamada 6: ERROR -> cargar_tabla() got multiple values for argument 'tabla'
```

Ejercicio propuesto 6.7

Un job tiene que procesar varios ficheros de tamaños muy distintos y dispone de varios ejecutores. Para equilibrar la carga se usa una regla sencilla: se ordenan los ficheros de mayor a menor y cada uno se asigna al ejecutor que **menos megas lleva acumulados** en ese momento. Si hay empate, se elige el de número más bajo.

```python
# (fichero, tamaño en MB)
ficheros = [
    ("f1.parquet", 512), ("f2.parquet", 128), ("f3.parquet", 900), ("f4.parquet", 256),
    ("f5.parquet", 640), ("f6.parquet", 300), ("f7.parquet", 128),
]
```

56. Escribe `asignar_tareas(ficheros, num_ejecutores=3)`, que devuelva un diccionario `"ejecutor_N" → {"ficheros": [...], "mb": total}`.
57. Escribe `desequilibrio(asignacion)`, que devuelva la carga máxima dividida entre la mínima, con dos decimales.
58. Muestra el reparto con 3 ejecutores y compara su desequilibrio con el de 4 ejecutores. **Salida esperada**

```
ejecutor_0:  900 MB -> ['f3.parquet']
ejecutor_1: 1024 MB -> ['f5.parquet', 'f4.parquet', 'f7.parquet']
ejecutor_2:  940 MB -> ['f1.parquet', 'f6.parquet', 'f2.parquet']
Desequilibrio (máx / mín): 1.14
Con 4 ejecutores: [900, 640, 640, 684] -> desequilibrio 1.41
```

✅ **Solución**

```python
ficheros = [
    ("f1.parquet", 512), ("f2.parquet", 128), ("f3.parquet", 900), ("f4.parquet", 256),
    ("f5.parquet", 640), ("f6.parquet", 300), ("f7.parquet", 128),
]

def asignar_tareas(ficheros, num_ejecutores=3):
    asignacion = {f"ejecutor_{i}": {"ficheros": [], "mb": 0} for i in range(num_ejecutores)}
    for nombre, mb in sorted(ficheros, key=lambda f: f[1], reverse=True):
        # el de menos carga; con empate gana el primero (número más bajo)
        destino = min(asignacion, key=lambda e: asignacion[e]["mb"])
        asignacion[destino]["ficheros"].append(nombre)
        asignacion[destino]["mb"] += mb
    return asignacion

def desequilibrio(asignacion):
    cargas = [e["mb"] for e in asignacion.values()]
    return round(max(cargas) / min(cargas), 2)

reparto = asignar_tareas(ficheros)
for ejecutor, datos in reparto.items():
    print(f"{ejecutor}: {datos['mb']:>4} MB -> {datos['ficheros']}")
print("Desequilibrio (máx / mín):", desequilibrio(reparto))
reparto4 = asignar_tareas(ficheros, 4)
print(f"Con 4 ejecutores: {[e['mb'] for e in reparto4.values()]} -> desequilibrio {desequilibrio(reparto4)}")
```

🖥️ Salida:

```
ejecutor_0:  900 MB -> ['f3.parquet']
ejecutor_1: 1024 MB -> ['f5.parquet', 'f4.parquet', 'f7.parquet']
ejecutor_2:  940 MB -> ['f1.parquet', 'f6.parquet', 'f2.parquet']
Desequilibrio (máx / mín): 1.14
Con 4 ejecutores: [900, 640, 640, 684] -> desequilibrio 1.41
```

Ejercicio propuesto 6.8

Para lanzar un job de Spark en un clúster se usa el comando `spark-submit`, con opciones como el gestor de recursos (`--master`), el modo de despliegue o el número de ejecutores, y parámetros de configuración `--conf clave=valor`.

Escribe `construir_spark_submit(script, *args_app, master="yarn", deploy_mode="cluster", num_executors=2, **conf)`. La función devuelve el comando como texto, con este orden: opciones fijas, opciones `--conf`, script y argumentos de la aplicación. En los nombres de `conf`, los `_` se sustituyen por `.` (`spark_executor_memory` → `spark.executor.memory`). Comprueba la función con estas llamadas:

```python
print(construir_spark_submit("carga_ventas.py"))
print(construir_spark_submit("carga_ventas.py", "--fecha", "2026-10-15",
                             num_executors=10, spark_executor_memory="8g",
                             spark_sql_shuffle_partitions=400))
print(construir_spark_submit("prueba.py", master="local[4]", deploy_mode="client"))
```

Salida esperada

```
spark-submit --master yarn --deploy-mode cluster --num-executors 2 carga_ventas.py
spark-submit --master yarn --deploy-mode cluster --num-executors 10 --conf spark.executor.memory=8g --conf
spark.sql.shuffle.partitions=400 carga_ventas.py --fecha 2026-10-15
spark-submit --master local[4] --deploy-mode client --num-executors 2 prueba.py
```

✅ Solución

```python
def construir_spark_submit(script, *args_app, master="yarn", deploy_mode="cluster",
                           num_executors=2, **conf):
    partes = ["spark-submit", "--master", master, "--deploy-mode", deploy_mode,
              "--num-executors", str(num_executors)]
    for clave, valor in conf.items():
        partes += ["--conf", f"{clave.replace('_', '.')}={valor}"]
    partes.append(script)
    partes += list(args_app)
```

```
    return " ".join(partes)

print(construir_spark_submit("carga_ventas.py"))
print(construir_spark_submit("carga_ventas.py", "--fecha", "2026-10-15",
                             num_executors=10, spark_executor_memory="8g",
                             spark_sql_shuffle_partitions=400))
print(construir_spark_submit("prueba.py", master="local[4]", deploy_mode="client"))
```

🖥️ Salida:

```
spark-submit --master yarn --deploy-mode cluster --num-executors 2 carga_ventas.py
spark-submit --master yarn --deploy-mode cluster --num-executors 10 --conf spark.executor.memory=8g --conf
spark.sql.shuffle.partitions=400 carga_ventas.py --fecha 2026-10-15
spark-submit --master local[4] --deploy-mode client --num-executors 2 prueba.py
```

Ejercicio propuesto 6.9

Los proveedores de modelos de lenguaje cobran por **tokens**, con un precio distinto para los de entrada (el prompt) y los de salida (la respuesta).

```
# USD por millón de tokens (entrada, salida). Tarifas ficticias para el ejercicio.
TARIFAS = {"modelo_ligero": (0.25, 1.25), "modelo_avanzado": (3.00, 15.00)}

# (modelo, tokens de entrada, tokens de salida) de las peticiones de un día
peticiones = [
    ("modelo_ligero", 1200, 300),
    ("modelo_avanzado", 5400, 900),
    ("modelo_ligero", 800, 150),
    ("modelo_avanzado", 12000, 2000),
    ("modelo_ligero", 2500, 600),
]
```

59. Escribe `coste_peticion(modelo, tokens_entrada, tokens_salida)`, que devuelva el coste en USD de una petición. Pruébala con la primera petición desempaquetando la tupla con .
60. Escribe `resumen_costes(peticiones, presupuesto_diario=0.05)`, que devuelva tres valores: un diccionario por modelo con su número de peticiones y su coste, el coste total y si se ha superado el presupuesto.
61. Escribe `proyectar(coste_diario, dias=30)`, que devuelva el coste estimado para ese número de días, con dos decimales. **Salida esperada**

```
Coste de la primera petición: 0.000675 USD
modelo_ligero    3 peticiones -> 0.002437 USD
modelo_avanzado  2 peticiones -> 0.095700 USD
Total del día: 0.098138 USD | ¿Supera el presupuesto? True
Proyección a 30 días: 2.94 USD
Proyección a 365 días: 35.82 USD
```

✅ Solución

```
TARIFAS = {"modelo_ligero": (0.25, 1.25), "modelo_avanzado": (3.00, 15.00)}
peticiones = [
    ("modelo_ligero", 1200, 300),
    ("modelo_avanzado", 5400, 900),
    ("modelo_ligero", 800, 150),
    ("modelo_avanzado", 12000, 2000),
    ("modelo_ligero", 2500, 600),
]

def coste_peticion(modelo, tokens_entrada, tokens_salida):
    precio_in, precio_out = TARIFAS[modelo]
    return (tokens_entrada * precio_in + tokens_salida * precio_out) / 1_000_000

def resumen_costes(peticiones, presupuesto_diario=0.05):
    por_modelo = {}
    for modelo, t_in, t_out in peticiones:
        datos = por_modelo.setdefault(modelo, {"peticiones": 0, "coste": 0})
        datos["peticiones"] += 1
        datos["coste"] += coste_peticion(modelo, t_in, t_out)
    total = sum(d["coste"] for d in por_modelo.values())
```

```
    return por_modelo, total, total > presupuesto_diario      # devuelve 3 valores (tupla)

def proyectar(coste_diario, dias=30):
    return round(coste_diario * dias, 2)

print(f"Coste de la primera petición: {coste_peticion(*peticiones[0]):.6f} USD")
por_modelo, total, supera = resumen_costes(peticiones)
for modelo, d in por_modelo.items():
    print(f"{modelo:<16} {d['peticiones']} peticiones -> {d['coste']:.6f} USD")
print(f"Total del día: {total:.6f} USD | ¿Supera el presupuesto? {supera}")
print(f"Proyección a 30 días: {proyectar(total)} USD")
print(f"Proyección a 365 días: {proyectar(total, 365)} USD")
```

🖥️ Salida:

```
Coste de la primera petición: 0.000675 USD
modelo_ligero    3 peticiones -> 0.002437 USD
modelo_avanzado  2 peticiones -> 0.095700 USD
Total del día: 0.098138 USD | ¿Supera el presupuesto? True
Proyección a 30 días: 2.94 USD
Proyección a 365 días: 35.82 USD
```

Ejercicio propuesto 6.10

Un LLM clasifica los tickets de soporte y devuelve un diccionario por ticket. Antes de guardar sus respuestas en la base de datos hay que comprobarlas: los modelos a veces omiten campos o inventan valores que no existen.

```
CATEGORIAS = {"facturacion", "tecnico", "envio"}
PRIORIDADES = {"baja", "media", "alta"}

# Respuestas de un LLM que clasifica tickets de soporte
respuestas = [
    {"ticket": "T1", "categoria": "envio", "prioridad": "alta", "resumen": "Pedido no entregado"},
    {"ticket": "T2", "categoria": "Facturación", "prioridad": "media", "resumen": "Cobro duplicado"},
    {"ticket": "T3", "categoria": "tecnico", "resumen": "Error al iniciar sesión"},
    {"ticket": "T4", "categoria": "tecnico", "prioridad": "urgente", "resumen": ""},
    {"ticket": "T5", "categoria": "facturacion", "prioridad": "baja", "resumen": "Pide factura"},
]
```

62. Escribe `validar_respuesta(respuesta, obligatorios=("categoria", "prioridad", "resumen"), **permitidos)`, que devuelva `(es_valida, errores)`. Un campo obligatorio falla si no existe o está vacío. Cada argumento de `permitidos` indica los valores aceptados de un campo, como `categoria=CATEGORIAS`.
63. Escribe `procesar_respuestas(respuestas, **permitidos)`, que separe los tickets aceptados de los que necesitan revisión humana. Debe pasar `permitidos` a `validar_respuesta()` con `*`.
64. Muestra los resultados y la tasa de respuestas válidas. **Salida esperada**

```
Aceptadas: ['T1', 'T5']
A revisión humana:
  T2: ["'categoria' no válido: Facturación"]
  T3: ["falta 'prioridad'"]
  T4: ["falta 'resumen'", "'prioridad' no válido: urgente"]
Tasa de respuestas válidas: 40 %
```

✅ Solución

```
CATEGORIAS = {"facturacion", "tecnico", "envio"}
PRIORIDADES = {"baja", "media", "alta"}
respuestas = [
    {"ticket": "T1", "categoria": "envio", "prioridad": "alta", "resumen": "Pedido no entregado"},
    {"ticket": "T2", "categoria": "Facturación", "prioridad": "media", "resumen": "Cobro duplicado"},
    {"ticket": "T3", "categoria": "tecnico", "resumen": "Error al iniciar sesión"},
    {"ticket": "T4", "categoria": "tecnico", "prioridad": "urgente", "resumen": ""},
    {"ticket": "T5", "categoria": "facturacion", "prioridad": "baja", "resumen": "Pide factura"},
]

def validar_respuesta(respuesta, obligatorios=("categoria", "prioridad", "resumen"), **permitidos):
    errores = [f"falta '{c}'" for c in obligatorios if not respuesta.get(c)]
    for campo, validos in permitidos.items():
```

```
        valor = respuesta.get(campo)
        if valor and valor not in validos:
            errores.append(f"'{campo}' no válido: {valor}")
    return not errores, errores

def procesar_respuestas(respuestas, **permitidos):
    aceptadas, revision = [], []
    for r in respuestas:
        ok, errores = validar_respuesta(r, **permitidos)     # reenvía los kwargs con **
        if ok:
            aceptadas.append(r["ticket"])
        else:
            revision.append((r["ticket"], errores))
    return aceptadas, revision

aceptadas, revision = procesar_respuestas(respuestas, categoria=CATEGORIAS, prioridad=PRIORIDADES)
print("Aceptadas:", aceptadas)
print("A revisión humana:")
for ticket, errores in revision:
    print(f"  {ticket}: {errores}")
print(f"Tasa de respuestas válidas: {len(aceptadas) * 100 // len(respuestas)} %")
```

🖥️ **Salida:**

```
Aceptadas: ['T1', 'T5']
A revisión humana:
  T2: ["'categoria' no válido: Facturación"]
  T3: ["falta 'prioridad'"]
  T4: ["falta 'resumen'", "'prioridad' no válido: urgente"]
Tasa de respuestas válidas: 40 %
```

Sección 7. Recursión

Una función recursiva es una función que **se llama a sí misma**. Siempre tiene dos partes:

- **Caso base:** la situación en la que la función ya no necesita llamarse y devuelve un resultado directamente. Sin él, la recursión no termina nunca.
- **Caso recursivo:** la función resuelve una parte del problema y se llama a sí misma con un problema **más pequeño**.

```
def funcion(datos):
    if condicion_de_parada:              # caso base
        return resultado_directo
    return combinar(parte, funcion(resto))   # caso recursivo
```

La recursión es útil en dos tipos de problema:

1. **Problemas que se dividen en subproblemas iguales pero más pequeños.** Por ejemplo, el tamaño de una carpeta es la suma del tamaño de sus subcarpetas.
2. **Problemas con un número desconocido de bucles anidados.** Por ejemplo, recorrer un JSON del que no se sabe cuántos niveles tiene, o combinar los valores de un número variable de parámetros. En ingeniería de datos y en IA aparece al recorrer **estructuras anidadas**: documentos JSON, árboles de carpetas de un data lake, grafos de dependencias, planes de un agente o trazas de ejecución.

Ejemplos resueltos

```
def total_bytes(tamanos):
    if not tamanos:                                  # caso base: lista vacía
        return 0
    return tamanos[0] + total_bytes(tamanos[1:])     # caso recursivo

print(total_bytes([512, 1024, 256]))

# Misma función con traza para ver las llamadas
def total_bytes_traza(tamanos, nivel=0):
    sangria = "  " * nivel
    print(f"{sangria}total_bytes({tamanos})")
```

```
    if not tamanos:
        print(f"{sangria}-> 0  (caso base)")
        return 0
    resultado = tamanos[0] + total_bytes_traza(tamanos[1:], nivel + 1)
    print(f"{sangria}->{resultado}")
    return resultado

total_bytes_traza([512, 1024, 256])
```

Paso a paso

3. Caso base: una lista vacía suma 0 bytes.
4. Caso recursivo: el total es el primer elemento más el total del resto de la lista (`tamanos[1:]`), que es una lista más corta.
5. La traza muestra que las llamadas **bajan** hasta el caso base y que las sumas se resuelven al **subir**: 0 → 256 → 1280 → 1792. Mientras tanto, cada suma queda pendiente esperando el resultado de la llamada siguiente.
6. Cada llamada tiene sus propias variables. No se comparten entre llamadas. **Salida esperada**

```
1792
total_bytes([512, 1024, 256])
  total_bytes([1024, 256])
    total_bytes([256])
      total_bytes([])
      -> 0  (caso base)
    -> 256
  -> 1280
-> 1792
```

Según dónde esté la llamada recursiva, cambia el orden en que se hace el trabajo.

```
def procesar_cola(lotes):
    """Recursión de cola: primero procesa, después se llama."""
    if not lotes:
        return
    print("cola     ->", lotes[0])
    procesar_cola(lotes[1:])

def procesar_cabeza(lotes):
    """Recursión de cabeza: primero se llama, después procesa."""
    if not lotes:
        return
    procesar_cabeza(lotes[1:])
    print("cabeza   ->", lotes[0])

particiones = ["day=01", "day=02", "day=03"]
procesar_cola(particiones)
procesar_cabeza(particiones)
```

Paso a paso

7. **Recursión de cola:** la función procesa el primer elemento y **después** se llama con el resto. El trabajo se hace al bajar, en el orden original, como en un bucle.
8. **Recursión de cabeza:** la función se llama **primero** y procesa el elemento después, cuando la llamada interior ya ha terminado. El trabajo se hace al subir, así que el orden se invierte.
9. Las dos versiones tienen el mismo caso base y la misma llamada. Solo cambia el lugar del `print()`. **Salida esperada**

```
cola     -> day=01
cola     -> day=02
cola     -> day=03
cabeza   -> day=03
cabeza   -> day=02
cabeza   -> day=01
```

Una política de reintentos con *backoff* exponencial duplica la espera tras cada fallo: 2 s, 4 s, 8 s... Queremos saber cuánto tiempo se espera en total.

```python
# Versión iterativa: espera total de una política de reintentos
def espera_total_iterativa(reintentos, espera_inicial):
    total = 0
    espera = espera_inicial
    while reintentos > 0:
        total += espera
        espera *= 2
        reintentos -= 1
    return total

# Versión recursiva, siguiendo los pasos de conversión
def espera_total(reintentos, espera):
    if reintentos == 0:                                  # condición del bucle -> caso base
        return 0
    return espera + espera_total(reintentos - 1, espera * 2)   # cuerpo -> caso recursivo

print("Iterativa:", espera_total_iterativa(5, 2), "s")
print("Recursiva:", espera_total(5, 2), "s")
for n in range(1, 6):
    print(f"{n} reintentos ->{espera_total(n, 2):>2} s de espera acumulada")
```

Paso a paso

Para convertir un bucle en recursión se siguen estos pasos:

10. Las variables que cambia el bucle (`reintentos`, `espera`) pasan a ser **parámetros** de la función recursiva.
11. La condición en la que el bucle termina (`reintentos == 0`) se convierte en el **caso base**.
12. El cuerpo del bucle se convierte en el **caso recursivo**: se suma la espera actual y se llama con un reintento menos y la espera duplicada.
13. Se comprueba que las dos versiones dan el mismo resultado y se eliminan las variables que ya no hacen falta. La versión recursiva no necesita el acumulador `total`. **Salida esperada**

```
Iterativa: 62 s
Recursiva: 62 s
  1 reintentos ->  2 s de espera acumulada
  2 reintentos ->  6 s de espera acumulada
  3 reintentos -> 14 s de espera acumulada
  4 reintentos -> 30 s de espera acumulada
  5 reintentos -> 62 s de espera acumulada
```

```python
# Carpetas = diccionarios; ficheros = tamaño en MB (int)
lake = {
    "bronce": {
        "ventas": {"2026-09.csv": 120, "2026-10.csv": 135},
        "clientes.csv": 40,
    },
    "plata": {
        "ventas": {"part-0000.parquet": 30, "part-0001.parquet": 28},
    },
    "oro": {"kpis_mensuales.parquet": 5},
}

def tamano_total(nodo):
    if isinstance(nodo, int):          # caso base: es un fichero
        return nodo
    total = 0
    for hijo in nodo.values():         # caso recursivo: es una carpeta
        total += tamano_total(hijo)
    return total

print("Data lake completo:", tamano_total(lake), "MB")
for capa, contenido in lake.items():
    print(f"{capa:<6}{tamano_total(contenido):>4} MB")
```

Paso a paso

14. `isinstance(nodo, int)` distingue un fichero, que es el caso base, de una carpeta, que es un diccionario.
15. Para una carpeta se suman los tamaños de todos sus hijos, y cada hijo puede ser a su vez otra carpeta.

16. Con un bucle `for` normal habría que saber de antemano cuántos niveles hay. La recursión funciona con cualquier profundidad. **Salida esperada**

```
Data lake completo: 358 MB
  bronce  295 MB
  plata    58 MB
  oro       5 MB
```

Antes de cargar un JSON en una tabla, hay que convertir sus objetos anidados en columnas planas, como hace la función `flatten` de muchas herramientas ETL.

```
evento = {
    "id": "e-1001",
    "cliente": {
        "id": "C001",
        "direccion": {"ciudad": "Madrid", "cp": "28001"},
    },
    "importe": 99.9,
    "dispositivo": {"tipo": "movil", "so": {"nombre": "Android", "version": "14"}},
}

def aplanar(diccionario, prefijo=""):
    plano = {}
    for clave, valor in diccionario.items():
        nombre = f"{prefijo}.{clave}" if prefijo else clave
        if isinstance(valor, dict):
            plano.update(aplanar(valor, nombre))    # caso recursivo
        else:
            plano[nombre] = valor                   # caso base
    return plano

for columna, valor in aplanar(evento).items():
    print(f"{columna:<26}{valor}")
```

Paso a paso

17. `prefijo` acumula la ruta de claves: `cliente` → `cliente.direccion` → `cliente.direccion.ciudad`.
18. Si el valor es un diccionario, se aplana recursivamente y su resultado se incorpora con `update()`.
19. Si el valor es simple, es una hoja: se guarda con su nombre completo. **Salida esperada**

```
id                         e-1001
cliente.id                 C001
cliente.direccion.ciudad   Madrid
cliente.direccion.cp       28001
importe                    99.9
dispositivo.tipo           movil
dispositivo.so.nombre      Android
dispositivo.so.version     14
```

En un orquestador como Airflow, una tarea solo puede ejecutarse cuando han terminado todas sus dependencias.

```
dependencias = {
    "kpis_ventas": ["ventas_limpias", "clientes_limpios"],
    "ventas_limpias": ["ventas_raw"],
    "clientes_limpios": ["clientes_raw"],
    "ventas_raw": [],
    "clientes_raw": [],
}

def orden_ejecucion(tarea, ejecutadas):
    for previa in dependencias[tarea]:
        if previa not in ejecutadas:
            orden_ejecucion(previa, ejecutadas)     # primero, sus dependencias
    ejecutadas.append(tarea)                        # después, la propia tarea
    return ejecutadas

orden = orden_ejecucion("kpis_ventas", [])
for paso, tarea in enumerate(orden, start=1):
    print(paso, tarea)
```

Paso a paso

20. Antes de ejecutar una tarea, se ejecutan recursivamente sus dependencias.
21. La comprobación `if previa not in ejecutadas` evita repetir una tarea que comparte dependencias con otra.
22. La tarea se añade a la lista **después** de sus dependencias. Por eso `kpis_ventas` aparece en último lugar. Es una recursión de cabeza: el trabajo se hace al volver de las llamadas. **Salida esperada**

```
1 ventas_raw
2 ventas_limpias
3 clientes_raw
4 clientes_limpios
5 kpis_ventas
```

Un fichero de log guarda un índice ordenado de *offsets*: la posición en bytes en la que empieza cada bloque. Buscar en él por mitades es mucho más rápido que recorrerlo entero.

```
import sys

def buscar(ordenada, objetivo, inicio, fin):
    if inicio > fin:                    # caso base 1: no está
        return -1
    medio = (inicio + fin) // 2
    if ordenada[medio] == objetivo:     # caso base 2: encontrado
        return medio
    if objetivo < ordenada[medio]:
        return buscar(ordenada, objetivo, inicio, medio - 1)
    return buscar(ordenada, objetivo, medio + 1, fin)

offsets = [0, 120, 245, 380, 512, 640, 777, 901, 1024]
print("640 está en la posición", buscar(offsets, 640, 0, len(offsets) - 1))
print("300 está en la posición", buscar(offsets, 300, 0, len(offsets) - 1))
print("Límite de recursión:", sys.getrecursionlimit())

def profundidad_lineal(n):
    if n == 0:
        return 0
    return 1 + profundidad_lineal(n - 1)

print(profundidad_lineal(900))
# profundidad_lineal(5000)  -> RecursionError: maximum recursion depth exceeded

sys.setrecursionlimit(10_000)      # se puede subir, con precaución
print(profundidad_lineal(5000))
```

Paso a paso

23. En cada llamada se mira el elemento central. Si no es el buscado, se repite la búsqueda solo en la mitad donde puede estar.
24. Hay dos casos base: el elemento se encuentra, o el rango queda vacío (`inicio > fin`).
25. Con 1.000.000 de elementos bastan unas 20 llamadas, porque cada una descarta la mitad de los datos.
26. Python limita la profundidad de recursión a 1000 llamadas por defecto. `profundidad_lineal(900)` funciona, pero con 5000 se produce un `RecursionError`.
27. `sys.setrecursionlimit()` permite subir el límite, pero cada llamada pendiente ocupa memoria, y un límite demasiado alto puede hacer que el programa se cierre de golpe. Por eso la recursión es adecuada para estructuras con pocos niveles, como un JSON o un árbol de carpetas, y no para recorrer millones de elementos de uno en uno. **Salida esperada**

```
640 está en la posición 5
300 está en la posición -1
Límite de recursión: 1000
900
5000
```

Antes de llevar un job a producción, se prueba con distintas configuraciones de recursos. Con tres parámetros harían falta tres bucles anidados, pero si mañana se añade un cuarto parámetro, habría que reescribir el código. Es un problema con un **número desconocido de bucles**.

```
parametros = {
    "executors": [4, 8],
    "memoria": ["4g", "8g"],
    "shuffle_partitions": [200, 400],
}

def combinaciones(nombres, parametros, actual, resultado):
    if len(actual) == len(nombres):          # caso base: ya hay un valor por parámetro
        resultado.append(dict(zip(nombres, actual)))
        return
    nombre = nombres[len(actual)]            # siguiente parámetro a fijar
    for valor in parametros[nombre]:
        actual.append(valor)
        combinaciones(nombres, parametros, actual, resultado)
        actual.pop()                         # deshacer para probar el siguiente valor

configuraciones = []
combinaciones(list(parametros), parametros, [], configuraciones)
for i, conf in enumerate(configuraciones, start=1):
    print(i, conf)

# Añadir un parámetro más no exige añadir un bucle más
parametros["compresion"] = ["snappy", "zstd"]
configuraciones = []
combinaciones(list(parametros), parametros, [], configuraciones)
print("Con 4 parámetros:", len(configuraciones), "configuraciones")
```

Paso a paso

28. `actual` es la configuración que se está construyendo. Cada nivel de recursión fija el valor de un parámetro.
29. **Caso base:** cuando `actual` tiene un valor por cada parámetro, la configuración está completa y se guarda como diccionario con `dict(zip(...))`.
30. **Caso recursivo:** para cada valor del siguiente parámetro, se añade a `actual`, se llama a la función para fijar el resto y, al volver, se quita con `pop()` para probar el siguiente valor. Esta técnica se llama *backtracking*.
31. Al añadir `compresion`, el código no cambia y genera 2 × 2 × 2 × 2 = 16 configuraciones. **Salida esperada**

```
1 {'executors': 4, 'memoria': '4g', 'shuffle_partitions': 200}
2 {'executors': 4, 'memoria': '4g', 'shuffle_partitions': 400}
3 {'executors': 4, 'memoria': '8g', 'shuffle_partitions': 200}
4 {'executors': 4, 'memoria': '8g', 'shuffle_partitions': 400}
5 {'executors': 8, 'memoria': '4g', 'shuffle_partitions': 200}
6 {'executors': 8, 'memoria': '4g', 'shuffle_partitions': 400}
7 {'executors': 8, 'memoria': '8g', 'shuffle_partitions': 200}
8 {'executors': 8, 'memoria': '8g', 'shuffle_partitions': 400}
Con 4 parámetros: 16 configuraciones
```

Cuando Spark ordena un dataset grande, cada partición se ordena por separado y después se combinan los resultados. La función recursiva divide la lista de particiones por la mitad hasta quedarse con una sola, y al volver fusiona los resultados de dos en dos.

```
def fusionar(a, b):
    """Une dos listas ordenadas en una sola lista ordenada."""
    resultado = []
    i = j = 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            resultado.append(a[i])
            i += 1
        else:
            resultado.append(b[j])
            j += 1
    return resultado + a[i:] + b[j:]

def fusionar_particiones(particiones, nivel=0):
    print("  " * nivel + f"fusionar{len(particiones)} particiones")
    if len(particiones) == 1:                        # caso base
        return particiones[0]
    mitad = len(particiones) // 2
    izquierda = fusionar_particiones(particiones[:mitad], nivel + 1)
```

```
    derecha = fusionar_particiones(particiones[mitad:], nivel + 1)
    return fusionar(izquierda, derecha)

# Timestamps ya ordenados dentro de cada partición
particiones = [[3, 9, 15], [1, 12], [5, 6, 20], [2, 18]]
ordenado = fusionar_particiones(particiones)
print("Resultado:", ordenado)
```

Paso a paso

32. `fusionar()` no es recursiva: une dos listas ya ordenadas comparando sus primeros elementos y tomando siempre el menor.
33. `fusionar_particiones()` divide la lista de particiones en dos mitades y se llama con cada una.
34. **Caso base:** una sola partición ya está ordenada y se devuelve tal cual.
35. Al volver de las dos llamadas, se fusionan los dos resultados. La traza muestra el árbol de llamadas: 4 → 2 + 2 → 1 + 1 + 1 + 1.
36. Este mismo esquema, dividir el problema, resolver cada parte y combinar los resultados, se llama **divide y vencerás**, y es la base del algoritmo *merge sort*. **Salida esperada**

```
fusionar 4 particiones
  fusionar 2 particiones
    fusionar 1 particiones
    fusionar 1 particiones
  fusionar 2 particiones
    fusionar 1 particiones
    fusionar 1 particiones
Resultado: [1, 2, 3, 5, 6, 9, 12, 15, 18, 20]
```

Un agente de IA que recibe una tarea compleja la **descompone** en subtareas, y cada subtarea puede descomponerse a su vez, hasta llegar a acciones concretas que se resuelven con una herramienta (una consulta SQL, una búsqueda web, una llamada al LLM…). El plan resultante es un árbol.

```
plan = {
    "tarea": "Informe de ventas de octubre",
    "subtareas": [
        {"tarea": "Obtener datos", "subtareas": [
            {"tarea": "Consultar ventas", "herramienta": "sql"},
            {"tarea": "Consultar objetivos", "herramienta": "api_crm"},
        ]},
        {"tarea": "Analizar", "subtareas": [
            {"tarea": "Calcular KPIs", "herramienta": "python"},
            {"tarea": "Detectar anomalías", "subtareas": [
                {"tarea": "Comparar con septiembre", "herramienta": "python"},
                {"tarea": "Buscar causas en noticias", "herramienta": "web_search"},
            ]},
        ]},
        {"tarea": "Redactar el informe", "herramienta": "llm"},
    ],
}

def mostrar_plan(nodo, nivel=0):
    if "herramienta" in nodo:                          # caso base: acción concreta
        print("  " * nivel + f"-{nodo['tarea']}  [{nodo['herramienta']}]")
        return 1
    print("  " * nivel + f"+{nodo['tarea']}")
    acciones = 0
    for sub in nodo["subtareas"]:                      # caso recursivo: descomponer
        acciones += mostrar_plan(sub, nivel + 1)
    return acciones

total = mostrar_plan(plan)
print("Acciones que ejecutará el agente:", total)
```

Paso a paso

37. Cada nodo del plan es un diccionario. Si tiene la clave `herramienta`, es una **acción concreta**: el caso base. Se imprime y cuenta como 1.
38. Si tiene `subtareas`, es una tarea compuesta: se imprime y se llama a la función con cada subtarea, un nivel más adentro.

39. `nivel` controla la sangría, así que la salida refleja la estructura del árbol.
40. La función devuelve el número de acciones de cada rama, y la tarea compuesta suma las de sus hijos. Así sabemos cuántas herramientas tendrá que ejecutar el agente. **Salida esperada**

```
+ Informe de ventas de octubre
  + Obtener datos
    - Consultar ventas  [sql]
    - Consultar objetivos  [api_crm]
  + Analizar
    - Calcular KPIs  [python]
    + Detectar anomalías
      - Comparar con septiembre  [python]
      - Buscar causas en noticias  [web_search]
  - Redactar el informe  [llm]
Acciones que ejecutará el agente: 6
```

Los *splitters* recursivos de herramientas como LangChain intentan cortar un documento por los separadores más "naturales" primero: párrafos, después líneas, después frases y, solo si no queda más remedio, palabras. Así cada chunk respeta al máximo la estructura del texto.

```python
def trocear(texto, max_caracteres, separadores=("\n\n", "\n", ". ", " ")):
    if len(texto) <= max_caracteres:                  # caso base: ya cabe
        return [texto]
    if not separadores:                               # caso base: corte duro
        return [texto[i:i + max_caracteres] for i in range(0, len(texto), max_caracteres)]
    separador = separadores[0]
    piezas = []
    for parte in texto.split(separador):              # caso recursivo
        if parte.strip():
            piezas.extend(trocear(parte.strip(), max_caracteres, separadores[1:]))
    chunks = []                                       # juntar piezas pequeñas
    for pieza in piezas:
        if chunks and len(chunks[-1]) + len(separador) + len(pieza) <= max_caracteres:
            chunks[-1] = chunks[-1] + separador + pieza
        else:
            chunks.append(pieza)
    return chunks

manual = """Política de envíos. Los pedidos se envían en 24 horas.

Devoluciones. El plazo es de 30 días. El producto debe estar sin usar. Los gastos de devolución corren a
cargo del cliente salvo defecto de fábrica.

Garantía. Dos años."""

for i, chunk in enumerate(trocear(manual, 60)):
    print(f"{i} ({len(chunk):>2} car.){chunk!r}")
```

Paso a paso

41. **Caso base 1:** si el texto ya cabe en el límite, se devuelve tal cual.
42. **Caso base 2:** si no quedan separadores, se corta a la fuerza cada `max_caracteres`.
43. **Caso recursivo:** se divide el texto con el primer separador y cada parte se trocea de nuevo con los separadores **restantes**. Una frase demasiado larga acabará dividida por espacios.
44. Al volver, se juntan las piezas pequeñas consecutivas mientras quepan en el límite. Sin este paso, la frase larga quedaría convertida en palabras sueltas.
45. Se muestra cada chunk con `!r` para que se vea el salto de párrafo (`\n\n`) del último. **Salida esperada**

```
0 (54 car.) 'Política de envíos. Los pedidos se envían en 24 horas.'
1 (36 car.) 'Devoluciones. El plazo es de 30 días'
2 (31 car.) 'El producto debe estar sin usar'
3 (57 car.) 'Los gastos de devolución corren a cargo del cliente salvo'
4 (40 car.) 'defecto de fábrica.\n\nGarantía. Dos años.'
```

Ejercicios propuestos

Ejercicio propuesto 7.1

Algunos sistemas de destino limitan el número de niveles de anidamiento que aceptan. Escribe `profundidad(nodo)` con estas reglas: un valor simple tiene profundidad 0, y un diccionario o una lista tienen profundidad 1 más la mayor profundidad de sus elementos. Un diccionario o una lista vacíos tienen profundidad 1.

```
documento = {
    "id": 1,
    "cliente": {"nombre": "Acme", "contacto": {"email": "a@acme.com", "tel": {"pref": "+34"}}},
    "items": [{"sku": "A1", "atributos": {"color": "rojo"}}, {"sku": "B2"}],
}
```

Calcula la profundidad de `documento`, de `documento["items"]` y del valor `42`.

Salida esperada

```
Profundidad: 4
Profundidad de 'items': 3
Profundidad de un valor simple: 0
```

✅ Solución

```
documento = {
    "id": 1,
    "cliente": {"nombre": "Acme", "contacto": {"email": "a@acme.com", "tel": {"pref": "+34"}}},
    "items": [{"sku": "A1", "atributos": {"color": "rojo"}}, {"sku": "B2"}],
}

def profundidad(nodo):
    if isinstance(nodo, dict):
        hijos = list(nodo.values())
    elif isinstance(nodo, list):
        hijos = nodo
    else:
        return 0                                    # caso base: valor simple
    return 1 + max((profundidad(h) for h in hijos), default=0)   # vacío → 1 + 0

print("Profundidad:", profundidad(documento))
print("Profundidad de 'items':", profundidad(documento["items"]))
print("Profundidad de un valor simple:", profundidad(42))
```

🖥️ Salida:

```
Profundidad: 4
Profundidad de 'items': 3
Profundidad de un valor simple: 0
```

Ejercicio propuesto 7.2

```
lake = {
    "bronce": {"ventas": {"2026-09.csv": 120, "2026-10.csv": 135}, "clientes.csv": 40},
    "plata": {
        "ventas": {"year=2026": {"part-0000.parquet": 30, "part-0001.parquet": 28}},
        "clientes": {"part-0000.parquet": 12},
    },
    "oro": {"kpis_mensuales.parquet": 5, "README.md": 1},
}
```

Escribe `buscar_ficheros(nodo, extension, ruta="")`, que devuelva la lista de rutas completas de los ficheros con esa extensión. Usa `nombre.endswith(extension)`. Muestra las rutas `.parquet` y su total, y después solo el total de `.csv`.

Salida esperada

```
/plata/ventas/year=2026/part-0000.parquet
/plata/ventas/year=2026/part-0001.parquet
/plata/clientes/part-0000.parquet
/oro/kpis_mensuales.parquet
Total .parquet: 4
Total .csv: 3
```

✅ Solución

```
lake = {
    "bronce": {"ventas": {"2026-09.csv": 120, "2026-10.csv": 135}, "clientes.csv": 40},
    "plata": {
        "ventas": {"year=2026": {"part-0000.parquet": 30, "part-0001.parquet": 28}},
        "clientes": {"part-0000.parquet": 12},
    },
    "oro": {"kpis_mensuales.parquet": 5, "README.md": 1},
}

def buscar_ficheros(nodo, extension, ruta=""):
    encontrados = []
    for nombre, contenido in nodo.items():
        ruta_actual = f"{ruta}/{nombre}"
        if isinstance(contenido, dict):                        # carpeta → recursión
            encontrados += buscar_ficheros(contenido, extension, ruta_actual)
        elif nombre.endswith(extension):                       # fichero
            encontrados.append(ruta_actual)
    return encontrados

parquet = buscar_ficheros(lake, ".parquet")
for r in parquet:
    print(r)
print("Total .parquet:", len(parquet))
print("Total .csv:", len(buscar_ficheros(lake, ".csv")))
```

🖥️ Salida:

```
/plata/ventas/year=2026/part-0000.parquet
/plata/ventas/year=2026/part-0001.parquet
/plata/clientes/part-0000.parquet
/oro/kpis_mensuales.parquet
Total .parquet: 4
Total .csv: 3
```

Ejercicio propuesto 7.3

Este diccionario indica las tablas de origen directas de cada tabla.

```
origenes = {
    "dashboard_ventas": ["kpis_ventas", "dim_tiempo"],
    "kpis_ventas": ["ventas_limpias", "clientes_limpios"],
    "ventas_limpias": ["ventas_raw", "tipos_cambio"],
    "clientes_limpios": ["clientes_raw"],
    "dim_tiempo": [],
    "ventas_raw": [],
    "clientes_raw": [],
    "tipos_cambio": [],
}
```

Escribe `ancestros(tabla)`, que devuelva un `set` con todas las tablas de las que depende, directa o indirectamente. Muestra los ancestros de `dashboard_ventas`, de `ventas_limpias` y de `ventas_raw`. Por último, con una comprehension, obtén las fuentes originales del dashboard: los ancestros que no tienen ningún origen.

Salida esperada

```
['clientes_limpios', 'clientes_raw', 'dim_tiempo', 'kpis_ventas', 'tipos_cambio', 'ventas_limpias',
'ventas_raw']
```

```
['tipos_cambio', 'ventas_raw']
[]
Fuentes originales del dashboard: ['clientes_raw', 'dim_tiempo', 'tipos_cambio', 'ventas_raw']
```

✅ Solución

```python
origenes = {
    "dashboard_ventas": ["kpis_ventas", "dim_tiempo"],
    "kpis_ventas": ["ventas_limpias", "clientes_limpios"],
    "ventas_limpias": ["ventas_raw", "tipos_cambio"],
    "clientes_limpios": ["clientes_raw"],
    "dim_tiempo": [], "ventas_raw": [], "clientes_raw": [], "tipos_cambio": [],
}

def ancestros(tabla):
    resultado = set()
    for origen in origenes[tabla]:            # sin orígenes → bucle vacío (caso base)
        resultado.add(origen)
        resultado |= ancestros(origen)
    return resultado

print(sorted(ancestros("dashboard_ventas")))
print(sorted(ancestros("ventas_limpias")))
print(sorted(ancestros("ventas_raw")))
print("Fuentes originales del dashboard:",
      sorted(t for t in ancestros("dashboard_ventas") if not origenes[t]))
```

🖥️ Salida:

```
['clientes_limpios', 'clientes_raw', 'dim_tiempo', 'kpis_ventas', 'tipos_cambio', 'ventas_limpias',
'ventas_raw']
['tipos_cambio', 'ventas_raw']
[]
Fuentes originales del dashboard: ['clientes_raw', 'dim_tiempo', 'tipos_cambio', 'ventas_raw']
```

Ejercicio propuesto 7.4

La respuesta mezcla diccionarios y listas a varios niveles.

```python
respuesta_api = {
    "pedido": 9001,
    "cliente": {"nombre": "Acme", "email": None, "telefonos": ["600111222", None]},
    "lineas": [
        {"sku": "A1", "precio": 10.0, "descuento": None},
        {"sku": None, "precio": None, "descuento": 0.1},
    ],
    "notas": None,
}
```

Escribe `contar_nulos(nodo)`, que cuente todos los `None` de la estructura, estén dentro de diccionarios o de listas.

Salida esperada

```
Nulos en la respuesta: 6
Nulos en 'lineas': 3
```

✅ Solución

```python
respuesta_api = {
    "pedido": 9001,
    "cliente": {"nombre": "Acme", "email": None, "telefonos": ["600111222", None]},
    "lineas": [
        {"sku": "A1", "precio": 10.0, "descuento": None},
        {"sku": None, "precio": None, "descuento": 0.1},
    ],
    "notas": None,
}
```

```python
def contar_nulos(nodo):
    if nodo is None:
        return 1
    if isinstance(nodo, dict):
        return sum(contar_nulos(v) for v in nodo.values())
    if isinstance(nodo, list):
        return sum(contar_nulos(v) for v in nodo)
    return 0

print("Nulos en la respuesta:", contar_nulos(respuesta_api))
print("Nulos en 'lineas':", contar_nulos(respuesta_api["lineas"]))
```

🖥️ Salida:

```
Nulos en la respuesta: 6
Nulos en 'lineas': 3
```

Ejercicio propuesto 7.5

Un orquestador rechaza un flujo de tareas si sus dependencias forman un ciclo (por ejemplo, `b → c → d → b`), porque ninguna de esas tareas podría arrancar nunca.

```python
dag_ok = {"a": ["b", "c"], "b": ["d"], "c": ["d"], "d": []}
dag_ko = {"a": ["b"], "b": ["c"], "c": ["d"], "d": ["b"]}
```

46. Escribe `tiene_ciclo(grafo, nodo, camino)`, que devuelva `True` si, partiendo de `nodo`, se vuelve a un nodo que ya está en el camino actual. `camino` es un `set` con los nodos visitados en la rama que se está recorriendo.
47. Escribe `validar_dag(grafo)`, que compruebe todos los nodos y devuelva `True` si no hay ningún ciclo.
    Salida esperada

```
dag_ok es un DAG válido: True
dag_ko es un DAG válido: False
```

✅ Solución

```python
dag_ok = {"a": ["b", "c"], "b": ["d"], "c": ["d"], "d": []}
dag_ko = {"a": ["b"], "b": ["c"], "c": ["d"], "d": ["b"]}

def tiene_ciclo(grafo, nodo, camino):
    if nodo in camino:                          # caso base: volvemos a un nodo de la rama
        return True
    camino = camino | {nodo}                    # copia: cada rama tiene su propio camino
    return any(tiene_ciclo(grafo, sig, camino) for sig in grafo[nodo])

def validar_dag(grafo):
    return not any(tiene_ciclo(grafo, nodo, set()) for nodo in grafo)

print("dag_ok es un DAG válido:", validar_dag(dag_ok))
print("dag_ko es un DAG válido:", validar_dag(dag_ko))
```

🖥️ Salida:

```
dag_ok es un DAG válido: True
dag_ko es un DAG válido: False
```

Ejercicio propuesto 7.6

Los sistemas de almacenamiento informan del tamaño en bytes, pero en los informes se muestran en la unidad más adecuada: KB, MB, GB…

```python
tamanos_bytes = [512, 2048, 5_368_709_120, 1_099_511_627_776, 734_003_200]
```

Escribe `legible(valor, nivel=0)` de forma **recursiva**, usando la lista de unidades `["B", "KB", "MB", "GB", "TB", "PB"]`:

- **Caso base:** si el valor es menor que 1024, o ya no quedan unidades mayores, se devuelve con un decimal y su unidad.
- **Caso recursivo:** se divide el valor entre 1024 y se llama con el nivel siguiente. **Salida esperada**

```
           512 -> 512 B
          2048 -> 2.0 KB
    5368709120 -> 5.0 GB
 1099511627776 -> 1.0 TB
     734003200 -> 700.0 MB
```

✅ Solución

```
UNIDADES = ["B", "KB", "MB", "GB", "TB", "PB"]
tamanos_bytes = [512, 2048, 5_368_709_120, 1_099_511_627_776, 734_003_200]

def legible(valor, nivel=0):
    if valor < 1024 or nivel == len(UNIDADES) - 1:    # caso base
        return f"{valor:.1f} {UNIDADES[nivel]}" if nivel else f"{valor} {UNIDADES[nivel]}"
    return legible(valor / 1024, nivel + 1)          # caso recursivo

for t in tamanos_bytes:
    print(f"{t:>16} -> {legible(t)}")
```

🖥️ Salida:

```
           512 -> 512 B
          2048 -> 2.0 KB
    5368709120 -> 5.0 GB
 1099511627776 -> 1.0 TB
     734003200 -> 700.0 MB
```

Ejercicio propuesto 7.7

Cuando hay muchas particiones, combinar todos sus resultados en un solo nodo puede saturarlo. Spark ofrece `treeReduce()`, que combina los resultados parciales por parejas, en varios niveles, como un torneo.

```
# Conteo de eventos calculado por cada una de las 6 particiones
parciales = [
    {"click": 120, "view": 400},
    {"click": 98, "compra": 7},
    {"view": 310, "compra": 3},
    {"click": 150, "view": 280, "compra": 12},
    {"view": 95},
    {"click": 40, "compra": 1},
]
```

48. Escribe `combinar(a, b)`, que devuelva un diccionario nuevo con la suma de los conteos de los dos diccionarios. No es recursiva.
49. Escribe `tree_reduce(parciales, nivel=0)`, que divida la lista en dos mitades, se llame con cada una y combine los dos resultados. El caso base es una lista con un solo diccionario. Imprime en cada llamada el nivel y el número de parciales que recibe, con una sangría de dos espacios por nivel.
50. Muestra el resultado final con las claves ordenadas. **Salida esperada**

```
nivel 0: 6 parciales
  nivel 1: 3 parciales
    nivel 2: 1 parciales
    nivel 2: 2 parciales
      nivel 3: 1 parciales
      nivel 3: 1 parciales
  nivel 1: 3 parciales
    nivel 2: 1 parciales
    nivel 2: 2 parciales
      nivel 3: 1 parciales
```

```
      nivel 3: 1 parciales
Resultado: {'click': 408, 'compra': 23, 'view': 1085}
```

✅ **Solución**

```python
parciales = [
    {"click": 120, "view": 400},
    {"click": 98, "compra": 7},
    {"view": 310, "compra": 3},
    {"click": 150, "view": 280, "compra": 12},
    {"view": 95},
    {"click": 40, "compra": 1},
]

def combinar(a, b):
    resultado = dict(a)
    for clave, valor in b.items():
        resultado[clave] = resultado.get(clave, 0) + valor
    return resultado

def tree_reduce(parciales, nivel=0):
    print("  " * nivel + f"nivel {nivel}: {len(parciales)} parciales")
    if len(parciales) == 1:                        # caso base
        return parciales[0]
    mitad = len(parciales) // 2
    izq = tree_reduce(parciales[:mitad], nivel + 1)
    der = tree_reduce(parciales[mitad:], nivel + 1)
    return combinar(izq, der)

final = tree_reduce(parciales)
print("Resultado:", dict(sorted(final.items())))
```

🖥️ **Salida:**

```
nivel 0: 6 parciales
  nivel 1: 3 parciales
    nivel 2: 1 parciales
    nivel 2: 2 parciales
      nivel 3: 1 parciales
      nivel 3: 1 parciales
  nivel 1: 3 parciales
    nivel 2: 1 parciales
    nivel 2: 2 parciales
      nivel 3: 1 parciales
      nivel 3: 1 parciales
Resultado: {'click': 408, 'compra': 23, 'view': 1085}
```

Ejercicio propuesto 7.8

En HDFS y en los data lakes, miles de ficheros diminutos degradan el rendimiento: cada fichero supone metadatos y una tarea de lectura. Se recomienda compactar las carpetas donde predominan los ficheros pequeños.

```python
# Carpetas = diccionarios; ficheros = tamaño en MB
tabla_eventos = {
    "fecha=2026-10-13": {"part-0.parquet": 210, "part-1.parquet": 190},
    "fecha=2026-10-14": {f"part-{i}.parquet": 3 for i in range(6)},
    "fecha=2026-10-15": {
        "hora=00": {"part-0.parquet": 2, "part-1.parquet": 4},
        "hora=01": {"part-0.parquet": 150},
    },
}
umbral_mb = 32
```

51. Escribe `ficheros_pequenos(nodo, umbral)`, que recorra una carpeta y todas sus subcarpetas y devuelva una tupla `(total_de_ficheros, ficheros_por_debajo_del_umbral)`.
52. Aplícala a cada partición de fecha y marca `COMPACTAR` las que tengan más de la mitad de sus ficheros por debajo del umbral.

53. Aplícala también a la tabla completa. **Salida esperada**

```
fecha=2026-10-13: 2 ficheros, 0 pequeños -> ok
fecha=2026-10-14: 6 ficheros, 6 pequeños -> COMPACTAR
fecha=2026-10-15: 3 ficheros, 2 pequeños -> COMPACTAR
Tabla completa (ficheros, pequeños): (11, 8)
```

✅ Solución

```python
tabla_eventos = {
    "fecha=2026-10-13": {"part-0.parquet": 210, "part-1.parquet": 190},
    "fecha=2026-10-14": {f"part-{i}.parquet": 3 for i in range(6)},
    "fecha=2026-10-15": {
        "hora=00": {"part-0.parquet": 2, "part-1.parquet": 4},
        "hora=01": {"part-0.parquet": 150},
    },
}
umbral_mb = 32

def ficheros_pequenos(nodo, umbral):
    total, pequenos = 0, 0
    for contenido in nodo.values():
        if isinstance(contenido, dict):
            t, p = ficheros_pequenos(contenido, umbral)     # subcarpeta → recursión
            total += t
            pequenos += p
        else:
            total += 1
            pequenos += contenido < umbral                  # True cuenta como 1
    return total, pequenos

for particion, contenido in tabla_eventos.items():
    total, pequenos = ficheros_pequenos(contenido, umbral_mb)
    estado = "COMPACTAR" if pequenos > total / 2 else "ok"
    print(f"{particion}: {total} ficheros, {pequenos} pequeños -> {estado}")
print("Tabla completa (ficheros, pequeños):", ficheros_pequenos(tabla_eventos, umbral_mb))
```

🖥️ Salida:

```
fecha=2026-10-13: 2 ficheros, 0 pequeños -> ok
fecha=2026-10-14: 6 ficheros, 6 pequeños -> COMPACTAR
fecha=2026-10-15: 3 ficheros, 2 pequeños -> COMPACTAR
Tabla completa (ficheros, pequeños): (11, 8)
```

Ejercicio propuesto 7.9

Las plataformas de observabilidad de agentes guardan cada ejecución como un árbol de pasos: llamadas al LLM, llamadas a herramientas y subagentes, que a su vez lanzan otros pasos.

```python
# Traza de ejecución de un agente: cada paso puede lanzar sub-pasos
traza = {
    "paso": "agente_principal", "tipo": "llm", "tokens": 1200,
    "hijos": [
        {"paso": "buscar_clientes", "tipo": "tool", "herramienta": "sql", "tokens": 0, "hijos": []},
        {"paso": "subagente_analista", "tipo": "llm", "tokens": 2300, "hijos": [
            {"paso": "calcular_churn", "tipo": "tool", "herramienta": "python", "tokens": 0, "hijos": []},
            {"paso": "buscar_contexto", "tipo": "tool", "herramienta": "rag", "tokens": 450, "hijos": [
                {"paso": "reescribir_consulta", "tipo": "llm", "tokens": 300, "hijos": []},
            ]},
        ]},
        {"paso": "redactar_respuesta", "tipo": "llm", "tokens": 1800, "hijos": []},
    ],
}
```

Escribe cuatro funciones recursivas:

- `tokens_totales(nodo)`: suma los tokens del nodo y de todos sus descendientes.
- `profundidad(nodo)`: un paso sin hijos tiene profundidad 1.

- `herramientas(nodo)`: devuelve el `set` de herramientas usadas en los pasos de tipo `tool`.
- `llamadas_llm(nodo)`: cuenta los pasos de tipo `llm`. Muestra los resultados para la traza completa y los tokens consumidos solo por `subagente_analista`.

Salida esperada

```
Tokens totales: 6050
Profundidad máxima: 4
Herramientas usadas: ['python', 'rag', 'sql']
Llamadas al LLM: 4
Tokens del subagente: 3050
```

✅ **Solución**

```
traza = {
    "paso": "agente_principal", "tipo": "llm", "tokens": 1200,
    "hijos": [
        {"paso": "buscar_clientes", "tipo": "tool", "herramienta": "sql", "tokens": 0, "hijos": []},
        {"paso": "subagente_analista", "tipo": "llm", "tokens": 2300, "hijos": [
            {"paso": "calcular_churn", "tipo": "tool", "herramienta": "python", "tokens": 0, "hijos": []},
            {"paso": "buscar_contexto", "tipo": "tool", "herramienta": "rag", "tokens": 450, "hijos": [
                {"paso": "reescribir_consulta", "tipo": "llm", "tokens": 300, "hijos": []},
            ]},
        ]},
        {"paso": "redactar_respuesta", "tipo": "llm", "tokens": 1800, "hijos": []},
    ],
}

def tokens_totales(nodo):
    return nodo["tokens"] + sum(tokens_totales(h) for h in nodo["hijos"])

def profundidad(nodo):
    return 1 + max((profundidad(h) for h in nodo["hijos"]), default=0)

def herramientas(nodo):
    propias = {nodo["herramienta"]} if nodo["tipo"] == "tool" else set()
    for h in nodo["hijos"]:
        propias |= herramientas(h)
    return propias

def llamadas_llm(nodo):
    return (nodo["tipo"] == "llm") + sum(llamadas_llm(h) for h in nodo["hijos"])

print("Tokens totales:", tokens_totales(traza))
print("Profundidad máxima:", profundidad(traza))
print("Herramientas usadas:", sorted(herramientas(traza)))
print("Llamadas al LLM:", llamadas_llm(traza))
subagente = next(h for h in traza["hijos"] if h["paso"] == "subagente_analista")
print("Tokens del subagente:", tokens_totales(subagente))
```

🖥️ **Salida:**

```
Tokens totales: 6050
Profundidad máxima: 4
Herramientas usadas: ['python', 'rag', 'sql']
Llamadas al LLM: 4
Tokens del subagente: 3050
```

Ejercicio propuesto 7.10

Cuando un documento no cabe en el contexto de un LLM, se usa un resumen jerárquico: se resumen los fragmentos por grupos, después se resumen esos resúmenes, y así hasta obtener uno solo. Como aquí no llamamos a un modelo real, la función `resumir()` lo simula quedándose con las 3 primeras palabras de cada texto.

```
# Simulamos el LLM: "resume" un grupo de textos quedándose con
# las 3 primeras palabras de cada uno
def resumir(textos):
```

```
    return " ".join([" ".join(t.split()[:3]) for t in textos])

capitulos = [
    "Ventas crecen un 12% en el canal online",
    "Tiendas físicas estables respecto al trimestre anterior",
    "Devoluciones bajan tras el cambio de política",
    "Coste logístico sube por el precio del combustible",
    "Nuevo almacén en Valencia reduce plazos de entrega",
]
```

54. Escribe `resumen_jerarquico(textos, grupo=2, nivel=0)`. En cada llamada imprime el nivel y cuántos textos recibe. Si solo queda un texto, lo devuelve. Si no, resume los textos en grupos de `grupo` elementos y se llama con la lista de resúmenes.
55. Ejecútala con grupos de 2 y de 3 y compara cuántos niveles necesita cada una.
56. Observa el resumen final: ¿qué información de los capítulos se ha perdido? Con un LLM real ocurre algo parecido, por eso el tamaño de grupo y las instrucciones de cada nivel influyen en la calidad del resultado.
    Salida esperada

```
Nivel 0: 5 textos
Nivel 1: 3 textos
Nivel 2: 2 textos
Nivel 3: 1 textos
Resumen final: Ventas crecen un Nuevo almacén en

Nivel 0: 5 textos
Nivel 1: 2 textos
Nivel 2: 1 textos
Resumen final: Ventas crecen un Coste logístico sube
```

✅ Solución

```
def resumir(textos):
    return " ".join([" ".join(t.split()[:3]) for t in textos])

capitulos = [
    "Ventas crecen un 12% en el canal online",
    "Tiendas físicas estables respecto al trimestre anterior",
    "Devoluciones bajan tras el cambio de política",
    "Coste logístico sube por el precio del combustible",
    "Nuevo almacén en Valencia reduce plazos de entrega",
]

def resumen_jerarquico(textos, grupo=2, nivel=0):
    print(f"Nivel {nivel}: {len(textos)} textos")
    if len(textos) == 1:                                    # caso base
        return textos[0]
    resumenes = [resumir(textos[i:i + grupo]) for i in range(0, len(textos), grupo)]
    return resumen_jerarquico(resumenes, grupo, nivel + 1)

print("Resumen final:", resumen_jerarquico(capitulos, 2))
print()
print("Resumen final:", resumen_jerarquico(capitulos, 3))
# Con grupo=2 hacen falta 3 niveles; con grupo=3, solo 2. En cada nivel se pierde información:
# las cifras (12 %), Tiendas, Devoluciones y la mayoría de los detalles desaparecen.
```

🖥️ Salida:

```
Nivel 0: 5 textos
Nivel 1: 3 textos
Nivel 2: 2 textos
Nivel 3: 1 textos
Resumen final: Ventas crecen un Nuevo almacén en

Nivel 0: 5 textos
Nivel 1: 2 textos
Nivel 2: 1 textos
Resumen final: Ventas crecen un Coste logístico sube
```

Estructura sugerida del repo:

```
Practica-13_Sprints_de_Python/
├── README.md
├── .gitignore
├── sprint_01_estructuras_datos/
│   └── sprint_01_estructuras_datos.ipynb    # listas, tuplas, set y diccionarios
├── sprint_02_comprehensions/
│   └── sprint_02_comprehensions.ipynb
├── sprint_03_funciones/
│   └── sprint_03_funciones.ipynb
└── sprint_04_recursion/
    └── sprint_04_recursion.ipynb
```

Parte 6 · Teoría de las clases 18 a 22 (material del profesor)

> Se omiten los pasos de preparación del entorno y los enunciados de ejercicios que ya están resueltos en las partes 2 a 5.

Clase 18 · Clases y objetos

Parte 01 - Clases y objetos

¿Qué son las clases y los objetos?

> El mundo está orientado a objetos. Está lleno de objetos como Gorrión, Rosa, Guitarra, Teclado, etc.-

- Cada objeto es una instancia específica de una clase. Por ejemplo, el Gorrión es una instancia específica de la clase Pájaro o Rosa es una instancia específica de la clase Flor.
- Pájaro es una clase. Gorrión, Cuervo, Águila son objetos de la clase Pájaro.
- Futbolista es una clase. Messi, Ronaldo, Iniesta son objetos de la clase Futbolista.
- Flor es una clase. Rosas, Tulipanes, Girasoles son objetos de la clase Flor.
- Instrumento musical es una clase: Guitarra, Piano y Flauta son objetos de la clase Instrumento musical.

> Una clase describe dos cosas: la forma que tendrá un objeto creado a partir de ella y la funcionalidad que tendrá. Por ejemplo, la clase Pájaro puede especificar la forma en términos de peso, color, número de plumas, mientras que la funcionalidad nos habla de volar, saltar, comer, entre otros.

- A la 'forma' suele denominarse como **propiedades** y a la funcionalidad: **métodos**. Una clase nos permite agrupar propiedades y funcionalidad juntos.
- Cuando se crean objetos como Gorrión o Águila a partir de la clase Pájaro, las propiedades tendrán valores. Los métodos nos permiten acceder o manipular estos valores. Por ejemplo, la propiedad peso tendrá un valor de 250 gramos para un objeto Gorrión, mientras que para un objeto Águila su peso es de 10 Kg. > Así pues, una clase es de naturaleza genérica, mientras que un objeto es de naturaleza específica. Se pueden crear múltiples objetos a partir de una clase. El proceso de creación de un objeto a partir de una clase se denomina **instanciación.**

Clases y Objetos en Python

- En Python cada tipo de dato es una clase. Así, `int`, `float`, `complex`, `bool`, `str`, `list`, `tuple`, `set`, `dict` son todos clases.
- Una clase tiene un nombre, mientras que los objetos no tienen nombre. Como los objetos no tienen nombre, se hace referencia a ellos utilizando sus direcciones en memoria.

Ejemplos programáticos de clases y objetos:

```
a = 3.14  # a es un objeto de la clase float
s = "Marge"  # s es un objeto de la clase str
lst = [19, 20, 30]  # lst es un objeto de la clase list
tpl = ('a', 'b', 'c')  # tpl es un objeto de la clase tuple
```

- Diferentes objetos de un tipo particular pueden contener diferentes datos, pero los mismos métodos:

```
s1 = 'Sonia'  # s1 es un objeto de tipo str
s2 = 'Greta'  # s2 es un objeto de tipo str
```

> Aquí s1 y s2 son objetos de la clase **str** que contienen datos diferentes, pero los mismos métodos como upper(), lower(), capitalize(), etc.

> Los datos específicos de un objeto a menudo se denominan **datos de instancia o propiedades** del objeto o estado del objeto o **atributos** del objeto. Los *métodos de un objeto* se denominan **métodos de instancia**.

Clases definidas por el usuario

- Además de proporcionar clases prefabricadas como `int`, `str`, `list`, `tuple`, etc., Python nos permite definir nuestras propias clases y crear objetos a partir de ellas.
- Las clases que definimos o creamos se llaman *user-defined data types*. Las reglas para definir y utilizar una clase definida por el usuario y una clase estándar de Python son las mismas.

Definamos una clase que llamaremos Empleado:

```
class Empleado:
    def set_data(self, n, e, s):
        self.nombre = n
        self.edad = e
        self.salario = s

    def display_data(self):
        print(self.nombre, self.edad, self.salario)

e1 = Empleado()
e1.set_data('Julian', 23, 25000)
e1.display_data()

e2 = Empleado()
e2.set_data('Thomas', 25, 30000)
e2.display_data()
```

Salida:

```
Julian 23 25000
Thomas 25 30000
```

> La clase Empleado contiene dos métodos `set_data()` y `display_data()` que se utilizan para establecer y mostrar los datos presentes en los objetos creados a partir de la clase Empleado.

> Se crean dos objetos mediante las sentencias:

```
e1 = Empleado()
e2 = Empleado()
```

- Las direcciones de los objetos se almacenan en `e1` y `e2`.
- En principio, ambos objetos deberían contener los datos de instancia `nombre`, `edad`, `salario` y los métodos de instancia `set_data()` y `display_data()`.

> En la práctica, cada objeto tiene sus propios datos de instancia: nombre, edad y salario, mientras que los métodos set_data() y display_data() se comparten entre los objetos.

> Los datos de instancia no se comparten, ya que los valores de los datos de instancia serían diferentes de un objeto a otro.

- La sintaxis para llamar al método de un objeto es `object.method()`, tal como en `e1.set_data()`.
- Dentro de `set_data()`, `self` contiene la dirección del primer objeto. Del mismo modo, cuando `set_data()` es llamado usando `e2`, `self` contiene la dirección del segundo objeto.

Convenio de acceso.

Hemos accedido a los métodos de instancia `set_data()` y `display_data()` desde fuera de la clase. Incluso los datos de instancia `nombre`, `edad` y `salario` son accesibles desde fuera de la clase. Por lo tanto, las siguientes sentencias funcionarán:

```
e3 = Empleado()
e3.nombre = 'Ana'
e3.edad = 25
e3.salario = 30000
e3.display_data()
```

Salida:

```
Ana 25 30000
```

> Sin embargo, es una buena idea mantener los datos de una clase inaccesibles desde fuera de la clase y acceder a ellos sólo a través de funciones miembro de la clase.

- No hay ningún mecanismo o palabra clave disponible en Python para hacer cumplir esto. Por lo tanto, se utiliza una convención para iniciar los identificadores de datos de instancia con dos guiones bajos (a menudo llamado `underscore`, abreviatura de double underscore). Ejemplo: `__nombre`, `__edad` y `__salario`.

Inicialización de objetos:

Hay dos maneras de inicializar un objeto:

> **inicializar un objeto** significa **crear una instancia con datos iniciales**.

- **Método 1**: Usando métodos como `get_data()`/`set_data()`. > Primero se crea el objeto vacío (por ejemplo: `e = Empleado()`). Luego se usa un **método** para **cargarle datos**:

`get_data()` → suele pedir los datos por teclado (con `input()`).

`set_data()` → recibe valores como parámetros y los asigna a los atributos.

```
# Ejemplo ilustrativo
e = Empleado()
e.set_data('Ana', 25, 30000)
```

- **Método 2**: Usando el método especial `__init__()`. > `__init__()` es el **constructor** de Python: se ejecuta **automáticamente** cuando se crea el objeto. Recibe parámetros **directamente al crear el objeto**, así que no necesitas llamar a un método aparte.

```python
class Empleado:
    def __init__(self, nombre, edad, salario):
        self.nombre = nombre
        self.edad = edad
        self.salario = salario

e = Empleado('Ana', 25, 30000)  # Se inicializa de inmediato
```

- El beneficio de inicializar un objeto usando el método especial `__init__()` es que garantiza la inicialización, ya que `__init__()` siempre se llama cuando se crea un objeto.
- `__init__()` es llamado una sola vez durante toda la vida de un objeto.
- El método `__del__()` es llamado automáticamente cuando un objeto sale del ámbito. La actividad de limpieza, si la hay, debe hacerse en `__del__()`.

Variables y métodos de clase

Definición de variable de clase

> 💡 Una variable de clase es aquella que se **comparte entre todas las instancias de una clase**. Se define a nivel de clase y no está asociada a un objeto específico, lo que significa que su valor es común para todos los objetos creados a partir de esa clase.

Ejemplo:

```python
class Coche:
    total_coches = 0  # Variable de clase

    def __init__(self, marca):
        self.marca = marca  # Variable de instancia
        Coche.total_coches += 1  # Incrementa la variable de clase

# Creando instancias
coche1 = Coche("Toyota")
coche2 = Coche("Honda")

# Accediendo a la variable de clase
print(Coche.total_coches)  # Salida: 2
```

Salida:

```
2
```

Definición de método de clase

> 💡 Un método de clase es una función que pertenece a la clase en su conjunto y no a una instancia específica. Se define utilizando el decorador `@classmethod` y recibe el parámetro `cls` como referencia a la clase, en lugar de `self`.

Ejemplo:

```python
class Estudiante:
    total_estudiantes = 0  # Variable de clase

    @classmethod
    def incrementar_contador(cls):
        cls.total_estudiantes += 1
        return cls.total_estudiantes
```

```python
    @classmethod
    def obtener_total(cls):
        return cls.total_estudiantes

# Usando los métodos de clase
Estudiante.incrementar_contador()  # Añade 1 estudiante
Estudiante.incrementar_contador()  # Añade otro estudiante

print(Estudiante.obtener_total())  # Salida: 2
```

Salida:

```
2
```

Diferencia con variables/métodos de instancia

> 💡 **Variables de instancia**: Están asociadas a objetos individuales y se declaran con `self`. Cada objeto tiene su propia copia de estas variables.

> **Métodos de instancia**: Operan sobre un objeto específico y requieren `self` como primer argumento.

Ejemplo:

```python
class Perro:
    # Variable de clase
    total_perros = 0

    def __init__(self, nombre):
        # Variable de instancia
        self.nombre = nombre
        Perro.total_perros += 1

    # Método de instancia
    def ladrar(self):
        return f"{self.nombre} está ladrando!"

    # Método de clase
    @classmethod
    def contar_perros(cls):
        return f"Total de perros: {cls.total_perros}"

# Creando instancias
perro1 = Perro("Rex")
perro2 = Perro("Luna")

# Accediendo a variables y métodos
print(perro1.nombre)  # Salida: Rex (variable de instancia única por objeto)
print(perro2.nombre)  # Salida: Luna (variable de instancia única por objeto)
print(Perro.total_perros)  # Salida: 2 (variable de clase compartida)
print(perro1.ladrar())  # Salida: Rex está ladrando! (método de instancia)
print(Perro.contar_perros())  # Salida: Total de perros: 2 (método de clase)
```

Salida:

```
Rex
Luna
2
Rex está ladrando!
Total de perros: 2
```

Se puede destacar que:

- Para compartir una variable entre todos los objetos de una clase, se debe declarar como variable de clase o atributo de clase.
- Las variables de clase se declaran sin `self` y no forman parte de los objetos de la clase.
- El acceso a variables de clase se realiza con la sintaxis `classname.varname`.
- Los métodos de clase no reciben el argumento `self`, a diferencia de los métodos de instancia.
- El acceso a métodos de clase se hace con `classname.methodname()`.
- Las variables de clase pueden usarse para contar cuántos objetos se han creado a partir de una clase.

Funciones `vars()` y `dir()`

> 💡 Existen dos útiles funciones integradas (built-in) `vars()` y `dir()`. De ellas, `vars()` devuelve un **diccionario de atributos y sus valores,** mientras que `dir()` devuelve una **lista de atributos.**

Ejemplo aplicado a una clase propia de la librería estandar de Python.

```
import math # módulo estándar

print("--- Atributos del módulo 'math' (vars) ---")
print("Estos son los atributos y sus valores definidos en el diccionario interno de 'math':")
print(vars(math))
print("\n" + "="*50 + "\n") # Separador para mayor claridad

print("--- Lista de atributos y métodos del módulo 'math' (dir) ---")
print("Estos son todos los nombres (atributos y métodos) accesibles en el módulo 'math':")
print(dir(math))
print("\n" + "="*50 + "\n") # Separador final
```

Salida:

```
--- Atributos del módulo 'math' (vars) ---
Estos son los atributos y sus valores definidos en el diccionario interno de 'math':
{'__name__': 'math', '__doc__': 'This module provides access to the mathematical functions\ndefined by the C
standard.', '__package__': '', '__loader__': <_frozen_importlib_external.ExtensionFileLoader object at
0x7fd4f30697e0>, '__spec__': ModuleSpec(name='math', loader=<_frozen_importlib_external.ExtensionFileLoader
object at 0x7fd4f30697e0>, origin='/usr/local/lib/python3.10/lib-dynload/math.cpython-310-x86_64-linux-
gnu.so'), 'acos': <built-in function acos>, 'acosh': <built-in function acosh>, 'asin': <built-in function
asin>, 'asinh': <built-in function asinh>, 'atan': <built-in function atan>, 'atan2': <built-in function
atan2>, 'atanh': <built-in function atanh>, 'ceil': <built-in function ceil>, 'copysign': <built-in function
copysign>, 'cos': <built-in function cos>, 'cosh': <built-in function cosh>, 'degrees': <built-in function
degrees>, 'dist': <built-in function dist>, 'erf': <built-in function erf>, 'erfc': <built-in function erfc>,
'exp': <built-in function exp>, 'expm1': <built-in function expm1>, 'fabs': <built-in function fabs>,
'factorial': <built-in function factorial>, 'floor': <built-in function floor>, 'fmod': <built-in function
fmod>, 'frexp': <built-in function frexp>, 'fsum': <built-in function fsum>, 'gamma': <built-in function
gamma>, 'gcd': <built-in function gcd>, 'hypot': <built-in function hypot>, 'isclose': <built-in function
isclose>, 'isfinite': <built-in function isfinite>, 'isinf': <built-in function isinf>, 'isnan': <built-in
function isnan>, 'isqrt': <built-in function isqrt>, 'lcm': <built-in function lcm>, 'ldexp': <built-in
function ldexp>, 'lgamma': <built-in function lgamma>, 'log': <built-in function log>, 'log1p': <built-in
function log1p>, 'log10': <built-in function log10>, 'log2': <built-in function log2>, 'modf': <built-in
function modf>, 'pow': <built-in function pow>, 'radians': <built-in function radians>, 'remainder': <built-
in function remainder>, 'sin': <built-in function sin>, 'sinh': <built-in function sinh>, 'sqrt': <built-in
function sqrt>, 'tan': <built-in function tan>, 'tanh': <built-in function tanh>, 'trunc': <built-in function
trunc>, 'prod': <built-in function prod>, 'perm': <built-in function perm>, 'comb': <built-in function comb>,
'nextafter': <built-in function nextafter>, 'ulp': <built-in function ulp>, '__file__':
'/usr/local/lib/python3.10/lib-dynload/math.cpython-310-x86_64-linux-gnu.so', 'pi': 3.141592653589793, 'e':
2.718281828459045, 'tau': 6.283185307179586, 'inf': inf, 'nan': nan}

==================================================

--- Lista de atributos y métodos del módulo 'math' (dir) ---
Estos son todos los nombres (atributos y métodos) accesibles en el módulo 'math':
['__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__', 'acos', 'acosh', 'asin',
'asinh', 'atan', 'atan2', 'atanh', 'ceil', 'comb', 'copysign', 'cos', 'cosh', 'degrees', 'dist', 'e', 'erf',
```

```
'erfc', 'exp', 'expm1', 'fabs', 'factorial', 'floor', 'fmod', 'frexp', 'fsum', 'gamma', 'gcd', 'hypot',
'inf', 'isclose', 'isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma', 'log', 'log10', 'log1p',
'log2', 'modf', 'nan', 'nextafter', 'perm', 'pi', 'pow', 'prod', 'radians', 'remainder', 'sin', 'sinh',
'sqrt', 'tan', 'tanh', 'tau', 'trunc', 'ulp']

================================================
```

Ejemplo aplicado a una clase creada por un usuario:

```python
class Fruta:
    count = 0

    def __init__(self, nombre='', tamaño=0, color=''):
        self._nombre = nombre
        self._tamaño = tamaño
        self._color = color
        Fruta.count += 1

    def display():
        print(Fruta.count)

f1 = Fruta('Banana', 5, 'Amarillo')
print(vars(Fruta))
print(dir(Fruta))
```

Salida:

```
{'__module__': '__main__', 'count': 1, '__init__': <function Fruta.__init__ at 0x7fd4a1eabd90>, 'display':
<function Fruta.display at 0x7fd4a1eabf40>, '__dict__': <attribute '__dict__' of 'Fruta' objects>,
'__weakref__': <attribute '__weakref__' of 'Fruta' objects>, '__doc__': None}
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__',
'__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
'__subclasshook__', '__weakref__', 'count', 'display']
```

```python
print(vars(f1))
print(dir(f1))
```

Salida:

```
{'_nombre': 'Banana', '_tamaño': 5, '_color': 'Amarillo'}
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__',
'__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
'__subclasshook__', '__weakref__', '_color', '_nombre', '_tamaño', 'count', 'display']
```

Ejemplo 2.1 · Crear objetos de una clase vacía

> 💡 Un sistema de gestión de pedidos necesita empezar definiendo el tipo `Pedido`, aunque todavía no tenga atributos ni comportamiento. Crea una clase `Pedido` **vacía**, usando `pass` como cuerpo.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_2_1_crear_objetos.py`

```python
class Pedido:
    pass

pedido_a = Pedido()
pedido_b = Pedido()
```

```
print(type(pedido_a))
print(type(pedido_a).__name__)
print(isinstance(pedido_a, Pedido))
print(isinstance("PED-1001", Pedido))
print(pedido_a is pedido_b)
```

```
<class '__main__.Pedido'>
Pedido
True
False
False
```

Ejemplo 2.2 · Atributos añadidos a mano y el problema que generan

> 💡 En Python, a un objeto se le pueden añadir atributos en cualquier momento con la notación `objeto.atributo = valor`. En este ejercicio vas a comprobar qué problema genera hacerlo así.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_2_2_atributos_a_mano.py`

```
class Pedido:
    pass

pedido_a = Pedido()
pedido_a.codigo = "PED-1001"
pedido_a.importe = 149.90

pedido_b = Pedido()
pedido_b.codigo = "PED-1002"

print(pedido_a.codigo, pedido_a.importe)
print(pedido_b.codigo)
print(hasattr(pedido_a, "importe"))
print(hasattr(pedido_b, "importe"))
print(vars(pedido_a))
print(vars(pedido_b))
```

Salida esperada:

```
PED-1001 149.9
PED-1002
True
False
{'codigo': 'PED-1001', 'importe': 149.9}
{'codigo': 'PED-1002'}
```

Sección 3. El constructor `__init__` y `self`

Definición

El **constructor** es un método especial llamado `__init__` (dos guiones bajos a cada lado) que Python ejecuta **automáticamente** cada vez que se crea un objeto. Su función es dar a cada objeto sus atributos iniciales, de modo que **todos los objetos de la clase nazcan con la misma estructura**.

El primer parámetro de `__init__`, y de cualquier método de instancia, es **`self`**: una referencia al objeto que se está creando o usando. `self.nombre = nombre` significa «guarda en **este** objeto un atributo `nombre` con el valor recibido».

Sintaxis

```
class NombreDeLaClase:
    def __init__(self, parametro1, parametro2, parametro_opcional=valor_por_defecto):
        self.atributo1 = parametro1
        self.atributo2 = parametro2
        self.atributo3 = parametro_opcional
        self.atributo_calculado = ...       # no tiene por qué venir de un parámetro

objeto = NombreDeLaClase(valor1, valor2)    # self NO se pasa: lo pone Python
```

Qué ocurre al ejecutar `objeto = NombreDeLaClase(valor1, valor2)`:

13. Python crea un objeto vacío de la clase.
14. Llama a `__init__` pasando ese objeto como `self`, seguido de `valor1` y `valor2`.
15. `__init__` rellena los atributos del objeto.
16. El objeto ya inicializado se asigna a la variable `objeto`. > `__init__` no lleva `return`: su trabajo es inicializar el objeto, no devolver un valor.

Ejemplo 3.1 · Fuentes de datos de una plataforma

> 💡💡 El equipo de datos quiere registrar las fuentes de las que se alimenta su *data lake*. Crea una clase `FuenteDatos` cuyo constructor reciba tres **parámetros obligatorios** y los guarde como atributos del objeto: - `nombre`: identificador de la fuente. - `tipo`: forma de acceso a los datos (por ejemplo, `"api"` o `"fichero"`). - `ubicacion`: URL o ruta donde se encuentran los datos. Después, crea dos fuentes:

- `crm`: `"crm_clientes"`, de tipo `"api"`, ubicada en `"https://api.crm.empresa.com/v2"`.
- `erp`: `"erp_facturas"`, de tipo `"fichero"`, ubicada en `"/landing/erp/facturas/"`. Por último:

1. Muestra el nombre, tipo y ubicación de cada fuente en una línea, separados por `|`.
2. Usa la función `vars()` sobre `erp` para mostrar todos sus atributos en forma de diccionario.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_3_1_fuente_datos.py`

```
class FuenteDatos:
    def __init__(self, nombre, tipo, ubicacion):
        self.nombre = nombre
        self.tipo = tipo
        self.ubicacion = ubicacion

crm = FuenteDatos("crm_clientes", "api", "https://api.crm.empresa.com/v2")
erp = FuenteDatos("erp_facturas", "fichero", "/landing/erp/facturas/")

print(crm.nombre, "|", crm.tipo, "|", crm.ubicacion)
print(erp.nombre, "|", erp.tipo, "|", erp.ubicacion)
print(vars(erp))
```

Salida esperada:

```
crm_clientes | api | https://api.crm.empresa.com/v2
erp_facturas | fichero | /landing/erp/facturas/
{'nombre': 'erp_facturas', 'tipo': 'fichero', 'ubicacion': '/landing/erp/facturas/'}
```

Ejemplo 3.2 · Valores por defecto y atributos de estado

> 💡 Crea una clase `TareaIngesta` que represente una tarea de carga de datos. El constructor debe recibir: - `nombre` y `origen` como **parámetros obligatorios**. - `formato` y `reintentos_max` como **parámetros opcionales**, con valores por defecto `"csv"` y `3`. Además, el constructor debe crear dos **atributos de estado** que **no se reciben como parámetro** y que toda tarea nueva tiene con el mismo valor inicial:

- `estado`, con el valor `"pendiente"`.
- `filas_cargadas`, con el valor `0`. Después, crea dos tareas:
- `t1`: `"ventas_diarias"` desde `"sftp_tiendas"`, usando los valores por defecto.
- `t2`: `"clics_web"` desde `"blob_logs"`, con formato `"json"` y `5` reintentos. Por último, recorre ambas tareas con un bucle `for` y muestra en una línea por tarea su nombre, formato, reintentos, estado y filas cargadas, alineando las columnas como en la salida esperada.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_3_2_tarea_ingesta.py`

```python
class TareaIngesta:
    def __init__(self, nombre, origen, formato="csv", reintentos_max=3):
        self.nombre = nombre
        self.origen = origen
        self.formato = formato
        self.reintentos_max = reintentos_max
        self.estado = "pendiente"
        self.filas_cargadas = 0

t1 = TareaIngesta("ventas_diarias", "sftp_tiendas")
t2 = TareaIngesta("clics_web", "blob_logs", formato="json", reintentos_max=5)

for tarea in [t1, t2]:
    print(f"{tarea.nombre:<15} formato={tarea.formato:<5} reintentos={tarea.reintentos_max} "
          f"estado={tarea.estado} filas={tarea.filas_cargadas}")
```

Salida esperada:

```
ventas_diarias  formato=csv   reintentos=3 estado=pendiente filas=0
clics_web       formato=json  reintentos=5 estado=pendiente filas=0
```

Ejemplo 3.3 · Atributos calculados en el constructor

> 💡 Cuando llega un fichero a la zona *landing*, el equipo de datos necesita extraer automáticamente información derivada de su ruta y su tamaño. La **zona *landing*** (zona de aterrizaje o de llegada) es el primer área de almacenamiento de un *data lake* o de una plataforma de datos: el lugar donde se depositan los ficheros **tal como llegan de los sistemas de origen**, sin transformar ni validar.

Crea una clase `ArchivoLanding` cuyo constructor reciba dos **parámetros obligatorios**: - `ruta`: ruta completa del fichero. - `tamano_bytes`: tamaño del fichero en bytes. Además de guardar ambos valores, el constructor debe crear cuatro **atributos calculados** a partir de ellos, sin recibirlos como parámetro:

- `nombre`: el nombre del fichero, es decir, la última parte de la ruta tras la última `/`.
- `extension`: la extensión del fichero, obtenida a partir de `nombre` y convertida a minúsculas.
- `tamano_mb`: el tamaño en megabytes, redondeado a 2 decimales (1 MB = 1024 × 1024 bytes).
- `es_grande`: `True` si el fichero supera los 100 MB y `False` en caso contrario. Ten en cuenta que algunos atributos dependen de otros, así que piensa en qué orden debes crearlos.

Después, crea un objeto `archivo` con la ruta `"/landing/ventas/2026/10/ventas_tienda_07.CSV"` y un tamaño de `157_286_400` bytes, y muestra por pantalla su nombre, extensión, tamaño en MB y si es grande.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_3_3_archivo_landing.py`

```python
class ArchivoLanding:
    def __init__(self, ruta, tamano_bytes):
        self.ruta = ruta
        self.tamano_bytes = tamano_bytes
        self.nombre = ruta.split("/")[-1]
        self.extension = self.nombre.split(".")[-1].lower()
        self.tamano_mb = round(tamano_bytes / (1024 * 1024), 2)
        self.es_grande = self.tamano_mb > 100

archivo = ArchivoLanding("/landing/ventas/2026/10/ventas_tienda_07.CSV", 157_286_400)

print(archivo.nombre)
print(archivo.extension)
print(archivo.tamano_mb)
print(archivo.es_grande)
```

Salida esperada:

```
ventas_tienda_07.CSV
csv
150.0
True
```

Sección 4. Métodos de instancia

Definición

> 💡 Un **método de instancia** es una función definida dentro de la clase que trabaja con los datos de un objeto concreto. Igual que `__init__`, recibe `self` como primer parámetro, y a través de él puede **leer** y **modificar** los atributos del objeto o **llamar a otros métodos** del mismo objeto.

Sintaxis

```python
class NombreDeLaClase:
    def __init__(self, ...):
        ...

    def nombre_metodo(self, parametro):
        # usa self.atributo y parametro
        return resultado

objeto = NombreDeLaClase(...)
objeto.nombre_metodo(argumento)          # forma habitual
NombreDeLaClase.nombre_metodo(objeto, argumento)   # forma equivalente
```

> La forma habitual `objeto.metodo(argumento)` es una abreviatura: Python la traduce a `Clase.metodo(objeto, argumento)`. Por eso `self` aparece en la definición pero no en la llamada.

Tipo de método	Qué hace	Ejemplo
Consulta	Lee atributos y devuelve un resultado	`importe_total()`

Tipo de método	Qué hace	Ejemplo
Modificador	Cambia el estado del objeto	`cargar(filas)`
Compuesto	Llama a otros métodos con `self.metodo()`	`total()` usa `subtotal()`

Ejemplo 4.1 · Lote de registros con métodos de consulta

> 💡 💡 Un proceso de ingesta agrupa los tickets de venta de cada día en **lotes**. Crea una clase `LoteDatos` que permita ir añadiendo registros a un lote y consultar información sobre ellos. El constructor debe recibir el parámetro obligatorio `id_lote` y crear el atributo `registros` como una **lista vacía** propia de cada lote.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_4_1_lote_datos.py`

```python
class LoteDatos:
    def __init__(self, id_lote):
        self.id_lote = id_lote
        self.registros = []

    def agregar(self, registro):
        self.registros.append(registro)

    def total_registros(self):
        return len(self.registros)

    def importe_total(self):
        total = 0
        for registro in self.registros:
            total += registro["importe"]
        return round(total, 2)

lote = LoteDatos("L-2026-10-26-01")
lote.agregar({"ticket": "T1", "importe": 23.50})
lote.agregar({"ticket": "T2", "importe": 8.75})
lote.agregar({"ticket": "T3", "importe": 112.00})

print(lote.id_lote)
print(lote.total_registros())
print(lote.importe_total())
print(LoteDatos.total_registros(lote))
```

Salida esperada:

```
L-2026-10-26-01
3
144.25
3
```

Ejemplo 4.2 · Métodos que controlan el ciclo de vida de una tarea

> 💡 Una tarea de ingesta pasa por distintos estados a lo largo de su vida: `"pendiente"` → `"en_ejecucion"` → `"completada"`. Crea una clase `TareaIngesta` cuyos **métodos protejan sus propias reglas**, de forma que la tarea no permita acciones que no correspondan a su estado actual. El constructor debe recibir el parámetro obligatorio `nombre` y crear dos atributos de estado con valor inicial fijo: - `estado`, con el valor `"pendiente"`. - `filas_cargadas`, con el valor `0`. La clase debe tener tres **métodos**:

- `iniciar()`: solo puede ejecutarse si la tarea está `"pendiente"`. En ese caso, cambia el estado a `"en_ejecucion"` y muestra `[nombre] Iniciada`. Si no, muestra `[nombre] No se puede iniciar: estado`

`actual '<estado>'` y no cambia nada.

- `cargar(filas)`: solo puede ejecutarse si la tarea está `"en_ejecucion"`. En ese caso, suma `filas` a `filas_cargadas` y muestra `[nombre] +<filas> filas (acumulado:<filas_cargadas>)`. Si no, muestra `[nombre] Carga rechazada: la tarea no está en ejecución` y no suma nada.
- `finalizar()`: si la tarea está `"en_ejecucion"`, cambia el estado a `"completada"`. En cualquier caso, muestra `[nombre] Estado final:<estado> con<filas_cargadas> filas`. Después, crea una tarea llamada `"ventas_diarias"` y ejecuta, **en este orden**, las siguientes llamadas:

1. `cargar(500)`
2. `iniciar()`
3. `cargar(12000)`
4. `cargar(8500)`
5. `iniciar()`
6. `finalizar()` Antes de ejecutar el código, intenta predecir qué mensaje mostrará cada llamada. ¿Cuántas filas se habrán cargado al final? ¿Por qué no cuentan las 500 de la primera llamada?

📄 `clase_18_clases_objetos/teoria/ejemplo_4_2_ciclo_vida_tarea.py`

```python
class TareaIngesta:
    def __init__(self, nombre):
        self.nombre = nombre
        self.estado = "pendiente"
        self.filas_cargadas = 0

    def iniciar(self):
        if self.estado != "pendiente":
            print(f"[{self.nombre}] No se puede iniciar: estado actual '{self.estado}'")
            return
        self.estado = "en_ejecucion"
        print(f"[{self.nombre}] Iniciada")

    def cargar(self, filas):
        if self.estado != "en_ejecucion":
            print(f"[{self.nombre}] Carga rechazada: la tarea no está en ejecución")
            return
        self.filas_cargadas += filas
        print(f"[{self.nombre}] +{filas} filas (acumulado:{self.filas_cargadas})")

    def finalizar(self):
        if self.estado == "en_ejecucion":
            self.estado = "completada"
        print(f"[{self.nombre}] Estado final:{self.estado} con{self.filas_cargadas} filas")

tarea = TareaIngesta("ventas_diarias")
tarea.cargar(500)
tarea.iniciar()
tarea.cargar(12000)
tarea.cargar(8500)
tarea.iniciar()
tarea.finalizar()
```

Salida esperada:

```
[ventas_diarias] Carga rechazada: la tarea no está en ejecución
[ventas_diarias] Iniciada
[ventas_diarias] +12000 filas (acumulado: 12000)
[ventas_diarias] +8500 filas (acumulado: 20500)
[ventas_diarias] No se puede iniciar: estado actual 'en_ejecucion'
[ventas_diarias] Estado final: completada con 20500 filas
```

Ejemplo 4.3 · Métodos que llaman a otros métodos

Una factura calcula el total reutilizando el subtotal y los impuestos.

> 💡 El departamento de facturación necesita generar facturas que calculen automáticamente su subtotal, sus impuestos y su total. Crea una clase `Factura` en la que **unos métodos reutilicen a otros** en lugar de repetir los mismos cálculos.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_4_3_factura.py`

```
class Factura:
    def __init__(self, numero, cliente, tipo_iva=0.21):
        self.numero = numero
        self.cliente = cliente
        self.tipo_iva = tipo_iva
        self.lineas = []

    def agregar_linea(self, concepto, cantidad, precio_unitario):
        self.lineas.append({"concepto": concepto, "cantidad": cantidad, "precio": precio_unitario})

    def subtotal(self):
        return round(sum([linea["cantidad"] * linea["precio"] for linea in self.lineas]), 2)

    def impuestos(self):
        return round(self.subtotal() * self.tipo_iva, 2)

    def total(self):
        return round(self.subtotal() + self.impuestos(), 2)

    def imprimir(self):
        print(f"Factura{self.numero} · Cliente:{self.cliente}")
        for linea in self.lineas:
            importe = linea["cantidad"] * linea["precio"]
            print(f"{linea['concepto']:<22}{linea['cantidad']:>3} x{linea['precio']:>8.2f} ={importe:>9.2f}")
        print(f"{'Subtotal':<38}{self.subtotal():>9.2f}")
        print(f"{'IVA ' + str(int(self.tipo_iva * 100)) + '%':<38}{self.impuestos():>9.2f}")
        print(f"{'TOTAL':<38}{self.total():>9.2f}")

factura = Factura("F-2026-0412", "Logística Iberia S.L.")
factura.agregar_linea("Licencia ETL (usuario)", 3, 49.90)
factura.agregar_linea("Soporte anual", 1, 1200.00)
factura.agregar_linea("Formación (jornada)", 2, 350.00)
factura.imprimir()
```

Salida esperada:

```
Factura F-2026-0412 · Cliente: Logística Iberia S.L.
  Licencia ETL (usuario)   3 x    49.90 =    149.70
  Soporte anual            1 x  1200.00 =   1200.00
  Formación (jornada)      2 x   350.00 =    700.00
  Subtotal                                 2049.70
  IVA 21%                                   430.44
  TOTAL                                    2480.14
```

Sección 5. Atributos de clase y atributos de instancia

> 💡 ##### Definición

- Un **atributo de instancia** se crea con `self.atributo` y pertenece a **un objeto concreto**. Cada objeto tiene su propio valor.
- Un **atributo de clase** se define directamente dentro de la clase, fuera de cualquier método, y es **compartido por todos los objetos**. Se usa para constantes comunes (unidades, umbrales por defecto, versión de un esquema) o para contadores globales. Cuando se lee `objeto.atributo`, Python busca **primero en el objeto** y, si no lo encuentra, **en la clase**.

Sintaxis

```python
class NombreDeLaClase:
    atributo_de_clase = valor          # compartido

    def __init__(self, dato):
        self.atributo_de_instancia = dato   # propio de cada objeto
        NombreDeLaClase.atributo_de_clase += 1  # para modificar el compartido, se usa el nombre de la clase
```

	Atributo de instancia	Atributo de clase
Dónde se define	Dentro de los métodos, con `self.`	En el cuerpo de la clase
A quién pertenece	A cada objeto	A la clase (todos lo comparten)
Cómo se modifica	`self.atributo = valor`	`Clase.atributo = valor`
Uso típico	Datos de cada registro	Constantes, contadores, configuración común

Ejemplo 5.1 · Constante compartida y contador de objetos

> 💡 Una cadena logística monitoriza la temperatura de sus almacenes mediante sensores. Crea una clase `Sensor` que combine **atributos de clase**, compartidos por todos los sensores, con **atributos de instancia**, propios de cada uno.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_5_1_sensor.py`

```python
class Sensor:
    unidad = "°C"
    total_sensores = 0

    def __init__(self, codigo, ubicacion):
        self.codigo = codigo
        self.ubicacion = ubicacion
        Sensor.total_sensores += 1

    def describir(self, valor):
        return f"{self.codigo} ({self.ubicacion}):{valor}{self.unidad}"

s1 = Sensor("TMP-01", "Cámara frigorífica A")
s2 = Sensor("TMP-02", "Cámara frigorífica B")
s3 = Sensor("TMP-03", "Muelle de carga")

print(s1.describir(-18.4))
print(s3.describir(12.1))
print("Sensores registrados:", Sensor.total_sensores)
print(s2.total_sensores)
```

Salida esperada:

```
TMP-01 (Cámara frigorífica A): -18.4 °C
TMP-03 (Muelle de carga): 12.1 °C
Sensores registrados: 3
3
```

Ejemplo 5.2 · Un objeto puede «tapar» el atributo de clase

> 💡 Un equipo de calidad de datos define un **umbral máximo de nulos** que se aplica por defecto a todas las columnas, aunque algunas columnas concretas pueden tolerar más. En este ejercicio vas a comprobar qué ocurre cuando un objeto **“tapa”** un atributo de clase con uno propio.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_5_2_regla_calidad.py`

```python
class ReglaCalidad:
    umbral_nulos = 0.05

    def __init__(self, columna):
        self.columna = columna

    def evaluar(self, nulos, total):
        ratio = nulos / total
        resultado = "OK" if ratio <= self.umbral_nulos else "FALLO"
        return f"{self.columna}:{ratio:.1%} nulos (umbral{self.umbral_nulos:.0%}) ->{resultado}"

r_email = ReglaCalidad("email")
r_telefono = ReglaCalidad("telefono")
r_telefono.umbral_nulos = 0.20

print(r_email.evaluar(80, 1000))
print(r_telefono.evaluar(150, 1000))
print(vars(r_email))
print(vars(r_telefono))

ReglaCalidad.umbral_nulos = 0.10
print(r_email.evaluar(80, 1000))
print(r_telefono.evaluar(150, 1000))
```

Salida esperada:

```
email: 8.0% nulos (umbral 5%) -> FALLO
telefono: 15.0% nulos (umbral 20%) -> OK
{'columna': 'email'}
{'columna': 'telefono', 'umbral_nulos': 0.2}
email: 8.0% nulos (umbral 10%) -> OK
telefono: 15.0% nulos (umbral 20%) -> OK
```

Ejemplo 5.3 · La trampa de las listas como atributo de clase

> 💡 Un equipo de integración necesita una cola de mensajes independiente para cada *topic* (por ejemplo, `"pedidos"` y `"pagos"`). Un desarrollador ha definido la lista de mensajes **como atributo de clase** por error. En este ejercicio vas a reproducir el fallo y descubrir por qué ocurre.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_5_3_cola_mensajes_error.py`

```python
class ColaMensajes:
    mensajes = []          # atributo de clase mutable: compartido por todas las colas

    def __init__(self, topic):
        self.topic = topic

    def publicar(self, mensaje):
        self.mensajes.append(mensaje)
```

```
pedidos = ColaMensajes("pedidos")
pagos = ColaMensajes("pagos")
pedidos.publicar("pedido 501 creado")
pagos.publicar("pago 77 confirmado")

print(pedidos.topic, pedidos.mensajes)
print(pagos.topic, pagos.mensajes)
print(pedidos.mensajes is pagos.mensajes)
```

Salida obtenida (incorrecta):

```
pedidos ['pedido 501 creado', 'pago 77 confirmado']
pagos ['pedido 501 creado', 'pago 77 confirmado']
True
```

> 💡 `self.mensajes.append(...)` no crea un atributo nuevo: **modifica la lista de la clase**, que es la misma para todas las colas. Los mensajes de pagos aparecen en pedidos y viceversa. La corrección es crear la lista en el constructor, de modo que cada objeto tenga la suya:

> 📄 `clase_18_clases_objetos/teoria/ejemplo_5_3_cola_mensajes_corregida.py`

```
class ColaMensajes:
    def __init__(self, topic):
        self.topic = topic
        self.mensajes = []     # atributo de instancia: una lista por cola

    def publicar(self, mensaje):
        self.mensajes.append(mensaje)

pedidos = ColaMensajes("pedidos")
pagos = ColaMensajes("pagos")
pedidos.publicar("pedido 501 creado")
pagos.publicar("pago 77 confirmado")

print(pedidos.topic, pedidos.mensajes)
print(pagos.topic, pagos.mensajes)
print(pedidos.mensajes is pagos.mensajes)
```

Salida esperada:

```
pedidos ['pedido 501 creado']
pagos ['pago 77 confirmado']
False
```

> **Regla práctica:** las listas, diccionarios y `set` que deban ser propios de cada objeto se crean siempre dentro de `__init__` con `self.`.

Sección 6. Representar objetos como texto: `__str__` y `__repr__`

> 💡 ##### Definición

Si se imprime un objeto de una clase propia sin más, Python muestra algo como `<__main__.Producto object at 0x7f3a2c1b9d50>`: el nombre de la clase y una dirección de memoria que cambia en cada ejecución. Para controlar cómo se muestra un objeto, la clase puede definir dos métodos especiales:

Método	Lo usa	Pensado para	Estilo recomendado
`__str__`	`print(obj)`, `str(obj)`, f-strings	Personas: informes, logs, pantallas	Legible y amigable
`__repr__`	`repr(obj)`, consola interactiva, objetos dentro de listas o diccionarios	Desarrolladores: depuración	Preciso; idealmente, parecido al código que crea el objeto

Ambos deben **devolver** (`return`) un `str`, no imprimirlo. Si una clase solo define `__repr__`, Python también lo usa para `print`.

Sintaxis

```
class NombreDeLaClase:
    def __init__(self, ...):
        ...

    def __str__(self):
        return "texto legible"

    def __repr__(self):
        return "NombreDeLaClase(atributo=valor, ...)"
```

Ejemplo 6.1 · Producto de un catálogo

> 💡 Una tienda online necesita representar los productos de su catálogo de forma legible, tanto para mostrarlos a los usuarios como para depurarlos en el código. Crea una clase `Producto` que defina **dos representaciones en texto** distintas mediante los métodos especiales `__str__` y `__repr__`.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_6_1_producto.py`

```
class Producto:
    def __init__(self, sku, nombre, precio):
        self.sku = sku
        self.nombre = nombre
        self.precio = precio

    def __str__(self):
        return f"{self.nombre} ({self.sku}) -{self.precio:.2f} €"

    def __repr__(self):
        return f"Producto(sku='{self.sku}', nombre='{self.nombre}', precio={self.precio})"

p = Producto("SKU-4471", "Auriculares BT", 59.9)

print(p)
print(str(p))
print(repr(p))
print(f"Producto destacado:{p}")

catalogo = [p, Producto("SKU-1020", "Cargador USB-C", 19.95)]
print(catalogo)
```

Salida esperada:

```
Auriculares BT (SKU-4471) - 59.90 €
Auriculares BT (SKU-4471) - 59.90 €
Producto(sku='SKU-4471', nombre='Auriculares BT', precio=59.9)
Producto destacado: Auriculares BT (SKU-4471) - 59.90 €
```

```
[Producto(sku='SKU-4471', nombre='Auriculares BT', precio=59.9), Producto(sku='SKU-1020', nombre='Cargador
USB-C', precio=19.95)]
```

Ejemplo 6.2 · Eventos de log con formato homogéneo

> 💡 El equipo de plataforma necesita que todos los eventos de log de sus *pipelines* se muestren con un **formato homogéneo**, sin tener que repetir el formato en cada `print`. Crea una clase `EventoLog` que defina cómo se representa cada evento mediante los métodos especiales `__str__` y `__repr__`.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_6_2_evento_log.py`

```python
class EventoLog:
    def __init__(self, timestamp, nivel, servicio, mensaje):
        self.timestamp = timestamp
        self.nivel = nivel
        self.servicio = servicio
        self.mensaje = mensaje

    def __str__(self):
        return f"{self.timestamp} [{self.nivel:<5}]{self.servicio:<12} |{self.mensaje}"

    def __repr__(self):
        return f"EventoLog('{self.timestamp}', '{self.nivel}', '{self.servicio}')"

eventos = [
    EventoLog("2026-10-26 02:00:01", "INFO", "ingesta", "Inicio de carga de ventas"),
    EventoLog("2026-10-26 02:03:47", "WARN", "ingesta", "Fichero tienda_12 llega vacío"),
    EventoLog("2026-10-26 02:05:10", "ERROR", "transform", "Columna 'importe' con texto no numérico"),
    EventoLog("2026-10-26 02:09:33", "INFO", "carga", "Lote publicado en zona curated"),
]

for evento in eventos:
    print(evento)

errores = [e for e in eventos if e.nivel == "ERROR"]
print(errores)
```

Salida esperada:

```
2026-10-26 02:00:01 [INFO ] ingesta      | Inicio de carga de ventas
2026-10-26 02:03:47 [WARN ] ingesta      | Fichero tienda_12 llega vacío
2026-10-26 02:05:10 [ERROR] transform    | Columna 'importe' con texto no numérico
2026-10-26 02:09:33 [INFO ] carga        | Lote publicado en zona curated
[EventoLog('2026-10-26 02:05:10', 'ERROR', 'transform')]
```

Sección 7. Objetos como referencias, colecciones de objetos y composición

> 💡 ##### Definición

- Una variable **no contiene** el objeto: contiene una **referencia** a él. Si se hace `b = a`, las dos variables apuntan al **mismo** objeto, y un cambio hecho a través de `b` se ve desde `a`. Lo mismo ocurre al pasar un objeto a una función.
- `is` comprueba si dos variables apuntan al mismo objeto. `==` compara valores.
- Los objetos se pueden guardar en **listas** y **diccionarios** como cualquier otro valor, y procesarse con bucles y comprehensions.
- Hay **composición** cuando un objeto contiene otros objetos como atributos: un `Pipeline` tiene `Etapa`s, un `Pedido` tiene `LineaPedido`s. Es la forma natural de modelar sistemas formados por piezas.

Sintaxis

```
alias = objeto              # misma referencia, no una copia
alias is objeto             # True

coleccion = [Clase(...), Clase(...)]          # lista de objetos
indice = {obj.clave: obj for obj in coleccion} # diccionario de objetos

class Contenedor:
    def __init__(self):
        self.partes = []                      # composición: guarda objetos de otra clase

    def agregar(self, parte):
        self.partes.append(parte)
```

Ejemplo 7.1 · Referencias compartidas

> 💡 Un programa de fidelización gestiona los puntos que acumula cada cliente. En este ejercicio vas a comprobar que una variable **no contiene** el objeto, sino una **referencia** a él, y qué consecuencias tiene esto al asignar variables y al pasar objetos a funciones.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_7_1_referencias.py`

```
class ClienteFidelidad:
    def __init__(self, id_cliente, puntos=0):
        self.id_cliente = id_cliente
        self.puntos = puntos

    def sumar_puntos(self, cantidad):
        self.puntos += cantidad

def aplicar_bono_bienvenida(cliente):
    cliente.sumar_puntos(200)

original = ClienteFidelidad("C-88", 100)
alias = original
alias.sumar_puntos(50)
print(original.puntos)
print(alias is original)

otro = ClienteFidelidad("C-88", 150)
print(otro is original)
print(otro.puntos == original.puntos)

aplicar_bono_bienvenida(otro)
print(otro.puntos)
print(original.puntos)
```

Salida esperada:

```
150
True
False
True
350
150
```

Ejemplo 7.2 · Procesar una lista de objetos

> 💡 Una empresa de paquetería necesita analizar los envíos del día. Crea una clase `Envio` y procesa una **lista de objetos** de esa clase para obtener distintos indicadores.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_7_2_envios.py`

```python
class Envio:
    def __init__(self, id_envio, destino, peso_kg, entregado):
        self.id_envio = id_envio
        self.destino = destino
        self.peso_kg = peso_kg
        self.entregado = entregado

    def __repr__(self):
        return f"Envio({self.id_envio},{self.destino},{self.peso_kg} kg)"

def obtener_peso(envio):
    return envio.peso_kg

envios = [
    Envio("E-01", "Madrid", 2.5, True),
    Envio("E-02", "Sevilla", 14.0, False),
    Envio("E-03", "Madrid", 0.8, True),
    Envio("E-04", "Bilbao", 7.2, False),
    Envio("E-05", "Sevilla", 3.1, True),
]

peso_total = sum([e.peso_kg for e in envios])
pendientes = [e.id_envio for e in envios if not e.entregado]

envios_por_destino = {}
for e in envios:
    envios_por_destino[e.destino] = envios_por_destino.get(e.destino, 0) + 1

mas_pesados = sorted(envios, key=obtener_peso, reverse=True)[:2]

print("Peso total:", round(peso_total, 2), "kg")
print("Pendientes:", pendientes)
print("Envíos por destino:", envios_por_destino)
print("Los 2 más pesados:", mas_pesados)
```

Salida esperada:

```
Peso total: 27.6 kg
Pendientes: ['E-02', 'E-04']
Envíos por destino: {'Madrid': 2, 'Sevilla': 2, 'Bilbao': 1}
Los 2 más pesados: [Envio(E-02, Sevilla, 14.0 kg), Envio(E-04, Bilbao, 7.2 kg)]
```

Ejemplo 7.3 · Composición: un pipeline formado por etapas

> 💡 El equipo de datos ejecuta cada noche un *pipeline* que procesa las ventas de las tiendas. Un *pipeline* está formado por varias **etapas** que se ejecutan en orden. Modela este sistema mediante **composición**: un objeto `Pipeline` que contiene objetos `Etapa` como parte de sus atributos.

> 📄 `clase_18_clases_objetos/teoria/ejemplo_7_3_pipeline_etapas.py`

```python
class Etapa:
    def __init__(self, nombre, duracion_seg):
```

```
        self.nombre = nombre
        self.duracion_seg = duracion_seg
        self.ejecutada = False

    def ejecutar(self):
        self.ejecutada = True
        return f"  OK{self.nombre} ({self.duracion_seg}s)"

class Pipeline:
    def __init__(self, nombre):
        self.nombre = nombre
        self.etapas = []

    def agregar_etapa(self, etapa):
        self.etapas.append(etapa)

    def duracion_total(self):
        return sum([etapa.duracion_seg for etapa in self.etapas])

    def ejecutar(self):
        print(f"Pipeline '{self.nombre}' ({len(self.etapas)} etapas)")
        for etapa in self.etapas:
            print(etapa.ejecutar())
        print(f"Duración total:{self.duracion_total()}s")

pipeline = Pipeline("ventas_nocturno")
pipeline.agregar_etapa(Etapa("extraer_ficheros_tiendas", 120))
pipeline.agregar_etapa(Etapa("validar_esquema", 35))
pipeline.agregar_etapa(Etapa("deduplicar_tickets", 48))
pipeline.agregar_etapa(Etapa("publicar_zona_curated", 60))

print([etapa.ejecutada for etapa in pipeline.etapas])
pipeline.ejecutar()
print([etapa.ejecutada for etapa in pipeline.etapas])
```

Salida esperada:

```
[False, False, False, False]
Pipeline 'ventas_nocturno' (4 etapas)
  OK  extraer_ficheros_tiendas (120s)
  OK  validar_esquema (35s)
  OK  deduplicar_tickets (48s)
  OK  publicar_zona_curated (60s)
Duración total: 263s
[True, True, True, True]
```

Clase 19 · OOP parte 2

Sesión 01: Detalles de clases y objetos

> En la Clase 18 se vio cómo definir clases, crear objetos, escribir métodos y representar objetos como texto. Esta clase profundiza en los **detalles** que distinguen un código orientado a objetos profesional:

- Cómo se nombran los **identificadores** y qué significa cada guion bajo.
- **Quién puede llamar a quién**: funciones globales, métodos de instancia y funciones definidas en la clase.
- Cómo enseñar a Python a **usar `+`, `<`, `==`, `in` o `len()` con nuestras propias clases**.
- Por qué en Python **todo es un objeto**.
- Cómo añadir y quitar atributos sobre la marcha.
- Cómo convertir objetos de nuestras clases en tipos nativos y viceversa.

Sección 1. Convenciones de nombres de los identificadores

Definición

> Un **identificador** es el nombre que damos a una variable, función, método, clase o atributo. Python no obliga a seguir un estilo concreto, pero la comunidad sigue unas **convenciones** que comunican la intención del programador. Al leer `self.__token` o `_reintentos`, cualquier desarrollador sabe, sin leer más, si ese atributo es para usarlo desde fuera o no.

Tipo de identificador	Convención	Ejemplos
Variables, funciones, métodos y atributos	`snake_case`, en minúsculas	`filas_cargadas`, `calcular_iva()`
Variable que se usa y se descarta	Un solo guion bajo: `_`	`for _ in range(3):`
Clases	`PascalCase`, primera letra en mayúscula	`TareaIngesta`, `ConexionProveedor`
Público	Empieza por letra	`endpoint`, `conectar()`
Uso interno («protegido»)	Empieza por **un** guion bajo	`_reintentos`, `_registrar_reintento()`
Privado	Empieza por **dos** guiones bajos	`__token`, `__token_valido()`
Nombres especiales del lenguaje	Empiezan **y** terminan por dos guiones bajos	`__init__()`, `__str__()`, `__add__()`

Puntos clave:

- **Público**: se puede usar desde dentro y desde fuera de la clase.
- **Uso interno o** «protegido»** (`_nombre`)**: es solo un aviso para otros programadores: «esto es un detalle interno, no lo uses desde fuera». Python** no lo impide**.
- **Privado (`__nombre`)**: Python aplica un mecanismo llamado **name mangling** (alteración del nombre). Dentro de la clase `ConexionProveedor`, el atributo `__token` se guarda realmente como `_ConexionProveedor__token`. Desde fuera, `objeto.__token` no existe y da `AttributeError`. No es una protección absoluta, pero evita usos accidentales.
- **Nombres especiales (`__nombre__`)**: los define el lenguaje. **No se llaman directamente**: Python los llama por nosotros. `__init__` se ejecuta al crear un objeto y `__str__` al hacer `print(objeto)`.

Analogía de la vida diaria

Sintaxis

```
class NombreClase:                      # PascalCase
    def __init__(self, dato, secreto):  # método especial: lo llama Python
        self.dato = dato                # público
        self._interno = 0               # uso interno (convención)
        self.__secreto = secreto        # privado: se guarda como _NombreClase__secreto

    def metodo_publico(self):
        ...

    def _metodo_interno(self):
        ...

    def __metodo_privado(self):           # se guarda como _NombreClase__metodo_privado
        ...
```

```python
for _ in range(3):                          # _ : variable que no se va a usar
    ...
```

Ejemplo 1.1 · Conexión a la API de un proveedor logístico

Una empresa de **Ingeniería de Datos** necesita conectarse a las APIs de diferentes proveedores logísticos para consultar información sobre envíos y entregas.

Para gestionar estas conexiones, se desarrolla una clase llamada `ConexionProveedor` que almacena el nombre del proveedor, la dirección de su API (*endpoint*) y un token de autenticación.

El objetivo es aplicar las **convenciones de nombres ,** diferenciando entre atributos y métodos públicos, de uso interno y privados.

La clase debe permitir:

- Consultar el nombre del proveedor y su endpoint.
- Almacenar el token como atributo privado (`__token`).
- Validar internamente que el token tenga al menos 12 caracteres.
- Registrar los intentos de conexión fallidos.
- Realizar una conexión simulada mediante el método público `conectar()` . > Se crearán dos proveedores, uno con un token válido y otro con un token incorrecto, para comprobar cómo se gestionan las conexiones y los reintentos.

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_1_1_convenciones_nombres.py`

```python
class ConexionProveedor:
    def __init__(self, proveedor, endpoint, token):
        self.proveedor = proveedor              # público
        self.endpoint = endpoint                # público
        self._reintentos = 0                    # uso interno (convención)
        self.__token = token                    # privado (name mangling)

    def __token_valido(self):                   # método privado
        return len(self.__token) >= 12

    def _registrar_reintento(self):             # método de uso interno
        self._reintentos += 1

    def conectar(self):                         # método público
        if not self.__token_valido():
            self._registrar_reintento()
            return f"{self.proveedor}: token no válido (reintentos:{self._reintentos})"
        return f"{self.proveedor}: conectado a{self.endpoint}"

aws = ConexionProveedor("logistica_sur", "https://api.logsur.com/v1", "tk_9f8e7d6c5b4a")
mal = ConexionProveedor("transportes_rio", "https://api.trio.es/v3", "tk_123")

print(aws.conectar())
print(mal.conectar())
print(mal.conectar())

for _ in range(3):
    print("-", end="")
print()
```

Salida esperada:

```
logistica_sur: conectado a https://api.logsur.com/v1
transportes_rio: token no válido (reintentos: 1)
transportes_rio: token no válido (reintentos: 2)

---
```

Highlights:

8. Desde fuera de la clase solo se usa el método público `conectar()`. Es la «puerta de entrada» del objeto.
9. Dentro de la clase, los métodos usan sin problema `self.__token` y `self.__token_valido()`: el doble guion bajo no limita el acceso **desde dentro** de la propia clase.
10. El token de `logistica_sur` tiene 15 caracteres y es válido. El de `transportes_rio` tiene 6, así que cada intento suma un reintento en `_reintentos`.
11. El bucle `for _ in range(3)` repite tres veces sin usar la variable de control. El nombre `_` comunica justo eso.

Ejemplo 1.2 · Name mangling: qué ocurre con `__token`

Una empresa de **Ingeniería de Datos** necesita proteger los tokens utilizados para conectarse a las APIs de sus proveedores logísticos, evitando que otros programadores accedan a ellos accidentalmente.

Para ello, se utiliza una clase llamada `ConexionProveedor`, que almacena el token como atributo privado mediante el prefijo de doble guion bajo (`__token`).

El objetivo es comprender cómo funciona el mecanismo de **name mangling** en Python y qué diferencias existen entre los atributos públicos, de uso interno y privados.

El ejemplo debe permitir:

- Inspeccionar los atributos reales del objeto mediante `vars()`.
- Comprobar que los atributos con un guion bajo (`_reintentos`) pueden consultarse directamente.
- Mostrar una versión enmascarada del token mediante un método público, dejando visibles únicamente sus tres primeros caracteres.
- Acceder al token utilizando el nombre generado por *name mangling* (`_ConexionProveedor__token`).
- Comprobar qué ocurre al intentar acceder directamente a `__token` desde fuera de la clase.

> Se creará una conexión con el proveedor `logistica_sur` y se realizarán diferentes consultas para observar cómo Python almacena y permite acceder a sus atributos.

Pregunta clave: ¿El doble guion bajo (`__`) hace que un atributo sea realmente inaccesible desde fuera de la clase, o simplemente modifica su nombre para evitar accesos accidentales?

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_1_2_name_mangling.py`

```python
class ConexionProveedor:
    def __init__(self, proveedor, token):
        self.proveedor = proveedor
        self._reintentos = 0
        self.__token = token

    def token_enmascarado(self):
        return self.__token[:3] + "*" * (len(self.__token) - 3)

c = ConexionProveedor("logistica_sur", "tk_9f8e7d6c5b4a")

print(vars(c))
print(c._reintentos)
print(c.token_enmascarado())
print(c._ConexionProveedor__token)
print(c.__token)
```

Salida esperada:

```
{'proveedor': 'logistica_sur', '_reintentos': 0, '_ConexionProveedor__token': 'tk_9f8e7d6c5b4a'}
0
tk_
tk_9f8e7d6c5b4a
Traceback (most recent call last):
  ...
AttributeError: 'ConexionProveedor' object has no attribute '__token'
```

Highlights:

12. `vars(c)` revela cómo se guardan realmente los atributos: `__token` aparece como `_ConexionProveedor__token`.
13. `c._reintentos` funciona. El guion bajo simple es solo una convención y Python no lo bloquea.
14. `token_enmascarado()` es la forma correcta de ofrecer el dato hacia fuera: el método público decide qué se muestra.
15. `c._ConexionProveedor__token` funciona porque usa el nombre real. Es posible, pero es **mala práctica**: se salta la intención de quien diseñó la clase.
16. `c.__token` falla: fuera de la clase ese nombre no existe. El programa se detiene con un `AttributeError`.

> En Data Engineering, las credenciales, cadenas de conexión y contadores internos de una clase se declaran con `_` o `__`. Lo que otros módulos del pipeline deben usar se expone con métodos públicos.

Sección 2. Funciones globales, métodos de instancia y funciones de la clase

Definición

En un programa con clases conviven tres tipos de funciones:

Tipo	Dónde se define	Recibe `self`	Cómo se llama
Función global	Fuera de cualquier clase	No	`funcion()`
Método de instancia	Dentro de la clase	Sí	`objeto.metodo()`
Función de la clase (método de clase)	Dentro de la clase, **sin** `self`	No	`Clase.funcion()`

> Una función de la clase no necesita los datos de ningún objeto: sirve para operaciones que pertenecen al concepto de la clase en general (su versión, los formatos que admite, una constante calculada...).

Reglas de llamada:

- Una **función global** puede llamar a métodos de instancia (a través de un objeto) y a funciones de la clase (a través del nombre de la clase). El siguiente ejemplo muestra cómo una función global puede llamar a métodos de una clase y cómo los métodos pueden llamar a otras funciones o métodos.
- Un **método** puede llamar a funciones globales directamente por su nombre.
- Dentro de una clase, un método **no** puede llamar a otro método solo por su nombre: necesita `self.metodo()` o `Clase.funcion()`. Si se escribe solo `metodo()`, Python lo busca como función global y da `NameError`.
- Una función de la clase **sin** `self` no se puede llamar desde un objeto: `objeto.funcion()` le pasa el objeto como primer argumento y da `TypeError`.

El siguiente ejemplo muestra cómo una función global puede llamar a métodos de una clase y cómo los métodos pueden llamar a otras funciones o métodos.

```python
# Función global
def saludar():
    print("Hola desde la función global")


class Ejemplo:

    # Método de instancia
    def metodo1(self):
        print("Ejecutando método 1")
        saludar()                  # Llama a una función global
        self.metodo2()            # Llama a otro método de instancia

    # Método de instancia
    def metodo2(self):
        print("Ejecutando método 2")

    # Función definida en la clase (sin self)
    def funcion():
        print("Función de la clase")


# Función global
def ejecutar():
    obj = Ejemplo()

    obj.metodo1()                 # Correcto: método de instancia
    Ejemplo.funcion()            # Correcto: función de la clase


ejecutar()
```

Salida esperada:

```
Ejecutando método 1
Hola desde la función global
Ejecutando método 2
Función de la clase
```

Resumen de las reglas

Desde	Llamada	Resultado
Función global	`obj.metodo1()`	Correcto
Función global	`Ejemplo.funcion()`	Correcto
Método de instancia	`saludar()`	Correcto
Método de instancia	`self.metodo2()`	Correcto
Método de instancia	`metodo2()`	`NameError`
Código externo	`obj.funcion()`	`TypeError`

```python
obj = Ejemplo()

# ERROR 1: llamar a otro método sin self
# Dentro de metodo1(), escribir:
# metodo2()                → NameError

# ERROR 2: llamar desde un objeto a una función sin self
obj.funcion()               # TypeError
```

> **Nota importante:** En Python moderno, si queremos definir una función asociada a una clase que no necesita `self`, lo recomendable es utilizar el decorador `@staticmethod`. Así podremos llamarla tanto con `Ejemplo.funcion()` como con `obj.funcion()` sin provocar un error.

Sintaxis

```python
def funcion_global():
    ...

class NombreClase:
    def metodo_instancia(self, dato):
        funcion_global()              # un método puede usar funciones globales
        self.otro_metodo()            # otro método del mismo objeto: con self.

    def funcion_de_clase():           # sin self
        ...

objeto = NombreClase()
objeto.metodo_instancia(1)            # correcto
NombreClase.funcion_de_clase()        # correcto
objeto.funcion_de_clase()             # TypeError
```

Ejemplo 2.1 · Sistema de alertas de un pipeline

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_2_1_funciones_y_metodos.py`

Una empresa de **Ingeniería de Datos** necesita desarrollar un sistema de notificaciones para informar sobre incidencias que se producen durante la ejecución de sus pipelines de datos, como retrasos en las cargas o problemas de almacenamiento.

Para ello, se utiliza una clase llamada `Notificador`, encargada de enviar mensajes de alerta a un canal de comunicación, como Slack o Teams.

El objetivo es comprender **cómo interactúan las funciones globales con los métodos de instancia y las funciones definidas dentro de una clase**.

El sistema debe permitir:

- Mostrar una cabecera común mediante la función global `cabecera()`.
- Crear notificadores asociados a diferentes canales de comunicación.
- Enviar mensajes mediante el método de instancia `enviar()`, utilizando `self` para acceder al canal.
- Mostrar la versión del sistema mediante la función `version()`, definida dentro de la clase sin `self`.
- Procesar una lista de alertas mediante la función global `procesar_alertas()`, que utiliza las funciones y métodos de la clase.

> Se creará un notificador asociado al canal `slack#datos` para comunicar dos incidencias: un retraso en la carga de ventas y un disco que ha alcanzado el 91 % de ocupación.

Pregunta clave: ¿Cómo puede una función global llamar a los métodos de un objeto y a las funciones de una clase? ¿Y cómo puede un método de instancia llamar a una función global?

```python
def cabecera():                                 # función global
    print("=== Sistema de alertas ===")

class Notificador:
    def __init__(self, canal):
        self.canal = canal

    def enviar(self, mensaje):                  # método de instancia: recibe self
        cabecera()                              # llama a una función global
        print(f"[{self.canal}]{mensaje}")

    def version():                              # función de la clase: sin self
        cabecera()
```

```
        print("Notificador v2.4")

def procesar_alertas(notificador, alertas):       # función global
    Notificador.version()                         # llama a la función de la clase
    for alerta in alertas:
        notificador.enviar(alerta)                # llama al método de instancia

slack = Notificador("slack#datos")
procesar_alertas(slack, ["Carga de ventas retrasada", "Disco al 91 %"])
```

Salida esperada:

```
=== Sistema de alertas ===
Notificador v2.4
=== Sistema de alertas ===
[slack#datos] Carga de ventas retrasada
=== Sistema de alertas ===
[slack#datos] Disco al 91 %
```

Highlights:

17. `procesar_alertas` es una función global que recibe un objeto `Notificador` y una lista de textos.
18. Primero llama a `Notificador.version()` usando el nombre de la clase, porque `version` no tiene `self`.
19. Después recorre las alertas y llama a `notificador.enviar(alerta)`: Python pasa `slack` como `self`.
20. Tanto `version()` como `enviar()` llaman a la función global `cabecera()` directamente por su nombre.

Ejemplo 2.2 · Llamar a una función de la clase desde un objeto

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_2_2_error_llamada.py`

Una empresa de **Ingeniería de Datos** utiliza una clase llamada `Notificador` para gestionar las alertas de sus pipelines.

La clase contiene una función `version()` que devuelve la versión actual del sistema de notificaciones.

Un programador necesita consultar esta información y prueba dos formas diferentes de llamar a la función:

- **Desde la clase:** utilizando `Notificador.version()`.
- **Desde un objeto:** creando un notificador para el canal `email` y utilizando `n.version()`.

Aunque ambas llamadas utilizan la misma función, **la primera funciona correctamente y la segunda produce un error**.

El objetivo del ejemplo es comprender cómo Python realiza las llamadas a funciones definidas dentro de una clase y por qué, cuando se invocan desde un objeto, Python pasa automáticamente la instancia como primer argumento.

Pregunta clave: ¿Por qué se produce un `TypeError` al ejecutar `n.version()`? ¿Qué relación tiene este error con el parámetro `self` de los métodos de instancia?

```
class Notificador:
    def __init__(self, canal):
        self.canal = canal

    def version():
        return "Notificador v2.4"

n = Notificador("email")
print(Notificador.version())
print(n.version())
```

Salida esperada:

```
Notificador v2.4
Traceback (most recent call last):
  ...
TypeError: Notificador.version() takes 0 positional arguments but 1 was given
```

Highlights:

21. `Notificador.version()` funciona: se llama a través de la clase y no recibe ningún argumento.
22. `n.version()` se traduce internamente a `Notificador.version(n)`. Python le pasa el objeto `n` como primer argumento, pero `version` no tiene ningún parámetro que lo reciba.
23. El mensaje del error lo explica: la función admite 0 argumentos posicionales y ha recibido 1.

Errores típicos de llamada:

Código	Error	Causa	Corrección
`objeto.funcion_sin_self()`	`TypeError: ... takes 0 positional arguments but 1 was given`	El objeto se pasa como argumento que nadie recibe	`Clase.funcion_sin_self()`
Dentro de un método: `otro_metodo()`	`NameError: name 'otro_metodo' is not defined`	Python lo busca como función global	`self.otro_metodo()`
`Clase.metodo_con_self()`	`TypeError: ... missing 1 required positional argument: 'self'`	Falta el objeto sobre el que trabajar	`objeto.metodo_con_self()`

Sección 3. Sobrecarga de operadores

Definición

Python sabe sumar dos `int` o concatenar dos `str`, pero no sabe qué significa sumar dos objetos de una clase nuestra. **Sobrecargar un operador** es definir en la clase el método especial que Python ejecuta cuando encuentra ese operador. Así, `resumen_lunes + resumen_martes` se convierte internamente en `resumen_lunes.__add__(resumen_martes)`.

> La sobrecarga hace que el código de negocio se lea de forma natural (`total = lunes + martes`, `if version_a < version_b`, `if "A-101" in lote`) y permite usar nuestras clases con funciones nativas como `sorted()`, `max()`, `min()` o `len()`.

Supongamos que `resumen_lunes` y `resumen_martes` son objetos de una clase `ResumenVentas` que implementa los métodos correspondientes.

Operador	Método especial	Ejemplo con objetos
`+`	`__add__(self, other)`	`resumen_lunes.__add__(resumen_martes)`
`-`	`__sub__(self, other)`	`resumen_lunes.__sub__(resumen_martes)`
`*`	`__mul__(self, other)`	`resumen_lunes.__mul__(resumen_martes)`
`/`	`__truediv__(self, other)`	`resumen_lunes.__truediv__(resumen_martes)`
`//`	`__floordiv__(self, other)`	`resumen_lunes.__floordiv__(resumen_martes)`
`%`	`__mod__(self, other)`	`resumen_lunes.__mod__(resumen_martes)`
`**`	`__pow__(self, other)`	`resumen_lunes.__pow__(resumen_martes)`

Supongamos que tenemos dos objetos de la clase `VersionPipeline`, llamados `version_a` y `version_b`.

Operador	Método especial	Ejemplo con objetos
`<`	`__lt__(self, other)`	`version_a.__lt__(version_b)`
`>`	`__gt__(self, other)`	`version_a.__gt__(version_b)`
`<=`	`__le__(self, other)`	`version_a.__le__(version_b)`
`>=`	`__ge__(self, other)`	`version_a.__ge__(version_b)`
`==`	`__eq__(self, other)`	`version_a.__eq__(version_b)`
`!=`	`__ne__(self, other)`	`version_a.__ne__(version_b)`

Supongamos que tenemos dos objetos de la clase `ResumenVentas`, llamados `resumen_total` y `resumen_martes`.

Operador	Método especial	Ejemplo con objetos
`+=`	`__iadd__(self, other)`	`resumen_total.__iadd__(resumen_martes)`
`-=`	`__isub__(self, other)`	`resumen_total.__isub__(resumen_martes)`
`*=`	`__imul__(self, other)`	`resumen_total.__imul__(resumen_martes)`
`/=`	`__itruediv__(self, other)`	`resumen_total.__itruediv__(resumen_martes)`
`//=`	`__ifloordiv__(self, other)`	`resumen_total.__ifloordiv__(resumen_martes)`
`%=`	`__imod__(self, other)`	`resumen_total.__imod__(resumen_martes)`
`**=`	`__ipow__(self, other)`	`resumen_total.__ipow__(resumen_martes)`

> Una operación como `resumen_total += resumen_martes` utiliza `__iadd__()` cuando está disponible y asigna a `resumen_total` el valor devuelto por el método. Si `__iadd__()` no está implementado, Python puede recurrir a `__add__()` y sus mecanismos alternativos.

Supongamos que `lote` es un objeto de la clase `LoteDatos` y `registro` es otro objeto que representa un registro.

Expresión	Método especial	Ejemplo con objetos
`registro in lote`	`__contains__(self, x)`	`lote.__contains__(registro)`
`len(lote)`	`__len__(self)`	`lote.__len__()`

Enunciado

Una empresa de **Ingeniería de Datos** necesita combinar los resúmenes de ventas generados diariamente por sus pipelines.

Cada resumen se representa mediante un objeto de la clase `ResumenVentas`, que almacena el importe total de ventas de un día.

El objetivo es implementar el método especial `__add__()` para permitir sumar dos objetos `ResumenVentas` utilizando directamente el operador `+`.

```python
class ResumenVentas:
    def __init__(self, total):
        self.total = total

    def __add__(self, other):
        return ResumenVentas(self.total + other.total)


# Crear dos objetos
resumen_lunes = ResumenVentas(100)
resumen_martes = ResumenVentas(200)

# Forma habitual: utilizando el operador +
resultado1 = resumen_lunes + resumen_martes
```

```python
# Forma explícita: llamando al método especial
resultado2 = resumen_lunes.__add__(resumen_martes)

# Mostrar resultados
print(resultado1.total)
print(resultado2.total)
```

Salida esperada

```
300
300
```

Highlights:

24. La clase `ResumenVentas` dispone de un atributo `total` que almacena el importe de las ventas.
25. Se crean dos objetos: `resumen_lunes` con 100 y `resumen_martes` con 200.
26. El método especial `__add__()` define cómo se suman dos objetos de la clase.
27. Cuando Python encuentra `resumen_lunes + resumen_martes`, llama al método `__add__()` del primer objeto, pasando el segundo como argumento.
28. Dentro del método se suman `self.total` y `other.total`.
29. El método devuelve un nuevo objeto `ResumenVentas` cuyo atributo `total` contiene 300.

Cuando escribimos:

```python
resultado = resumen_lunes + resumen_martes
```

Python ejecuta conceptualmente:

```python
resultado = resumen_lunes.__add__(resumen_martes)
```

Dentro del método:

```python
def __add__(self, other):
    return ResumenVentas(self.total + other.total)
```

- `self` hace referencia al objeto `resumen_lunes`.
- `other` hace referencia al objeto `resumen_martes`.
- `self.total` contiene `100`.
- `other.total` contiene `200`.
- Se crea un nuevo objeto `ResumenVentas(300)`.

Operador	Método especial	Expresión habitual	Llamada equivalente
`+`	`__add__()`	`lunes + martes`	`lunes.__add__(martes)`
`<`	`__lt__()`	`version_a < version_b`	`version_a.__lt__(version_b)`
`+=`	`__iadd__()`	`total += martes`	`total = total.__iadd__(martes)`
`in`	`__contains__()`	`registro in lote`	`lote.__contains__(registro)`
`len()`	`__len__()`	`len(lote)`	`lote.__len__()`

> **Nota:** estas equivalencias representan el funcionamiento básico de los métodos especiales cuando están implementados. Python puede utilizar otros mecanismos, como métodos reflejados, cuando corresponde.

Idea clave

> **La sobrecarga de operadores permite definir cómo se comportan los operadores de Python cuando trabajan con objetos de nuestras propias clases.**

En lugar de escribir:

```
resultado = resumen_lunes.sumar(resumen_martes)
```

Podemos escribir:

```
resultado = resumen_lunes + resumen_martes
```

Esto hace que nuestro código sea más natural, expresivo y fácil de leer.

Detalles importantes:

- Los métodos aritméticos **devuelven un objeto nuevo** y no modifican los operandos.
- Los métodos de asignación compuesta (`__iadd__`, `__isub__`...) **modifican el propio objeto** y deben terminar con `return self`. Si no se definen, `a += b` usa `__add__` y crea un objeto nuevo.
- Si se define `__eq__`, Python obtiene `!=` automáticamente negando su resultado.
- `sorted()`, `min()` y `max()` solo necesitan que la clase sepa comparar con `<` (`__lt__`). **¿Qué operadores conviene sobrecargar?** Solo aquellos cuyo significado sea **evidente** para el negocio. Sumar dos resúmenes de carga o comparar dos versiones de un esquema es natural. Multiplicar dos clientes no lo es.

Sintaxis

```
class NombreClase:
    def __add__(self, otro):            # objeto + otro
        return NombreClase(...)         # devuelve un objeto NUEVO

    def __lt__(self, otro):             # objeto < otro
        return True o False

    def __iadd__(self, otro):           # objeto += otro
        ...                             # modifica self
        return self

    def __contains__(self, elemento):   # elemento in objeto
        return True o False

    def __len__(self):                  # len(objeto)
        return numero_entero
```

Ejemplo 3.1 · Sumar y comparar resúmenes de carga

Enunciado

Una empresa de **Ingeniería de Datos** ejecuta cada noche un pipeline que procesa las ventas del día y las almacena en un Data Warehouse.

Al finalizar cada ejecución, el pipeline genera un resumen con tres indicadores:

- **Filas:** número de registros procesados.
- **Rechazadas:** número de registros que no pudieron cargarse por errores de calidad.
- **Megas:** volumen de datos procesados en MB. El equipo necesita analizar los resultados de diferentes días, calcular diferencias entre ejecuciones y estimar el volumen de datos que podría procesarse durante un mes.

Para ello, se desarrolla una clase llamada `ResumenCarga` que utiliza la **sobrecarga de operadores** para realizar operaciones directamente entre objetos.

La clase debe permitir:

30. **Sumar dos resúmenes (`__add__`):** combinar los resultados de dos días, acumulando las filas, los rechazos y los megabytes procesados.
31. **Restar dos resúmenes (`__sub__`):** calcular las diferencias entre las ejecuciones de dos días.
32. **Multiplicar un resumen (`__mul__`):** estimar los resultados de varios días a partir de un resumen diario y un factor numérico.
33. **Mostrar un resumen (`__str__`):** presentar los resultados de forma legible cuando se utiliza `print()`.

Se dispone de los resultados de dos ejecuciones del pipeline:

Indicador	Lunes	Martes
Filas procesadas	120.000	98.500
Filas rechazadas	35	12
Volumen (MB)	48,60	40,15

Utilizando los objetos `lunes` y `martes`, se realizarán las siguientes operaciones:

- `lunes + martes`: obtener el total acumulado de ambos días.
- `lunes - martes`: conocer las diferencias entre las dos ejecuciones.
- `martes * 30`: proyectar los resultados de 30 días suponiendo que diariamente se procesa la misma cantidad de datos que el martes.
- `lunes.__add__(martes)`: comprobar que llamar directamente al método especial produce el mismo resultado que utilizar el operador `+`.

Pregunta clave: ¿Cómo consigue Python sumar, restar y multiplicar objetos de una clase creada por nosotros? ¿Qué papel desempeñan los métodos especiales `__add__()`, `__sub__()` y `__mul__()`?

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_3_1_resumen_carga.py`

```python
class ResumenCarga:
    def __init__(self, filas=0, rechazadas=0, megas=0.0):
        self.filas = filas
        self.rechazadas = rechazadas
        self.megas = megas

    def __add__(self, otro):
        return ResumenCarga(self.filas + otro.filas,
                            self.rechazadas + otro.rechazadas,
                            round(self.megas + otro.megas, 2))

    def __sub__(self, otro):
        return ResumenCarga(self.filas - otro.filas,
                            self.rechazadas - otro.rechazadas,
                            round(self.megas - otro.megas, 2))

    def __mul__(self, factor):
        return ResumenCarga(self.filas * factor,
                            self.rechazadas * factor,
                            round(self.megas * factor, 2))

    def __str__(self):
        return f"filas={self.filas:,} rechazadas={self.rechazadas} megas={self.megas}"

lunes = ResumenCarga(120_000, 35, 48.6)
martes = ResumenCarga(98_500, 12, 40.15)

semana = lunes + martes
diferencia = lunes - martes
```

```
proyeccion_mes = martes * 30

print("Total     :", semana)
print("Diferencia:", diferencia)
print("Proyección:", proyeccion_mes)
print("Equivale a:", lunes.__add__(martes))
```

Salida esperada:

```
Total     : filas=218,500 rechazadas=47 megas=88.75
Diferencia: filas=21,500 rechazadas=23 megas=8.45
Proyección: filas=2,955,000 rechazadas=360 megas=1204.5
Equivale a: filas=218,500 rechazadas=47 megas=88.75
```

Highlights:

34. `lunes + martes` ejecuta `lunes.__add__(martes)`: `self` es `lunes` y `otro` es `martes`. Devuelve un `ResumenCarga` **nuevo**; `lunes` y `martes` no cambian.
35. `lunes - martes` ejecuta `__sub__` de la misma forma.
36. `martes * 30` ejecuta `martes.__mul__(30)`. El segundo operando no tiene por qué ser de la misma clase: aquí es un número.
37. La última línea demuestra que el operador `+` y la llamada explícita a `__add__` son equivalentes. En la práctica siempre se usa el operador.
38. `{self.filas:,}` muestra el número con separador de miles.

Ejemplo 3.2 · Comparar versiones de un esquema de datos

Una empresa de **Ingeniería de Datos** utiliza diferentes aplicaciones para producir y consumir eventos de datos. Para garantizar que estas aplicaciones puedan intercambiar información correctamente, cada esquema de datos tiene asignado un número de versión.

Las versiones siguen el formato **mayor.menor.parche**, por ejemplo, `2.10.0`, donde:

- **Mayor:** indica cambios importantes en la estructura del esquema.
- **Menor:** indica cambios menores, como la incorporación de nuevos campos.
- **Parche:** indica pequeñas correcciones o ajustes.

El equipo necesita comparar las versiones utilizadas por los productores y consumidores de datos para identificar cuáles son más recientes, cuáles son anteriores y cuáles son iguales.

Para ello, se desarrolla una clase llamada `VersionEsquema` que utiliza la **sobrecarga de operadores de comparación**.

La clase debe permitir:

39. **Comparar igualdad (`__eq__`):** determinar si dos objetos representan la misma versión mediante `==`.
40. **Comparar versiones anteriores (`__lt__`):** identificar si una versión es menor que otra mediante `<`.
41. **Comparar versiones posteriores (`__gt__`):** identificar si una versión es mayor que otra mediante `>`.
42. **Representar versiones (`__repr__`):** mostrar los objetos con un formato legible, como `v2.10.0`.

Se dispone de las siguientes versiones:

Objeto	Versión	Descripción
`productor`	`2.10.0`	Versión utilizada por el productor de datos
`consumidor`	`2.9.3`	Versión utilizada por el consumidor
`copia`	`2.10.0`	Otro objeto con la misma versión del productor

El programa debe realizar las siguientes comprobaciones:

- Comparar `productor == copia` y `productor is copia` para distinguir entre **igualdad de valores e identidad de objetos**.
- Comprobar si las versiones del productor y del consumidor son diferentes mediante `!=`.
- Determinar qué versión es anterior o posterior utilizando `<` y `>`.
- Ordenar una lista de versiones mediante `sorted()`.
- Identificar la versión más reciente utilizando `max()`.

Pregunta clave: ¿Cómo podemos conseguir que Python compare y ordene correctamente objetos que representan versiones de esquemas de datos, utilizando operadores como `==`, `<` y `>`?

Importante: las versiones deben compararse utilizando sus componentes numéricos y no como cadenas de texto. Además, que una versión sea más reciente no garantiza por sí solo que sea compatible con otra aplicación.

Un productor de datos publica eventos con un esquema versionado (`mayor.menor.parche`). Los consumidores deben saber si su versión es anterior o posterior a la del productor.

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_3_2_version_esquema.py`

```python
class VersionEsquema:
    def __init__(self, mayor, menor, parche):
        self.mayor = mayor
        self.menor = menor
        self.parche = parche

    def __eq__(self, otra):
        return (self.mayor, self.menor, self.parche) == (otra.mayor, otra.menor, otra.parche)

    def __lt__(self, otra):
        return (self.mayor, self.menor, self.parche) < (otra.mayor, otra.menor, otra.parche)

    def __gt__(self, otra):
        return (self.mayor, self.menor, self.parche) > (otra.mayor, otra.menor, otra.parche)

    def __repr__(self):
        return f"v{self.mayor}.{self.menor}.{self.parche}"

productor = VersionEsquema(2, 10, 0)
consumidor = VersionEsquema(2, 9, 3)
copia = VersionEsquema(2, 10, 0)

print(productor == copia)
print(productor is copia)
print(productor != consumidor)
print(consumidor < productor)
print(productor > consumidor)

historico = [VersionEsquema(1, 4, 2), VersionEsquema(2, 10, 0), VersionEsquema(2, 9, 3), VersionEsquema(1,
12, 0)]
print(sorted(historico))
print(max(historico))
```

Salida esperada:

```
True
False
True
True
True
[v1.4.2, v1.12.0, v2.9.3, v2.10.0]
v2.10.0
```

Highlights:

43. Cada comparación construye una tupla `(mayor, menor, parche)` y compara tuplas. Python compara las tuplas elemento a elemento, que es exactamente la regla de las versiones.

44. `productor == copia` es `True` gracias a `__eq__`: tienen los mismos números. Sin embargo, `productor is copia` es `False`: son objetos distintos.
45. `!=` funciona aunque no se haya definido `__ne__`: Python niega el resultado de `__eq__`.
46. `sorted()` usa `__lt__` y `max()` usa `__gt__` para ordenar y elegir.
47. Las versiones se comparan como **números**. Si se compararan como texto, `"1.12.0" < "1.4.2"` sería `True`, porque el carácter `"1"` va antes que `"4"`. Con la clase, `v1.4.2` queda correctamente antes que `v1.12.0`.

Ejemplo 3.3 · Asignación compuesta, `in` y `len()` en un lote de tickets

Una empresa de **Ingeniería de Datos** recibe diariamente miles de tickets de compra procedentes de diferentes tiendas. Antes de enviarlos al sistema central, cada tienda agrupa sus tickets en lotes para facilitar su procesamiento.

Cada ticket contiene un **identificador único (`id`)** y el **importe de la compra (`importe`)**.

Para gestionar estos datos, se desarrolla una clase llamada `LoteTickets` que almacena los tickets de cada tienda en una lista privada y utiliza la **sobrecarga de operadores** para añadir, contar y buscar tickets de forma sencilla.

La clase debe permitir:

48. **Añadir tickets (`__iadd__`):** incorporar nuevos tickets al lote utilizando el operador `+=`, sin crear un nuevo objeto.
49. **Contar tickets (`__len__`):** obtener el número de tickets almacenados utilizando `len()`.
50. **Buscar tickets (`__contains__`):** comprobar si existe un ticket con un identificador determinado mediante `in` o `not in`.
51. **Mostrar información (`__str__`):** presentar el nombre de la tienda y la cantidad de tickets del lote al utilizar `print()`.

Se creará un lote de tickets para la tienda `T-07`, inicialmente vacío, al que se añadirán las siguientes compras:

ID del ticket	Importe (€)
A-100	23,50
A-101	8,75
A-102	112,00

Una vez añadidos los tickets, el programa debe:

- Mostrar el estado del lote antes y después de incorporar las compras.
- Consultar cuántos tickets contiene mediante `len(lote)`.
- Comprobar si el ticket `A-101` existe utilizando `in`.
- Comprobar si el ticket `Z-999` existe utilizando `in` y `not in`.

Pregunta clave: ¿Cómo podemos utilizar operadores como `+=`, `in` y funciones como `len()` con objetos de nuestras propias clases, de manera similar a como trabajamos con las listas de Python?

Cada tienda envía sus tickets al sistema central agrupados en lotes.

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_3_3_lote_operadores.py`

```python
class LoteTickets:
    def __init__(self, tienda):
        self.tienda = tienda
        self.__tickets = []

    def __iadd__(self, ticket):
        self.__tickets.append(ticket)
```

```
        return self

    def __len__(self):
        return len(self.__tickets)

    def __contains__(self, id_ticket):
        return id_ticket in [t["id"] for t in self.__tickets]

    def __str__(self):
        return f"Lote{self.tienda}:{len(self)} tickets"

lote = LoteTickets("T-07")
print(lote)

lote += {"id": "A-100", "importe": 23.5}
lote += {"id": "A-101", "importe": 8.75}
lote += {"id": "A-102", "importe": 112.0}

print(lote)
print(len(lote))
print("A-101" in lote)
print("Z-999" in lote)
print("Z-999" not in lote)
```

Salida esperada:

```
Lote T-07: 0 tickets
Lote T-07: 3 tickets
3
True
False
True
```

Highlights:

52. `lote += {...}` ejecuta `lote = lote.__iadd__({...})`. El método añade el ticket a la lista privada y devuelve `self`, de modo que `lote` sigue apuntando al mismo objeto.
53. Si `__iadd__` olvidara el `return self`, devolvería `None` y la variable `lote` pasaría a valer `None`.
54. `len(lote)` ejecuta `lote.__len__()`. El propio `__str__` lo reutiliza con `len(self)`.
55. `"A-101" in lote` ejecuta `lote.__contains__("A-101")`. `not in` funciona automáticamente negando el resultado.
56. La lista `__tickets` es privada: desde fuera solo se puede añadir con `+=`, preguntar con `in` y contar con `len()`.

Ejemplo 3.4 · Operadores ya sobrecargados en los tipos nativos

En Python, los tipos de datos nativos como `int`, `str`, `list` y `tuple` también son clases que tienen implementados sus propios métodos especiales.

Esto explica por qué **un mismo operador puede comportarse de manera diferente dependiendo del tipo de objeto con el que trabaje**.

Por ejemplo, el operador `+` permite sumar números, concatenar cadenas de texto o unir listas. Sin embargo, no todos los operadores están disponibles para todos los tipos de datos.

El objetivo de este ejemplo es comprender cómo Python utiliza la **sobrecarga de operadores en sus clases nativas**, sin necesidad de que nosotros implementemos los métodos especiales.

El programa debe permitir:

57. **Operador de suma (`__add__`):** comprobar cómo `+` realiza diferentes operaciones con números, cadenas, listas y tuplas.
58. **Operador de multiplicación (`__mul__`):** observar cómo permite repetir cadenas de texto y listas.
59. **Llamadas explícitas:** utilizar directamente métodos como `int.__add__()`, `str.__add__()` y `list.__mul__()` para comprobar su relación con los operadores.

60. **Operaciones no permitidas:** intentar restar dos cadenas de texto y analizar el `TypeError` que produce Python.

Se realizarán operaciones con diferentes tipos de datos, utilizando ejemplos relacionados con números, nombres de recursos de datos, listas y fechas.

Posteriormente, se repetirán algunas operaciones llamando directamente a sus métodos especiales para comprobar que producen los mismos resultados.

Finalmente, se intentará ejecutar `"ventas" - "s"` para observar qué sucede cuando utilizamos un operador que no está implementado para un determinado tipo de dato.

Pregunta clave: ¿Por qué el operador `+` suma números, concatena cadenas y une listas, mientras que el operador `-` funciona con números, pero produce un error cuando intentamos restar dos cadenas de texto?

Los tipos nativos de Python también son clases con métodos especiales. Por eso el mismo `+` suma números, concatena textos y une listas.

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_3_4_operadores_nativos.py`

```
print(10 + 20)
print("data" + "_" + "lake")
print([1, 2] + [3, 4])
print((2026, 10) + (29,))

print("-" * 20)
print([0] * 5)

print(int.__add__(10, 20))
print(str.__add__("data", "_lake"))
print(list.__mul__([0], 5))

print("ventas" - "s")
```

Salida esperada:

```
30
data_lake
[1, 2, 3, 4]
(2026, 10, 29)
--------------------
[0, 0, 0, 0, 0]
30
data_lake
[0, 0, 0, 0, 0]
Traceback (most recent call last):
  ...
TypeError: unsupported operand type(s) for -: 'str' and 'str'
```

Highlights:

61. `+` está sobrecargado en `int`, `str`, `list` y `tuple`, con un significado distinto en cada clase: sumar, concatenar o unir.
62. está sobrecargado en `str` y `list` para repetir su contenido.
63. Las tres llamadas explícitas (`int.__add__`, `str.__add__`, `list.__mul__`) demuestran qué método se ejecuta detrás de cada operador.
64. La clase `str` **no** define `__sub__`. Restar dos textos no tiene un significado evidente, así que Python da `TypeError`.

Sección 4. Todo es un objeto

Definición

En Python, **todo lo que se puede asignar a una variable es un objeto**: números, textos, listas, diccionarios, `None`, funciones, clases e incluso módulos importados. Cada objeto tiene:

- Un **tipo** (la clase a la que pertenece), que se obtiene con `type(objeto)`.
- Una **identidad**, un número único mientras el objeto existe, que se obtiene con `id(objeto)`. En CPython corresponde a su dirección de memoria.
- Un conjunto de **atributos y métodos**, que se listan con `dir(objeto)`.

Una variable no guarda el objeto, sino una **referencia** a él. Esto tiene consecuencias distintas según el tipo:

	Inmutables	Mutables
Tipos	`int`, `float`, `complex`, `bool`, `str`, `tuple`	`list`, `dict`, `set`, objetos de nuestras clases
¿Se pueden cambiar?	No. «Cambiar» el valor crea un objeto nuevo	Sí, el mismo objeto se modifica
Si dos variables apuntan al mismo objeto	Reasignar una no afecta a la otra	Modificar desde una se ve desde la otra

---

Analogía de la vida diaria

Sintaxis

```
type(objeto)              # clase del objeto
type(objeto).__name__     # nombre de la clase como texto
id(objeto)                # identidad única del objeto
dir(objeto)               # lista de atributos y métodos
a is b                    # True si id(a) == id(b)
globals()                 # diccionario con los nombres globales del programa
```

Ejemplo 4.1 · El tipo de cada cosa

Una empresa de **Ingeniería de Datos** desarrolla una aplicación para gestionar información de sus proveedores logísticos. Durante el procesamiento de los datos, el programa necesita trabajar con diferentes elementos: números, textos, listas, diccionarios, funciones, clases y módulos.

Aunque estos elementos parecen muy diferentes, en Python **todos son objetos y cada uno pertenece a una clase determinada**.

El objetivo de este ejemplo es comprender el principio **«Todo es un objeto en Python»** y aprender a utilizar la función `type()` para identificar a qué clase pertenece cada elemento.

El programa debe permitir:

65. **Definir una clase** `Proveedor` y crear un objeto a partir de ella.
66. **Crear una función** `validar_fichero()` que compruebe si una ruta corresponde a un archivo CSV.
67. **Importar el módulo** `math` para comprobar que los módulos también son objetos.
68. **Almacenar diferentes elementos en una misma lista**, incluyendo tipos de datos nativos, una función, una clase, una instancia y un módulo.

69. **Recorrer la lista con un bucle** `for` y utilizar `type()` para mostrar la clase a la que pertenece cada elemento.

El programa analizará una lista que contiene diferentes tipos de datos utilizados habitualmente en aplicaciones de Ingeniería de Datos:

- **Datos simples:** números enteros, decimales, complejos, cadenas de texto y valores booleanos.
- **Colecciones:** listas, tuplas, conjuntos y diccionarios.
- **Otros objetos:** `None`, una función, una clase, una instancia de esa clase y un módulo importado. Se utilizará `type()` para identificar el tipo de cada elemento y comprobar que incluso las funciones, las clases y los módulos son objetos en Python.

Pregunta clave: Si en Python todo es un objeto, ¿qué diferencia existe entre el tipo de una función, el de una clase y el de un objeto creado a partir de esa clase? ¿Por qué `type(Proveedor)` y `type(Proveedor())` devuelven resultados diferentes?

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_4_1_todo_es_objeto.py`

```
import math

class Proveedor:
    pass

def validar_fichero(ruta):
    return ruta.endswith(".csv")

valores = [
    250,
    19.99,
    3 + 2j,
    "Logística Sur",
    True,
    [10, 20, 30],
    (40.41, -3.70),
    {"csv", "json"},
    {"tienda": "T-07"},
    None,
    validar_fichero,
    Proveedor,
    Proveedor(),
    math,
]

for v in valores:
    print(type(v))
```

Salida esperada:

```
<class 'int'>
<class 'float'>
<class 'complex'>
<class 'str'>
<class 'bool'>
<class 'list'>
<class 'tuple'>
<class 'set'>
<class 'dict'>
<class 'NoneType'>
<class 'function'>
<class 'type'>
<class '__main__.Proveedor'>
<class 'module'>
```

Highlights:

70. Una sola lista guarda números, textos, colecciones, `None`, una función, una clase, un objeto y un módulo. Esto solo es posible porque todos son objetos.
71. La función `validar_fichero` es un objeto de la clase `function`: se puede guardar en una lista, pasar como argumento o asignar a otra variable.
72. La clase `Proveedor` es un objeto de la clase `type`. En cambio, `Proveedor()` es un objeto de la clase `Proveedor`.
73. El módulo `math` es un objeto de la clase `module`.

Ejemplo 4.2 · Referencias con objetos inmutables y mutables

Una empresa de **Ingeniería de Datos** desarrolla una aplicación para configurar y supervisar sus pipelines. Durante su funcionamiento, el programa utiliza diferentes variables para almacenar límites de procesamiento, listas de columnas y umbrales de calidad de los datos.

El equipo necesita comprender qué sucede cuando **dos variables hacen referencia al mismo objeto** y cómo cambia su comportamiento dependiendo de si el objeto es mutable o inmutable.

El objetivo de este ejemplo es comprender cómo funcionan las **referencias a objetos en Python**, diferenciando entre la reasignación de variables y la modificación de objetos compartidos.

El programa debe permitir:

74. **Objetos inmutables (`int`):** asignar el mismo número a dos variables y comprobar qué ocurre cuando una de ellas recibe un nuevo valor.
75. **Objetos mutables (`list`):** crear dos variables que apuntan a la misma lista de columnas y observar cómo una modificación afecta a ambas.
76. **Objetos de nuestras propias clases:** crear dos instancias de la clase `Umbral` con el mismo valor y comprobar si son realmente el mismo objeto.
77. **Referencias compartidas:** asignar un objeto `Umbral` a otra variable y modificar uno de sus atributos para observar cómo afecta al objeto original.
78. **Identidad de objetos:** utilizar `is` e `id()` para comprobar cuándo dos variables hacen referencia al mismo objeto.

Se realizarán tres pruebas relacionadas con la configuración de un pipeline:

Prueba 1. Límite de registros

Un pipeline tiene un límite inicial de 500 registros. Se crea una segunda variable que hace referencia al mismo número y posteriormente se le asigna el valor 800.

Se comprobará si el límite original también cambia.

Prueba 2. Columnas de una tabla

Un proceso utiliza una lista con las columnas `id` y `fecha`. Se crea otra variable que apunta a esa misma lista y se añade la columna `importe`.

Se comprobará si ambas variables reflejan la modificación.

Prueba 3. Umbrales de calidad

Se crean dos objetos independientes de la clase `Umbral`, ambos con un valor inicial de `0.05`.

Posteriormente, se crea una tercera variable que apunta al primer objeto y se modifica su valor a `0.10`.

Se comprobará qué objetos se ven afectados y cuáles mantienen su valor original.

Pregunta clave: ¿Por qué reasignar una variable que contiene un número no modifica otra variable que apuntaba al mismo objeto, mientras que modificar una lista o los atributos de un objeto compartido sí afecta a todas las variables que lo referencian?

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_4_2_referencias.py`

```python
class Umbral:
    def __init__(self, valor):
        self.valor = valor

# Objetos inmutables: int
limite = 500
alias = limite
print(alias is limite)
alias = 800
print(limite, alias)
print(alias is limite)

# Objetos mutables: list
columnas = ["id", "fecha"]
mismas = columnas
mismas.append("importe")
print(columnas)
print(mismas is columnas)

# Dos objetos con los mismos datos
u1 = Umbral(0.05)
u2 = Umbral(0.05)
print(u1 is u2, id(u1) == id(u2))
u3 = u1
u3.valor = 0.10
print(u1.valor, u2.valor)
print(id(u3) == id(u1))
```

Salida esperada:

```
True
500 800
False
['id', 'fecha', 'importe']
True
False False
0.1 0.05
True
```

Higjhlights:

79. `alias = limite` hace que las dos variables apunten al mismo objeto `500`. Al hacer `alias = 800`, `alias` pasa a apuntar a un objeto nuevo; `limite` sigue en `500`.
80. Con la lista ocurre lo contrario: `mismas.append(...)` **modifica** el objeto compartido, así que el cambio se ve desde `columnas`.
81. `u1` y `u2` tienen el mismo dato, pero son dos objetos con identidades distintas.
82. `u3 = u1` comparte el objeto: cambiar `u3.valor` cambia `u1.valor`. `u2` no se ve afectado.
83. No se imprimen los valores de `id()` porque cambian en cada ejecución; se comparan entre sí, que es lo que interesa.

Ejemplo 4.3 · Los atributos de los objetos nativos

Como todo es un objeto, los números, los textos y las funciones tienen sus propios métodos especiales. Los operadores y funciones nativas los llaman por nosotros, pero se pueden ver con `dir()` y llamar explícitamente.

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_4_3_atributos_de_objetos.py`

```python
def validar_fichero(ruta):
    return ruta.endswith(".csv")
```

```python
print("__add__" in dir(55))
print("__abs__" in dir(-5.67))
print("__call__" in dir(validar_fichero))
print("upper" in dir("texto"))

print((55).__add__(45))
print((-5.67).__abs__())
print("ventas".__len__())
print(validar_fichero.__call__("ventas_07.csv"))
print(validar_fichero.__name__)

funcion = globals()["validar_fichero"]
print(funcion("clientes.json"))
```

Salida esperada:

```
True
True
True
True
100
5.67
6
True
validar_fichero
False
```

Highlights:

84. `dir()` devuelve una lista de nombres. Con `in` se comprueba si un objeto tiene un atributo concreto.
85. `(55).__add__(45)` es lo que Python ejecuta al escribir `55 + 45`. El número va entre paréntesis para que el punto no se confunda con un decimal.
86. `(-5.67).__abs__()` es lo que ejecuta `abs(-5.67)` y `"ventas".__len__()` lo que ejecuta `len("ventas")`.
87. Las funciones tienen el método `__call__`: es lo que se ejecuta al poner paréntesis detrás de su nombre. También tienen atributos como `__name__`.
88. `globals()` devuelve un diccionario con todos los nombres globales del programa. Al buscar `"validar_fichero"` se obtiene el objeto función, que se guarda en otra variable y se llama con ella.

Sección 5. Imitar una estructura: atributos dinámicos

> **Definición**

En Python se pueden **crear, modificar y eliminar atributos de un objeto en tiempo de ejecución**, después de crearlo. Con una clase vacía (`pass`) se obtiene un contenedor de datos con acceso por punto (`registro.cliente`), parecido a las *structs* de otros lenguajes.

> Además de la notación con punto, Python ofrece funciones para trabajar con atributos cuyo **nombre está en una variable**, algo muy habitual cuando los nombres vienen de la cabecera de un CSV o de las claves de un JSON:

Operación	Con nombre fijo	Con nombre en una variable
Crear o modificar	`obj.cliente = "C-1"`	`setattr(obj, nombre, valor)`
Leer	`obj.cliente`	`getattr(obj, nombre)`
Leer con valor por defecto	—	`getattr(obj, nombre, defecto)`
Eliminar	`del obj.cliente`	`delattr(obj, nombre)`
Comprobar si existe	—	`hasattr(obj, nombre)`

> Los atributos dinámicos son útiles para prototipos y para cargar datos de estructura variable. Para entidades estables (pedidos, clientes, tareas), es mejor definir todos los atributos en `__init__`: así todos los objetos tienen la misma forma.

Analogía de la vida diaria

Sintaxis

```python
class Estructura:
    pass

obj = Estructura()
obj.campo = valor                  # crear
obj.campo = nuevo_valor            # modificar
del obj.campo                      # eliminar

setattr(obj, "campo", valor)       # crear o modificar con el nombre en un str
getattr(obj, "campo", defecto)     # leer; si no existe, devuelve defecto
delattr(obj, "campo")              # eliminar
hasattr(obj, "campo")              # True / False
vars(obj)                          # diccionario con todos los atributos
```

Ejemplo 5.1 · Evento de clic en una tienda online

Una empresa de **Ingeniería de Datos** desarrolla un sistema para recopilar y analizar los clics que realizan los usuarios mientras navegan por una tienda online.

Cada vez que un usuario visita una página, se genera un evento que contiene información como su identificador, la página visitada, el dispositivo utilizado y la campaña publicitaria de procedencia.

Sin embargo, **no todos los eventos tienen necesariamente los mismos atributos**, y algunos datos pueden cambiar o dejar de ser necesarios durante el procesamiento.

Para gestionar esta información, se utiliza una clase vacía llamada `EventoClic`, a la que se pueden añadir, modificar y eliminar atributos dinámicamente.

El objetivo es comprender cómo funcionan los **atributos dinámicos en Python**, sin necesidad de definirlos previamente en el constructor `__init__()`.

El programa debe permitir:

89. **Crear atributos:** añadir al objeto los datos del usuario, la página visitada, el dispositivo y la campaña publicitaria.
90. **Consultar atributos:** utilizar `vars()` para mostrar los atributos almacenados en el objeto.
91. **Modificar atributos:** actualizar la página visitada y el dispositivo utilizado.
92. **Eliminar atributos:** eliminar la campaña publicitaria mediante `del`.
93. **Comprobar atributos:** utilizar `hasattr()` para verificar si un atributo sigue existiendo.

Se registra un evento de navegación con los siguientes datos iniciales:

Atributo	Valor
`usuario`	`u_4471`
`pagina`	`/checkout`
`dispositivo`	`movil`
`campania`	`black_friday`

Durante el procesamiento del evento, se realizan las siguientes operaciones:

- Se actualiza la página a `/checkout/pago`.
- Se cambia el dispositivo de `movil` a `tablet`.
- Se elimina el atributo `campania` porque ya no es necesario para este procesamiento.
- Se consulta el estado final del objeto para comprobar los cambios realizados. **Pregunta clave:** ¿Cómo puede un objeto de una clase vacía almacenar información, modificar sus atributos y eliminar aquellos que ya no necesita, sin que estos hayan sido definidos previamente en `__init__()`?

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_5_1_atributos_dinamicos.py`

```python
class EventoClic:
    pass

e = EventoClic()

# crear atributos sobre la marcha
e.usuario = "u_4471"
e.pagina = "/checkout"
e.dispositivo = "movil"
e.campania = "black_friday"
print(vars(e))

# modificar atributos
e.pagina = "/checkout/pago"
e.dispositivo = "tablet"

# eliminar un atributo
del e.campania

print(vars(e))
print(hasattr(e, "campania"))
print(e.pagina)
```

Salida esperada:

```
{'usuario': 'u_4471', 'pagina': '/checkout', 'dispositivo': 'movil', 'campania': 'black_friday'}
{'usuario': 'u_4471', 'pagina': '/checkout/pago', 'dispositivo': 'tablet'}
False
/checkout/pago
```

Highlights:

94. La clase `EventoClic` está vacía. Los cuatro atributos se crean después de construir el objeto.
95. Asignar a un atributo que ya existe lo modifica.
96. `del e.campania` lo elimina del objeto. A partir de ahí, `hasattr` devuelve `False`, y leer `e.campania` daría `AttributeError`.

Ejemplo 5.2 · Convertir una línea de CSV en un objeto

Una empresa de **Ingeniería de Datos** recibe diariamente archivos CSV con información sobre los pedidos realizados en una tienda online.

Cada archivo contiene una **cabecera con los nombres de las columnas** y varias líneas con los datos de los pedidos. Sin embargo, los nombres de las columnas pueden variar entre archivos, por lo que no siempre es posible definir previamente los atributos de una clase.

Para solucionar este problema, se utiliza una clase vacía llamada `Registro`, cuyos atributos se crean dinámicamente a partir de los nombres de las columnas del CSV.

El objetivo es aprender a utilizar **`setattr()`, `getattr()`, `delattr()` y `hasattr()`** para crear, consultar, modificar y eliminar atributos de un objeto.

El programa debe permitir:

97. **Separar los datos:** utilizar `split(",")` para convertir la cabecera y la línea del CSV en dos listas.
98. **Crear atributos dinámicamente:** recorrer las columnas y utilizar `setattr()` para asignar a cada atributo su valor correspondiente.
99. **Consultar atributos:** utilizar `getattr()` para obtener el cliente y consultar un atributo inexistente mediante un valor por defecto.
100. **Modificar atributos:** convertir el importe del pedido de texto (`str`) a número decimal (`float`).
101. **Eliminar atributos:** utilizar `delattr()` para eliminar el atributo `canal`.
102. **Comprobar atributos:** utilizar `hasattr()` para verificar si el atributo eliminado sigue existiendo.

Se recibe el siguiente registro de un archivo CSV:

id_pedido	cliente	importe	canal
PED-9001	C-188	149.90	web

El programa debe convertir este registro en un objeto `Registro`, utilizando los nombres de las columnas como atributos.

Posteriormente, se realizarán las siguientes operaciones:

- Consultar el identificador del cliente.
- Consultar el atributo `descuento`, devolviendo `"sin descuento"` si no existe.
- Convertir el atributo `importe` a `float` para permitir operaciones numéricas.
- Eliminar el atributo `canal`.
- Mostrar los atributos finales mediante `vars()` y comprobar que `canal` ya no existe.

Pregunta clave: ¿Cómo podemos convertir automáticamente las columnas de un archivo CSV en atributos de un objeto Python cuando sus nombres no se conocen de antemano, sin tener que definir cada atributo manualmente en una clase?

> Este ejemplo utiliza `split(",")` para simplificar el aprendizaje. En un proyecto real de Ingeniería de Datos conviene utilizar el módulo `csv` de Python, especialmente `csv.DictReader`, porque gestiona correctamente campos entrecomillados, separadores dentro de los valores y otras particularidades del formato CSV.

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_5_2_setattr_getattr.py`

```python
class Registro:
    pass

cabecera = "id_pedido,cliente,importe,canal"
linea = "PED-9001,C-188,149.90,web"

columnas = cabecera.split(",")
valores = linea.split(",")

r = Registro()
for i in range(len(columnas)):
    setattr(r, columnas[i], valores[i])

print(vars(r))
print(getattr(r, "cliente"))
print(getattr(r, "descuento", "sin descuento"))

setattr(r, "importe", float(r.importe))
delattr(r, "canal")
```

```
print(vars(r))
print(hasattr(r, "canal"))
```

Salida esperada:

```
{'id_pedido': 'PED-9001', 'cliente': 'C-188', 'importe': '149.90', 'canal': 'web'}
C-188
sin descuento
{'id_pedido': 'PED-9001', 'cliente': 'C-188', 'importe': 149.9}
False
```

Highlights:

103. `split(",")` separa la cabecera y la línea en dos listas del mismo tamaño.
104. En cada vuelta, `setattr(r, columnas[i], valores[i])` crea un atributo cuyo nombre es el texto de la columna. Es equivalente a escribir `r.id_pedido = "PED-9001"`, `r.cliente = "C-188"`...
105. Todo lo leído de un CSV es texto: `importe` vale `'149.90'`.
106. `getattr(r, "descuento", "sin descuento")` no falla aunque el atributo no exista: devuelve el valor por defecto.
107. `setattr` también sirve para modificar: `importe` pasa a ser un `float`. `delattr` elimina la columna `canal`.

Sección 6. Conversión de tipos

> 💡 ##### Definición

Convertir un dato de un tipo a otro es una tarea diaria en Data Engineering: todo lo que llega de un fichero de texto o de una API es `str`, y hay que pasarlo a `int`, `float` o a objetos de nuestras clases. Hay tres tipos de conversión:

Conversión	Cómo se hace	Ejemplo
Entre tipos nativos simples	Funciones `int()`, `float()`, `str()`, `bool()`, `complex()`	`int("250")`
Entre tipos nativos y contenedores	Funciones `list()`, `tuple()`, `set()`, `dict()`	`set(["a", "a", "b"])`
Entre tipos nativos y clases propias	**Nativo → clase**: el constructor `__init__`. **Clase → nativo**: métodos especiales	`DuracionTarea("00:12:40")`, `int(duracion)`

Métodos especiales de conversión de una clase a un tipo nativo:

Función nativa	Método especial	Debe devolver
`int(obj)`	`__int__(self)`	`int`
`float(obj)`	`__float__(self)`	`float`
`str(obj)`	`__str__(self)`	`str`
`bool(obj)` y `if obj:`	`__bool__(self)`	`bool`

Detalles de las conversiones nativas:

- `int(9.99)` **trunca** (quita los decimales) y da `9`. Para redondear se usa `round()`.
- `int(" 1520 ")` admite espacios alrededor, pero `int("3.75")` da error: primero hay que pasar por `float()`.
- `bool()` de un valor «vacío» (`0`, `0.0`, `""`, `[]`, `{}`, `None`) es `False`. Cualquier texto no vacío es `True`, incluso `"0"`.
- `int(texto, base)` interpreta el texto en otra base, por ejemplo hexadecimal.

Analogía de la vida diaria

Sintaxis

```
# Entre tipos nativos
int("250"), float("19.99"), str(2026), bool(0)
list("CSV"), tuple(lista), set(lista), dict(lista_de_pares)

# Clases propias
class MiClase:
    def __init__(self, texto):        # nativo -> MiClase
        ...
    def __int__(self):                # MiClase -> int
        return entero
    def __float__(self):              # MiClase -> float
        return decimal
    def __bool__(self):               # MiClase -> bool (y en if)
        return True o False
    def __str__(self):                # MiClase -> str
        return texto

obj = MiClase("valor")
int(obj), float(obj), bool(obj), str(obj)
```

Ejemplo 6.1 · Conversiones entre tipos nativos

Una empresa de **Ingeniería de Datos** recibe información procedente de archivos CSV, APIs y bases de datos. Durante el procesamiento, algunos valores llegan como cadenas de texto, mientras que otros necesitan transformarse en números, listas, tuplas, conjuntos o diccionarios.

Para poder utilizar estos datos correctamente, es necesario realizar **conversiones entre los tipos nativos de Python**.

El objetivo de este ejemplo es comprender cómo funcionan las principales funciones de conversión de Python y qué ocurre cuando transformamos un valor de un tipo a otro.

El programa debe permitir:

108. **Convertir números y cadenas:** utilizar `int()`, `float()` y `str()` para transformar valores entre distintos tipos.
109. **Diferenciar truncamiento y redondeo:** comprobar la diferencia entre `int()` y `round()` al trabajar con números decimales.
110. **Convertir valores a booleanos:** utilizar `bool()` para identificar qué valores se consideran `True` o `False`.
111. **Convertir colecciones:** utilizar `list()`, `tuple()`, `set()` y `dict()` para transformar diferentes estructuras de datos.
112. **Eliminar duplicados:** comprobar cómo `set()` permite obtener valores únicos a partir de una lista.
113. **Convertir números de otras bases:** utilizar `int()` para interpretar una cadena hexadecimal y obtener su equivalente decimal.

Se realizarán diferentes conversiones sobre datos habituales en un pipeline:

- Un número de registros recibido como texto (`"250"`) se convierte a entero.
- Un importe recibido como texto (`"19.99"`) se convierte a decimal.
- Un valor decimal (`9.99`) se trunca y se redondea para comparar los resultados.
- Se comprueba cómo Python interpreta valores vacíos y cadenas como `"0"` al convertirlos a booleanos.
- Se transforman cadenas y colecciones para obtener listas, tuplas, conjuntos y diccionarios.
- Se eliminan ciudades duplicadas de una lista.
- Se convierte un número hexadecimal (`"0x1F"`) a su valor decimal.

Pregunta clave: ¿Cómo podemos transformar datos entre los diferentes tipos nativos de Python y qué diferencias debemos tener en cuenta para evitar errores durante el procesamiento de información en un pipeline de datos?

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_6_1_conversiones_nativas.py`

```
print(int("250"))
print(float("19.99"))
print(int(9.99))
print(round(9.99))
print(str(2026))
print(bool(0), bool(""), bool("0"), bool([]))

print(list("CSV"))
print(tuple(["id", "fecha"]))
print(set(["madrid", "sevilla", "madrid"]) == {"madrid", "sevilla"})
print(dict([("csv", 12), ("json", 3)]))
print(list({"csv": 12, "json": 3}))
print(int("0x1F", 16))
```

Salida esperada:

```
250
19.99
9
10
2026
False False True False
['C', 'S', 'V']
('id', 'fecha')
True
{'csv': 12, 'json': 3}
['csv', 'json']
31
```

Highlights:

114. `int(9.99)` trunca a `9`, mientras que `round(9.99)` redondea a `10`.
115. `bool("0")` es `True`: es un texto con un carácter. Esto causa errores frecuentes al leer flags de ficheros de texto.
116. `list("CSV")` recorre el texto y crea una lista de caracteres.
117. `set(...)` elimina duplicados. Se compara con `==` en lugar de imprimirlo, porque un `set` no tiene orden fijo.
118. `dict(...)` crea un diccionario a partir de una lista de pares. `list(diccionario)` devuelve sus claves.
119. `int("0x1F", 16)` interpreta el texto como hexadecimal: `1F` = 31.

Ejemplo 6.2 · Duración de una tarea: de texto a objeto y de objeto a número

Una empresa de **Ingeniería de Datos** utiliza un orquestador para ejecutar diferentes tareas dentro de sus pipelines, como la extracción, transformación y carga de datos.

El sistema registra la duración de cada tarea como una cadena de texto con el formato `hh:mm:ss` (horas, minutos y segundos).

Sin embargo, el equipo necesita **convertir estas duraciones a diferentes tipos de datos** para realizar cálculos, generar informes y comprobar si alguna tarea tiene una duración de cero segundos.

Para ello, se desarrolla una clase llamada `DuracionTarea` que permite transformar una duración expresada como texto en un objeto Python y convertir posteriormente ese objeto a diferentes tipos nativos mediante **métodos especiales de conversión**.

El programa debe permitir:

120. **Texto a objeto (`__init__`):** recibir una duración en formato `hh:mm:ss`, convertirla a segundos y almacenarla en un atributo privado.
121. **Objeto a entero (`__int__`):** obtener la duración total expresada en segundos mediante `int()`.
122. **Objeto a decimal (`__float__`):** obtener la duración expresada en horas decimales mediante `float()`.
123. **Objeto a booleano (`__bool__`):** devolver `True` cuando la duración sea mayor que cero y `False` cuando sea igual a cero.
124. **Objeto a texto (`__str__`):** mostrar la duración en un formato legible utilizando minutos y segundos.
125. **Calcular duraciones acumuladas:** convertir varios objetos a segundos y utilizar `sum()` para obtener el tiempo total de ejecución.

Se dispone de tres tareas de un pipeline con las siguientes duraciones:

Tarea	Duración (`hh:mm:ss`)	Estado considerado en el ejemplo
Extracción	`00:12:40`	Con duración registrada
Transformación	`01:05:00`	Con duración registrada
Tarea sin ejecutar	`00:00:00`	Sin duración registrada

El programa debe realizar las siguientes operaciones:

- Convertir la duración de extracción a segundos.
- Convertir la duración de transformación a horas decimales.
- Mostrar la duración de extracción como texto legible.
- Comprobar qué tareas tienen una duración mayor que cero utilizando `bool()`.
- Calcular la duración total acumulada de las tres tareas.
- Mostrar un mensaje de advertencia cuando una tarea tenga una duración igual a cero.

Pregunta clave: ¿Cómo podemos transformar una duración recibida como texto en un objeto de nuestra propia clase y conseguir que Python pueda convertir ese objeto a `int`, `float`, `str` o `bool` utilizando los métodos especiales correspondientes?

El orquestador de pipelines registra la duración de cada tarea como texto `hh:mm:ss`. El equipo necesita sumar duraciones, expresarlas en horas y detectar tareas que no se ejecutaron.

> 📄 `clase_19_detalles_clases_objetos/teoria/ejemplo_6_2_duracion_tarea.py`

```python
class DuracionTarea:
    def __init__(self, texto="00:00:00"):            # conversión str -> DuracionTarea
        horas, minutos, segundos = texto.split(":")
        self.__segundos = int(horas) * 3600 + int(minutos) * 60 + int(segundos)

    def __int__(self):                                # DuracionTarea -> int
        return self.__segundos

    def __float__(self):                              # DuracionTarea -> float (horas)
        return round(self.__segundos / 3600, 2)

    def __bool__(self):                               # DuracionTarea -> bool
        return self.__segundos > 0

    def __str__(self):                                # DuracionTarea -> str
        return f"{self.__segundos // 60} min{self.__segundos % 60} s"

extraccion = DuracionTarea("00:12:40")
transformacion = DuracionTarea("01:05:00")
sin_ejecutar = DuracionTarea()

print(int(extraccion))
print(float(transformacion))
print(str(extraccion))
print(bool(transformacion), bool(sin_ejecutar))

total_segundos = sum([int(d) for d in [extraccion, transformacion, sin_ejecutar]])
```

```
print(total_segundos)

if not sin_ejecutar:
    print("Hay tareas que no han llegado a ejecutarse")
```

Salida esperada:

```
760
1.08
12 min 40 s
True False
4660
Hay tareas que no han llegado a ejecutarse
```

Highlights:

126. El **constructor** hace la conversión de `str` a `DuracionTarea`: separa horas, minutos y segundos y guarda el total en segundos en un atributo privado.
127. `int(extraccion)` ejecuta `__int__` y devuelve los segundos: 12 × 60 + 40 = 760.
128. `float(transformacion)` ejecuta `__float__` y devuelve las horas: 3900 / 3600 ≈ 1.08.
129. `bool()` ejecuta `__bool__`. Una duración de cero segundos es `False`.
130. `if not sin_ejecutar:` también usa `__bool__`: el objeto se comporta como cualquier valor nativo en una condición.
131. La comprehension convierte cada duración a `int` y `sum()` suma los segundos: 760 + 3900 + 0 = 4660.

Clase 20 · Herencia y polimorfismo

Sesión 1: Herencia

📌 Definición de Herencia

> Es un mecanismo que permite crear nuevas clases como una extensión de otras clases ya definidas. La clase que se extiende (hija) hereda los métodos y atributos de la clase extendida (Padre), pudiendo sobrescribirlos y cambiar así su comportamiento.

> Para indicar que una clase hereda de otra se usa el mismo nombre de la clase de la que se hereda entre paréntesis después del nombre de la nueva clase.

```
class Mamifero:
    def __init__(self, sangre):
        self.sangre = sangre
    def tipo_sangre(self):
        print(self.sangre)

class Elefante(Mamifero):
    def tipo_sangre(self):
        print('El tipo de sangre es', self.sangre)
j = Elefante('Frío')
j.tipo_sangre()
```

Salida esperada:

```
El tipo de sangre es Frío
```

📌 Sobrescribir métodos y añadir atributos

> Si se quiere especificar un nuevo atributo cuando se crea un objeto Elefante bastaría con escribir un nuevo método `__init__` para la clase que se ejecutaría en lugar del `__init__` heredado.

```
class Mamifero:
    def __init__(self, sangre):
        self.sangre = sangre
    def tipo_sangre(self):
        print(self.sangre)
class Elefante(Mamifero):
    def __init__(self, sangre, sexo):
        self.sangre = sangre
        self.sexo = sexo
    def tipo_sangre(self):
        print('El tipo de sangre es', self.sangre)
j = Elefante('Frío', 'hembra')
j.sexo  # 'hembra'
```

Salida esperada:

```
'hembra'
```

📌 Ejemplo sin herencia

> La herencia es la capacidad que tiene una clase de heredar los atributos y métodos de otra, algo que nos permite reutilizar código y hacer programas óptimos.

> **Ejemplo sin herencia:** Supongamos que nos piden diseñar una estructura para una tienda que vendía tres tipos de productos: adornos, alimentos y libros. Si partimos de una clase que contenga todos los atributos, quedaría más o menos así:

```
class Producto:
    def __init__(self, referencia, tipo, nombre,
                 pvp, descripcion, productor=None,
                 distribuidor=None, isbn=None, autor=None):
        self.referencia = referencia
        self.tipo = tipo
        self.nombre = nombre
        self.pvp = pvp
        self.descripcion = descripcion
        self.productor = productor
        self.distribuidor = distribuidor
        self.isbn = isbn
        self.autor = autor
adorno = Producto('000A', 'ADORNO', 'Vaso Adornado', 15,
                  'Vaso de porcelana con dibujos')
print(adorno)
print(adorno.tipo)
print(vars(adorno))
```

Salida esperada:

```
<__main__.Producto object at 0x7f7d2f34b610>
ADORNO
{'referencia': '000A', 'tipo': 'ADORNO', 'nombre': 'Vaso Adornado', 'pvp': 15, 'descripcion': 'Vaso de
porcelana con dibujos', 'productor': None, 'distribuidor': None, 'isbn': None, 'autor': None}
```

📌 Superclases

> La idea de la herencia es identificar una clase base (la superclase o clase Padre) con los atributos comunes y luego crear las demás clases heredando de ella (las subclases o clases hijas) extendiendo sus campos específicos. En nuestro caso esa clase Padre o Superclase sería el Producto en sí mismo:

```
class Producto:
    def __init__(self, referencia, nombre, pvp, descripcion):
        self.referencia = referencia
        self.nombre = nombre
        self.pvp = pvp
        self.descripcion = descripcion

    def __str__(self):
        return f"REFERENCIA\t {self.referencia}\n" \
               f"NOMBRE\t\t {self.nombre}\n" \
               f"PVP\t\t {self.pvp}\n" \
               f"DESCRIPCIÓN\t {self.descripcion}\n"
```

📌 Subclases

> Para heredar los atributos y métodos de una clase en otra sólo tenemos que pasarla entre paréntesis durante la definición:

```
class Adorno(Producto):
    pass

adorno = Adorno(2034, "Vaso adornado", 15, "Vaso de porcelana")
print(adorno)
```

Salida esperada:

```
REFERENCIA    2034
NOMBRE        Vaso adornado
PVP           15
DESCRIPCIÓN   Vaso de porcelana
```

> Como se puede apreciar es posible utilizar el comportamiento de una superclase sin definir nada en la subclase.

📌 Subclases con atributos adicionales

```
class Alimento(Producto):
    productor = ""
    distribuidor = ""

    def __str__(self):
        return f"REFERENCIA\t {self.referencia}\n" \
               f"NOMBRE\t\t {self.nombre}\n" \
               f"PVP\t\t {self.pvp}\n" \
               f"DESCRIPCIÓN\t {self.descripcion}\n" \
               f"PRODUCTOR\t {self.productor}\n" \
               f"DISTRIBUIDOR\t {self.distribuidor}\n"

class Libro(Producto):
    isbn = ""
    autor = ""
```

```
    def __str__(self):
        return f"REFERENCIA\t {self.referencia}\n" \
               f"NOMBRE\t\t {self.nombre}\n" \
               f"PVP\t\t {self.pvp}\n" \
               f"DESCRIPCIÓN\t {self.descripcion}\n" \
               f"ISBN\t\t {self.isbn}\n" \
               f"AUTOR\t\t {self.autor}\n"
```

📌 Usar las subclases

> Para utilizarlas simplemente tendríamos que establecer los atributos después de crear los objetos:

```
alimento = Alimento(2035, "Botella de Aceite de Oliva", 5, "250 ML")
alimento.productor = "La Aceitera"
alimento.distribuidor = "Distribuciones SA"
print(alimento)

libro = Libro(2036, "Cocina Mediterránea", 9, "Recetas sanas y buenas")
libro.isbn = "0-123456-78-9"
libro.autor = "Doña Juana"
print(libro)
```

Salida esperada:

```
REFERENCIA    2035
NOMBRE        Botella de Aceite de Oliva
PVP           5
DESCRIPCIÓN   250 ML
PRODUCTOR     La Aceitera
DISTRIBUIDOR  Distribuciones SA

REFERENCIA    2036
NOMBRE        Cocina Mediterránea
PVP           9
DESCRIPCIÓN   Recetas sanas y buenas
ISBN          0-123456-78-9
AUTOR         Doña Juana
```

> Ejercicio 1 . Tomando como referencia el ejemplo anterior crear una Superclase llamada Vehículo con sus respectivos atributos comunes. Luego crear al menos tres subclases (con sus atributos adicionales) del tipo: Coche, Moto, Camión, Autobús. Crear por lo menos un objeto de cada subclase y muestra en pantalla los resultados.

Trabajando en conjunto

> Gracias a la flexibilidad de Python podemos manejar objetos de distintas clases masivamente de una forma muy simple. Vamos a empezar creando una lista con nuestros tres productos de subclases distintas:

```
productos = [adorno, alimento]
productos.append(libro)

print(productos)
```

Salida:

```
[<__main__.Adorno object at 0x0000027EB304D150>, <__main__.Alimento object at 0x0000027EB3071F10>,
<__main__.Libro object at 0x0000027EB3022150>]
```

Ahora si queremos recorrer todos los productos de la lista podemos usar un bucle `for`.

```
for producto in productos:
    print(producto, "\n")
```

Salida:

```
REFERENCIA      2034
NOMBRE          Vaso adornado
PVP             15
DESCRIPCIÓN     Vaso de porcelana

REFERENCIA      2035
NOMBRE          Botella de Aceite de Oliva
PVP             5
DESCRIPCIÓN     250 ML
PRODUCTOR       La Aceitera
DISTRIBUIDOR    Distribuciones SA

REFERENCIA      2036
NOMBRE          Cocina Mediterránea
PVP             9
DESCRIPCIÓN     Recetas sanas y buenas
ISBN            0-123456-78-9
AUTOR           Doña Juana
```

> También podemos acceder a los atributos, siempre que sean compartidos entre todos los objetos:

```
for producto in productos:
    print(producto.referencia, producto.nombre)
```

Salida:

```
2034 Vaso adornado
2035 Botella de Aceite de Oliva
2036 Cocina Mediterránea
```

> Si un objeto no tiene el atributo al que queremos acceder nos dará error:

```
for producto in productos:
    print(producto.autor)
```

Salida:

```
AttributeError
Cell In[7], line 2
----> 2     print(producto.autor)

AttributeError: 'Adorno' object has no attribute 'autor'
```

Por suerte podemos hacer una comprobación con la función `isinstance()` para determinar si una instancia es de una determinada clase y así mostrar unos atributos u otros:

```
for producto in productos:
    if isinstance(producto, Adorno):
        print(producto.referencia, producto.nombre)
    elif isinstance(producto, Alimento):
        print(producto.referencia, producto.nombre, producto.productor)
    elif isinstance(producto, Libro):
        print(producto.referencia, producto.nombre, producto.isbn)
```

Salida:

```
2034 Vaso adornado
2035 Botella de Aceite de Oliva La Aceitera
2036 Cocina Mediterránea 0-123456-78-9
```

> *(Ejercicios P1, P2 y P3: enunciado y solución en la Parte 3.)*

Sesión 2: Polimorfismo

Polimorfismo.

> 💡 El polimorfismo es una propiedad de la herencia por la que objetos de distintas subclases pueden responder a una misma acción. La polimorfia es implícita en Python, ya que todas las clases son subclases de una superclase común llamada `Object`. Por ejemplo, la siguiente función aplica una rebaja al precio de un producto:

Ejemplo: Sistema de notificaciones

Enunciado

> 💡 Una empresa necesita enviar notificaciones a sus usuarios mediante tres canales: **correo electrónico, SMS y WhatsApp**. Vamos a resolver el mismo problema de dos formas:

1. **Sin polimorfismo:** el programa comprueba qué tipo de notificación recibe y decide qué método ejecutar.
2. **Con polimorfismo:** todas las clases ofrecen el mismo método `enviar()`, pero cada una lo implementa de manera diferente. > En ambos casos utilizaremos **herencia simple**. No utilizaremos `super()` ni herencia múltiple.

Solución 1. Sin polimorfismo

> 💡 La clase padre almacena el destinatario. Cada clase hija tiene un método **con un nombre diferente** para enviar su notificación.

```python
class Notificacion:
    def __init__(self, destinatario):
        self.destinatario = destinatario


class Email(Notificacion):
    def enviar_email(self):
        print(f"Enviando EMAIL a{self.destinatario}")


class SMS(Notificacion):
    def enviar_sms(self):
        print(f"Enviando SMS a{self.destinatario}")


class WhatsApp(Notificacion):
    def enviar_whatsapp(self):
        print(f"Enviando WhatsApp a{self.destinatario}")


# Objetos de diferentes clases
notificaciones = [
    Email("alumno@correo.com"),
```

```
    SMS("600123456"),
    WhatsApp("600987654")
]


# Hay que identificar el tipo de objeto antes de actuar
for notificacion in notificaciones:
    if isinstance(notificacion, Email):
        notificacion.enviar_email()
    elif isinstance(notificacion, SMS):
        notificacion.enviar_sms()
    elif isinstance(notificacion, WhatsApp):
        notificacion.enviar_whatsapp()
```

Salida esperada:

```
Enviando EMAIL a alumno@correo.com
Enviando SMS a 600123456
Enviando WhatsApp a 600987654
```

> 💡 ##### ¿Cuál es el problema?

El código que procesa las notificaciones **necesita conocer todas las clases** para saber qué método llamar.

Si mañana añadimos una clase `Telegram`, también tendremos que modificar el bloque `if / elif` para contemplarla.

Solución 2. Con polimorfismo

> 💡 Ahora definimos un método común, `enviar()`, en la clase padre. Las clases hijas **sobrescriben ese método** con su propio comportamiento.

```
class Notificacion:
    def __init__(self, destinatario):
        self.destinatario = destinatario

    def enviar(self):
        print("Enviando notificación genérica")


class Email(Notificacion):
    def enviar(self):
        print(f"Enviando EMAIL a{self.destinatario}")


class SMS(Notificacion):
    def enviar(self):
        print(f"Enviando SMS a{self.destinatario}")


class WhatsApp(Notificacion):
    def enviar(self):
        print(f"Enviando WhatsApp a{self.destinatario}")


# Objetos de diferentes clases
notificaciones = [
    Email("alumno@correo.com"),
    SMS("600123456"),
    WhatsApp("600987654")
]
```

```python
# Una misma llamada sirve para todos los objetos
for notificacion in notificaciones:
    notificacion.enviar()
```

Salida esperada:

```
Enviando EMAIL a alumno@correo.com
Enviando SMS a 600123456
Enviando WhatsApp a 600987654
```

¿Dónde está el polimorfismo?

> 💡 Observa esta instrucción:

```python
notificacion.enviar()
```

Se escribe **exactamente igual** para los tres objetos, pero Python ejecuta un método diferente dependiendo de la clase real del objeto:

- `Email` → `Email.enviar()`.
- `SMS` → `SMS.enviar()`.
- `WhatsApp` → `WhatsApp.enviar()`. **Misma acción, diferentes comportamientos.** No necesitamos preguntar qué tipo de notificación tenemos.

Ampliación: añadir otro canal

Podemos incorporar una nueva clase sin cambiar el bucle que procesa las notificaciones:

```python
class Telegram(Notificacion):
    def enviar(self):
        print(f"Enviando TELEGRAM a{self.destinatario}")


notificaciones.append(Telegram("@alumno"))

for notificacion in notificaciones:
    notificacion.enviar()
```

El bucle sigue siendo el mismo: cada objeto responde a `enviar()` a su manera.

Conceptos de repaso

> 💡 - **Herencia:** permite crear clases hijas a partir de una clase padre. - **Sobrescritura:** permite que una clase hija redefina un método heredado. - **Polimorfismo:** permite utilizar una misma operación (`enviar()`) con objetos de diferentes clases, obteniendo comportamientos distintos.

> Con polimorfismo, el código que utiliza los objetos no necesita decidir cómo trabaja cada clase. Simplemente solicita la acción común y cada objeto responde con su propia implementación.

> 💡 **Nota:** Python también admite polimorfismo sin herencia mediante *duck typing*. Aquí nos centramos en la herencia simple y la sobrescritura de métodos.

> 📖 Ejercicio 2. Crea 2 ejemplos de polimorfismo mediante ***duck typing*** **.**

Mas ejemplos:

```
class Empleado:
    def trabajar(self):
        print("El empleado trabaja en la oficina.")

class Programador(Empleado):
    def trabajar(self):
        print("El programador escribe código.")

class Diseñador(Empleado):
    def trabajar(self):
        print("El diseñador crea prototipos.")

# Lista de empleados de distintos tipos
empleados = [Programador(), Diseñador(), Empleado()]

for e in empleados:
    e.trabajar()
```

Salida

```
El programador escribe código.
El diseñador crea prototipos.
El empleado trabaja en la oficina.
```

---

2 Con la librería `math`

```
import math

class Figura:
    def area(self):
        pass  # método general, no hace nada

class Cuadrado(Figura):
    def __init__(self, lado):
        self.lado = lado

    def area(self):
        return self.lado ** 2

class Circulo(Figura):
    def __init__(self, radio):
        self.radio = radio

    def area(self):
        return math.pi * (self.radio ** 2)

# Lista de figuras
figuras = [Cuadrado(4), Circulo(3)]

for f in figuras:
    print(f"Área: {f.area():.2f}")
```

Salida:

```
Área: 16.00
Área: 28.27
```

---

3 Polimorfismo con una función externa

```
class Guitarra:
    def tocar(self):
        return "Tocando un solo de guitarra."

class Bateria:
    def tocar(self):
        return "Tocando un ritmo de batería."

class Piano:
    def tocar(self):
        return "Tocando una melodía suave."

def concierto(instrumento):
    print(instrumento.tocar())

instrumentos = [Guitarra(), Bateria(), Piano()]

for i in instrumentos:
    concierto(i)
```

Salida:

```
Tocando un solo de guitarra.
Tocando un ritmo de batería.
Tocando una melodía suave.
```

> *(Ejercicio P4: enunciado y solución en la Parte 3.)*

Herencia Múltiple

> 📖 La herencia múltiple es la capacidad de una subclase de heredar de múltiples superclases. Esto conlleva un problema, y es que, si varias superclases tienen los mismos atributos o métodos, la subclase sólo podrá heredar de una de ellas. En estos casos Python dará prioridad a las clases más a la izquierda en el momento de la declaración de la subclase:

```
class A:
    def __init__(self):
        print("Soy de clase A")

class B:
    def __init__(self):
        print("Soy de clase B")

class C(A,B):
    pass

c = C()
```

Salida:

```
Soy de clase A
```

Si cambiamos la “B” por la “A”:

```
class A:
    def __init__(self):
        print("Soy de clase A")

class B:
```

```
    def __init__(self):
        print("Soy de clase B")

class C(B,A):
    pass

c = C()
```

Salida:

```
Soy de clase B
```

Para comprobar cómo se heredan todos los métodos de las superclases, podemos añadir algunos métodos específicos de clase, por ejemplo:

```
class A:
    def __init__(self):
        print("Soy de clase A")
    def a(self):
        print("Este método lo heredo de A")

class B:
    def __init__(self):
        print("Soy de clase B")
    def b(self):
        print("Este método lo heredo de B")

class C(B,A):
    def c(self):
        print("Este método es de C")

c = C()
c.a()
```

Salida:

```
Soy de clase B
Este método lo heredo de A
```

Por ejemplo

```
c.b()
```

Salida:

```
Este método lo heredo de B
```

---

otro ejemplo:

```
c.c()
```

Salida:

```
Este método es de C
```

Mas ejemplos:

Herencia múltiple simple:

```
class Padre1:
    def saludar(self):
        print("Hola desde Padre1")

class Padre2:
    def despedir(self):
        print("Adiós desde Padre2")

class Hijo(Padre1, Padre2):
    pass

h = Hijo()
h.saludar()
h.despedir()
```

Salida:

```
Hola desde Padre1
Adiós desde Padre2
```

Herencia múltiple con atributos

```
class Persona:
    def __init__(self, nombre):
        self.nombre = nombre

class Estudiante:
    def estudiar(self):
        print(f"{self.nombre} está estudiando.")

class Deportista:
    def entrenar(self):
        print(f"{self.nombre} está entrenando.")

class EstudianteDeportista(Persona, Estudiante, Deportista):
    print("No me interesa")

e = EstudianteDeportista("Carlos")
e.estudiar()
e.entrenar()
```

Salida:

```
No me interesa
Carlos está estudiando.
Carlos está entrenando.
```

Uso de `super()` en herencia múltiple

```
class A:
    def accion(self):
        print("Acción en A")

class B(A):
    def accion(self):
        print("Acción en B")
        super().accion()
```

```python
class C(A):
    def accion(self):
        print("Acción en C")
        super().accion()

class D(B, C):
    def accion(self):
        print("Acción en D")
        super().accion()

d = D()
d.accion()
```

Salida:

```
Acción en D
Acción en B
Acción en C
Acción en A
```

> Utilizando la herencia múltiple podemos gestionar atributos y métodos heredados de varias clases a la vez.

Ejemplo resuelto 1: `super()` en herencia múltiple — Un coche

> 📖 #### Enunciado

Queremos representar un coche que tiene dos funcionalidades: **arrancar el motor** y **encender las luces**.

Para ello, utilizaremos cuatro clases:

- `Vehiculo`: clase padre que representa un vehículo.
- `Motor`: hereda de `Vehiculo`.
- `Luces`: hereda de `Vehiculo`.
- `Coche`: hereda de `Motor` y `Luces` (herencia múltiple). El objetivo es comprobar cómo `super()` permite ejecutar los métodos de todas las clases siguiendo el orden establecido por Python.

Diagrama de herencia

Código completo

```python
class Vehiculo:
    def iniciar(self):
        print("Vehículo preparado")


class Motor(Vehiculo):
    def iniciar(self):
        print("Motor arrancado")
        super().iniciar()


class Luces(Vehiculo):
    def iniciar(self):
        print("Luces encendidas")
        super().iniciar()


class Coche(Motor, Luces):
    def iniciar(self):
        print("Iniciando coche")
        super().iniciar()
```

```python
# Crear objeto
coche = Coche()

# Ejecutar método
coche.iniciar()
```

Salida:

```
Iniciando coche
Motor arrancado
Luces encendidas
Vehículo preparado
```

¿Qué pasaría si eliminamos `super()` de `Motor`?

Modificamos únicamente esta clase:

```python
class Motor(Vehiculo):
    def iniciar(self):
        print("Motor arrancado")
        # Sin super()
```

Ahora la salida sería:

```
Iniciando coche
Motor arrancado
```

Ni `Luces` ni `Vehiculo` ejecutarían sus métodos, porque la cadena de llamadas se interrumpe en `Motor`.

> 📖 En herencia múltiple, `super()` permite encadenar métodos de diferentes clases respetando el **MRO**. Para que funcione, las clases deben colaborar utilizando `super()`. En este ejemplo, gracias a `super()`, el coche ejecuta las funcionalidades de `Motor`, `Luces` y `Vehiculo` sin tener que llamarlas individualmente desde `Coche`.

Mismo ejemplo sin super()

Para conseguir el mismo resultado **sin utilizar `super()`**, debemos llamar explícitamente a los métodos de las clases que queremos ejecutar.

Código sin `super()`

```python
class Vehiculo:
    def iniciar(self):
        print("Vehículo preparado")


class Motor(Vehiculo):
    def iniciar(self):
        print("Motor arrancado")


class Luces(Vehiculo):
    def iniciar(self):
        print("Luces encendidas")


class Coche(Motor, Luces):
    def iniciar(self):
        print("Iniciando coche")

        Motor.iniciar(self)
        Luces.iniciar(self)
        Vehiculo.iniciar(self)
```

```
# Crear objeto
coche = Coche()

# Ejecutar método
coche.iniciar()
```

Salida:

```
Iniciando coche
Motor arrancado
Luces encendidas
Vehículo preparado
```

¿Qué diferencia existe?

En el código original, utilizábamos:

```
class Coche(Motor, Luces):
    def iniciar(self):
        print("Iniciando coche")
        super().iniciar()
```

Python recorría automáticamente el MRO:

`Coche → Motor → Luces → Vehiculo → object`

En cambio, sin `super()`:

```
class Coche(Motor, Luces):
    def iniciar(self):
        print("Iniciando coche")

        Motor.iniciar(self)
        Luces.iniciar(self)
        Vehiculo.iniciar(self)
```

Estamos indicando manualmente qué métodos ejecutar y en qué orden.

¿Por qué es preferible `super()`?

Imagina que añadimos una tercera funcionalidad:

```
class GPS(Vehiculo):
    def iniciar(self):
        print("GPS activado")
```

Sin `super()`, tendríamos que modificar `Coche`:

```
class Coche(Motor, Luces, GPS):
    def iniciar(self):
        print("Iniciando coche")

        Motor.iniciar(self)
        Luces.iniciar(self)
        GPS.iniciar(self)
        Vehiculo.iniciar(self)
```

Con una implementación cooperativa de `super()`, bastaría con modificar la herencia:

```
class Coche(Motor, Luces, GPS):
    def iniciar(self):
        print("Iniciando coche")
        super().iniciar()
```

Esto funciona siempre que `Motor`, `Luces` y `GPS` utilicen también `super()` correctamente.

> 📖 Sin `super()`, podemos conseguir el mismo resultado mediante llamadas explícitas a los métodos, pero perdemos la principal ventaja de la herencia múltiple cooperativa: permitir que Python determine automáticamente el recorrido de métodos según el MRO.

Ejemplo resuelto 2: Uso de `super()` en herencia múltiple: Dispositivos IoT

> 📖 #### Enunciado

Una empresa de tecnología desarrolla un sistema para gestionar **dispositivos inteligentes IoT**.

Cada dispositivo puede incorporar distintas funcionalidades:

- **`Dispositivo`**: proporciona el funcionamiento básico común.
- **`Sensor`**: permite obtener mediciones.
- **`Conectividad`**: permite enviar datos a la nube.
- **`SensorInteligente`**: combina las funcionalidades anteriores mediante **herencia múltiple**.

Estructura de clases

`SensorInteligente` hereda de `Sensor` y de `Conectividad`, que a su vez comparten la clase padre `Dispositivo`.

Paso 1: Crear la clase padre `Dispositivo`

```
class Dispositivo:
    def ejecutar(self):
        print("4. Dispositivo: funcionamiento básico")
```

Esta clase contiene el método `ejecutar()`, que representa el comportamiento básico de cualquier dispositivo.

Paso 2: Crear la clase `Sensor`

```
class Sensor(Dispositivo):
    def ejecutar(self):
        print("2. Sensor: obtener mediciones")
        super().ejecutar()
```

La clase `Sensor` hereda de `Dispositivo` y sobrescribe el método `ejecutar()`.

La instrucción `super().ejecutar()` permite continuar la búsqueda y ejecución de `ejecutar()` en la **siguiente clase del MRO**.

> **Importante:** `super()` no significa necesariamente «llamar a la clase padre directa». La siguiente clase depende del MRO del objeto utilizado.

Paso 3: Crear la clase `Conectividad`

```
class Conectividad(Dispositivo):
    def ejecutar(self):
        print("3. Conectividad: enviar datos a la nube")
        super().ejecutar()
```

También hereda directamente de `Dispositivo`. Primero ejecuta su propio código y después continúa la cadena con `super()`.

Paso 4: Crear la clase `SensorInteligente`

```
class SensorInteligente(Sensor, Conectividad):
    def ejecutar(self):
        print("1. SensorInteligente: iniciar operación")
        super().ejecutar()
```

La declaración `class SensorInteligente(Sensor, Conectividad):` establece una **herencia múltiple**. Python utiliza la estructura de herencia y el orden de las clases base para calcular el MRO.

Paso 5: Código completo

```
class Dispositivo:
    def ejecutar(self):
        print("4. Dispositivo: funcionamiento básico")


class Sensor(Dispositivo):
    def ejecutar(self):
        print("2. Sensor: obtener mediciones")
        super().ejecutar()


class Conectividad(Dispositivo):
    def ejecutar(self):
        print("3. Conectividad: enviar datos a la nube")
        super().ejecutar()


class SensorInteligente(Sensor, Conectividad):
    def ejecutar(self):
        print("1. SensorInteligente: iniciar operación")
        super().ejecutar()


# Crear objeto
sensor = SensorInteligente()

# Ejecutar
sensor.ejecutar()
```

Salida esperada:

```
1. SensorInteligente: iniciar operación
2. Sensor: obtener mediciones
3. Conectividad: enviar datos a la nube
4. Dispositivo: funcionamiento básico
```

Paso 6: Comprender el recorrido de `super()`

Al invocar `sensor.ejecutar()` ocurre lo siguiente:

25. **`SensorInteligente.ejecutar()`** imprime «iniciar operación» y llama a `super()`.
26. **`Sensor.ejecutar()`** imprime «obtener mediciones» y llama a `super()`.
27. **`Conectividad.ejecutar()`** imprime «enviar datos a la nube» y llama a `super()`.
28. **`Dispositivo.ejecutar()`** imprime «funcionamiento básico» y finaliza la cadena. **Recorrido de llamadas:**

```
SensorInteligente → Sensor → Conectividad → Dispositivo
```

¿Por qué `super()` en `Sensor` ejecuta `Conectividad` y no `Dispositivo`?

Aunque `Sensor` hereda directamente de `Dispositivo`, cuando el objeto es de tipo `SensorInteligente`, Python sigue el MRO de esta última clase:

```
print(SensorInteligente.__mro__)
```

Resultado simplificado:

```
SensorInteligente → Sensor → Conectividad → Dispositivo → object
```

Por eso, dentro de `Sensor`, `super().ejecutar()` continúa con `Conectividad.ejecutar()`.

Paso 7: ¿Qué sucede si no utilizamos `super()`?

Supongamos que sustituimos `super()` en `Sensor` por una llamada directa a `Dispositivo`:

```
class Sensor(Dispositivo):
    def ejecutar(self):
        print("2. Sensor: obtener mediciones")
        Dispositivo.ejecutar(self)
```

Manteniendo el resto de las clases del ejemplo, la salida sería:

```
1. SensorInteligente: iniciar operación
2. Sensor: obtener mediciones
4. Dispositivo: funcionamiento básico
```

¿Qué ha pasado? No se ha ejecutado `Conectividad.ejecutar()`, porque la llamada directa a `Dispositivo` **interrumpe la cadena cooperativa del MRO**. Además, en jerarquías más complejas, llamar directamente a los padres puede provocar ejecuciones duplicadas.

Clase 21 · Programación funcional

Sesión 1 - Introd a la Programación Funcional

¿Qué son las comprehensions?

> 💡 - Las **comprehensions** ofrecen una forma fácil y compacta de crear **listas, conjuntos y diccionarios**. - Una comprehension funciona **recorriendo o iterando** sobre elementos y **asignándolos a un contenedor** como lista, conjunto o diccionario. - Este contenedor **no puede ser una tupla**, ya que al ser inmutable no puede recibir asignaciones.

List Comprehension (Comprensión de listas)

- Una comprensión de lista consiste en **corchetes** que contienen una **expresión** seguida de una cláusula `for`, y cero o más cláusulas `for` o `if`.
- La forma general de una comprensión de lista es:

```
lst = [expresión for var in secuencia [opcional for and/or if]]
```

Ejemplos de list comprehension:

```
# Generar 20 números aleatorios en el rango de 10 a 100
import random
a = [random.randint(10, 100) for n in range(20)]
```

```
[63, 19, 53, 95, 84, 37, 22, 32, 67, 34, 93, 35, 20, 97, 35, 49, 62, 51, 93, 15]
```

```python
# Generar cuadrado y cubo de todos
# los números entre 0 y 10
a = [(x, x**2, x**3) for x in range(10)]
print(a)
```

```
[(0, 0, 0), (1, 1, 1), (2, 4, 8), (3, 9, 27), (4, 16, 64), (5, 25, 125), (6, 36, 216), (7, 49, 343), (8, 64,
512), (9, 81, 729)]
```

```python
# Convertir una lista de strings a una lista de enteros
a = [int(x) for x in ['10', '20', '30', '40']]
print(a)
```

```
[10, 20, 30, 40]
```

Ejemplos del uso de if en list comprehension:

```python
# Generar una lista de números pares en el rango de 10 a 30
a = [n for n in range(10, 30) if n % 2 == 0]
print(a)
```

```
[10, 12, 14, 16, 18, 20, 22, 24, 26, 28]
```

```python
# De una lista, eliminar todos los números con valor entre 20 y 50
a = [num for num in a if num < 20 or num > 50]
print(a)
```

```
[10, 12, 14, 16, 18]
```

Ejemplo del uso de if-else en list comprehension:

```python
# Cuando se usan if-else, se colocan antes del for
# Reemplazar una vocal en una cadena con !
a = ['!' if alphabet in 'aeiou' else alphabet for alphabet in 'Technical']
print(a)
```

```
['T', '!', 'c', 'h', 'n', '!', 'c', '!', 'l']
```

Ejemplo del uso de múltiples for y if en list comprehension:

```python
# Aplanar una lista de listas
arr = [[1,2,3,4], [5,6,7,8], [10, 11, 12, 13]]
b = [n for ele in arr for n in ele]  # una forma
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13]
```

```python
# * puede usarse para desempaquetar una lista
c = [*arr[0], *arr[1], *arr[2]]  # otra forma
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 10, 11, 12, 13]
```

Nota la diferencia entre for anidado en list comprehension y comprensión anidada:

```
# Produce [4, 5, 6, 5, 6, 7, 6, 7, 8]. Usa for anidado
lst = [a + b for a in [1, 2, 3] for b in [3, 4, 5]]
print(lst)
```

```
[4, 5, 6, 5, 6, 7, 6, 7, 8]
```

```
# Produce [[4, 5, 6], [5, 6, 7], [6, 7, 8]]. Usa comprensión anidada
lst = [[a + b for a in [1, 2, 3]] for b in [3, 4, 5]]
print(lst)
```

```
[[4, 5, 6], [5, 6, 7], [6, 7, 8]]
```

> Piensa en el primer for como bucle externo y el segundo como bucle interno.

Ejemplo del uso de múltiples for y if en list comprehension:

```
# Generar todas las combinaciones únicas de 1, 2 y 3
a = [(i, j, k) for i in [1,2,3] for j in [1,2,3] for k in [1, 2, 3] if i != j and j != k and k != i]
```

```
[(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)]
```

🔹 Set Comprehension (Comprensión de conjuntos)

> - Al igual que las **list comprehensions**, las **set comprehensions** ofrecen una forma fácil de crear **conjuntos**.
> - Consisten en **llaves** `{}` que contienen una **expresión** seguida de una cláusula `for`, y opcionalmente más `for` o `if`.

Sintaxis general:

```
s = {expresión for var in secuencia [if condición] [for ...]}
```

✅ Ejemplos de set comprehension

```
# Generar un conjunto con los cuadrados de números del 0 al 9
a = {x**2 for x in range(10)}
# → {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}
```

```
{0, 1, 64, 4, 36, 9, 16, 49, 81, 25}
```

```
# De un conjunto, eliminar todos los números entre 20 y 50
a = {num for num in a if num > 20 and num < 50}
```

```
{49, 36, 25}
```

🔹 Dictionary Comprehension (Comprensión de diccionarios)

> - Permite crear **diccionarios** de forma compacta a partir de iterables.

- Usa **llaves** {} y la sintaxis: {clave: valor for (clave, valor) in iterable}

Sintaxis General

```
dict_var = {clave: valor for (clave, valor) in diccionario.items()}
```

✅ Ejemplos de dictionary comprehension

```
# Diccionario inicial
d = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

```
# Obtener diccionario con cada valor al cubo
d1 = {k: v ** 3 for (k, v) in d.items()}
print(d1)
```

```
{'a': 1, 'b': 8, 'c': 27, 'd': 64}
```

```
# Solo los valores mayores a 3, al cubo
d2 = {k: v ** 3 for (k, v) in d.items() if v > 3}
print(d2)
```

```
{'d': 64}
```

```
# Identificar entradas pares e impares
d3 = {k: ('Even' if v % 2 == 0 else 'Odd') for (k, v) in d.items()}
print(d3)
```

```
{'a': 'Odd', 'b': 'Even', 'c': 'Odd', 'd': 'Even'}
```

Programación Funcional

> 💡 En la programación funcional, un problema se trata como la evaluación de una o más funciones. Por lo tanto, un problema dado se descompone en un conjunto de funciones. Estas funciones proporcionan la principal fuente de lógica en el programa.

Funciones como Valores de Primera Clase

> Python facilita la programación funcional al tratar las funciones como valores de datos “de primera clase”. Esto significa que: las funciones pueden asignarse a variables y luego llamarse usando estas variables.

```
def saludar():
    print("¡Hola!")

# Asignamos la función a una variable
mi_funcion = saludar()
```

```
¡Hola!
```

Las funciones pueden pasarse como argumentos a funciones y devolverse desde funciones.

```
def aplicar(funcion, valor):
    return funcion(valor)
```

```
def cuadrado(x):
    return x * x
```

Pasamos la función `cuadrado` como argumento

```
resultado = aplicar(cuadrado, 5)
```

```
25
```

Función que devuelve otra función

```
def crear_multiplicador(n):
    def multiplicar(x):
        return x * n
    return multiplicar
```

```
duplicar = crear_multiplicador(2)
print(duplicar(10))
```

```
20
```

```
triplicar = crear_multiplicador(3)
print(triplicar(30))
```

```
90
```

> Las funciones pueden construirse en tiempo de ejecución, de la misma manera que se pueden construir listas, tuplas, etc. Creamos funciones dentro de otra función, en tiempo de ejecución

```
def construir_funcion_operacion(operador):
    def sumar(x, y):
        return x + y

    def restar(x, y):
        return x - y

    if operador == 'sumar':
        return sumar
    elif operador == 'restar':
        return restar
    else:
        raise ValueError("Operador no soportado")
```

Usamos la función para crear otra función en tiempo de ejecución

```
operacion = construir_funcion_operacion('restar')
print(operacion(10, 4))
```

```
6
```

```
otra_operacion = construir_funcion_operacion('sumar')
print(otra_operacion(3, 7))
```

```
10
```

Ejemplo resuelto 1. Procesamiento de registros de servidores en un Data Center

Enunciado

> 📖 Una empresa tecnológica administra varios **Data Centers** que disponen de servidores encargados de ejecutar aplicaciones y procesar grandes volúmenes de datos. Cada servidor genera registros que contienen información sobre el consumo de memoria RAM, expresado en megabytes (MB).

El equipo de Ingeniería de Datos necesita procesar estos registros para identificar los consumos válidos y transformar sus valores a gigabytes (GB).

Se necesita desarrollar un programa que permita:

1. Crear una función que convierta el consumo de memoria de MB a GB.
2. Crear una función genérica que reciba otra función como argumento y la aplique a un valor.
3. Utilizar **list comprehension** para seleccionar registros válidos y convertir el consumo a GB.
4. Utilizar **set comprehension** para identificar los Data Centers que tienen registros válidos.
5. Utilizar **dictionary comprehension** para generar un diccionario con el identificador de cada servidor y su consumo de memoria convertido. Se consideran válidos los registros cuyo consumo de memoria esté comprendido entre **0 y 32768 MB**, ambos incluidos. Este rango es una regla de validación simplificada para el ejercicio.

Código

```
# Datos originales de servidores
registros = [
    {"servidor": "SRV01", "datacenter": "Madrid", "memoria": 8192},
    {"servidor": "SRV02", "datacenter": "Barcelona", "memoria": 16384},
    {"servidor": "SRV03", "datacenter": "Madrid", "memoria": -1024},
    {"servidor": "SRV04", "datacenter": "Valencia", "memoria": 4096},
    {"servidor": "SRV05", "datacenter": "Barcelona", "memoria": 65536},
    {"servidor": "SRV06", "datacenter": "Valencia", "memoria": 12288}
]


# 1. Función para convertir MB a GB
# Se emplea la equivalencia didáctica 1024 MB = 1 GB.
def mb_a_gb(megabytes):
    return megabytes / 1024


# 2. Función que recibe otra función como argumento
def aplicar_conversion(funcion, valor):
    return funcion(valor)


# 3. List comprehension: filtrar y transformar
memoria_gb = [
    aplicar_conversion(mb_a_gb, r["memoria"])
    for r in registros
    if 0 <= r["memoria"] <= 32768
]


# 4. Set comprehension: Data Centers únicos
datacenters = {
    r["datacenter"]
    for r in registros
    if 0 <= r["memoria"] <= 32768
}


# 5. Dictionary comprehension: servidor -> memoria GB
memoria_por_servidor = {
    r["servidor"]: aplicar_conversion(mb_a_gb, r["memoria"])
    for r in registros
```

```
    if 0 <= r["memoria"] <= 32768
}


# Resultados
print("Memoria en GB:", memoria_gb)
print("Data Centers:", sorted(datacenters))
print("Memoria por servidor:", memoria_por_servidor)
```

Salida esperada

```
Memoria en GB: [8.0, 16.0, 4.0, 12.0]
Data Centers: ['Barcelona', 'Madrid', 'Valencia']
Memoria por servidor: {'SRV01': 8.0, 'SRV02': 16.0, 'SRV04': 4.0, 'SRV06': 12.0}
```

Relación con Ingeniería de Datos

Ejemplo resuelto 2. Análisis de ventas mediante funciones dinámicas

Enunciado

> 📖 Una empresa de comercio electrónico dispone de un conjunto de ventas realizadas en distintas categorías de productos. El departamento de Ingeniería de Datos necesita implementar un sistema que permita aplicar diferentes porcentajes de impuestos o descuentos sobre los importes.

El programa debe:

1. Crear una función que **devuelva otra función** para aplicar un porcentaje de incremento o descuento.
2. Generar dinámicamente dos funciones: una para aplicar un **IVA del 21 %** y otra para aplicar un **descuento del 10 %**.
3. Utilizar **list comprehension** para calcular los importes con IVA de las ventas superiores a 100 €.
4. Utilizar **set comprehension** para identificar las categorías con ventas superiores a 100 €.
5. Utilizar **dictionary comprehension** para obtener los importes con descuento de esas ventas. En este ejemplo, IVA y descuento se aplican **por separado** al importe original.

Código resuelto

```
# Datos originales
ventas = [
    {"id": 101, "categoria": "Hardware", "importe": 250},
    {"id": 102, "categoria": "Software", "importe": 80},
    {"id": 103, "categoria": "Hardware", "importe": 400},
    {"id": 104, "categoria": "Servicios", "importe": 150},
    {"id": 105, "categoria": "Software", "importe": 60},
    {"id": 106, "categoria": "Servicios", "importe": 300}
]


# 1. Función que devuelve otra función
def crear_ajustador(porcentaje):

    def ajustar(importe):
        return round(importe * (1 + porcentaje / 100), 2)

    return ajustar


# 2. Crear funciones dinámicamente
aplicar_iva = crear_ajustador(21)
aplicar_descuento = crear_ajustador(-10)
```

```python
# 3. List comprehension: ventas con IVA
ventas_con_iva = [
    aplicar_iva(v["importe"])
    for v in ventas
    if v["importe"] > 100
]


# 4. Set comprehension: categorías únicas
categorias = {
    v["categoria"]
    for v in ventas
    if v["importe"] > 100
}


# 5. Dictionary comprehension: ventas con descuento
ventas_con_descuento = {
    v["id"]: aplicar_descuento(v["importe"])
    for v in ventas
    if v["importe"] > 100
}


# Resultados
print("Ventas con IVA:", ventas_con_iva)
print("Categorías:", sorted(categorias))
print("Ventas con descuento:", ventas_con_descuento)
```

Salida esperada

```
Ventas con IVA: [302.5, 484.0, 181.5, 363.0]
Categorías: ['Hardware', 'Servicios']
Ventas con descuento: {101: 225.0, 103: 360.0, 104: 135.0, 106: 270.0}
```

Ejemplo resuelto 3. Análisis de ejecuciones de pipelines de datos

Enunciado

> 📖 Una empresa utiliza varios **pipelines de Ingeniería de Datos** para procesar información de ventas, inventario y clientes. Cada ejecución de un pipeline registra su identificador, el nombre del pipeline, la duración en minutos, el número de registros procesados y su estado (`OK` o `ERROR`). Se necesita desarrollar un programa en Python que combine **programación orientada a objetos**, **comprehensions** y **programación funcional** para analizar estas ejecuciones.

El programa debe:

1. Definir la clase `EjecucionPipeline`, que represente una ejecución y permita calcular cuántos registros procesa por minuto.
2. Crear varios **objetos** a partir de los datos proporcionados.
3. Definir una función `aplicar(funcion, objeto)` que reciba una función como argumento y la ejecute sobre un objeto.
4. Utilizar una **list comprehension** para obtener la velocidad de procesamiento de las ejecuciones correctas.
5. Utilizar una **set comprehension** para identificar los nombres de los pipelines que terminaron correctamente.
6. Utilizar una **dictionary comprehension** para asociar cada identificador de ejecución correcta con su número de registros procesados.
7. Crear una función que **devuelva otra función** para comprobar si una ejecución alcanza una velocidad mínima y utilizarla dentro de una comprehension. > **Reglas:** una ejecución es correcta cuando su estado es `OK`. Todos los datos tienen una duración positiva para que pueda calcularse la velocidad de procesamiento. En este ejercicio, los identificadores de ejecución son únicos.

Datos de entrada

ID	Pipeline	Duración (min)	Registros	Estado
E01	Ventas	5	10000	OK
E02	Inventario	4	6000	OK
E03	Clientes	6	9000	ERROR
E04	Ventas	8	12000	OK
E05	Clientes	3	7500	OK
E06	Inventario	2	2000	ERROR

Código

```
# 1. Clase que representa una ejecución de un pipeline
class EjecucionPipeline:
    def __init__(self, id_ejecucion, pipeline, duracion, registros, estado):
        self.id_ejecucion = id_ejecucion
        self.pipeline = pipeline
        self.duracion = duracion
        self.registros = registros
        self.estado = estado

    def velocidad(self):
        """Devuelve los registros procesados por minuto."""
        return self.registros / self.duracion


# 2. Creación de objetos
ejecuciones = [
    EjecucionPipeline("E01", "Ventas", 5, 10000, "OK"),
    EjecucionPipeline("E02", "Inventario", 4, 6000, "OK"),
    EjecucionPipeline("E03", "Clientes", 6, 9000, "ERROR"),
    EjecucionPipeline("E04", "Ventas", 8, 12000, "OK"),
    EjecucionPipeline("E05", "Clientes", 3, 7500, "OK"),
    EjecucionPipeline("E06", "Inventario", 2, 2000, "ERROR")
]


# 3. Función de orden superior: recibe otra función
def aplicar(funcion, objeto):
    return funcion(objeto)


# 4. List comprehension + función como argumento
# Resultado: velocidades de ejecuciones correctas
velocidades = [
    aplicar(lambda ejecucion: ejecucion.velocidad(), e)
    for e in ejecuciones
    if e.estado == "OK"
]


# 5. Set comprehension: nombres sin duplicados
pipelines_correctos = {
    e.pipeline
    for e in ejecuciones
    if e.estado == "OK"
}


# 6. Dictionary comprehension: ID -> registros procesados
registros_por_ejecucion = {
    e.id_ejecucion: e.registros
    for e in ejecuciones
    if e.estado == "OK"
}
```

```python
# 7. Función que devuelve otra función (closure)
def crear_filtro_velocidad(minima):
    def cumple(ejecucion):
        return ejecucion.velocidad() >= minima
    return cumple


# Crear una función para identificar ejecuciones rápidas
es_rapida = crear_filtro_velocidad(1800)

# List comprehension + función devuelta por otra función
ids_rapidos = [
    e.id_ejecucion
    for e in ejecuciones
    if e.estado == "OK" and es_rapida(e)
]


# Resultados
print("Velocidades (registros/min):", velocidades)
print("Pipelines correctos:", sorted(pipelines_correctos))
print("Registros por ejecución:", registros_por_ejecucion)
print("Ejecuciones rápidas (>= 1800 registros/min):", ids_rapidos)
```

Salida esperada

```
Velocidades (registros/min): [2000.0, 1500.0, 1500.0, 2500.0]
Pipelines correctos: ['Clientes', 'Inventario', 'Ventas']
Registros por ejecución: {'E01': 10000, 'E02': 6000, 'E04': 12000, 'E05': 7500}
Ejecuciones rápidas (>= 1800 registros/min): ['E01', 'E05']
```

> *(Ejercicio P1: enunciado y solución en la Parte 3.)*

Sesión 2: lambda, map(), fillter(), y reduce()

Ejemplos de construcción de funciones en tiempo de ejecución: Funciones Lambda

> - Las funciones normales tienen nombre. Se definen con la palabra clave `def`.
> - Las funciones `lambda` se utilizan normalmente para funciones cortas que es conveniente definir en el punto en el que se llaman.
> - Las funciones `lambda` también se denominan funciones anónimas o funciones en línea.
> - Una función `lambda` puede recibir cualquier número de argumentos, pero sólo puede devolver un valor.

Otros ejemplos de funciones lambda

Función que calcula el cubo de un número:

```python
def cubo(a):
    return a**3

b = cubo
print(b(8))
```

```python
cubo = lambda a: a**3

b = cubo
print(b(8))
```

Salida: Salida:

```
512
```

```
512
```

Otros ejemplos de funciones lambda

Función que recibe 3 argumentos y devuelve el promedio de ellos: Con lambda:

```
def promedio(x, y, z):
    return (x + y + z) / 3

print(promedio(2, 4, 11))
```

```
promedio = lambda x, y, z: (x + y + z) / 3

print(promedio(2, 4, 11))
```

Salida: Salida:

```
5.666666666666667
```

```
5.666666666666667
```

Mas ejemplos de funciones lambda

Función que recibe nombre y apellido para imprimirlo

```
def nomb_comp(Nombre, Apellido):
    return f'El nombre completo es: {Nombre} {Apellido}'

x = nomb_comp
print(x('Nicolás', 'Pérez'))
```

Salida:

```
El nombre completo es: Nicolás Pérez
```

Con lambda:

```
nomb_comp = lambda Nombre, Apellido: f'El nombre completo es: {Nombre} {Apellido}'

t = nomb_comp('Nicolás', 'Pérez')
print(t)
```

Salida:

```
El nombre completo es: Nicolás Pérez
```

También pueden ser pasadas directamente al `print()`

```
def promedio(x, y, z):
    return (x + y + z) / 3

print(promedio(2, 4, 11))
```

Salida:

```
5.666666666666667
```

Con lambda + print()

```
print((lambda x, y, z: (x + y + z) / 3)(2, 4, 11))
```

Salida:

```
5.666666666666667
```

Mas ejemplos:

```
def cubo(a):
    return a**3

b = cubo
print(b(8))
```

salida:

```
512
```

Se puede hacer así:

```
print((lambda a: a**3)(8))
```

Salida:

```
512
```

Lambda también puede asignarse a una variable y luego invocarse:

```
p = lambda a: a**3
print(p(8))
```

Salida:

```
512
```

```
q = lambda x, y, z: (x + y + z) / 3
print(q(2, 4, 11))
```

Salida:

```
5.666666666666667
```

Los datos tipos contenedores (listas, tuplas, entre otros) también pueden pasarse a una función `lambda`. Por ejemplo, una función `lambda` que calcula la media de los números de una lista puede pasarse a la función `print()`:

```
lista_1 = [1, 2, 3, 4, 5]
print((lambda L1: sum(L1)/len(L1))(lista_1))
```

salida:

```
3.0
```

```
tupla_2 = (1, 2, 3, 4, 5)
print((lambda t2: sum(t2)/len(t2))(tupla_2))
```

salida:

```
3.0
```

Funciones de orden superior.

Una función de orden superior es una función que puede recibir otras funciones como argumentos o devolverlos:

```
def incrementar(x):
    return x + 1

def func_orden_sup(x, func):
    return x + func(x)

resultado = func_orden_sup(2, incrementar)
# 2 + (2 + 1)
print(resultado)
```

salida:

```
5
```

```
incrementar = lambda x: x + 1

func_orden_sup = lambda x, func: x + func(x)

resultado = func_orden_sup(2, incrementar)
print(resultado)
```

salida:

```
5
```

Otro ejemplo, podemos pasar una función lambda a la función `built-in` llamada `sorted()` para ordenar un diccionario por valores:

```
d = {'Aceite': 230, 'Pan': 150, 'Salmón': 175, 'Jamón': 35}
d1 = sorted(d.items(), key = lambda a: a[1])
print(d1)
```

salida:

```
[('Jamón', 35), ('Pan', 150), ('Salmón', 175), ('Aceite', 230)
```

> Para facilitar la programación funcional, Python proporciona 3 funciones de orden superior: `map()`, `filter()` y `reduce()`.

`map()`: su objetivo principal es hacer transformaciones a una lista dada de elementos.

🧾 Sintaxis general de `map()`

```
map(funcion, iterable)
```

✅ **Parámetros:**

- `funcion`: Una función que se aplicará a cada elemento del iterable. Puede ser:
- una función definida con `def`
- una función anónima `lambda`
- `iterable`: Una secuencia como una lista, tupla, conjunto, etc. > Nota: Si usas map() en Python 3, devuelve un objeto iterable tipo map, por lo que normalmente se convierte a lista con list().

```
numeros = [1, 2, 3, 4]
numeros_2 = []
for i in numeros:
    numeros_2.append(i * 2)
print(numeros)
print(numeros_2)
```

salida:

```
[1, 2, 3, 4]
[2, 4, 6, 8]
```

Esto se puede hacer prácticamente en una sola línea usando `map()` y `lambda`:

```
numeros = [1, 2, 3, 4]
numeros_3 = list(map(lambda i: i*2, numeros))
print(numeros_3)
```

salida:

```
[2, 4, 6, 8]
```

Con set()

```
numeros = [1, 2, 3, 4]
numeros_3 = set(map(lambda i: i*2, numeros))
print(numeros_3)
```

salida:

```
{8, 2, 4, 6}
```

```
numeros_3 = tuple(map(lambda i: i*2, numeros))
print(numeros_3)
```

salida:

```
(2, 4, 6, 8)
```

Mas ejemplos con `map()`

```
numeros_4 = [1, 2, 3, 4]
numeros_5 = [5, 6, 7]
print(numeros_4)
print(numeros_5)
print()
resultado = list(map(lambda x, y: x + y, numeros_4, numeros_5))
print(resultado)
```

salida:

```
[1, 2, 3, 4]
[5, 6, 7]

[6, 8, 10]
```

Uso de `map()` con diccionarios, compara con el codigo sin usar lambda y map

```python
# Items ordenes de compra almacenada en una lista de diccionarios
items = [
    {
        'producto': 'camisa',
        'precio': 100,
    },
    {
        'producto': 'pantalones',
        'precio': 300
    },
    {
        'producto': 'pantalones 2',
        'precio': 200
    }
]

# Quiero transformar esa información a una lista que
# solo contenga los precios: precios = [100, 200, 300]

precios = list(map(lambda item: item['precio'], items))
print(precios)
```

salida:

```
[100, 300, 200]
```

```python
# Items ordenes de compra almacenadas en una lista de diccionarios
items = [
    {
        'producto': 'camisa',
        'precio': 100,
    },
    {
        'producto': 'pantalones',
        'precio': 300,
    },
    {
        'producto': 'pantalones 2',
        'precio': 200
    }
]

# Función que recibe la lista de items y extrae los precios
def obtener_precios(lista_items):
    precios = []
    for item in lista_items:
        precios.append(item['precio'])
    return precios

# Usamos la función
precios = obtener_precios(items)
print(precios)
```

Supongamos que a la lista original le falta añadir un atributo, digamos: impuestos. En este caso no se puede hacer solo con lambda ya que la operación de añadir esa clave impuestos a la lista no se hace en una sola línea:

```python
def añadir_impuestos(item):
    item['impuestos'] = item['precio'] * .15
    return item
```

```
new_items = list(map(añadir_impuestos, items))
print(new_items)
```

Salida:

```
[{'producto': 'camisa', 'precio': 100, 'impuestos': 15.0}, {'producto': 'pantalones', 'precio': 300,
'impuestos': 45.0}, {'producto': 'pantalones 2', 'precio': 200, 'impuestos': 30.0}]
```

`filter()`. Filtrar elementos de una lista. Se seleccionan ciertos elementos para que pertenezcan a una nueva lista.

🧾 Sintaxis de `filter()`

```
filter(funcion, iterable)
```

✅ Parámetros:

- `funcion`: Una función que devuelve `True` o `False` para cada elemento. Solo los elementos que hagan que esta función devuelva `True` serán incluidos en el resultado.
- Puede ser una función definida con `def` o una `lambda`.
- `iterable`: Una secuencia como una lista, tupla, etc. > 🔁 Al igual que map(), filter() devuelve un objeto iterable tipo filter, por lo tanto, se suele convertir en list() para ver el resultado.

```
numbers = [1,2,3,4,5]
print(numbers)
new_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(new_numbers)
```

salida:

```
[1, 2, 3, 4, 5]
[2, 4]
```

```
numbers = [1, 2, 3, 4, 5]
print(numbers)

def es_par(x):
    return x % 2 == 0

new_numbers = []
for numero in numbers:
    if es_par(numero):
        new_numbers.append(numero)

print(new_numbers)
```

Otro ejemplo:

Dada la siguiente lista de diccionarios:

```
matches = [
    {
        'home_team': 'España',
        'away_team': 'Italia',
        'home_team_score': 3,
        'away_team_score': 1,
        'home_team_result': 'Win'
    },
    {
        'home_team': 'Francia',
        'away_team': 'Alemania',
```

```
        'home_team_score': 1,
        'away_team_score': 1,
        'home_team_result': 'Draw'
    },
    {
        'home_team': 'Portugal',
        'away_team': 'Suiza',
        'home_team_score': 5,
        'away_team_score': 0,
        'home_team_result': 'Win'
    },
]
# Necesitamos filtrar solo los equipos locales que ganaron
```

Necesitamos filtrar solo los equipos locales que ganaron

Veamos el estado original

```
# Veamos el estado original
print(matches)
```

salida:

```
[{'home_team': 'España', 'away_team': 'Italia', 'home_team_score': 3, 'away_team_score': 1,
'home_team_result': 'Win'}, {'home_team': 'Francia', 'away_team': 'Alemania', 'home_team_score': 1,
'away_team_score': 1, 'home_team_result': 'Draw'}, {'home_team': 'Portugal', 'away_team': 'Suiza',
'home_team_score': 5, 'away_team_score': 0, 'home_team_result': 'Win'}]
```

Veamos el nro de partidos

```
print(len(matches))
```

salida:

```
3
```

```
new_list = list(filter(lambda item: item['home_team_result'] == 'Win', matches))
print(new_list)
print(len(new_list))
```

salida:

```
[{'home_team': 'España', 'away_team': 'Italia', 'home_team_score': 3, 'away_team_score': 1,
'home_team_result': 'Win'}, {'home_team': 'Portugal', 'away_team': 'Suiza', 'home_team_score': 5,
'away_team_score': 0, 'home_team_result': 'Win'}]
2
```

```
lst1 = ['A', 'X', 'Y', '3', 'M', '4', 'D']
f1 = filter(str.isalpha, lst1)
print(list(f1))
```

salida:

```
['A', 'X', 'Y', 'M', 'D']
```

Código con filter y def (sin lambda)

```
def fun(n):
    if n % 5 == 0:
```

```
        return True
    else:
        return False

lst2 = [5, 10, 18, 27, 25]
f2 = filter(fun, lst2)
print(list(f2))
```

salida:

```
[5, 10, 25]
```

Código con lambda + filter

```
lst2 = [5, 10, 18, 27, 25]

# Usamos lambda en lugar de la función 'fun'
f2 = filter(lambda n: n % 5 == 0, lst2)

print(list(f2))
```

salida:

```
[5, 10, 25]
```

`reduce()`. “Reducir a un único valor”.

🧾 Sintaxis general de `reduce()`

```
from functools import reduce

reduce(funcion, iterable[, valor_inicial])
```

✅ Parámetros:

15. **`funcion`**: Una función que toma dos argumentos y devuelve un valor. Esta función se aplica acumulativamente a los elementos del iterable.
16. **`iterable`**: Una secuencia como lista, tupla, etc.
17. **`valor_inicial`** (opcional): Valor inicial que se usa como primer argumento en la primera llamada.

Ejemplo: Se tiene una lista de números enteros `numbers = [1, 2, 3, 4]` Utiliza la función `reduce()` del módulo `functools` junto con una función `lambda` para calcular la **suma acumulada** de todos los elementos de la lista. La función `lambda` debe recibir dos parámetros: un **acumulador (`counter`)** y un **elemento actual (`item`)**, y devolver la suma de ambos.

```
import functools

numbers = [1, 2, 3, 4]
result = functools.reduce(lambda counter, item: counter + item, numbers)
print(result)
```

salida:

```
10
```

Iteration	Counter	Item	Return
1	0	1	1

Iteration	Counter	Item	Return
2	1	2	3
3	3	3	6
4	6	4	10

Versión sin funciones de orden superior

```
numbers = [1, 2, 3, 4]

def sumar_lista(lista):
    total = 0
    for numero in lista:
        total += numero
    return total

result = sumar_lista(numbers)
print(result)
```

salida:

```
10
```

Iteration	Counter	Item	Return
1	0	1	1
2	1	2	3
3	3	3	6
4	6	4	10

Veamos otro ejemplo de `reduce()` con un diccionario para calcular el precio total:

```
import functools
items = [
    {
        'producto': 'camisa',
        'precio': 100,
    },
    {
        'producto': 'pantalones',
        'precio': 300
    },
    {
        'producto': 'pantalones 2',
        'precio': 200
    }
]

def accum(counter, item):
    print('counter => ',counter)
    print('item => ',item)
    return counter + item['precio']

total = functools.reduce(accum, items, 0)
print(total)
```

salida:

```
counter => 0
item => {'producto': 'camisa', 'precio': 100}
counter => 100
item => {'producto': 'pantalones', 'precio': 300}
counter => 400
item => {'producto': 'pantalones 2', 'precio': 200}
600
```

(Ejercicios P2 y P3: enunciado y solución en la Parte 3.)

Clase 22 · Programación modular y paquetes

Sesión 1: Programación modular

En las clases 18, 19 y 20 se trabajó la **programación orientada a objetos** y en la Clase 21 la **programación funcional**. Todos los ejemplos cabían en un solo archivo. Los programas reales, en cambio, tienen miles de líneas, los escriben varias personas y reutilizan código entre proyectos. Esta clase enseña a **repartir un programa en varios archivos y carpetas** que se importan entre sí: la **programación modular**.

Sección 1. ¿Qué es la programación modular?

Definición

La **programación modular** consiste en dividir un programa en **piezas independientes**, llamadas **módulos**, cada una responsable de **una tarea concreta**. Las piezas se conectan entre sí **importándose**.

Término	Significado en Python
Módulo	Un archivo `.py`. Su nombre es el del archivo sin la extensión: `tarifas.py` es el módulo `tarifas`
Importar	Pedirle a Python que cargue un módulo para usar lo que contiene (variables, funciones, clases)
Paquete	Una carpeta que agrupa varios módulos relacionados (Sección 7)
Biblioteca o **librería**	Conjunto de módulos y paquetes listos para usar: la **biblioteca estándar** viene con Python; las **externas** se instalan con `pip`
Espacio de nombres	El «apellido» de cada nombre: `tarifas.coste_envio` y `aduanas.coste_envio` son dos funciones distintas aunque se llamen igual

Las tres piezas encajan unas dentro de otras: una **librería** contiene **paquetes**, y cada paquete contiene **módulos**.

Cuando se desarrolla un programa en Python, normalmente se reparte en varios archivos: algunos se convierten en **módulos**, otros se agrupan en **paquetes** y uno de ellos es el **punto de entrada** (*entry point*) o **programa principal** (*main*). Es buena práctica llamar `main.py` a ese archivo.

Ventajas de dividir el programa en módulos: 1. **Reutilización**: una función escrita una vez en un módulo se importa en todos los programas que la necesiten. 2. **Mantenimiento**: si cambia una regla (por ejemplo, una tarifa), se modifica en **un solo sitio**. 3. **Legibilidad**: archivos cortos, cada uno con un propósito claro. 4. **Trabajo en equipo**: cada persona trabaja en un módulo sin pisar el código de las demás. 5. **Pruebas**: cada módulo se puede probar por separado.

Sintaxis

```
mi_programa/
├── main.py          ← punto de entrada: importa y coordina
├── tarifas.py       ← módulo: reglas de precios
└── clientes.py      ← módulo: gestión de clientes
```

```python
# main.py
# Importar módulos
import tarifas
import clientes

# Función propia del programa principal
def procesar_envio(peso):
    print("=== PROCESAMIENTO DE ENVÍO ===")

    # Utilizar una función del módulo clientes
    clientes.mostrar_cliente()

    # Utilizar una función del módulo tarifas
    coste = tarifas.coste_envio(peso)

    # Obtener descuento del cliente
    descuento = clientes.obtener_descuento()

    # Aplicar descuento
    total = tarifas.aplicar_descuento(coste, descuento)

    print("------------------------------")
    print("Peso:", peso, "kg")
    print("Coste inicial:", coste, "€")
    print("Descuento:", descuento, "%")
    print("Total a pagar:", round(total, 2), "€")

# Llamar a la función principal
procesar_envio(2.0)
```

Ejemplo 1.1 · Del script único al primer módulo

> Una empresa de paquetería calcula el coste de cada envío con una tarifa base, un precio por kilo y un recargo para los envíos urgentes. Hasta ahora todo estaba en un único archivo, `monolito.py`. El departamento de atención al cliente y el de facturación necesitan **la misma función** de cálculo. En lugar de copiarla en cada programa, se saca a un módulo `tarifas.py`, y `main.py` lo importa.

> 📁 `ejemplo_1_1_primer_modulo/`

```
ejemplo_1_1_primer_modulo/
├── monolito.py
├── tarifas.py
└── main.py
```

Antes: `monolito.py` (todo en un archivo)

```python
TARIFA_BASE = 3.50
PRECIO_POR_KG = 0.80
RECARGO_URGENTE = 0.25


def coste_envio(peso_kg, urgente=False):
```

```
    coste = TARIFA_BASE + peso_kg * PRECIO_POR_KG
    if urgente:
        coste = coste * (1 + RECARGO_URGENTE)
    return round(coste, 2)


pedidos = [("A-101", 2.0, False), ("A-102", 7.5, True), ("A-103", 0.5, False)]
for codigo, peso, urgente in pedidos:
    print(f"{codigo}:{coste_envio(peso, urgente)} €")
```

Después: `tarifas.py` (el módulo, solo con las reglas de negocio)

```
"""Tarifas de envío de la empresa de paquetería."""

TARIFA_BASE = 3.50
PRECIO_POR_KG = 0.80
RECARGO_URGENTE = 0.25


def coste_envio(peso_kg, urgente=False):
    coste = TARIFA_BASE + peso_kg * PRECIO_POR_KG
    if urgente:
        coste = coste * (1 + RECARGO_URGENTE)
    return round(coste, 2)
```

Después: `main.py` (el programa que lo usa)

```
import tarifas

pedidos = [("A-101", 2.0, False), ("A-102", 7.5, True), ("A-103", 0.5, False)]
for codigo, peso, urgente in pedidos:
    print(f"{codigo}:{tarifas.coste_envio(peso, urgente)} €")

print("Tarifa base:", tarifas.TARIFA_BASE)
```

Ejecución:

```
python monolito.py
python main.py
```

Salida esperada de `monolito.py`:

```
A-101: 5.1 €
A-102: 11.88 €
A-103: 3.9 €
```

Salida esperada de `main.py`:

```
A-101: 5.1 €
A-102: 11.88 €
A-103: 3.9 €
Tarifa base: 3.5
```

Highlights:

1. `import tarifas` se escribe **sin** la extensión `.py`. Python busca el archivo `tarifas.py`, en este caso en la misma carpeta que `main.py`.
2. Todo lo que está definido en el módulo se usa con la forma **`modulo.nombre`**: `tarifas.coste_envio()`, `tarifas.TARIFA_BASE`.
3. El resultado es idéntico: modularizar **no cambia lo que hace** el programa, cambia **cómo está organizado**.
4. Si mañana sube la tarifa base, se cambia en `tarifas.py` y **todos** los programas que lo importan reciben el cambio.

5. La cadena entre comillas triples de la primera línea es el **docstring del módulo**: describe para qué sirve el archivo.

Sección 2. Crear e importar módulos

Definición

> Cualquier archivo `.py` es un módulo y puede contener **variables**, **funciones** y **clases**. Hay cuatro formas de importarlo:

Forma	Qué hace	Cómo se usa después
`import modulo`	Carga el módulo completo	`modulo.funcion()`
`import modulo as alias`	Igual, pero con un nombre más corto	`alias.funcion()`
`from modulo import nombre`	Trae **solo** ese nombre al programa	`funcion()` directamente
`from modulo import nombre as alias`	Trae ese nombre con otro nombre	`alias()`

> Existe una quinta forma, `from modulo import *`, que trae **todos** los nombres públicos del módulo. **No se recomienda**: no se sabe qué nombres entran y puede sobrescribir otros sin avisar.

Un módulo importado es un **objeto** de la clase `module`. Como cualquier objeto, tiene atributos:

Atributo	Contenido
`modulo.__name__`	Nombre del módulo
`modulo.__doc__`	Su docstring
`modulo.__file__`	Ruta del archivo `.py`
`dir(modulo)`	Lista de todos los nombres que contiene

```
import pyspark

# 1. Nombre del paquete
print(pyspark.__name__)

# 2. Documentación del paquete
print(pyspark.__doc__)

# 3. Ubicación del archivo
print(pyspark.__file__)

# 4. Contenido del paquete
print(dir(pyspark))
# ['Accumulator', 'AccumulatorParam', 'Any', 'BarrierTaskContext', 'BarrierTaskInfo', 'BasicProfiler',
'Broadcast', 'CPickleSerializer', 'Callable', 'HiveContext', 'InheritableThread', 'MarshalSerializer',
'Profiler', 'RDD', 'RDDBarrier', 'Row', 'SQLContext', 'SparkConf', 'SparkContext', 'SparkFiles',
'SparkJobInfo', 'SparkStageInfo', 'StatusTracker', 'StorageLevel', 'TaskContext', 'TypeVar', 'Union', '_F',
'_NoValue', '__all__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__',
'__package__', '__path__', '__spec__', '__version__', '_globals', 'accumulators', 'broadcast', 'cast',
'cloudpickle', 'conf', 'context', 'core', 'errors', 'files', 'find_spark_home', 'inheritable_thread_target',
'is_remote_only', 'java_gateway', 'join', 'keyword_only', 'logger', 'loose_version', 'profiler', 'rdd',
'rddsampler', 'resource', 'resultiterable', 'serializers', 'shuffle', 'since', 'sql', 'statcounter',
'status', 'storagelevel', 'sys', 'taskcontext', 'traceback_utils', 'util', 'version', 'wraps']
```

Analogía

> Pedir herramientas a un **almacén de bricolaje**: - `import ferreteria` es traerte **la caja de herramientas entera**: para usar el martillo dices «el martillo **de la caja**» (`ferreteria.martillo`). - `from ferreteria import martillo` es coger **solo el martillo** y dejarlo en tu mesa: lo usas directamente. - `from ferreteria import *` es **vaciar la caja entera sobre tu mesa**: tienes todo a mano, pero si ya tenías un martillo propio, ya no sabes cuál es cuál.

> La diferencia principal entre `import ferreteria` y `from ferreteria import *` está en **cómo accedes a las funciones, variables y clases del módulo y cómo se incorporan a tu espacio de nombres (*namespace*)**.

Vamos a verlo con un ejemplo sencillo.

```
# ferreteria.py

iva = 0.21

def calcular_precio(precio):
    return precio * (1 + iva)

def mostrar_producto(nombre):
    print(f"Producto: {nombre}")
```

Importa el módulo como un objeto y permite acceder a sus componentes utilizando su nombre.

```
import ferreteria

print(ferreteria.iva)

precio = ferreteria.calcular_precio(100)
print(precio)

ferreteria.mostrar_producto("Martillo")
```

Ventajas:

- Sabes de qué módulo procede cada función o variable.
- Evitas conflictos entre nombres de diferentes módulos.
- Es más fácil mantener y comprender el código. Importa directamente al espacio de nombres actual todos los nombres públicos exportados por el módulo.

```
from ferreteria import *

print(iva)

precio = calcular_precio(100)
print(precio)

mostrar_producto("Martillo")
```

Desventajas:

- No queda claro de dónde procede cada función o variable.
- Puede sobrescribir nombres que ya existían.
- Dificulta la lectura y el mantenimiento del código. Por defecto, `import *` no importa nombres que comienzan por `_`, salvo que el módulo defina `__all__` para controlar explícitamente qué nombres exporta.

Característica	`import ferreteria`	`from ferreteria import *`
Importa el módulo como objeto accesible	Sí	No directamente

Característica	`import ferreteria`	`from ferreteria import *`
Acceso a funciones	`ferreteria.calcular_precio()`	`calcular_precio()`
Acceso a variables	`ferreteria.iva`	`iva`
Riesgo de conflictos	Bajo	Alto
Recomendado	Sí	Generalmente no

Sintaxis

```
import modulo
import modulo as alias
from modulo import nombre1, nombre2
from modulo import nombre as alias
from modulo import *                    # desaconsejado

modulo.__name__
dir(modulo)
```

Ejemplo 2.1 · Primer contacto: un módulo con una función y una clase

> Crear un módulo es tan sencillo como crear un script: basta con escribir funciones o clases en un archivo `.py`. Se crea el módulo `saludos.py` con una función `saludar()` y una clase `Saludo` cuyo `__init__` muestra un mensaje. Desde `main.py`, **en la misma carpeta**, se usan de las dos formas básicas: importando el módulo completo y con `from ... import`.

> 📁 `ejemplo_2_1_modulo_saludos/`

```
ejemplo_2_1_modulo_saludos/
├── saludos.py
└── main.py
```

`saludos.py`

```python
def saludar():
    print("Hola, te estoy saludando desde la función saludar() "
          "del módulo saludos")


class Saludo:
    def __init__(self):
        print("Hola, te estoy saludando desde el __init__ "
              "de la clase Saludo")

    def saludar_persona(self, nombre):
        print(f"Hola {nombre}, desde la clase Saludo")
```

`main.py`

```python
# Forma 1: importar el módulo completo

import saludos

# Llamar a la función del módulo
saludos.saludar()

# Crear un objeto de la clase
objeto = saludos.Saludo()
```

```python
# Llamar al método del objeto
objeto.saludar_persona("Ana")
```

```python
# Forma 2: importar solo la función y la clase

from saludos import Saludo

objeto = Saludo()

objeto.saludar_persona("Ana")
```

Ejemplo 2.2 · Un módulo de funciones financieras

> El departamento de ventas de una cadena de electrodomésticos usa siempre los mismos cálculos: precio con IVA, aplicar un descuento y calcular el margen de beneficio. Se agrupan en un módulo `finanzas.py` de **funciones puras** (Clase 21) y constantes. Desde `main.py` se inspecciona el módulo como objeto y se usan sus funciones, incluso como argumento de `map()`.

> 📁 `ejemplo_2_2_modulo_finanzas/`

```
ejemplo_2_2_modulo_finanzas/
├── finanzas.py
└── main.py
```

`finanzas.py`

```python
"""Funciones financieras del departamento de ventas."""

IVA_GENERAL = 0.21
IVA_REDUCIDO = 0.10


def con_iva(importe, tipo=IVA_GENERAL):
    return round(importe * (1 + tipo), 2)


def aplicar_descuento(importe, porcentaje):
    return round(importe * (1 - porcentaje / 100), 2)


def margen(precio_venta, coste):
    return round((precio_venta - coste) / precio_venta * 100, 1)
```

`main.py`

```python
import finanzas

print(type(finanzas))
print(finanzas.__name__)
print(finanzas.__doc__)
print([nombre for nombre in dir(finanzas) if not nombre.startswith("_")])

print(finanzas.con_iva(100))
print(finanzas.con_iva(100, finanzas.IVA_REDUCIDO))
print(finanzas.aplicar_descuento(250, 15))
print(finanzas.margen(150, 90), "%")

precios = [120.0, 45.5, 80.0]
print(list(map(finanzas.con_iva, precios)))
```

Salida esperada:

```
<class 'module'>
finanzas
Funciones financieras del departamento de ventas.
['IVA_GENERAL', 'IVA_REDUCIDO', 'aplicar_descuento', 'con_iva', 'margen']
121.0
110.0
212.5
40.0 %
[145.2, 55.05, 96.8]
```

Highlights:

> 1. `type(finanzas)` confirma que un módulo es un **objeto** de la clase `module`, igual que en la Clase 19 se vio que todo en Python es un objeto.
> 2. `dir()` devuelve todos los nombres del módulo. Se filtran los que empiezan por `_` (atributos internos como `__name__` o `__file__`) para ver solo lo que ha escrito el programador.
> 3. Las constantes del módulo se escriben en **MAYÚSCULAS** por convención: indican que no deben modificarse.
> 4. `map(finanzas.con_iva, precios)` pasa la función del módulo **sin paréntesis**, como cualquier función (Clase 21). Una función no deja de ser un objeto por estar en otro archivo.

Ejemplo 2.3 · Las formas de importar y el conflicto de nombres

> La empresa de paquetería tiene dos módulos de tarifas, uno para envíos nacionales y otro para internacionales. **Los dos** tienen una función que se llama `calcular_coste()`. Se prueban las cuatro formas de importar y se provoca a propósito un **conflicto de nombres**.

> 📁 `ejemplo_2_3_formas_de_importar/`

```
ejemplo_2_3_formas_de_importar/
├── envios_nacionales.py
├── envios_internacionales.py
└── main.py
```

`envios_nacionales.py`

```python
"""Tarifas para envíos dentro de España."""

ZONA = "nacional"


def calcular_coste(peso_kg):
    return round(4.00 + peso_kg * 0.60, 2)
```

`envios_internacionales.py`

```python
"""Tarifas para envíos fuera de España."""

ZONA = "internacional"


def calcular_coste(peso_kg):
    return round(12.00 + peso_kg * 2.50, 2)
```

`main.py`

```
# 1. import modulo
import envios_nacionales
print("1.", envios_nacionales.ZONA, envios_nacionales.calcular_coste(4))

# 2. import modulo as alias
import envios_internacionales as intl
print("2.", intl.ZONA, intl.calcular_coste(4))

# 3. from modulo import nombre
from envios_nacionales import calcular_coste
print("3.", calcular_coste(4))

# 4. from modulo import nombre as alias
from envios_internacionales import calcular_coste as coste_intl
print("4.", coste_intl(4))

# 5. Conflicto: se importa otro nombre igual al que ya existía
from envios_internacionales import calcular_coste
print("5.", calcular_coste(4))
print("   ¿De qué módulo viene ahora?", calcular_coste.__module__)
```

Salida esperada:

```
1. nacional 6.4
2. internacional 22.0
3. 6.4
4. 22.0
5. 22.0
   ¿De qué módulo viene ahora? envios_internacionales
```

Highlights:

> 1. Con `import modulo` (formas 1 y 2) **no hay conflicto posible**: el nombre del módulo actúa como apellido y distingue las dos funciones `calcular_coste`.
> 2. Con `from ... import` (formas 3 y 4) la función se usa sin prefijo. Es más corto, pero se pierde la información de **dónde viene**.
> 3. En el paso 5, el segundo `from ... import calcular_coste` **sobrescribe sin avisar** el nombre del paso 3. Es el mismo problema que con `from modulo import *`, pero multiplicado por todos los nombres del módulo.
> 4. Toda función guarda en `__module__` el nombre del módulo donde se definió. Sirve para depurar este tipo de errores.
> 5. Las importaciones van normalmente **al principio** del archivo. Aquí se reparten por el código solo para comparar las formas una a una.

Ejemplo 2.4 · Un módulo de utilidades con comprehensions y lambdas

> El equipo de datos de una tienda online exporta del CRM la lista de clientes con nombres, emails y ciudades escritos de cualquier manera, y con algún cliente duplicado. Se crea el módulo `limpieza.py` con funciones puras que resuelven cada tarea con un tipo distinto de **comprehension**:

Función	Qué hace	Herramienta
`normalizar()`	Limpia espacios, mayúsculas y minúsculas	Comprehension de **lista**
`dominios()`	Dominios de email distintos	Comprehension de **set**
`indexar_por_email()`	Diccionario `email → cliente`	Comprehension de **diccionario**

Función	Qué hace	Herramienta
`gasto_por_ciudad()`	Gasto total de cada ciudad	Comprehension de diccionario + expresión generadora
`ordenar()`	Ordena según un criterio	Diccionario `ORDENES` de **lambdas**

`main.py` importa el módulo con un alias y, además, la constante `ORDENES`.

> 📁 `ejemplo_2_4_modulo_comprehensions/`

```
ejemplo_2_4_modulo_comprehensions/
├── limpieza.py
└── main.py
```

`limpieza.py`

```python
"""Utilidades para limpiar y organizar los datos de clientes del CRM."""

ORDENES = {
    "nombre": lambda c: c["nombre"],
    "gasto": lambda c: c["gasto"],
}


def normalizar(clientes):
    return [
        {
            "nombre": c["nombre"].strip().title(),
            "email": c["email"].strip().lower(),
            "ciudad": c["ciudad"].strip().title(),
            "gasto": c["gasto"],
        }
        for c in clientes
    ]


def dominios(clientes):
    return {c["email"].split("@")[1] for c in clientes}


def indexar_por_email(clientes):
    return {c["email"]: c for c in clientes}


def gasto_por_ciudad(clientes):
    ciudades = sorted({c["ciudad"] for c in clientes})
    return {
        ciudad: round(sum(c["gasto"] for c in clientes if c["ciudad"] == ciudad), 2)
        for ciudad in ciudades
    }


def ordenar(clientes, criterio, descendente=False):
    return sorted(clientes, key=ORDENES[criterio], reverse=descendente)
```

`main.py`

```python
import limpieza as lp
from limpieza import ORDENES

crudos = [
    {"nombre": "  ana lópez", "email": "ANA@Gmail.com ", "ciudad": "madrid", "gasto": 320.5},
    {"nombre": "LUIS PÉREZ ", "email": "luis@empresa.es", "ciudad": " Sevilla", "gasto": 89.9},
    {"nombre": "marta ruiz", "email": "marta@gmail.com", "ciudad": "Madrid ", "gasto": 145.0},
    {"nombre": "ana lópez", "email": "ana@gmail.com", "ciudad": "madrid", "gasto": 60.0},
]
```

```
clientes = lp.normalizar(crudos)
print([c["nombre"] for c in clientes])
print("Dominios:", sorted(lp.dominios(clientes)))

indice = lp.indexar_por_email(clientes)
print(len(clientes), "registros ->", len(indice), "emails distintos")
print("Gasto por ciudad:", lp.gasto_por_ciudad(clientes))

mejores = lp.ordenar(clientes, "gasto", descendente=True)
print("Por gasto:", [(c["nombre"], c["gasto"]) for c in mejores[:2]])
print("Criterios disponibles:", list(ORDENES))
```

Salida esperada:

```
['Ana López', 'Luis Pérez', 'Marta Ruiz', 'Ana López']
Dominios: ['empresa.es', 'gmail.com']
4 registros -> 3 emails distintos
Gasto por ciudad: {'Madrid': 525.5, 'Sevilla': 89.9}
Por gasto: [('Ana López', 320.5), ('Marta Ruiz', 145.0)]
Criterios disponibles: ['nombre', 'gasto']
```

Highlights:

> 1. Las cuatro funciones son **puras**: reciben la lista y devuelven datos nuevos sin modificar `crudos` (Clase 21).
> 2. `dominios()` devuelve un **set**, así que `gmail.com` aparece una sola vez aunque lo usen tres clientes. Se ordena con `sorted()` para mostrarlo siempre igual, porque un set no tiene orden.
> 3. En `indexar_por_email()`, los dos registros de Ana tienen el mismo email tras normalizar. En un diccionario las claves no se repiten, así que **el último registro sustituye al primero**: hay 4 registros pero 3 emails distintos. Al deduplicar así hay que decidir conscientemente cuál se conserva.
> 4. `gasto_por_ciudad()` combina una comprehension de diccionario con una expresión generadora dentro de `sum()`. Madrid suma los tres registros: 320,5 + 145 + 60.
> 5. El módulo exporta tanto **funciones** como un **diccionario de lambdas**. Desde fuera se puede consultar qué criterios hay (`list(ORDENES)`) sin leer el código del módulo.
> 6. `import limpieza as lp` y `from limpieza import ORDENES` conviven sin problema: son dos formas de acceder al mismo módulo.

Sección 3. Qué ocurre al importar un módulo

Definición

> Cuando Python ejecuta `import modulo` por **primera vez** en un programa:

6. **Busca** el archivo `modulo.py` (Sección 6 explica dónde).
7. **Ejecuta todo su código**, de arriba abajo. Las líneas `def` y `class` crean funciones y clases; un `print()` suelto se ejecuta de verdad.
8. Crea un **objeto módulo** con todos los nombres resultantes.
9. Lo guarda en un diccionario llamado **`sys.modules`**, que funciona como caché de módulos ya cargados.

- Si después se vuelve a importar el mismo módulo, **desde cualquier archivo del programa**, Python lo encuentra en `sys.modules` y **no lo vuelve a ejecutar**: entrega el mismo objeto.
- Consecuencias: > • Un módulo se ejecuta **una sola vez** por programa. • Todos los archivos que importan un módulo **comparten el mismo objeto**. Si uno modifica un dato mutable del módulo (por ejemplo, un diccionario), los demás ven el cambio. Además, Python guarda una versión compilada del módulo en la carpeta **`__pycache__/`** (archivos `.pyc`) para cargarlo más rápido la próxima vez. Esa carpeta se crea sola, se puede borrar sin problema y **no se sube a Git**.

10. **Busca** el archivo `modulo.py` (Sección 6 explica dónde).

11. **Ejecuta todo su código**, de arriba abajo. Las líneas `def` y `class` crean funciones y clases; un `print()` suelto se ejecuta de verdad.
12. Crea un **objeto módulo** con todos los nombres resultantes.
13. Lo guarda en un diccionario llamado **`sys.modules`**, que funciona como caché de módulos ya cargados.

    - Un módulo se ejecuta **una sola vez** por programa.
    - Todos los archivos que importan un módulo **comparten el mismo objeto**. Si uno modifica un dato mutable del módulo (por ejemplo, un diccionario), los demás ven el cambio.

14. Lo guarda en un diccionario llamado **`sys.modules`**, que funciona como caché de módulos ya cargados.

Sintaxis

```
import sys

"nombre_modulo" in sys.modules        # ¿ya se ha cargado?
sys.modules["nombre_modulo"]          # el objeto módulo guardado en la caché
modulo_a.modulo_b is modulo_b         # True: es el mismo objeto
```

Ejemplo 3.1 · Un módulo de configuración compartido

> Una aplicación de informes financieros tiene un módulo `configuracion.py` con la moneda y el número de decimales con que se muestran los importes. El módulo `informes.py` lo importa para formatear cantidades, y `main.py` importa ambos. Se añade un `print()` en `configuracion.py` para ver **cuántas veces** se ejecuta su código y se comprueba que un cambio en la configuración hecho desde `main.py` se nota en `informes.py`.

> 📁 `ejemplo_3_1_que_pasa_al_importar/`

```
ejemplo_3_1_que_pasa_al_importar/
├── configuracion.py
├── informes.py
└── main.py
```

`configuracion.py`

```
print("  [cargando el módulo configuracion]")

CONFIG = {"moneda": "EUR", "decimales": 2}
```

`informes.py`

```
import configuracion


def formatear(importe):
    decimales = configuracion.CONFIG["decimales"]
    moneda = configuracion.CONFIG["moneda"]
    return f"{importe:.{decimales}f}{moneda}"
```

`main.py`

```
import sys

print("Inicio del programa")
import configuracion
import configuracion            # segunda vez: no se vuelve a ejecutar
```

```
import informes                # informes también importa configuracion
print("Fin de los import")

print("¿Está en sys.modules?", "configuracion" in sys.modules)
print("¿Es el mismo objeto?", informes.configuracion is configuracion)

print(informes.formatear(1234.7))
configuracion.CONFIG["moneda"] = "USD"
configuracion.CONFIG["decimales"] = 0
print(informes.formatear(1234.7))
```

Salida esperada:

```
Inicio del programa
  [cargando el módulo configuracion]
Fin de los import
¿Está en sys.modules? True
¿Es el mismo objeto? True
1234.70 EUR
1235 USD
```

Highlights:

> 1. El mensaje `[cargando el módulo configuracion]` aparece **una sola vez**, aunque el módulo se importa tres veces (dos en `main.py` y una dentro de `informes.py`).
> 2. `informes.configuracion is configuracion` es `True`: `main.py` e `informes.py` trabajan con **el mismo objeto módulo**, no con copias.
> 3. Por eso, al cambiar `CONFIG` desde `main.py`, `informes.formatear()` usa la moneda y los decimales nuevos.
> 4. Esta característica es útil para compartir configuración, pero peligrosa si se abusa: cualquier archivo puede cambiar un dato que otros usan. Es el mismo problema de las **funciones impuras** de la Clase 21.
> 5. Después de ejecutar el ejemplo aparece una carpeta `__pycache__/` junto a los módulos. Es normal.
> 6. Poner un `print()` suelto en un módulo solo tiene sentido para este experimento. En la práctica, un módulo **no debe ejecutar nada** al importarse, solo definir cosas. La Sección 4 explica cómo conseguirlo.

Ejemplo 3.2 · Un objeto compartido por todos los módulos

> Una tienda online quiere un **registro de auditoría** único en el que todos los departamentos anoten lo que hacen: pagos, envíos... Como un módulo se ejecuta **una sola vez**, si el módulo `auditoria.py` crea un objeto al importarse, **todos** los módulos que lo importen recibirán **el mismo objeto**. - `auditoria.py`: la clase `RegistroAuditoria` y una instancia única, `registro`. - `pagos.py` y `envios.py`: importan `registro` y anotan en él cada operación. - `main.py`: ejecuta operaciones y consulta el registro.

> 📁 `ejemplo_3_2_objeto_compartido/`

```
ejemplo_3_2_objeto_compartido/
├── auditoria.py
├── pagos.py
├── envios.py
└── main.py
```

`auditoria.py`

```
"""Registro de auditoría compartido por todo el programa."""
```

```python
class RegistroAuditoria:
    def __init__(self):
        self._eventos = []

    def anotar(self, origen, mensaje):
        self._eventos.append((len(self._eventos) + 1, origen, mensaje))

    def filtrar(self, origen):
        return [evento for evento in self._eventos if evento[1] == origen]

    def __len__(self):
        return len(self._eventos)

    def __iter__(self):
        for evento in self._eventos:
            yield evento


registro = RegistroAuditoria()          # instancia única, creada al importar el módulo
```

pagos.py

```python
"""Cobros a clientes."""

from auditoria import registro


def cobrar(cliente, importe):
    registro.anotar("pagos", f"cobro de{importe} € a{cliente}")
    return round(importe, 2)
```

envios.py

```python
"""Preparación de envíos."""

from auditoria import registro


def preparar_envio(cliente, articulos):
    registro.anotar("envios", f"{len(articulos)} artículos para{cliente}")
    return sorted(articulos, key=lambda articulo: articulo.lower())
```

main.py

```python
import auditoria
import pagos
from auditoria import registro
from envios import preparar_envio

pagos.cobrar("Bodegas Sol", 245.0)
print(preparar_envio("Bodegas Sol", ["vino tinto", "Cava", "aceite"]))
pagos.cobrar("Talleres Norte", 99.9)

print("Eventos registrados:", len(registro))
for numero, origen, mensaje in registro:
    print(f"{numero}. [{origen}]{mensaje}")

print("Solo pagos:", len(registro.filtrar("pagos")))
print("¿Mismo objeto en todos los módulos?", registro is auditoria.registro is pagos.registro)
```

Salida esperada:

```
['aceite', 'Cava', 'vino tinto']
Eventos registrados: 3
  1. [pagos] cobro de 245.0 € a Bodegas Sol
  2. [envios] 3 artículos para Bodegas Sol
```

```
  3. [pagos] cobro de 99.9 € a Talleres Norte
Solo pagos: 2
¿Mismo objeto en todos los módulos? True
```

Highlights:

> 1. La línea `registro = RegistroAuditoria()` se ejecuta **una sola vez**, la primera vez que algún archivo importa `auditoria`. Los demás imports reciben el objeto ya creado desde `sys.modules`.
> 2. `pagos.py` y `envios.py` anotan en **el mismo registro** sin conocerse entre sí: por eso los eventos salen numerados del 1 al 3 en el orden en que ocurrieron.
> 3. La comparación encadenada `a is b is c` confirma que `main.py`, `auditoria` y `pagos` apuntan al mismo objeto.
> 4. `__iter__` es un **generador** y `filtrar()` usa una **comprehension**: la clase permite recorrer y consultar los eventos sin dar acceso a la lista interna `_eventos`.
> 5. En `envios.py`, `key=lambda articulo: articulo.lower()` ordena sin distinguir mayúsculas. Sin la lambda, `Cava` saldría primero porque las mayúsculas van antes que las minúsculas.
> 6. Este patrón (un módulo que crea un objeto único y compartido) se usa mucho para registros (*logs*), configuraciones y conexiones. Es la forma más sencilla en Python de tener **una sola instancia** de una clase en todo el programa.

Sección 4. `__name__` y el patrón `if __name__ == "__main__":`

Definición

> Todo módulo tiene una variable especial `__name__` que Python rellena automáticamente:

Cómo se usa el archivo	Valor de `__name__`
Se **ejecuta directamente**: `python validaciones.py`	`"__main__"`
Se **importa** desde otro archivo: `import validaciones`	`"validaciones"` (el nombre del módulo)

Esto permite escribir un bloque de código que **solo se ejecuta cuando el archivo se lanza directamente**, y que se ignora cuando se importa:

```python
if __name__ == "__main__":
    # pruebas o programa principal
```

Usos habituales:

15. Añadir **pruebas rápidas** al final de un módulo, para comprobarlo de forma aislada sin que esas pruebas molesten al importarlo.
16. Marcar el **punto de entrada** de un programa en su `main.py`, normalmente llamando a una función `main()`.

Analogía.

Un **actor** de una compañía de teatro. Cuando ensaya **solo en su casa**, recita su papel completo de principio a fin para comprobar que se lo sabe. Cuando actúa **dentro de la obra**, no recita de golpe todo su papel: espera a que el director le dé paso en cada escena. El actor es el mismo, pero se comporta distinto según si «lo ejecutan» solo o «lo importan» en la obra.

Sintaxis

```python
# mi_modulo.py
def funcion_util():
    ...

if __name__ == "__main__":
    print(funcion_util())       # solo con: python mi_modulo.py
```

```python
# main.py
def main():
    ...                         # programa principal

if __name__ == "__main__":
    main()
```

Ejemplo 4.1 · Un módulo de validaciones con pruebas propias

> El equipo de datos de una distribuidora mayorista recibe fichas de clientes con códigos postales y teléfonos escritos de muchas formas. Se crea un módulo `validaciones.py` con dos funciones: - `es_codigo_postal_valido(cp)`: 5 dígitos y las dos primeras cifras entre 01 y 52 (las provincias españolas). - `normalizar_telefono(telefono)`: deja solo los dígitos y devuelve el número con el prefijo `+34`, o `None` si no tiene al menos 9 cifras. El módulo incluye pruebas rápidas dentro de `if __name__ == "__main__":`. Se ejecuta de dos formas: **directamente** y **importado** desde `main.py`.

> 📁 `ejemplo_4_1_name_main/`

```
ejemplo_4_1_name_main/
├── validaciones.py
└── main.py
```

`validaciones.py`

```python
"""Validaciones de los datos de clientes."""

print(f"  [validaciones.py] __name__ ={__name__!r}")


def es_codigo_postal_valido(cp):
    return len(cp) == 5 and cp.isdigit() and 1 <= int(cp[:2]) <= 52


def normalizar_telefono(telefono):
    digitos = "".join(c for c in telefono if c.isdigit())
    if len(digitos) < 9:
        return None
    return "+34" + digitos[-9:]


if __name__ == "__main__":
    print("  Pruebas rápidas del módulo:")
    print("  ", es_codigo_postal_valido("28013"), es_codigo_postal_valido("99001"))
    print("  ", normalizar_telefono("612 34 56 78"), normalizar_telefono("123"))
```

`main.py`

```python
import validaciones
```

```
print(f"[main.py] __name__ ={__name__!r}")

clientes = [
    ("Ferretería Ruiz", "28013", "91-555-12-34"),
    ("Bodegas Sol", "99001", "600 11 22 33"),
    ("Talleres Norte", "48001", "944 000"),
]

validos = [
    (nombre, validaciones.normalizar_telefono(telefono))
    for nombre, cp, telefono in clientes
    if validaciones.es_codigo_postal_valido(cp)
]
print(validos)
```

Ejecución 1: el módulo directamente

```
python validaciones.py
```

Salida esperada:

```
  [validaciones.py] __name__ = '__main__'
  Pruebas rápidas del módulo:
   True False
   +34612345678 None
```

Ejecución 2: el programa que lo importa

```
python main.py
```

Salida esperada:

```
  [validaciones.py] __name__ = 'validaciones'
[main.py] __name__ = '__main__'
[('Ferretería Ruiz', '+34915551234'), ('Talleres Norte', None)]
```

Highlights:

> 1. El **mismo archivo** `validaciones.py` tiene un `__name__` distinto según cómo se use: `'__main__'` al ejecutarlo y `'validaciones'` al importarlo.
> 2. Al importarlo desde `main.py`, las pruebas del bloque `if` **no se ejecutan**. El módulo solo aporta sus funciones.
> 3. El archivo que se lanza con `python` siempre es el `'__main__'` del programa: en la ejecución 2 lo es `main.py`.
> 4. `Bodegas Sol` se descarta porque su código postal empieza por `99`. `Talleres Norte` pasa el filtro, pero su teléfono tiene solo 6 cifras y queda como `None`: un dato pendiente de revisar.
> 5. El `print()` suelto de la línea 3 está solo para ver el valor de `__name__`. En un módulo real se quitaría.

Ejemplo 4.2 · Un módulo con una clase y sus propias pruebas

> El departamento financiero de una empresa exportadora convierte sus facturas en euros a otras divisas. Se crea el módulo `conversor.py` con la clase `ConversorDivisas`: - `convertir()`: convierte un importe a una divisa. - `convertir_lote()`: convierte una lista de importes, con una comprehension de **lista**. - `tabla()`: muestra cuánto vale un importe en todas las divisas, con una comprehension de **diccionario**. Al final del módulo, dentro de `if __name__ == "__main__":`, hay unas pruebas rápidas que comparan cada resultado con el valor esperado. `main.py` importa la clase y la usa con las facturas del día, combinándola con comprehensions, `zip()`, `filter()` y una `lambda`.

> 📁 `ejemplo_4_2_clase_con_pruebas/`

```
ejemplo_4_2_clase_con_pruebas/
├── conversor.py
└── main.py
```

`conversor.py`

```python
"""Conversión de importes en euros a otras divisas."""


class ConversorDivisas:
    def __init__(self, tasas):
        self.tasas = tasas                      # {divisa: unidades por 1 €}

    def convertir(self, importe_eur, divisa):
        return round(importe_eur * self.tasas[divisa], 2)

    def convertir_lote(self, importes_eur, divisa):
        return [self.convertir(importe, divisa) for importe in importes_eur]

    def tabla(self, importe_eur):
        return {divisa: self.convertir(importe_eur, divisa) for divisa in self.tasas}


if __name__ == "__main__":
    prueba = ConversorDivisas({"USD": 1.10, "GBP": 0.85})
    print("Prueba convertir:", prueba.convertir(100, "USD") == 110.0)
    print("Prueba lote:     ", prueba.convertir_lote([10, 20], "GBP") == [8.5, 17.0])
    print("Prueba tabla:    ", prueba.tabla(100) == {"USD": 110.0, "GBP": 85.0})
```

`main.py`

```python
from conversor import ConversorDivisas

tasas_hoy = {"USD": 1.08, "GBP": 0.86, "JPY": 162.5}
conversor = ConversorDivisas(tasas_hoy)

facturas = [("F-01", 1200.0), ("F-02", 349.9), ("F-03", 75.0)]

print("100 € equivalen a:", conversor.tabla(100))

en_usd = conversor.convertir_lote([importe for _, importe in facturas], "USD")
print("Facturas en USD:", en_usd)

grandes_gbp = {numero: conversor.convertir(importe, "GBP")
               for numero, importe in facturas if importe > 100}
print("Mayores de 100 € en GBP:", grandes_gbp)

numeros = [numero for numero, _ in facturas]
superan_300_usd = list(filter(lambda par: par[1] > 300, zip(numeros, en_usd)))
print("Superan 300 USD:", superan_300_usd)
```

Ejecución 1: el módulo directamente

```
python conversor.py
```

Salida esperada:

```
Prueba convertir: True
Prueba lote:      True
Prueba tabla:     True
```

Ejecución 2: el programa que lo importa

```
python main.py
```

Salida esperada:

```
100 € equivalen a: {'USD': 108.0, 'GBP': 86.0, 'JPY': 16250.0}
Facturas en USD: [1296.0, 377.89, 81.0]
Mayores de 100 € en GBP: {'F-01': 1032.0, 'F-02': 300.91}
Superan 300 USD: [('F-01', 1296.0), ('F-02', 377.89)]
```

Highlights:

1. Las pruebas usan **tasas inventadas y fáciles de calcular** (1,10 y 0,85), distintas de las de `main.py`. Así se comprueba la lógica de la clase con números que se pueden verificar de cabeza.
2. Al importar `conversor` desde `main.py`, las pruebas **no se ejecutan** y la salida queda limpia.
3. Cada prueba compara el resultado con el esperado y muestra `True` o `False`. Si algún día se modifica la clase y una prueba muestra `False`, se detecta el error antes de usar el módulo en otro programa.
4. Dentro de la clase, `convertir_lote()` y `tabla()` **reutilizan** `convertir()` en lugar de repetir la fórmula: si cambia el redondeo, se cambia en un solo método.
5. En `main.py`, `zip(numeros, en_usd)` empareja cada número de factura con su importe en dólares, y `filter()` con una `lambda` se queda con los pares que superan 300 USD.

Sección 5. Módulos con clases y funciones

Definición

Un módulo puede contener **clases**, igual que funciones. Al organizar un programa orientado a objetos en módulos, se suele seguir este reparto:

Módulo	Contenido típico
`modelos.py`	Las clases que representan los **datos** del negocio (`Producto`, `Cliente`, `Venta`…) y su jerarquía de herencia
`servicios.py`, `inventario.py`…	Clases o funciones que **trabajan con** esos modelos
`reglas.py`, `politicas.py`, `utilidades.py`…	**Funciones puras** con reglas de negocio y cálculos
`main.py`	El punto de entrada: crea los objetos y coordina

Reglas importantes:

- Para usar una clase de otro módulo se importa como cualquier otro nombre: `from modelos import Producto`.
- Una clase **derivada** puede estar en un módulo distinto de su clase **base**: basta con importar la base.
- `isinstance()` funciona con clases importadas, y reconoce también a las derivadas (Clase 20).
- La programación modular **combina** bien con la orientada a objetos y con la funcional: las clases organizan los datos, las funciones puras organizan las reglas y los módulos organizan los archivos.

Analogía.

En una **fábrica de muebles**, el departamento de diseño guarda los **planos** de cada mueble (las clases), el taller tiene las **máquinas** que cortan y montan siguiendo esos planos (los servicios) y la oficina técnica

> mantiene el **manual de normas** de calidad (las funciones puras). Ningún departamento guarda copias de los planos de otro: cuando el taller necesita uno, lo **pide** a diseño.

Sintaxis

```
# modelos.py
class Base:
    ...

class Derivada(Base):
    ...
```

```
# servicios.py
from modelos import Base

class Gestor:
    def agregar(self, obj):
        if not isinstance(obj, Base):
            raise TypeError(...)
```

```
# main.py
from modelos import Base, Derivada
from servicios import Gestor
```

Ejemplo 5.1 · Inventario de un almacén repartido en módulos

> Una cadena de supermercados gestiona el inventario de sus almacenes. El programa se reparte en dos módulos: - `modelos.py`: la clase `Producto` y su derivada `ProductoPerecedero`, que vende con un **30 % de descuento** los productos a los que les quedan **3 días o menos** para caducar. - `inventario.py`: la clase `Inventario`, que guarda los productos y ofrece consultas. Usa herramientas de la Clase 21: `filter()`, expresiones generadoras, `sorted()` con `key` y un generador en `__iter__`. `main.py` importa ambos módulos, crea el inventario y lo consulta.

> 📁 `ejemplo_5_1_modulos_con_clases/`

```
ejemplo_5_1_modulos_con_clases/
├── modelos.py
├── inventario.py
└── main.py
```

`modelos.py`

```
"""Clases del catálogo de productos."""


class Producto:
    def __init__(self, sku, nombre, precio, stock):
        self.sku = sku
        self.nombre = nombre
        self.precio = precio
        self.stock = stock

    @property
    def valor_stock(self):
        return round(self.precio * self.stock, 2)
```

```
    def precio_venta(self):
        return self.precio

    def __repr__(self):
        return f"{type(self).__name__}({self.sku!r},{self.nombre!r})"


class ProductoPerecedero(Producto):
    def __init__(self, sku, nombre, precio, stock, dias_para_caducar):
        super().__init__(sku, nombre, precio, stock)
        self.dias_para_caducar = dias_para_caducar

    def precio_venta(self):
        if self.dias_para_caducar <= 3:
            return round(self.precio * 0.70, 2)
        return self.precio
```

inventario.py

```
"""Gestión del inventario de un almacén."""

from modelos import Producto


class Inventario:
    CRITERIOS = {
        "precio": lambda p: p.precio_venta(),
        "stock": lambda p: p.stock,
        "valor": lambda p: p.valor_stock,
    }

    def __init__(self):
        self._productos = []

    def agregar(self, producto):
        if not isinstance(producto, Producto):
            raise TypeError("Solo se admiten objetos Producto")
        self._productos.append(producto)

    def __len__(self):
        return len(self._productos)

    def __iter__(self):
        for producto in self._productos:
            yield producto

    def bajo_stock(self, umbral):
        return list(filter(lambda p: p.stock < umbral, self._productos))

    def valor_total(self):
        return round(sum(p.valor_stock for p in self._productos), 2)

    def ordenar_por(self, criterio, descendente=False):
        return sorted(self._productos, key=self.CRITERIOS[criterio], reverse=descendente)
```

main.py

```
from modelos import Producto, ProductoPerecedero
from inventario import Inventario

almacen = Inventario()
almacen.agregar(Producto("SKU-01", "Aceite 1L", 6.40, 120))
almacen.agregar(ProductoPerecedero("SKU-02", "Yogur pack 4", 2.20, 15, dias_para_caducar=2))
almacen.agregar(Producto("SKU-03", "Arroz 1kg", 1.35, 8))
almacen.agregar(ProductoPerecedero("SKU-04", "Queso curado", 9.90, 30, dias_para_caducar=40))

print("Productos en almacén:", len(almacen))
for p in almacen:
    print(f"{p.nombre:<14} precio venta:{p.precio_venta():>5}")
```

```
print("Bajo stock (<20):", almacen.bajo_stock(20))
print("Valor total:", almacen.valor_total())
print("Por valor:", [p.sku for p in almacen.ordenar_por("valor", descendente=True)])

try:
    almacen.agregar("Leche")
except TypeError as error:
    print("TypeError:", error)
```

Salida esperada:

```
Productos en almacén: 4
  Aceite 1L      precio venta:   6.4
  Yogur pack 4   precio venta:  1.54
  Arroz 1kg      precio venta:  1.35
  Queso curado   precio venta:   9.9
Bajo stock (<20): [ProductoPerecedero('SKU-02', 'Yogur pack 4'), Producto('SKU-03', 'Arroz 1kg')]
Valor total: 1108.8
Por valor: ['SKU-01', 'SKU-04', 'SKU-02', 'SKU-03']
TypeError: Solo se admiten objetos Producto
```

Highlights:

1. `inventario.py` importa `Producto` desde `modelos.py` para poder comprobar los tipos con `isinstance()`. Un `ProductoPerecedero` también pasa la comprobación porque **es un** `Producto` (Clase 20).
2. `main.py` importa de **dos** módulos. Cada archivo tiene una responsabilidad: `modelos.py` sabe **qué es** un producto; `inventario.py` sabe **cómo gestionar** muchos.
3. El bucle `for` llama al precio de venta de cada objeto y cada clase responde con su versión: es **polimorfismo**, aunque las clases estén en otro archivo.
4. `__iter__` es un **generador** (Clase 21): permite recorrer el inventario con `for` sin dar acceso directo a la lista interna `_productos`.
5. El diccionario `CRITERIOS` guarda **lambdas** como valores: añadir un criterio nuevo de ordenación es añadir una entrada, sin tocar `ordenar_por()`.

Ejemplo 5.2 · Un módulo de reglas funcionales usado por una clase

Una tienda online de material deportivo cambia de **política de precios** según la campaña: precio normal, rebajas, Black Friday u outlet. El equipo decide separar las reglas de la lógica: - `politicas_precio.py`: **solo funciones puras**. Incluye una función que **fabrica** descuentos (clausura, Clase 21) y una función `componer()` que encadena varias políticas usando `reduce()`. Al final, un diccionario `POLITICAS` con las campañas disponibles. - `tarificador.py`: la clase `Tarificador`, que **recibe una política** (una función) y la aplica a un catálogo. Así, marketing puede añadir campañas en `politicas_precio.py` sin tocar la clase.

📁 `ejemplo_5_2_clases_y_funciones/`

```
ejemplo_5_2_clases_y_funciones/
├── politicas_precio.py
├── tarificador.py
└── main.py
```

`politicas_precio.py`

```
"""Políticas de precio: funciones puras que reciben un precio y devuelven otro."""

from functools import reduce
```

```python
def precio_normal(precio):
    return precio


def redondeo_comercial(precio):
    return int(precio) + 0.99


def crear_descuento(porcentaje):
    def descontar(precio):
        return round(precio * (1 - porcentaje / 100), 2)
    return descontar


def componer(*politicas):
    return lambda precio: reduce(lambda acumulado, f: f(acumulado), politicas, precio)


POLITICAS = {
    "normal": precio_normal,
    "rebajas": crear_descuento(20),
    "black_friday": crear_descuento(35),
    "outlet": componer(crear_descuento(40), redondeo_comercial),
}
```

---

tarificador.py

```python
"""Clase que aplica una política de precios a un catálogo."""

from politicas_precio import POLITICAS, precio_normal


class Tarificador:
    def __init__(self, politica=precio_normal):
        self.politica = politica

    def cambiar_politica(self, nombre):
        self.politica = POLITICAS[nombre]

    def tarifar(self, catalogo):
        return {producto: self.politica(precio) for producto, precio in catalogo.items()}
```

---

main.py

```python
from tarificador import Tarificador
from politicas_precio import POLITICAS, crear_descuento

catalogo = {"Zapatillas": 89.90, "Mochila": 45.00, "Gorra": 19.50}

caja = Tarificador()
print("Normal:      ", caja.tarifar(catalogo))

caja.cambiar_politica("black_friday")
print("Black Friday:", caja.tarifar(catalogo))

caja.cambiar_politica("outlet")
print("Outlet:      ", caja.tarifar(catalogo))

caja.politica = crear_descuento(10)               # política creada al momento
print("Socios -10%: ", caja.tarifar(catalogo))

print("Políticas disponibles:", list(POLITICAS))
```

Salida esperada:

```
Normal:       {'Zapatillas': 89.9, 'Mochila': 45.0, 'Gorra': 19.5}
Black Friday: {'Zapatillas': 58.44, 'Mochila': 29.25, 'Gorra': 12.68}
Outlet:       {'Zapatillas': 53.99, 'Mochila': 27.99, 'Gorra': 11.99}
```

```
Socios -10%:  {'Zapatillas': 80.91, 'Mochila': 40.5, 'Gorra': 17.55}
Políticas disponibles: ['normal', 'rebajas', 'black_friday', 'outlet']
```

Highlights:

1. Un módulo puede **importar otros módulos**: `politicas_precio.py` importa `reduce` de la biblioteca estándar y `tarificador.py` importa `politicas_precio.py`. Se forma una cadena de dependencias.
2. El atributo `politica` de la clase guarda **una función**. Cambiar de campaña es asignar otra función: la clase no sabe ni le importa cómo se calcula el precio.
3. `componer(crear_descuento(40), redondeo_comercial)` crea una política nueva encadenando dos: primero descuenta un 40 % y después redondea a `,99`. `reduce()` pasa el precio por cada función en orden.
4. El diccionario `POLITICAS` se crea **al importar** el módulo, una sola vez (Sección 3).
5. La regla de diseño es: **lo que cambia a menudo** (las reglas de negocio) va en su propio módulo, separado de **lo que cambia poco** (la clase que las aplica).

Sección 6. Dónde busca Python los módulos

Definición

Al ejecutar `import nombre`, Python busca el módulo **en orden** en una lista de carpetas guardada en **`sys.path`**. Usa el **primero** que encuentra: 1. **La carpeta del script que se está ejecutando** (`sys.path[0]`). Por eso `main.py` encuentra los módulos de su misma carpeta, y por eso **un módulo guardado en otra carpeta no se encuentra** con un simple `import`. La solución ordenada es agrupar los módulos en **paquetes** (Sección 7) colocados junto al `main.py`. 2. Las carpetas de la **biblioteca estándar** de Python. 3. La carpeta **`site-packages`** del entorno virtual activo, donde `pip` instala las librerías externas.

Errores típicos al importar:

Error	Causa habitual
`ModuleNotFoundError: No module named 'x'`	El archivo no existe, el nombre está mal escrito, se ejecuta desde otra carpeta, o la librería externa no está instalada en el `.venv` activo
`ImportError: cannot import name 'y' from 'x'`	El módulo existe, pero no contiene el nombre `y`
`AttributeError: module 'x' has no attribute 'y'`	Igual que el anterior, usando `x.y`. Muy a menudo: **un archivo propio tiene el mismo nombre que un módulo estándar** y lo está tapando

⚠️ **Nunca** llames a tus archivos como módulos de Python o de librerías: `random.py`, `math.py`, `statistics.py`, `json.py`, `csv.py`, `datetime.py`, `pandas.py`… Como la carpeta del script se mira **primero**, tu archivo «tapa» al módulo original.

Analogía

Cuando buscas unas **llaves**, miras primero en el **bolsillo**, después en el **recibidor** y por último en el **cajón de la cocina**. Te quedas con las primeras que encuentras. Si en el bolsillo llevas unas llaves que se parecen a las de casa pero son de la oficina, **nunca llegarás al recibidor**, y la puerta de casa no se abrirá.

Sintaxis

```
import sys

sys.path              # lista de carpetas donde se buscan módulos, en orden
sys.path[0]           # la carpeta del script que se está ejecutando

modulo.__file__       # ruta del archivo desde el que se cargó el módulo
```

Ejemplo 6.1 · Un archivo que tapa a un módulo estándar

> Un analista quería probar el módulo `random` para hacer un sorteo entre clientes y, sin pensarlo, llamó a su archivo de pruebas **`random.py`**. Después, en otro archivo de la misma carpeta, intenta usar el `random` de Python. Se inspecciona `sys.path`, se comprueba **de dónde** se ha cargado `random` y se provoca también un `ModuleNotFoundError`.

> 📁 `ejemplo_6_1_donde_busca_python/`

```
ejemplo_6_1_donde_busca_python/
├── random.py        ← ¡nombre peligroso!
└── main.py
```

`random.py`

```
# Archivo creado por error con el mismo nombre que un módulo estándar
def sorteo():
    return "Esto NO es el módulo random de Python"
```

`main.py`

```
import os
import sys

print("Primera ruta de búsqueda:", os.path.basename(sys.path[0]))
print("Número de rutas > 1:", len(sys.path) > 1)

import random
print("random se ha cargado desde:", os.path.basename(random.__file__))
try:
    print(random.randint(1, 6))
except AttributeError as error:
    print("AttributeError:", error)

try:
    import tarifas_2027
except ModuleNotFoundError as error:
    print("ModuleNotFoundError:", error)
```

Salida esperada:

```
Primera ruta de búsqueda: ejemplo_6_1_donde_busca_python
Número de rutas > 1: True
random se ha cargado desde: random.py
AttributeError: module 'random' has no attribute 'randint'
ModuleNotFoundError: No module named 'tarifas_2027'
```

> En Python 3.13 o posterior, el mensaje del `AttributeError` añade una sugerencia: renombrar el archivo porque tapa al módulo estándar.

Highlights:

> 1. `os.path.basename()` se queda solo con el último trozo de una ruta. Se usa para no mostrar la ruta completa, que es distinta en cada ordenador.
> 2. `sys.path[0]` es la carpeta del ejemplo: Python mira ahí **antes** que en la biblioteca estándar.
> 3. `random.__file__` delata el problema: se ha cargado **nuestro** `random.py`, que no tiene `randint()`.
> 4. La solución es **renombrar el archivo** (por ejemplo, `prueba_sorteo.py`) y **borrar la carpeta** `__pycache__/` que se creó con el nombre antiguo.
> 5. Cuando aparezca un `ModuleNotFoundError` con una librería externa, lo primero es comprobar que la terminal muestra `(.venv)`: si el entorno virtual no está activo, Python no mira en su `site-packages`.

Sesión 2: Paquetes

Sección 7. Definición de Paquete en Python

> Un **paquete** es una **carpeta que contiene módulos** y que Python trata como una unidad. Cuando el número de módulos crece, hace falta poner orden: agruparlos por funcionalidad o según la aplicación donde se usan. Python aprovecha para ello la **estructura de carpetas** del sistema operativo.

Ventajas de usar paquetes:

17. **Agrupan** distintos módulos bajo un mismo nombre y permiten crear **jerarquías** de módulos, subpaquetes y submódulos.
18. Permiten **distribuir** el código como una librería instalable, de forma que otros programas lo usen igual que un módulo estándar.

- La carpeta lleva un archivo especial `__init__.py`. Puede estar vacío. Marca la carpeta como paquete y se **ejecuta automáticamente** la primera vez que se importa algo del paquete.
- Los módulos de un paquete se importan con **puntos**, como una ruta: `paquete.modulo`.
- Un paquete puede contener otros paquetes (**subpaquetes**), cada uno con su propio `__init__.py`: `paquete.subpaquete.modulo`. La jerarquía puede crecer tanto como se necesite.

En la imagen, `app` es una carpeta normal con dos módulos, mientras que `paquete` y `paquete/subpaquete` son paquetes porque contienen `__init__.py`. El `modulo3.py` se importaría como `paquete.subpaquete.modulo3`.

Imports absolutos y relativos

Tipo	Ejemplo	Cuándo se usa
Absoluto	`from facturacion.modelos import Cliente`	Desde fuera del paquete (por ejemplo, en `main.py`). Siempre funciona
Relativo	`from .modelos import Cliente`	Desde **dentro** del paquete, para importar un módulo hermano. El punto significa «este mismo paquete»

Los imports relativos solo funcionan cuando el módulo se usa **como parte del paquete**. Si se ejecuta directamente un archivo que los contiene (`python facturacion/factura.py`), Python lanza un `ImportError`.

El `__init__.py` como recepción del paquete

En `__init__.py` se pueden importar las clases y funciones principales del paquete para que quien lo use escriba `from facturacion import Factura` en lugar de `from facturacion.factura import Factura`. También se suelen definir:

- `__version__`: la versión del paquete.
- `__all__`: la lista de nombres públicos, que es lo que se importa con `from paquete import *`.

Sintaxis

```
main.py
mi_paquete/
├── __init__.py
├── modulo_a.py
├── modulo_b.py
└── subpaquete/
    ├── __init__.py
    └── modulo_c.py
```

```python
# Imports absolutos (desde fuera del paquete)
import mi_paquete.modulo_a
import mi_paquete.modulo_a as ma
from mi_paquete.modulo_a import ClaseA
from mi_paquete import ClaseA           # si __init__.py la importa
from mi_paquete.subpaquete.modulo_c import funcion_c

# Imports relativos (desde dentro del paquete)
from .modulo_a import ClaseA            # módulo hermano
from . import modulo_a                  # el módulo completo

# __init__.py
from .modulo_a import ClaseA
__version__ = "1.0.0"
__all__ = ["ClaseA"]
```

Ejemplo 7.1 · Un primer paquete de logística

> Una empresa de transporte agrupa su código de flota en un paquete `logistica` con dos módulos: - `vehiculos.py`: la clase `Vehiculo` y sus derivadas `Furgoneta` y `Camion`, cada una con su coste por kilómetro. - `rutas.py`: funciones puras para calcular la distancia y el coste de una ruta. Reciben cualquier vehículo y le piden su coste por kilómetro (polimorfismo).

El `__init__.py` está **vacío**. Desde `main.py` se importan los módulos del paquete de varias formas.

> 📁 `ejemplo_7_1_primer_paquete/`

```
ejemplo_7_1_primer_paquete/
├── logistica/
│   ├── __init__.py      (vacío)
│   ├── vehiculos.py
│   └── rutas.py
└── main.py
```

Para crear el `__init__.py` vacío desde la terminal:

```powershell
New-Item logistica\__init__.py -ItemType File
```

---

logistica/vehiculos.py

```
"""Vehículos de la flota."""


class Vehiculo:
    def __init__(self, matricula, capacidad_kg):
        self.matricula = matricula
        self.capacidad_kg = capacidad_kg

    def coste_por_km(self):
        return 0.0

    def puede_cargar(self, kg):
        return kg <= self.capacidad_kg

    def __str__(self):
        return f"{type(self).__name__}{self.matricula}"


class Furgoneta(Vehiculo):
    def coste_por_km(self):
        return 0.35


class Camion(Vehiculo):
    def coste_por_km(self):
        return 0.90
```

---

logistica/rutas.py

```
"""Cálculos sobre rutas de reparto (funciones puras)."""


def distancia_total(tramos_km):
    return round(sum(tramos_km), 1)


def coste_ruta(vehiculo, tramos_km):
    return round(vehiculo.coste_por_km() * distancia_total(tramos_km), 2)
```

---

main.py

```
import logistica
import logistica.rutas as rutas
from logistica.vehiculos import Furgoneta, Camion

flota = [Furgoneta("1234-KLM", 1200), Camion("5678-PQR", 18000)]
tramos = [120, 85.5, 40]
carga_kg = 3500

print("Distancia:", rutas.distancia_total(tramos), "km")
for vehiculo in flota:
    print(f"{vehiculo} ->{rutas.coste_ruta(vehiculo, tramos)} €")

aptos = [v.matricula for v in flota if v.puede_cargar(carga_kg)]
print(f"Aptos para{carga_kg} kg:", aptos)

print(type(logistica))
print(logistica.__name__, "|", rutas.__name__, "|", Furgoneta.__module__)
```

Salida esperada:

```
Distancia: 245.5 km
  Furgoneta 1234-KLM -> 85.92 €
  Camion 5678-PQR -> 220.95 €
Aptos para 3500 kg: ['5678-PQR']
```

```
<class 'module'>
logistica | logistica.rutas | logistica.vehiculos
```

Highlights:

> 1. `logistica.vehiculos` se lee como una ruta: «el módulo `vehiculos` del paquete `logistica`».
> 2. Un paquete importado también es un objeto `module`. Su nombre completo aparece en `__name__` y en el `__module__` de sus clases: `logistica.vehiculos`.
> 3. `rutas.py` **no importa** `vehiculos.py`: no le hace falta. Solo llama a `coste_por_km()`, y cualquier objeto que tenga ese método sirve (polimorfismo).
> 4. El alias `import logistica.rutas as rutas` evita escribir `logistica.rutas.coste_ruta(...)` cada vez.

Ejemplo 7.2 · Subpaquetes

> Se amplía el módulo de saludos del Ejemplo 2.1. Ahora el código se organiza en un paquete `paquete` con **dos subpaquetes**: `hola`, con el módulo `saludos.py`, y `adios`, con el módulo `despedidas.py`. Cada carpeta lleva su `__init__.py` **vacío**. Desde `main.py` se importa una función de un subpaquete y una clase del otro.

> 📁 `ejemplo_7_2_subpaquetes/`

```
ejemplo_7_2_subpaquetes/
├── main.py
└── paquete/
    ├── __init__.py
    ├── hola/
    │   ├── __init__.py
    │   └── saludos.py
    └── adios/
        ├── __init__.py
        └── despedidas.py
```

`paquete/hola/saludos.py`

```python
def saludar():
    print("Hola, te estoy saludando desde la función saludar() "
          "del módulo saludos")


class Saludo:
    def __init__(self):
        print("Hola, te estoy saludando desde el __init__ "
              "de la clase Saludo")
```

`paquete/adios/despedidas.py`

```python
def despedir():
    print("Adiós, me estoy despidiendo desde la función despedir() "
          "del módulo despedidas")


class Despedida:
    def __init__(self):
        print("Adiós, me estoy despidiendo desde el __init__ "
              "de la clase Despedida")
```

`main.py`

```
from paquete.hola.saludos import saludar
from paquete.adios.despedidas import Despedida

saludar()
Despedida()
```

Salida esperada:

```
Hola, te estoy saludando desde la función saludar() del módulo saludos
Adiós, me estoy despidiendo desde el __init__ de la clase Despedida
```

Highlights:

> 1. La ruta del import sigue la ruta de carpetas: `paquete.hola.saludos` es el archivo `paquete/hola/saludos.py`.
> 2. Cada nivel de la jerarquía tiene su `__init__.py`, aunque esté vacío. Así queda claro qué carpetas son paquetes.
> 3. `main.py` está **al lado** de la carpeta `paquete`: por eso Python la encuentra (Sección 6).
> 4. Solo se importa lo que se usa: la función `despedir()` y la clase `Saludo` existen, pero `main.py` no las necesita.

Ejemplo 7.3 · El `__init__.py` como recepción e imports relativos

> Una empresa de suministros para hostelería tiene un paquete `facturacion` con tres módulos que se usan entre sí: - `modelos.py`: las clases `Cliente` y `LineaFactura`. - `calculos.py`: funciones puras para la base imponible, el IVA y un **descuento por volumen** por tramos (5 % desde 1 000 € y 10 % desde 5 000 €), calculado con `reduce()`. - `factura.py`: la clase `Factura`, que importa los otros dos módulos con **imports relativos**. El `__init__.py` expone las clases principales, la versión del paquete y la lista `__all__`, de forma que `main.py` solo necesita `from facturacion import Cliente, Factura`.

> 📁 `ejemplo_7_3_init_e_imports_relativos/`

```
ejemplo_7_3_init_e_imports_relativos/
├── facturacion/
│   ├── __init__.py
│   ├── modelos.py
│   ├── calculos.py
│   └── factura.py
└── main.py
```

`facturacion/modelos.py`

```
"""Clases de datos de la facturación."""


class Cliente:
    def __init__(self, nif, nombre, tipo_iva=0.21):
        self.nif = nif
        self.nombre = nombre
        self.tipo_iva = tipo_iva


class LineaFactura:
    def __init__(self, concepto, unidades, precio_unitario):
        self.concepto = concepto
        self.unidades = unidades
        self.precio_unitario = precio_unitario
```

```
    @property
    def importe(self):
        return round(self.unidades * self.precio_unitario, 2)
```

facturacion/calculos.py

```
"""Funciones puras para calcular importes."""

from functools import reduce

TRAMOS_DESCUENTO = [(1000, 0.05), (5000, 0.10)]       # (base mínima, porcentaje)


def base_imponible(lineas):
    return round(sum(linea.importe for linea in lineas), 2)


def descuento_por_volumen(base):
    porcentaje = reduce(
        lambda actual, tramo: tramo[1] if base >= tramo[0] else actual,
        TRAMOS_DESCUENTO,
        0.0,
    )
    return round(base * porcentaje, 2)


def cuota_iva(base, tipo):
    return round(base * tipo, 2)
```

facturacion/factura.py

```
"""La clase Factura, que usa los otros módulos del paquete."""

from .modelos import Cliente, LineaFactura
from .calculos import base_imponible, descuento_por_volumen, cuota_iva


class Factura:
    _contador = 0

    def __init__(self, cliente):
        if not isinstance(cliente, Cliente):
            raise TypeError("Se esperaba un Cliente")
        Factura._contador += 1
        self.numero = f"F-{Factura._contador:04d}"
        self.cliente = cliente
        self.lineas = []

    def agregar_linea(self, concepto, unidades, precio_unitario):
        self.lineas.append(LineaFactura(concepto, unidades, precio_unitario))
        return self                                    # permite encadenar llamadas

    def total(self):
        base = base_imponible(self.lineas)
        base_neta = round(base - descuento_por_volumen(base), 2)
        return round(base_neta + cuota_iva(base_neta, self.cliente.tipo_iva), 2)

    def __str__(self):
        return f"{self.numero} |{self.cliente.nombre} | líneas:{len(self.lineas)} | total:{self.total()} €"
```

facturacion/__init__.py

```
"""Paquete de facturación de la empresa."""

from .modelos import Cliente, LineaFactura
from .factura import Factura
```

```
__version__ = "1.0.0"
__all__ = ["Cliente", "LineaFactura", "Factura"]
```

`main.py`

```
import facturacion
from facturacion import Cliente, Factura

print("Versión del paquete:", facturacion.__version__)

hosteleria = Cliente("B12345678", "Hostelería Levante SL")
panaderia = Cliente("A87654321", "Panadería Ortiz", tipo_iva=0.10)

f1 = (Factura(hosteleria)
      .agregar_linea("Cafetera industrial", 2, 950.00)
      .agregar_linea("Molinillo", 1, 320.00)
      .agregar_linea("Mantenimiento anual", 1, 180.00))
f2 = Factura(panaderia).agregar_linea("Amasadora", 1, 640.00)

for factura in [f1, f2]:
    print(factura)

print(facturacion.__all__)
print(Factura.__module__)
```

Salida esperada:

```
Versión del paquete: 1.0.0
F-0001 | Hostelería Levante SL | líneas: 3 | total: 2758.8 €
F-0002 | Panadería Ortiz | líneas: 1 | total: 704.0 €
['Cliente', 'LineaFactura', 'Factura']
facturacion.factura
```

Prueba adicional: desde la carpeta del ejemplo, ejecuta directamente el módulo que contiene imports relativos:

```
python facturacion\factura.py
```

Salida esperada (última línea del error):

```
ImportError: attempted relative import with no known parent package
```

Highlights:

1. Gracias al `__init__.py`, `main.py` importa `Factura` desde `facturacion` sin saber en qué módulo interno está. Si mañana `Factura` se mueve a otro archivo del paquete, `main.py` **no cambia**.
2. Aun así, `Factura.__module__` revela su ubicación real: `facturacion.factura`.
3. Cálculo de `F-0001`: base 2 400 € → descuento del 5 % (120 €) → base neta 2 280 € → IVA del 21 % (478,80 €) → **2 758,80 €**. `F-0002` no llega a 1 000 €, así que no tiene descuento y lleva IVA del 10 %.
4. Dentro del paquete, `factura.py` usa `from .modelos import ...`. El punto significa «el paquete en el que estoy», así que el código no depende del nombre de la carpeta.
5. El `ImportError` de la prueba adicional ocurre porque, al ejecutar `factura.py` directamente, Python no sabe que pertenece a un paquete. Los módulos internos de un paquete **se importan, no se ejecutan**: el punto de entrada es `main.py`.
6. `agregar_linea()` devuelve `self`, lo que permite **encadenar** varias llamadas en una sola expresión. Los paréntesis alrededor de `f1 = (...)` permiten repartirla en varias líneas.

Sección 8. La biblioteca estándar: módulos que ya vienen con Python

Definición

> Python incluye cientos de módulos listos para usar, sin instalar nada: la **biblioteca estándar**. Se importan exactamente igual que los módulos propios. Algunos de los más usados en ingeniería de datos:

Módulo	Para qué sirve	Ejemplos
`math`	Funciones matemáticas	`math.ceil()`, `math.floor()`, `math.sqrt()`
`statistics`	Estadística básica	`statistics.mean()`, `statistics.median()`, `statistics.stdev()`
`collections`	Estructuras de datos especializadas	`Counter` (contar), `defaultdict`, `namedtuple`
`datetime`	Fechas y horas	`date`, `datetime`, `timedelta`
`functools`	Herramientas para funciones	`reduce()`, `partial()`
`itertools`	Herramientas para iteradores	`chain()`, `islice()`, `groupby()`
`random`	Números aleatorios	`random.randint()`, `random.choice()`
`os`, `sys`	Sistema operativo e intérprete	`os.path`, `sys.path`

Las librerías **externas** (pandas, PySpark…) no vienen con Python: las publica la comunidad, se instalan con `pip install` dentro del `.venv` y se anotan en `requirements.txt`, como se hizo en la Clase 18. Una vez instaladas, se importan con la misma sintaxis.

Dónde consultarlas:

- Biblioteca estándar: The Python Standard Library (también en español).
- Librerías de terceros: PyPI · The Python Package Index.

Sintaxis

```
import math
import statistics
from collections import Counter
from datetime import date, timedelta
from functools import partial

partial(funcion, argumento=valor)     # nueva función con ese argumento ya fijado
```

Ejemplo 8.1 · El cierre semanal de una cafetería

> Una cadena de cafeterías analiza los tickets de un local para preparar el pedido semanal. Se resuelven cinco tareas, cada una con un módulo distinto de la biblioteca estándar: 1. Los **dos productos más vendidos** (`collections.Counter`). 2. El **ticket medio** y la **mediana** (`statistics`). 3. Cuántos **sacos de café de 2 kg** pedir si se consumen 7,3 kg a la semana (`math.ceil`, que redondea hacia arriba). 4. La **fecha del próximo pedido**, una semana después del último (`datetime`). 5. Una función de IVA con el **tipo de hostelería (10 %) ya fijado**, a partir de una función general (`functools.partial`).

> 📁 `ejemplo_8_1_biblioteca_estandar/`

```
ejemplo_8_1_biblioteca_estandar/
└── main.py
```

`main.py`

```python
import math
import statistics
from collections import Counter
from datetime import date, timedelta
from functools import partial

tickets = [("café", 1.40), ("tostada", 2.10), ("café", 1.40), ("zumo", 2.80),
           ("café", 1.40), ("tostada", 2.10), ("croissant", 1.80)]

# collections: contar sin escribir el bucle
mas_vendidos = Counter(producto for producto, _ in tickets)
print("Más vendidos:", mas_vendidos.most_common(2))

# statistics: medidas estadísticas
importes = [precio for _, precio in tickets]
print("Ticket medio:", round(statistics.mean(importes), 2),
      "| mediana:", statistics.median(importes))

# math: funciones matemáticas
consumo_kg, saco_kg = 7.3, 2
print("Sacos de café a pedir:", math.ceil(consumo_kg / saco_kg))

# datetime: fechas
ultimo_pedido = date(2026, 11, 10)
proximo_pedido = ultimo_pedido + timedelta(days=7)
print("Próximo pedido:", proximo_pedido.isoformat())

# functools: fijar argumentos de una función ya existente
def con_iva(importe, tipo):
    return round(importe * (1 + tipo), 2)

con_iva_hosteleria = partial(con_iva, tipo=0.10)
print("Con IVA del 10 %:", list(map(con_iva_hosteleria, [1.40, 2.10, 2.80])))
```

Salida esperada:

```
Más vendidos: [('café', 3), ('tostada', 2)]
Ticket medio: 1.86 | mediana: 1.8
Sacos de café a pedir: 4
Próximo pedido: 2026-11-17
Con IVA del 10 %: [1.54, 2.31, 3.08]
```

Highlights:

1. `Counter` recibe una expresión generadora (Clase 21) y cuenta cuántas veces aparece cada producto. `most_common(2)` devuelve los dos primeros como pares `(producto, veces)`.
2. `math.ceil(3.65)` da `4`: aunque sobre algo de café, no se puede pedir medio saco.
3. Sumar un `timedelta` a una `date` da otra fecha: Python se encarga de los cambios de mes y de año.
4. `partial()` es una alternativa a las clausuras de la Clase 21: crea una función nueva a partir de otra, con algún argumento ya fijado. `con_iva_hosteleria` solo necesita el importe, así que encaja directamente en `map()`.
5. Antes de escribir una función de utilidad, conviene comprobar si la biblioteca estándar ya la tiene.

Sección 9. Un proyecto modular completo: pipeline ETL

Definición

> Un **pipeline ETL** (*Extract, Transform, Load*) es el proceso típico de la ingeniería de datos: 1. **Extracción**: leer los datos en bruto de su origen. 2. **Transformación**: limpiar, validar y convertir los datos. 3. **Carga**: guardar el resultado donde se va a consultar. Cada etapa es una responsabilidad distinta, así que encaja de forma natural en **un módulo por etapa**, agrupados en un paquete.

Buenas prácticas de organización

Práctica	Por qué
Un módulo, una responsabilidad	Si el nombre de un módulo necesita un «y» (`clientes_y_facturas.py`), probablemente son dos módulos
Nombres de módulos en **minúsculas con guiones bajos** (`snake_case`), sin espacios ni tildes	Son nombres de variables para Python: `import mis datos` no es válido
Imports **al principio** del archivo, en tres bloques separados por una línea en blanco: 1) biblioteca estándar, 2) librerías externas, 3) módulos propios	Es lo que recomienda la guía PEP 8: de un vistazo se ve de qué depende el archivo
Evitar `from modulo import *`	No se sabe qué nombres entran ni de dónde vienen
Los módulos **solo definen**; el código que se ejecuta va en `main()` bajo `if __name__ == "__main__":`	Importar un módulo no debe tener efectos inesperados
Evitar **imports circulares** (A importa B y B importa A)	Python puede encontrarse un módulo a medio cargar y lanzar `ImportError`. Suele indicar que hay que sacar lo común a un tercer módulo
Nombres internos con **un guion bajo** (`_es_decimal`)	Convención de la Clase 19: indica que es un detalle interno del módulo y no forma parte de lo que ofrece

---

Sintaxis

> La estructura habitual del programa principal (`main.py`) es: primero los imports, ordenados en tres bloques; después el código propio (funciones y clases); y al final el punto de entrada real, protegido con `if __name__ == "__main__":`.

Estructura del programa principal

Aplicada a un archivo real:

```
"""Docstring del módulo: qué hace este archivo."""

# 1. Biblioteca estándar
import math
from datetime import date

# 2. Librerías externas (instaladas con pip)
# import pandas as pd

# 3. Módulos propios
from mi_paquete import mi_funcion


def main():
    ...
```

```python
if __name__ == "__main__":
    main()
```

Ejemplo 9.1 · Pipeline de ventas de una cadena de tiendas

> Una cadena de tiendas de café e infusiones recibe cada noche las ventas de sus locales en un texto con formato CSV. Los datos llegan **sucios**: espacios sobrantes, mayúsculas mezcladas y algunas filas con las unidades escritas en letra o vacías.

Se construye el paquete `pipeline_ventas` con un módulo por responsabilidad:

Módulo	Responsabilidad	Técnica principal
`modelos.py`	Clase `Venta`	OOP
`extraccion.py`	Entregar las líneas de datos una a una	Generador
`transformacion.py`	Limpiar, validar y convertir a objetos `Venta`	Funciones puras + `map()` / `filter()`
`carga.py`	Guardar las ventas y responder consultas	Clase `AlmacenVentas`
`orquestador.py`	Unir las tres etapas	Composición de funciones
`__init__.py`	Exponer solo `ejecutar_pipeline()`	`__all__`

`main.py` solo ejecuta el pipeline y muestra los resultados.

> 📁 `ejemplo_9_1_pipeline_modular/`

```
ejemplo_9_1_pipeline_modular/
├── pipeline_ventas/
│   ├── __init__.py
│   ├── modelos.py
│   ├── extraccion.py
│   ├── transformacion.py
│   ├── carga.py
│   └── orquestador.py
└── main.py
```

`pipeline_ventas/modelos.py`

```python
"""Clases de datos del pipeline."""


class Venta:
    def __init__(self, tienda, producto, unidades, precio):
        self.tienda = tienda
        self.producto = producto
        self.unidades = unidades
        self.precio = precio

    @property
    def importe(self):
        return round(self.unidades * self.precio, 2)

    def __repr__(self):
        return f"Venta({self.tienda},{self.producto},{self.importe})"
```

`pipeline_ventas/extraccion.py`

```python
"""Extracción: lee los registros en bruto sin cargarlos todos a la vez."""


def leer_registros(texto):
    lineas = texto.strip().split("\n")
    for linea in lineas[1:]:                  # se salta la cabecera
        yield linea
```

pipeline_ventas/transformacion.py

```python
"""Transformación: funciones puras que limpian y convierten los registros."""

from .modelos import Venta


def parsear(linea):
    return [campo.strip() for campo in linea.split(",")]


def _es_decimal(texto):
    try:
        float(texto)
        return True
    except ValueError:
        return False


def es_valido(campos):
    return len(campos) == 4 and campos[2].isdigit() and _es_decimal(campos[3])


def a_venta(campos):
    tienda, producto, unidades, precio = campos
    return Venta(tienda.title(), producto.lower(), int(unidades), float(precio))


def transformar(lineas):
    campos = map(parsear, lineas)
    validos = filter(es_valido, campos)
    return map(a_venta, validos)
```

pipeline_ventas/carga.py

```python
"""Carga: almacena las ventas y ofrece consultas."""


class AlmacenVentas:
    def __init__(self):
        self._ventas = []

    def cargar(self, ventas):
        for venta in ventas:
            self._ventas.append(venta)

    def __len__(self):
        return len(self._ventas)

    def total_por_tienda(self):
        totales = {}
        for venta in self._ventas:
            totales[venta.tienda] = round(totales.get(venta.tienda, 0) + venta.importe, 2)
        return totales

    def top_productos(self, n):
        totales = {}
        for venta in self._ventas:
            totales[venta.producto] = round(totales.get(venta.producto, 0) + venta.importe, 2)
        return sorted(totales.items(), key=lambda par: par[1], reverse=True)[:n]
```

pipeline_ventas/orquestador.py

```
"""Orquestador: une las tres etapas del pipeline."""

from .extraccion import leer_registros
from .transformacion import transformar
from .carga import AlmacenVentas


def ejecutar_pipeline(texto):
    almacen = AlmacenVentas()
    almacen.cargar(transformar(leer_registros(texto)))
    return almacen
```

---

pipeline_ventas/__init__.py

```
"""Pipeline ETL de ventas: extracción, transformación y carga."""

from .orquestador import ejecutar_pipeline

__all__ = ["ejecutar_pipeline"]
```

---

main.py

```
"""Punto de entrada del pipeline de ventas."""

from pipeline_ventas import ejecutar_pipeline

DATOS = """tienda,producto,unidades,precio
madrid, Café Molido ,3,4.50
barcelona,té verde,2,3.20
madrid,café molido,1,4.50
valencia,cacao,dos,2.80
barcelona,Café Molido,4,4.50
sevilla,té verde,,3.20
valencia,cacao,5,2.80
"""


def main():
    almacen = ejecutar_pipeline(DATOS)
    print("Ventas cargadas:", len(almacen))
    print("Total por tienda:", almacen.total_por_tienda())
    print("Top 2 productos:", almacen.top_productos(2))


if __name__ == "__main__":
    main()
```

Salida esperada:

```
Ventas cargadas: 5
Total por tienda: {'Madrid': 18.0, 'Barcelona': 24.4, 'Valencia': 14.0}
Top 2 productos: [('café molido', 36.0), ('cacao', 14.0)]
```

Highlights:

> 1. De 7 filas se cargan 5: la de Valencia con unidades `dos` y la de Sevilla con unidades vacías no pasan `es_valido()`. Por eso Sevilla no aparece en los totales.
> 2. `" Café Molido "`, `"café molido"` y `"Café Molido"` acaban siendo el mismo producto gracias a `strip()` y `lower()` en la transformación.
> 3. Toda la cadena es **perezosa** (Clase 21): `leer_registros()` es un generador y `transformar()` devuelve un `map`. Ninguna línea se procesa hasta que `AlmacenVentas.cargar()` recorre el resultado, y nunca se guarda una lista intermedia.

4. `main.py` **no conoce** las etapas internas: solo importa `ejecutar_pipeline`. Se podría cambiar por completo cómo se transforman los datos sin tocar `main.py`.
5. `_es_decimal()` lleva guion bajo porque es un detalle interno de `transformacion.py`. Fíjate en que el `__init__.py` tampoco lo expone.
6. Cada módulo se puede probar por separado. Por ejemplo, `transformacion.parsear(" madrid ,cacao,5,2.80")` se comprueba sin necesitar ni la extracción ni la carga.
7. Esta misma estructura (un paquete con extracción, transformación y carga, y un `main.py` que lo lanza) es la que se usará en los proyectos de datos del máster, cambiando el texto de ejemplo por ficheros, bases de datos o Spark.

Ejemplo 9.2 : del notebook a producción

En la clase se dijo que en ingeniería de datos **todo el código de producción es modular**: un pipeline se reparte en módulos de **extracción**, **transformación** y **carga**. Este ejemplo lo muestra de principio a fin: se parte de un análisis hecho en un **notebook de Jupyter** (`.ipynb`) y se convierte, paso a paso, en un **proyecto modular** listo para ejecutarse en producción.

Sección 1. El punto de partida: un notebook

Contexto

> **FrescoYa** es un supermercado online que reparte en Madrid, Sevilla y Valencia. Una analista de datos ha preparado en un notebook el **informe diario de ventas por ciudad**: 1. Lee los pedidos del día, que llegan en formato CSV. 2. Descarta los pedidos con el importe vacío o mal escrito. 3. Descarta los pedidos cancelados. 4. Calcula el importe de cada pedido con un **21 % de IVA**. 5. Suma las ventas por ciudad, de mayor a menor.

El notebook funciona, y el informe gusta tanto que la dirección pide que se **ejecute automáticamente cada noche**. Es el momento de llevarlo a producción.

> En un proyecto real, los pedidos llegarían de un fichero, una base de datos o una API. Aquí se escriben en un texto dentro del código para que el ejemplo se pueda ejecutar sin nada más.

El notebook `informe_ventas.ipynb`

Un notebook se ejecuta **celda a celda**, y cada celda muestra su salida debajo.

Celda [1] · Cargar los datos

```
datos = """id,cliente,ciudad,importe,estado
P001, Ana López ,madrid,45.50,entregado
P002,Luis Pérez,Sevilla,12.00,cancelado
P003,Marta Ruiz,MADRID,30.25,entregado
P004,Jorge Gil,valencia,,entregado
P005,Elena Sanz,sevilla,80.00,entregado
P006,Pablo Ríos,Valencia,22.40,en reparto
P007,Sara Mora,madrid,abc,entregado"""
lineas = datos.split("\n")[1:]
print("Pedidos leídos:", len(lineas))
```

```
Pedidos leídos: 7
```

Celda [2] · Limpiar y descartar errores

```
filas = []
rechazados = []
for l in lineas:
    c = [x.strip() for x in l.split(",")]
    try:
        imp = float(c[3])
        filas.append([c[0], c[1].title(), c[2].title(), imp, c[4].lower()])
    except ValueError:
        rechazados.append(c[0])
print("Rechazados por datos erróneos:", rechazados)
```

```
Rechazados por datos erróneos: ['P004', 'P007']
```

Celda [3] · Quitar cancelados y calcular el IVA

```
cancelados = [f[0] for f in filas if f[4] == "cancelado"]
filas = [f for f in filas if f[4] != "cancelado"]
for f in filas:
    f.append(round(f[3] * 1.21, 2))
print("Pedidos cancelados:", cancelados)
print("Pedidos procesados:", len(filas))
```

```
Pedidos cancelados: ['P002']
Pedidos procesados: 4
```

Celda [4] · Agregar por ciudad

```
resumen = {}
for f in filas:
    resumen[f[2]] = round(resumen.get(f[2], 0) + f[5], 2)
resumen = dict(sorted(resumen.items(), key=lambda x: x[1], reverse=True))
```

Celda [5] · Mostrar el informe

```
print("Ventas con IVA por ciudad:")
for ciudad, total in resumen.items():
    print(f"  {ciudad:<9}{total:>8.2f} €")
```

```
Ventas con IVA por ciudad:
  Sevilla     96.80 €
  Madrid      91.65 €
  Valencia    27.10 €
```

Sección 2. Por qué este notebook no puede ir a producción tal cual

> El notebook es perfecto para **explorar** datos y probar ideas. Pero tiene problemas cuando el código se tiene que ejecutar solo, todas las noches y durante años:

Problema en el notebook	Dónde se ve	Consecuencia en producción
Depende del orden de ejecución	La celda 4 usa `f[5]`, una columna que solo existe si antes se ejecutó la celda 3. Si se salta, falla con `IndexError`; si se repite la 3, cada pedido acumula columnas de IVA duplicadas	Resultados o errores distintos según quién lo ejecute y cómo
Todo son variables globales	`filas`, `resumen` y `rechazados` viven en el notebook y cualquier celda puede cambiarlas	Errores difíciles de encontrar (son funciones impuras, Clase 21)
Números «mágicos» repartidos	El `1.21` del IVA y el texto `"cancelado"` están escritos dentro del código	Si cambia el IVA, hay que buscarlo celda a celda

Problema en el notebook	Dónde se ve	Consecuencia en producción
Datos sin estructura	Un pedido es una lista: `f[2]` es la ciudad y `f[5]` el total con IVA	Nadie recuerda qué es `f[5]`, y añadir un campo descoloca todos los índices
Nada se puede reutilizar	La limpieza solo existe dentro de este notebook	Otro informe que necesite limpiar pedidos tiene que copiar y pegar
No se puede probar por partes	La validación está mezclada con la lectura y la limpieza	No se puede comprobar una regla sin ejecutarlo todo
Difícil de automatizar	Un `.ipynb` está pensado para abrirse y ejecutarse a mano	Las herramientas de automatización lanzan programas `.py` con un punto de entrada claro

Sección 3. Paso a paso de la conversión

Paso	Qué se hace	Dónde acaba
1	Identificar las **etapas** del notebook: qué celdas extraen, cuáles transforman y cuáles cargan	Un módulo por etapa
2	Sacar los números y textos fijos a un **módulo de configuración**	`config.py`
3	Sustituir las listas por posiciones (`f[2]`, `f[5]`) por una **clase** con atributos con nombre	`modelos.py`
4	Convertir las celdas de limpieza en **funciones puras** que reciben datos y devuelven datos nuevos	`transformacion.py`
5	Llevar la agregación a una **clase** que guarda los pedidos y responde consultas	`carga.py`
6	Escribir una función que una las etapas **siempre en el mismo orden**	`orquestador.py`
7	Agrupar todo en un **paquete** y exponer solo lo necesario	`etl/__init__.py`
8	Crear el **punto de entrada** con `main()` y `if __name__ == "__main__":`	`main.py`
9	Comprobar que la versión modular da **exactamente la misma salida** que el notebook	Ejecución final

Correspondencia entre celdas y módulos:

Celda del notebook	Módulo de producción
Celda 1: cargar los datos	`extraccion.py`
Celda 2: limpiar y descartar errores	`transformacion.py` → `parsear()`, `es_valido()`, `a_pedido()`
Celda 3: quitar cancelados y calcular el IVA	`transformacion.py` → `transformar()` y `modelos.py` → `Pedido.total_con_iva`
Celda 4: agregar por ciudad	`carga.py` → `ResumenVentas.por_ciudad()`
Celda 5: mostrar el informe	`main.py` → `mostrar_informe()`
El `1.21` y el texto `"cancelado"`	`config.py`

Sección 4. El proyecto modular

Estructura

```
pedidos_etl/
├── main.py                ← punto de entrada
└── etl/                   ← paquete del pipeline
    ├── __init__.py
    ├── config.py          ← reglas de negocio configurables
    ├── modelos.py         ← clase Pedido
    ├── extraccion.py      ← E: obtener los datos
    ├── transformacion.py  ← T: limpiar, validar y convertir
    ├── carga.py           ← L: guardar y resumir
    └── orquestador.py     ← une E, T y L
```

`etl/config.py`

```python
"""Parámetros del pipeline. Si cambia una regla de negocio, se cambia aquí."""

IVA = 0.21
ESTADO_CANCELADO = "cancelado"
```

`etl/modelos.py`

```python
"""Clases de datos del pipeline."""

from .config import IVA


class Pedido:
    def __init__(self, id_pedido, cliente, ciudad, importe, estado):
        self.id_pedido = id_pedido
        self.cliente = cliente
        self.ciudad = ciudad
        self.importe = importe
        self.estado = estado

    @property
    def total_con_iva(self):
        return round(self.importe * (1 + IVA), 2)

    def __repr__(self):
        return f"Pedido({self.id_pedido}, {self.ciudad}, {self.total_con_iva})"
```

`etl/extraccion.py`

```python
"""Extracción: obtiene las líneas de datos en bruto, sin la cabecera."""


def leer_lineas(texto):
    return texto.strip().split("\n")[1:]
```

`etl/transformacion.py`

```python
"""Transformación: funciones puras que limpian, validan y convierten los datos."""

from .config import ESTADO_CANCELADO
from .modelos import Pedido


def parsear(linea):
```

```
    return [campo.strip() for campo in linea.split(",")]


def es_valido(campos):
    if len(campos) != 5:
        return False
    try:
        float(campos[3])
        return True
    except ValueError:
        return False


def a_pedido(campos):
    id_pedido, cliente, ciudad, importe, estado = campos
    return Pedido(id_pedido, cliente.title(), ciudad.title(), float(importe), estado.lower())


def transformar(lineas):
    registros = list(map(parsear, lineas))
    rechazados = [r[0] for r in registros if not es_valido(r)]
    pedidos = [a_pedido(r) for r in registros if es_valido(r)]
    cancelados = [p.id_pedido for p in pedidos if p.estado == ESTADO_CANCELADO]
    activos = list(filter(lambda p: p.estado != ESTADO_CANCELADO, pedidos))
    return activos, rechazados, cancelados
```

etl/carga.py

```
"""Carga: guarda los pedidos procesados y genera el resumen."""


class ResumenVentas:
    def __init__(self):
        self._pedidos = []

    def cargar(self, pedidos):
        self._pedidos.extend(pedidos)

    def __len__(self):
        return len(self._pedidos)

    def por_ciudad(self):
        totales = {}
        for pedido in self._pedidos:
            totales[pedido.ciudad] = round(totales.get(pedido.ciudad, 0) + pedido.total_con_iva, 2)
        return dict(sorted(totales.items(), key=lambda par: par[1], reverse=True))
```

etl/orquestador.py

```
"""Orquestador: une las tres etapas del pipeline en el orden correcto."""

from .extraccion import leer_lineas
from .transformacion import transformar
from .carga import ResumenVentas


def ejecutar_pipeline(texto):
    lineas = leer_lineas(texto)
    activos, rechazados, cancelados = transformar(lineas)
    resumen = ResumenVentas()
    resumen.cargar(activos)
    return {
        "leidos": len(lineas),
        "rechazados": rechazados,
        "cancelados": cancelados,
        "resumen": resumen,
    }
```

`etl/__init__.py`

```
"""Pipeline ETL de pedidos online: extracción, transformación y carga."""

from .orquestador import ejecutar_pipeline

__all__ = ["ejecutar_pipeline"]
```

`main.py`

```
"""Punto de entrada del pipeline de pedidos."""

from etl import ejecutar_pipeline

# En producción, estos datos llegarían de un fichero, una base de datos o una API
DATOS = """id,cliente,ciudad,importe,estado
P001, Ana López ,madrid,45.50,entregado
P002,Luis Pérez,Sevilla,12.00,cancelado
P003,Marta Ruiz,MADRID,30.25,entregado
P004,Jorge Gil,valencia,,entregado
P005,Elena Sanz,sevilla,80.00,entregado
P006,Pablo Ríos,Valencia,22.40,en reparto
P007,Sara Mora,madrid,abc,entregado"""


def mostrar_informe(resultado):
    print("Pedidos leídos:", resultado["leidos"])
    print("Rechazados por datos erróneos:", resultado["rechazados"])
    print("Pedidos cancelados:", resultado["cancelados"])
    print("Pedidos procesados:", len(resultado["resumen"]))
    print("Ventas con IVA por ciudad:")
    for ciudad, total in resultado["resumen"].por_ciudad().items():
        print(f"  {ciudad:<9}{total:>8.2f} €")


def main():
    resultado = ejecutar_pipeline(DATOS)
    mostrar_informe(resultado)


if __name__ == "__main__":
    main()
```

Ejecución

```
cd pedidos_etl
python main.py
```

Salida esperada:

```
Pedidos leídos: 7
Rechazados por datos erróneos: ['P004', 'P007']
Pedidos cancelados: ['P002']
Pedidos procesados: 4
Ventas con IVA por ciudad:
  Sevilla     96.80 €
  Madrid      91.65 €
  Valencia    27.10 €
```

Es **exactamente la misma salida** que la del notebook. Modularizar no cambia lo que hace el programa: cambia cómo está organizado.

Highlights:

> 1. **Cada etapa en su módulo.** `extraccion.py`, `transformacion.py` y `carga.py` son la E, la T y la L del pipeline. Si mañana los pedidos llegan de una base de datos en lugar de un texto, solo cambia `extraccion.py`.
> 2. **Las reglas de negocio en un solo sitio.** El IVA está en `config.py` y lo usa la clase `Pedido`. Si cambia, se modifica una línea.
> 3. **Una clase en lugar de posiciones.** `pedido.ciudad` y `pedido.total_con_iva` se entienden solos; `f[2]` y `f[5]` no. Además, el total con IVA es una `@property` que **se calcula** cuando se pide: no es una columna que se añade a mano y que puede faltar o duplicarse según el orden de las celdas.
> 4. **Funciones puras en la transformación.** `parsear()`, `es_valido()` y `a_pedido()` no modifican nada fuera de ellas. `transformar()` las combina con `map()`, comprehensions y `filter()` con una `lambda`.
> 5. **El orden lo fija el código.** En el notebook, el orden dependía de quién pulsaba las celdas. En producción lo fija `ejecutar_pipeline()`, y siempre es el mismo.
> 6. **Una sola puerta de entrada.** `main.py` solo importa `ejecutar_pipeline` y no necesita saber cómo está hecho el pipeline por dentro.
> 7. **Listo para automatizar.** Programar la ejecución nocturna consiste en lanzar `python main.py`. Gracias a `if __name__ == "__main__":`, el paquete también se puede importar desde otros programas sin que se ejecute nada.
> 8. **Mejoras en un único sitio.** Al separar `es_valido()`, se ha añadido una comprobación que el notebook no tenía: que cada línea tenga exactamente 5 campos (Sección 5). Con estos datos la salida no cambia, pero el pipeline es más robusto.

Sección 5. Probar una pieza por separado

> Una ventaja directa de la versión modular es que cada función se puede probar **sin ejecutar el pipeline completo**. Por ejemplo: ¿qué pasa si alguien escribe el importe con **coma decimal**? Con una coma de más, la línea se parte en 6 campos en lugar de 5.

Desde la carpeta `pedidos_etl`, en un archivo de pruebas o en la consola de Python:

```
from etl.transformacion import parsear, es_valido

print(es_valido(parsear("P010,Rosa Vidal,bilbao,19.95,entregado")))
print(es_valido(parsear("P009,Rosa Vidal,bilbao,9,95,entregado")))
print(es_valido(parsear("P011,Iván Paz,bilbao,,entregado")))
```

Salida esperada:

```
True
False
False
```

En el notebook, la línea de `P009` habría pasado la validación: tomaría `9` como importe y `95` como estado, sin dar ningún error, y el pedido se sumaría con un importe equivocado. Encontrar ese fallo habría obligado a revisar todas las celdas.

Sección 6. ¿Y el notebook? Sigue siendo útil

> El notebook **no se tira**: sigue siendo la mejor herramienta para explorar datos, probar ideas y hacer gráficos. Lo que cambia es su papel:

	Notebook (`.ipynb`)	Proyecto modular (`.py`)
Para qué	Explorar, analizar, prototipar	Ejecutar en producción de forma automática y repetible

	Notebook (`.ipynb`)	Proyecto modular (`.py`)
Quién lo ejecuta	Una persona, celda a celda	Un programador de tareas, siempre igual
Dónde viven las reglas	Repartidas por las celdas	En módulos con nombre, en un solo sitio
Cómo se reutiliza	Copiando y pegando	Importando

La práctica habitual en los equipos de datos es que, una vez existe el paquete, **el notebook lo importa** en lugar de repetir el código. Si el notebook está en la carpeta `pedidos_etl`, una celda podría ser:

```
from etl import ejecutar_pipeline

resultado = ejecutar_pipeline(datos)
resultado["resumen"].por_ciudad()
```

Así, el análisis del notebook y el proceso de producción usan **las mismas reglas**, y un cambio en el paquete llega a los dos.

Sección 7. Lo que viene después

> Más adelante en el máster, los pipelines trabajarán con volúmenes de datos mucho mayores y con herramientas específicas de Big Data. La organización será **la misma** que en este ejemplo: - Un **paquete** con un módulo por etapa. - Las **reglas de negocio** en configuración. - La **transformación** como funciones que reciben datos y devuelven datos nuevos. - Un **orquestador** que une las etapas y un `main.py` como punto de entrada. Cambiará **qué hay dentro** de cada módulo, no **cómo se organizan**. Por eso aprender a modularizar ahora con Python puro es la base de todo lo que viene.

Parte 7 · Examen de práctica · Modelo A

Máster Big Data & IA (Tajamar) · Preparación de Héctor Miguel Tudela · Examen práctico del 13 de noviembre de 2026

Examen inventado con el **mismo formato que el real**: **10 ejercicios, uno por sección**, del mismo estilo que el modelo 16.1, con ejercicios distintos a los del modelo y a los del simulacro.

Indicaciones

- Entrega en un cuaderno `.ipynb`. Puedes trabajar en un `.py` con celdas `# %%`, con el autocompletado de IA **desactivado**.
- En el ejercicio 1 escribe tu predicción como comentario **antes** de ejecutar.
- Comprueba tu salida con la **salida esperada** carácter a carácter.
- Hazlo primero sin mirar la **Parte II (Solucionario)**. Todas las soluciones se han ejecutado y las salidas son reales.
- Antes de entregar: **Reiniciar kernel → Run All**.

Nº	Sección	Ejercicio
1	Lectura de código	Pedidos de una tienda
2	Comprehensions	Ventas de una cadena de tiendas
3	Programación funcional	Vuelos de una aerolínea
4	Recursión	Máximo común divisor y máximo de una lista
5	Clases y objetos	Gestor de tareas (composición)
6	Sobrecarga de operadores	Fracciones
7	Herencia y polimorfismo	Alquiler de vehículos
8	Refactorización	Cuotas de un gimnasio
9	Ejercicio integrador	Tienda de videojuegos
10	Programación modular	El paquete `tienda`

Parte I · Enunciados

Ejercicio 1 · Lectura de código · Pedidos de una tienda

Sin ejecutarlo, escribe lo que se muestra por pantalla y responde a las preguntas.

```python
class Pedido:
    contador = 0
    estados = []

    def __init__(self, cliente):
        Pedido.contador += 1
        self.id = Pedido.contador
        self.cliente = cliente
        self.estados.append("creado")

    def __str__(self):
        return f"#{self.id} {self.cliente}"

class PedidoUrgente(Pedido):
    def __init__(self, cliente):
        super().__init__(cliente)
        self.estados = self.estados + ["urgente"]

p1 = Pedido("Ana")
p2 = PedidoUrgente("Luis")
p3 = Pedido("Eva")

print(p1, p2, p3)
```

```
print(p1.estados)
print(p2.estados)
print(Pedido.contador, p2.contador, len(Pedido.estados))
print(p1.estados is Pedido.estados, p2.estados is Pedido.estados)
```

1. ¿Por qué `p1.estados` contiene el estado del pedido de Eva?
2. ¿Por qué `p2.estados` no tiene el `"creado"` de Eva?
3. ¿Qué diferencia hay entre `self.estados.append(...)` y `self.estados = self.estados + [...]`?

Ejercicio 2 · Comprehensions · Ventas de una cadena de tiendas

```
ventas = [
    ("Madrid", "portátil", 2, 899.0),
    ("Sevilla", "ratón", 10, 19.9),
    ("Madrid", "ratón", 5, 19.9),
    ("Bilbao", "monitor", 3, 189.0),
    ("Sevilla", "portátil", 1, 899.0),
    ("Bilbao", "teclado", 4, 49.5),
    ("Madrid", "monitor", 1, 189.0),
]
```

Resuelve cada apartado con **comprehensions**:

1. Un diccionario `tienda → facturación total` (2 decimales), con las tiendas en orden alfabético.
2. Un set con los productos vendidos en **más de una** tienda. Muéstralo ordenado.
3. Una lista de tuplas `(tienda, producto)` de las líneas de más de 500 €.
4. Un diccionario anidado `tienda → {producto: unidades}`.

Salida esperada

```
{'Bilbao': 765.0, 'Madrid': 2086.5, 'Sevilla': 1098.0}
['monitor', 'portátil', 'ratón']
[('Madrid', 'portátil'), ('Bilbao', 'monitor'), ('Sevilla', 'portátil')]
{'Bilbao': {'monitor': 3, 'teclado': 4}, 'Madrid': {'portátil': 2, 'ratón': 5, 'monitor': 1}, 'Sevilla':
{'ratón': 10, 'portátil': 1}}
```

Ejercicio 3 · Programación funcional · Vuelos de una aerolínea

```
vuelos = [
    {"codigo": "IB101", "destino": "Roma",    "plazas": 180, "ocupadas": 171, "precio": 89.0},
    {"codigo": "IB202", "destino": "Lisboa",  "plazas": 150, "ocupadas": 60,  "precio": 59.0},
    {"codigo": "IB303", "destino": "París",   "plazas": 200, "ocupadas": 200, "precio": 120.0},
    {"codigo": "IB404", "destino": "Berlín",  "plazas": 180, "ocupadas": 99,  "precio": 75.0},
    {"codigo": "IB505", "destino": "Londres", "plazas": 220, "ocupadas": 198, "precio": 110.0},
]
```

1. **lambda + `sorted()`:** los códigos ordenados por **porcentaje de ocupación**, de mayor a menor.
2. **`map()`:** textos con el formato `"IB101 → Roma (95.0 %)"`.
3. **`filter()`:** los destinos de los vuelos con **menos del 70 %** de ocupación.
4. **`reduce()`:** los ingresos totales (ocupadas × precio) y el vuelo con más plazas libres.
5. **Función que devuelve una función:** `crear_descuento_ultima_hora(porcentaje)`, que devuelva una función que reciba un vuelo y devuelva su precio rebajado (2 decimales). Aplica un 30 % con `map()` a los vuelos no completos.

Salida esperada

```
['IB303', 'IB101', 'IB505', 'IB404', 'IB202']
['IB101 → Roma (95.0 %)', 'IB202 → Lisboa (40.0 %)', 'IB303 → París (100.0 %)', 'IB404 → Berlín (55.0 %)',
'IB505 → Londres (90.0 %)']
['Lisboa', 'Berlín']
Ingresos: 71964.00 €
Más plazas libres: IB202
[('IB101', 62.3), ('IB202', 41.3), ('IB404', 52.5), ('IB505', 77.0)]
```

Ejercicio 4 · Recursión · Máximo común divisor y máximo de una lista

Sin bucles:

1. `mcd(a, b)` recursiva con el algoritmo de Euclides: `mcd(a, 0) = a` y `mcd(a, b) = mcd(b, a % b)`.
2. `maximo(lista)` recursiva, sin usar `max()`: el máximo es el mayor entre el primer elemento y el máximo del resto.
3. `simplificar(num, den)` que use `mcd` y devuelva la fracción simplificada como texto `"num/den"`.

```
print(mcd(48, 18), mcd(17, 5), mcd(100, 25))
print(maximo([7, 42, 3, 19, 42, 8]), maximo([-5]))
print(simplificar(48, 18), simplificar(100, 25))
```

Salida esperada

```
6 1 25
42 -5
8/3 4/1
```

Pregunta: ¿qué pasaría con `maximo([])`? ¿Cómo lo protegerías?

Ejercicio 5 · Clases y objetos · Gestor de tareas (composición)

1. Crea la clase `Tarea` con:
   - atributo de clase `PRIORIDADES = ("baja", "media", "alta")` y contador `creadas`;
   - atributos `titulo`, `prioridad` (por defecto `"media"`) y `completada` (`False`);
   - `@staticmethod es_prioridad_valida(p)`;
   - `@classmethod desde_texto("Comprar café|alta")`, un constructor alternativo;
   - método `completar()`, que devuelve `False` si ya estaba completada;
   - `__str__` con el formato `[x] Comprar café (alta)` o `[ ] ...`.
2. Crea la clase `Proyecto`, que **contenga** una lista de tareas, con:
   - `agregar(tarea)`, que solo añade la tarea si su prioridad es válida y devuelve `True`/`False`;
   - `@property progreso`, el porcentaje de tareas completadas (1 decimal);
   - `pendientes(prioridad=None)`, la lista de títulos pendientes, filtrada por prioridad si se indica;
   - `len()` y `__str__` (ver salida).

```
p = Proyecto("Mudanza")
print(p.agregar(Tarea("Pedir presupuestos", "alta")))
print(p.agregar(Tarea.desde_texto("Comprar cajas|baja")))
print(p.agregar(Tarea("Avisar al casero", "urgente")))
p.agregar(Tarea("Dar de baja la luz"))
p.agregar(Tarea.desde_texto("Contratar internet|alta"))
print(p.tareas[0].completar(), p.tareas[0].completar())
p.tareas[1].completar()
print(p)
print(p.progreso, len(p), Tarea.creadas)
print(p.pendientes(), p.pendientes("alta"))
```

Salida esperada

```
True
True
False
True False
Proyecto Mudanza (50.0 %)
  [x] Pedir presupuestos (alta)
  [x] Comprar cajas (baja)
  [ ] Dar de baja la luz (media)
  [ ] Contratar internet (alta)
50.0 4 5
['Dar de baja la luz', 'Contratar internet'] ['Contratar internet']
```

Ejercicio 6 · Sobrecarga de operadores · Fracciones

Crea una clase `Fraccion(num, den)` que **siempre se guarde simplificada** (usa `math.gcd` en el constructor) y que permita:

- `a + b` y `a * b`, que devuelven una nueva `Fraccion`;
- `a == b` (con `Fraccion(2, 4) == Fraccion(1, 2)` → `True`) y `a < b`;
- `float(a)` y `str(a)` → `3/4`, o solo `2` si el denominador es 1;
- `@staticmethod desde_texto("3/4")`;
- `sum(lista)`, para lo que necesitas `__radd__`.

```
a = Fraccion(1, 2)
b = Fraccion(3, 4)
print(a + b, a * b, Fraccion(6, 8), Fraccion(4, 2))
print(a == Fraccion(2, 4), a < b, float(b))
fracciones = [Fraccion.desde_texto(t) for t in ["1/3", "1/6", "1/2", "5/6"]]
print(sum(fracciones), max(fracciones), sorted(fracciones)[0])
print([str(f) for f in sorted(fracciones)])
```

Salida esperada

```
5/4 3/8 3/4 2
True True 0.75
11/6 5/6 1/6
['1/6', '1/3', '1/2', '5/6']
```

Ejercicio 7 · Herencia y herencia múltiple · Alquiler de vehículos

1. Superclase `Vehiculo(matricula, dias)` con `PRECIO_DIA = 40` (atributo de clase), `precio()` = días × `PRECIO_DIA`, `extras()` que devuelve `[]` y `__str__` con el formato `Coche 1111-AAA · 3 días · 120.00 € · extras: ninguno`.
2. Subclases:
   - `Coche`: no cambia nada.
   - `Furgoneta`: `PRECIO_DIA = 65`, y un **10 % de descuento** si se alquila 7 días o más (usa `super().precio()`).
   - `Moto`: `PRECIO_DIA = 25`.
3. Dos clases "complemento" que heredan de `Vehiculo`:
   - `ConGPS`: suma 5 €/día al precio y añade `"GPS"` a los extras;
   - `ConSeguro`: suma 12 €/día y añade `"seguro"`.

   Las dos usan `super()`.
4. Clases combinadas: `CocheCompleto(ConGPS, ConSeguro, Coche)` y `FurgonetaGPS(ConGPS, Furgoneta)`.
5. Recorre la flota con un único bucle, muestra cada alquiler y el total, y muestra el MRO de `CocheCompleto`.

```
flota = [Coche("1111-AAA", 3), Furgoneta("2222-BBB", 7), Moto("3333-CCC", 2),
         CocheCompleto("4444-DDD", 3), FurgonetaGPS("5555-EEE", 7)]
```

Salida esperada

```
Coche 1111-AAA · 3 días · 120.00 € · extras: ninguno
Furgoneta 2222-BBB · 7 días · 409.50 € · extras: ninguno
Moto 3333-CCC · 2 días · 50.00 € · extras: ninguno
CocheCompleto 4444-DDD · 3 días · 171.00 € · extras: seguro, GPS
FurgonetaGPS 5555-EEE · 7 días · 444.50 € · extras: GPS
Total: 1195.00 €
['CocheCompleto', 'ConGPS', 'ConSeguro', 'Coche', 'Vehiculo', 'object']
```

Pregunta: en `FurgonetaGPS`, ¿el descuento del 10 % se aplica también a los 5 €/día del GPS? ¿Por qué?

Ejercicio 8 · Refactorización · Cuotas de un gimnasio

El programa funciona, pero está escrito con listas, índices y `if/elif`:

```
# [tipo, socio, clases_dirigidas_al_mes]
socios = [
```

```
    ["basica", "Ana", 4],
    ["premium", "Luis", 12],
    ["estudiante", "Eva", 6],
    ["premium", "Marta", 5],
    ["basica", "Iker", 0],
]

total = 0
for s in socios:
    if s[0] == "basica":
        cuota = 30.0 + s[2] * 3
        print("BASICA | " + s[1] + " | " + str(s[2]) + " clases | " + str(round(cuota, 2)) + " €")
    elif s[0] == "premium":
        cuota = 50.0
        if s[2] > 8:
            cuota = cuota + 10
        print("PREMIUM | " + s[1] + " | " + str(s[2]) + " clases | " + str(round(cuota, 2)) + " €")
    elif s[0] == "estudiante":
        cuota = (30.0 + s[2] * 3) * 0.8
        print("ESTUDIANTE | " + s[1] + " | " + str(s[2]) + " clases | " + str(round(cuota, 2)) + " €")
    total = total + cuota
print("TOTAL MENSUAL: " + str(round(total, 2)) + " €")
```

Tipo	Cuota
Básica	30 € + 3 € por clase dirigida
Premium	50 € (clases incluidas) + 10 € si hace más de 8 clases
Estudiante	la cuota básica con un 20 % de descuento

1. Crea una superclase `Socio(nombre, clases)` con `cuota()` y `__str__` (el tipo sale del nombre de la clase, sin la palabra `Socio`).
2. Crea `SocioBasica`, `SocioPremium` y `SocioEstudiante`. `SocioEstudiante` debe **heredar de `SocioBasica`** y reutilizar su cuota con `super()`.
3. Crea `Gimnasio` (composición) con `alta(socio)`, `listado()` y `total()`.
4. La salida debe ser **idéntica** a la del original.
5. Ampliación: `SocioFamiliar(nombre, clases, miembros)` = 70 € + 15 € por cada miembro a partir del segundo, sin tocar `Socio` ni `Gimnasio`. Añade `SocioFamiliar("Familia Ruiz", 10, 4)`.

Restricciones: sin `if/elif` por tipo ni `isinstance`, y cada regla de precio escrita en un único sitio.

Salida esperada

```
BASICA | Ana | 4 clases | 42.0 €
PREMIUM | Luis | 12 clases | 60.0 €
ESTUDIANTE | Eva | 6 clases | 38.4 €
PREMIUM | Marta | 5 clases | 50.0 €
BASICA | Iker | 0 clases | 30.0 €
TOTAL MENSUAL: 220.4 €
--- Con familiar ---
BASICA | Ana | 4 clases | 42.0 €
PREMIUM | Luis | 12 clases | 60.0 €
ESTUDIANTE | Eva | 6 clases | 38.4 €
PREMIUM | Marta | 5 clases | 50.0 €
BASICA | Iker | 0 clases | 30.0 €
FAMILIAR | Familia Ruiz | 10 clases | 115.0 €
TOTAL MENSUAL: 335.4 €
```

Ejercicio 9 · Ejercicio integrador · Tienda de videojuegos

Parte A. Superclase `Juego(titulo, precio)` con `precio_final()` y `__str__` (`Titulo - 59.99 €`). Subclases: - `JuegoDigital`: un 15 % más barato. - `EdicionColeccionista`: recargo fijo de 40 €. - `JuegoSegundaMano(titulo, precio, estado)`: la mitad del precio si `estado == "bueno"` y un 30 % del precio si no.

Parte B. Datos

```
ventas = [
    {"cliente": "Ana",  "juego": JuegoDigital("Zelda", 69.99),                  "uds": 1},
    {"cliente": "Luis", "juego": Juego("FIFA", 59.99),                          "uds": 2},
```

```
    {"cliente": "Eva",  "juego": EdicionColeccionista("Elden Ring", 79.99),     "uds": 1},
    {"cliente": "Ana",  "juego": JuegoSegundaMano("Mario Kart", 49.99, "bueno"), "uds": 1},
    {"cliente": "Iker", "juego": JuegoSegundaMano("GTA V", 29.99, "regular"),    "uds": 3},
]
```

Parte C. Con comprehensions: lista de importes (`precio_final × uds`, 2 decimales); diccionario `cliente → gasto total`; set de títulos vendidos en formato digital (`isinstance`).

Parte D. Con `filter()`, las ventas de más de 50 €. Con `map()` y `reduce()`, la facturación total. Con una función `crear_filtro_precio(maximo)` que devuelva otra función, los títulos con precio final ≤ 30 €.

Parte E. `unidades(ventas)` recursiva, sin bucles.

Parte F. Informe:

Salida esperada

```
=== JUEGOS VENDIDOS ===
Zelda - 59.49 €
FIFA - 59.99 €
Elden Ring - 119.99 €
Mario Kart - 25.00 €
GTA V - 9.00 €

Importes: [59.49, 119.98, 119.99, 25.0, 27.0]
Gasto por cliente: {'Ana': 84.49, 'Eva': 119.99, 'Iker': 27.0, 'Luis': 119.98}
Digitales: ['Zelda']
Ventas > 50 €: ['Zelda', 'FIFA', 'Elden Ring']
Facturación: 351.46 €
Precio ≤ 30 €: ['Mario Kart', 'GTA V']
Unidades vendidas: 8
```

Ejercicio 10 · Programación modular · El paquete `tienda`

Tienes este proyecto:

```
proyecto/
├── main.py
└── tienda/
    ├── __init__.py
    ├── config.py
    ├── productos.py
    └── carrito.py
```

`tienda/__init__.py`

```
print("Cargando paquete tienda")
from .config import IVA
from .productos import Producto
from .carrito import Carrito

__all__ = ["IVA", "Producto", "Carrito"]
```

`tienda/config.py`

```
print("Cargando config")
IVA = 0.21
ENVIO_GRATIS_DESDE = 50
GASTOS_ENVIO = 4.95
```

`tienda/productos.py`

```
from .config import IVA
print("Cargando productos")


class Producto:
    def __init__(self, nombre, precio):
```

```
        self.nombre = nombre
        self.precio = precio

    @property
    def precio_con_iva(self):
        return round(self.precio * (1 + IVA), 2)

    def __repr__(self):
        return f"Producto({self.nombre!r}, {self.precio})"
```

`tienda/carrito.py`

```
from .productos import Producto
from .config import ENVIO_GRATIS_DESDE, GASTOS_ENVIO
print("Cargando carrito")


class Carrito:
    def __init__(self):
        self.lineas = []

    def agregar(self, producto, cantidad=1):
        self.lineas.append((producto, cantidad))

    def total(self):
        subtotal = sum(p.precio_con_iva * c for p, c in self.lineas)
        envio = 0 if subtotal >= ENVIO_GRATIS_DESDE else GASTOS_ENVIO
        return round(subtotal + envio, 2)


if __name__ == "__main__":
    c = Carrito()
    c.agregar(Producto("Funda", 10))
    print("Prueba carrito:", c.total())
```

`main.py`

```
from tienda import Producto, Carrito, IVA
import tienda.config
from tienda.productos import Producto as P

print("IVA:", IVA, "| mismo Producto:", Producto is P)
c = Carrito()
c.agregar(Producto("Ratón", 20), 2)
c.agregar(Producto("Teclado", 35))
print("Total carrito:", c.total())

tienda.config.IVA = 0.10
print(Producto("Cable", 10).precio_con_iva, IVA, tienda.config.IVA)
```

1. ¿Qué se muestra al ejecutar `python main.py` desde la carpeta `proyecto`? ¿Cuántas veces aparece cada mensaje “Cargando...”?
2. ¿Por qué `IVA` y `Producto` se pueden importar directamente desde `tienda`?
3. ¿Qué ocurre si ejecutas `python tienda/carrito.py`? ¿Y `python -m tienda.carrito`?
4. Un compañero añade `from .carrito import Carrito` en `productos.py`. ¿Qué problema aparece?
5. ¿Qué ventaja tiene que `IVA` esté solo en `config.py`?

Parte II · Solucionario

Solución 1 · Lectura de código · Pedidos de una tienda

🖥️ Se muestra por pantalla:

```
#1 Ana #2 Luis #3 Eva
['creado', 'creado', 'creado']
['creado', 'creado', 'urgente']
```

```
3 3 3
True False
```

Explicación:

1. `estados` es un **atributo de clase**: una única lista compartida. `self.estados.append(...)` no encuentra `estados` en el objeto, lo busca en la clase y **modifica esa lista común**. Los tres pedidos añaden "creado" a la misma lista.
2. En `PedidoUrgente`, `self.estados + ["urgente"]` crea una **lista nueva**, que es una copia de la lista de clase en ese momento (con 2 "creado"). La asignación `self.estados = ...` la guarda como atributo de **instancia** de `p2`. Cuando después se crea `p3`, se modifica la lista de clase, pero `p2` ya tiene la suya.
3. `append` **modifica** la lista existente, que aquí es la compartida. `+` crea una lista **nueva**, y la asignación con `self.` crea o tapa un atributo de instancia.

- `id` sí es distinto en cada pedido porque se asigna con `self.id = ...`, y `contador` se suma en la **clase** (`Pedido.contador += 1`), no en el objeto.

Solución 2 · Comprehensions · Ventas de una cadena de tiendas

```python
ventas = [
    ("Madrid", "portátil", 2, 899.0),
    ("Sevilla", "ratón", 10, 19.9),
    ("Madrid", "ratón", 5, 19.9),
    ("Bilbao", "monitor", 3, 189.0),
    ("Sevilla", "portátil", 1, 899.0),
    ("Bilbao", "teclado", 4, 49.5),
    ("Madrid", "monitor", 1, 189.0),
]

tiendas = sorted({t for t, _, _, _ in ventas})
facturacion = {t: round(sum(u * p for ti, _, u, p in ventas if ti == t), 2) for t in tiendas}

productos = [prod for _, prod, _, _ in ventas]
repetidos = {p for p in productos if len({t for t, pr, _, _ in ventas if pr == p}) > 1}

grandes = [(t, prod) for t, prod, u, p in ventas if u * p > 500]

detalle = {t: {prod: u for ti, prod, u, _ in ventas if ti == t} for t in tiendas}

print(facturacion)
print(sorted(repetidos))
print(grandes)
print(detalle)
```

🖥️ **Salida:**

```
{'Bilbao': 765.0, 'Madrid': 2086.5, 'Sevilla': 1098.0}
['monitor', 'portátil', 'ratón']
[('Madrid', 'portátil'), ('Bilbao', 'monitor'), ('Sevilla', 'portátil')]
{'Bilbao': {'monitor': 3, 'teclado': 4}, 'Madrid': {'portátil': 2, 'ratón': 5, 'monitor': 1}, 'Sevilla':
{'ratón': 10, 'portátil': 1}}
```

Explicación:

- En el apartado 2, para cada producto se construye un **set de tiendas** donde aparece. Si tiene más de un elemento, el producto se ha vendido en varias tiendas.
- El apartado 4 es una **comprehension de diccionario dentro de otra**: la exterior recorre las tiendas y la interior, las líneas de cada tienda.
- `_` se usa como nombre de variable para los campos que no se utilizan.

Solución 3 · Programación funcional · Vuelos de una aerolínea

```python
vuelos = [
    {"codigo": "IB101", "destino": "Roma",    "plazas": 180, "ocupadas": 171, "precio": 89.0},
    {"codigo": "IB202", "destino": "Lisboa",  "plazas": 150, "ocupadas": 60,  "precio": 59.0},
    {"codigo": "IB303", "destino": "París",   "plazas": 200, "ocupadas": 200, "precio": 120.0},
    {"codigo": "IB404", "destino": "Berlín",  "plazas": 180, "ocupadas": 99,  "precio": 75.0},
```

```
    {"codigo": "IB505", "destino": "Londres", "plazas": 220, "ocupadas": 198, "precio": 110.0},
]

from functools import reduce

ocupacion = lambda v: round(v["ocupadas"] / v["plazas"] * 100, 1)

print([v["codigo"] for v in sorted(vuelos, key=ocupacion, reverse=True)])
print(list(map(lambda v: f"{v['codigo']} → {v['destino']} ({ocupacion(v)} %)", vuelos)))
print(list(map(lambda v: v["destino"], filter(lambda v: ocupacion(v) < 70, vuelos))))

ingresos = reduce(lambda acum, v: acum + v["ocupadas"] * v["precio"], vuelos, 0)
mas_libres = reduce(lambda a, b: a if a["plazas"] - a["ocupadas"] >= b["plazas"] - b["ocupadas"] else b,
vuelos)
print(f"Ingresos: {ingresos:.2f} €")
print("Más plazas libres:", mas_libres["codigo"])

def crear_descuento_ultima_hora(porcentaje):
    def rebajar(vuelo):
        return round(vuelo["precio"] * (1 - porcentaje / 100), 2)
    return rebajar                          # closure: recuerda porcentaje

rebaja_30 = crear_descuento_ultima_hora(30)
no_completos = filter(lambda v: v["ocupadas"] < v["plazas"], vuelos)
print(list(map(lambda v: (v["codigo"], rebaja_30(v)), no_completos)))
```

🖥️ **Salida:**

```
['IB303', 'IB101', 'IB505', 'IB404', 'IB202']
['IB101 → Roma (95.0 %)', 'IB202 → Lisboa (40.0 %)', 'IB303 → París (100.0 %)', 'IB404 → Berlín (55.0 %)',
'IB505 → Londres (90.0 %)']
['Lisboa', 'Berlín']
Ingresos: 71964.00 €
Más plazas libres: IB202
[('IB101', 62.3), ('IB202', 41.3), ('IB404', 52.5), ('IB505', 77.0)]
```

Explicación:

- `ocupacion` es una lambda guardada en una variable. Se reutiliza como `key` de `sorted()`, sin paréntesis, y también se llama directamente.
- En `reduce` sin valor inicial, el acumulador empieza siendo el **primer vuelo**; en cada paso se queda con el que tiene más plazas libres.
- `filter()` devuelve un iterador, y `map()` lo recorre directamente sin convertirlo antes a lista.

Solución 4 · Recursión · Máximo común divisor y máximo de una lista

```
def mcd(a, b):
    if b == 0:                              # caso base
        return a
    return mcd(b, a % b)                    # el problema se reduce en cada llamada

def maximo(lista):
    if len(lista) == 1:                     # caso base: un solo elemento
        return lista[0]
    resto = maximo(lista[1:])
    return lista[0] if lista[0] > resto else resto

def simplificar(num, den):
    divisor = mcd(num, den)
    return f"{num // divisor}/{den // divisor}"

print(mcd(48, 18), mcd(17, 5), mcd(100, 25))
print(maximo([7, 42, 3, 19, 42, 8]), maximo([-5]))
print(simplificar(48, 18), simplificar(100, 25))
```

🖥️ **Salida:**

```
6 1 25
42 -5
```

```
8/3 4/1
```

Explicación:

`mcd(48, 18)` → `mcd(18, 12)` → `mcd(12, 6)` → `mcd(6, 0)` → 6.

Con `maximo([])` nunca se llega al caso base de un elemento: `lista[1:]` de una lista vacía sigue siendo `[]` y la función se llama sin fin hasta un **`RecursionError`** (o da `IndexError` si se accede a `lista[0]`). Se protege con un caso base para la lista vacía, por ejemplo `if not lista: return None`, o lanzando un `ValueError`.

Solución 5 · Clases y objetos · Gestor de tareas (composición)

```python
class Tarea:
    PRIORIDADES = ("baja", "media", "alta")
    creadas = 0

    def __init__(self, titulo, prioridad="media"):
        self.titulo = titulo
        self.prioridad = prioridad
        self.completada = False
        Tarea.creadas += 1

    @staticmethod
    def es_prioridad_valida(prioridad):
        return prioridad in Tarea.PRIORIDADES

    @classmethod
    def desde_texto(cls, texto):
        titulo, prioridad = texto.split("|")
        return cls(titulo, prioridad)

    def completar(self):
        if self.completada:
            return False
        self.completada = True
        return True

    def __str__(self):
        marca = "x" if self.completada else " "
        return f"[{marca}] {self.titulo} ({self.prioridad})"

class Proyecto:
    def __init__(self, nombre):
        self.nombre = nombre
        self.tareas = []                        # composición

    def agregar(self, tarea):
        if not Tarea.es_prioridad_valida(tarea.prioridad):
            return False
        self.tareas.append(tarea)
        return True

    @property
    def progreso(self):
        if not self.tareas:
            return 0.0
        return round(len([t for t in self.tareas if t.completada]) / len(self.tareas) * 100, 1)

    def pendientes(self, prioridad=None):
        return [t.titulo for t in self.tareas
                if not t.completada and (prioridad is None or t.prioridad == prioridad)]

    def __len__(self):
        return len(self.tareas)

    def __str__(self):
        return f"Proyecto {self.nombre} ({self.progreso} %)\n" + "\n".join(f"  {t}" for t in self.tareas)

p = Proyecto("Mudanza")
print(p.agregar(Tarea("Pedir presupuestos", "alta")))
print(p.agregar(Tarea.desde_texto("Comprar cajas|baja")))
```

```
print(p.agregar(Tarea("Avisar al casero", "urgente")))
p.agregar(Tarea("Dar de baja la luz"))
p.agregar(Tarea.desde_texto("Contratar internet|alta"))
print(p.tareas[0].completar(), p.tareas[0].completar())
p.tareas[1].completar()
print(p)
print(p.progreso, len(p), Tarea.creadas)
print(p.pendientes(), p.pendientes("alta"))
```

🖥️ **Salida:**

```
True
True
False
True False
Proyecto Mudanza (50.0 %)
  [x] Pedir presupuestos (alta)
  [x] Comprar cajas (baja)
  [ ] Dar de baja la luz (media)
  [ ] Contratar internet (alta)
50.0 4 5
['Dar de baja la luz', 'Contratar internet'] ['Contratar internet']
```

Explicación:

- `Tarea.creadas` vale **5** aunque el proyecto solo tenga 4 tareas: la tarea "urgente" sí se creó (se ejecutó su `__init__`), pero `agregar()` la rechazó.
- `progreso` es una `@property`: se usa sin paréntesis y se calcula en el momento de pedirla.
- `desde_texto` usa `cls(...)`, así que si mañana hay una subclase de `Tarea`, creará objetos de esa subclase.

Solución 6 · Sobrecarga de operadores · Fracciones

```
import math

class Fraccion:
    def __init__(self, num, den):
        divisor = math.gcd(num, den)
        self.num = num // divisor
        self.den = den // divisor

    @staticmethod
    def desde_texto(texto):
        num, den = texto.split("/")
        return Fraccion(int(num), int(den))

    def __add__(self, otra):
        return Fraccion(self.num * otra.den + otra.num * self.den, self.den * otra.den)

    def __radd__(self, otro):                # sum() empieza con 0 + fracción
        return self + Fraccion(otro, 1)

    def __mul__(self, otra):
        return Fraccion(self.num * otra.num, self.den * otra.den)

    def __eq__(self, otra):
        return self.num == otra.num and self.den == otra.den   # ya están simplificadas

    def __lt__(self, otra):
        return self.num * otra.den < otra.num * self.den       # sin decimales

    def __float__(self):
        return self.num / self.den

    def __str__(self):
        return str(self.num) if self.den == 1 else f"{self.num}/{self.den}"

a = Fraccion(1, 2)
b = Fraccion(3, 4)
print(a + b, a * b, Fraccion(6, 8), Fraccion(4, 2))
print(a == Fraccion(2, 4), a < b, float(b))
fracciones = [Fraccion.desde_texto(t) for t in ["1/3", "1/6", "1/2", "5/6"]]
```

```
print(sum(fracciones), max(fracciones), sorted(fracciones)[0])
print([str(f) for f in sorted(fracciones)])
```

🖥️ **Salida:**

```
5/4 3/8 3/4 2
True True 0.75
11/6 5/6 1/6
['1/6', '1/3', '1/2', '5/6']
```

Explicación:

- Simplificar en el constructor hace que `__eq__` sea trivial: dos fracciones iguales tienen el mismo numerador y denominador.
- `__lt__` compara con productos cruzados (a/b < c/d ⇔ a·d < c·b) para evitar errores de redondeo de los `float`.
- `max()` y `sorted()` funcionan solo con `__lt__`. Sin `__radd__`, `sum()` daría `TypeError` al hacer `0 + Fraccion`.

Solución 7 · Herencia y herencia múltiple · Alquiler de vehículos

```
class Vehiculo:
    PRECIO_DIA = 40

    def __init__(self, matricula, dias):
        self.matricula = matricula
        self.dias = dias

    def precio(self):
        return self.dias * self.PRECIO_DIA          # self.PRECIO_DIA → el de la clase real

    def extras(self):
        return []

    def __str__(self):
        extras = ", ".join(self.extras()) or "ninguno"
        return (f"{type(self).__name__} {self.matricula} · {self.dias} días · "
                f"{self.precio():.2f} € · extras: {extras}")

class Coche(Vehiculo):
    pass

class Furgoneta(Vehiculo):
    PRECIO_DIA = 65

    def precio(self):
        base = super().precio()
        return base * 0.9 if self.dias >= 7 else base

class Moto(Vehiculo):
    PRECIO_DIA = 25

class ConGPS(Vehiculo):
    def precio(self):
        return super().precio() + 5 * self.dias

    def extras(self):
        return super().extras() + ["GPS"]

class ConSeguro(Vehiculo):
    def precio(self):
        return super().precio() + 12 * self.dias

    def extras(self):
        return super().extras() + ["seguro"]

class CocheCompleto(ConGPS, ConSeguro, Coche):
    pass

class FurgonetaGPS(ConGPS, Furgoneta):
    pass
```

```python
flota = [Coche("1111-AAA", 3), Furgoneta("2222-BBB", 7), Moto("3333-CCC", 2),
         CocheCompleto("4444-DDD", 3), FurgonetaGPS("5555-EEE", 7)]

for v in flota:
    print(v)                                    # polimorfismo: precio() y extras() de cada clase
print(f"Total: {sum(v.precio() for v in flota):.2f} €")
print([c.__name__ for c in CocheCompleto.__mro__])
```

🖥️ **Salida:**

```
Coche 1111-AAA · 3 días · 120.00 € · extras: ninguno
Furgoneta 2222-BBB · 7 días · 409.50 € · extras: ninguno
Moto 3333-CCC · 2 días · 50.00 € · extras: ninguno
CocheCompleto 4444-DDD · 3 días · 171.00 € · extras: seguro, GPS
FurgonetaGPS 5555-EEE · 7 días · 444.50 € · extras: GPS
Total: 1195.00 €
['CocheCompleto', 'ConGPS', 'ConSeguro', 'Coche', 'Vehiculo', 'object']
```

Explicación:

- MRO de `CocheCompleto`: `CocheCompleto → ConGPS → ConSeguro → Coche → Vehiculo`. El `precio()` de `ConGPS` llama con `super()` a `ConSeguro`, este a `Coche` y este a `Vehiculo`: 3×40 + 3×12 + 3×5 = 171 €.
- Los extras se construyen a la vuelta, por eso salen en orden inverso al MRO: `seguro, GPS`.
- En `FurgonetaGPS` el MRO es `ConGPS → Furgoneta → Vehiculo`. El descuento lo aplica `Furgoneta.precio()` **antes** de que `ConGPS` sume los 5 €/día, así que **no** se descuenta el GPS: 7×65×0.9 + 35 = 444.50 €. Si quisieras descontarlo, el orden de herencia o el diseño tendrían que cambiar.

Solución 8 · Refactorización · Cuotas de un gimnasio

```python
class Socio:
    def __init__(self, nombre, clases):
        self.nombre = nombre
        self.clases = clases

    def cuota(self):
        return 0.0

    def __str__(self):
        tipo = type(self).__name__.replace("Socio", "").upper()
        return f"{tipo} | {self.nombre} | {self.clases} clases | {round(self.cuota(), 2)} €"

class SocioBasica(Socio):
    BASE = 30.0
    POR_CLASE = 3

    def cuota(self):
        return self.BASE + self.clases * self.POR_CLASE

class SocioPremium(Socio):
    BASE = 50.0
    LIMITE_CLASES = 8
    SUPLEMENTO = 10

    def cuota(self):
        return self.BASE + (self.SUPLEMENTO if self.clases > self.LIMITE_CLASES else 0)

class SocioEstudiante(SocioBasica):            # un estudiante ES una básica con descuento
    DESCUENTO = 0.20

    def cuota(self):
        return super().cuota() * (1 - self.DESCUENTO)

class Gimnasio:                                # composición
    def __init__(self):
        self.socios = []

    def alta(self, socio):
        self.socios.append(socio)
```

```
    def listado(self):
        for s in self.socios:
            print(s)

    def total(self):
        return sum(s.cuota() for s in self.socios)

gym = Gimnasio()
for s in [SocioBasica("Ana", 4), SocioPremium("Luis", 12), SocioEstudiante("Eva", 6),
          SocioPremium("Marta", 5), SocioBasica("Iker", 0)]:
    gym.alta(s)
gym.listado()
print(f"TOTAL MENSUAL: {round(gym.total(), 2)} €")

# Ampliación sin tocar Socio ni Gimnasio
class SocioFamiliar(Socio):
    BASE = 70.0
    POR_MIEMBRO_EXTRA = 15

    def __init__(self, nombre, clases, miembros):
        super().__init__(nombre, clases)
        self.miembros = miembros

    def cuota(self):
        return self.BASE + (self.miembros - 1) * self.POR_MIEMBRO_EXTRA

print("--- Con familiar ---")
gym.alta(SocioFamiliar("Familia Ruiz", 10, 4))
gym.listado()
print(f"TOTAL MENSUAL: {round(gym.total(), 2)} €")
```

🖥️ **Salida:**

```
BASICA | Ana | 4 clases | 42.0 €
PREMIUM | Luis | 12 clases | 60.0 €
ESTUDIANTE | Eva | 6 clases | 38.4 €
PREMIUM | Marta | 5 clases | 50.0 €
BASICA | Iker | 0 clases | 30.0 €
TOTAL MENSUAL: 220.4 €
--- Con familiar ---
BASICA | Ana | 4 clases | 42.0 €
PREMIUM | Luis | 12 clases | 60.0 €
ESTUDIANTE | Eva | 6 clases | 38.4 €
PREMIUM | Marta | 5 clases | 50.0 €
BASICA | Iker | 0 clases | 30.0 €
FAMILIAR | Familia Ruiz | 10 clases | 115.0 €
TOTAL MENSUAL: 335.4 €
```

Explicación:

`type(self).__name__.replace("Socio", "").upper()` convierte `SocioEstudiante` en `ESTUDIANTE`. Añadir el plan familiar ha consistido en **añadir una clase**: el listado y el total funcionan sin cambios gracias al polimorfismo.

Solución 9 · Ejercicio integrador · Tienda de videojuegos

```
from functools import reduce

class Juego:
    def __init__(self, titulo, precio):
        self.titulo = titulo
        self.precio = precio

    def precio_final(self):
        return self.precio

    def __str__(self):
        return f"{self.titulo} - {self.precio_final():.2f} €"

class JuegoDigital(Juego):
    def precio_final(self):
        return round(super().precio_final() * 0.85, 2)
```

```python
class EdicionColeccionista(Juego):
    def precio_final(self):
        return super().precio_final() + 40

class JuegoSegundaMano(Juego):
    def __init__(self, titulo, precio, estado):
        super().__init__(titulo, precio)
        self.estado = estado

    def precio_final(self):
        factor = 0.5 if self.estado == "bueno" else 0.3
        return round(super().precio_final() * factor, 2)

ventas = [
    {"cliente": "Ana",  "juego": JuegoDigital("Zelda", 69.99),                    "uds": 1},
    {"cliente": "Luis", "juego": Juego("FIFA", 59.99),                           "uds": 2},
    {"cliente": "Eva",  "juego": EdicionColeccionista("Elden Ring", 79.99),     "uds": 1},
    {"cliente": "Ana",  "juego": JuegoSegundaMano("Mario Kart", 49.99, "bueno"), "uds": 1},
    {"cliente": "Iker", "juego": JuegoSegundaMano("GTA V", 29.99, "regular"),    "uds": 3},
]

importe = lambda v: round(v["juego"].precio_final() * v["uds"], 2)

# C. Comprehensions
importes = [importe(v) for v in ventas]
clientes = {v["cliente"] for v in ventas}
gasto = {c: round(sum(importe(v) for v in ventas if v["cliente"] == c), 2) for c in sorted(clientes)}
digitales = {v["juego"].titulo for v in ventas if isinstance(v["juego"], JuegoDigital)}

# D. Funcional
grandes = list(filter(lambda v: importe(v) > 50, ventas))
facturacion = reduce(lambda acum, x: acum + x, map(importe, ventas), 0)

def crear_filtro_precio(maximo):
    return lambda v: v["juego"].precio_final() <= maximo

baratos = [v["juego"].titulo for v in filter(crear_filtro_precio(30), ventas)]

# E. Recursión
def unidades(ventas):
    return 0 if not ventas else ventas[0]["uds"] + unidades(ventas[1:])

# F. Informe
print("=== JUEGOS VENDIDOS ===")
for v in ventas:
    print(v["juego"])
print()
print("Importes:", importes)
print("Gasto por cliente:", gasto)
print("Digitales:", sorted(digitales))
print("Ventas > 50 €:", [v["juego"].titulo for v in grandes])
print(f"Facturación: {round(facturacion, 2)} €")
print("Precio ≤ 30 €:", baratos)
print("Unidades vendidas:", unidades(ventas))
```

🖥️ Salida:

```
=== JUEGOS VENDIDOS ===
Zelda - 59.49 €
FIFA - 59.99 €
Elden Ring - 119.99 €
Mario Kart - 25.00 €
GTA V - 9.00 €

Importes: [59.49, 119.98, 119.99, 25.0, 27.0]
Gasto por cliente: {'Ana': 84.49, 'Eva': 119.99, 'Iker': 27.0, 'Luis': 119.98}
Digitales: ['Zelda']
Ventas > 50 €: ['Zelda', 'FIFA', 'Elden Ring']
Facturación: 351.46 €
Precio ≤ 30 €: ['Mario Kart', 'GTA V']
Unidades vendidas: 8
```

Solución 10 · Programación modular · El paquete `tienda`

🖥️ **`python main.py`:**

```
Cargando paquete tienda
Cargando config
Cargando productos
Cargando carrito
IVA: 0.21 | mismo Producto: True
Total carrito: 90.75
12.1 0.21 0.1
```

🖥️ **`python -m tienda.carrito`:**

```
Cargando paquete tienda
Cargando config
Cargando productos
Cargando carrito
<frozen runpy>:130: RuntimeWarning: 'tienda.carrito' found in sys.modules after import of package 'tienda',
but prior to execution of 'tienda.carrito'; this may result in unpredictable behaviour
Cargando carrito
Prueba carrito: 17.05
```

🖥️ **`python tienda/carrito.py` (última línea del error):**

```
ImportError: attempted relative import with no known parent package
```

Explicación:

1. Al importar `tienda`, primero se ejecuta `tienda/__init__.py`, que es el primer mensaje, y este importa `config`, `productos` y `carrito` **en ese orden**. Cada módulo se ejecuta **una sola vez**, aunque `main.py` lo vuelva a importar después (`import tienda.config`, `from tienda.productos import ...`): Python lo reutiliza desde `sys.modules`. Por eso `Producto is P` es `True`. Al final, `tienda.config.IVA = 0.10` **no** afecta a `Producto`, que sigue en 12.1: `productos.py` hizo `from .config import IVA`, que copia el **valor** al importar, y el `IVA` de `main.py` también se copió así (0.21). Solo `tienda.config.IVA` vale 0.1. Para que el cambio se notara habría que usar `config.IVA` en el momento del cálculo (`from . import config`).
2. `tienda/__init__.py` es la **recepción** del paquete: importa `Producto`, `Carrito` e `IVA` desde sus módulos internos. Así quien usa el paquete puede escribir `from tienda import ...` sin conocer la estructura interna.
3. `python tienda/carrito.py` ejecuta el archivo como **script suelto**, sin paquete padre, así que los imports relativos fallan con `ImportError: attempted relative import with no known parent package`. Con `python -m tienda.carrito` se ejecuta **como módulo del paquete**: los imports relativos funcionan y, como `__name__ == "__main__"`, se ejecutan las pruebas del bloque final. Ojo: como `tienda/__init__.py` ya importa `carrito`, ese módulo se carga **dos veces**, una como `tienda.carrito` y otra como `__main__`. Por eso "Cargando carrito" sale repetido y Python avisa con un `RuntimeWarning`.
4. Sería un **import circular**: `carrito` importa `productos`, y `productos` importaría `carrito`. Uno de los dos quedaría a medio cargar y daría `ImportError`. Las dependencias deben ir en un solo sentido: `config → productos → carrito → __init__ → main`.
5. Si el IVA cambia, se modifica en **un único sitio** y todos los módulos usan el valor nuevo. No hay "números mágicos" repetidos.

Parte 8 · Examen de práctica · Modelo B

Máster Big Data & IA (Tajamar) · Preparación de Héctor Miguel Tudela · Examen práctico del 13 de noviembre de 2026

Examen inventado con el **mismo formato que el real**: **10 ejercicios, uno por sección**, del mismo estilo que el modelo 16.1, con ejercicios distintos a los del modelo y a los del simulacro.

Indicaciones

- Entrega en un cuaderno `.ipynb`. Puedes trabajar en un `.py` con celdas `# %%`, con el autocompletado de IA **desactivado**.
- En el ejercicio 1 escribe tu predicción como comentario **antes** de ejecutar.
- Comprueba tu salida con la **salida esperada** carácter a carácter.
- Hazlo primero sin mirar la **Parte II (Solucionario)**. Todas las soluciones se han ejecutado y las salidas son reales.
- Antes de entregar: **Reiniciar kernel → Run All**.

Nº	Sección	Ejercicio
1	Lectura de código	La cadena de `super()` rota
2	Comprehensions	Logs de un servidor web
3	Programación funcional	Liga de fútbol
4	Recursión	Binario, listas crecientes y presupuestos
5	Clases y objetos	Hotel y habitaciones (composición)
6	Sobrecarga de operadores	Temperaturas
7	Herencia y polimorfismo	Envío de notificaciones
8	Refactorización	Matrículas de una academia de idiomas
9	Ejercicio integrador	Taquilla de un cine
10	Programación modular	Del script al paquete `almacen`

Parte I · Enunciados

Ejercicio 1 · Lectura de código · La cadena de `super()` rota

Sin ejecutarlo, escribe lo que se muestra por pantalla y responde a las preguntas.

```python
class A:
    def saludo(self):
        return ["A"]

class B(A):
    def saludo(self):
        return super().saludo() + ["B"]

class C(A):
    def saludo(self):
        return ["C"]                # no llama a super()

class D(B, C):
    def saludo(self):
        return super().saludo() + ["D"]

class E(C, B):
    pass

print(D().saludo())
print(E().saludo())
print(B().saludo())
```

```
print([k.__name__ for k in D.__mro__])
print(isinstance(D(), C), issubclass(E, B), E.__mro__[1].__name__)
```

1. ¿Por qué en `D().saludo()` no aparece `"A"`?
2. ¿Por qué `E().saludo()` devuelve una sola letra?
3. ¿A qué clase llama el `super()` de `B` en un objeto `B`? ¿Y en un objeto `D`?

Ejercicio 2 · Comprehensions · Logs de un servidor web

```
logs = [
    "2026-11-12 10:01 GET /productos 200 0.120",
    "2026-11-12 10:02 POST /login 401 0.045",
    "2026-11-12 10:02 GET /productos 200 0.098",
    "2026-11-12 10:03 GET /carrito 500 1.502",
    "2026-11-12 10:05 POST /login 200 0.051",
    "2026-11-12 10:07 GET /productos/15 404 0.020",
    "2026-11-12 10:08 GET /carrito 200 0.310",
]
```

Con **comprehensions**:

1. Una lista de tuplas `(hora, metodo, ruta, codigo, segundos)`, con `codigo` como `int` y `segundos` como `float`.
2. Un set con las rutas que han dado algún error (código ≥ 400), ordenado al mostrarlo.
3. Un diccionario `ruta → nº de peticiones`.
4. Un diccionario `ruta → tiempo medio` (3 decimales).
5. Las horas de las peticiones **lentas** (más de 0.3 s) junto con su código.

Salida esperada

```
('10:01', 'GET', '/productos', 200, 0.12)
['/carrito', '/login', '/productos/15']
{'/productos': 2, '/login': 2, '/carrito': 2, '/productos/15': 1}
{'/productos': 0.109, '/login': 0.048, '/carrito': 0.906, '/productos/15': 0.02}
[('10:03', 500), ('10:08', 200)]
```

Ejercicio 3 · Programación funcional · Liga de fútbol

```
partidos = [
    {"jornada": 1, "local": "Atlético", "visitante": "Betis",    "gl": 3, "gv": 1},
    {"jornada": 1, "local": "Girona",   "visitante": "Sevilla",  "gl": 2, "gv": 2},
    {"jornada": 2, "local": "Betis",    "visitante": "Girona",   "gl": 0, "gv": 1},
    {"jornada": 2, "local": "Sevilla",  "visitante": "Atlético", "gl": 1, "gv": 2},
    {"jornada": 3, "local": "Atlético", "visitante": "Girona",   "gl": 1, "gv": 1},
    {"jornada": 3, "local": "Betis",    "visitante": "Sevilla",  "gl": 4, "gv": 0},
]
```

1. **lambda:** `resultado(p)`, que devuelva `"1"`, `"X"` o `"2"`. Con `sorted()`, los partidos ordenados de **más a menos goles** y, a igualdad de goles, por jornada. Muestra `(jornada, local, goles totales)`.
2. **`map()`:** los marcadores con el formato `"Atlético 3-1 Betis"`.
3. **`filter()`:** los empates y las victorias visitantes (muestra el marcador).
4. **`reduce()`:** los goles totales de la liga y el partido con más goles.
5. **Función que devuelve una función:** `crear_contador_puntos(equipo)` devuelve una función que recibe un partido y devuelve los puntos de ese equipo (3 por victoria, 1 por empate, 0 si pierde o no juega). Úsala con `map()` y `sum()` para construir la clasificación ordenada de mayor a menor.

Salida esperada

```
['1', 'X', '2', '2', 'X', '1']
[(1, 'Atlético', 4), (1, 'Girona', 4), (3, 'Betis', 4), (2, 'Sevilla', 3), (3, 'Atlético', 2), (2, 'Betis',
1)]
['Atlético 3-1 Betis', 'Girona 2-2 Sevilla', 'Betis 0-1 Girona', 'Sevilla 1-2 Atlético', 'Atlético 1-1
Girona', 'Betis 4-0 Sevilla']
Empates: ['Girona 2-2 Sevilla', 'Atlético 1-1 Girona']
Gana visitante: ['Betis 0-1 Girona', 'Sevilla 1-2 Atlético']
```

```
Goles totales: 18 | Partido con más goles: Atlético 3-1 Betis
[('Atlético', 7), ('Girona', 5), ('Betis', 3), ('Sevilla', 1)]
```

Ejercicio 4 · Recursión · Binario, listas crecientes y presupuestos

Sin bucles, salvo en el apartado 3, donde se permite un `for` para recorrer los hijos:

1. `binario(n)`: el número decimal `n ≥ 0` en binario, como texto. Pista: `binario(n // 2)` seguido de `n % 2`.
2. `es_creciente(lista)`: `True` si cada elemento es mayor o igual que el anterior.
3. `coste_total(departamento)`: suma su `coste` y el de todos sus subdepartamentos, a cualquier profundidad.

```python
empresa = {"nombre": "Empresa", "coste": 50000, "subdepartamentos": [
    {"nombre": "IT", "coste": 120000, "subdepartamentos": [
        {"nombre": "Datos", "coste": 80000, "subdepartamentos": []},
        {"nombre": "Sistemas", "coste": 60000, "subdepartamentos": [
            {"nombre": "Ciberseguridad", "coste": 45000, "subdepartamentos": []}]}]},
    {"nombre": "Marketing", "coste": 70000, "subdepartamentos": []},
]}

print(binario(10), binario(255), binario(0), binario(1))
print(es_creciente([1, 3, 3, 7]), es_creciente([1, 5, 2]), es_creciente([]))
print(coste_total(empresa), coste_total(empresa["subdepartamentos"][0]))
```

Salida esperada

```
1010 11111111 0 1
True False True
425000 305000
```

Ejercicio 5 · Clases y objetos · Hotel y habitaciones (composición)

1. Clase `Habitacion` con:
   - atributo de clase `PRECIOS = {"individual": 60, "doble": 90, "suite": 180}`;
   - atributos `numero`, `tipo` y `huesped` (empieza en `None`), y `noches` (0);
   - `@property planta`, el número de la habitación dividido entre 100 (división entera);
   - `@property libre`, que es `True` si no hay huésped;
   - `ocupar(huesped, noches)`, que devuelve `False` si ya estaba ocupada;
   - `liberar()`, que devuelve el importe de la estancia (noches × precio) y deja la habitación libre;
   - `__str__` con el formato `101 (doble, planta 1): libre` o `...: Ana · 3 noches`.
2. Clase `Hotel` (composición) con:
   - `agregar(habitacion)`;
   - `check_in(huesped, tipo, noches)`, que ocupa la **primera habitación libre** de ese tipo y devuelve un mensaje (ver salida);
   - `check_out(numero)`, que devuelve el importe;
   - `@property ocupacion`, el porcentaje de habitaciones ocupadas;
   - `len()`.

```python
h = Hotel("Brisa")
for numero, tipo in [(101, "doble"), (102, "individual"), (201, "doble"), (301, "suite")]:
    h.agregar(Habitacion(numero, tipo))
print(h.check_in("Ana", "doble", 3))
print(h.check_in("Luis", "doble", 2))
print(h.check_in("Eva", "doble", 1))
print(h.check_in("Iker", "suite", 2))
for hab in h.habitaciones:
    print(hab)
print("Ocupación:", h.ocupacion, "%")
print("Check-out 101:", h.check_out(101), "€ | ocupación:", h.ocupacion, "%")
```

Salida esperada

```
Ana → habitación 101
Luis → habitación 201
Eva: no quedan habitaciones doble
Iker → habitación 301
```

```
101 (doble, planta 1): Ana · 3 noches
102 (individual, planta 1): libre
201 (doble, planta 2): Luis · 2 noches
301 (suite, planta 3): Iker · 2 noches
Ocupación: 75.0 %
Check-out 101: 270 € | ocupación: 50.0 %
```

Ejercicio 6 · Sobrecarga de operadores · Temperaturas

Crea una clase `Temperatura` que guarde el valor **en grados Celsius** y permita:

- `@staticmethod desde_texto(texto)`: acepta `"25C"` o `"77F"` (convierte: `C = (F − 32) × 5/9`).
- `t + numero` → nueva `Temperatura` con esos grados más.
- `t1 - t2` → la **diferencia** en grados, como `float` y no como `Temperatura`.
- `==` (redondeando a 1 decimal) y `<`.
- `float(t)` → celsius; `str(t)` → `25.0 °C`; `bool(t)` → `True` si está **por encima de 0 °C**.
- Método `a_fahrenheit()`.

```python
lecturas = [Temperatura.desde_texto(t) for t in ["25C", "77F", "-3C", "32F", "38.5C"]]
print([str(t) for t in lecturas])
print(lecturas[0] == lecturas[1], lecturas[2] < lecturas[3])
print(lecturas[4] - lecturas[2], lecturas[0] + 5, float(lecturas[3]))
print("Máxima:", max(lecturas), "| mínima:", min(lecturas))
print("Sin helada:", len([t for t in lecturas if t]), "de", len(lecturas))
print(lecturas[4].a_fahrenheit())
```

Salida esperada

```
['25.0 °C', '25.0 °C', '-3.0 °C', '0.0 °C', '38.5 °C']
True True
41.5 30.0 °C 0.0
Máxima: 38.5 °C | mínima: -3.0 °C
Sin helada: 3 de 5
101.3
```

Ejercicio 7 · Herencia y polimorfismo · Envío de notificaciones

1. Superclase `Notificacion(destinatario, mensaje)` con:
   - `canal()`, que devuelve el nombre de la clase en mayúsculas;
   - `coste()`, que devuelve 0;
   - `enviar()`, que devuelve `"[EMAIL] ana@x.com: Factura disponible (0.00 €)"`.
2. Subclases que usan `super()`:
   - `Email`: `enviar()` añade `" + adjunto"` si se creó con `adjunto=True`. El constructor recibe ese parámetro extra.
   - `SMS`: cuesta 0.05 € por cada bloque de 160 caracteres (redondeando hacia arriba con `math.ceil`).
   - `Push`: cuesta 0.001 € y su `enviar()` pone el mensaje en mayúsculas si `urgente=True`.
3. Una clase `Slack` **sin herencia** con los mismos métodos `enviar()` y `coste()` (coste 0.002 €). Su formato: `"[SLACK] #canal: mensaje"`.
4. Una función `difundir(notificaciones)` que muestre cada envío y devuelva el coste total, sin `if` ni `isinstance`.

```python
avisos = [
    Email("ana@x.com", "Factura disponible", adjunto=True),
    SMS("600111222", "Su pedido 1234 ha salido del almacén. " * 5),
    Push("móvil de Luis", "oferta flash", urgente=True),
    Slack("#ventas", "Nuevo cliente VIP"),
    Email("eva@x.com", "Bienvenida"),
]
print(f"Coste total: {difundir(avisos):.3f} €")
```

Salida esperada

```
[EMAIL] ana@x.com: Factura disponible (0.00 €) + adjunto
[SMS] 600111222: Su pedido 1234 ha salido del almacén. Su pedido 1234 ha salido del almacén. Su pedido 1234
ha salido del almacén. Su pedido 1234 ha salido del almacén. Su pedido 1234 ha salido del almacén.  (0.10 €)
```

```
[PUSH] móvil de Luis: OFERTA FLASH (0.00 €)
[SLACK] #ventas: Nuevo cliente VIP
[EMAIL] eva@x.com: Bienvenida (0.00 €)
Coste total: 0.103 €
```

Pregunta: ¿qué pasaría si `Slack` no tuviera método `coste()`?

Ejercicio 8 · Refactorización · Matrículas de una academia de idiomas

```python
# [modalidad, alumno, horas_al_mes]
matriculas = [
    ["grupo", "Ana", 12],
    ["individual", "Luis", 8],
    ["online", "Eva", 20],
    ["individual", "Marta", 10],
    ["grupo", "Iker", 6],
]

total = 0
caras = 0
for m in matriculas:
    if m[0] == "grupo":
        precio = m[2] * 8.0
        print("GRUPO | " + m[1] + " | " + str(m[2]) + " h | " + str(round(precio, 2)) + " €")
    elif m[0] == "individual":
        precio = m[2] * 25.0
        if m[2] >= 10:
            precio = precio * 0.9
        print("INDIVIDUAL | " + m[1] + " | " + str(m[2]) + " h | " + str(round(precio, 2)) + " €")
    elif m[0] == "online":
        precio = m[2] * 6.0 + 15
        print("ONLINE | " + m[1] + " | " + str(m[2]) + " h | " + str(round(precio, 2)) + " €")
    total = total + precio
    if precio > 100:
        caras = caras + 1
print("FACTURACIÓN: " + str(round(total, 2)) + " €")
print("Matrículas de más de 100 €: " + str(caras))
```

Modalidad	Precio
Grupo	8 €/hora
Individual	25 €/hora, con un 10 % de descuento desde 10 horas
Online	6 €/hora + 15 € de plataforma

1. Indica tres problemas del código.
2. Crea `Matricula(alumno, horas)` con `PRECIO_HORA` como atributo de clase, `precio()` y `__str__`. Crea `MatriculaGrupo`, `MatriculaIndividual` y `MatriculaOnline`, y sobrescribe `precio()` solo cuando haga falta, usando `super()`.
3. Crea `Academia` (composición) con `matricular()`, `listado()`, `facturacion()` y `contar_mayores_que(importe)`.
4. La salida debe ser **idéntica**.
5. Ampliación: `MatriculaIntensiva`, que es un grupo (hereda de `MatriculaGrupo`) con un recargo del 25 %. Añade `MatriculaIntensiva("Sara", 20)` sin tocar `Matricula` ni `Academia`.

Salida esperada

```
GRUPO | Ana | 12 h | 96.0 €
INDIVIDUAL | Luis | 8 h | 200.0 €
ONLINE | Eva | 20 h | 135.0 €
INDIVIDUAL | Marta | 10 h | 225.0 €
GRUPO | Iker | 6 h | 48.0 €
FACTURACIÓN: 704.0 €
Matrículas de más de 100 €: 3
--- Con intensiva ---
GRUPO | Ana | 12 h | 96.0 €
INDIVIDUAL | Luis | 8 h | 200.0 €
ONLINE | Eva | 20 h | 135.0 €
INDIVIDUAL | Marta | 10 h | 225.0 €
GRUPO | Iker | 6 h | 48.0 €
INTENSIVA | Sara | 20 h | 200.0 €
```

```
FACTURACIÓN: 904.0 €
Matrículas de más de 100 €: 4
```

Ejercicio 9 · Ejercicio integrador · Taquilla de un cine

Parte A. Superclase `Entrada(pelicula, precio_base)` con `precio()` y `__str__` (`Pelicula - 9.50 €`). Subclases: - `EntradaReducida`: un 30 % de descuento (2 decimales). - `Entrada3D`: +2.50 € de suplemento. - `EntradaVIP(pelicula, precio_base, menu)`: el doble del precio base, más 6 € si `menu=True`.

Parte B. Datos

```
ventas = [
    {"cliente": "Ana",  "entrada": Entrada("Dune 3", 9.50),                "cantidad": 2},
    {"cliente": "Luis", "entrada": EntradaReducida("Dune 3", 9.50),        "cantidad": 3},
    {"cliente": "Eva",  "entrada": Entrada3D("Avatar 4", 9.50),            "cantidad": 2},
    {"cliente": "Ana",  "entrada": EntradaVIP("Avatar 4", 9.50, menu=True), "cantidad": 1},
    {"cliente": "Iker", "entrada": EntradaVIP("Oppenheimer 2", 8.00, menu=False), "cantidad": 2},
]
```

Parte C. Con comprehensions: los importes (`precio × cantidad`, 2 decimales); un diccionario `película → entradas vendidas`; un set de clientes con alguna entrada VIP.

Parte D. Con `filter()` y `map()`, los clientes que han gastado más de 20 € en una compra. Con `reduce()`, la recaudación total. Con `crear_filtro_pelicula(titulo)`, que devuelve una función, la recaudación de “Avatar 4”.

Parte E. `total_entradas(ventas)` recursiva, sin bucles.

Parte F. Informe:

Salida esperada

```
=== TAQUILLA ===
Dune 3 - 9.50 €
Dune 3 - 6.65 €
Avatar 4 - 12.00 €
Avatar 4 - 25.00 €
Oppenheimer 2 - 16.00 €

Importes: [19.0, 19.95, 24.0, 25.0, 32.0]
Entradas por película: {'Avatar 4': 3, 'Dune 3': 5, 'Oppenheimer 2': 2}
Clientes VIP: ['Ana', 'Iker']
Compras de más de 20 €: ['Eva', 'Ana', 'Iker']
Recaudación: 119.95 €
Recaudación Avatar 4: 49.0 €
Entradas vendidas: 10
```

Ejercicio 10 · Programación modular · Del script al paquete `almacen`

Un almacén controla su stock con este script, que funciona pero está todo seguido:

```
datos = """codigo;nombre;stock;precio
P1;Tornillos 5mm;120;0.08
P2;Martillo;4;12.5
P3;Sierra;;18.0
P4;Destornillador;9;6.75
P5;Taladro;2;89.9
P6;Llave inglesa;15;x"""
filas = datos.split("\n")[1:]
productos = []
malos = []
for f in filas:
    c = f.split(";")
    try:
        productos.append([c[0].strip().upper(), c[1].strip().title(), int(c[2]), float(c[3])])
    except ValueError:
        malos.append(c[0])
print("Productos leídos:", len(filas), "| descartados:", malos)
total = 0
for p in productos:
```

```
    total = total + p[2] * p[3] * 1.21
print("Valor del stock (IVA incl.):", round(total, 2), "€")
reponer = [p[1] for p in productos if p[2] < 10]
print("A reponer:", reponer)
```

Salida del script:

```
Productos leídos: 6 | descartados: ['P3', 'P6']
Valor del stock (IVA incl.): 363.18 €
A reponer: ['Martillo', 'Destornillador', 'Taladro']
```

1. Indica al menos **cuatro problemas** del script pensando en producción.
2. Conviértelo en este proyecto, que debe dar **la misma salida**:

```
almacen_proyecto/
├── main.py              ← DATOS, mostrar(resultado), main() e if __name__ == "__main__"
├── pruebas.py           ← prueba funciones sueltas de almacen.servicios
└── almacen/
    ├── __init__.py      ← expone procesar, __version__ y __all__
    ├── config.py        ← STOCK_MINIMO, IVA, NUM_CAMPOS
    ├── modelos.py       ← Producto con @property valor (con IVA) y @property reponer
    └── servicios.py     ← parsear, es_valida, a_producto, cargar (map + filter con lambda),
                           valor_total, a_reponer, procesar(texto)
```

3. Dentro del paquete usa imports **relativos**; en `main.py`, solo `from almacen import procesar`.
4. Añade un bloque `if __name__ == "__main__":` con pruebas en `servicios.py` e indica cómo se ejecuta.
5. Con `pruebas.py` comprueba:

```
from almacen.servicios import parsear, es_valida, a_producto

print(es_valida(parsear("P9;Taladro;3;45.0")))
print(es_valida(parsear("P9;Taladro;tres;45.0")))
print(es_valida(parsear("P9;Taladro;3")))
print(a_producto(parsear(" p9 ; taladro ;3;45")))
```

Parte II · Solucionario

Solución 1 · Lectura de código · La cadena de `super()` rota

🖥️ Se muestra por pantalla:

```
['C', 'B', 'D']
['C']
['A', 'B']
['D', 'B', 'C', 'A', 'object']
True True C
```

Explicación:

- MRO de `D`: `D → B → C → A → object`. `D.saludo()` llama con `super()` a `B`, y el `super()` de `B` no llama a `A`, sino a la **siguiente clase del MRO de D**, que es `C`. `C` no llama a `super()`, así que la cadena se **corta** ahí y `A` nunca se ejecuta. Resultado: `["C"] + ["B"] + ["D"]`.
- MRO de `E`: `E → C → B → A`. `E` no define `saludo()`, así que se usa el de `C`, que devuelve `["C"]` y no sigue la cadena.
- En un objeto `B`, el MRO es `B → A` y `super()` llama a `A` → `['A', 'B']`. En un objeto `D`, el mismo `super()` llama a `C`. **`super()` depende del MRO del objeto, no de la clase donde está escrito.**
- `isinstance(D(), C)` es `True` porque `C` está entre los ancestros de `D`.

Solución 2 · Comprehensions · Logs de un servidor web

```
logs = [
    "2026-11-12 10:01 GET /productos 200 0.120",
    "2026-11-12 10:02 POST /login 401 0.045",
    "2026-11-12 10:02 GET /productos 200 0.098",
```

```
    "2026-11-12 10:03 GET /carrito 500 1.502",
    "2026-11-12 10:05 POST /login 200 0.051",
    "2026-11-12 10:07 GET /productos/15 404 0.020",
    "2026-11-12 10:08 GET /carrito 200 0.310",
]

registros = [(l.split()[1], l.split()[2], l.split()[3], int(l.split()[4]), float(l.split()[5])) for l in
logs]

con_error = {ruta for _, _, ruta, codigo, _ in registros if codigo >= 400}
rutas = [r[2] for r in registros]
peticiones = {ruta: rutas.count(ruta) for ruta in rutas}
tiempo_medio = {ruta: round(sum(r[4] for r in registros if r[2] == ruta) / peticiones[ruta], 3)
                for ruta in peticiones}
lentas = [(hora, codigo) for hora, _, _, codigo, seg in registros if seg > 0.3]

print(registros[0])
print(sorted(con_error))
print(peticiones)
print(tiempo_medio)
print(lentas)
```

🖥️ Salida:

```
('10:01', 'GET', '/productos', 200, 0.12)
['/carrito', '/login', '/productos/15']
{'/productos': 2, '/login': 2, '/carrito': 2, '/productos/15': 1}
{'/productos': 0.109, '/login': 0.048, '/carrito': 0.906, '/productos/15': 0.02}
[('10:03', 500), ('10:08', 200)]
```

Explicación:

El apartado 1 llama varias veces a `l.split()`, como permite la técnica vista en clase. En el apartado 4 se reutiliza el diccionario `peticiones` del apartado 3 para dividir. En el 3, la clave repetida se sobrescribe con el mismo valor, así que cada ruta aparece una vez.

Solución 3 · Programación funcional · Liga de fútbol

```
partidos = [
    {"jornada": 1, "local": "Atlético", "visitante": "Betis",    "gl": 3, "gv": 1},
    {"jornada": 1, "local": "Girona",   "visitante": "Sevilla",  "gl": 2, "gv": 2},
    {"jornada": 2, "local": "Betis",    "visitante": "Girona",   "gl": 0, "gv": 1},
    {"jornada": 2, "local": "Sevilla",  "visitante": "Atlético", "gl": 1, "gv": 2},
    {"jornada": 3, "local": "Atlético", "visitante": "Girona",   "gl": 1, "gv": 1},
    {"jornada": 3, "local": "Betis",    "visitante": "Sevilla",  "gl": 4, "gv": 0},
]

from functools import reduce

resultado = lambda p: "1" if p["gl"] > p["gv"] else "X" if p["gl"] == p["gv"] else "2"
goles = lambda p: p["gl"] + p["gv"]
marcador = lambda p: f"{p['local']} {p['gl']}-{p['gv']} {p['visitante']}"

print([resultado(p) for p in partidos])
print([(p["jornada"], p["local"], goles(p)) for p in sorted(partidos, key=lambda p: (-goles(p),
p["jornada"]))])
print(list(map(marcador, partidos)))
print("Empates:", list(map(marcador, filter(lambda p: resultado(p) == "X", partidos))))
print("Gana visitante:", list(map(marcador, filter(lambda p: resultado(p) == "2", partidos))))

total_goles = reduce(lambda acum, p: acum + goles(p), partidos, 0)
mas_goles = reduce(lambda a, b: a if goles(a) >= goles(b) else b, partidos)
print("Goles totales:", total_goles, "| Partido con más goles:", marcador(mas_goles))

def crear_contador_puntos(equipo):
    def puntos(p):
        if equipo == p["local"]:
            propios, ajenos = p["gl"], p["gv"]
        elif equipo == p["visitante"]:
            propios, ajenos = p["gv"], p["gl"]
        else:
```

```
            return 0                                # no juega ese partido
        return 3 if propios > ajenos else 1 if propios == ajenos else 0
    return puntos

equipos = {p["local"] for p in partidos} | {p["visitante"] for p in partidos}
clasificacion = {e: sum(map(crear_contador_puntos(e), partidos)) for e in equipos}
print(sorted(clasificacion.items(), key=lambda kv: (-kv[1], kv[0])))
```

🖥️ **Salida:**

```
['1', 'X', '2', '2', 'X', '1']
[(1, 'Atlético', 4), (1, 'Girona', 4), (3, 'Betis', 4), (2, 'Sevilla', 3), (3, 'Atlético', 2), (2, 'Betis',
1)]
['Atlético 3-1 Betis', 'Girona 2-2 Sevilla', 'Betis 0-1 Girona', 'Sevilla 1-2 Atlético', 'Atlético 1-1
Girona', 'Betis 4-0 Sevilla']
Empates: ['Girona 2-2 Sevilla', 'Atlético 1-1 Girona']
Gana visitante: ['Betis 0-1 Girona', 'Sevilla 1-2 Atlético']
Goles totales: 18 | Partido con más goles: Atlético 3-1 Betis
[('Atlético', 7), ('Girona', 5), ('Betis', 3), ('Sevilla', 1)]
```

Explicación:

`crear_contador_puntos("Atlético")` fabrica una función **especializada** en ese equipo, ya que el closure recuerda `equipo`. En la clasificación se fabrica una función por equipo y se aplica con `map()` a todos los partidos. En los `sorted` la clave es una **tupla**: el signo `-` ordena de mayor a menor y el segundo elemento deshace los empates.

Solución 4 · Recursión · Binario, listas crecientes y presupuestos

```
def binario(n):
    if n < 2:                                   # caso base: 0 → "0", 1 → "1"
        return str(n)
    return binario(n // 2) + str(n % 2)

def es_creciente(lista):
    if len(lista) < 2:                          # caso base: 0 o 1 elementos
        return True
    return lista[0] <= lista[1] and es_creciente(lista[1:])

def coste_total(departamento):
    total = departamento["coste"]
    for sub in departamento["subdepartamentos"]:
        total += coste_total(sub)               # sin hijos → no entra (caso base)
    return total

empresa = {"nombre": "Empresa", "coste": 50000, "subdepartamentos": [
    {"nombre": "IT", "coste": 120000, "subdepartamentos": [
        {"nombre": "Datos", "coste": 80000, "subdepartamentos": []},
        {"nombre": "Sistemas", "coste": 60000, "subdepartamentos": [
            {"nombre": "Ciberseguridad", "coste": 45000, "subdepartamentos": []}]}]},
    {"nombre": "Marketing", "coste": 70000, "subdepartamentos": []},
]}

print(binario(10), binario(255), binario(0), binario(1))
print(es_creciente([1, 3, 3, 7]), es_creciente([1, 5, 2]), es_creciente([]))
print(coste_total(empresa), coste_total(empresa["subdepartamentos"][0]))
```

🖥️ **Salida:**

```
1010 11111111 0 1
True False True
425000 305000
```

Explicación:

`binario(10)` → `binario(5) + "0"` → `binario(2) + "1" + "0"` → `binario(1) + "0" + "1" + "0"` = `"1010"`. En `es_creciente`, el `and` hace **cortocircuito**: si un par ya falla, no se sigue llamando a la función.

Solución 5 · Clases y objetos · Hotel y habitaciones (composición)

```python
class Habitacion:
    PRECIOS = {"individual": 60, "doble": 90, "suite": 180}

    def __init__(self, numero, tipo):
        self.numero = numero
        self.tipo = tipo
        self.huesped = None
        self.noches = 0

    @property
    def planta(self):
        return self.numero // 100

    @property
    def libre(self):
        return self.huesped is None

    def ocupar(self, huesped, noches):
        if not self.libre:
            return False
        self.huesped, self.noches = huesped, noches
        return True

    def liberar(self):
        importe = self.noches * Habitacion.PRECIOS[self.tipo]
        self.huesped, self.noches = None, 0
        return importe

    def __str__(self):
        estado = "libre" if self.libre else f"{self.huesped} · {self.noches} noches"
        return f"{self.numero} ({self.tipo}, planta {self.planta}): {estado}"

class Hotel:
    def __init__(self, nombre):
        self.nombre = nombre
        self.habitaciones = []                    # composición

    def agregar(self, habitacion):
        self.habitaciones.append(habitacion)

    def check_in(self, huesped, tipo, noches):
        for hab in self.habitaciones:
            if hab.tipo == tipo and hab.ocupar(huesped, noches):
                return f"{huesped} → habitación {hab.numero}"
        return f"{huesped}: no quedan habitaciones {tipo}"

    def check_out(self, numero):
        for hab in self.habitaciones:
            if hab.numero == numero:
                return hab.liberar()
        return 0

    @property
    def ocupacion(self):
        ocupadas = len([h for h in self.habitaciones if not h.libre])
        return round(ocupadas / len(self) * 100, 1)

    def __len__(self):
        return len(self.habitaciones)

h = Hotel("Brisa")
for numero, tipo in [(101, "doble"), (102, "individual"), (201, "doble"), (301, "suite")]:
    h.agregar(Habitacion(numero, tipo))
print(h.check_in("Ana", "doble", 3))
print(h.check_in("Luis", "doble", 2))
print(h.check_in("Eva", "doble", 1))
print(h.check_in("Iker", "suite", 2))
for hab in h.habitaciones:
    print(hab)
```

```
print("Ocupación:", h.ocupacion, "%")
print("Check-out 101:", h.check_out(101), "€ | ocupación:", h.ocupacion, "%")
```

🖥️ **Salida:**

```
Ana → habitación 101
Luis → habitación 201
Eva: no quedan habitaciones doble
Iker → habitación 301
101 (doble, planta 1): Ana · 3 noches
102 (individual, planta 1): libre
201 (doble, planta 2): Luis · 2 noches
301 (suite, planta 3): Iker · 2 noches
Ocupación: 75.0 %
Check-out 101: 270 € | ocupación: 50.0 %
```

Explicación:

En `check_in`, la condición `hab.tipo == tipo and hab.ocupar(...)` aprovecha el **cortocircuito** del `and`: solo se intenta ocupar si el tipo coincide, y `ocupar()` devuelve `False` si ya está ocupada, así que el bucle sigue buscando. `planta` y `libre` son `@property`: se usan como atributos, sin paréntesis.

Solución 6 · Sobrecarga de operadores · Temperaturas

```
class Temperatura:
    def __init__(self, celsius):
        self.celsius = celsius

    @staticmethod
    def desde_texto(texto):
        valor, unidad = float(texto[:-1]), texto[-1].upper()
        if unidad == "F":
            valor = (valor - 32) * 5 / 9
        return Temperatura(valor)

    def __add__(self, grados):                    # Temperatura + número
        return Temperatura(self.celsius + grados)

    def __sub__(self, otra):                      # Temperatura - Temperatura → float
        return round(self.celsius - otra.celsius, 1)

    def __eq__(self, otra):
        return round(self.celsius, 1) == round(otra.celsius, 1)

    def __lt__(self, otra):
        return self.celsius < otra.celsius

    def __float__(self):
        return float(self.celsius)

    def __bool__(self):
        return self.celsius > 0

    def __str__(self):
        return f"{self.celsius:.1f} °C"

    def a_fahrenheit(self):
        return round(self.celsius * 9 / 5 + 32, 1)

lecturas = [Temperatura.desde_texto(t) for t in ["25C", "77F", "-3C", "32F", "38.5C"]]
print([str(t) for t in lecturas])
print(lecturas[0] == lecturas[1], lecturas[2] < lecturas[3])
print(lecturas[4] - lecturas[2], lecturas[0] + 5, float(lecturas[3]))
print("Máxima:", max(lecturas), "| mínima:", min(lecturas))
print("Sin helada:", len([t for t in lecturas if t]), "de", len(lecturas))
print(lecturas[4].a_fahrenheit())
```

🖥️ **Salida:**

```
['25.0 °C', '25.0 °C', '-3.0 °C', '0.0 °C', '38.5 °C']
True True
41.5 30.0 °C 0.0
Máxima: 38.5 °C | mínima: -3.0 °C
Sin helada: 3 de 5
101.3
```

Explicación:

- Un operador puede devolver **cualquier tipo**: aquí `+` devuelve una `Temperatura` y `-` devuelve un número. Lo decide quien diseña la clase.
- `32F` son exactamente 0 °C, y `bool()` devuelve `False` porque no está **por encima** de 0.
- `if t` dentro de la comprehension usa `__bool__`.

Solución 7 · Herencia y polimorfismo · Envío de notificaciones

```python
import math

class Notificacion:
    def __init__(self, destinatario, mensaje):
        self.destinatario = destinatario
        self.mensaje = mensaje

    def canal(self):
        return type(self).__name__.upper()

    def coste(self):
        return 0

    def enviar(self):
        return f"[{self.canal()}] {self.destinatario}: {self.mensaje} ({self.coste():.2f} €)"

class Email(Notificacion):
    def __init__(self, destinatario, mensaje, adjunto=False):
        super().__init__(destinatario, mensaje)
        self.adjunto = adjunto

    def enviar(self):
        return super().enviar() + (" + adjunto" if self.adjunto else "")

class SMS(Notificacion):
    def coste(self):
        return math.ceil(len(self.mensaje) / 160) * 0.05

class Push(Notificacion):
    def __init__(self, destinatario, mensaje, urgente=False):
        super().__init__(destinatario, mensaje)
        self.urgente = urgente

    def coste(self):
        return 0.001

    def enviar(self):
        if self.urgente:
            self.mensaje = self.mensaje.upper()
        return super().enviar()

class Slack:                                        # duck typing: sin herencia
    def __init__(self, canal_slack, mensaje):
        self.canal_slack = canal_slack
        self.mensaje = mensaje

    def coste(self):
        return 0.002

    def enviar(self):
        return f"[SLACK] {self.canal_slack}: {self.mensaje}"

def difundir(notificaciones):
    total = 0
    for n in notificaciones:
```

```
        print(n.enviar())                          # polimorfismo / duck typing
        total += n.coste()
    return total

avisos = [
    Email("ana@x.com", "Factura disponible", adjunto=True),
    SMS("600111222", "Su pedido 1234 ha salido del almacén. " * 5),
    Push("móvil de Luis", "oferta flash", urgente=True),
    Slack("#ventas", "Nuevo cliente VIP"),
    Email("eva@x.com", "Bienvenida"),
]
print(f"Coste total: {difundir(avisos):.3f} €")
```

🖥️ **Salida:**

```
[EMAIL] ana@x.com: Factura disponible (0.00 €) + adjunto
[SMS] 600111222: Su pedido 1234 ha salido del almacén. Su pedido 1234 ha salido del almacén. Su pedido 1234
ha salido del almacén. Su pedido 1234 ha salido del almacén. Su pedido 1234 ha salido del almacén.  (0.10 €)
[PUSH] móvil de Luis: OFERTA FLASH (0.00 €)
[SLACK] #ventas: Nuevo cliente VIP
[EMAIL] eva@x.com: Bienvenida (0.00 €)
Coste total: 0.103 €
```

Explicación:

- El SMS tiene 190 caracteres → `ceil(190 / 160)` = 2 bloques → 0.10 €. El `enviar()` de la superclase llama a `self.coste()` y `self.canal()`, que en cada objeto ejecutan la versión de su clase.
- `Slack` funciona en `difundir()` sin heredar de nada porque tiene los métodos que se usan (**duck typing**). Sin `coste()`, al llegar a ese objeto se lanzaría `AttributeError: 'Slack' object has no attribute 'coste'`, aunque su `enviar()` ya se habría mostrado.

Solución 8 · Refactorización · Matrículas de una academia de idiomas

```
class Matricula:
    PRECIO_HORA = 0

    def __init__(self, alumno, horas):
        self.alumno = alumno
        self.horas = horas

    def precio(self):
        return self.horas * self.PRECIO_HORA

    def __str__(self):
        tipo = type(self).__name__.replace("Matricula", "").upper()
        return f"{tipo} | {self.alumno} | {self.horas} h | {round(self.precio(), 2)} €"

class MatriculaGrupo(Matricula):
    PRECIO_HORA = 8.0

class MatriculaIndividual(Matricula):
    PRECIO_HORA = 25.0
    HORAS_DESCUENTO = 10
    DESCUENTO = 0.10

    def precio(self):
        base = super().precio()
        return base * (1 - self.DESCUENTO) if self.horas >= self.HORAS_DESCUENTO else base

class MatriculaOnline(Matricula):
    PRECIO_HORA = 6.0
    PLATAFORMA = 15

    def precio(self):
        return super().precio() + self.PLATAFORMA

class Academia:
    def __init__(self):
        self.matriculas = []
```

```
    def matricular(self, matricula):
        self.matriculas.append(matricula)

    def listado(self):
        for m in self.matriculas:
            print(m)

    def facturacion(self):
        return sum(m.precio() for m in self.matriculas)

    def contar_mayores_que(self, importe):
        return len([m for m in self.matriculas if m.precio() > importe])

academia = Academia()
for m in [MatriculaGrupo("Ana", 12), MatriculaIndividual("Luis", 8), MatriculaOnline("Eva", 20),
          MatriculaIndividual("Marta", 10), MatriculaGrupo("Iker", 6)]:
    academia.matricular(m)
academia.listado()
print(f"FACTURACIÓN: {round(academia.facturacion(), 2)} €")
print(f"Matrículas de más de 100 €: {academia.contar_mayores_que(100)}")

class MatriculaIntensiva(MatriculaGrupo):      # ampliación
    RECARGO = 0.25

    def precio(self):
        return super().precio() * (1 + self.RECARGO)

print("--- Con intensiva ---")
academia.matricular(MatriculaIntensiva("Sara", 20))
academia.listado()
print(f"FACTURACIÓN: {round(academia.facturacion(), 2)} €")
print(f"Matrículas de más de 100 €: {academia.contar_mayores_que(100)}")
```

🖥️ **Salida:**

```
GRUPO | Ana | 12 h | 96.0 €
INDIVIDUAL | Luis | 8 h | 200.0 €
ONLINE | Eva | 20 h | 135.0 €
INDIVIDUAL | Marta | 10 h | 225.0 €
GRUPO | Iker | 6 h | 48.0 €
FACTURACIÓN: 704.0 €
Matrículas de más de 100 €: 3
--- Con intensiva ---
GRUPO | Ana | 12 h | 96.0 €
INDIVIDUAL | Luis | 8 h | 200.0 €
ONLINE | Eva | 20 h | 135.0 €
INDIVIDUAL | Marta | 10 h | 225.0 €
GRUPO | Iker | 6 h | 48.0 €
INTENSIVA | Sara | 20 h | 200.0 €
FACTURACIÓN: 904.0 €
Matrículas de más de 100 €: 4
```

Explicación:

Problemas del original: índices mágicos (`m[2]` son las horas); tarifas y descuentos escritos a mano, sin nombre; el `print` está repetido tres veces casi igual; mezcla de cálculo y presentación; y para añadir una modalidad hay que tocar el `if/elif`. En la versión con clases, `MatriculaIntensiva` reutiliza el precio de grupo con `super()` y solo añade el recargo.

Solución 9 · Ejercicio integrador · Taquilla de un cine

```
from functools import reduce

class Entrada:
    def __init__(self, pelicula, precio_base):
        self.pelicula = pelicula
        self.precio_base = precio_base

    def precio(self):
        return self.precio_base
```

```python
    def __str__(self):
        return f"{self.pelicula} - {self.precio():.2f} €"

class EntradaReducida(Entrada):
    def precio(self):
        return round(super().precio() * 0.70, 2)

class Entrada3D(Entrada):
    def precio(self):
        return super().precio() + 2.50

class EntradaVIP(Entrada):
    def __init__(self, pelicula, precio_base, menu=False):
        super().__init__(pelicula, precio_base)
        self.menu = menu

    def precio(self):
        return super().precio() * 2 + (6 if self.menu else 0)

ventas = [
    {"cliente": "Ana",  "entrada": Entrada("Dune 3", 9.50),                "cantidad": 2},
    {"cliente": "Luis", "entrada": EntradaReducida("Dune 3", 9.50),        "cantidad": 3},
    {"cliente": "Eva",  "entrada": Entrada3D("Avatar 4", 9.50),            "cantidad": 2},
    {"cliente": "Ana",  "entrada": EntradaVIP("Avatar 4", 9.50, menu=True), "cantidad": 1},
    {"cliente": "Iker", "entrada": EntradaVIP("Oppenheimer 2", 8.00, menu=False), "cantidad": 2},
]

importe = lambda v: round(v["entrada"].precio() * v["cantidad"], 2)

# C. Comprehensions
importes = [importe(v) for v in ventas]
peliculas = {v["entrada"].pelicula for v in ventas}
por_pelicula = {p: sum(v["cantidad"] for v in ventas if v["entrada"].pelicula == p) for p in
sorted(peliculas)}
vip = {v["cliente"] for v in ventas if isinstance(v["entrada"], EntradaVIP)}

# D. Funcional
grandes = list(map(lambda v: v["cliente"], filter(lambda v: importe(v) > 20, ventas)))
recaudacion = reduce(lambda acum, v: acum + importe(v), ventas, 0)

def crear_filtro_pelicula(titulo):
    return lambda v: v["entrada"].pelicula == titulo

avatar = reduce(lambda acum, v: acum + importe(v), filter(crear_filtro_pelicula("Avatar 4"), ventas), 0)

# E. Recursión
def total_entradas(ventas):
    return 0 if not ventas else ventas[0]["cantidad"] + total_entradas(ventas[1:])

# F. Informe
print("=== TAQUILLA ===")
for v in ventas:
    print(v["entrada"])
print()
print("Importes:", importes)
print("Entradas por película:", por_pelicula)
print("Clientes VIP:", sorted(vip))
print("Compras de más de 20 €:", grandes)
print(f"Recaudación: {round(recaudacion, 2)} €")
print(f"Recaudación Avatar 4: {round(avatar, 2)} €")
print("Entradas vendidas:", total_entradas(ventas))
```

🖥️ Salida:

```
=== TAQUILLA ===
Dune 3 - 9.50 €
Dune 3 - 6.65 €
Avatar 4 - 12.00 €
Avatar 4 - 25.00 €
Oppenheimer 2 - 16.00 €

Importes: [19.0, 19.95, 24.0, 25.0, 32.0]
Entradas por película: {'Avatar 4': 3, 'Dune 3': 5, 'Oppenheimer 2': 2}
```

```
Clientes VIP: ['Ana', 'Iker']
Compras de más de 20 €: ['Eva', 'Ana', 'Iker']
Recaudación: 119.95 €
Recaudación Avatar 4: 49.0 €
Entradas vendidas: 10
```

Solución 10 · Programación modular · Del script al paquete `almacen`

1. Problemas del script:

Problema	Consecuencia en producción
Datos escritos dentro del código	Para otro fichero hay que editar el programa
Índices mágicos (`p[2]`, `p[3]`)	Ilegible; si cambian las columnas falla en silencio
`1.21` y `10` escritos a mano	Cambiar el IVA o el stock mínimo obliga a buscarlos por todo el código
Validación incompleta (solo `int`/`float`)	Una línea con columnas de más pasaría como válida
Variables globales modificadas en bucles (`total`, `productos`)	No se puede reutilizar ni probar por partes
Todo en un bloque, sin funciones	Nada se puede importar desde otro informe o notebook

`almacen/config.py`

```python
"""Parámetros del negocio: solo constantes."""
STOCK_MINIMO = 10
IVA = 0.21
NUM_CAMPOS = 4
SEPARADOR = ";"
```

`almacen/modelos.py`

```python
from .config import IVA, STOCK_MINIMO


class Producto:
    def __init__(self, codigo, nombre, stock, precio):
        self.codigo = codigo
        self.nombre = nombre
        self.stock = stock
        self.precio = precio

    @property
    def valor(self):
        """Valor del stock con IVA, calculado al pedirlo."""
        return self.stock * self.precio * (1 + IVA)

    @property
    def reponer(self):
        return self.stock < STOCK_MINIMO

    def __repr__(self):
        return f"Producto({self.codigo!r}, {self.nombre!r}, stock={self.stock}, precio={self.precio})"
```

`almacen/servicios.py`

```python
from .config import NUM_CAMPOS, SEPARADOR
from .modelos import Producto


def parsear(linea):
    return [campo.strip() for campo in linea.split(SEPARADOR)]


def es_valida(campos):
    if len(campos) != NUM_CAMPOS:
        return False
    try:
```

```
        int(campos[2])
        float(campos[3])
        return True
    except ValueError:
        return False


def a_producto(campos):
    return Producto(campos[0].upper(), campos[1].title(), int(campos[2]), float(campos[3]))


def cargar(lineas):
    """Devuelve (productos_validos, codigos_descartados)."""
    campos = list(map(parsear, lineas))
    descartados = [c[0] for c in campos if not es_valida(c)]
    productos = list(map(a_producto, filter(lambda c: es_valida(c), campos)))
    return productos, descartados


def valor_total(productos):
    return round(sum(p.valor for p in productos), 2)


def a_reponer(productos):
    return [p.nombre for p in productos if p.reponer]


def procesar(texto):
    lineas = texto.strip().split("\n")[1:]
    productos, descartados = cargar(lineas)
    return {"leidos": len(lineas), "descartados": descartados,
            "valor": valor_total(productos), "reponer": a_reponer(productos)}


if __name__ == "__main__":          # python -m almacen.servicios (desde almacen_proyecto)
    print(parsear("P1; Tornillos ;5;0.1"))
    print(a_reponer([a_producto(["P1", "tornillos", "5", "0.1"])]))
```

`almacen/__init__.py`

```
"""Recepción del paquete almacen."""
from .servicios import procesar

__version__ = "1.0.0"
__all__ = ["procesar"]
```

`main.py`

```
from almacen import procesar

DATOS = """codigo;nombre;stock;precio
P1;Tornillos 5mm;120;0.08
P2;Martillo;4;12.5
P3;Sierra;;18.0
P4;Destornillador;9;6.75
P5;Taladro;2;89.9
P6;Llave inglesa;15;x"""


def mostrar(resultado):
    print("Productos leídos:", resultado["leidos"], "| descartados:", resultado["descartados"])
    print("Valor del stock (IVA incl.):", resultado["valor"], "€")
    print("A reponer:", resultado["reponer"])


def main():
    mostrar(procesar(DATOS))


if __name__ == "__main__":
    main()
```

`pruebas.py`

```
from almacen.servicios import parsear, es_valida, a_producto

print(es_valida(parsear("P9;Taladro;3;45.0")))
print(es_valida(parsear("P9;Taladro;tres;45.0")))
print(es_valida(parsear("P9;Taladro;3")))
print(a_producto(parsear(" p9 ; taladro ;3;45")))
```

🖥️ **`python main.py`** (idéntica al script original):

```
Productos leídos: 6 | descartados: ['P3', 'P6']
Valor del stock (IVA incl.): 363.18 €
A reponer: ['Martillo', 'Destornillador', 'Taladro']
```

🖥️ **`python pruebas.py`:**

```
True
False
False
Producto('P9', 'Taladro', stock=3, precio=45.0)
```

🖥️ **`python -m almacen.servicios`** (pruebas propias del módulo; se ejecuta con `-m` desde `almacen_proyecto` porque usa imports relativos):

```
<frozen runpy>:130: RuntimeWarning: 'almacen.servicios' found in sys.modules after import of package
'almacen', but prior to execution of 'almacen.servicios'; this may result in unpredictable behaviour
['P1', 'Tornillos', '5', '0.1']
['Tornillos']
```

> El `RuntimeWarning` aparece porque `almacen/__init__.py` ya importa `servicios`: el módulo se carga una vez como `almacen.servicios` y otra como `__main__`. No es un error. Las pruebas de `pruebas.py` evitan ese doble cargado.

Con un `notebook.ipynb` en la misma carpeta se podría reutilizar todo sin copiar código: `from almacen import procesar` o `from almacen.servicios import cargar`.